

Pandas 中文文档

Pandas 官方文档 (中文: teadocs)

- [Pandas 中文文档](#)
- [新版本特性](#)
- [v0.25.0 版本特性 \(2019年7月18日\)](#)
- [安装方法](#)
- [安装](#)
- [快速入门](#)
- [快速入门](#)
- [Pandas 概览](#)
- [十分钟入门 Pandas](#)
- [基础用法](#)
- [数据结构简介](#)
- [与其他工具比较](#)
- [教程资料](#)
- [用户指南](#)
- [Pandas 用户指南目录](#)
- [IO工具 \(文本, CSV, HDF5, ...\)](#)
- [索引和数据选择器](#)
- [多层级索引和高级索引](#)
- [Merge, join, and concatenate](#)
- [Reshaping and pivot tables](#)
- [Pandas 处理文本字符串](#)
- [Working with missing data](#)
- [Categorical data](#)
- [Nullable整型数据类型](#)
- [Visualization](#)
- [Computational tools](#)
- [Group By: split-apply-combine](#)
- [时间序列与日期用法](#)
- [时间差](#)
- [Styling](#)
- [Options and settings](#)
- [Enhancing performance](#)
- [Sparse data structures](#)
- [Frequently Asked Questions \(FAQ\)](#)
- [烹饪指南](#)
- [Pandas 生态](#)
- [Pandas 生态圈](#)

- [API 参考手册](#)
- [API 参考手册](#)
- [开发者文档](#)
- [开发者文档](#)
- [版本发布日志](#)
- [发布日志](#)

Pandas 中文文档

Pandas 官方文档中文版 • 在线 <https://www.py pandas.cn> 许可: CC BY-NC-SA 3.0 CN

新版本特性

v0.25.0 版本特性（2019年7月18日）

::: danger 警告

从0.25.x系列版本开始，Pandas仅支持Python 3.5.3及更高版本。有关更多详细信息，请参见[计划移除对Python 2.7的支持](#)。

:::

::: danger 警告

在未来的版本中，支持的最低Python版本将提高到3.6。

:::

::: danger 警告

面板（Panel）已完全删除。对于N-D标记的数据结构，请使用 [xarray](#)。

:::

::: danger 警告

`read_pickle()` 和 `read_msgpack()` 方法仅保证向后兼容的 Pandas 版本为0.20.3（[GH27082](#)）。

:::

这些是 Pandas v0.25.0 版本的改变。有关完整的更新日志(包括其他版本的Pandas)，请参见[发布日志](#)。

增强

具有重新标记的Groupby聚合

Pandas添加了特殊的groupby行为，称为“命名聚合”，用于在将多个聚合函数应用于特定列时命名输出列（[GH18366](#), [GH26512](#)）。

```
In [1]: animals = pd.DataFrame({'kind': ['cat', 'dog', 'cat', 'dog'],
...:                           'height': [9.1, 6.0, 9.5, 34.0],
...:                           'weight': [7.9, 7.5, 9.9, 198.0]})
...:
```

```
In [2]: animals
Out[2]:
```

	kind	height	weight
0	cat	9.1	7.9
1	dog	6.0	7.5
2	cat	9.5	9.9
3	dog	34.0	198.0

```
[4 rows x 3 columns]
```

```
In [3]: animals.groupby("kind").agg(
...:     min_height=pd.NamedAgg(column='height', aggfunc='min'),
...:     max_height=pd.NamedAgg(column='height', aggfunc='max'),
...:     average_weight=pd.NamedAgg(column='weight', aggfunc=np.mean),
...: )
...:
```

```
Out[3]:
```

	min_height	max_height	average_weight
kind			
cat	9.1	9.5	8.90
dog	6.0	34.0	102.75

```
[2 rows x 3 columns]
```

将所需的列名称作为 `**kwargs` 传递给 `.agg`。`**kwargs` 的值应该是元组，其中第一个元素是列选择，第二个元素是要应用的聚合函数。Pandas提供了 `pandas.NamedAgg`（命名为元组），使函数的参数更清晰，但也接受了普通元组。

```
In [4]: animals.groupby("kind").agg(
...:     min_height=('height', 'min'),
...:     max_height=('height', 'max'),
...:     average_weight=('weight', np.mean),
...: )
Out[4]:
```

	min_height	max_height	average_weight
kind			
cat	9.1	9.5	8.90
dog	6.0	34.0	102.75

```
[2 rows x 3 columns]
```

命名聚合是建议替代不推荐使用的“dict-of-dicts”方法来命名特定于列的聚合的输出（[重命名时使用字典弃用groupby.agg\(\)](#)）。

类似的方法现在也可用于Series Groupby对象。因为不需要选择列，所以值可以只是要应用的函数。

```
In [5]: animals.groupby("kind").height.agg(
...:     min_height="min",
...:     max_height="max",
...: )
Out[5]:
```

	min_height	max_height
kind		
cat	9.1	9.5
dog	6.0	34.0

```
[2 rows x 2 columns]
```

在将dict传递给Series groupby聚合（[重命名时使用字典时不推荐使用groupby.agg\(\)方法](#)）时，建议使用这种类型的聚合来替代不建议使用的方法和操作。

有关更多信息，请参见[命名聚合](#)。

具有多个Lambda的Groupby聚合

您现在可以在 `pandas.core.groupby.GroupBy.agg` ([GH26430](#)) 中为类似列表的聚合提供多个lambda函数。

```

In [6]: animals.groupby('kind').height.agg([
...:     lambda x: x.iloc[0], lambda x: x.iloc[-1]
...: ])
...:
Out[6]:
      <lambda_0>  <lambda_1>
kind
cat           9.1         9.5
dog           6.0        34.0

[2 rows x 2 columns]

In [7]: animals.groupby('kind').agg([
...:     lambda x: x.iloc[0] - x.iloc[1],
...:     lambda x: x.iloc[0] + x.iloc[1]
...: ])
...:
Out[7]:
      height      weight
      <lambda_0> <lambda_1> <lambda_0> <lambda_1>
kind
cat      -0.4      18.6      -2.0      17.8
dog     -28.0      40.0     -190.5     205.5

[2 rows x 4 columns]

```

以前的版本，这些行为会引发 `SpecificationError` 异常。

更好的多索引 repr

`MultiIndex` 实例的打印现在将会显示每行的元组数据，并确保元组项垂直对齐，因此现在更容易理解 `MultiIndex` 的结构。([GH13480](#)):

`repr`现在看起来像这样：

```
In [8]: pd.MultiIndex.from_product(['a', 'abc'], range(500))
Out[8]:
MultiIndex([( 'a',  0),
             ( 'a',  1),
             ( 'a',  2),
             ( 'a',  3),
             ( 'a',  4),
             ( 'a',  5),
             ( 'a',  6),
             ( 'a',  7),
             ( 'a',  8),
             ( 'a',  9),
             ...
             ('abc', 490),
             ('abc', 491),
             ('abc', 492),
             ('abc', 493),
             ('abc', 494),
             ('abc', 495),
             ('abc', 496),
             ('abc', 497),
             ('abc', 498),
             ('abc', 499)],
            length=1000)
```

在以前的版本中，输出 `MultiIndex` 操作会打印 `MultiIndex` 的所有级别和代码，这在视觉和排版上没有吸引力，并使输出的内容更难以定位。例如（将范围限制为5）：

```
In [1]: pd.MultiIndex.from_product(['a', 'abc'], range(5))
Out[1]: MultiIndex(levels=[['a', 'abc'], [0, 1, 2, 3]],
...:               codes=[[0, 0, 0, 0, 1, 1, 1, 1], [0, 1, 2, 3, 0, 1, 2, 3]])
```

在新的repr中，如果行数小于 `options.display.max_seq_items`（默认值：100个项目），则将显示所有值。水平方向上，如果输出比 `options.display.width` 宽（默认值：80个字符），则输出将被截断。

用于Series和DataFrame的较短截断 repr

目前，pandas的默认显示选项确保当Series或DataFrame具有超过60行时，其repr将被截断为最多60行（`display.max_rows`选项）。然而，这仍然给出一个占据垂直屏幕区域很大一部分的repr。因此，引入了一个新选项 `display.min_rows`，默认值为10，它确定截断的repr中显示的行数：

- 对于较小的 Series 或 DataFrame，最多显示 `max_rows` 数行（默认值：60）。
- 对于长度大于 `max_rows` 的长度较大的DataFrame Series，仅限显示 `min_rows` 数行（默认值：10，即第一个和最后一个5行）。

这个双重选项允许仍然可以看到相对较小的对象的全部内容（例如 `df.head(20)` 显示所有20行），同时为大对象提供简短的repr。

要恢复单个阈值的先前行为，请设置 `pd.options.display.min_rows = None`。

使用max_level参数支持进行JSON规范化

`json_normalize()` normalizes the provided input dict to all nested levels. The new `max_level` parameter provides more control over which level to end normalization ([GH23843](#)):

The repr now looks like this:

```
In [9]: from pandas.io.json import json_normalize

In [10]: data = [{
.....:     'CreatedBy': {'Name': 'User001'},
.....:     'Lookup': {'TextField': 'Some text',
.....:                'UserField': {'Id': 'ID001', 'Name': 'Name001'}},
.....:     'Image': {'a': 'b'}
.....: }]

In [11]: json_normalize(data, max_level=1)
Out[11]:
  CreatedBy.Name Lookup.TextField      Lookup.UserField Image.a
0      User001   Some text  {'Id': 'ID001', 'Name': 'Name001'}      b

[1 rows x 4 columns]
```

Series.explode 将类似列表的值拆分为行

`Series` and `DataFrame` have gained the `DataFrame.explode()` methods to transform list-likes to individual rows. See [section on Exploding list-like column](#) in docs for more information ([GH16538](#), [GH10511](#))

Here is a typical usecase. You have comma separated string in a column.

```
In [12]: df = pd.DataFrame([{'var1': 'a,b,c', 'var2': 1},
.....:                      {'var1': 'd,e,f', 'var2': 2}])
.....:

In [13]: df
Out[13]:
   var1  var2
0  a,b,c     1
1  d,e,f     2

[2 rows x 2 columns]
```

Creating a long form `DataFrame` is now straightforward using chained operations

```
In [14]: df.assign(var1=df.var1.str.split(',')).explode('var1')
Out[14]:
  var1 var2
0    a    1
0    b    1
0    c    1
1    d    2
1    e    2
1    f    2

[6 rows x 2 columns]
```

其他增强功能

- `DataFrame.plot()` keywords `logy`, `logx` and `loglog` can now accept the value `'sym'` for symlog scaling. (GH24867)
- Added support for ISO week year format (`'%G-%V-%u'`) when parsing datetimes using `to_datetime()` (GH16607)
- Indexing of `DataFrame` and `Series` now accepts zerodim `np.ndarray` (GH24919)
- `Timestamp.replace()` now supports the `fold` argument to disambiguate DST transition times (GH25017)
- `DataFrame.at_time()` and `Series.at_time()` now support `datetime.time` objects with timezones (GH24043)
- `DataFrame.pivot_table()` now accepts an `observed` parameter which is passed to underlying calls to `DataFrame.groupby()` to speed up grouping categorical data. (GH24923)
- `Series.str` has gained `Series.str.casefold()` method to removes all case distinctions present in a string (GH25405)
- `DataFrame.set_index()` now works for instances of `abc.Iterator`, provided their output is of the same length as the calling frame (GH22484, GH24984)
- `DatetimeIndex.union()` now supports the `sort` argument. The behavior of the `sort` parameter matches that of `Index.union()` (GH24994)
- `RangeIndex.union()` now supports the `sort` argument. If `sort=False` an unsorted `Int64Index` is always returned. `sort=None` is the default and returns a monotonically increasing `RangeIndex` if possible or a sorted `Int64Index` if not (GH24471)
- `TimedeltaIndex.intersection()` now also supports the `sort` keyword (GH24471)
- `DataFrame.rename()` now supports the `errors` argument to raise errors when attempting to rename nonexistent keys (GH13473)
- Added `Sparse accessor` for working with a `DataFrame` whose values are sparse (GH25681)
- `RangeIndex` has gained `start`, `stop`, and `step` attributes (GH25710)
- `datetime.timezone` objects are now supported as arguments to timezone methods and constructors (GH25065)
- `DataFrame.query()` and `DataFrame.eval()` now supports quoting column names with backticks to refer to names with spaces (GH6508)

- `merge_asof()` now gives a more clear error message when merge keys are categoricals that are not equal ([GH26136](#))
- `pandas.core.window.Rolling()` supports exponential (or Poisson) window type ([GH21303](#))
- Error message for missing required imports now includes the original import error's text ([GH23868](#))
- `DatetimeIndex` and `TimedeltaIndex` now have a `mean` method ([GH24757](#))
- `DataFrame.describe()` now formats integer percentiles without decimal point ([GH26660](#))
- Added support for reading SPSS .sav files using `read_spss()` ([GH26537](#))
- Added new option `plotting.backend` to be able to select a plotting backend different than the existing `matplotlib` one. Use `pandas.set_option('plotting.backend', '')` where ``GH14130)
- `pandas.offsets.BusinessHour` supports multiple opening hours intervals ([GH15481](#))
- `read_excel()` can now use `openpyxl` to read Excel files via the `engine='openpyxl'` argument. This will become the default in a future release ([GH11499](#))
- `pandas.io.excel.read_excel()` supports reading OpenDocument tables. Specify `engine='odf'` to enable. Consult the [IO User Guide](#) for more details ([GH9070](#))
- `Interval`, `IntervalIndex`, and `IntervalArray` have gained an `is_empty` attribute denoting if the given interval(s) are empty ([GH27219](#))

向后不兼容的API更改

使用UTC偏移量对日期字符串进行索引

Indexing a `DataFrame` or `Series` with a `DatetimeIndex` with a date string with a UTC offset would previously ignore the UTC offset. Now, the UTC offset is respected in indexing. ([GH24076](#), [GH16785](#))

```
In [15]: df = pd.DataFrame([0], index=pd.DatetimeIndex(['2019-01-01'], tz='US/Pacific'))

In [16]: df
Out[16]:
```

	0
2019-01-01 00:00:00-08:00	0

```
[1 rows x 1 columns]
```

Previous behavior:

```
In [3]: df['2019-01-01 00:00:00+04:00':'2019-01-01 01:00:00+04:00']
Out[3]:
```

	0
2019-01-01 00:00:00-08:00	0

New behavior:

```
In [17]: df['2019-01-01 12:00:00+04:00':'2019-01-01 13:00:00+04:00']
Out[17]:
                0
2019-01-01 00:00:00-08:00  0

[1 rows x 1 columns]
```

MultiIndex由级别和代码构造

Constructing a `MultiIndex` with `NaN` levels or codes value `< -1` was allowed previously. Now, construction with codes value `< -1` is not allowed and `NaN` levels' corresponding codes would be reassigned as `-1`. ([GH19387](#))

Previous behavior:

```
In [1]: pd.MultiIndex(levels=[[np.nan, None, pd.NaT, 128, 2]],
...:                  codes=[[0, -1, 1, 2, 3, 4]])
...:
Out[1]: MultiIndex(levels=[[nan, None, NaT, 128, 2]],
                  codes=[[0, -1, 1, 2, 3, 4]])

In [2]: pd.MultiIndex(levels=[[1, 2]], codes=[[0, -2]])
Out[2]: MultiIndex(levels=[[1, 2]],
                  codes=[[0, -2]])
```

New behavior:

```

In [18]: pd.MultiIndex(levels=[[np.nan, None, pd.NaT, 128, 2]],
.....:                  codes=[[0, -1, 1, 2, 3, 4]])
.....:
Out[18]:
MultiIndex([(nan,),
            (nan,),
            (nan,),
            (nan,),
            (128,),
            ( 2,)],
           )

In [19]: pd.MultiIndex(levels=[[1, 2]], codes=[[0, -2]])
-----
ValueError                                Traceback (most recent call last)
<ipython-input-19-225a01af3975> in <module>
----> 1 pd.MultiIndex(levels=[[1, 2]], codes=[[0, -2]])

/pandas/pandas/util/_decorators.py in wrapper(*args, **kwargs)
    206         else:
    207             kwargs[new_arg_name] = new_arg_value
--> 208         return func(*args, **kwargs)
    209
    210     return wrapper

/pandas/pandas/core/indexes/multi.py in __new__(cls, levels, codes, sortorder, names, dtype, copy, name,
        verify_integrity, _set_identity)
    270
    271     if verify_integrity:
--> 272         new_codes = result._verify_integrity()
    273         result._codes = new_codes
    274

/pandas/pandas/core/indexes/multi.py in _verify_integrity(self, codes, levels)
    348         raise ValueError(
    349             "On level {level}, code value {code}"
--> 350             " < -1".format(level=i, code=level_codes.min())
    351         )
    352         if not level.is_unique:

ValueError: On level 0, code value (-2) < -1

```

DataFrame 上的 Groupby.apply 只对第一组求值一次

The implementation of `DataFrameGroupBy.apply()` previously evaluated the supplied function consistently twice on the first group to infer if it is safe to use a fast code path. Particularly for functions with side effects, this was an undesired behavior and may have led to surprises. ([GH2936](#), [GH2656](#), [GH7739](#), [GH10519](#), [GH12155](#), [GH20084](#), [GH21417](#))

Now every group is evaluated only a single time.

```
In [20]: df = pd.DataFrame({"a": ["x", "y"], "b": [1, 2]})

In [21]: df
Out[21]:
   a  b
0  x  1
1  y  2

[2 rows x 2 columns]

In [22]: def func(group):
....:     print(group.name)
....:     return group
....:
```

Previous behavior:

```
In [3]: df.groupby('a').apply(func)
x
x
y
Out[3]:
   a  b
0  x  1
1  y  2
```

New behavior:

```
In [23]: df.groupby("a").apply(func)
x
y
Out[23]:
   a  b
0  x  1
1  y  2

[2 rows x 2 columns]
```

连接稀疏值

When passed DataFrames whose values are sparse, `concat()` will now return a [Series](#) or [DataFrame](#) with sparse values, rather than a `SparseDataFrame` ([GH25702](#)).

```
In [24]: df = pd.DataFrame({"A": pd.SparseArray([0, 1])})
```

Previous behavior:

```
In [2]: type(pd.concat([df, df]))
pandas.core.sparse.frame.SparseDataFrame
```

New behavior:

```
In [25]: type(pd.concat([df, df]))
Out[25]: pandas.core.frame.DataFrame
```

This now matches the existing behavior of `concat` on `Series` with sparse values. `concat()` will continue to return a `SparseDataFrame` when all the values are instances of `SparseDataFrame`.

This change also affects routines using `concat()` internally, like `get_dummies()`, which now returns a `DataFrame` in all cases (previously a `SparseDataFrame` was returned if all the columns were dummy encoded, and a `DataFrame` otherwise).

Providing any `SparseSeries` or `SparseDataFrame` to `concat()` will cause a `SparseSeries` or `SparseDataFrame` to be returned, as before.

`.str`-访问器执行更严格的类型检查

Due to the lack of more fine-grained dtypes, `Series.str` so far only checked whether the data was of `object` dtype. `Series.str` will now infer the dtype data within the Series; in particular, `'bytes'`-only data will raise an exception (except for `Series.str.decode()`, `Series.str.get()`, `Series.str.len()`, `Series.str.slice()`), see [GH23163](#), [GH23011](#), [GH23551](#).

Previous behavior:

```
In [1]: s = pd.Series(np.array(['a', 'ba', 'cba'], 'S'), dtype=object)

In [2]: s
Out[2]:
0    b'a'
1    b'ba'
2    b'cba'
dtype: object

In [3]: s.str.startswith(b'a')
Out[3]:
0    True
1    False
2    False
dtype: bool
```

New behavior:

```

In [26]: s = pd.Series(np.array(['a', 'ba', 'cba'], 'S'), dtype=object)

In [27]: s
Out[27]:
0    b'a'
1    b'ba'
2    b'cba'
Length: 3, dtype: object

In [28]: s.str.startswith(b'a')
-----
TypeError                                Traceback (most recent call last)
<ipython-input-28-ac784692b361> in <module>
----> 1 s.str.startswith(b'a')

/pandas/pandas/core/strings.py in wrapper(self, *args, **kwargs)
    1840         )
    1841     )
-> 1842         raise TypeError(msg)
    1843     return func(self, *args, **kwargs)
    1844

TypeError: Cannot use .str.startswith with values of inferred dtype 'bytes'.

```

在groupby期间保留分类dtypes

Previously, columns that were categorical, but not the groupby key(s) would be converted to `object` dtype during groupby operations. Pandas now will preserve these dtypes. ([GH18502](#))

```

In [29]: cat = pd.Categorical(["foo", "bar", "bar", "qux"], ordered=True)

In [30]: df = pd.DataFrame({'payload': [-1, -2, -1, -2], 'col': cat})

In [31]: df
Out[31]:
   payload  col
0        -1  foo
1        -2  bar
2        -1  bar
3        -2  qux

[4 rows x 2 columns]

In [32]: df.dtypes
Out[32]:
payload    int64
col        category
Length: 2, dtype: object

```

Previous Behavior:

```
In [5]: df.groupby('payload').first().col.dtype
Out[5]: dtype('O')
```

New Behavior:

```
In [33]: df.groupby('payload').first().col.dtype
Out[33]: CategoricalDtype(categories=['bar', 'foo', 'qux'], ordered=True)
```

不兼容的索引类型联合

When performing `Index.union()` operations between objects of incompatible dtypes, the result will be a base `Index` of dtype `object`. This behavior holds true for unions between `Index` objects that previously would have been prohibited. The dtype of empty `Index` objects will now be evaluated before performing union operations rather than simply returning the other `Index` object. `Index.union()` can now be considered commutative, such that `A.union(B) == B.union(A)` ([GH23525](#)).

Previous behavior:

```
In [1]: pd.period_range('19910905', periods=2).union(pd.Int64Index([1, 2, 3]))
...
ValueError: can only call with other PeriodIndex-ed objects

In [2]: pd.Index([], dtype=object).union(pd.Index([1, 2, 3]))
Out[2]: Int64Index([1, 2, 3], dtype='int64')
```

New behavior:

```
In [34]: pd.period_range('19910905', periods=2).union(pd.Int64Index([1, 2, 3]))
Out[34]: Index([1991-09-05, 1991-09-06, 1, 2, 3], dtype='object')

In [35]: pd.Index([], dtype=object).union(pd.Index([1, 2, 3]))
Out[35]: Index([1, 2, 3], dtype='object')
```

Note that integer- and floating-dtype indexes are considered “compatible”. The integer values are coerced to floating point, which may result in loss of precision. See [Set operations on Index objects](#) for more.

DataFrame groupby ffill/bfill不再返回组标签

The methods `ffill`, `bfill`, `pad` and `backfill` of `DataFrameGroupBy` previously included the group labels in the return value, which was inconsistent with other groupby transforms. Now only the filled values are returned. ([GH21521](#))

```
In [36]: df = pd.DataFrame({"a": ["x", "y"], "b": [1, 2]})

In [37]: df
Out[37]:
   a  b
0  x  1
1  y  2

[2 rows x 2 columns]
```

Previous behavior:

```
In [3]: df.groupby("a").ffill()
Out[3]:
   a  b
0  x  1
1  y  2
```

New behavior:

```
In [38]: df.groupby("a").ffill()
Out[38]:
   b
0  1
1  2

[2 rows x 1 columns]
```

DataFrame 在空的分类/对象列上描述将返回top和freq

When calling `DataFrame.describe()` with an empty categorical / object column, the ‘top’ and ‘freq’ columns were previously omitted, which was inconsistent with the output for non-empty columns. Now the ‘top’ and ‘freq’ columns will always be included, with `numpy.nan` in the case of an empty `DataFrame` ([GH26397](#))

```
In [39]: df = pd.DataFrame({"empty_col": pd.Categorical([])})

In [40]: df
Out[40]:
Empty DataFrame
Columns: [empty_col]
Index: []

[0 rows x 1 columns]
```

Previous behavior:


```
In [3]: df.describe()
Out[3]:
      empty_col
count         0
unique         0
```

New behavior:

```
In [41]: df.describe()
Out[41]:
      empty_col
count         0
unique         0
top          NaN
freq          NaN

[4 rows x 1 columns]
```

`__str__`方法现在调用`__repr__`而不是反之亦然

Pandas has until now mostly defined string representations in a Pandas objects' s `__str__` / `__unicode__` / `__bytes__` methods, and called `__str__` from the `__repr__` method, if a specific `__repr__` method is not found. This is not needed for Python3. In Pandas 0.25, the string representations of Pandas objects are now generally defined in `__repr__`, and calls to `__str__` in general now pass the call on to the `__repr__`, if a specific `__str__` method doesn't exist, as is standard for Python. This change is backward compatible for direct usage of Pandas, but if you subclass Pandas objects and give your subclasses specific `__str__` / `__repr__` methods, you may have to adjust your `__str__` / `__repr__` methods ([GH26495](#)).

使用Interval对象索引IntervalIndex

Indexing methods for `IntervalIndex` have been modified to require exact matches only for `Interval` queries. `IntervalIndex` methods previously matched on any overlapping `Interval`. Behavior with scalar points, e.g. querying with an integer, is unchanged ([GH16316](#)).

```
In [42]: ii = pd.IntervalIndex.from_tuples([(0, 4), (1, 5), (5, 8)])

In [43]: ii
Out[43]:
IntervalIndex([(0, 4], (1, 5], (5, 8]],
              closed='right',
              dtype='interval[int64]')
```

The `in` operator (`__contains__`) now only returns `True` for exact matches to `Intervals` in the `IntervalIndex`, whereas this would previously return `True` for any `Interval` overlapping an `Interval` in the `IntervalIndex`.

Previous behavior:

```
In [4]: pd.Interval(1, 2, closed='neither') in ii
Out[4]: True

In [5]: pd.Interval(-10, 10, closed='both') in ii
Out[5]: True
```

New behavior:

```
In [44]: pd.Interval(1, 2, closed='neither') in ii
Out[44]: False

In [45]: pd.Interval(-10, 10, closed='both') in ii
Out[45]: False
```

The `get_loc()` method now only returns locations for exact matches to `Interval` queries, as opposed to the previous behavior of returning locations for overlapping matches. A `KeyError` will be raised if an exact match is not found.

Previous behavior:

```
In [6]: ii.get_loc(pd.Interval(1, 5))
Out[6]: array([0, 1])

In [7]: ii.get_loc(pd.Interval(2, 6))
Out[7]: array([0, 1, 2])
```

New behavior:

```
In [6]: ii.get_loc(pd.Interval(1, 5))
Out[6]: 1

In [7]: ii.get_loc(pd.Interval(2, 6))
-----
KeyError: Interval(2, 6, closed='right')
```

Likewise, `get_indexer()` and `get_indexer_non_unique()` will also only return locations for exact matches to `Interval` queries, with `-1` denoting that an exact match was not found.

These indexing changes extend to querying a `Series` or `DataFrame` with an `IntervalIndex` index.

```
In [46]: s = pd.Series(list('abc'), index=ii)
```

```
In [47]: s
Out[47]:
(0, 4]    a
(1, 5]    b
(5, 8]    c
Length: 3, dtype: object
```

Selecting from a `Series` or `DataFrame` using `[]` (`__getitem__`) or `loc` now only returns exact matches for `Interval` queries.

Previous behavior:

```
In [8]: s[pd.Interval(1, 5)]
Out[8]:
(0, 4]    a
(1, 5]    b
dtype: object

In [9]: s.loc[pd.Interval(1, 5)]
Out[9]:
(0, 4]    a
(1, 5]    b
dtype: object
```

New behavior:

```
In [48]: s[pd.Interval(1, 5)]
Out[48]: 'b'

In [49]: s.loc[pd.Interval(1, 5)]
Out[49]: 'b'
```

Similarly, a `KeyError` will be raised for non-exact matches instead of returning overlapping matches.

Previous behavior:

```
In [9]: s[pd.Interval(2, 3)]
Out[9]:
(0, 4]    a
(1, 5]    b
dtype: object

In [10]: s.loc[pd.Interval(2, 3)]
Out[10]:
(0, 4]    a
(1, 5]    b
dtype: object
```

New behavior:

```
In [6]: s[pd.Interval(2, 3)]
-----
KeyError: Interval(2, 3, closed='right')

In [7]: s.loc[pd.Interval(2, 3)]
-----
KeyError: Interval(2, 3, closed='right')
```

The `overlaps()` method can be used to create a boolean indexer that replicates the previous behavior of returning overlapping matches.

New behavior:

```
In [50]: idxr = s.index.overlaps(pd.Interval(2, 3))

In [51]: idxr
Out[51]: array([ True,  True, False])

In [52]: s[idxr]
Out[52]:
(0, 4]    a
(1, 5]    b
Length: 2, dtype: object

In [53]: s.loc[idxr]
Out[53]:
(0, 4]    a
(1, 5]    b
Length: 2, dtype: object
```

Series 上的二进制ufunc现在对齐

Applying a binary ufunc like `numpy.power()` now aligns the inputs when both are [Series](#) (GH23293).

```
In [54]: s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])

In [55]: s2 = pd.Series([3, 4, 5], index=['d', 'c', 'b'])

In [56]: s1
Out[56]:
a    1
b    2
c    3
Length: 3, dtype: int64

In [57]: s2
Out[57]:
d    3
c    4
b    5
Length: 3, dtype: int64
```

Previous behavior

```
In [5]: np.power(s1, s2)
Out[5]:
a      1
b     16
c    243
dtype: int64
```

New behavior

```
In [58]: np.power(s1, s2)
Out[58]:
a      1.0
b     32.0
c     81.0
d      NaN
Length: 4, dtype: float64
```

This matches the behavior of other binary operations in pandas, like `Series.add()`. To retain the previous behavior, convert the other `Series` to an array before applying the ufunc.

```
In [59]: np.power(s1, s2.array)
Out[59]:
a      1
b     16
c    243
Length: 3, dtype: int64
```

Categorical.argsort现在在最后放置缺失值

`Categorical.argsort()` now places missing values at the end of the array, making it consistent with NumPy and the rest of pandas ([GH21801](#)).

```
In [60]: cat = pd.Categorical(['b', None, 'a'], categories=['a', 'b'], ordered=True)
```

Previous behavior

```
In [2]: cat = pd.Categorical(['b', None, 'a'], categories=['a', 'b'], ordered=True)

In [3]: cat.argsort()
Out[3]: array([1, 2, 0])

In [4]: cat[cat.argsort()]
Out[4]:
[NaN, a, b]
categories (2, object): [a < b]
```

New behavior

```
In [61]: cat.argsort()
Out[61]: array([2, 0, 1])

In [62]: cat[cat.argsort()]
Out[62]:
[a, b, NaN]
Categories (2, object): [a < b]
```

将字典列表传递给DataFrame时，将保留列顺序

Starting with Python 3.7 the key-order of `dict` is [guaranteed](#). In practice, this has been true since Python 3.6. The `DataFrame` constructor now treats a list of dicts in the same way as it does a list of `OrderedDict`, i.e. preserving the order of the dicts. This change applies only when pandas is running on Python ≥ 3.6 ([GH27309](#)).

```
In [63]: data = [
.....:     {'name': 'Joe', 'state': 'NY', 'age': 18},
.....:     {'name': 'Jane', 'state': 'KY', 'age': 19, 'hobby': 'Minecraft'},
.....:     {'name': 'Jean', 'state': 'OK', 'age': 20, 'finances': 'good'}
.....: ]
.....:
```

Previous Behavior:

The columns were lexicographically sorted previously,

```
In [1]: pd.DataFrame(data)
Out[1]:
   age finances      hobby name state
0   18      NaN      NaN   Joe   NY
1   19      NaN  Minecraft  Jane   KY
2   20     good      NaN   Jean   OK
```

New Behavior:

The column order now matches the insertion-order of the keys in the `dict`, considering all the records from top to bottom. As a consequence, the column order of the resulting `DataFrame` has changed compared to previous pandas versions.

```
In [64]: pd.DataFrame(data)
Out[64]:
   name state age      hobby finances
0   Joe   NY  18      NaN      NaN
1  Jane   KY  19  Minecraft      NaN
2  Jean   OK  20      NaN     good

[3 rows x 5 columns]
```

增加了依赖项的最低版本

Due to dropping support for Python 2.7, a number of optional dependencies have updated minimum versions ([GH25725](#), [GH24942](#), [GH25752](#)). Independently, some minimum supported versions of dependencies were updated ([GH23519](#), [GH25554](#)). If installed, we now require:

Package	Minimum Version	Required
numpy	1.13.3	X
pytz	2015.4	X
python-dateutil	2.6.1	X
bottleneck	1.2.1	
numexpr	2.6.2	
pytest (dev)	4.0.2	

For [optional libraries](#) the general recommendation is to use the latest version. The following table lists the lowest version per library that is currently being tested throughout the development of pandas. Optional libraries below the lowest tested version may still work, but are not considered supported.

Package	Minimum Version
beautifulsoup4	4.6.0
fastparquet	0.2.1
gcsfs	0.2.2
lxml	3.8.0
matplotlib	2.2.2
openpyxl	2.4.8
pyarrow	0.9.0
pymysql	0.7.1
pytables	3.4.2
scipy	0.19.0
sqlalchemy	1.1.4
xarray	0.8.2
xlrd	1.1.0
xlsxwriter	0.9.8
xlwt	1.2.0

See [Dependencies](#) and [Optional dependencies](#) for more.

其他API更改

- `DatetimeTZDtype` will now standardize pytz timezones to a common timezone instance ([GH24713](#))
- `Timestamp` and `Timedelta` scalars now implement the `to_numpy()` method as aliases to `Timestamp.to_datetime64()` and `Timedelta.to_timedelta64()`, respectively. ([GH24653](#))
- `Timestamp.strptime()` will now raise a `NotImplementedError` ([GH25016](#))
- Comparing `Timestamp` with unsupported objects now returns `NotImplemented` instead of raising `TypeError`. This implies that unsupported rich comparisons are delegated to the other object, and are now consistent with Python 3 behavior for `datetime` objects ([GH24011](#))
- Bug in `DatetimeIndex.snap()` which didn't preserving the `name` of the input `Index` ([GH25575](#))
- The `arg` argument in `pandas.core.groupby.DataFrameGroupBy.agg()` has been renamed to `func` ([GH26089](#))
- The `arg` argument in `pandas.core.window.Window.aggregate()` has been renamed to `func` ([GH26372](#))
- Most Pandas classes had a `__bytes__` method, which was used for getting a python2-style bytestring representation of the object. This method has been removed as a part of dropping Python2 ([GH26447](#))
- The `.str`-accessor has been disabled for 1-level `MultiIndex`, use `MultiIndex.to_flat_index()` if necessary ([GH23679](#))
- Removed support of gtk package for clipboards ([GH26563](#))
- Using an unsupported version of Beautiful Soup 4 will now raise an `ImportError` instead of a `ValueError` ([GH27063](#))
- `Series.to_excel()` and `DataFrame.to_excel()` will now raise a `ValueError` when saving timezone aware data. ([GH27008](#), [GH7056](#))
- `ExtensionArray.argsort()` places NA values at the end of the sorted array. ([GH21801](#))
- `DataFrame.to_hdf()` and `Series.to_hdf()` will now raise a `NotImplementedError` when saving a `MultiIndex` with extension data types for a `fixed` format. ([GH7775](#))
- Passing duplicate `names` in `read_csv()` will now raise a `ValueError` ([GH17346](#))

弃用

Sparse的子类

The `SparseSeries` and `SparseDataFrame` subclasses are deprecated. Their functionality is better-provided by a `Series` or `DataFrame` with sparse values.

Previous way

```
In [65]: df = pd.SparseDataFrame({"A": [0, 0, 1, 2]})

In [66]: df.dtypes
Out[66]:
A    Sparse[int64, nan]
Length: 1, dtype: object
```


New way

```
In [67]: df = pd.DataFrame({"A": pd.SparseArray([0, 0, 1, 2])})

In [68]: df.dtypes
Out[68]:
A    Sparse[int64, 0]
Length: 1, dtype: object
```

The memory usage of the two approaches is identical. See [Migrating](#) for more ([GH19239](#)).

msgpack格式

The msgpack format is deprecated as of 0.25 and will be removed in a future version. It is recommended to use pyarrow for on-the-wire transmission of pandas objects. ([GH27084](#))

其他弃用

- The deprecated `.ix[]` indexer now raises a more visible `FutureWarning` instead of `DeprecationWarning` ([GH26438](#)).
- Deprecated the `units=M` (months) and `units=Y` (year) parameters for `units` of `pandas.to_timedelta()`, `pandas.Timedelta()` and `pandas.TimedeltaIndex()` ([GH16344](#))
- `pandas.concat()` has deprecated the `join_axes`-keyword. Instead, use `DataFrame.reindex()` or `DataFrame.reindex_like()` on the result or on the inputs ([GH21951](#))
- The `SparseArray.values` attribute is deprecated. You can use `np.asarray(...)` or the `SparseArray.to_dense()` method instead ([GH26421](#)).
- The functions `pandas.to_datetime()` and `pandas.to_timedelta()` have deprecated the `box` keyword. Instead, use `to_numpy()` or `Timestamp.to_datetime64()` or `Timedelta.to_timedelta64()`. ([GH24416](#))
- The `DataFrame.compound()` and `Series.compound()` methods are deprecated and will be removed in a future version ([GH26405](#)).
- The internal attributes `_start`, `_stop` and `_step` attributes of `RangeIndex` have been deprecated. Use the public attributes `start`, `stop` and `step` instead ([GH26581](#)).
- The `Series.ftype()`, `Series.ftypes()` and `DataFrame.ftypes()` methods are deprecated and will be removed in a future version. Instead, use `Series.dtype()` and `DataFrame.dtypes()` ([GH26705](#)).
- The `Series.get_values()`, `DataFrame.get_values()`, `Index.get_values()`, `SparseArray.get_values()` and `Categorical.get_values()` methods are deprecated. One of `np.asarray(...)` or `to_numpy()` can be used instead ([GH19617](#)).
- The ‘outer’ method on NumPy ufuncs, e.g. `np.subtract.outer` has been deprecated on `Series` objects. Convert the input to an array with `Series.array` first ([GH27186](#))
- `Timedelta.resolution()` is deprecated and replaced with `Timedelta.resolution_string()`. In a future version, `Timedelta.resolution()` will be changed to behave like the standard library `datetime.timedelta.resolution` ([GH21344](#))
- `read_table()` has been undeprecated. ([GH25220](#))
- `Index.dtype_str` is deprecated. ([GH18262](#))

- `Series.imag` and `Series.real` are deprecated. (GH18262)
- `Series.put()` is deprecated. (GH18262)
- `Index.item()` and `Series.item()` is deprecated. (GH18262)
- The default value `ordered=None` in `CategoricalDtype` has been deprecated in favor of `ordered=False`. When converting between categorical types `ordered=True` must be explicitly passed in order to be preserved. (GH26336)
- `Index.contains()` is deprecated. Use `key in index` (`__contains__`) instead (GH17753).
- `DataFrame.get_dtype_counts()` is deprecated. (GH18262)
- `Categorical.ravel()` will return a `Categorical` instead of a `np.ndarray` (GH27199)

删除先前版本的弃用/更改

- Removed `Panel` (GH25047, GH25191, GH25231)
- Removed the previously deprecated `sheetname` keyword in `read_excel()` (GH16442, GH20938)
- Removed the previously deprecated `TimeGrouper` (GH16942)
- Removed the previously deprecated `parse_cols` keyword in `read_excel()` (GH16488)
- Removed the previously deprecated `pd.options.html.border` (GH16970)
- Removed the previously deprecated `convert_objects` (GH11221)
- Removed the previously deprecated `select` method of `DataFrame` and `Series` (GH17633)
- Removed the previously deprecated behavior of `Series` treated as list-like in `rename_categories()` (GH17982)
- Removed the previously deprecated `DataFrame.reindex_axis` and `Series.reindex_axis` (GH17842)
- Removed the previously deprecated behavior of altering column or index labels with `Series.rename_axis()` or `DataFrame.rename_axis()` (GH17842)
- Removed the previously deprecated `tupleize_cols` keyword argument in `read_html()`, `read_csv()`, and `DataFrame.to_csv()` (GH17877, GH17820)
- Removed the previously deprecated `DataFrame.from_csv` and `Series.from_csv` (GH17812)
- Removed the previously deprecated `raise_on_error` keyword argument in `DataFrame.where()` and `DataFrame.mask()` (GH17744)
- Removed the previously deprecated `ordered` and `categories` keyword arguments in `astype` (GH17742)
- Removed the previously deprecated `cdate_range` (GH17691)
- Removed the previously deprecated `True` option for the `dropna` keyword argument in `SeriesGroupBy.nth()` (GH17493)
- Removed the previously deprecated `convert` keyword argument in `Series.take()` and `DataFrame.take()` (GH17352)

性能改进

- Significant speedup in `SparseArray` initialization that benefits most operations, fixing performance regression introduced in v0.20.0 (GH24985)

- `DataFrame.to_stata()` is now faster when outputting data with any string or non-native endian columns ([GH25045](#))
- Improved performance of `Series.searchsorted()`. The speedup is especially large when the dtype is int8/int16/int32 and the searched key is within the integer bounds for the dtype ([GH22034](#))
- Improved performance of `pandas.core.groupby.GroupBy.quantile()` ([GH20405](#))
- Improved performance of slicing and other selected operation on a `RangeIndex` ([GH26565](#), [GH26617](#), [GH26722](#))
- `RangeIndex` now performs standard lookup without instantiating an actual hashtable, hence saving memory ([GH16685](#))
- Improved performance of `read_csv()` by faster tokenizing and faster parsing of small float numbers ([GH25784](#))
- Improved performance of `read_csv()` by faster parsing of N/A and boolean values ([GH25804](#))
- Improved performance of `IntervalIndex.is_monotonic`, `IntervalIndex.is_monotonic_increasing` and `IntervalIndex.is_monotonic_decreasing` by removing conversion to `MultiIndex` ([GH24813](#))
- Improved performance of `DataFrame.to_csv()` when writing datetime dtypes ([GH25708](#))
- Improved performance of `read_csv()` by much faster parsing of `MM/YYYY` and `DD/MM/YYYY` datetime formats ([GH25922](#))
- Improved performance of `nanops` for dtypes that cannot store NaNs. Speedup is particularly prominent for `Series.all()` and `Series.any()` ([GH25070](#))
- Improved performance of `Series.map()` for dictionary mappers on categorical series by mapping the categories instead of mapping all values ([GH23785](#))
- Improved performance of `IntervalIndex.intersection()` ([GH24813](#))
- Improved performance of `read_csv()` by faster concatenating date columns without extra conversion to string for integer/float zero and float `NaN`; by faster checking the string for the possibility of being a date ([GH25754](#))
- Improved performance of `IntervalIndex.is_unique` by removing conversion to `MultiIndex` ([GH24813](#))
- Restored performance of `DatetimeIndex.__iter__()` by re-enabling specialized code path ([GH26702](#))
- Improved performance when building `MultiIndex` with at least one `CategoricalIndex` level ([GH22044](#))
- Improved performance by removing the need for a garbage collect when checking for `SettingWithCopyWarning` ([GH27031](#))
- For `to_datetime()` changed default value of cache parameter to `True` ([GH26043](#))
- Improved performance of `DatetimeIndex` and `PeriodIndex` slicing given non-unique, monotonic data ([GH27136](#)).
- Improved performance of `pd.read_json()` for index-oriented data. ([GH26773](#))
- Improved performance of `MultiIndex.shape()` ([GH27384](#)).

Bug修复

Categorical相关

- Bug in `DataFrame.at()` and `Series.at()` that would raise exception if the index was a `CategoricalIndex` (GH20629)
- Fixed bug in comparison of ordered `Categorical` that contained missing values with a scalar which sometimes incorrectly resulted in `True` (GH26504)
- Bug in `DataFrame.dropna()` when the `DataFrame` has a `CategoricalIndex` containing `Interval` objects incorrectly raised a `TypeError` (GH25087)

和Datetime相关的

- Bug in `to_datetime()` which would raise an (incorrect) `ValueError` when called with a date far into the future and the `format` argument specified instead of raising `OutOfBoundsDatetime` (GH23830)
- Bug in `to_datetime()` which would raise `InvalidIndexError: Reindexing only valid with uniquely valued Index objects` when called with `cache=True`, with `arg` including at least two different elements from the set `{None, numpy.nan, pandas.NaT}` (GH22305)
- Bug in `DataFrame` and `Series` where timezone aware data with `dtype='datetime64[ns]` was not cast to naive (GH25843)
- Improved `Timestamp` type checking in various datetime functions to prevent exceptions when using a subclassed `datetime` (GH25851)
- Bug in `Series` and `DataFrame repr` where `np.datetime64('NaT')` and `np.timedelta64('NaT')` with `dtype=object` would be represented as `NaN` (GH25445)
- Bug in `to_datetime()` which does not replace the invalid argument with `NaT` when error is set to `coerce` (GH26122)
- Bug in adding `DateOffset` with nonzero month to `DatetimeIndex` would raise `ValueError` (GH26258)
- Bug in `to_datetime()` which raises unhandled `OverflowError` when called with mix of invalid dates and `NaN` values with `format='%Y%m%d'` and `error='coerce'` (GH25512)
- Bug in `isin()` for datetimelike indexes; `DatetimeIndex`, `TimedeltaIndex` and `PeriodIndex` where the `levels` parameter was ignored. (GH26675)
- Bug in `to_datetime()` which raises `TypeError` for `format='%Y%m%d'` when called for invalid integer dates with length `>= 6` digits with `errors='ignore'`
- Bug when comparing a `PeriodIndex` against a zero-dimensional numpy array (GH26689)
- Bug in constructing a `Series` or `DataFrame` from a numpy `datetime64` array with a non-ns unit and out-of-bound timestamps generating rubbish data, which will now correctly raise an `OutOfBoundsDatetime` error (GH26206).
- Bug in `date_range()` with unnecessary `OverflowError` being raised for very large or very small dates (GH26651)
- Bug where adding `Timestamp` to a `np.timedelta64` object would raise instead of returning a `Timestamp` (GH24775)

- Bug where comparing a zero-dimensional numpy array containing a `np.datetime64` object to a `Timestamp` would incorrectly raise `TypeError` ([GH26916](#))
- Bug in `to_datetime()` which would raise `ValueError: Tz-aware datetime.datetime cannot be converted to datetime64 unless utc=True` when called with `cache=True`, with `arg` including datetime strings with different offset ([GH26097](#))

Timedelta相关

- Bug in `TimedeltaIndex.intersection()` where for non-monotonic indices in some cases an empty `Index` was returned when in fact an intersection existed ([GH25913](#))
- Bug with comparisons between `Timedelta` and `NaT` raising `TypeError` ([GH26039](#))
- Bug when adding or subtracting a `BusinessHour` to a `Timestamp` with the resulting time landing in a following or prior day respectively ([GH26381](#))
- Bug when comparing a `TimedeltaIndex` against a zero-dimensional numpy array ([GH26689](#))

Timezones相关

- Bug in `DatetimeIndex.to_frame()` where timezone aware data would be converted to timezone naive data ([GH25809](#))
- Bug in `to_datetime()` with `utc=True` and datetime strings that would apply previously parsed UTC offsets to subsequent arguments ([GH24992](#))
- Bug in `Timestamp.tz_localize()` and `Timestamp.tz_convert()` does not propagate `freq` ([GH25241](#))
- Bug in `Series.at()` where setting `Timestamp` with timezone raises `TypeError` ([GH25506](#))
- Bug in `DataFrame.update()` when updating with timezone aware data would return timezone naive data ([GH25807](#))
- Bug in `to_datetime()` where an uninformative `RuntimeError` was raised when passing a naive `Timestamp` with datetime strings with mixed UTC offsets ([GH25978](#))
- Bug in `to_datetime()` with `unit='ns'` would drop timezone information from the parsed argument ([GH26168](#))
- Bug in `DataFrame.join()` where joining a timezone aware index with a timezone aware column would result in a column of `NaN` ([GH26335](#))
- Bug in `date_range()` where ambiguous or nonexistent start or end times were not handled by the `ambiguous` or `nonexistent` keywords respectively ([GH27088](#))
- Bug in `DatetimeIndex.union()` when combining a timezone aware and timezone unaware `DatetimeIndex` ([GH21671](#))
- Bug when applying a numpy reduction function (e.g. `numpy.minimum()`) to a timezone aware `Series` ([GH15552](#))

Numeric相关

- Bug in `to_numeric()` in which large negative numbers were being improperly handled ([GH24910](#))

- Bug in `to_numeric()` in which numbers were being coerced to float, even though `errors` was not `coerce` (GH24910)
- Bug in `to_numeric()` in which invalid values for `errors` were being allowed (GH26466)
- Bug in `format` in which floating point complex numbers were not being formatted to proper display precision and trimming (GH25514)
- Bug in error messages in `DataFrame.corr()` and `Series.corr()`. Added the possibility of using a callable. (GH25729)
- Bug in `Series.divmod()` and `Series.rdivmod()` which would raise an (incorrect) `ValueError` rather than return a pair of `Series` objects as result (GH25557)
- Raises a helpful exception when a non-numeric index is sent to `interpolate()` with methods which require numeric index. (GH21662)
- Bug in `eval()` when comparing floats with scalar operators, for example: `x < -0.1` (GH25928)
- Fixed bug where casting all-boolean array to integer extension array failed (GH25211)
- Bug in `divmod` with a `Series` object containing zeros incorrectly raising `AttributeError` (GH26987)
- Inconsistency in `Series` floor-division (`//`) and `divmod` filling positive//zero with `NaN` instead of `Inf` (GH27321)
-

转换相关

- Bug in `DataFrame.astype()` when passing a dict of columns and types the `errors` parameter was ignored. (GH25905)

字符串相关

- Bug in the `__name__` attribute of several methods of `Series.str`, which were set incorrectly (GH23551)
- Improved error message when passing `Series` of wrong dtype to `Series.str.cat()` (GH22722)

“间隔” 相关

- Construction of `Interval` is restricted to numeric, `Timestamp` and `Timedelta` endpoints (GH23013)
- Fixed bug in `Series/DataFrame` not displaying `NaN` in `IntervalIndex` with missing values (GH25984)
- Bug in `IntervalIndex.get_loc()` where a `KeyError` would be incorrectly raised for a decreasing `IntervalIndex` (GH25860)
- Bug in `Index` constructor where passing mixed closed `Interval` objects would result in a `ValueError` instead of an `object` dtype `Index` (GH27172)

索引相关

- Improved exception message when calling `DataFrame.iloc()` with a list of non-numeric objects (GH25753).

- Improved exception message when calling `.iloc` or `.loc` with a boolean indexer with different length (GH26658).
- Bug in `KeyError` exception message when indexing a `MultiIndex` with a non-existent key not displaying the original key (GH27250).
- Bug in `.iloc` and `.loc` with a boolean indexer not raising an `IndexError` when too few items are passed (GH26658).
- Bug in `DataFrame.loc()` and `Series.loc()` where `KeyError` was not raised for a `MultiIndex` when the key was less than or equal to the number of levels in the `MultiIndex` (GH14885).
- Bug in which `DataFrame.append()` produced an erroneous warning indicating that a `KeyError` will be thrown in the future when the data to be appended contains new columns (GH22252).
- Bug in which `DataFrame.to_csv()` caused a segfault for a reindexed data frame, when the indices were single-level `MultiIndex` (GH26303).
- Fixed bug where assigning a `arrays.PandasArray` to a `pandas.core.frame.DataFrame` would raise error (GH26390)
- Allow keyword arguments for callable local reference used in the `DataFrame.query()` string (GH26426)
- Fixed a `KeyError` when indexing a ``MultiIndex`` level with a list containing exactly one label, which is missing (GH27148)
- Bug which produced `AttributeError` on partial matching `Timestamp` in a `MultiIndex` (GH26944)
- Bug in `Categorical` and `CategoricalIndex` with `Interval` values when using the `in` operator (`__contains__`) with objects that are not comparable to the values in the `Interval` (GH23705)
- Bug in `DataFrame.loc()` and `DataFrame.iloc()` on a `DataFrame` with a single timezone-aware `datetime64[ns]` column incorrectly returning a scalar instead of a `Series` (GH27110)
- Bug in `CategoricalIndex` and `Categorical` incorrectly raising `ValueError` instead of `TypeError` when a list is passed using the `in` operator (`__contains__`) (GH21729)
- Bug in setting a new value in a `Series` with a `Timedelta` object incorrectly casting the value to an integer (GH22717)
- Bug in `Series` setting a new key (`__setitem__`) with a timezone-aware datetime incorrectly raising `ValueError` (GH12862)
- Bug in `DataFrame.iloc()` when indexing with a read-only indexer (GH17192)
- Bug in `Series` setting an existing tuple key (`__setitem__`) with timezone-aware datetime values incorrectly raising `TypeError` (GH20441)

缺失 (Missing) 相关

- Fixed misleading exception message in `Series.interpolate()` if argument `order` is required, but omitted (GH10633, GH24014).
- Fixed class type displayed in exception message in `DataFrame.dropna()` if invalid `axis` parameter passed (GH25555)
- A `ValueError` will now be thrown by `DataFrame.fillna()` when `limit` is not a positive integer (GH27042)
-

多索引 (MultiIndex) 相关

- Bug in which incorrect exception raised by `Timedelta` when testing the membership of `MultiIndex` ([GH24570](#))
-

输入输出 (I/O) 相关

- Bug in `DataFrame.to_html()` where values were truncated using display options instead of outputting the full content ([GH17004](#))
- Fixed bug in missing text when using `to_clipboard()` if copying utf-16 characters in Python 3 on Windows ([GH25040](#))
- Bug in `read_json()` for `orient='table'` when it tries to infer dtypes by default, which is not applicable as dtypes are already defined in the JSON schema ([GH21345](#))
- Bug in `read_json()` for `orient='table'` and float index, as it infers index dtype by default, which is not applicable because index dtype is already defined in the JSON schema ([GH25433](#))
- Bug in `read_json()` for `orient='table'` and string of float column names, as it makes a column name type conversion to `Timestamp`, which is not applicable because column names are already defined in the JSON schema ([GH25435](#))
- Bug in `json_normalize()` for `errors='ignore'` where missing values in the input data, were filled in resulting `DataFrame` with the string "nan" instead of `numpy.nan` ([GH25468](#))
- `DataFrame.to_html()` now raises `TypeError` when using an invalid type for the `classes` parameter instead of `AssertionError` ([GH25608](#))
- Bug in `DataFrame.to_string()` and `DataFrame.to_latex()` that would lead to incorrect output when the `header` keyword is used ([GH16718](#))
- Bug in `read_csv()` not properly interpreting the UTF8 encoded filenames on Windows on Python 3.6+ ([GH15086](#))
- Improved performance in `pandas.read_stata()` and `pandas.io.stata.StataReader` when converting columns that have missing values ([GH25772](#))
- Bug in `DataFrame.to_html()` where header numbers would ignore display options when rounding ([GH17280](#))
- Bug in `read_hdf()` where reading a table from an HDF5 file written directly with PyTables fails with a `ValueError` when using a sub-selection via the `start` or `stop` arguments ([GH11188](#))
- Bug in `read_hdf()` not properly closing store after a `KeyError` is raised ([GH25766](#))
- Improved the explanation for the failure when value labels are repeated in Stata dta files and suggested work-arounds ([GH25772](#))
- Improved `pandas.read_stata()` and `pandas.io.stata.StataReader` to read incorrectly formatted 118 format files saved by Stata ([GH25960](#))
- Improved the `col_space` parameter in `DataFrame.to_html()` to accept a string so CSS length values can be set correctly ([GH25941](#))
- Fixed bug in loading objects from S3 that contain `#` characters in the URL ([GH25945](#))

- Adds `use_bqstorage_api` parameter to `read_gbq()` to speed up downloads of large data frames. This feature requires version 0.10.0 of the `pandas-gbq` library as well as the `google-cloud-bigquery-storage` and `fastavro` libraries. (GH26104)
- Fixed memory leak in `DataFrame.to_json()` when dealing with numeric data (GH24889)
- Bug in `read_json()` where date strings with `Z` were not converted to a UTC timezone (GH26168)
- Added `cache_dates=True` parameter to `read_csv()`, which allows to cache unique dates when they are parsed (GH25990)
- `DataFrame.to_excel()` now raises a `ValueError` when the caller's dimensions exceed the limitations of Excel (GH26051)
- Fixed bug in `pandas.read_csv()` where a BOM would result in incorrect parsing using `engine='python'` (GH26545)
- `read_excel()` now raises a `ValueError` when input is of type `pandas.io.excel.ExcelFile` and `engine` param is passed since `pandas.io.excel.ExcelFile` has an engine defined (GH26566)
- Bug while selecting from `HDFStore` with `where=''` specified (GH26610).
- Fixed bug in `DataFrame.to_excel()` where custom objects (i.e. `PeriodIndex`) inside merged cells were not being converted into types safe for the Excel writer (GH27006)
- Bug in `read_hdf()` where reading a timezone aware `DatetimeIndex` would raise a `TypeError` (GH11926)
- Bug in `to_msgpack()` and `read_msgpack()` which would raise a `ValueError` rather than a `FileNotFoundError` for an invalid path (GH27160)
- Fixed bug in `DataFrame.to_parquet()` which would raise a `ValueError` when the dataframe had no columns (GH27339)
- Allow parsing of `PeriodDtype` columns when using `read_csv()` (GH26934)

绘图 (Plotting) 相关

- Fixed bug where `api.extensions.ExtensionArray` could not be used in matplotlib plotting (GH25587)
- Bug in an error message in `DataFrame.plot()`. Improved the error message if non-numerics are passed to `DataFrame.plot()` (GH25481)
- Bug in incorrect ticklabel positions when plotting an index that are non-numeric / non-datetime (GH7612, GH15912, GH22334)
- Fixed bug causing plots of `PeriodIndex` timeseries to fail if the frequency is a multiple of the frequency rule code (GH14763)
- Fixed bug when plotting a `DatetimeIndex` with `datetime.timezone.utc` timezone (GH17173)

分组/重采样/滚动

- Bug in `pandas.core.resample.Resampler.agg()` with a timezone aware index where `OverflowError` would raise when passing a list of functions (GH22660)
- Bug in `pandas.core.groupby.DataFrameGroupBy.nunique()` in which the names of column levels were lost (GH23222)
- Bug in `pandas.core.groupby.GroupBy.agg()` when applying an aggregation function to timezone aware data (GH23683)

- Bug in `pandas.core.groupby.GroupBy.first()` and `pandas.core.groupby.GroupBy.last()` where timezone information would be dropped ([GH21603](#))
- Bug in `pandas.core.groupby.GroupBy.size()` when grouping only NA values ([GH23050](#))
- Bug in `Series.groupby()` where `observed` kwarg was previously ignored ([GH24880](#))
- Bug in `Series.groupby()` where using `groupby` with a `MultiIndex` Series with a list of labels equal to the length of the series caused incorrect grouping ([GH25704](#))
- Ensured that ordering of outputs in `groupby` aggregation functions is consistent across all versions of Python ([GH25692](#))
- Ensured that result group order is correct when grouping on an ordered `Categorical` and specifying `observed=True` ([GH25871](#), [GH25167](#))
- Bug in `pandas.core.window.Rolling.min()` and `pandas.core.window.Rolling.max()` that caused a memory leak ([GH25893](#))
- Bug in `pandas.core.window.Rolling.count()` and `pandas.core.window.Expanding.count` was previously ignoring the `axis` keyword ([GH13503](#))
- Bug in `pandas.core.groupby.GroupBy.idxmax()` and `pandas.core.groupby.GroupBy.idxmin()` with datetime column would return incorrect dtype ([GH25444](#), [GH15306](#))
- Bug in `pandas.core.groupby.GroupBy.cumsum()`, `pandas.core.groupby.GroupBy.cumprod()`, `pandas.core.groupby.GroupBy.cummin()` and `pandas.core.groupby.GroupBy.cummax()` with categorical column having absent categories, would return incorrect result or segfault ([GH16771](#))
- Bug in `pandas.core.groupby.GroupBy.nth()` where NA values in the grouping would return incorrect results ([GH26011](#))
- Bug in `pandas.core.groupby.SeriesGroupBy.transform()` where transforming an empty group would raise a `ValueError` ([GH26208](#))
- Bug in `pandas.core.frame.DataFrame.groupby()` where passing a `pandas.core.groupby.grouper.Grouper` would return incorrect groups when using the `.groups` accessor ([GH26326](#))
- Bug in `pandas.core.groupby.GroupBy.agg()` where incorrect results are returned for uint64 columns. ([GH26310](#))
- Bug in `pandas.core.window.Rolling.median()` and `pandas.core.window.Rolling.quantile()` where `MemoryError` is raised with empty window ([GH26005](#))
- Bug in `pandas.core.window.Rolling.median()` and `pandas.core.window.Rolling.quantile()` where incorrect results are returned with `closed='left'` and `closed='neither'` ([GH26005](#))
- Improved `pandas.core.window.Rolling`, `pandas.core.window.Window` and `pandas.core.window.EWM` functions to exclude nuisance columns from results instead of raising errors and raise a `DataError` only if all columns are nuisance ([GH12537](#))
- Bug in `pandas.core.window.Rolling.max()` and `pandas.core.window.Rolling.min()` where incorrect results are returned with an empty variable window ([GH26005](#))
- Raise a helpful exception when an unsupported weighted window function is used as an argument of `pandas.core.window.Window.aggregate()` ([GH26597](#))

重塑 (Reshaping) 相关

- Bug in `pandas.merge()` adds a string of `None`, if `None` is assigned in suffixes instead of remain the column name as-is ([GH24782](#)).
- Bug in `merge()` when merging by index name would sometimes result in an incorrectly numbered index (missing index values are now assigned NA) ([GH24212](#), [GH25009](#))
- `to_records()` now accepts dtypes to its `column_dtypes` parameter ([GH24895](#))
- Bug in `concat()` where order of `OrderedDict` (and `dict` in Python 3.6+) is not respected, when passed in as `objs` argument ([GH21510](#))
- Bug in `pivot_table()` where columns with `NaN` values are dropped even if `dropna` argument is `False`, when the `aggfunc` argument contains a `list` ([GH22159](#))
- Bug in `concat()` where the resulting `freq` of two `DatetimeIndex` with the same `freq` would be dropped ([GH3232](#)).
- Bug in `merge()` where merging with equivalent Categorical dtypes was raising an error ([GH22501](#))
- bug in `DataFrame` instantiating with a dict of iterators or generators (e.g. `pd.DataFrame({'A': reversed(range(3))})`) raised an error ([GH26349](#)).
- Bug in `DataFrame` instantiating with a `range` (e.g. `pd.DataFrame(range(3))`) raised an error ([GH26342](#)).
- Bug in `DataFrame` constructor when passing non-empty tuples would cause a segmentation fault ([GH25691](#))
- Bug in `Series.apply()` failed when the series is a timezone aware `DatetimeIndex` ([GH25959](#))
- Bug in `pandas.cut()` where large bins could incorrectly raise an error due to an integer overflow ([GH26045](#))
- Bug in `DataFrame.sort_index()` where an error is thrown when a multi-indexed `DataFrame` is sorted on all levels with the initial level sorted last ([GH26053](#))
- Bug in `Series.nlargest()` treats `True` as smaller than `False` ([GH26154](#))
- Bug in `DataFrame.pivot_table()` with a `IntervalIndex` as pivot index would raise `TypeError` ([GH25814](#))
- Bug in which `DataFrame.from_dict()` ignored order of `OrderedDict` when `orient='index'` ([GH8425](#)).
- Bug in `DataFrame.transpose()` where transposing a `DataFrame` with a timezone-aware datetime column would incorrectly raise `ValueError` ([GH26825](#))
- Bug in `pivot_table()` when pivoting a timezone aware column as the `values` would remove timezone information ([GH14948](#))
- Bug in `merge_asof()` when specifying multiple `by` columns where one is `datetime64[ns, tz]` dtype ([GH26649](#))

零散 (Sparse)

- Significant speedup in `SparseArray` initialization that benefits most operations, fixing performance regression introduced in v0.20.0 ([GH24985](#))
- Bug in `SparseFrame` constructor where passing `None` as the data would cause `default_fill_value` to be ignored ([GH16807](#))

- Bug in `SparseDataFrame` when adding a column in which the length of values does not match length of index, `AssertionError` is raised instead of raising `ValueError` ([GH25484](#))
- Introduce a better error message in `Series.sparse.from_coo()` so it returns a `TypeError` for inputs that are not coo matrices ([GH26554](#))
- Bug in `numpy.modf()` on a `SparseArray`. Now a tuple of `SparseArray` is returned ([GH26946](#)).

构建相关更改

- Fix install error with PyPy on macOS ([GH26536](#))

扩展数组

- Bug in `factorize()` when passing an `ExtensionArray` with a custom `na_sentinel` ([GH25696](#)).
- `Series.count()` miscounts NA values in `ExtensionArrays` ([GH26835](#))
- Added `Series.__array_ufunc__` to better handle NumPy ufuncs applied to Series backed by extension arrays ([GH23293](#)).
- Keyword argument `deep` has been removed from `ExtensionArray.copy()` ([GH27083](#))

其他

- Removed unused C functions from vendored UltraJSON implementation ([GH26198](#))
- Allow `Index` and `RangeIndex` to be passed to numpy `min` and `max` functions ([GH26125](#))
- Use actual class name in repr of empty objects of a `Series` subclass ([GH27001](#)).
- Bug in `DataFrame` where passing an object array of timezone-aware datetime objects would incorrectly raise `ValueError` ([GH13287](#))

贡献者

(译者注：官方未公布)

安装方法

安装

安装Pandas的最简单方法是将其安装为Anaconda发行版的一部分，这是一种用于数据分析和科学计算的跨平台发行版。这是大多数用户的推荐安装方法。

还提供了从源，PyPI，ActivePython，各种Linux发行版或开发版本进行安装的说明。

计划移除对Python 2.7的支持

Python核心团队计划在2020年1月1日停止支持Python 2.7。按照NumPy的计划，2018年12月31日之前的所有Pandas版本都仍支持Python 2（译者注：之后的版本将不再支持）。

2018年12月31日之前的最终版本将是支持Python 2的最后一个版本。已发布的软件包将继续在PyPI和conda上提供。

- 从 **2019年1月1日** 开始，所有版本都只支持Python 3。

如果有人对2018年12月31日之后对Python 2.7的持续支持感兴趣（反向移植错误修正或资金支持），请联系问题跟踪器上的维护人员。

有关更多信息，请参见 [Python 3 语法](#) 和 [移植到Python 3指南](#)。

Python版本支持

正式的 Python 2.7、3.5、3.6和3.7 版本。

安装 Pandas

通过Anaconda安装

对于没有经验的新手朋友来说，安装Pandas以及其余的NumPy和SciPy等工具包可能会有点困难。

最简单的方法不是直接安装Pandas，而是安装Python和构成SciPy数据科学技术栈的最流行的工具包（IPython，NumPy，Matplotlib，...）的集合Anaconda，它是一个跨平台（Linux，Mac OS X，Windows）的Python发行版，用于数据分析和科学计算。

在运行安装程序之后，用户将可以访问Pandas和SciPy工具包的任何工具，而不需要安装任何其他东西，也不需要等待任何软件编译，非常的方便。

可以在[此处](#)找到Anaconda的安装说明。

可以在[这里](#)找到作为Anaconda发行版一部分的可用软件包的完整列表。

安装Anaconda的另一个优点是安装它不需要管理员权限。Anaconda可以安装在用户的主目录中，如果您决定删除Anaconda（只需删除该文件夹就可以），这就非常方便了。

通过Miniconda安装

上一节概述了如何安装Pandas作为Anaconda 发行版的一部分。但是，这种方法意味着您将安装超过一百个包，并涉及下载几百兆字节的安装程序。

如果您只想要安装你需要的包，或者具有有限的互联网带宽，那么使用Miniconda安装Pandas可能是更好的解决方案。

Conda是Anaconda发行版所基于的软件包管理器。它是一个跨平台的且和语言无关的包管理器（它可以起到与pip和virtualenv组合类似的作用）。

Miniconda允许您创建最小的Python安装包，然后使用Conda命令安装其他软件包。

首先你需要安装Conda，下载并运行Miniconda会帮你做这件事。安装程序可以在[这里](#)找到。

下一步是创建一个新的conda环境。conda环境就像一个virtualenv，它允许您指定特定版本的Python和你需要的第三方库。从终端窗口运行以下命令：

```
$ conda create -n name_of_my_env python
```

这将创建一个只安装了Python的最小环境。把你自己放在这个环境中（激活环境）：

```
$ source activate name_of_my_env
```

在Windows上，命令是：

```
$ activate name_of_my_env
```

最后一步是安装Pandas，可以使用以下命令完成：

```
$ conda install Pandas
```

要安装特定的Pandas版本：

```
$ conda install Pandas=0.20.3
```

要安装其他软件包，例如IPython：

```
$ conda install ipython
```

要安装完整的Anaconda发行版：

```
$ conda install anaconda
```

如果您需要可用于pip而不是conda的软件包，请安装pip，然后使用pip安装这些软件包：

```
$ conda install pip  
$ pip install django
```

从PyPI安装

Pandas可以通过PyPI的pip安装。

```
$ pip install Pandas
```

通过ActivePython安装

可以在[此处](#)找到ActivePython的安装说明。版本2.7和3.5 包含了 pandas。

使用Linux发行版的包管理器进行安装。

此表中的命令将从您的发行版安装Python 3版本的Pandas。要安装Python 2版本的Pandas，您可能需要使用 `python-Pandas` 包。

发行版名称	状态	下载 / 仓库地址	安装方法
Debian	stable	official Debian repository	<code>sudo apt-get install</code>
Debian & Ubuntu	unstable (latest packages)	NeuroDebian	
Ubuntu	stable	official Ubuntu repository	<code>sudo apt-get install python3-Pandas</code>
OpenSuse	stable	OpenSuse Repository	<code>zypper in python3-Pandas</code>
Fedora	stable	official Fedora repository	<code>dnf install python3-Pandas</code>
Centos/RHEL	stable	EPEL repository	<code>yum install python3-Pandas</code>

However，Linux的包管理器中的包通常落后几个版本，因此要获得最新版本的Pandas，建议使用上面描述的 `pip` 或 `conda` 方法进行安装。

从源码安装

有关从git源代码树构建的完整说明，请参阅[贡献指南](#)。此外，如果您希望创建一个pandas开发环境，请参阅[创建开发环境](#)。

运行测试套件

大Pandas配备了一套详尽的单元测试，涵盖了撰写本文时约97%的代码库。要在您的计算机上运行它以验证一切正常（并且您已经安装了所有依赖项，软的和硬的），请确保您有 `pytest >= 4.0.2` 和 `Hypothesis >= 3.58`，然后运行：

```
>>> import Pandas as pd
>>> pd.test()
running: pytest --skip-slow --skip-network C:\Users\TP\Anaconda3\envs\py36\Lib\site-packages\Pandas
===== test session starts =====
platform win32 -- Python 3.6.2, pytest-3.2.1, py-1.4.34, pluggy-0.4.0
rootdir: C:\Users\TP\Documents\Python\Pandasdev\Pandas, inifile: setup.cfg
collected 12145 items / 3 skipped

.....S.....
.....S.....
.....

===== 12130 passed, 12 skipped in 368.339 seconds =====
```

依赖

Package	最低支持版本
setuptools	24.2.0
NumPy	1.13.3
python-dateutil	2.6.1
pytz	2017.2

推荐的依赖关系

- [numexpr](#): 用于加速某些数值运算。`numexpr` 使用多个内核以及智能分块和缓存来实现大型加速。如果已安装，则必须为2.6.2或更高版本。
- [bottleneck](#): 加速某些类型的 `nan` 评估。`bottleneck` 使用专门的cython例程来实现大的加速。如果已安装，则必须为1.2.1或更高版本。

::: tip 注意

强烈建议您安装这些库，因为它们可以提高处理速度，尤其是在处理大型数据集时。

:::

可选的依赖项

Pandas有许多可选的依赖项，仅用于特定的方法。例如，`pandas.read_hdf()` 需要 `pytables` 包。如果未安装可选依赖项，则在调用需要该依赖项的方法时，pandas将引发 `ImportError`。

依赖名称	最低版本	注意
BeautifulSoup4	4.6.0	HTML parser for read_html (see note)
Jinja2		Conditional formatting with DataFrame.style
PyQt4		Clipboard I/O
PyQt5		Clipboard I/O
PyTables	3.4.2	HDF5-based reading / writing
SQLAlchemy	1.1.4	SQL support for databases other than sqlite
SciPy	0.19.0	Miscellaneous statistical functions
XLsxWriter	0.9.8	Excel writing
blosc		Compression for msgpack
fastparquet	0.2.1	Parquet reading / writing
gcsfs	0.2.2	Google Cloud Storage access
html5lib		HTML parser for read_html (see note)
lxml	3.8.0	HTML parser for read_html (see note)
matplotlib	2.2.2	Visualization
openpyxl	2.4.8	Reading / writing for xlsx files
pandas-gbq	0.8.0	Google Big Query access
psycopg2		PostgreSQL engine for sqlalchemy
pyarrow	0.9.0	Parquet and feather reading / writing
pymysql	0.7.11	MySQL engine for sqlalchemy
pyreadstat		SPSS files (.sav) reading
pytables	3.4.2	HDF5 reading / writing
qtpy		Clipboard I/O
s3fs	0.0.8	Amazon S3 access
xarray	0.8.2	pandas-like API for N-dimensional data
xclip		Clipboard I/O on linux
xlrd	1.1.0	Excel reading
xlwt	1.2.0	Excel writing
xsel		Clipboard I/O on linux
zlib		Compression for msgpack

用于解析HTML的可选依赖项

要使用顶级[read_html\(\)](#)函数，需要以下一种库组合：

Changed in version 0.23.0.

- [BeautifulSoup4](#) 和 [html5lib](#)
- [BeautifulSoup4](#) 和 [lxml](#)
- [BeautifulSoup4](#) 和 [html5lib](#) 和 [lxml](#)
- 只有 [lxml](#)，请参阅[HTML表解析](#)，以了解为什么您可能**不应该**采用这种方法。

::: danger 警告

- 如果您安装[BeautifulSoup4](#)，您必须安装[lxml](#)或[html5lib](#)或两个都安装。[read_html\(\)](#) 不能只安装 [BeautifulSoup4](#)。
- 我们强烈建议您阅读 [HTML表解析之坑](#)。它解释了有关上述三个库的安装和使用的问题。

:::

快速入门

快速入门

- [Pandas概览](#)
 - [数据结构](#)
 - [大小可变与数据复制](#)
 - [获得支持](#)
 - [社区](#)
 - [项目监管](#)
 - [开发团队](#)
 - [机构合作伙伴](#)
 - [许可协议](#)
- [十分钟入门Pandas](#)
 - [对象创建](#)
 - [查看数据](#)
 - [选择](#)
 - [缺失值](#)
 - [操作](#)
 - [合并 \(Merge\)](#)
 - [分组 \(Grouping\)](#)
 - [重塑 \(Reshaping\)](#)
 - [时间序列 \(Time Series\)](#)
 - [分类](#)

- 绘图
- 数据输入 / 输出
- 坑 (Gotchas)
- 基础用法
 - Head 与 Tail
 - 属性与底层数据
 - 加速操作
 - 二进制操作
 - 描述性统计
 - 函数
 - 重置索引与更换标签
 - 迭代
 - .dt 访问器
 - 矢量化字符串方法
 - 排序
 - 复制
 - 数据类型
 - 基于 dtype 选择列
- 数据结构简介
 - Series
 - DataFrame
- 与其它工具比较
 - 与 R 语言比较
 - 与 SQL 比较
 - 与 SAS 比较
 - 与 Stata 比较
- 教程资料
 - 官方指南
 - 社区指南

Pandas 概览

Pandas 是 **Python** 的核心数据分析支持库，提供了快速、灵活、明确的数据结构，旨在简单、直观地处理关系型、标记型数据。Pandas 的目标是成为 Python 数据分析实践与实战的必备高级工具，其长远目标是成为**最强大、最灵活、可以支持任何语言的开源数据分析工具**。经过多年不懈的努力，Pandas 离这个目标已经越来越近了。

Pandas 适用于处理以下类型的数据：

- 与 SQL 或 Excel 表类似的，含异构列的表格数据；
- 有序和无序（非固定频率）的时间序列数据；
- 带行列标签的矩阵数据，包括同构或异构型数据；

- 任意其它形式的观测、统计数据集, 数据转入 Pandas 数据结构时不必事先标记。

Pandas 的主要数据结构是 [Series](#)（一维数据）与 [DataFrame](#)（二维数据），这两种数据结构足以处理金融、统计、社会科学、工程等领域里的大多数典型用例。对于 R 用户，DataFrame 提供了比 R 语言 `data.frame` 更丰富的功能。Pandas 基于 [NumPy](#) 开发，可以与其它第三方科学计算支持库完美集成。

Pandas 就像一把万能瑞士军刀，下面仅列出了它的部分优势：

- 处理浮点与非浮点数据里的**缺失数据**，表示为 `NaN`；
- 大小可变：**插入或删除** DataFrame 等多维对象的列；
- 自动、显式**数据对齐**：显式地将对象与一组标签对齐，也可以忽略标签，在 Series、DataFrame 计算时自动与数据对齐；
- 强大、灵活的分组（group by）功能：**拆分-应用-组合**数据集，聚合、转换数据；
- 把 Python 和 NumPy 数据结构里不规则、不同索引的数据**轻松地**转换为 DataFrame 对象；
- 基于智能标签，对大型数据集进行**切片、花式索引、子集分解**等操作；
- 直观地**合并（merge）**、****连接（join）****数据集；
- 灵活地**重塑（reshape）**、****透视（pivot）****数据集；
- **轴**支持结构化标签：一个刻度支持多个标签；
- 成熟的 IO 工具：读取**文本文件**（CSV 等支持分隔符的文件）、Excel 文件、数据库等来源的数据，利用超快的 **HDF5** 格式保存 / 加载数据；
- **时间序列**：支持日期范围生成、频率转换、移动窗口统计、移动窗口线性回归、日期位移等时间序列功能。

这些功能主要是为了解决其它编程语言、科研环境的痛点。处理数据一般分为几个阶段：数据整理与清洗、数据分析与建模、数据可视化与制表，Pandas 是处理数据的理想工具。

其它说明：

- Pandas **速度很快**。Pandas 的很多底层算法都用 [Cython](#) 优化过。然而，为了保持通用性，必然要牺牲一些性能，如果专注某一功能，完全可以开发出比 Pandas 更快的专用工具。
- Pandas 是 [statsmodels](#) 的依赖项，因此，Pandas 也是 Python 中统计计算生态系统的重要组成部分。
- Pandas 已广泛应用于金融领域。

数据结构

维数	名称	描述
1	Series	带标签的一维同构数组
2	DataFrame	带标签的，大小可变的，二维异构表格

为什么有多个数据结构？

Pandas 数据结构就像是低维数据的容器。比如，DataFrame 是 Series 的容器，Series 则是标量的容器。使用这种方式，可以在容器中以字典的形式插入或删除对象。

此外，通用 API 函数的默认操作要顾及时间序列与截面数据集的方向。多维数组存储二维或三维数据时，编写函数要注意数据集的方向，这对用户来说是一种负担；如果不考虑 C 或 Fortran 中连续性对性能的影响，一般情况下，不同的轴在程序里其实没有什么区别。Pandas 里，轴的概念主要是为了给数据赋予更直观的语义，即用“更恰当”的方式表示数据集的方向。这样做可以让用户编写数据转换函数时，少费点脑子。

处理 DataFrame 等表格数据时，**index**（行）或 **columns**（列）比 **axis 0** 和 **axis 1** 更直观。用这种方式迭代 DataFrame 的列，代码更易读易懂：

```
for col in df.columns:
    series = df[col]
    # do something with series
```

大小可变与数据复制

Pandas 所有数据结构的值都是可变的，但数据结构的大小并非都是可变的，比如，Series 的长度不可改变，但 DataFrame 里就可以插入列。

Pandas 里，绝大多数方法都不改变原始的输入数据，而是复制数据，生成新的对象。一般来说，原始输入数据**不变更**稳妥。

获得支持

发现 Pandas 的问题或有任何建议，请反馈到 [Github 问题跟踪器](#)。日常应用问题请在 [Stack Overflow](#) 上咨询 Pandas 社区专家。

社区

Pandas 如今由来自全球的同道中人组成的社区提供支持，社区里的每个人都贡献了宝贵的时间和精力，正因如此，才成就了开源 Pandas，在此，我们要感谢[所有贡献者](#)。

若您有意为 Pandas 贡献自己的力量，请先阅读[贡献指南](#)。

Pandas 是 [NumFOCUS](#) 赞助的项目。有了稳定的资金来源，就确保了 Pandas，这一世界级开源项目的成功，为本项目[捐款](#)也更有保障。

项目监管

自 2008 年以来，Pandas 沿用的监管流程已正式编纂为[项目监管文档](#)。这些文件阐明了如何决策，如何处理营利组织与非营利实体进行开源协作开发的关系等内容。

Wes McKinney 是仁慈的终身独裁者。

开发团队

核心团队成員列表及詳細信息可在 Github 仓库的[人員页面](#)上查詢。

机构合作伙伴

現有机构合作伙伴信息可在 [Pandas 网站页面](#)上查詢。

许可协议

BSD 3-Clause License

Copyright (c) 2008-2012, AQR Capital Management, LLC, Lambda Foundry, Inc. and PyData Development Team
All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- * Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- * Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- * Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

十分钟入门 Pandas

本节是帮助 Pandas 新手快速上手的简介。[烹饪指南](#)里介绍了更多实用案例。

本节以下列方式导入 Pandas 与 NumPy:

```
In [1]: import numpy as np

In [2]: import pandas as pd
```

生成对象

详见[数据结构简介](#)文档。

用值列表生成 [Series](#) 时, Pandas 默认自动生成整数索引:

```
In [3]: s = pd.Series([1, 3, 5, np.nan, 6, 8])

In [4]: s
Out[4]:
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
dtype: float64
```

用含日期时间索引与标签的 NumPy 数组生成 [DataFrame](#):

```
In [5]: dates = pd.date_range('20130101', periods=6)

In [6]: dates
Out[6]:
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')

In [7]: df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list('ABCD'))

In [8]: df
Out[8]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401
2013-01-06	-0.673690	0.113648	-1.478427	0.524988

用 Series 字典对象生成 DataFrame:

```
In [9]: df2 = pd.DataFrame({'A': 1.,
...:                        'B': pd.Timestamp('20130102'),
...:                        'C': pd.Series(1, index=list(range(4)), dtype='float32'),
...:                        'D': np.array([3] * 4, dtype='int32'),
...:                        'E': pd.Categorical(["test", "train", "test", "train"]),
...:                        'F': 'foo'})
```

```
In [10]: df2
```

```
Out[10]:
```

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	1.0	2013-01-02	1.0	3	train	foo
2	1.0	2013-01-02	1.0	3	test	foo
3	1.0	2013-01-02	1.0	3	train	foo

DataFrame 的列有不同数据类型。

```
In [11]: df2.dtypes
```

```
Out[11]:
```

```
A          float64
B    datetime64[ns]
C          float32
D           int32
E          category
F           object
dtype: object
```

IPython支持 tab 键自动补全列名与公共属性。下面是部分可自动补全的属性：

```
In [12]: df2.<TAB> # noqa: E225, E999
df2.A          df2.bool
df2.abs        df2.boxplot
df2.add        df2.C
df2.add_prefix df2.clip
df2.add_suffix df2.clip_lower
df2.align      df2.clip_upper
df2.all        df2.columns
df2.any        df2.combine
df2.append     df2.combine_first
df2.apply      df2.compound
df2.applymap   df2.consolidate
df2.D
```

列 A、B、C、D 和 E 都可以自动补全；为简洁起见，此处只显示了部分属性。

查看数据

详见[基础用法](#)文档。

下列代码说明如何查看 DataFrame 头部和尾部数据：


```
In [13]: df.head()
Out[13]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401

```
In [14]: df.tail(3)
Out[14]:
```

	A	B	C	D
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401
2013-01-06	-0.673690	0.113648	-1.478427	0.524988

显示索引与列名：

```
In [15]: df.index
Out[15]:
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')

In [16]: df.columns
Out[16]: Index(['A', 'B', 'C', 'D'], dtype='object')
```

`DataFrame.to_numpy()` 输出底层数据的 NumPy 对象。注意，`DataFrame` 的列由多种数据类型组成时，该操作耗费系统资源较大，这也是 Pandas 和 NumPy 的本质区别：**NumPy 数组只有一种数据类型，DataFrame 每列的数据类型各不相同**。调用 `DataFrame.to_numpy()` 时，Pandas 查找支持 DataFrame 里所有数据类型的 NumPy 数据类型。还有一种数据类型是 `object`，可以把 DataFrame 列里的值强制转换为 Python 对象。

下面的 `df` 这个 `DataFrame` 里的值都是浮点数，`DataFrame.to_numpy()` 的操作会很快，而且不复制数据。

```
In [17]: df.to_numpy()
Out[17]:
array([[ 0.4691, -0.2829, -1.5091, -1.1356],
       [ 1.2121, -0.1732,  0.1192, -1.0442],
       [-0.8618, -2.1046, -0.4949,  1.0718],
       [ 0.7216, -0.7068, -1.0396,  0.2719],
       [-0.425 ,  0.567 ,  0.2762, -1.0874],
       [-0.6737,  0.1136, -1.4784,  0.525 ]])
```

`df2` 这个 `DataFrame` 包含了多种类型，`DataFrame.to_numpy()` 操作就会耗费较多资源。

```
In [18]: df2.to_numpy()
Out[18]:
array([[1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo']], dtype=object)
```

::: tip 提醒

`DataFrame.to_numpy()` 的输出不包含行索引和列标签。

:::

`describe()` 可以快速查看数据的统计摘要：

```
In [19]: df.describe()
Out[19]:
```

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	0.073711	-0.431125	-0.687758	-0.233103
std	0.843157	0.922818	0.779887	0.973118
min	-0.861849	-2.104569	-1.509059	-1.135632
25%	-0.611510	-0.600794	-1.368714	-1.076610
50%	0.022070	-0.228039	-0.767252	-0.386188
75%	0.658444	0.041933	-0.034326	0.461706
max	1.212112	0.567020	0.276232	1.071804

转置数据：

```
In [20]: df.T
Out[20]:
```

	2013-01-01	2013-01-02	2013-01-03	2013-01-04	2013-01-05	2013-01-06
A	0.469112	1.212112	-0.861849	0.721555	-0.424972	-0.673690
B	-0.282863	-0.173215	-2.104569	-0.706771	0.567020	0.113648
C	-1.509059	0.119209	-0.494929	-1.039575	0.276232	-1.478427
D	-1.135632	-1.044236	1.071804	0.271860	-1.087401	0.524988

按轴排序：

```
In [21]: df.sort_index(axis=1, ascending=False)
Out[21]:
```

	D	C	B	A
2013-01-01	-1.135632	-1.509059	-0.282863	0.469112
2013-01-02	-1.044236	0.119209	-0.173215	1.212112
2013-01-03	1.071804	-0.494929	-2.104569	-0.861849
2013-01-04	0.271860	-1.039575	-0.706771	0.721555
2013-01-05	-1.087401	0.276232	0.567020	-0.424972
2013-01-06	0.524988	-1.478427	0.113648	-0.673690

按值排序：

```
In [22]: df.sort_values(by='B')
Out[22]:
```

	A	B	C	D
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-06	-0.673690	0.113648	-1.478427	0.524988
2013-01-05	-0.424972	0.567020	0.276232	-1.087401

选择

⋮ tip 提醒

选择、设置标准 Python / Numpy 的表达式已经非常直观，交互也很方便，但对于生产代码，我们还是推荐优化过的 Pandas 数据访问方法：`.at`、`.iat`、`.loc` 和 `.iloc`。

⋮

详见[索引与选择数据](#)、[多层索引与高级索引](#)文档。

获取数据

选择单列，产生 `Series`，与 `df.A` 等效：

```
In [23]: df['A']
Out[23]:
```

2013-01-01	0.469112
2013-01-02	1.212112
2013-01-03	-0.861849
2013-01-04	0.721555
2013-01-05	-0.424972
2013-01-06	-0.673690

Freq: D, Name: A, dtype: float64

用 `[]` 切片行：

```
In [24]: df[0:3]
Out[24]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804

```
In [25]: df['20130102':'20130104']
Out[25]:
```

	A	B	C	D
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860

按标签选择

详见[按标签选择](#)。

用标签提取一行数据：

```
In [26]: df.loc[dates[0]]
Out[26]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632

Name: 2013-01-01 00:00:00, dtype: float64

用标签选择多列数据：

```
In [27]: df.loc[:, ['A', 'B']]
Out[27]:
```

	A	B
2013-01-01	0.469112	-0.282863
2013-01-02	1.212112	-0.173215
2013-01-03	-0.861849	-2.104569
2013-01-04	0.721555	-0.706771
2013-01-05	-0.424972	0.567020
2013-01-06	-0.673690	0.113648

用标签切片，包含行与列结束点：

```
In [28]: df.loc['20130102':'20130104', ['A', 'B']]
Out[28]:
```

	A	B
2013-01-02	1.212112	-0.173215
2013-01-03	-0.861849	-2.104569
2013-01-04	0.721555	-0.706771

返回对象降维：

```
In [29]: df.loc['20130102', ['A', 'B']]
Out[29]:
A    1.212112
B   -0.173215
Name: 2013-01-02 00:00:00, dtype: float64
```

提取标量值：

```
In [30]: df.loc[dates[0], 'A']
Out[30]: 0.46911229990718628
```

快速访问标量，与上述方法等效：

```
In [31]: df.at[dates[0], 'A']
Out[31]: 0.46911229990718628
```

按位置选择

详见[按位置选择](#)。

用整数位置选择：

```
In [32]: df.iloc[3]
Out[32]:
A    0.721555
B   -0.706771
C   -1.039575
D    0.271860
Name: 2013-01-04 00:00:00, dtype: float64
```

类似 NumPy / Python，用整数切片：

```
In [33]: df.iloc[3:5, 0:2]
Out[33]:
```

	A	B
2013-01-04	0.721555	-0.706771
2013-01-05	-0.424972	0.567020

类似 NumPy / Python，用整数列表按位置切片：

```
In [34]: df.iloc[[1, 2, 4], [0, 2]]
Out[34]:
```

	A	C
2013-01-02	1.212112	0.119209
2013-01-03	-0.861849	-0.494929
2013-01-05	-0.424972	0.276232

显式整行切片：

```
In [35]: df.iloc[1:3, :]  
Out[35]:
```

	A	B	C	D
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804

显式整列切片：

```
In [36]: df.iloc[:, 1:3]  
Out[36]:
```

	B	C
2013-01-01	-0.282863	-1.509059
2013-01-02	-0.173215	0.119209
2013-01-03	-2.104569	-0.494929
2013-01-04	-0.706771	-1.039575
2013-01-05	0.567020	0.276232
2013-01-06	0.113648	-1.478427

显式提取值：

```
In [37]: df.iloc[1, 1]  
Out[37]: -0.17321464905330858
```

快速访问标量，与上述方法等效：

```
In [38]: df.iat[1, 1]  
Out[38]: -0.17321464905330858
```

布尔索引

用单列的值选择数据：

```
In [39]: df[df.A > 0]  
Out[39]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-04	0.721555	-0.706771	-1.039575	0.271860

选择 DataFrame 里满足条件的值：

```
In [40]: df[df > 0]
Out[40]:
```

	A	B	C	D
2013-01-01	0.469112	NaN	NaN	NaN
2013-01-02	1.212112	NaN	0.119209	NaN
2013-01-03	NaN	NaN	NaN	1.071804
2013-01-04	0.721555	NaN	NaN	0.271860
2013-01-05	NaN	0.567020	0.276232	NaN
2013-01-06	NaN	0.113648	NaN	0.524988

用 `isin()` 筛选：

```
In [41]: df2 = df.copy()

In [42]: df2['E'] = ['one', 'one', 'two', 'three', 'four', 'three']

In [43]: df2
Out[43]:
```

	A	B	C	D	E
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632	one
2013-01-02	1.212112	-0.173215	0.119209	-1.044236	one
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804	two
2013-01-04	0.721555	-0.706771	-1.039575	0.271860	three
2013-01-05	-0.424972	0.567020	0.276232	-1.087401	four
2013-01-06	-0.673690	0.113648	-1.478427	0.524988	three

```
In [44]: df2[df2['E'].isin(['two', 'four'])]
Out[44]:
```

	A	B	C	D	E
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804	two
2013-01-05	-0.424972	0.567020	0.276232	-1.087401	four

赋值

用索引自动对齐新增列的数据：

```
In [45]: s1 = pd.Series([1, 2, 3, 4, 5, 6], index=pd.date_range('20130102', periods=6))

In [46]: s1
Out[46]:
```

2013-01-02	1
2013-01-03	2
2013-01-04	3
2013-01-05	4
2013-01-06	5
2013-01-07	6

```
Freq: D, dtype: int64

In [47]: df['F'] = s1
```

按标签赋值：

```
In [48]: df.at[dates[0], 'A'] = 0
```

按位置赋值：

```
In [49]: df.iat[0, 1] = 0
```

按 NumPy 数组赋值：

```
In [50]: df.loc[:, 'D'] = np.array([5] * len(df))
```

上述赋值结果：

```
In [51]: df
Out[51]:
```

	A	B	C	D	F
2013-01-01	0.000000	0.000000	-1.509059	5	NaN
2013-01-02	1.212112	-0.173215	0.119209	5	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5	2.0
2013-01-04	0.721555	-0.706771	-1.039575	5	3.0
2013-01-05	-0.424972	0.567020	0.276232	5	4.0
2013-01-06	-0.673690	0.113648	-1.478427	5	5.0

用 where 条件赋值：

```
In [52]: df2 = df.copy()

In [53]: df2[df2 > 0] = -df2

In [54]: df2
Out[54]:
```

	A	B	C	D	F
2013-01-01	0.000000	0.000000	-1.509059	-5	NaN
2013-01-02	-1.212112	-0.173215	-0.119209	-5	-1.0
2013-01-03	-0.861849	-2.104569	-0.494929	-5	-2.0
2013-01-04	-0.721555	-0.706771	-1.039575	-5	-3.0
2013-01-05	-0.424972	-0.567020	-0.276232	-5	-4.0
2013-01-06	-0.673690	-0.113648	-1.478427	-5	-5.0

缺失值

Pandas 主要用 `np.nan` 表示缺失数据。计算时，默认不包含空值。详见[缺失数据](#)。

重建索引（reindex）可以更改、添加、删除指定轴的索引，并返回数据副本，即不更改原数据。


```
In [55]: df1 = df.reindex(index=dates[0:4], columns=list(df.columns) + ['E'])
```

```
In [56]: df1.loc[dates[0]:dates[1], 'E'] = 1
```

```
In [57]: df1
```

```
Out[57]:
```

	A	B	C	D	F	E
2013-01-01	0.000000	0.000000	-1.509059	5	NaN	1.0
2013-01-02	1.212112	-0.173215	0.119209	5	1.0	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5	2.0	NaN
2013-01-04	0.721555	-0.706771	-1.039575	5	3.0	NaN

删除所有含缺失值的行：

```
In [58]: df1.dropna(how='any')
```

```
Out[58]:
```

	A	B	C	D	F	E
2013-01-02	1.212112	-0.173215	0.119209	5	1.0	1.0

填充缺失值：

```
In [59]: df1.fillna(value=5)
```

```
Out[59]:
```

	A	B	C	D	F	E
2013-01-01	0.000000	0.000000	-1.509059	5	5.0	1.0
2013-01-02	1.212112	-0.173215	0.119209	5	1.0	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5	2.0	5.0
2013-01-04	0.721555	-0.706771	-1.039575	5	3.0	5.0

提取 nan 值的布尔掩码：

```
In [60]: pd.isna(df1)
```

```
Out[60]:
```

	A	B	C	D	F	E
2013-01-01	False	False	False	False	True	False
2013-01-02	False	False	False	False	False	False
2013-01-03	False	False	False	False	False	True
2013-01-04	False	False	False	False	False	True

运算

详见[二进制操作](#)。

统计

一般情况下，运算时排除缺失值。

描述性统计：

```
In [61]: df.mean()
Out[61]:
A    -0.004474
B    -0.383981
C    -0.687758
D     5.000000
F     3.000000
dtype: float64
```

在另一个轴(即，行)上执行同样的操作：

```
In [62]: df.mean(1)
Out[62]:
2013-01-01    0.872735
2013-01-02    1.431621
2013-01-03    0.707731
2013-01-04    1.395042
2013-01-05    1.883656
2013-01-06    1.592306
Freq: D, dtype: float64
```

不同维度对象运算时，要先对齐。此外，Pandas 自动沿指定维度广播。

```
In [63]: s = pd.Series([1, 3, 5, np.nan, 6, 8], index=dates).shift(2)

In [64]: s
Out[64]:
2013-01-01    NaN
2013-01-02    NaN
2013-01-03     1.0
2013-01-04     3.0
2013-01-05     5.0
2013-01-06    NaN
Freq: D, dtype: float64

In [65]: df.sub(s, axis='index')
Out[65]:
```

	A	B	C	D	F
2013-01-01	NaN	NaN	NaN	NaN	NaN
2013-01-02	NaN	NaN	NaN	NaN	NaN
2013-01-03	-1.861849	-3.104569	-1.494929	4.0	1.0
2013-01-04	-2.278445	-3.706771	-4.039575	2.0	0.0
2013-01-05	-5.424972	-4.432980	-4.723768	0.0	-1.0
2013-01-06	NaN	NaN	NaN	NaN	NaN

Apply 函数

Apply 函数处理数据：

```
In [66]: df.apply(np.cumsum)
Out[66]:
```

	A	B	C	D	F
2013-01-01	0.000000	0.000000	-1.509059	5	NaN
2013-01-02	1.212112	-0.173215	-1.389850	10	1.0
2013-01-03	0.350263	-2.277784	-1.884779	15	3.0
2013-01-04	1.071818	-2.984555	-2.924354	20	6.0
2013-01-05	0.646846	-2.417535	-2.648122	25	10.0
2013-01-06	-0.026844	-2.303886	-4.126549	30	15.0

```
In [67]: df.apply(lambda x: x.max() - x.min())
Out[67]:
```

A	2.073961
B	2.671590
C	1.785291
D	0.000000
F	4.000000

dtype: float64

直方图

详见[直方图与离散化](#)。

```
In [68]: s = pd.Series(np.random.randint(0, 7, size=10))
```

```
In [69]: s
```

```
Out[69]:
```

```
0    4
1    2
2    1
3    2
4    6
5    4
6    4
7    6
8    4
9    4
dtype: int64
```

```
In [70]: s.value_counts()
```

```
Out[70]:
```

```
4    5
6    2
2    2
1    1
dtype: int64
```

字符串方法

Series 的 `str` 属性包含一组字符串处理功能，如下列代码所示。注意，`str` 的模式匹配默认使用[正则表达式](#)。详见[矢量字符串方法](#)。

```
In [71]: s = pd.Series(['A', 'B', 'C', 'Aaba', 'Baca', np.nan, 'CABA', 'dog', 'cat'])

In [72]: s.str.lower()
Out[72]:
0      a
1      b
2      c
3    aaba
4    baca
5     NaN
6    caba
7     dog
8     cat
dtype: object
```

合并（Merge）

结合（Concat）

Pandas 提供了多种将 Series、DataFrame 对象组合在一起的功能，用索引与关联代数功能的多种设置逻辑可执行连接（join）与合并（merge）操作。

详见[合并](#)。

`concat()` 用于连接 Pandas 对象：

```
In [73]: df = pd.DataFrame(np.random.randn(10, 4))
```

```
In [74]: df
```

```
Out[74]:
```

	0	1	2	3
0	-0.548702	1.467327	-1.015962	-0.483075
1	1.637550	-1.217659	-0.291519	-1.745505
2	-0.263952	0.991460	-0.919069	0.266046
3	-0.709661	1.669052	1.037882	-1.705775
4	-0.919854	-0.042379	1.247642	-0.009920
5	0.290213	0.495767	0.362949	1.548106
6	-1.131345	-0.089329	0.337863	-0.945867
7	-0.932132	1.956030	0.017587	-0.016692
8	-0.575247	0.254161	-1.143704	0.215897
9	1.193555	-0.077118	-0.408530	-0.862495

```
# 分解为多组
```

```
In [75]: pieces = [df[:3], df[3:7], df[7:]]
```

```
In [76]: pd.concat(pieces)
```

```
Out[76]:
```

	0	1	2	3
0	-0.548702	1.467327	-1.015962	-0.483075
1	1.637550	-1.217659	-0.291519	-1.745505
2	-0.263952	0.991460	-0.919069	0.266046
3	-0.709661	1.669052	1.037882	-1.705775
4	-0.919854	-0.042379	1.247642	-0.009920
5	0.290213	0.495767	0.362949	1.548106
6	-1.131345	-0.089329	0.337863	-0.945867
7	-0.932132	1.956030	0.017587	-0.016692
8	-0.575247	0.254161	-1.143704	0.215897
9	1.193555	-0.077118	-0.408530	-0.862495

连接 (join)

SQL 风格的合并。详见[数据库风格连接](#)。

```

In [77]: left = pd.DataFrame({'key': ['foo', 'foo'], 'lval': [1, 2]})

In [78]: right = pd.DataFrame({'key': ['foo', 'foo'], 'rval': [4, 5]})

In [79]: left
Out[79]:
   key  lval
0  foo     1
1  foo     2

In [80]: right
Out[80]:
   key  rval
0  foo     4
1  foo     5

In [81]: pd.merge(left, right, on='key')
Out[81]:
   key  lval  rval
0  foo     1     4
1  foo     1     5
2  foo     2     4
3  foo     2     5

```

这里还有一个例子：

```

In [82]: left = pd.DataFrame({'key': ['foo', 'bar'], 'lval': [1, 2]})

In [83]: right = pd.DataFrame({'key': ['foo', 'bar'], 'rval': [4, 5]})

In [84]: left
Out[84]:
   key  lval
0  foo     1
1  bar     2

In [85]: right
Out[85]:
   key  rval
0  foo     4
1  bar     5

In [86]: pd.merge(left, right, on='key')
Out[86]:
   key  lval  rval
0  foo     1     4
1  bar     2     5

```

追加 (Append)

为 DataFrame 追加行。详见[追加](#)文档。

```
In [87]: df = pd.DataFrame(np.random.randn(8, 4), columns=['A', 'B', 'C', 'D'])
```

```
In [88]: df
```

```
Out[88]:
```

	A	B	C	D
0	1.346061	1.511763	1.627081	-0.990582
1	-0.441652	1.211526	0.268520	0.024580
2	-1.577585	0.396823	-0.105381	-0.532532
3	1.453749	1.208843	-0.080952	-0.264610
4	-0.727965	-0.589346	0.339969	-0.693205
5	-0.339355	0.593616	0.884345	1.591431
6	0.141809	0.220390	0.435589	0.192451
7	-0.096701	0.803351	1.715071	-0.708758

```
In [89]: s = df.iloc[3]
```

```
In [90]: df.append(s, ignore_index=True)
```

```
Out[90]:
```

	A	B	C	D
0	1.346061	1.511763	1.627081	-0.990582
1	-0.441652	1.211526	0.268520	0.024580
2	-1.577585	0.396823	-0.105381	-0.532532
3	1.453749	1.208843	-0.080952	-0.264610
4	-0.727965	-0.589346	0.339969	-0.693205
5	-0.339355	0.593616	0.884345	1.591431
6	0.141809	0.220390	0.435589	0.192451
7	-0.096701	0.803351	1.715071	-0.708758
8	1.453749	1.208843	-0.080952	-0.264610

分组 (Grouping)

“group by” 指的是涵盖下列一项或多项步骤的处理流程：

- **分割**：按条件把数据分割成多组；
- **应用**：为每组单独应用函数；
- **组合**：将处理结果组合成一个数据结构。

详见[分组](#)。

```
In [91]: df = pd.DataFrame({'A': ['foo', 'bar', 'foo', 'bar',
....:                             'foo', 'bar', 'foo', 'foo'],
....:                      'B': ['one', 'one', 'two', 'three',
....:                             'two', 'two', 'one', 'three'],
....:                      'C': np.random.randn(8),
....:                      'D': np.random.randn(8)})
....:
```

```
In [92]: df
```

```
Out[92]:
```

	A	B	C	D
0	foo	one	-1.202872	-0.055224
1	bar	one	-1.814470	2.395985
2	foo	two	1.018601	1.552825
3	bar	three	-0.595447	0.166599
4	foo	two	1.395433	0.047609
5	bar	two	-0.392670	-0.136473
6	foo	one	0.007207	-0.561757
7	foo	three	1.928123	-1.623033

先分组，再用 `sum()` 函数计算每组的汇总数据：

```
In [93]: df.groupby('A').sum()
```

```
Out[93]:
```

	C	D
A		
bar	-2.802588	2.42611
foo	3.146492	-0.63958

多列分组后，生成多层索引，也可以应用 `sum` 函数：

```
In [94]: df.groupby(['A', 'B']).sum()
```

```
Out[94]:
```

		C	D
A	B		
bar	one	-1.814470	2.395985
	three	-0.595447	0.166599
	two	-0.392670	-0.136473
foo	one	-1.195665	-0.616981
	three	1.928123	-1.623033
	two	2.414034	1.600434

重塑 (Reshaping)

详见[多层索引与重塑](#)。

堆叠 (Stack)

```
In [95]: tuples = list(zip(*[['bar', 'bar', 'baz', 'baz',
....:                        'foo', 'foo', 'qux', 'qux'],
....:                        ['one', 'two', 'one', 'two',
....:                        'one', 'two', 'one', 'two']]))
....:

In [96]: index = pd.MultiIndex.from_tuples(tuples, names=['first', 'second'])

In [97]: df = pd.DataFrame(np.random.randn(8, 2), index=index, columns=['A', 'B'])

In [98]: df2 = df[:4]

In [99]: df2
Out[99]:
```

		A	B
bar	one	0.029399	-0.542108
	two	0.282696	-0.087302
baz	one	-1.575170	1.771208
	two	0.816482	1.100230

`stack()`方法把 DataFrame 列压缩至一层:

```
In [100]: stacked = df2.stack()

In [101]: stacked
Out[101]:
```

first	second		
	two	B	-0.542108
		A	0.282696
baz	one	B	-0.087302
		A	-1.575170
	two	B	1.771208
		A	0.816482
	two	B	1.100230
		A	0.282696

dtype: float64

压缩后的 DataFrame 或 Series 具有多层索引, `stack()` 的逆操作是 `unstack()`, 默认为拆叠最后一层:

```
In [102]: stacked.unstack()
Out[102]:
```

		A	B
first	second		
bar	one	0.029399	-0.542108
	two	0.282696	-0.087302
baz	one	-1.575170	1.771208
	two	0.816482	1.100230

```
In [103]: stacked.unstack(1)
Out[103]:
```

		one	two
second	first		
bar	A	0.029399	0.282696
	B	-0.542108	-0.087302
baz	A	-1.575170	0.816482
	B	1.771208	1.100230

```
In [104]: stacked.unstack(0)
Out[104]:
```

		bar	baz
first	second		
one	A	0.029399	-1.575170
	B	-0.542108	1.771208
two	A	0.282696	0.816482
	B	-0.087302	1.100230

数据透视表 (Pivot Tables)

详见[数据透视表](#)。

```
In [105]: df = pd.DataFrame({'A': ['one', 'one', 'two', 'three'] * 3,
.....:                      'B': ['A', 'B', 'C'] * 4,
.....:                      'C': ['foo', 'foo', 'foo', 'bar', 'bar', 'bar'] * 2,
.....:                      'D': np.random.randn(12),
.....:                      'E': np.random.randn(12)})
.....:
```

```
In [106]: df
```

```
Out[106]:
```

	A	B	C	D	E
0	one	A	foo	1.418757	-0.179666
1	one	B	foo	-1.879024	1.291836
2	two	C	foo	0.536826	-0.009614
3	three	A	bar	1.006160	0.392149
4	one	B	bar	-0.029716	0.264599
5	one	C	bar	-1.146178	-0.057409
6	two	A	foo	0.100900	-1.425638
7	three	B	foo	-1.035018	1.024098
8	one	C	foo	0.314665	-0.106062
9	one	A	bar	-0.773723	1.824375
10	two	B	bar	-1.170653	0.595974
11	three	C	bar	0.648740	1.167115

用上述数据生成数据透视表非常简单：

```
In [107]: pd.pivot_table(df, values='D', index=['A', 'B'], columns=['C'])
```

```
Out[107]:
```

		bar	foo
one	A	-0.773723	1.418757
	B	-0.029716	-1.879024
	C	-1.146178	0.314665
three	A	1.006160	NaN
	B	NaN	-1.035018
	C	0.648740	NaN
two	A	NaN	0.100900
	B	-1.170653	NaN
	C	NaN	0.536826

时间序列(TimeSeries)

Pandas 为频率转换时重采样提供了虽然简单易用，但强大高效的功能，如，将秒级的数据转换为 5 分钟为频率的数据。这种操作常见于财务应用程序，但又不仅限于此。详见[时间序列](#)。

```
In [108]: rng = pd.date_range('1/1/2012', periods=100, freq='S')

In [109]: ts = pd.Series(np.random.randint(0, 500, len(rng)), index=rng)

In [110]: ts.resample('5Min').sum()
Out[110]:
2012-01-01    25083
Freq: 5T, dtype: int64
```

时区表示：

```
In [111]: rng = pd.date_range('3/6/2012 00:00', periods=5, freq='D')

In [112]: ts = pd.Series(np.random.randn(len(rng)), rng)

In [113]: ts
Out[113]:
2012-03-06    0.464000
2012-03-07    0.227371
2012-03-08   -0.496922
2012-03-09    0.306389
2012-03-10   -2.290613
Freq: D, dtype: float64

In [114]: ts_utc = ts.tz_localize('UTC')

In [115]: ts_utc
Out[115]:
2012-03-06 00:00:00+00:00    0.464000
2012-03-07 00:00:00+00:00    0.227371
2012-03-08 00:00:00+00:00   -0.496922
2012-03-09 00:00:00+00:00    0.306389
2012-03-10 00:00:00+00:00   -2.290613
Freq: D, dtype: float64
```

转换成其它时区：

```
In [116]: ts_utc.tz_convert('US/Eastern')
Out[116]:
2012-03-05 19:00:00-05:00    0.464000
2012-03-06 19:00:00-05:00    0.227371
2012-03-07 19:00:00-05:00   -0.496922
2012-03-08 19:00:00-05:00    0.306389
2012-03-09 19:00:00-05:00   -2.290613
Freq: D, dtype: float64
```

转换时间段：

```
In [117]: rng = pd.date_range('1/1/2012', periods=5, freq='M')
```

```
In [118]: ts = pd.Series(np.random.randn(len(rng)), index=rng)
```

```
In [119]: ts
```

```
Out[119]:
```

```
2012-01-31    -1.134623
2012-02-29    -1.561819
2012-03-31    -0.260838
2012-04-30     0.281957
2012-05-31     1.523962
Freq: M, dtype: float64
```

```
In [120]: ps = ts.to_period()
```

```
In [121]: ps
```

```
Out[121]:
```

```
2012-01    -1.134623
2012-02    -1.561819
2012-03    -0.260838
2012-04     0.281957
2012-05     1.523962
Freq: M, dtype: float64
```

```
In [122]: ps.to_timestamp()
```

```
Out[122]:
```

```
2012-01-01    -1.134623
2012-02-01    -1.561819
2012-03-01    -0.260838
2012-04-01     0.281957
2012-05-01     1.523962
Freq: MS, dtype: float64
```

Pandas 函数可以很方便地转换时间段与时间戳。下例把以 11 月为结束年份的季度频率转换为下一季度月末上午 9 点：

```
In [123]: prng = pd.period_range('1990Q1', '2000Q4', freq='Q-NOV')
```

```
In [124]: ts = pd.Series(np.random.randn(len(prng)), prng)
```

```
In [125]: ts.index = (prng.asfreq('M', 'e') + 1).asfreq('H', 's') + 9
```

```
In [126]: ts.head()
```

```
Out[126]:
```

```
1990-03-01 09:00    -0.902937
1990-06-01 09:00     0.068159
1990-09-01 09:00    -0.057873
1990-12-01 09:00    -0.368204
1991-03-01 09:00    -1.144073
Freq: H, dtype: float64
```

类别型 (Categoricals)

Pandas 的 DataFrame 里可以包含类别数据。完整文档详见[类别简介](#) 和 [API 文档](#)。

```
In [127]: df = pd.DataFrame({"id": [1, 2, 3, 4, 5, 6],
.....:                      "raw_grade": ['a', 'b', 'b', 'a', 'a', 'e']})
.....:
```

将 `grade` 的原生数据转换为类别型数据：

```
In [128]: df["grade"] = df["raw_grade"].astype("category")

In [129]: df["grade"]
Out[129]:
0    a
1    b
2    b
3    a
4    a
5    e
Name: grade, dtype: category
Categories (3, object): [a, b, e]
```

用有含义的名字重命名不同类型，调用 `Series.cat.categories`。

```
In [130]: df["grade"].cat.categories = ["very good", "good", "very bad"]
```

重新排序各类别，并添加缺失类，`Series.cat` 的方法默认返回新 `Series`。

```
In [131]: df["grade"] = df["grade"].cat.set_categories(["very bad", "bad", "medium",
.....:                                                "good", "very good"])
.....:

In [132]: df["grade"]
Out[132]:
0    very good
1         good
2         good
3    very good
4    very good
5    very bad
Name: grade, dtype: category
Categories (5, object): [very bad, bad, medium, good, very good]
```

注意，这里是按生成类别时的顺序排序，不是按词汇排序：

```
In [133]: df.sort_values(by="grade")
Out[133]:
```

	id	raw_grade	grade
5	6	e	very bad
1	2	b	good
2	3	b	good
0	1	a	very good
3	4	a	very good
4	5	a	very good

按类别分组 (groupby) 时, 即便某类别为空, 也会显示:

```
In [134]: df.groupby("grade").size()
Out[134]:
```

grade	
very bad	1
bad	0
medium	0
good	2
very good	3

dtype: int64

可视化

详见[可视化文档](#)。

```
In [135]: ts = pd.Series(np.random.randn(1000),
.....:                  index=pd.date_range('1/1/2000', periods=1000))
.....:

In [136]: ts = ts.cumsum()

In [137]: ts.plot()
Out[137]: <matplotlib.axes._subplots.AxesSubplot at 0x7f2b5771ac88>
```

可视化

DataFrame 的 `plot()` 方法可以快速绘制所有带标签的列:

```

In [138]: df = pd.DataFrame(np.random.randn(1000, 4), index=ts.index,
.....:                      columns=['A', 'B', 'C', 'D'])
.....:

In [139]: df = df.cumsum()

In [140]: plt.figure()
Out[140]: <Figure size 640x480 with 0 Axes>

In [141]: df.plot()
Out[141]: <matplotlib.axes._subplots.AxesSubplot at 0x7f2b53a2d7f0>

In [142]: plt.legend(loc='best')
Out[142]: <matplotlib.legend.Legend at 0x7f2b539728d0>

```

可视化2

数据输入 / 输出

CSV

写入 CSV 文件。

```
In [143]: df.to_csv('foo.csv')
```

读取 CSV 文件数据：

```

In [144]: pd.read_csv('foo.csv')
Out[144]:
   Unnamed: 0      A      B      C      D
0  2000-01-01  0.266457 -0.399641 -0.219582  1.186860
1  2000-01-02 -1.170732 -0.345873  1.653061 -0.282953
2  2000-01-03 -1.734933  0.530468  2.060811 -0.515536
3  2000-01-04 -1.555121  1.452620  0.239859 -1.156896
4  2000-01-05  0.578117  0.511371  0.103552 -2.428202
5  2000-01-06  0.478344  0.449933 -0.741620 -1.962409
6  2000-01-07  1.235339 -0.091757 -1.543861 -1.084753
..  ...      ...      ...      ...      ...
993 2002-09-20 -10.628548 -9.153563 -7.883146  28.313940
994 2002-09-21 -10.390377 -8.727491 -6.399645  30.914107
995 2002-09-22 -8.985362 -8.485624 -4.669462  31.367740
996 2002-09-23 -9.558560 -8.781216 -4.499815  30.518439
997 2002-09-24 -9.902058 -9.340490 -4.386639  30.105593
998 2002-09-25 -10.216020 -9.480682 -3.933802  29.758560
999 2002-09-26 -11.856774 -10.671012 -3.216025  29.369368

[1000 rows x 5 columns]

```


HDF5

详见 [HDFStores](#) 文档。

写入 HDF5 Store:

```
In [145]: df.to_hdf('foo.h5', 'df')
```

读取 HDF5 Store:

```
In [146]: pd.read_hdf('foo.h5', 'df')
Out[146]:
```

	A	B	C	D
2000-01-01	0.266457	-0.399641	-0.219582	1.186860
2000-01-02	-1.170732	-0.345873	1.653061	-0.282953
2000-01-03	-1.734933	0.530468	2.060811	-0.515536
2000-01-04	-1.555121	1.452620	0.239859	-1.156896
2000-01-05	0.578117	0.511371	0.103552	-2.428202
2000-01-06	0.478344	0.449933	-0.741620	-1.962409
2000-01-07	1.235339	-0.091757	-1.543861	-1.084753
...	...	...	...	...
2002-09-20	-10.628548	-9.153563	-7.883146	28.313940
2002-09-21	-10.390377	-8.727491	-6.399645	30.914107
2002-09-22	-8.985362	-8.485624	-4.669462	31.367740
2002-09-23	-9.558560	-8.781216	-4.499815	30.518439
2002-09-24	-9.902058	-9.340490	-4.386639	30.105593
2002-09-25	-10.216020	-9.480682	-3.933802	29.758560
2002-09-26	-11.856774	-10.671012	-3.216025	29.369368

[1000 rows x 4 columns]

Excel

详见 [Excel](#) 文档。

写入 Excel 文件:

```
In [147]: df.to_excel('foo.xlsx', sheet_name='Sheet1')
```

读取 Excel 文件:

```
In [148]: pd.read_excel('foo.xlsx', 'Sheet1', index_col=None, na_values=['NA'])
Out[148]:
   Unnamed: 0      A      B      C      D
0  2000-01-01  0.266457 -0.399641 -0.219582  1.186860
1  2000-01-02 -1.170732 -0.345873  1.653061 -0.282953
2  2000-01-03 -1.734933  0.530468  2.060811 -0.515536
3  2000-01-04 -1.555121  1.452620  0.239859 -1.156896
4  2000-01-05  0.578117  0.511371  0.103552 -2.428202
5  2000-01-06  0.478344  0.449933 -0.741620 -1.962409
6  2000-01-07  1.235339 -0.091757 -1.543861 -1.084753
...
993 2002-09-20 -10.628548 -9.153563 -7.883146  28.313940
994 2002-09-21 -10.390377 -8.727491 -6.399645  30.914107
995 2002-09-22 -8.985362 -8.485624 -4.669462  31.367740
996 2002-09-23 -9.558560 -8.781216 -4.499815  30.518439
997 2002-09-24 -9.902058 -9.340490 -4.386639  30.105593
998 2002-09-25 -10.216020 -9.480682 -3.933802  29.758560
999 2002-09-26 -11.856774 -10.671012 -3.216025  29.369368

[1000 rows x 5 columns]
```

各种坑 (Gotchas)

执行某些操作，将触发异常，如：

```
>>> if pd.Series([False, True, False]):
...     print("I was true")
Traceback
...
ValueError: The truth value of an array is ambiguous. Use a.empty, a.any() or a.all().
```

参阅[比较操作](#)文档，查看错误提示与解决方案。

详见[各种坑](#)文档。

基础用法

本节介绍 Pandas 数据结构的基础用法。下列代码创建上一节用过的示例数据对象：

```
In [1]: index = pd.date_range('1/1/2000', periods=8)

In [2]: s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])

In [3]: df = pd.DataFrame(np.random.randn(8, 3), index=index,
...:                       columns=['A', 'B', 'C'])
...:
```

Head 与 Tail

`head()` 与 `tail()` 用于快速预览 Series 与 DataFrame，默认显示 5 条数据，也可以指定显示数据的数量。

```
In [4]: long_series = pd.Series(np.random.randn(1000))

In [5]: long_series.head()
Out[5]:
0    -1.157892
1    -1.344312
2     0.844885
3     1.075770
4    -0.109050
dtype: float64

In [6]: long_series.tail(3)
Out[6]:
997    -0.289388
998    -1.020544
999     0.589993
dtype: float64
```

属性与底层数据

Pandas 可以通过多个属性访问元数据：

- **shape:**
 - 输出对象的轴维度，与 ndarray 一致
- **轴标签**
 - **Series:** Index (仅有此轴)
 - **DataFrame:** Index (行) 与列

注意：为属性赋值是安全的！

```

In [7]: df[:2]
Out[7]:
           A          B          C
2000-01-01 -0.173215  0.119209 -1.044236
2000-01-02 -0.861849 -2.104569 -0.494929

In [8]: df.columns = [x.lower() for x in df.columns]

In [9]: df
Out[9]:
           a          b          c
2000-01-01 -0.173215  0.119209 -1.044236
2000-01-02 -0.861849 -2.104569 -0.494929
2000-01-03  1.071804  0.721555 -0.706771
2000-01-04 -1.039575  0.271860 -0.424972
2000-01-05  0.567020  0.276232 -1.087401
2000-01-06 -0.673690  0.113648 -1.478427
2000-01-07  0.524988  0.404705  0.577046
2000-01-08 -1.715002 -1.039268 -0.370647

```

Pandas 对象（[Index](#)，[Series](#)，[DataFrame](#)）相当于数组的容器，用于存储数据、执行计算。大部分类型的底层数组都是 `numpy.ndarray`。不过，Pandas 与第三方支持库一般都会扩展 NumPy 类型系统，添加自定义数组（见[数据类型](#)）。

`.array` 属性用于提取 [Index](#) 或 [Series](#) 里的数据。

```

In [10]: s.array
Out[10]:
<PandasArray>
[ 0.4691122999071863, -0.2828633443286633, -1.5090585031735124,
 -1.1356323710171934,  1.2121120250208506]
Length: 5, dtype: float64

In [11]: s.index.array
Out[11]:
<PandasArray>
['a', 'b', 'c', 'd', 'e']
Length: 5, dtype: object

```

`array` 一般指 [ExtensionArray](#)。至于什么是 [ExtensionArray](#) 及 Pandas 为什么要用 [ExtensionArray](#) 不是本节要说明的内容。更多信息请参阅[数据类型](#)。

提取 NumPy 数组，用 `to_numpy()` 或 `numpy.asarray()`。

```

In [12]: s.to_numpy()
Out[12]: array([ 0.4691, -0.2829, -1.5091, -1.1356,  1.2121])

In [13]: np.asarray(s)
Out[13]: array([ 0.4691, -0.2829, -1.5091, -1.1356,  1.2121])

```

`Series` 与 `Index` 的类型是 [ExtensionArray](#) 时，`to_numpy()` 会复制数据，并强制转换值。详情见[数据类型](#)。

`to_numpy()` 可以控制 `numpy.ndarray` 生成的数据类型。以带时区的 `datetime` 为例，NumPy 未提供时区信息的 `datetime` 数据类型，Pandas 则提供了两种表现形式：

1. 一种是带 `Timestamp` 的 `numpy.ndarray`，提供了正确的 `tz` 信息。
2. 另一种是 `datetime64[ns]`，这也是一种 `numpy.ndarray`，值被转换为 UTC，但去掉了时区信息。

时区信息可以用 `dtype=object` 保存。

```
In [14]: ser = pd.Series(pd.date_range('2000', periods=2, tz="CET"))

In [15]: ser.to_numpy(dtype=object)
Out[15]:
array([Timestamp('2000-01-01 00:00:00+0100', tz='CET', freq='D'),
       Timestamp('2000-01-02 00:00:00+0100', tz='CET', freq='D')],
      dtype=object)
```

或用 `dtype='datetime64[ns]'` 去除。

```
In [16]: ser.to_numpy(dtype="datetime64[ns]")
Out[16]:
array(['1999-12-31T23:00:00.000000000', '2000-01-01T23:00:00.000000000'],
      dtype='datetime64[ns]')
```

提取 `DataFrame` 里的原数据稍微有点复杂。`DataFrame` 里所有列的数据类型都一样时，`DataFrame.to_numpy()` 返回底层数据：

```
In [17]: df.to_numpy()
Out[17]:
array([[ -0.1732,  0.1192, -1.0442],
       [ -0.8618, -2.1046, -0.4949],
       [  1.0718,  0.7216, -0.7068],
       [ -1.0396,  0.2719, -0.425 ],
       [  0.567 ,  0.2762, -1.0874],
       [ -0.6737,  0.1136, -1.4784],
       [  0.525 ,  0.4047,  0.577 ],
       [ -1.715 , -1.0393, -0.3706]])
```

`DataFrame` 为同构型数据时，Pandas 直接修改原始 `ndarray`，所做修改会直接反应在数据结构里。对于异质型数据，即 `DataFrame` 列的数据类型不一样时，就不是这种操作模式了。与轴标签不同，不能为值的属性赋值。

∴ tip 注意

处理异质型数据时，输出结果 `ndarray` 的数据类型适用于涉及的各类数据。若 `DataFrame` 里包含字符串，输出结果的数据类型就是 `object`。要是只有浮点数或整数，则输出结果的数据类型是浮点数。

∴

以前，Pandas 推荐用 `Series.values` 或 `DataFrame.values` 从 Series 或 DataFrame 里提取数据。旧有代码库或在线教程里仍在用这种操作，但 Pandas 已改进了此功能，现在，推荐用 `.array` 或 `to_numpy` 提取数据，别再用 `.values` 了。`.values` 有以下几个缺点：

1. Series 含扩展类型时，`Series.values` 无法判断到底是该返回 NumPy array，还是返回 `ExtensionArray`。而 `Series.array` 则只返回 `ExtensionArray`，且不会复制数据。`Series.to_numpy()` 则返回 NumPy 数组，其代价是需要复制、并强制转换数据的值。
2. DataFrame 含多种数据类型时，`DataFrame.values` 会复制数据，并将数据的值强制转换同一种数据类型，这是一种代价较高的操作。`DataFrame.to_numpy()` 则返回 NumPy 数组，这种方式更清晰，也不会把 DataFrame 里的数据都当作一种类型。

加速操作

借助 `numexpr` 与 `bottleneck` 支持库，Pandas 可以加速特定类型的二进制数值与布尔操作。

处理大型数据集时，这两个支持库特别有用，加速效果也非常明显。`numexpr` 使用智能分块、缓存与多核技术。`bottleneck` 是一组专属 cython 例程，处理含 `nans` 值的数组时，特别快。

请看下面这个例子（DataFrame 包含 100 列 X 10 万行数据）：

操作	0.11.0版 (ms)	旧版 (ms)	提升比率
<code>df1 > df2</code>	13.32	125.35	0.1063
<code>df1 * df2</code>	21.71	36.63	0.5928
<code>df1 + df2</code>	22.04	36.50	0.6039

强烈建议安装这两个支持库，更多信息，请参阅[推荐支持库](#)。

这两个支持库默认为启用状态，可用以下选项设置：

0.20.0 版新增。

```
pd.set_option('compute.use_bottleneck', False)
pd.set_option('compute.use_numexpr', False)
```

二进制操作

Pandas 数据结构之间执行二进制操作，要注意下列两个关键点：

- 多维（DataFrame）与低维（Series）对象之间的广播机制；
- 计算中的缺失值处理。

这两个问题可以同时处理，但下面先介绍怎么分开处理。

匹配/广播机制

DataFrame 支持 `add()`、`sub()`、`mul()`、`div()` 及 `radd()`、`rsub()` 等方法执行二进制操作。广播机制重点关注输入的 Series。通过 `axis` 关键字，匹配 `index` 或 `columns` 即可调用这些函数。

```

In [18]: df = pd.DataFrame({
.....:     'one': pd.Series(np.random.randn(3), index=['a', 'b', 'c']),
.....:     'two': pd.Series(np.random.randn(4), index=['a', 'b', 'c', 'd']),
.....:     'three': pd.Series(np.random.randn(3), index=['b', 'c', 'd'])})
.....:

In [19]: df
Out[19]:
      one      two      three
a  1.394981  1.772517      NaN
b  0.343054  1.912123 -0.050390
c  0.695246  1.478369  1.227435
d      NaN  0.279344 -0.613172

In [20]: row = df.iloc[1]

In [21]: column = df['two']

In [22]: df.sub(row, axis='columns')
Out[22]:
      one      two      three
a  1.051928 -0.139606      NaN
b  0.000000  0.000000  0.000000
c  0.352192 -0.433754  1.277825
d      NaN -1.632779 -0.562782

In [23]: df.sub(row, axis=1)
Out[23]:
      one      two      three
a  1.051928 -0.139606      NaN
b  0.000000  0.000000  0.000000
c  0.352192 -0.433754  1.277825
d      NaN -1.632779 -0.562782

In [24]: df.sub(column, axis='index')
Out[24]:
      one  two      three
a -0.377535  0.0      NaN
b -1.569069  0.0 -1.962513
c -0.783123  0.0 -0.250933
d      NaN  0.0 -0.892516

In [25]: df.sub(column, axis=0)
Out[25]:
      one  two      three
a -0.377535  0.0      NaN
b -1.569069  0.0 -1.962513
c -0.783123  0.0 -0.250933
d      NaN  0.0 -0.892516

```

还可以用 Series 对齐多层索引 DataFrame 的某一层级。


```

In [26]: dfmi = df.copy()

In [27]: dfmi.index = pd.MultiIndex.from_tuples([(1, 'a'), (1, 'b'),
....:                                          (1, 'c'), (2, 'a')],
....:                                          names=['first', 'second'])
....:

In [28]: dfmi.sub(column, axis=0, level='second')
Out[28]:

```

		one	two	three
first	second			
1	a	-0.377535	0.000000	NaN
	b	-1.569069	0.000000	-1.962513
	c	-0.783123	0.000000	-0.250933
2	a	NaN	-1.493173	-2.385688

Series 与 Index 还支持 `divmod()` 内置函数，该函数同时执行向下取整除与模运算，返回两个与左侧类型相同的元组。示例如下：

```
In [29]: s = pd.Series(np.arange(10))
```

```
In [30]: s
```

```
Out[30]:
```

```
0    0
1    1
2    2
3    3
4    4
5    5
6    6
7    7
8    8
9    9
```

```
dtype: int64
```

```
In [31]: div, rem = divmod(s, 3)
```

```
In [32]: div
```

```
Out[32]:
```

```
0    0
1    0
2    0
3    1
4    1
5    1
6    2
7    2
8    2
9    3
```

```
dtype: int64
```

```
In [33]: rem
```

```
Out[33]:
```

```
0    0
1    1
2    2
3    0
4    1
5    2
6    0
7    1
8    2
9    0
```

```
dtype: int64
```

```
In [34]: idx = pd.Index(np.arange(10))
```

```
In [35]: idx
```

```
Out[35]: Int64Index([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], dtype='int64')
```

```
In [36]: div, rem = divmod(idx, 3)
```

```
In [37]: div
```

```
Out[37]: Int64Index([0, 0, 0, 1, 1, 1, 2, 2, 2, 3], dtype='int64')
```

```
In [38]: rem
Out[38]: Int64Index([0, 1, 2, 0, 1, 2, 0, 1, 2, 0], dtype='int64')
```

`divmod()` 还支持元素级运算：

```
In [39]: div, rem = divmod(s, [2, 2, 3, 3, 4, 4, 5, 5, 6, 6])
```

```
In [40]: div
```

```
Out[40]:
```

```
0    0
1     0
2     0
3     1
4     1
5     1
6     1
7     1
8     1
9     1
```

```
dtype: int64
```

```
In [41]: rem
```

```
Out[41]:
```

```
0    0
1    1
2    2
3    0
4    0
5    1
6    1
7    2
8    2
9    3
```

```
dtype: int64
```

缺失值与填充缺失值操作

Series 与 DataFrame 的算数函数支持 `fill_value` 选项，即用指定值替换某个位置的缺失值。比如，两个 DataFrame 相加，除非两个 DataFrame 里同一个位置都有缺失值，其相加的和仍为 `NaN`，如果只有一个 DataFrame 里存在缺失值，则可以用 `fill_value` 指定一个值来替代 `NaN`，当然，也可以用 `fillna` 把 `NaN` 替换为想要的值。

⋮ tip 注意

下面第 43 条代码里，Pandas 官档没有写 `df2` 是哪来的，这里补上，与 `df` 类似。 ``python df2 = pd.DataFrame({: 'one': pd.Series(np.random.randn(3), index=['a', 'b', 'c']),: 'two': pd.Series(np.random.randn(4), index=['a', 'b', 'c', 'd']),: 'three': pd.Series(np.random.randn(4), index=['a', 'b', 'c', 'd'])}):

```

:::

```python
In [42]: df
Out[42]:
 one two three
a 1.394981 1.772517 NaN
b 0.343054 1.912123 -0.050390
c 0.695246 1.478369 1.227435
d NaN 0.279344 -0.613172

In [43]: df2
Out[43]:
 one two three
a 1.394981 1.772517 1.000000
b 0.343054 1.912123 -0.050390
c 0.695246 1.478369 1.227435
d NaN 0.279344 -0.613172

In [44]: df + df2
Out[44]:
 one two three
a 2.789963 3.545034 NaN
b 0.686107 3.824246 -0.100780
c 1.390491 2.956737 2.454870
d NaN 0.558688 -1.226343

In [45]: df.add(df2, fill_value=0)
Out[45]:
 one two three
a 2.789963 3.545034 1.000000
b 0.686107 3.824246 -0.100780
c 1.390491 2.956737 2.454870
d NaN 0.558688 -1.226343

```

## 比较操作

与上一小节的算数运算类似，Series 与 DataFrame 还支持 `eq`、`ne`、`lt`、`gt`、`le`、`ge` 等二进制比较操作的方法：

序号	缩写	英文	中文
1	eq	equal to	等于
2	ne	not equal to	不等于
3	lt	less than	小于
4	gt	greater than	大于
5	le	less than or equal to	小于等于
6	ge	greater than or equal to	大于等于

```
In [46]: df.gt(df2)
Out[46]:
 one two three
a False False False
b False False False
c False False False
d False False False
```

```
In [47]: df2.ne(df)
Out[47]:
 one two three
a False False True
b False False False
c False False False
d True False False
```

这些操作生成一个与左侧输入对象类型相同的 Pandas 对象，即，dtype 为 `bool`。`boolean` 对象可用于索引操作，参阅[布尔索引](#)。

## 布尔简化

`empty`、`any()`、`all()`、`bool()` 可以把数据汇总简化至单个布尔值。

```
In [48]: (df > 0).all()
Out[48]:
one False
two True
three False
dtype: bool
```

```
In [49]: (df > 0).any()
Out[49]:
one True
two True
three True
dtype: bool
```

还可以进一步把上面的结果简化为单个布尔值。

```
In [50]: (df > 0).any().any()
Out[50]: True
```

通过 `empty` 属性，可以验证 Pandas 对象是否为空。

```
In [51]: df.empty
Out[51]: False

In [52]: pd.DataFrame(columns=list('ABC')).empty
Out[52]: True
```

用 `bool()` 方法验证单元素 pandas 对象的布尔值。

```
In [53]: pd.Series([True]).bool()
Out[53]: True

In [54]: pd.Series([False]).bool()
Out[54]: False

In [55]: pd.DataFrame([[True]]).bool()
Out[55]: True

In [56]: pd.DataFrame([[False]]).bool()
Out[56]: False
```

::: danger 警告

以下代码：

```
>>> if df:
... pass
```

或

```
>>> df and df2
```

上述代码试图比对多个值，因此，这两种操作都会触发错误：

```
ValueError: The truth value of an array is ambiguous. Use a.empty, a.any() or a.all().
```

:::

了解详情，请参阅[各种坑](#)小节的内容。

## 比较对象是否等效

一般情况下，多种方式都能得出相同的结果。以 `df + df` 与 `df * 2` 为例。应用上一小节学到的知识，测试这两种计算方式的结果是否一致，一般人都会用 `(df + df == df * 2).all()`，不过，这个表达式的结果是 `False`：

```

In [57]: df + df == df * 2
Out[57]:
 one two three
a True True False
b True True True
c True True True
d False True True

In [58]: (df + df == df * 2).all()
Out[58]:
one False
two True
three False
dtype: bool

```

注意：布尔型 DataFrame `df + df == df * 2` 中有 `False` 值！这是因为两个 `NaN` 值的比较结果为**不等**：

```

In [59]: np.nan == np.nan
Out[59]: False

```

为了验证数据是否等效，Series 与 DataFrame 等 N 维框架提供了 `equals()` 方法，用这个方法验证 `NaN` 值的结果为**相等**。

```

In [60]: (df + df).equals(df * 2)
Out[60]: True

```

注意：Series 与 DataFrame 索引的顺序必须一致，验证结果才能为 `True`：

```

In [61]: df1 = pd.DataFrame({'col': ['foo', 0, np.nan]})

In [62]: df2 = pd.DataFrame({'col': [np.nan, 0, 'foo']}, index=[2, 1, 0])

In [63]: df1.equals(df2)
Out[63]: False

In [64]: df1.equals(df2.sort_index())
Out[64]: True

```

## 比较 array 型对象

用标量值与 Pandas 数据结构对比数据元素非常简单：

```
In [65]: pd.Series(['foo', 'bar', 'baz']) == 'foo'
Out[65]:
0 True
1 False
2 False
dtype: bool

In [66]: pd.Index(['foo', 'bar', 'baz']) == 'foo'
Out[66]: array([True, False, False])
```

Pandas 还能对比两个等长 `array` 对象里的数据元素：

```
In [67]: pd.Series(['foo', 'bar', 'baz']) == pd.Index(['foo', 'bar', 'qux'])
Out[67]:
0 True
1 True
2 False
dtype: bool

In [68]: pd.Series(['foo', 'bar', 'baz']) == np.array(['foo', 'bar', 'qux'])
Out[68]:
0 True
1 True
2 False
dtype: bool
```

对比不等长的 `Index` 或 `Series` 对象会触发 `ValueError`：

```
In [55]: pd.Series(['foo', 'bar', 'baz']) == pd.Series(['foo', 'bar'])
ValueError: Series lengths must match to compare

In [56]: pd.Series(['foo', 'bar', 'baz']) == pd.Series(['foo'])
ValueError: Series lengths must match to compare
```

注意：这里的操作与 NumPy 的广播机制不同：

```
In [69]: np.array([1, 2, 3]) == np.array([2])
Out[69]: array([False, True, False])
```

NumPy 无法执行广播操作时，返回 `False`：

```
In [70]: np.array([1, 2, 3]) == np.array([1, 2])
Out[70]: False
```

## 合并重叠数据集

有时，要合并两个相似的数据集，两个数据集里的其中一个的数据比另一个多。比如，展示特定经济指标的两个数据序列，其中一个“高质量”指标，另一个是“低质量”指标。一般来说，低质量序列可能包含更多的历史数据，或覆盖更广的数据。因此，要合并这两个 `DataFrame` 对象，其中一个 `DataFrame` 中



的缺失值将按指定条件用另一个 DataFrame 里类似标签中的数据进行填充。要实现这一操作，请用下列代码中的 `combine_first()` 函数。

```
In [71]: df1 = pd.DataFrame({'A': [1., np.nan, 3., 5., np.nan],
....: 'B': [np.nan, 2., 3., np.nan, 6.]})
....:

In [72]: df2 = pd.DataFrame({'A': [5., 2., 4., np.nan, 3., 7.],
....: 'B': [np.nan, np.nan, 3., 4., 6., 8.]})
....:

In [73]: df1
Out[73]:
 A B
0 1.0 NaN
1 NaN 2.0
2 3.0 3.0
3 5.0 NaN
4 NaN 6.0

In [74]: df2
Out[74]:
 A B
0 5.0 NaN
1 2.0 NaN
2 4.0 3.0
3 NaN 4.0
4 3.0 6.0
5 7.0 8.0

In [75]: df1.combine_first(df2)
Out[75]:
 A B
0 1.0 NaN
1 2.0 2.0
2 3.0 3.0
3 5.0 4.0
4 3.0 6.0
5 7.0 8.0
```

## DataFrame 通用合并方法

上述 `combine_first()` 方法调用了更普适的 `DataFrame.combine()` 方法。该方法提取另一个 DataFrame 及合并器函数，并将之与输入的 DataFrame 对齐，再传递与 Series 配对的合并器函数（比如，名称相同的列）。

下面的代码复现了上述的 `combine_first()` 函数：

```
In [76]: def combiner(x, y):
....: return np.where(pd.isna(x), y, x)
....:
```

## 描述性统计

`Series` 与 `DataFrame` 支持大量计算描述性统计的方法与操作。这些方法大部分都是 `sum()`、`mean()`、`quantile()` 等聚合函数，其输出结果比原始数据集小；此外，还有输出结果与原始数据集同样大小的 `cumsum()`、`cumprod()` 等函数。这些方法都基本上都接受 `axis` 参数，如，`ndarray.{sum,std,...}`，但这里的 `axis` 可以用名称或整数指定：

- **Series**: 无需 `axis` 参数
- **DataFrame**:
  - `index`，即 `axis=0`，默认值
  - `columns`，即 `axis=1`

示例如下：

```
In [77]: df
Out[77]:
```

	one	two	three
a	1.394981	1.772517	NaN
b	0.343054	1.912123	-0.050390
c	0.695246	1.478369	1.227435
d	NaN	0.279344	-0.613172

```
In [78]: df.mean(0)
Out[78]:
```

one	0.811094
two	1.360588
three	0.187958

```
dtype: float64

In [79]: df.mean(1)
Out[79]:
```

a	1.583749
b	0.734929
c	1.133683
d	-0.166914

```
dtype: float64
```

上述方法都支持 `skipna` 关键字，指定是否要排除缺失数据，默认值为 `True`。

```
In [80]: df.sum(0, skipna=False)
Out[80]:
one NaN
two 5.442353
three NaN
dtype: float64

In [81]: df.sum(axis=1, skipna=True)
Out[81]:
a 3.167498
b 2.204786
c 3.401050
d -0.333828
dtype: float64
```

结合广播机制或算数操作，可以描述不同统计过程，比如标准化，即渲染数据零均值与标准差 1，这种操作非常简单：

```
In [82]: ts_stand = (df - df.mean()) / df.std()

In [83]: ts_stand.std()
Out[83]:
one 1.0
two 1.0
three 1.0
dtype: float64

In [84]: xs_stand = df.sub(df.mean(1), axis=0).div(df.std(1), axis=0)

In [85]: xs_stand.std(1)
Out[85]:
a 1.0
b 1.0
c 1.0
d 1.0
dtype: float64
```

注：`cumsum()` 与 `cumprod()` 等方法保留 NaN 值的位置。这与 `expanding()` 和 `rolling()` 略显不同，详情请参阅[本文](#)。

```
In [86]: df.cumsum()
Out[86]:
 one two three
a 1.394981 1.772517 NaN
b 1.738035 3.684640 -0.050390
c 2.433281 5.163008 1.177045
d NaN 5.442353 0.563873
```

下表为常用函数汇总表。每个函数都支持 `level` 参数，仅在数据对象为[结构化 Index](#)时使用。

函数	描述
<code>count</code>	统计非空值数量
<code>sum</code>	汇总值
<code>mean</code>	平均值
<code>mad</code>	平均绝对偏差
<code>median</code>	算数中位数
<code>min</code>	最小值
<code>max</code>	最大值
<code>mode</code>	众数
<code>abs</code>	绝对值
<code>prod</code>	乘积
<code>std</code>	贝塞尔校正的样本标准偏差
<code>var</code>	无偏方差
<code>sem</code>	平均值的标准误差
<code>skew</code>	样本偏度 (第三阶)
<code>kurt</code>	样本峰度 (第四阶)
<code>quantile</code>	样本分位数 (不同 % 的值)
<code>cumsum</code>	累加
<code>cumprod</code>	累乘
<code>cummax</code>	累积最大值
<code>cummin</code>	累积最小值

注意：NumPy 的 `mean`、`std`、`sum` 等方法默认不统计 Series 里的空值。

```
In [87]: np.mean(df['one'])
Out[87]: 0.8110935116651192

In [88]: np.mean(df['one'].to_numpy())
Out[88]: nan
```

`Series.nunique()` 返回 Series 里所有非空值的唯一值。

```
In [89]: series = pd.Series(np.random.randn(500))

In [90]: series[20:500] = np.nan

In [91]: series[10:20] = 5

In [92]: series.nunique()
Out[92]: 11
```

## 数据总结：describe

`describe()` 函数计算 Series 与 DataFrame 数据列的各种数据统计量，注意，这里排除了空值。

```
In [93]: series = pd.Series(np.random.randn(1000))

In [94]: series[::2] = np.nan

In [95]: series.describe()
Out[95]:
count 500.000000
mean -0.021292
std 1.015906
min -2.683763
25% -0.699070
50% -0.069718
75% 0.714483
max 3.160915
dtype: float64

In [96]: frame = pd.DataFrame(np.random.randn(1000, 5),
....: columns=['a', 'b', 'c', 'd', 'e'])
....:

In [97]: frame.iloc[::2] = np.nan

In [98]: frame.describe()
Out[98]:
```

	a	b	c	d	e
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	0.033387	0.030045	-0.043719	-0.051686	0.005979
std	1.017152	0.978743	1.025270	1.015988	1.006695
min	-3.000951	-2.637901	-3.303099	-3.159200	-3.188821
25%	-0.647623	-0.576449	-0.712369	-0.691338	-0.691115
50%	0.047578	-0.021499	-0.023888	-0.032652	-0.025363
75%	0.729907	0.775880	0.618896	0.670047	0.649748
max	2.740139	2.752332	3.004229	2.728702	3.240991

此外，还可以指定输出结果包含的分位数：

```
In [99]: series.describe(percentiles=[.05, .25, .75, .95])
Out[99]:
count 500.000000
mean -0.021292
std 1.015906
min -2.683763
5% -1.645423
25% -0.699070
50% -0.069718
75% 0.714483
95% 1.711409
max 3.160915
dtype: float64
```

一般情况下，默认值包含**中位数**。

对于非数值型 Series 对象，`describe()` 返回值的总数、唯一值数量、出现次数最多的值及出现的次数。

```
In [100]: s = pd.Series(['a', 'a', 'b', 'b', 'a', 'a', np.nan, 'c', 'd', 'a'])

In [101]: s.describe()
Out[101]:
count 9
unique 4
top a
freq 5
dtype: object
```

注意：对于混合型的 DataFrame 对象，`describe()` 只返回数值列的汇总统计量，如果没有数值列，则只显示类别型的列。

```
In [102]: frame = pd.DataFrame({'a': ['Yes', 'Yes', 'No', 'No'], 'b': range(4)})

In [103]: frame.describe()
Out[103]:
```

	b
count	4.000000
mean	1.500000
std	1.290994
min	0.000000
25%	0.750000
50%	1.500000
75%	2.250000
max	3.000000

`include/exclude` 参数的值为列表，用该参数可以控制包含或排除的数据类型。这里还有一个特殊值，`all`：

```
In [104]: frame.describe(include=['object'])
```

```
Out[104]:
```

	a
count	4
unique	2
top	Yes
freq	2

```
In [105]: frame.describe(include=['number'])
```

```
Out[105]:
```

	b
count	4.000000
mean	1.500000
std	1.290994
min	0.000000
25%	0.750000
50%	1.500000
75%	2.250000
max	3.000000

```
In [106]: frame.describe(include='all')
```

```
Out[106]:
```

	a	b
count	4	4.000000
unique	2	NaN
top	Yes	NaN
freq	2	NaN
mean	NaN	1.500000
std	NaN	1.290994
min	NaN	0.000000
25%	NaN	0.750000
50%	NaN	1.500000
75%	NaN	2.250000
max	NaN	3.000000

本功能依托于 [select\\_dtypes](#)，要了解该参数接受哪些输入内容请参阅[本文](#)。

## 最大值与最小值对应的索引

Series 与 DataFrame 的 `idxmax()` 与 `idxmin()` 函数计算最大值与最小值对应的索引。

```

In [107]: s1 = pd.Series(np.random.randn(5))

In [108]: s1
Out[108]:
0 1.118076
1 -0.352051
2 -1.242883
3 -1.277155
4 -0.641184
dtype: float64

In [109]: s1.idxmin(), s1.idxmax()
Out[109]: (3, 0)

In [110]: df1 = pd.DataFrame(np.random.randn(5, 3), columns=['A', 'B', 'C'])

In [111]: df1
Out[111]:
 A B C
0 -0.327863 -0.946180 -0.137570
1 -0.186235 -0.257213 -0.486567
2 -0.507027 -0.871259 -0.111110
3 2.000339 -2.430505 0.089759
4 -0.321434 -0.033695 0.096271

In [112]: df1.idxmin(axis=0)
Out[112]:
A 2
B 3
C 1
dtype: int64

In [113]: df1.idxmax(axis=1)
Out[113]:
0 C
1 A
2 C
3 A
4 C
dtype: object

```

多行或多列中存在多个最大值或最小值时，`idxmax()` 与 `idxmin()` 只返回匹配到的第一个值的 Index：



```
In [114]: df3 = pd.DataFrame([2, 1, 1, 3, np.nan], columns=['A'], index=list('edcba'))

In [115]: df3
Out[115]:
 A
e 2.0
d 1.0
c 1.0
b 3.0
a NaN

In [116]: df3['A'].idxmin()
Out[116]: 'd'
```

::: tip 注意

`idxmin` 与 `idxmax` 对应 NumPy 里的 `argmin` 与 `argmax`。

:::

## 值计数（直方图）与众数

Series 的 `value_counts()` 方法及顶级函数计算一维数组中数据值的直方图，还可以用作常规数组的函数：

```

In [117]: data = np.random.randint(0, 7, size=50)

In [118]: data
Out[118]:
array([6, 6, 2, 3, 5, 3, 2, 5, 4, 5, 4, 3, 4, 5, 0, 2, 0, 4, 2, 0, 3, 2,
 2, 5, 6, 5, 3, 4, 6, 4, 3, 5, 6, 4, 3, 6, 2, 6, 6, 2, 3, 4, 2, 1,
 6, 2, 6, 1, 5, 4])

In [119]: s = pd.Series(data)

In [120]: s.value_counts()
Out[120]:
6 10
2 10
4 9
5 8
3 8
0 3
1 2
dtype: int64

In [121]: pd.value_counts(data)
Out[121]:
6 10
2 10
4 9
5 8
3 8
0 3
1 2
dtype: int64

```

与上述操作类似，还可以统计 Series 或 DataFrame 的众数，即出现频率最高的值：

```

In [122]: s5 = pd.Series([1, 1, 3, 3, 3, 5, 5, 7, 7, 7])

In [123]: s5.mode()
Out[123]:
0 3
1 7
dtype: int64

In [124]: df5 = pd.DataFrame({"A": np.random.randint(0, 7, size=50),
.....: "B": np.random.randint(-10, 15, size=50)})
.....:

In [125]: df5.mode()
Out[125]:
 A B
0 1.0 -9
1 NaN 10
2 NaN 13

```

## 离散化与分位数

`cut()` 函数（以值为依据实现分箱）及 `qcut()` 函数（以样本分位数为依据实现分箱）用于连续值的离散化：

```
In [126]: arr = np.random.randn(20)

In [127]: factor = pd.cut(arr, 4)

In [128]: factor
Out[128]:
[(-0.251, 0.464], (-0.968, -0.251], (0.464, 1.179], (-0.251, 0.464], (-0.968, -0.251], ..., (-0.251, 0.464], (-0.968, -0.251], (-0.968, -0.251], (-0.968, -0.251], (-0.968, -0.251]]
Length: 20
Categories (4, interval[float64]): [(-0.968, -0.251] < (-0.251, 0.464] < (0.464, 1.179] < (1.179, 1.893]]

In [129]: factor = pd.cut(arr, [-5, -1, 0, 1, 5])

In [130]: factor
Out[130]:
[(0, 1], (-1, 0], (0, 1], (0, 1], (-1, 0], ..., (-1, 0], (-1, 0], (-1, 0], (-1, 0], (-1, 0], (-1, 0]]
Length: 20
Categories (4, interval[int64]): [(-5, -1] < (-1, 0] < (0, 1] < (1, 5]]
```

`qcut()` 计算样本分位数。比如，下列代码按等距分位数分割正态分布的数据：

```
In [131]: arr = np.random.randn(30)

In [132]: factor = pd.qcut(arr, [0, .25, .5, .75, 1])

In [133]: factor
Out[133]:
[(0.569, 1.184], (-2.278, -0.301], (-2.278, -0.301], (0.569, 1.184], (0.569, 1.184], ..., (-0.301, 0.569], (1.184, 2.346], (1.184, 2.346], (-0.301, 0.569], (-2.278, -0.301]]
Length: 30
Categories (4, interval[float64]): [(-2.278, -0.301] < (-0.301, 0.569] < (0.569, 1.184] < (1.184, 2.346]]

In [134]: pd.value_counts(factor)
Out[134]:
(1.184, 2.346] 8
(-2.278, -0.301] 8
(0.569, 1.184] 7
(-0.301, 0.569] 7
dtype: int64
```

定义分箱时，还可以传递无穷值：

```
In [135]: arr = np.random.randn(20)

In [136]: factor = pd.cut(arr, [-np.inf, 0, np.inf])

In [137]: factor
Out[137]:
[(-inf, 0.0], (0.0, inf], (0.0, inf], (-inf, 0.0], (-inf, 0.0], ..., (-inf, 0.0], (-inf, 0.0], (-inf,
0.0], (0.0, inf], (0.0, inf]]
Length: 20
Categories (2, interval[float64]): [(-inf, 0.0] < (0.0, inf]]
```

## 函数应用

不管是为 Pandas 对象应用自定义函数，还是应用第三方函数，都离不开以下三种方法。用哪种方法取决于操作的对象是 `DataFrame`，还是 `Series`；是行、列，还是元素。

1. 表级函数应用：`pipe()`
2. 行列级函数应用：`apply()`
3. 聚合 API：`agg()` 与 `transform()`
4. 元素级函数应用：`applymap()`

### 表级函数应用

虽然可以把 `DataFrame` 与 `Series` 传递给函数，不过链式调用函数时，最好使用 `pipe()` 方法。对比以下两种方式：

```
f、g、h 是提取、返回 `DataFrames` 的函数
>>> f(g(h(df), arg1=1), arg2=2, arg3=3)
```

下列代码与上述代码等效：

```
>>> (df.pipe(h)
... .pipe(g, arg1=1)
... .pipe(f, arg2=2, arg3=3))
```

Pandas 鼓励使用第二种方式，即链式方法。在链式方法中调用自定义函数或第三方支持库函数时，用 `pipe` 更容易，与用 Pandas 自身方法一样。

上例中，`f`、`g` 与 `h` 这几个函数都把 `DataFrame` 当作首位参数。要是想把数据作为第二个参数，该怎么办？本例中，`pipe` 为元组（`callable, data_keyword`）形式。`.pipe` 把 `DataFrame` 作为元组里指定的参数。

下例用 `statsmodels` 拟合回归。该 API 先接收一个公式，`DataFrame` 是第二个参数，`data`。要传递函数，则要用 `pipe` 接收关键词对（`sm.ols, 'data'`）。

```
In [138]: import statsmodels.formula.api as sm

In [139]: bb = pd.read_csv('data/baseball.csv', index_col='id')

In [140]: (bb.query('h > 0')
.....: .assign(ln_h=lambda df: np.log(df.h))
.....: .pipe((sm.ols, 'data'), 'hr ~ ln_h + year + g + C(lg)')
.....: .fit()
.....: .summary()
.....:)
.....:
Out[140]:
<class 'statsmodels.iolib.summary.Summary'>
"""

 OLS Regression Results
=====
Dep. Variable: hr R-squared: 0.685
Model: OLS Adj. R-squared: 0.665
Method: Least Squares F-statistic: 34.28
Date: Thu, 22 Aug 2019 Prob (F-statistic): 3.48e-15
Time: 15:48:59 Log-Likelihood: -205.92
No. Observations: 68 AIC: 421.8
Df Residuals: 63 BIC: 432.9
Df Model: 4
Covariance Type: nonrobust
=====
 coef std err t P>|t| [0.025 0.975]

Intercept -8484.7720 4664.146 -1.819 0.074 -1.78e+04 835.780
C(lg)[T.NL] -2.2736 1.325 -1.716 0.091 -4.922 0.375
ln_h -1.3542 0.875 -1.547 0.127 -3.103 0.395
year 4.2277 2.324 1.819 0.074 -0.417 8.872
g 0.1841 0.029 6.258 0.000 0.125 0.243
=====
Omnibus: 10.875 Durbin-Watson: 1.999
Prob(Omnibus): 0.004 Jarque-Bera (JB): 17.298
Skew: 0.537 Prob(JB): 0.000175
Kurtosis: 5.225 Cond. No. 1.49e+07
=====

Warnings:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.49e+07. This might indicate that there are
strong multicollinearity or other numerical problems.
"""
```

unix 的 `pipe` 与后来出现的 `dplyr` 及 `magrittr` 启发了 `pipe` 方法，在此，引入了 R 语言里用于读取 pipe 的操作符 (`%>%`)。 `pipe` 的实现思路非常清晰，仿佛 Python 源生的一样。强烈建议大家阅读 `pipe()` 的源代码。

## 行列级函数应用

`apply()` 方法沿着 DataFrame 的轴应用函数，比如，描述性统计方法，该方法支持 `axis` 参数。

```

In [141]: df.apply(np.mean)
Out[141]:
one 0.811094
two 1.360588
three 0.187958
dtype: float64

In [142]: df.apply(np.mean, axis=1)
Out[142]:
a 1.583749
b 0.734929
c 1.133683
d -0.166914
dtype: float64

In [143]: df.apply(lambda x: x.max() - x.min())
Out[143]:
one 1.051928
two 1.632779
three 1.840607
dtype: float64

In [144]: df.apply(np.cumsum)
Out[144]:
 one two three
a 1.394981 1.772517 NaN
b 1.738035 3.684640 -0.050390
c 2.433281 5.163008 1.177045
d NaN 5.442353 0.563873

In [145]: df.apply(np.exp)
Out[145]:
 one two three
a 4.034899 5.885648 NaN
b 1.409244 6.767440 0.950858
c 2.004201 4.385785 3.412466
d NaN 1.322262 0.541630

```

`apply()` 方法还支持通过函数名字串调用函数。

```
In [146]: df.apply('mean')
Out[146]:
one 0.811094
two 1.360588
three 0.187958
dtype: float64

In [147]: df.apply('mean', axis=1)
Out[147]:
a 1.583749
b 0.734929
c 1.133683
d -0.166914
dtype: float64
```

默认情况下，`apply()` 调用的函数返回的类型会影响 `DataFrame.apply` 输出结果的类型。

- 函数返回的是 `Series` 时，最终输出结果是 `DataFrame`。输出的列与函数返回的 `Series` 索引相匹配。
- 函数返回其它任意类型时，输出结果是 `Series`。

`result_type` 会覆盖默认行为，该参数有三个选项：`reduce`、`broadcast`、`expand`。这些选项决定了列表型返回值是否扩展为 `DataFrame`。

用好 `apply()` 可以了解数据集的很多信息。比如可以提取每列的最大值对应的日期：

```
In [148]: tsdf = pd.DataFrame(np.random.randn(1000, 3), columns=['A', 'B', 'C'],
.....: index=pd.date_range('1/1/2000', periods=1000))
.....:

In [149]: tsdf.apply(lambda x: x.idxmax())
Out[149]:
A 2000-08-06
B 2001-01-18
C 2001-07-18
dtype: datetime64[ns]
```

还可以向 `apply()` 方法传递额外的参数与关键字参数。比如下例中要应用的这个函数：

```
def subtract_and_divide(x, sub, divide=1):
 return (x - sub) / divide
```

可以用下列方式应用该函数：

```
df.apply(subtract_and_divide, args=(5,), divide=3)
```

为每行或每列执行 `Series` 方法的功能也很实用：

```
In [150]: tsdf
Out[150]:
```

	A	B	C
2000-01-01	-0.158131	-0.232466	0.321604
2000-01-02	-1.810340	-3.105758	0.433834
2000-01-03	-1.209847	-1.156793	-0.136794
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	-0.653602	0.178875	1.008298
2000-01-09	1.007996	0.462824	0.254472
2000-01-10	0.307473	0.600337	1.643950

```
In [151]: tsdf.apply(pd.Series.interpolate)
Out[151]:
```

	A	B	C
2000-01-01	-0.158131	-0.232466	0.321604
2000-01-02	-1.810340	-3.105758	0.433834
2000-01-03	-1.209847	-1.156793	-0.136794
2000-01-04	-1.098598	-0.889659	0.092225
2000-01-05	-0.987349	-0.622526	0.321243
2000-01-06	-0.876100	-0.355392	0.550262
2000-01-07	-0.764851	-0.088259	0.779280
2000-01-08	-0.653602	0.178875	1.008298
2000-01-09	1.007996	0.462824	0.254472
2000-01-10	0.307473	0.600337	1.643950

`apply()` 有一个参数 `raw`，默认值为 `False`，在应用函数前，使用该参数可以将每行或列转换为 `Series`。该参数为 `True` 时，传递的函数接收 `ndarray` 对象，若不需要索引功能，这种操作能显著提高性能。

## 聚合 API

0.20.0 版新增。

聚合 API 可以快速、简洁地执行多个聚合操作。Pandas 对象支持多个类似的 API，如 [groupby API](#)、[window functions API](#)、[resample API](#)。聚合函数为 `DataFrame.aggregate()`，它的别名是 `DataFrame.agg()`。

此处用与上例类似的 `DataFrame`：



```
In [152]: tsdf = pd.DataFrame(np.random.randn(10, 3), columns=['A', 'B', 'C'],
.....: index=pd.date_range('1/1/2000', periods=10))
.....:
```

```
In [153]: tsdf.iloc[3:7] = np.nan
```

```
In [154]: tsdf
```

```
Out[154]:
```

	A	B	C
2000-01-01	1.257606	1.004194	0.167574
2000-01-02	-0.749892	0.288112	-0.757304
2000-01-03	-0.207550	-0.298599	0.116018
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.814347	-0.257623	0.869226
2000-01-09	-0.250663	-1.206601	0.896839
2000-01-10	2.169758	-1.333363	0.283157

应用单个函数时，该操作与 `apply()` 等效，这里也可以用字符串表示聚合函数名。下面的聚合函数输出的结果为 `Series`：

```
In [155]: tsdf.agg(np.sum)
```

```
Out[155]:
```

```
A 3.033606
```

```
B -1.803879
```

```
C 1.575510
```

```
dtype: float64
```

```
In [156]: tsdf.agg('sum')
```

```
Out[156]:
```

```
A 3.033606
```

```
B -1.803879
```

```
C 1.575510
```

```
dtype: float64
```

```
因为应用的是单个函数，该操作与`.sum()`是等效的
```

```
In [157]: tsdf.sum()
```

```
Out[157]:
```

```
A 3.033606
```

```
B -1.803879
```

```
C 1.575510
```

```
dtype: float64
```

`Series` 单个聚合操作返回标量值：

```
In [158]: tsdf.A.agg('sum')
```

```
Out[158]: 3.033606102414146
```

## 多函数聚合

还可以用列表形式传递多个聚合函数。每个函数在输出结果 `DataFrame` 里以行的形式显示，行名是每个聚合函数的函数名。

```
In [159]: tsdf.agg(['sum'])
Out[159]:
```

	A	B	C
sum	3.033606	-1.803879	1.57551

多个函数输出多行：

```
In [160]: tsdf.agg(['sum', 'mean'])
Out[160]:
```

	A	B	C
sum	3.033606	-1.803879	1.575510
mean	0.505601	-0.300647	0.262585

`Series` 聚合多函数返回结果还是 `Series`，索引为函数名：

```
In [161]: tsdf.A.agg(['sum', 'mean'])
Out[161]:
```

sum	3.033606
mean	0.505601

Name: A, dtype: float64

传递 `lambda` 函数时，输出名为 `<lambda>` 的行：

```
In [162]: tsdf.A.agg(['sum', lambda x: x.mean()])
Out[162]:
```

sum	3.033606
<lambda>	0.505601

Name: A, dtype: float64

应用自定义函数时，该函数名为输出结果的行名：

```
In [163]: def mymean(x):
.....: return x.mean()
.....:

In [164]: tsdf.A.agg(['sum', mymean])
Out[164]:
```

sum	3.033606
mymean	0.505601

Name: A, dtype: float64

## 用字典实现聚合

指定为哪些列应用哪些聚合函数时，需要把包含列名与标量（或标量列表）的字典传递给 `DataFrame.agg`。

注意：这里输出结果的顺序不是固定的，要想让输出顺序与输入顺序一致，请使用 `OrderedDict`。

```
In [165]: tsdf.agg({'A': 'mean', 'B': 'sum'})
Out[165]:
A 0.505601
B -1.803879
dtype: float64
```

输入的参数是列表时，输出结果为 `DataFrame`，并以矩阵形式显示所有聚合函数的计算结果，且输出结果由所有唯一函数组成。未执行聚合操作的列输出结果为 `NaN` 值：

```
In [166]: tsdf.agg({'A': ['mean', 'min'], 'B': 'sum'})
Out[166]:
```

	A	B
mean	0.505601	NaN
min	-0.749892	NaN
sum	NaN	-1.803879

## 多种数据类型 (Dtype)

与 `groupby` 的 `.agg` 操作类似，`DataFrame` 含不能执行聚合的数据类型时，`.agg` 只计算可聚合的列：

```
In [167]: mdf = pd.DataFrame({'A': [1, 2, 3],
.....: 'B': [1., 2., 3.],
.....: 'C': ['foo', 'bar', 'baz'],
.....: 'D': pd.date_range('20130101', periods=3)})
.....:

In [168]: mdf.dtypes
Out[168]:
A int64
B float64
C object
D datetime64[ns]
dtype: object
```

```
In [169]: mdf.agg(['min', 'sum'])
Out[169]:
```

	A	B	C	D
min	1	1.0	bar	2013-01-01
sum	6	6.0	foobarbaz	NaT

## 自定义 Describe

`.agg()` 可以创建类似于内置 `describe` 函数的自定义 `describe` 函数。

```
In [170]: from functools import partial

In [171]: q_25 = partial(pd.Series.quantile, q=0.25)

In [172]: q_25.__name__ = '25%'

In [173]: q_75 = partial(pd.Series.quantile, q=0.75)

In [174]: q_75.__name__ = '75%'

In [175]: tsdf.agg(['count', 'mean', 'std', 'min', q_25, 'median', q_75, 'max'])
Out[175]:
```

	A	B	C
count	6.000000	6.000000	6.000000
mean	0.505601	-0.300647	0.262585
std	1.103362	0.887508	0.606860
min	-0.749892	-1.333363	-0.757304
25%	-0.239885	-0.979600	0.128907
median	0.303398	-0.278111	0.225365
75%	1.146791	0.151678	0.722709
max	2.169758	1.004194	0.896839

## Transform API

0.20.0 版新增。

`transform()` 方法的返回结果与原始数据的索引相同，大小相同。与 `agg` API 类似，该 API 支持同时处理多种操作，不用一个一个操作。

下面，先创建一个 DataFrame：

```
In [176]: tsdf = pd.DataFrame(np.random.randn(10, 3), columns=['A', 'B', 'C'],
.....: index=pd.date_range('1/1/2000', periods=10))
.....:

In [177]: tsdf.iloc[3:7] = np.nan

In [178]: tsdf
Out[178]:
```

	A	B	C
2000-01-01	-0.428759	-0.864890	-0.675341
2000-01-02	-0.168731	1.338144	-1.279321
2000-01-03	-1.621034	0.438107	0.903794
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.254374	-1.240447	-0.201052
2000-01-09	-0.157795	0.791197	-1.144209
2000-01-10	-0.030876	0.371900	0.061932

这里转换的是整个 DataFrame。 `.transform()` 支持 NumPy 函数、字符串函数及自定义函数。

```
In [179]: tsdf.transform(np.abs)
```

```
Out[179]:
```

	A	B	C
2000-01-01	0.428759	0.864890	0.675341
2000-01-02	0.168731	1.338144	1.279321
2000-01-03	1.621034	0.438107	0.903794
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.254374	1.240447	0.201052
2000-01-09	0.157795	0.791197	1.144209
2000-01-10	0.030876	0.371900	0.061932

```
In [180]: tsdf.transform('abs')
```

```
Out[180]:
```

	A	B	C
2000-01-01	0.428759	0.864890	0.675341
2000-01-02	0.168731	1.338144	1.279321
2000-01-03	1.621034	0.438107	0.903794
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.254374	1.240447	0.201052
2000-01-09	0.157795	0.791197	1.144209
2000-01-10	0.030876	0.371900	0.061932

```
In [181]: tsdf.transform(lambda x: x.abs())
```

```
Out[181]:
```

	A	B	C
2000-01-01	0.428759	0.864890	0.675341
2000-01-02	0.168731	1.338144	1.279321
2000-01-03	1.621034	0.438107	0.903794
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.254374	1.240447	0.201052
2000-01-09	0.157795	0.791197	1.144209
2000-01-10	0.030876	0.371900	0.061932

这里的 `transform()` 接受单个函数；与 `ufunc` 等效。

```
In [182]: np.abs(tsdf)
Out[182]:
```

	A	B	C
2000-01-01	0.428759	0.864890	0.675341
2000-01-02	0.168731	1.338144	1.279321
2000-01-03	1.621034	0.438107	0.903794
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.254374	1.240447	0.201052
2000-01-09	0.157795	0.791197	1.144209
2000-01-10	0.030876	0.371900	0.061932

`.transform()` 向 Series 传递单个函数时，返回的结果也是单个 Series。

```
In [183]: tsdf.A.transform(np.abs)
Out[183]:
```

2000-01-01	0.428759
2000-01-02	0.168731
2000-01-03	1.621034
2000-01-04	NaN
2000-01-05	NaN
2000-01-06	NaN
2000-01-07	NaN
2000-01-08	0.254374
2000-01-09	0.157795
2000-01-10	0.030876

Freq: D, Name: A, dtype: float64

## 多函数 Transform

`transform()` 调用多个函数时，生成多层索引 DataFrame。第一层是原始数据集的列名；第二层是 `transform()` 调用的函数名。

```
In [184]: tsdf.transform([np.abs, lambda x: x + 1])
Out[184]:
```

	A		B		C	
	absolute	<lambda>	absolute	<lambda>	absolute	<lambda>
2000-01-01	0.428759	0.571241	0.864890	0.135110	0.675341	0.324659
2000-01-02	0.168731	0.831269	1.338144	2.338144	1.279321	-0.279321
2000-01-03	1.621034	-0.621034	0.438107	1.438107	0.903794	1.903794
2000-01-04	NaN	NaN	NaN	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN	NaN	NaN	NaN
2000-01-08	0.254374	1.254374	1.240447	-0.240447	0.201052	0.798948
2000-01-09	0.157795	0.842205	0.791197	1.791197	1.144209	-0.144209
2000-01-10	0.030876	0.969124	0.371900	1.371900	0.061932	1.061932

为 Series 应用多个函数时，输出结果是 DataFrame，列名是 `transform()` 调用的函数名。

```
In [185]: tsdf.A.transform([np.abs, lambda x: x + 1])
Out[185]:
```

	absolute	<lambda>
2000-01-01	0.428759	0.571241
2000-01-02	0.168731	0.831269
2000-01-03	1.621034	-0.621034
2000-01-04	NaN	NaN
2000-01-05	NaN	NaN
2000-01-06	NaN	NaN
2000-01-07	NaN	NaN
2000-01-08	0.254374	1.254374
2000-01-09	0.157795	0.842205
2000-01-10	0.030876	0.969124

## 用字典执行 transform 操作

函数字典可以为每列执行指定 transform() 操作。

```
In [186]: tsdf.transform({'A': np.abs, 'B': lambda x: x + 1})
Out[186]:
```

	A	B
2000-01-01	0.428759	0.135110
2000-01-02	0.168731	2.338144
2000-01-03	1.621034	1.438107
2000-01-04	NaN	NaN
2000-01-05	NaN	NaN
2000-01-06	NaN	NaN
2000-01-07	NaN	NaN
2000-01-08	0.254374	-0.240447
2000-01-09	0.157795	1.791197
2000-01-10	0.030876	1.371900

transform() 的参数是列表字典时，生成的是以 transform() 调用的函数为名的多层索引 DataFrame。

```
In [187]: tsdf.transform({'A': np.abs, 'B': [lambda x: x + 1, 'sqrt']})
Out[187]:
```

	A	B	
	absolute	<lambda>	sqrt
2000-01-01	0.428759	0.135110	NaN
2000-01-02	0.168731	2.338144	1.156782
2000-01-03	1.621034	1.438107	0.661897
2000-01-04	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN
2000-01-08	0.254374	-0.240447	NaN
2000-01-09	0.157795	1.791197	0.889493
2000-01-10	0.030876	1.371900	0.609836

## 元素级函数应用

并非所有函数都能矢量化，即接受 NumPy 数组，返回另一个数组或值，DataFrame 的 `applymap()` 及 Series 的 `map()`，支持任何接收单个值并返回单个值的 Python 函数。

示例如下：

```
In [188]: df4
Out[188]:
```

	one	two	three
a	1.394981	1.772517	NaN
b	0.343054	1.912123	-0.050390
c	0.695246	1.478369	1.227435
d	NaN	0.279344	-0.613172

```
In [189]: def f(x):
.....: return len(str(x))
.....:

In [190]: df4['one'].map(f)
Out[190]:
```

a	18
b	19
c	18
d	3

Name: one, dtype: int64

```
In [191]: df4.applymap(f)
Out[191]:
```

	one	two	three
a	18	17	3
b	19	18	20
c	18	18	16
d	3	19	19

`Series.map()` 还有个功能，可以“连接”或“映射”第二个 Series 定义的值。这与 `merging / joining` 功能联系非常紧密：



```

In [192]: s = pd.Series(['six', 'seven', 'six', 'seven', 'six'],
.....: index=['a', 'b', 'c', 'd', 'e'])
.....:

In [193]: t = pd.Series({'six': 6., 'seven': 7.})

In [194]: s
Out[194]:
a six
b seven
c six
d seven
e six
dtype: object

In [195]: s.map(t)
Out[195]:
a 6.0
b 7.0
c 6.0
d 7.0
e 6.0
dtype: float64

```

## 重置索引与更换标签

`reindex()` 是 Pandas 里实现数据对齐的基本方法，该方法执行几乎所有功能都要用到的标签对齐功能。

`reindex` 指的是沿着指定轴，让数据与给定的一组标签进行匹配。该功能完成以下几项操作：

- 让现有数据匹配一组新标签，并重新排序；
- 在无数据但有标签的位置插入缺失值（`NA`）标记；
- 如果指定，则按逻辑填充无标签的数据，该操作多见于时间序列数据。

示例如下：

```

In [196]: s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])

In [197]: s
Out[197]:
a 1.695148
b 1.328614
c 1.234686
d -0.385845
e -1.326508
dtype: float64

In [198]: s.reindex(['e', 'b', 'f', 'd'])
Out[198]:
e -1.326508
b 1.328614
f NaN
d -0.385845
dtype: float64

```

本例中，原 Series 里没有标签 `f`，因此，输出结果里 `f` 对应的值为 `NaN`。

DataFrame 支持同时 `reindex` 索引与列：

```

In [199]: df
Out[199]:
 one two three
a 1.394981 1.772517 NaN
b 0.343054 1.912123 -0.050390
c 0.695246 1.478369 1.227435
d NaN 0.279344 -0.613172

In [200]: df.reindex(index=['c', 'f', 'b'], columns=['three', 'two', 'one'])
Out[200]:
 three two one
c 1.227435 1.478369 0.695246
f NaN NaN NaN
b -0.050390 1.912123 0.343054

```

`reindex` 还支持 `axis` 关键字：

```

In [201]: df.reindex(['c', 'f', 'b'], axis='index')
Out[201]:
 one two three
c 0.695246 1.478369 1.227435
f NaN NaN NaN
b 0.343054 1.912123 -0.050390

```

注意：不同对象可以共享 Index 包含的轴标签。比如，有一个 Series，还有一个 DataFrame，可以执行下列操作：

```
In [202]: rs = s.reindex(df.index)
```

```
In [203]: rs
```

```
Out[203]:
```

```
a 1.695148
```

```
b 1.328614
```

```
c 1.234686
```

```
d -0.385845
```

```
dtype: float64
```

```
In [204]: rs.index is df.index
```

```
Out[204]: True
```

这里指的是，重置后，Series 的索引与 DataFrame 的索引是同一个 Python 对象。

0.21.0 版新增。

`DataFrame.reindex()` 还支持“轴样式”调用习语，可以指定单个 `labels` 参数，并指定应用于哪个 `axis`。

```
In [205]: df.reindex(['c', 'f', 'b'], axis='index')
```

```
Out[205]:
```

	one	two	three
c	0.695246	1.478369	1.227435
f	NaN	NaN	NaN
b	0.343054	1.912123	-0.050390

```
In [206]: df.reindex(['three', 'two', 'one'], axis='columns')
```

```
Out[206]:
```

	three	two	one
a	NaN	1.772517	1.394981
b	-0.050390	1.912123	0.343054
c	1.227435	1.478369	0.695246
d	-0.613172	0.279344	NaN

⋮ tip 注意

[多层索引与高级索引](#)介绍了怎样用更简洁的方式重置索引。

⋮

⋮ tip 注意

编写注重性能的代码时，最好花些时间深入理解 `reindex`：预对齐数据后，操作会更快。两个未对齐的 DataFrame 相加，后台操作会执行 `reindex`。探索性分析时很难注意到这点有什么不同，这是因为 `reindex` 已经进行了高度优化，但需要注重 CPU 周期时，显式调用 `reindex` 还是有一些影响的。

⋮

## 重置索引，并与其它对象对齐

提取一个对象，并用另一个具有相同标签的对象 `reindex` 该对象的轴。这种操作的语法虽然简单，但未免有些啰嗦。这时，最好用 `reindex_like()` 方法，这是一种既有效，又简单的方式：

```
In [207]: df2
Out[207]:
```

	one	two
a	1.394981	1.772517
b	0.343054	1.912123
c	0.695246	1.478369

```
In [208]: df3
Out[208]:
```

	one	two
a	0.583888	0.051514
b	-0.468040	0.191120
c	-0.115848	-0.242634

```
In [209]: df.reindex_like(df2)
Out[209]:
```

	one	two
a	1.394981	1.772517
b	0.343054	1.912123
c	0.695246	1.478369

## 用 `align` 对齐多个对象

`align()` 方法是对齐两个对象最快的方式，该方法支持 `join` 参数（请参阅 [joining 与 merging](#)）：

- `join='outer'`：使用两个对象索引的合集，默认值
- `join='left'`：使用左侧调用对象的索引
- `join='right'`：使用右侧传递对象的索引
- `join='inner'`：使用两个对象索引的交集

该方法返回重置索引后的两个 Series 元组：

```
In [210]: s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])
```

```
In [211]: s1 = s[:4]
```

```
In [212]: s2 = s[1:]
```

```
In [213]: s1.align(s2)
```

```
Out[213]:
```

```
(a -0.186646
 b -1.692424
 c -0.303893
 d -1.425662
 e NaN
dtype: float64, a NaN
 b -1.692424
 c -0.303893
 d -1.425662
 e 1.114285
dtype: float64)
```

```
In [214]: s1.align(s2, join='inner')
```

```
Out[214]:
```

```
(b -1.692424
 c -0.303893
 d -1.425662
dtype: float64, b -1.692424
 c -0.303893
 d -1.425662
dtype: float64)
```

```
In [215]: s1.align(s2, join='left')
```

```
Out[215]:
```

```
(a -0.186646
 b -1.692424
 c -0.303893
 d -1.425662
dtype: float64, a NaN
 b -1.692424
 c -0.303893
 d -1.425662
dtype: float64)
```

默认条件下，`join` 方法既应用于索引，也应用于列：

```
In [216]: df.align(df2, join='inner')
```

```
Out[216]:
```

```
(one two
a 1.394981 1.772517
b 0.343054 1.912123
c 0.695246 1.478369, one two
a 1.394981 1.772517
b 0.343054 1.912123
c 0.695246 1.478369)
```

align 方法还支持 axis 选项，用来指定要对齐的轴：

```
In [217]: df.align(df2, join='inner', axis=0)
Out[217]:
(
 one two three
a 1.394981 1.772517 NaN
b 0.343054 1.912123 -0.050390
c 0.695246 1.478369 1.227435, one two
a 1.394981 1.772517
b 0.343054 1.912123
c 0.695246 1.478369)
```

如果把 Series 传递给 DataFrame.align(), 可以用 axis 参数选择是在 DataFrame 的索引，还是列上对齐两个对象：

```
In [218]: df.align(df2.iloc[0], axis=1)
Out[218]:
(
 one three two
a 1.394981 NaN 1.772517
b 0.343054 -0.050390 1.912123
c 0.695246 1.227435 1.478369
d NaN -0.613172 0.279344, one 1.394981
three NaN
two 1.772517
Name: a, dtype: float64)
```

方法	动作
pad / ffill	先前填充
bfill / backfill	向后填充
nearest	从最近的索引值填充

下面用一个简单的 Series 展示 fill 方法：

```
In [219]: rng = pd.date_range('1/3/2000', periods=8)

In [220]: ts = pd.Series(np.random.randn(8), index=rng)

In [221]: ts2 = ts[[0, 3, 6]]
```

```
In [222]: ts
Out[222]:
2000-01-03 0.183051
2000-01-04 0.400528
2000-01-05 -0.015083
2000-01-06 2.395489
2000-01-07 1.414806
2000-01-08 0.118428
2000-01-09 0.733639
2000-01-10 -0.936077
Freq: D, dtype: float64
```

```
In [223]: ts2
Out[223]:
2000-01-03 0.183051
2000-01-06 2.395489
2000-01-09 0.733639
dtype: float64
```

```
In [224]: ts2.reindex(ts.index)
Out[224]:
2000-01-03 0.183051
2000-01-04 NaN
2000-01-05 NaN
2000-01-06 2.395489
2000-01-07 NaN
2000-01-08 NaN
2000-01-09 0.733639
2000-01-10 NaN
Freq: D, dtype: float64
```

```
In [225]: ts2.reindex(ts.index, method='ffill')
Out[225]:
2000-01-03 0.183051
2000-01-04 0.183051
2000-01-05 0.183051
2000-01-06 2.395489
2000-01-07 2.395489
2000-01-08 2.395489
2000-01-09 0.733639
2000-01-10 0.733639
Freq: D, dtype: float64
```

```
In [226]: ts2.reindex(ts.index, method='bfill')
Out[226]:
2000-01-03 0.183051
2000-01-04 2.395489
2000-01-05 2.395489
2000-01-06 2.395489
```

```
2000-01-07 0.733639
2000-01-08 0.733639
2000-01-09 0.733639
2000-01-10 NaN
Freq: D, dtype: float64
```

```
In [227]: ts2.reindex(ts.index, method='nearest')
```

```
Out[227]:
2000-01-03 0.183051
2000-01-04 0.183051
2000-01-05 2.395489
2000-01-06 2.395489
2000-01-07 2.395489
2000-01-08 0.733639
2000-01-09 0.733639
2000-01-10 0.733639
Freq: D, dtype: float64
```

上述操作要求索引按递增或递减排序。

注意：除了 `method='nearest'`，用 `fillna` 或 `interpolate` 也能实现同样的效果：

```
In [228]: ts2.reindex(ts.index).fillna(method='ffill')
```

```
Out[228]:
2000-01-03 0.183051
2000-01-04 0.183051
2000-01-05 0.183051
2000-01-06 2.395489
2000-01-07 2.395489
2000-01-08 2.395489
2000-01-09 0.733639
2000-01-10 0.733639
Freq: D, dtype: float64
```

如果索引不是按递增或递减排序，`reindex()` 会触发 `ValueError` 错误。`fillna()` 与 `interpolate()` 则不检查索引的排序。

## 重置索引填充的限制

`limit` 与 `tolerance` 参数可以控制 `reindex` 的填充操作。`limit` 限制了连续匹配的最大数量：

```
In [229]: ts2.reindex(ts.index, method='ffill', limit=1)
```

```
Out[229]:
2000-01-03 0.183051
2000-01-04 0.183051
2000-01-05 NaN
2000-01-06 2.395489
2000-01-07 2.395489
2000-01-08 NaN
2000-01-09 0.733639
2000-01-10 0.733639
Freq: D, dtype: float64
```



反之，`tolerance` 限定了索引与索引器值之间的最大距离：

```
In [230]: ts2.reindex(ts.index, method='ffill', tolerance='1 day')
Out[230]:
2000-01-03 0.183051
2000-01-04 0.183051
2000-01-05 NaN
2000-01-06 2.395489
2000-01-07 2.395489
2000-01-08 NaN
2000-01-09 0.733639
2000-01-10 0.733639
Freq: D, dtype: float64
```

注意：索引为 `DatetimeIndex`、`TimedeltaIndex` 或 `PeriodIndex` 时，`tolerance` 会尽可能将这些索引强制转换为 `Timedelta`，这里要求用户用恰当的字符串设定 `tolerance` 参数。

## 去掉轴上的标签

`drop()` 函数与 `reindex` 经常配合使用，该函数用于删除轴上的一组标签：

```
In [231]: df
Out[231]:
 one two three
a 1.394981 1.772517 NaN
b 0.343054 1.912123 -0.050390
c 0.695246 1.478369 1.227435
d NaN 0.279344 -0.613172

In [232]: df.drop(['a', 'd'], axis=0)
Out[232]:
 one two three
b 0.343054 1.912123 -0.050390
c 0.695246 1.478369 1.227435

In [233]: df.drop(['one'], axis=1)
Out[233]:
 two three
a 1.772517 NaN
b 1.912123 -0.050390
c 1.478369 1.227435
d 0.279344 -0.613172
```

注意：下面的代码可以运行，但不够清晰：

```
In [234]: df.reindex(df.index.difference(['a', 'd']))
Out[234]:
 one two three
b 0.343054 1.912123 -0.050390
c 0.695246 1.478369 1.227435
```

## 重命名或映射标签

`rename()` 方法支持按不同的轴基于映射（字典或 Series）调整标签。

```
In [235]: s
Out[235]:
a -0.186646
b -1.692424
c -0.303893
d -1.425662
e 1.114285
dtype: float64

In [236]: s.rename(str.upper)
Out[236]:
A -0.186646
B -1.692424
C -0.303893
D -1.425662
E 1.114285
dtype: float64
```

如果调用的是函数，该函数在处理标签时，必须返回一个值，而且生成的必须是一组唯一值。此外，`rename()` 还可以调用字典或 Series。

```
In [237]: df.rename(columns={'one': 'foo', 'two': 'bar'},
.....: index={'a': 'apple', 'b': 'banana', 'd': 'durian'})
.....:
Out[237]:
```

	foo	bar	three
apple	1.394981	1.772517	NaN
banana	0.343054	1.912123	-0.050390
c	0.695246	1.478369	1.227435
durian	NaN	0.279344	-0.613172

Pandas 不会重命名标签未包含在映射里的列或索引。注意，映射里多出的标签不会触发错误。

0.21.0 版新增。

`DataFrame.rename()` 还支持“轴式”习语，用这种方式可以指定单个 `mapper`，及执行映射的 `axis`。

```
In [238]: df.rename({'one': 'foo', 'two': 'bar'}, axis='columns')
Out[238]:
```

	foo	bar	three
a	1.394981	1.772517	NaN
b	0.343054	1.912123	-0.050390
c	0.695246	1.478369	1.227435
d	NaN	0.279344	-0.613172

```
In [239]: df.rename({'a': 'apple', 'b': 'banana', 'd': 'durian'}, axis='index')
Out[239]:
```

	one	two	three
apple	1.394981	1.772517	NaN
banana	0.343054	1.912123	-0.050390
c	0.695246	1.478369	1.227435
durian	NaN	0.279344	-0.613172

`rename()` 方法还提供了 `inplace` 命名参数，默认为 `False`，并会复制底层数据。`inplace=True` 时，会直接在原数据上重命名。

0.18.0 版新增。

`rename()` 还支持用标量或列表更改 `Series.name` 属性。

```
In [240]: s.rename("scalar-name")
Out[240]:
```

a	-0.186646
b	-1.692424
c	-0.303893
d	-1.425662
e	1.114285

Name: scalar-name, dtype: float64

0.24.0 版新增。

`rename_axis()` 方法支持指定 多层索引 名称，与标签相对应。

```
In [241]: df = pd.DataFrame({'x': [1, 2, 3, 4, 5, 6],
.....: 'y': [10, 20, 30, 40, 50, 60]},
.....: index=pd.MultiIndex.from_product([['a', 'b', 'c'], [1, 2]],
.....: names=['let', 'num']))
.....:
```

```
In [242]: df
```

```
Out[242]:
```

	x	y
let num		
a 1	1	10
2	2	20
b 1	3	30
2	4	40
c 1	5	50
2	6	60

```
In [243]: df.rename_axis(index={'let': 'abc'})
```

```
Out[243]:
```

	x	y
abc num		
a 1	1	10
2	2	20
b 1	3	30
2	4	40
c 1	5	50
2	6	60

```
In [244]: df.rename_axis(index=str.upper)
```

```
Out[244]:
```

	x	y
LET NUM		
a 1	1	10
2	2	20
b 1	3	30
2	4	40
c 1	5	50
2	6	60

## 迭代

Pandas 对象基于类型进行迭代操作。Series 迭代时被视为数组，基础迭代生成值。DataFrame 则遵循字典式习语，用对象的 `key` 实现迭代操作。

简言之，基础迭代（`for i in object`）生成：

- **Series**：值
- **DataFrame**：列标签

例如，DataFrame 迭代时输出列名：

```
In [245]: df = pd.DataFrame({'col1': np.random.randn(3),
.....: 'col2': np.random.randn(3)}, index=['a', 'b', 'c'])
.....:

In [246]: for col in df:
.....: print(col)
.....:
col1
col2
```

Pandas 对象还支持字典式的 `items()` 方法，通过键值对迭代。

用下列方法可以迭代 DataFrame 里的行：

- `iterrows()`：把 DataFrame 里的行当作 (index, Series) 对进行迭代。该操作把行转为 Series，同时改变数据类型，并对性能有影响。
- `itertuples()` 把 DataFrame 的行当作值的命名元组进行迭代。该操作比 `iterrows()` 快的多，建议尽量用这种方法迭代 DataFrame 的值。

::: danger 警告

Pandas 对象迭代的速度较慢。大部分情况下，没必要对行执行迭代操作，建议用以下几种替代方式：

- 矢量化：很多操作可以用内置方法或 NumPy 函数，布尔索引……
- 调用的函数不能在完整的 DataFrame / Series 上运行时，最好用 `apply()`，不要对值进行迭代操作。请参阅[函数应用](#)文档。
- 如果必须对值进行迭代，请务必注意代码的性能，建议在 cython 或 numba 环境下实现内循环。参阅[性能优化](#)一节，查看这种操作方法的示例。

:::

::: danger 警告

**永远不要修改**迭代的内容，这种方式不能确保所有操作都能正常运作。基于数据类型，迭代器返回的是复制 (copy) 的结果，不是视图 (view)，这种写入可能不会生效！

下例中的赋值就不会生效：

```
In [247]: df = pd.DataFrame({'a': [1, 2, 3], 'b': ['a', 'b', 'c']})

In [248]: for index, row in df.iterrows():
.....: row['a'] = 10
.....:

In [249]: df
Out[249]:
a b
0 1 a
1 2 b
2 3 c
```

...

## 项目 (items)

与字典接口类似，`items()` 通过键值对进行迭代：

- **Series**: (Index, 标量值) 对
- **DataFrame**: (列, Series) 对

示例如下：

```
In [250]: for label, ser in df.items():
.....: print(label)
.....: print(ser)
.....:

a
0 1
1 2
2 3
Name: a, dtype: int64
b
0 a
1 b
2 c
Name: b, dtype: object
```

## iterrows

`iterrows()` 迭代 DataFrame 或 Series 里的每一行数据。这个操作返回一个迭代器，生成索引值及包含每行数据的 Series：

```
In [251]: for row_index, row in df.iterrows():
.....: print(row_index, row, sep='\n')
.....:
0
a 1
b a
Name: 0, dtype: object
1
a 2
b b
Name: 1, dtype: object
2
a 3
b c
Name: 2, dtype: object
```

⋮ tip 注意

`iterrows()` 返回的是 Series 里的每一行数据，该操作**不**保留每行数据的数据类型，因为数据类型是通过 DataFrame 的列界定的。

示例如下：

```
In [252]: df_orig = pd.DataFrame([[1, 1.5]], columns=['int', 'float'])

In [253]: df_orig.dtypes
Out[253]:
int int64
float float64
dtype: object

In [254]: row = next(df_orig.iterrows())[1]

In [255]: row
Out[255]:
int 1.0
float 1.5
Name: 0, dtype: float64
```

`row` 里的值以 Series 形式返回，并被转换为浮点数，原始的整数值则在列 X：

```
In [256]: row['int'].dtype
Out[256]: dtype('float64')

In [257]: df_orig['int'].dtype
Out[257]: dtype('int64')
```

要想在行迭代时保存数据类型，最好用 `itertuples()`，这个函数返回值的命名元组，总的来说，该操作比 `iterrows()` 速度更快。

⋮

下例展示了怎样转置 DataFrame:

```
In [258]: df2 = pd.DataFrame({'x': [1, 2, 3], 'y': [4, 5, 6]})

In [259]: print(df2)
 x y
0 1 4
1 2 5
2 3 6

In [260]: print(df2.T)
 0 1 2
x 1 2 3
y 4 5 6

In [261]: df2_t = pd.DataFrame({idx: values for idx, values in df2.iterrows()})

In [262]: print(df2_t)
 0 1 2
x 1 2 3
y 4 5 6
```

## itertuples

`itertuples()` 方法返回为 DataFrame 里每行数据生成命名元组的迭代器。该元组的第一个元素是行的索引值，其余的值则是行的值。

示例如下:

```
In [263]: for row in df.itertuples():
.....: print(row)
.....:
Pandas(Index=0, a=1, b='a')
Pandas(Index=1, a=2, b='b')
Pandas(Index=2, a=3, b='c')
```

该方法不会把行转换为 Series，只是返回命名元组里的值。`itertuples()` 保存值的数据类型，而且比 `iterrows()` 快。

⋮ tip 注意

包含无效 Python 识别符的列名、重复的列名及以下划线开头的列名，会被重命名为位置名称。如果列数较大，比如大于 255 列，则返回正则元组。

⋮



## .dt 访问器

`Series` 提供一个可以简单、快捷地返回 `datetime` 属性值的访问器。这个访问器返回的也是 `Series`，索引与现有的 `Series` 一样。

```
datetime
In [264]: s = pd.Series(pd.date_range('20130101 09:10:12', periods=4))

In [265]: s
Out[265]:
0 2013-01-01 09:10:12
1 2013-01-02 09:10:12
2 2013-01-03 09:10:12
3 2013-01-04 09:10:12
dtype: datetime64[ns]

In [266]: s.dt.hour
Out[266]:
0 9
1 9
2 9
3 9
dtype: int64

In [267]: s.dt.second
Out[267]:
0 12
1 12
2 12
3 12
dtype: int64

In [268]: s.dt.day
Out[268]:
0 1
1 2
2 3
3 4
dtype: int64
```

用下列表达式进行筛选非常方便：

```
In [269]: s[s.dt.day == 2]
Out[269]:
1 2013-01-02 09:10:12
dtype: datetime64[ns]
```

时区转换也很轻松：

```

In [270]: stz = s.dt.tz_localize('US/Eastern')

In [271]: stz
Out[271]:
0 2013-01-01 09:10:12-05:00
1 2013-01-02 09:10:12-05:00
2 2013-01-03 09:10:12-05:00
3 2013-01-04 09:10:12-05:00
dtype: datetime64[ns, US/Eastern]

In [272]: stz.dt.tz
Out[272]: <DstTzInfo 'US/Eastern' LMT-1 day, 19:04:00 STD>

```

可以把这些操作连在一起：

```

In [273]: s.dt.tz_localize('UTC').dt.tz_convert('US/Eastern')
Out[273]:
0 2013-01-01 04:10:12-05:00
1 2013-01-02 04:10:12-05:00
2 2013-01-03 04:10:12-05:00
3 2013-01-04 04:10:12-05:00
dtype: datetime64[ns, US/Eastern]

```

还可以用 `Series.dt.strftime()` 把 `datetime` 的值当成字符串进行格式化，支持与标准 `strftime()` 同样的格式。

```

DatetimeIndex
In [274]: s = pd.Series(pd.date_range('20130101', periods=4))

In [275]: s
Out[275]:
0 2013-01-01
1 2013-01-02
2 2013-01-03
3 2013-01-04
dtype: datetime64[ns]

In [276]: s.dt.strftime('%Y/%m/%d')
Out[276]:
0 2013/01/01
1 2013/01/02
2 2013/01/03
3 2013/01/04
dtype: object

```

```
PeriodIndex
In [277]: s = pd.Series(pd.period_range('20130101', periods=4))

In [278]: s
Out[278]:
0 2013-01-01
1 2013-01-02
2 2013-01-03
3 2013-01-04
dtype: period[D]

In [279]: s.dt.strftime('%Y/%m/%d')
Out[279]:
0 2013/01/01
1 2013/01/02
2 2013/01/03
3 2013/01/04
dtype: object
```

.dt 访问器还支持 period 与 timedelta。

```
period
In [280]: s = pd.Series(pd.period_range('20130101', periods=4, freq='D'))

In [281]: s
Out[281]:
0 2013-01-01
1 2013-01-02
2 2013-01-03
3 2013-01-04
dtype: period[D]

In [282]: s.dt.year
Out[282]:
0 2013
1 2013
2 2013
3 2013
dtype: int64

In [283]: s.dt.day
Out[283]:
0 1
1 2
2 3
3 4
dtype: int64
```

```
timedelta
In [284]: s = pd.Series(pd.timedelta_range('1 day 00:00:05', periods=4, freq='s'))

In [285]: s
Out[285]:
0 1 days 00:00:05
1 1 days 00:00:06
2 1 days 00:00:07
3 1 days 00:00:08
dtype: timedelta64[ns]

In [286]: s.dt.days
Out[286]:
0 1
1 1
2 1
3 1
dtype: int64

In [287]: s.dt.seconds
Out[287]:
0 5
1 6
2 7
3 8
dtype: int64

In [288]: s.dt.components
Out[288]:
 days hours minutes seconds milliseconds microseconds nanoseconds
0 1 0 0 5 0 0 0
1 1 0 0 6 0 0 0
2 1 0 0 7 0 0 0
3 1 0 0 8 0 0 0
```

::: tip 注意

用这个访问器处理不是 `datetime` 类型的值时，`Series.dt` 会触发 `TypeError` 错误。

:::

## 矢量化字符串方法

`Series` 支持字符串处理方法，可以非常方便地操作数组里的每个元素。这些方法会自动排除缺失值与空值，这也许是其最重要的特性。这些方法通过 `Series` 的 `str` 属性访问，一般情况下，这些操作的名称与内置的字符串方法一致。示例如下：

```
In [289]: s = pd.Series(['A', 'B', 'C', 'Aaba', 'Baca', np.nan, 'CABA', 'dog', 'cat'])

In [290]: s.str.lower()
Out[290]:
0 a
1 b
2 c
3 aaba
4 baca
5 NaN
6 caba
7 dog
8 cat
dtype: object
```

这里还提供了强大的模式匹配方法，但工业注意，模式匹配方法默认使用[正则表达式](#)。

参阅[矢量化字符串方法](#)，了解完整内容。

## 排序

---

Pandas 支持三种排序方式，按索引标签排序，按列里的值排序，按两种方式混合排序。

### 按索引排序

[Series.sort\\_index\(\)](#) 与 [DataFrame.sort\\_index\(\)](#) 方法用于按索引层级对 Pandas 对象排序。

```
In [291]: df = pd.DataFrame({
.....: 'one': pd.Series(np.random.randn(3), index=['a', 'b', 'c']),
.....: 'two': pd.Series(np.random.randn(4), index=['a', 'b', 'c', 'd']),
.....: 'three': pd.Series(np.random.randn(3), index=['b', 'c', 'd'])})
.....:
```

```
In [292]: unsorted_df = df.reindex(index=['a', 'd', 'c', 'b'],
.....: columns=['three', 'two', 'one'])
.....:
```

```
In [293]: unsorted_df
```

```
Out[293]:
```

	three	two	one
a	NaN	-1.152244	0.562973
d	-0.252916	-0.109597	NaN
c	1.273388	-0.167123	0.640382
b	-0.098217	0.009797	-1.299504

```
DataFrame
```

```
In [294]: unsorted_df.sort_index()
```

```
Out[294]:
```

	three	two	one
a	NaN	-1.152244	0.562973
b	-0.098217	0.009797	-1.299504
c	1.273388	-0.167123	0.640382
d	-0.252916	-0.109597	NaN

```
In [295]: unsorted_df.sort_index(ascending=False)
```

```
Out[295]:
```

	three	two	one
d	-0.252916	-0.109597	NaN
c	1.273388	-0.167123	0.640382
b	-0.098217	0.009797	-1.299504
a	NaN	-1.152244	0.562973

```
In [296]: unsorted_df.sort_index(axis=1)
```

```
Out[296]:
```

	one	three	two
a	0.562973	NaN	-1.152244
d	NaN	-0.252916	-0.109597
c	0.640382	1.273388	-0.167123
b	-1.299504	-0.098217	0.009797

```
Series
```

```
In [297]: unsorted_df['three'].sort_index()
```

```
Out[297]:
```

a	NaN
b	-0.098217
c	1.273388
d	-0.252916

```
Name: three, dtype: float64
```

## 按值排序

`Series.sort_values()` 方法用于按值对 Series 排序。`DataFrame.sort_values()` 方法用于按行列的值对 DataFrame 排序。`DataFrame.sort_values()` 的可选参数 `by` 用于指定按哪列排序，该参数的值可以是一列或多列数据。

```
In [298]: df1 = pd.DataFrame({'one': [2, 1, 1, 1],
.....: 'two': [1, 3, 2, 4],
.....: 'three': [5, 4, 3, 2]})
.....:
```

```
In [299]: df1.sort_values(by='two')
```

```
Out[299]:
```

	one	two	three
0	2	1	5
2	1	2	3
1	1	3	4
3	1	4	2

参数 `by` 支持列名列表，示例如下：

```
In [300]: df1[['one', 'two', 'three']].sort_values(by=['one', 'two'])
```

```
Out[300]:
```

	one	two	three
2	1	2	3
1	1	3	4
3	1	4	2
0	2	1	5

这些方法支持用 `na_position` 参数处理空值。

```
In [301]: s[2] = np.nan

In [302]: s.sort_values()
Out[302]:
0 A
3 Aaba
1 B
4 Baca
6 CABA
8 cat
7 dog
2 NaN
5 NaN
dtype: object

In [303]: s.sort_values(na_position='first')
Out[303]:
2 NaN
5 NaN
0 A
3 Aaba
1 B
4 Baca
6 CABA
8 cat
7 dog
dtype: object
```

## 按索引与值排序

0.23.0 版新增。

通过参数 `by` 传递给 `DataFrame.sort_values()` 的字符串可以引用列或索引层名。



```
创建 MultiIndex
In [304]: idx = pd.MultiIndex.from_tuples([('a', 1), ('a', 2), ('a', 2),
.....: ('b', 2), ('b', 1), ('b', 1)])
.....:

In [305]: idx.names = ['first', 'second']

创建 DataFrame
In [306]: df_multi = pd.DataFrame({'A': np.arange(6, 0, -1)},
.....: index=idx)
.....:

In [307]: df_multi
Out[307]:
```

		A
first	second	
a	1	6
	2	5
	2	4
b	2	3
	1	2
	1	1

按 `second`（索引）与 `A`（列）排序。

```
In [308]: df_multi.sort_values(by=['second', 'A'])
Out[308]:
```

		A
first	second	
b	1	1
	1	2
a	1	6
b	2	3
a	2	4
	2	5

::: tip 注意

字符串、列名、索引层名重名时，会触发警告提示，并以列名为准。后期版本中，这种情况将会触发模糊错误。

:::

## 搜索排序

Series 支持 `searchsorted()` 方法，这与 `numpy.ndarray.searchsorted()` 的操作方式类似。

```
In [309]: ser = pd.Series([1, 2, 3])

In [310]: ser.searchsorted([0, 3])
Out[310]: array([0, 2])

In [311]: ser.searchsorted([0, 4])
Out[311]: array([0, 3])

In [312]: ser.searchsorted([1, 3], side='right')
Out[312]: array([1, 3])

In [313]: ser.searchsorted([1, 3], side='left')
Out[313]: array([0, 2])

In [314]: ser = pd.Series([3, 1, 2])

In [315]: ser.searchsorted([0, 3], sorter=np.argsort(ser))
Out[315]: array([0, 2])
```

## 最大值与最小值

Series 支持 `nsmallest()` 与 `nlargest()` 方法，本方法返回 N 个最大或最小的值。对于数据量大的 Series 来说，该方法比先为整个 Series 排序，再调用 `head(n)` 这种方式的速度要快得多。

```
In [316]: s = pd.Series(np.random.permutation(10))
```

```
In [317]: s
```

```
Out[317]:
```

```
0 2
1 0
2 3
3 7
4 1
5 5
6 9
7 6
8 8
9 4
```

```
dtype: int64
```

```
In [318]: s.sort_values()
```

```
Out[318]:
```

```
1 0
4 1
0 2
2 3
9 4
5 5
7 6
3 7
8 8
6 9
```

```
dtype: int64
```

```
In [319]: s.nsmallest(3)
```

```
Out[319]:
```

```
1 0
4 1
0 2
```

```
dtype: int64
```

```
In [320]: s.nlargest(3)
```

```
Out[320]:
```

```
6 9
8 8
3 7
```

```
dtype: int64
```

`DataFrame` 也支持 `nlargest` 与 `nsmallest` 方法。

```
In [321]: df = pd.DataFrame({'a': [-2, -1, 1, 10, 8, 11, -1],
.....: 'b': list('abdceff'),
.....: 'c': [1.0, 2.0, 4.0, 3.2, np.nan, 3.0, 4.0]})
.....:
```

```
In [322]: df.nlargest(3, 'a')
```

```
Out[322]:
```

	a	b	c
5	11	f	3.0
3	10	c	3.2
4	8	e	NaN

```
In [323]: df.nlargest(5, ['a', 'c'])
```

```
Out[323]:
```

	a	b	c
5	11	f	3.0
3	10	c	3.2
4	8	e	NaN
2	1	d	4.0
6	-1	f	4.0

```
In [324]: df.nsmallest(3, 'a')
```

```
Out[324]:
```

	a	b	c
0	-2	a	1.0
1	-1	b	2.0
6	-1	f	4.0

```
In [325]: df.nsmallest(5, ['a', 'c'])
```

```
Out[325]:
```

	a	b	c
0	-2	a	1.0
1	-1	b	2.0
6	-1	f	4.0
2	1	d	4.0
4	8	e	NaN

## 用多层索引的列排序

列为多层索引时，可以显式排序，用 `by` 指定所有层级。

```
In [326]: df1.columns = pd.MultiIndex.from_tuples([('a', 'one'),
.....: ('a', 'two'),
.....: ('b', 'three')])
.....:
```

```
In [327]: df1.sort_values(by=('a', 'two'))
```

```
Out[327]:
 a b
one two three
0 2 1 5
2 1 2 3
1 1 3 4
3 1 4 2
```

## 复制

在 Pandas 对象上执行 `copy()` 方法，将复制底层数据（但不包括轴索引，因为轴索引不可变），并返回一个新的对象。注意，**复制对象这种操作一般来说不是必须的**。比如说，以下几种方式可以**就地 (inplace)** 改变 DataFrame：

- 插入、删除、修改列
- 为 `index` 或 `columns` 属性赋值
- 对于同质数据，用 `values` 属性或高级索引即可直接修改值

注意，用 Pandas 方法修改数据不会带来任何副作用，几乎所有方法都返回新的对象，不会修改原始数据对象。如果原始数据有所改动，唯一的可能就是用户显式指定了要修改原始数据。

## 数据类型

大多数情况下，Pandas 使用 NumPy 数组、Series 或 DataFrame 里某列的数据类型。NumPy 支持 `float`、`int`、`bool`、`timedelta[ns]`、`datetime64[ns]`，注意，NumPy 不支持带时区信息的 `datetime`。

Pandas 与第三方支持库扩充了 NumPy 类型系统，本节只介绍 Pandas 的内部扩展。如需了解如何编写与 Pandas 扩展类型，请参阅[扩展类型](#)，参阅[扩展数据类型](#)了解第三方支持库提供的扩展类型。

下表列出了 Pandas 扩展类型，参阅列出的文档内容，查看每种类型的详细说明。

数据种类	数据类型	标量	数组	文档
tz-aware datetime	<a href="#">DatetimeTZDtype</a>	<a href="#">Timestamp</a>	<a href="#">arrays.DatetimeArray</a>	<a href="#">Time zone handling</a>
Categorical	<a href="#">CategoricalDtype</a>	(无)	<a href="#">Categorical</a>	<a href="#">Categorical data</a>
period (time spans)	<a href="#">PeriodDtype</a>	<a href="#">Period</a>	<a href="#">arrays.PeriodArray</a>	<a href="#">Time span representation</a>
sparse	<a href="#">SparseDtype</a>	(无)	<a href="#">arrays.SparseArray</a>	<a href="#">Sparse data structures</a>
intervals	<a href="#">IntervalDtype</a>	<a href="#">Interval</a>	<a href="#">arrays.IntervalArray</a>	<a href="#">IntervalIndex</a>
nullable integer	<a href="#">Int64Dtype</a> , ...	(无)	<a href="#">arrays.IntegerArray</a>	<a href="#">Nullable integer data type</a>

Pandas 用 `object` 存储字符串。

虽然，`object` 数据类型能够存储任何对象，但应尽量避免这种操作，要了解与其它支持库与方法的性能与交互操作，参阅 [对象转换](#)。

`DataFrame` 的 `dtypes` 属性用起来很方便，以 `Series` 形式返回每列的数据类型。

```
In [328]: dft = pd.DataFrame({'A': np.random.rand(3),
.....: 'B': 1,
.....: 'C': 'foo',
.....: 'D': pd.Timestamp('20010102'),
.....: 'E': pd.Series([1.0] * 3).astype('float32'),
.....: 'F': False,
.....: 'G': pd.Series([1] * 3, dtype='int8')})
.....:
```

```
In [329]: dft
```

```
Out[329]:
```

	A	B	C	D	E	F	G
0	0.035962	1	foo	2001-01-02	1.0	False	1
1	0.701379	1	foo	2001-01-02	1.0	False	1
2	0.281885	1	foo	2001-01-02	1.0	False	1

```
In [330]: dft.dtypes
```

```
Out[330]:
```

A	float64
B	int64
C	object
D	datetime64[ns]
E	float32
F	bool
G	int8

dtype: object

要查看 `Series` 的数据类型，用 `dtype` 属性。

```
In [331]: dft['A'].dtype
Out[331]: dtype('float64')
```

Pandas 对象单列中含多种类型的数据时，该列的数据类型为可适配于各类数据的数据类型，通常为 `object`。

```
整数被强制转换为浮点数
In [332]: pd.Series([1, 2, 3, 4, 5, 6.])
Out[332]:
0 1.0
1 2.0
2 3.0
3 4.0
4 5.0
5 6.0
dtype: float64

字符串数据决定了该 Series 的数据类型为 ``object``
In [333]: pd.Series([1, 2, 3, 6., 'foo'])
Out[333]:
0 1
1 2
2 3
3 6
4 foo
dtype: object
```

`DataFrame.dtypes.value_counts()` 用于统计 DataFrame 里不同数据类型的列数。

```
In [334]: dft.dtypes.value_counts()
Out[334]:
float32 1
object 1
bool 1
int8 1
float64 1
datetime64[ns] 1
int64 1
dtype: int64
```

多种数值型数据类型可以在 DataFrame 里共存。如果只传递一种数据类型，不论是通过 `dtype` 关键字直接传递，还是通过 `ndarray` 或 `Series` 传递，都会保存至 DataFrame 操作。此外，不同数值型数据类型**不会**合并。示例如下：

```
In [335]: df1 = pd.DataFrame(np.random.randn(8, 1), columns=['A'], dtype='float32')
```

```
In [336]: df1
```

```
Out[336]:
```

```
 A
0 0.224364
1 1.890546
2 0.182879
3 0.787847
4 -0.188449
5 0.667715
6 -0.011736
7 -0.399073
```

```
In [337]: df1.dtypes
```

```
Out[337]:
```

```
A float32
dtype: object
```

```
In [338]: df2 = pd.DataFrame({'A': pd.Series(np.random.randn(8), dtype='float16'),
.....: 'B': pd.Series(np.random.randn(8)),
.....: 'C': pd.Series(np.array(np.random.randn(8),
.....: dtype='uint8'))})
```

```
In [339]: df2
```

```
Out[339]:
```

```
 A B C
0 0.823242 0.256090 0
1 1.607422 1.426469 0
2 -0.333740 -0.416203 255
3 -0.063477 1.139976 0
4 -1.014648 -1.193477 0
5 0.678711 0.096706 0
6 -0.040863 -1.956850 1
7 -0.357422 -0.714337 0
```

```
In [340]: df2.dtypes
```

```
Out[340]:
```

```
A float16
B float64
C uint8
dtype: object
```

## 默认值

整数的默认类型为 `int64`，浮点数的默认类型为 `float64`，这里的默认值与系统平台无关，不管是 32 位系统，还是 64 位系统都是一样的。下列代码返回的结果都是 `int64`：



```

In [341]: pd.DataFrame([1, 2], columns=['a']).dtypes
Out[341]:
a int64
dtype: object

In [342]: pd.DataFrame({'a': [1, 2]}).dtypes
Out[342]:
a int64
dtype: object

In [343]: pd.DataFrame({'a': 1}, index=list(range(2))).dtypes
Out[343]:
a int64
dtype: object

```

注意，NumPy 创建数组时，会根据系统选择类型。下列代码在 32 位系统上**将**返回 `int32`。

```

In [344]: frame = pd.DataFrame(np.array([1, 2]))

```

## 向上转型

与其它类型合并时，用的是向上转型，指的是从现有类型转换为另一种类型，如 `int` 变为 `float`。

```

In [345]: df3 = df1.reindex_like(df2).fillna(value=0.0) + df2

In [346]: df3
Out[346]:
 A B C
0 1.047606 0.256090 0.0
1 3.497968 1.426469 0.0
2 -0.150862 -0.416203 255.0
3 0.724370 1.139976 0.0
4 -1.203098 -1.193477 0.0
5 1.346426 0.096706 0.0
6 -0.052599 -1.956850 1.0
7 -0.756495 -0.714337 0.0

In [347]: df3.dtypes
Out[347]:
A float32
B float64
C float64
dtype: object

```

`DataFrame.to_numpy()` 返回多个数据类型里**用得最多的数据类型**，这里指的是，输出结果的数据类型，适用于所有同构 NumPy 数组的数据类型。此处强制执行**向上转型**。

```

In [348]: df3.to_numpy().dtype
Out[348]: dtype('float64')

```

## astype

`astype()` 方法显式地把一种数据类型转换为另一种，默认操作为复制数据，就算数据类型没有改变也会复制数据，`copy=False` 改变默认操作模式。此外，`astype` 无效时，会触发异常。

向上转型一般都遵循 **NumPy** 规则。操作中含有两种不同类型的数据时，返回更为通用的那种数据类型。

```
In [349]: df3
Out[349]:
```

	A	B	C
0	1.047606	0.256090	0.0
1	3.497968	1.426469	0.0
2	-0.150862	-0.416203	255.0
3	0.724370	1.139976	0.0
4	-1.203098	-1.193477	0.0
5	1.346426	0.096706	0.0
6	-0.052599	-1.956850	1.0
7	-0.756495	-0.714337	0.0

```
In [350]: df3.dtypes
Out[350]:
A float32
B float64
C float64
dtype: object

转换数据类型
In [351]: df3.astype('float32').dtypes
Out[351]:
A float32
B float32
C float32
dtype: object
```

用 `astype()` 把一列或多列转换为指定类型。

```
In [352]: dft = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6], 'c': [7, 8, 9]})

In [353]: dft[['a', 'b']] = dft[['a', 'b']].astype(np.uint8)

In [354]: dft
Out[354]:
 a b c
0 1 4 7
1 2 5 8
2 3 6 9

In [355]: dft.dtypes
Out[355]:
a uint8
b uint8
c int64
dtype: object
```

0.19.0 版新增。

`astype()` 通过字典指定哪些列转换为哪些类型。

```
In [356]: dft1 = pd.DataFrame({'a': [1, 0, 1], 'b': [4, 5, 6], 'c': [7, 8, 9]})

In [357]: dft1 = dft1.astype({'a': np.bool, 'c': np.float64})

In [358]: dft1
Out[358]:
 a b c
0 True 4 7.0
1 False 5 8.0
2 True 6 9.0

In [359]: dft1.dtypes
Out[359]:
a bool
b int64
c float64
dtype: object
```

::: tip 注意

用 `astype()` 与 `loc()` 为部分列转换指定类型时，会发生向上转型。

`loc()` 尝试分配当前的数据类型，而 `[]` 则会从右方获取数据类型并进行覆盖。因此，下列代码会产出意料之外的结果：

```

In [360]: dft = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6], 'c': [7, 8, 9]})

In [361]: dft.loc[:, ['a', 'b']].astype(np.uint8).dtypes
Out[361]:
a uint8
b uint8
dtype: object

In [362]: dft.loc[:, ['a', 'b']] = dft.loc[:, ['a', 'b']].astype(np.uint8)

In [363]: dft.dtypes
Out[363]:
a int64
b int64
c int64
dtype: object

```

...

## 对象转换

Pandas 提供了多种函数可以把 `object` 从一种类型强制转为另一种类型。这是因为，数据有时存储的是正确类型，但在保存时却存成了 `object` 类型，此时，用 `DataFrame.infer_objects()` 与 `Series.infer_objects()` 方法即可把数据软转换为正确的类型。

```

In [364]: import datetime

In [365]: df = pd.DataFrame([[1, 2],
.....: ['a', 'b'],
.....: [datetime.datetime(2016, 3, 2),
.....: datetime.datetime(2016, 3, 2)]]))

In [366]: df = df.T

In [367]: df
Out[367]:
 0 1 2
0 1 a 2016-03-02
1 2 b 2016-03-02

In [368]: df.dtypes
Out[368]:
0 object
1 object
2 datetime64[ns]
dtype: object

```

因为数据被转置，所以把原始列的数据类型改成了 `object`，但使用 `infer_objects` 后就变正确了。

```
In [369]: df.infer_objects().dtypes
Out[369]:
0 int64
1 object
2 datetime64[ns]
dtype: object
```

下列函数可以应用于一维数组与标量，执行硬转换，把对象转换为指定类型。

- `to_numeric()`，转换为数值型

```
In [370]: m = ['1.1', 2, 3]

In [371]: pd.to_numeric(m)
Out[371]: array([1.1, 2. , 3.])
```

- `to_datetime()`，转换为 `datetime` 对象

```
In [372]: import datetime

In [373]: m = ['2016-07-09', datetime.datetime(2016, 3, 2)]

In [374]: pd.to_datetime(m)
Out[374]: DatetimeIndex(['2016-07-09', '2016-03-02'], dtype='datetime64[ns]', freq=None)
```

- `to_timedelta()`，转换为 `timedelta` 对象。

```
In [375]: m = ['5us', pd.Timedelta('1day')]

In [376]: pd.to_timedelta(m)
Out[376]: TimedeltaIndex(['0 days 00:00:00.000005', '1 days 00:00:00'], dtype='timedelta64[ns]',
 freq=None)
```

如需强制转换，则要加入 `error` 参数，指定 Pandas 怎样处理不能转换为预期类型或对象的数据。`errors` 参数的默认值为 `False`，指的是在转换过程中，遇到任何问题都触发错误。设置为 `errors='coerce'` 时，pandas 会忽略错误，强制把问题数据转换为 `pd.NaT`（`datetime` 与 `timedelta`），或 `np.nan`（数值型）。读取数据时，如果大部分要转换的数据是数值型或 `datetime`，这种操作非常有用，但偶尔也会有非制式数据混合在一起，可能会导致展示数据缺失：

```

In [377]: import datetime

In [378]: m = ['apple', datetime.datetime(2016, 3, 2)]

In [379]: pd.to_datetime(m, errors='coerce')
Out[379]: DatetimeIndex(['NaT', '2016-03-02'], dtype='datetime64[ns]', freq=None)

In [380]: m = ['apple', 2, 3]

In [381]: pd.to_numeric(m, errors='coerce')
Out[381]: array([nan, 2., 3.])

In [382]: m = ['apple', pd.Timedelta('1day')]

In [383]: pd.to_timedelta(m, errors='coerce')
Out[383]: TimedeltaIndex([NaT, '1 days'], dtype='timedelta64[ns]', freq=None)

```

`error` 参数还有第三个选项, `error='ignore'`。转换数据时会忽略错误, 直接输出问题数据:

```

In [384]: import datetime

In [385]: m = ['apple', datetime.datetime(2016, 3, 2)]

In [386]: pd.to_datetime(m, errors='ignore')
Out[386]: Index(['apple', '2016-03-02 00:00:00'], dtype='object')

In [387]: m = ['apple', 2, 3]

In [388]: pd.to_numeric(m, errors='ignore')
Out[388]: array(['apple', 2, 3], dtype=object)

In [389]: m = ['apple', pd.Timedelta('1day')]

In [390]: pd.to_timedelta(m, errors='ignore')
Out[390]: array(['apple', Timedelta('1 days 00:00:00')], dtype=object)

```

执行转换操作时, `to_numeric()` 还有一个参数, `downcast`, 即向下转型, 可以把数值型转换为减少内存占用的数据类型:

```

In [391]: m = ['1', 2, 3]

In [392]: pd.to_numeric(m, downcast='integer') # smallest signed int dtype
Out[392]: array([1, 2, 3], dtype=int8)

In [393]: pd.to_numeric(m, downcast='signed') # same as 'integer'
Out[393]: array([1, 2, 3], dtype=int8)

In [394]: pd.to_numeric(m, downcast='unsigned') # smallest unsigned int dtype
Out[394]: array([1, 2, 3], dtype=uint8)

In [395]: pd.to_numeric(m, downcast='float') # smallest float dtype
Out[395]: array([1., 2., 3.], dtype=float32)

```

上述方法仅能应用于一维数组、列表或标量；不能直接用于 DataFrame 等多维对象。不过，用 `apply()`，可以快速为每列应用函数：

```
In [396]: import datetime

In [397]: df = pd.DataFrame([
.....: ['2016-07-09', datetime.datetime(2016, 3, 2)] * 2, dtype='O')
.....:

In [398]: df
Out[398]:
 0 1
0 2016-07-09 2016-03-02 00:00:00
1 2016-07-09 2016-03-02 00:00:00

In [399]: df.apply(pd.to_datetime)
Out[399]:
 0 1
0 2016-07-09 2016-03-02
1 2016-07-09 2016-03-02

In [400]: df = pd.DataFrame([['1.1', 2, 3]] * 2, dtype='O')

In [401]: df
Out[401]:
 0 1 2
0 1.1 2 3
1 1.1 2 3

In [402]: df.apply(pd.to_numeric)
Out[402]:
 0 1 2
0 1.1 2 3
1 1.1 2 3

In [403]: df = pd.DataFrame([['5us', pd.Timedelta('1day')]] * 2, dtype='O')

In [404]: df
Out[404]:
 0 1
0 5us 1 days 00:00:00
1 5us 1 days 00:00:00

In [405]: df.apply(pd.to_timedelta)
Out[405]:
 0 1
0 00:00:00.000005 1 days
1 00:00:00.000005 1 days
```

## 各种坑

对 `integer` 数据执行选择操作时，可以很轻而易举地把数据转换为 `floating`。Pandas 会保存输入数据的数据类型，以防未引入 `nan`s 的情况。参阅 [对整数 NA 空值的支持](#)。

```
In [406]: dfi = df3.astype('int32')
```

```
In [407]: dfi['E'] = 1
```

```
In [408]: dfi
```

```
Out[408]:
```

	A	B	C	E
0	1	0	0	1
1	3	1	0	1
2	0	0	255	1
3	0	1	0	1
4	-1	-1	0	1
5	1	0	0	1
6	0	-1	1	1
7	0	0	0	1

```
In [409]: dfi.dtypes
```

```
Out[409]:
```

```
A int32
```

```
B int32
```

```
C int32
```

```
E int64
```

```
dtype: object
```

```
In [410]: casted = dfi[dfi > 0]
```

```
In [411]: casted
```

```
Out[411]:
```

	A	B	C	E
0	1.0	NaN	NaN	1
1	3.0	1.0	NaN	1
2	NaN	NaN	255.0	1
3	NaN	1.0	NaN	1
4	NaN	NaN	NaN	1
5	1.0	NaN	NaN	1
6	NaN	NaN	1.0	1
7	NaN	NaN	NaN	1

```
In [412]: casted.dtypes
```

```
Out[412]:
```

```
A float64
```

```
B float64
```

```
C float64
```

```
E int64
```

```
dtype: object
```

浮点数类型未改变。



```

In [413]: dfa = df3.copy()

In [414]: dfa['A'] = dfa['A'].astype('float32')

In [415]: dfa.dtypes
Out[415]:
A float32
B float64
C float64
dtype: object

In [416]: casted = dfa[df2 > 0]

In [417]: casted
Out[417]:
 A B C
0 1.047606 0.256090 NaN
1 3.497968 1.426469 NaN
2 NaN NaN 255.0
3 NaN 1.139976 NaN
4 NaN NaN NaN
5 1.346426 0.096706 NaN
6 NaN NaN 1.0
7 NaN NaN NaN

In [418]: casted.dtypes
Out[418]:
A float32
B float64
C float64
dtype: object

```

## 基于 dtype 选择列

`select_dtypes()` 方法基于 `dtype` 选择列。

首先，创建一个由多种数据类型组成的 DataFrame：

```

In [419]: df = pd.DataFrame({'string': list('abc'),
.....: 'int64': list(range(1, 4)),
.....: 'uint8': np.arange(3, 6).astype('u1'),
.....: 'float64': np.arange(4.0, 7.0),
.....: 'bool1': [True, False, True],
.....: 'bool2': [False, True, False],
.....: 'dates': pd.date_range('now', periods=3),
.....: 'category': pd.Series(list("ABC")).astype('category')})
.....:

In [420]: df['tdeltas'] = df.dates.diff()

In [421]: df['uint64'] = np.arange(3, 6).astype('u8')

In [422]: df['other_dates'] = pd.date_range('20130101', periods=3)

In [423]: df['tz_aware_dates'] = pd.date_range('20130101', periods=3, tz='US/Eastern')

In [424]: df
Out[424]:
 string int64 uint8 float64 bool1 bool2 dates category tdeltas uint64
0 a 1 3 4.0 True False 2019-08-22 15:49:01.870038 A NaT 3
 2013-01-01 2013-01-01 00:00:00-05:00
1 b 2 4 5.0 False True 2019-08-23 15:49:01.870038 B 1 days 4
 2013-01-02 2013-01-02 00:00:00-05:00
2 c 3 5 6.0 True False 2019-08-24 15:49:01.870038 C 1 days 5
 2013-01-03 2013-01-03 00:00:00-05:00

```

该 DataFrame 的数据类型：

```

In [425]: df.dtypes
Out[425]:
string object
int64 int64
uint8 uint8
float64 float64
bool1 bool
bool2 bool
dates datetime64[ns]
category category
tdeltas timedelta64[ns]
uint64 uint64
other_dates datetime64[ns]
tz_aware_dates datetime64[ns, US/Eastern]
dtype: object

```

`select_dtypes()` 有两个参数，`include` 与 `exclude`，用于实现“提取这些数据类型的列”（`include`）或“提取不是这些数据类型的列”（`exclude`）。

选择 `bool` 型的列，示例如下：

```
In [426]: df.select_dtypes(include=[bool])
Out[426]:
 bool1 bool2
0 True False
1 False True
2 True False
```

该方法还支持输入 NumPy 数据类型的名称：

```
In [427]: df.select_dtypes(include=['bool'])
Out[427]:
 bool1 bool2
0 True False
1 False True
2 True False
```

`select_dtypes()` 还支持通用数据类型。

比如，选择所有数值型与布尔型的列，同时，排除无符号整数：

```
In [428]: df.select_dtypes(include=['number', 'bool'], exclude=['unsignedinteger'])
Out[428]:
 int64 float64 bool1 bool2 tdelatas
0 1 4.0 True False NaT
1 2 5.0 False True 1 days
2 3 6.0 True False 1 days
```

选择字符串型的列必须要用 `object`：

```
In [429]: df.select_dtypes(include=['object'])
Out[429]:
 string
0 a
1 b
2 c
```

要查看 `numpy.number` 等通用 `dtype` 的所有子类型，可以定义一个函数，返回子类型树：

```
In [430]: def subdtypes(dtype):
.....: subs = dtype.__subclasses__()
.....: if not subs:
.....: return dtype
.....: return [dtype, [subdtypes(dt) for dt in subs]]
.....:
```

所有 NumPy 数据类型都是 `numpy.generic` 的子类：

```

In [431]: subdtypes(np.generic)
Out[431]:
[numpy.generic,
 [[numpy.number,
 [[numpy.integer,
 [[numpy.signedinteger,
 [numpy.int8,
 numpy.int16,
 numpy.int32,
 numpy.int64,
 numpy.int64,
 numpy.timedelta64]],
 numpy.unsignedinteger,
 [numpy.uint8,
 numpy.uint16,
 numpy.uint32,
 numpy.uint64,
 numpy.uint64]]]],
 [numpy.inexact,
 [[numpy.floating,
 [numpy.float16, numpy.float32, numpy.float64, numpy.float128]],
 [numpy.complexfloating,
 [numpy.complex64, numpy.complex128, numpy.complex256]]]]],
 [numpy.flexible,
 [[numpy.character, [numpy.bytes_, numpy.str_]],
 [numpy.void, [numpy.record]]]],
 numpy.bool_,
 numpy.datetime64,
 numpy.object_]]

```

⋮ tip 注意

Pandas 支持 `category` 与 `datetime64[ns, tz]` 类型，但这两种类型未整合到 NumPy 架构，因此，上面的函数没有显示。

⋮

## 数据结构简介

本节介绍 Pandas 基础数据结构，包括各类对象的数据类型、索引、轴标记、对齐等基础操作。首先，导入 NumPy 和 Pandas：

```

In [1]: import numpy as np

In [2]: import pandas as pd

```

“数据对齐是内在的”，这一原则是根本。除非显式指定，Pandas 不会断开标签和数据之间的连接。

下文先简单介绍数据结构，然后再分门别类介绍每种功能与方法。

# Series

**Series** 是带标签的一维数组，可存储整数、浮点数、字符串、Python 对象等类型的数据。轴标签统称为**索引**。调用 `pd.Series` 函数即可创建 Series：

```
>>> s = pd.Series(data, index=index)
```

上述代码中，`data` 支持以下数据类型：

- Python 字典
- 多维数组
- 标量值（如，5）

`index` 是轴标签列表。不同**数据**可分为以下几种情况：

## 多维数组

`data` 是多维数组时，**index** 长度必须与 **data** 长度一致。没有指定 `index` 参数时，创建数值型索引，即 `[0, ..., len(data) - 1]`。

```
In [3]: s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])
```

```
In [4]: s
```

```
Out[4]:
```

```
a 0.469112
```

```
b -0.282863
```

```
c -1.509059
```

```
d -1.135632
```

```
e 1.212112
```

```
dtype: float64
```

```
In [5]: s.index
```

```
Out[5]: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')
```

```
In [6]: pd.Series(np.random.randn(5))
```

```
Out[6]:
```

```
0 -0.173215
```

```
1 0.119209
```

```
2 -1.044236
```

```
3 -0.861849
```

```
4 -2.104569
```

```
dtype: float64
```

⋮ tip 注意

Pandas 的索引值可以重复。不支持重复索引值的操作会触发异常。其原因主要与性能有关，有很多计算实例，比如 GroupBy 操作就不用索引。

⋮

## 字典

Series 可以用字典实例化：

```
In [7]: d = {'b': 1, 'a': 0, 'c': 2}

In [8]: pd.Series(d)
Out[8]:
b 1
a 0
c 2
dtype: int64
```

::: tip 注意

`data` 为字典，且未设置 `index` 参数时，如果 Python 版本  $\geq 3.6$  且 Pandas 版本  $\geq 0.23$ ，Series 按字典的插入顺序排序索引。

Python  $< 3.6$  或 Pandas  $< 0.23$ ，且未设置 `index` 参数时，Series 按字母顺序排序字典的键（key）列表。

:::

上例中，如果 Python  $< 3.6$  或 Pandas  $< 0.23$ ，Series 按字母排序字典的键。输出结果不是 `['b', 'a', 'c']`，而是 `['a', 'b', 'c']`。

如果设置了 `index` 参数，则按索引标签提取 `data` 里对应的值。

```
In [9]: d = {'a': 0., 'b': 1., 'c': 2.}

In [10]: pd.Series(d)
Out[10]:
a 0.0
b 1.0
c 2.0
dtype: float64

In [11]: pd.Series(d, index=['b', 'c', 'd', 'a'])
Out[11]:
b 1.0
c 2.0
d NaN
a 0.0
dtype: float64
```

::: tip 注意

Pandas 用 `NaN`（Not a Number）表示**缺失数据**。

:::

## 标量值

`data` 是标量值时，必须提供索引。`Series` 按索引长度重复该标量值。

```
In [12]: pd.Series(5., index=['a', 'b', 'c', 'd', 'e'])
Out[12]:
a 5.0
b 5.0
c 5.0
d 5.0
e 5.0
dtype: float64
```

## Series 类似多维数组

`Series` 操作与 `ndarray` 类似，支持大多数 NumPy 函数，还支持索引切片。

```
In [13]: s[0]
Out[13]: 0.4691122999071863

In [14]: s[:3]
Out[14]:
a 0.469112
b -0.282863
c -1.509059
dtype: float64

In [15]: s[s > s.median()]
Out[15]:
a 0.469112
e 1.212112
dtype: float64

In [16]: s[[4, 3, 1]]
Out[16]:
e 1.212112
d -1.135632
b -0.282863
dtype: float64

In [17]: np.exp(s)
Out[17]:
a 1.598575
b 0.753623
c 0.221118
d 0.321219
e 3.360575
dtype: float64
```

⋮ tip 注意

[索引与选择数据](#)一节介绍了 `s[[4, 3, 1]]` 等数组索引操作。

⋮

和 NumPy 数组一样，Series 也支持 `dtype`。

```
In [18]: s.dtype
Out[18]: dtype('float64')
```

Series 的数据类型一般是 NumPy 数据类型。不过，Pandas 和第三方库在一些方面扩展了 NumPy 类型系统，即[扩展数据类型](#)。比如，Pandas 的[类别型数据](#)与[可空整数数据类型](#)。更多信息，请参阅[数据类型](#)。

`Series.array` 用于提取 Series 数组。

```
In [19]: s.array
Out[19]:
<PandasArray>
[0.4691122999071863, -0.2828633443286633, -1.5090585031735124,
 -1.1356323710171934, 1.2121120250208506]
Length: 5, dtype: float64
```

执行不用索引的操作时，如禁用[自动对齐](#)，访问数组非常有用。

`Series.array` 一般是[扩展数组](#)。简单说，扩展数组是把 N 个 `numpy.ndarray` 包在一起的打包器。Pandas 知道怎么把扩展数组存储到 Series 或 DataFrame 的列里。更多信息，请参阅[数据类型](#)。

Series 只是类似于多维数组，提取[真正](#)的多维数组，要用 `Series.to_numpy()`。

```
In [20]: s.to_numpy()
Out[20]: array([0.4691, -0.2829, -1.5091, -1.1356, 1.2121])
```

Series 是[扩展数组](#)，`Series.to_numpy()` 返回的是 NumPy 多维数组。

## Series 类似字典

Series 类似固定大小的字典，可以用索引标签提取值或设置值：



```
In [21]: s['a']
Out[21]: 0.4691122999071863

In [22]: s['e'] = 12.

In [23]: s
Out[23]:
a 0.469112
b -0.282863
c -1.509059
d -1.135632
e 12.000000
dtype: float64

In [24]: 'e' in s
Out[24]: True

In [25]: 'f' in s
Out[25]: False
```

引用 `Series` 里没有的标签会触发异常：

```
>>> s['f']
KeyError: 'f'
```

`get` 方法可以提取 `Series` 里没有的标签，返回 `None` 或指定默认值：

```
In [26]: s.get('f')

In [27]: s.get('f', np.nan)
Out[27]: nan
```

更多信息，请参阅[属性访问](#)。

## 矢量操作与对齐 `Series` 标签

`Series` 和 NumPy 数组一样，都不用循环每个值，而且 `Series` 支持大多数 NumPy 多维数组的方法。

```

In [28]: s + s
Out[28]:
a 0.938225
b -0.565727
c -3.018117
d -2.271265
e 24.000000
dtype: float64

In [29]: s * 2
Out[29]:
a 0.938225
b -0.565727
c -3.018117
d -2.271265
e 24.000000
dtype: float64

In [30]: np.exp(s)
Out[30]:
a 1.598575
b 0.753623
c 0.221118
d 0.321219
e 162754.791419
dtype: float64

```

Series 和 多维数组的主要区别在于，Series 之间的操作会自动基于标签对齐数据。因此，不用顾及执行计算操作的 Series 是否有相同的标签。

```

In [31]: s[1:] + s[:-1]
Out[31]:
a NaN
b -0.565727
c -3.018117
d -2.271265
e NaN
dtype: float64

```

操作未对齐索引的 Series，其计算结果是所有涉及索引的**并集**。如果在 Series 里找不到标签，运算结果标记为 `NaN`，即缺失值。编写无需显式对齐数据的代码，给交互数据分析和研究提供了巨大的自由度和灵活性。Pandas 数据结构集成的数据对齐功能，是 Pandas 区别于大多数标签型数据处理工具的重要特性。

⋮ tip 注意

总之，让不同索引对象操作的默认结果生成索引**并集**，是为了避免信息丢失。就算缺失了数据，索引标签依然包含计算的重要信息。当然，也可以用 `** dropna **` 函数清除含有缺失值的标签。

⋮

## 名称属性

Series 支持 `name` 属性：

```
In [32]: s = pd.Series(np.random.randn(5), name='something')
```

```
In [33]: s
```

```
Out[33]:
```

```
0 -0.494929
```

```
1 1.071804
```

```
2 0.721555
```

```
3 -0.706771
```

```
4 -1.039575
```

```
Name: something, dtype: float64
```

```
In [34]: s.name
```

```
Out[34]: 'something'
```

一般情况下，Series 自动分配 `name`，特别是提取一维 DataFrame 切片时，详见下文。

0.18.0 版新增。

`pandas.Series.rename()` 方法用于重命名 Series。

```
In [35]: s2 = s.rename("different")
```

```
In [36]: s2.name
```

```
Out[36]: 'different'
```

注意，`s` 与 `s2` 指向不同的对象。

## DataFrame

**DataFrame** 是由多种类型的列构成的二维标签数据结构，类似于 Excel、SQL 表，或 Series 对象构成的字典。DataFrame 是最常用的 Pandas 对象，与 Series 一样，DataFrame 支持多种类型的输入数据：

- 一维 ndarray、列表、字典、Series 字典
- 二维 numpy.ndarray
- [结构多维数组或记录多维数组](#)
- Series
- DataFrame

除了数据，还可以有选择地传递 **index**（行标签）和 **columns**（列标签）参数。传递了索引或列，就可以确保生成的 DataFrame 里包含索引或列。Series 字典加上指定索引时，会丢弃与传递的索引不匹配的所有数据。

没有传递轴标签时，按常规依据输入数据进行构建。

⋮ tip 注意

Python  $\geq 3.6$ , 且 Pandas  $\geq 0.23$ , 数据是字典, 且未指定 `columns` 参数时, `DataFrame` 的列按字典的插入顺序排序。

Python  $< 3.6$  或 Pandas  $< 0.23$ , 且未指定 `columns` 参数时, `DataFrame` 的列按字典键的字母排序。

⋮

## 用 Series 字典或字典生成 DataFrame

生成的索引是每个 **Series** 索引的并集。先把嵌套字典转换为 Series。如果没有指定列, `DataFrame` 的列就是字典键的有序列表。

```
In [37]: d = {'one': pd.Series([1., 2., 3.], index=['a', 'b', 'c']),
.....: 'two': pd.Series([1., 2., 3., 4.], index=['a', 'b', 'c', 'd'])}
.....:
```

```
In [38]: df = pd.DataFrame(d)
```

```
In [39]: df
```

```
Out[39]:
```

	one	two
a	1.0	1.0
b	2.0	2.0
c	3.0	3.0
d	NaN	4.0

```
In [40]: pd.DataFrame(d, index=['d', 'b', 'a'])
```

```
Out[40]:
```

	one	two
d	NaN	4.0
b	2.0	2.0
a	1.0	1.0

```
In [41]: pd.DataFrame(d, index=['d', 'b', 'a'], columns=['two', 'three'])
```

```
Out[41]:
```

	two	three
d	4.0	NaN
b	2.0	NaN
a	1.0	NaN

**index** 和 **columns** 属性分别用于访问行、列标签：

⋮ tip 注意

指定列与数据字典一起传递时, 传递的列会覆盖字典的键。

⋮

```
In [42]: df.index
Out[42]: Index(['a', 'b', 'c', 'd'], dtype='object')

In [43]: df.columns
Out[43]: Index(['one', 'two'], dtype='object')
```

## 用多维数组字典、列表字典生成 DataFrame

多维数组的长度必须相同。如果传递了索引参数，`index` 的长度必须与数组一致。如果没有传递索引参数，生成的结果是 `range(n)`，`n` 为数组长度。

```
In [44]: d = {'one': [1., 2., 3., 4.],
....: 'two': [4., 3., 2., 1.]}
....:

In [45]: pd.DataFrame(d)
Out[45]:
 one two
0 1.0 4.0
1 2.0 3.0
2 3.0 2.0
3 4.0 1.0

In [46]: pd.DataFrame(d, index=['a', 'b', 'c', 'd'])
Out[46]:
 one two
a 1.0 4.0
b 2.0 3.0
c 3.0 2.0
d 4.0 1.0
```

## 用结构多维数组或记录多维数组生成 DataFrame

本例与数组字典的操作方式相同。

```
In [47]: data = np.zeros((2,), dtype=[('A', 'i4'), ('B', 'f4'), ('C', 'a10')])
```

```
In [48]: data[:] = [(1, 2., 'Hello'), (2, 3., "World")]
```

```
In [49]: pd.DataFrame(data)
```

```
Out[49]:
```

	A	B	C
0	1	2.0	b'Hello'
1	2	3.0	b'World'

```
In [50]: pd.DataFrame(data, index=['first', 'second'])
```

```
Out[50]:
```

	A	B	C
first	1	2.0	b'Hello'
second	2	3.0	b'World'

```
In [51]: pd.DataFrame(data, columns=['C', 'A', 'B'])
```

```
Out[51]:
```

	C	A	B
0	b'Hello'	1	2.0
1	b'World'	2	3.0

∴ tip 注意

DataFrame 的运作方式与 NumPy 二维数组不同。

∴

## 用列表字典生成 DataFrame

```
In [52]: data2 = [{'a': 1, 'b': 2}, {'a': 5, 'b': 10, 'c': 20}]
```

```
In [53]: pd.DataFrame(data2)
```

```
Out[53]:
```

	a	b	c
0	1	2	NaN
1	5	10	20.0

```
In [54]: pd.DataFrame(data2, index=['first', 'second'])
```

```
Out[54]:
```

	a	b	c
first	1	2	NaN
second	5	10	20.0

```
In [55]: pd.DataFrame(data2, columns=['a', 'b'])
```

```
Out[55]:
```

	a	b
0	1	2
1	5	10

## 用元组字典生成 DataFrame

元组字典可以自动创建多层索引 DataFrame。

```
In [56]: pd.DataFrame({'a', 'b'): {'(A', 'B)': 1, ('A', 'C)': 2},
.....: ('a', 'a'): {'(A', 'C)': 3, ('A', 'B)': 4},
.....: ('a', 'c'): {'(A', 'B)': 5, ('A', 'C)': 6},
.....: ('b', 'a'): {'(A', 'C)': 7, ('A', 'B)': 8},
.....: ('b', 'b'): {'(A', 'D)': 9, ('A', 'B)': 10}})
Out[56]:
```

		a		b		
		b	a	c	a	b
A	B	1.0	4.0	5.0	8.0	10.0
C		2.0	3.0	6.0	7.0	NaN
D		NaN	NaN	NaN	NaN	9.0

## 用 Series 创建 DataFrame

生成的 DataFrame 继承了输入的 Series 的索引，如果没有指定列名，默认列名是输入 Series 的名称。

### 缺失数据

更多内容，详见[缺失数据](#)。DataFrame 里的缺失值用 `np.nan` 表示。DataFrame 构建器以 `numpy.MaskedArray` 为参数时，被屏蔽的条目为缺失数据。

## 备选构建器

### DataFrame.from\_dict

`DataFrame.from_dict` 接收字典组成的字典或数组序列字典，并生成 DataFrame。除了 `orient` 参数默认为 `columns`，本构建器的操作与 `DataFrame` 构建器类似。把 `orient` 参数设置为 `'index'`，即可把字典的键作为行标签。

```
In [57]: pd.DataFrame.from_dict(dict([('A', [1, 2, 3]), ('B', [4, 5, 6]))))
Out[57]:
```

	A	B
0	1	4
1	2	5
2	3	6

`orient='index'` 时，键是行标签。本例还传递了列名：

```
In [58]: pd.DataFrame.from_dict(dict([('A', [1, 2, 3]), ('B', [4, 5, 6])]),
....: orient='index', columns=['one', 'two', 'three'])
....:
Out[58]:
```

	one	two	three
A	1	2	3
B	4	5	6

## DataFrame.from\_records

`DataFrame.from_records` 构建器支持元组列表或结构数据类型（dtype）的多维数组。本构建器与 `DataFrame` 构建器类似，只不过生成的 `DataFrame` 索引是结构数据类型指定的字段。例如：

```
In [59]: data
Out[59]:
array([(1, 2., b'Hello'), (2, 3., b'World')],
 dtype=[('A', '<i4'), ('B', '<f4'), ('C', 'S10')])

In [60]: pd.DataFrame.from_records(data, index='C')
Out[60]:
```

	A	B
C		
b'Hello'	1	2.0
b'World'	2	3.0

## 提取、添加、删除列

`DataFrame` 就像带索引的 `Series` 字典，提取、设置、删除列的操作与字典类似：

```
In [61]: df['one']
Out[61]:
a 1.0
b 2.0
c 3.0
d NaN
Name: one, dtype: float64

In [62]: df['three'] = df['one'] * df['two']

In [63]: df['flag'] = df['one'] > 2

In [64]: df
Out[64]:
```

	one	two	three	flag
a	1.0	1.0	1.0	False
b	2.0	2.0	4.0	False
c	3.0	3.0	9.0	True
d	NaN	4.0	NaN	False

删除（`del`、`pop`）列的方式也与字典类似：



```
In [65]: del df['two']

In [66]: three = df.pop('three')

In [67]: df
Out[67]:
```

	one	flag
a	1.0	False
b	2.0	False
c	3.0	True
d	NaN	False

标量值以广播的方式填充列：

```
In [68]: df['foo'] = 'bar'

In [69]: df
Out[69]:
```

	one	flag	foo
a	1.0	False	bar
b	2.0	False	bar
c	3.0	True	bar
d	NaN	False	bar

插入与 DataFrame 索引不同的 Series 时，以 DataFrame 的索引为准：

```
In [70]: df['one_trunc'] = df['one'][:2]

In [71]: df
Out[71]:
```

	one	flag	foo	one_trunc
a	1.0	False	bar	1.0
b	2.0	False	bar	2.0
c	3.0	True	bar	NaN
d	NaN	False	bar	NaN

可以插入原生多维数组，但长度必须与 DataFrame 索引长度一致。

默认在 DataFrame 尾部插入列。insert 函数可以指定插入列的位置：

```
In [72]: df.insert(1, 'bar', df['one'])

In [73]: df
Out[73]:
```

	one	bar	flag	foo	one_trunc
a	1.0	1.0	False	bar	1.0
b	2.0	2.0	False	bar	2.0
c	3.0	3.0	True	bar	NaN
d	NaN	NaN	False	bar	NaN

## 用方法链分配新列

受 `dplyr` 的 `mutate` 启发，`DataFrame` 提供了 `assign()` 方法，可以利用现有的列创建新列。

```
In [74]: iris = pd.read_csv('data/iris.data')

In [75]: iris.head()
Out[75]:
```

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
In [76]: (iris.assign(sepal_ratio=iris['SepalWidth'] / iris['SepalLength']))
.....: .head()
.....:
Out[76]:
```

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name	sepal_ratio
0	5.1	3.5	1.4	0.2	Iris-setosa	0.686275
1	4.9	3.0	1.4	0.2	Iris-setosa	0.612245
2	4.7	3.2	1.3	0.2	Iris-setosa	0.680851
3	4.6	3.1	1.5	0.2	Iris-setosa	0.673913
4	5.0	3.6	1.4	0.2	Iris-setosa	0.720000

上例中，插入了一个预计算的值。还可以传递带参数的函数，在 `assign` 的 `DataFrame` 上求值。

```
In [77]: iris.assign(sepal_ratio=lambda x: (x['SepalWidth'] / x['SepalLength'])).head()
Out[77]:
```

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name	sepal_ratio
0	5.1	3.5	1.4	0.2	Iris-setosa	0.686275
1	4.9	3.0	1.4	0.2	Iris-setosa	0.612245
2	4.7	3.2	1.3	0.2	Iris-setosa	0.680851
3	4.6	3.1	1.5	0.2	Iris-setosa	0.673913
4	5.0	3.6	1.4	0.2	Iris-setosa	0.720000

`assign` 返回的**都是**数据副本，原 `DataFrame` 不变。

未引用 `DataFrame` 时，传递可调用的，不是实际要插入的值。这种方式常见于在操作链中调用 `assign` 的操作。例如，将 `DataFrame` 限制为花萼长度大于 5 的观察值，计算比例，再制图：

```
In [78]: (iris.query('SepalLength > 5'))
.....: .assign(SepalRatio=lambda x: x.SepalWidth / x.SepalLength,
.....: PetalRatio=lambda x: x.PetalWidth / x.PetalLength)
.....: .plot(kind='scatter', x='SepalRatio', y='PetalRatio')
.....:
Out[78]: <matplotlib.axes._subplots.AxesSubplot at 0x7f66075a7978>
```

函数运算

上例用 `assign` 把函数传递给 `DataFrame`，并执行函数运算。这是要注意的是，该 `DataFrame` 是筛选了花瓣长度大于 5 以后的数据。首先执行的是筛选操作，再计算比例。这个例子就是对没有事先筛选 `DataFrame` 进行的引用。

`assign` 函数签名就是 `**kwargs`。键是新字段的列名，值为插入值（例如，`Series` 或 `NumPy` 数组），或把 `DataFrame` 当做调用参数的函数。返回结果是插入新值的 `DataFrame` 副本。

0.23.0 版新增。

从 3.6 版开始，Python 可以保存 `**kwargs` 顺序。这种操作允许依赖赋值，`**kwargs` 后的表达式，可以引用同一个 `assign()` 函数里之前创建的列。

```
In [79]: dfa = pd.DataFrame({"A": [1, 2, 3],
....: "B": [4, 5, 6]})
....:

In [80]: dfa.assign(C=lambda x: x['A'] + x['B'],
....: D=lambda x: x['A'] + x['C'])
....:
Out[80]:
 A B C D
0 1 4 5 6
1 2 5 7 9
2 3 6 9 12
```

第二个表达式里，`x['C']` 引用刚创建的列，与 `dfa['A'] + dfa['B']` 等效。

要兼容所有 Python 版本，可以把 `assign` 操作分为两部分。

```
In [81]: dependent = pd.DataFrame({"A": [1, 1, 1]})

In [82]: (dependent.assign(A=lambda x: x['A'] + 1)
....: .assign(B=lambda x: x['A'] + 2))
....:
Out[82]:
 A B
0 2 4
1 2 4
2 2 4
```

### ::: danger 警告

依赖赋值改变了 Python 3.6 及之后版本与 Python 3.6 之前版本的代码操作方式。

要想编写支持 3.6 之前或之后版本的 Python 代码，传递 `assign` 表达式时，要注意以下两点：

- 更新现有的列
- 在同一个 `assign` 引用刚建立的更新列

示例如下，更新列 “A”，然后，在创建 “B” 列时引用该列。

```
>>> dependent = pd.DataFrame({"A": [1, 1, 1]})
>>> dependent.assign(A=lambda x: x["A"] + 1, B=lambda x: x["A"] + 2)
```

Python 3.5 或更早版本的表达式在创建 B 列时引用的是 A 列的“旧”值 [1, 1, 1]。输出是：

```
A B
0 2 3
1 2 3
2 2 3
```

Python >= 3.6 的表达式创建 A 列时，引用的是 A 列的“新”值，[2, 2, 2]，输出是：

```
A B
0 2 4
1 2 4
2 2 4
```

...

## 索引 / 选择

索引基础用法如下：

操作	句法	结果
选择列	<code>df[col]</code>	Series
用标签选择行	<code>df.loc[label]</code>	Series
用整数位置选择行	<code>df.iloc[loc]</code>	Series
行切片	<code>df[5:10]</code>	DataFrame
用布尔向量选择行	<code>df[bool_vec]</code>	DataFrame

选择行返回 Series，索引是 DataFrame 的列：

```
In [83]: df.loc['b']
Out[83]:
one 2
bar 2
flag False
foo bar
one_trunc 2
Name: b, dtype: object
```

```
In [84]: df.iloc[2]
Out[84]:
one 3
bar 3
flag True
foo bar
one_trunc NaN
Name: c, dtype: object
```

高级索引、切片技巧，请参阅[索引](#)。[重建索引](#)介绍重建索引 / 遵循新标签集的基础知识。

## 数据对齐和运算

DataFrame 对象可以自动对齐\*\*列与索引（行标签）\*\*的数据。与上文一样，生成的结果是列和行标签的并集。

```
In [85]: df = pd.DataFrame(np.random.randn(10, 4), columns=['A', 'B', 'C', 'D'])
```

```
In [86]: df2 = pd.DataFrame(np.random.randn(7, 3), columns=['A', 'B', 'C'])
```

```
In [87]: df + df2
```

```
Out[87]:
```

	A	B	C	D
0	0.045691	-0.014138	1.380871	NaN
1	-0.955398	-1.501007	0.037181	NaN
2	-0.662690	1.534833	-0.859691	NaN
3	-2.452949	1.237274	-0.133712	NaN
4	1.414490	1.951676	-2.320422	NaN
5	-0.494922	-1.649727	-1.084601	NaN
6	-1.047551	-0.748572	-0.805479	NaN
7	NaN	NaN	NaN	NaN
8	NaN	NaN	NaN	NaN
9	NaN	NaN	NaN	NaN

DataFrame 和 Series 之间执行操作时，默认操作是在 DataFrame 的**列**上对齐 Series 的**索引**，按行执行**广播**操作。例如：

```
In [88]: df - df.iloc[0]
Out[88]:
```

	A	B	C	D
0	0.000000	0.000000	0.000000	0.000000
1	-1.359261	-0.248717	-0.453372	-1.754659
2	0.253128	0.829678	0.010026	-1.991234
3	-1.311128	0.054325	-1.724913	-1.620544
4	0.573025	1.500742	-0.676070	1.367331
5	-1.741248	0.781993	-1.241620	-2.053136
6	-1.240774	-0.869551	-0.153282	0.000430
7	-0.743894	0.411013	-0.929563	-0.282386
8	-1.194921	1.320690	0.238224	-1.482644
9	2.293786	1.856228	0.773289	-1.446531

时间序列是特例，DataFrame 索引包含日期时，按列广播：

```
In [89]: index = pd.date_range('1/1/2000', periods=8)

In [90]: df = pd.DataFrame(np.random.randn(8, 3), index=index, columns=list('ABC'))

In [91]: df
Out[91]:
```

	A	B	C
2000-01-01	-1.226825	0.769804	-1.281247
2000-01-02	-0.727707	-0.121306	-0.097883
2000-01-03	0.695775	0.341734	0.959726
2000-01-04	-1.110336	-0.619976	0.149748
2000-01-05	-0.732339	0.687738	0.176444
2000-01-06	0.403310	-0.154951	0.301624
2000-01-07	-2.179861	-1.369849	-0.954208
2000-01-08	1.462696	-1.743161	-0.826591

```
In [92]: type(df['A'])
Out[92]: Pandas.core.series.Series

In [93]: df - df['A']
Out[93]:
```

	2000-01-01 00:00:00	2000-01-02 00:00:00	2000-01-03 00:00:00	2000-01-04 00:00:00	...
2000-01-01	NaN	NaN	NaN	NaN	NaN
2000-01-02	NaN	NaN	NaN	NaN	NaN
2000-01-03	NaN	NaN	NaN	NaN	NaN
2000-01-04	NaN	NaN	NaN	NaN	NaN
2000-01-05	NaN	NaN	NaN	NaN	NaN
2000-01-06	NaN	NaN	NaN	NaN	NaN
2000-01-07	NaN	NaN	NaN	NaN	NaN
2000-01-08	NaN	NaN	NaN	NaN	NaN

```
[8 rows x 11 columns]
```

::: danger 警告

```
df - df['A']
```

已弃用，后期版本中会删除。实现此操作的首选方法是：

```
df.sub(df['A'], axis=0)
```

:::

有关匹配和广播操作的显式控制，请参阅[二进制操作](#)。

标量操作与其它数据结构一样：

```
In [94]: df * 5 + 2
```

```
Out[94]:
```

	A	B	C
2000-01-01	-4.134126	5.849018	-4.406237
2000-01-02	-1.638535	1.393469	1.510587
2000-01-03	5.478873	3.708672	6.798628
2000-01-04	-3.551681	-1.099880	2.748742
2000-01-05	-1.661697	5.438692	2.882222
2000-01-06	4.016548	1.225246	3.508122
2000-01-07	-8.899303	-4.849247	-2.771039
2000-01-08	9.313480	-6.715805	-2.132955

```
In [95]: 1 / df
```

```
Out[95]:
```

	A	B	C
2000-01-01	-0.815112	1.299033	-0.780489
2000-01-02	-1.374179	-8.243600	-10.216313
2000-01-03	1.437247	2.926250	1.041965
2000-01-04	-0.900628	-1.612966	6.677871
2000-01-05	-1.365487	1.454041	5.667510
2000-01-06	2.479485	-6.453662	3.315381
2000-01-07	-0.458745	-0.730007	-1.047990
2000-01-08	0.683669	-0.573671	-1.209788

```
In [96]: df ** 4
```

```
Out[96]:
```

	A	B	C
2000-01-01	2.265327	0.351172	2.694833
2000-01-02	0.280431	0.000217	0.000092
2000-01-03	0.234355	0.013638	0.848376
2000-01-04	1.519910	0.147740	0.000503
2000-01-05	0.287640	0.223714	0.000969
2000-01-06	0.026458	0.000576	0.008277
2000-01-07	22.579530	3.521204	0.829033
2000-01-08	4.577374	9.233151	0.466834

支持布尔运算符：



```

In [97]: df1 = pd.DataFrame({'a': [1, 0, 1], 'b': [0, 1, 1]}, dtype=bool)

In [98]: df2 = pd.DataFrame({'a': [0, 1, 1], 'b': [1, 1, 0]}, dtype=bool)

In [99]: df1 & df2
Out[99]:
 a b
0 False False
1 False True
2 True False

In [100]: df1 | df2
Out[100]:
 a b
0 True True
1 True True
2 True True

In [101]: df1 ^ df2
Out[101]:
 a b
0 True True
1 True False
2 False True

In [102]: ~df1
Out[102]:
 a b
0 False True
1 True False
2 False False

```

## 转置

类似于多维数组，`T` 属性（即 `transpose` 函数）可以转置 `DataFrame`：

```

only show the first 5 rows
In [103]: df[:5].T
Out[103]:
 2000-01-01 2000-01-02 2000-01-03 2000-01-04 2000-01-05
A -1.226825 -0.727707 0.695775 -1.110336 -0.732339
B 0.769804 -0.121306 0.341734 -0.619976 0.687738
C -1.281247 -0.097883 0.959726 0.149748 0.176444

```

## DataFrame 应用 NumPy 函数

`Series` 与 `DataFrame` 可使用 `log`、`exp`、`sqrt` 等多种元素级 NumPy 通用函数（ufunc），假设 `DataFrame` 的数据都是数字：

```

In [104]: np.exp(df)
Out[104]:
 A B C
2000-01-01 0.293222 2.159342 0.277691
2000-01-02 0.483015 0.885763 0.906755
2000-01-03 2.005262 1.407386 2.610980
2000-01-04 0.329448 0.537957 1.161542
2000-01-05 0.480783 1.989212 1.192968
2000-01-06 1.496770 0.856457 1.352053
2000-01-07 0.113057 0.254145 0.385117
2000-01-08 4.317584 0.174966 0.437538

In [105]: np.asarray(df)
Out[105]:
array([[-1.2268, 0.7698, -1.2812],
 [-0.7277, -0.1213, -0.0979],
 [0.6958, 0.3417, 0.9597],
 [-1.1103, -0.62 , 0.1497],
 [-0.7323, 0.6877, 0.1764],
 [0.4033, -0.155 , 0.3016],
 [-2.1799, -1.3698, -0.9542],
 [1.4627, -1.7432, -0.8266]])

```

DataFrame 不是多维数组的替代品，它的索引语义和数据模型与多维数组都不同。

`Series` 应用 `__array_ufunc__`，支持 NumPy 通用函数。

通用函数应用于 `Series` 的底层数组。

```

In [106]: ser = pd.Series([1, 2, 3, 4])

In [107]: np.exp(ser)
Out[107]:
0 2.718282
1 7.389056
2 20.085537
3 54.598150
dtype: float64

```

0.25.0 版更改：多个 `Series` 传递给 `ufunc` 时，会先进行对齐。

Pandas 可以自动对齐 `ufunc` 里的多个带标签输入数据。例如，两个标签排序不同的 `Series` 运算前，会先对齐标签。

```
In [108]: ser1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
```

```
In [109]: ser2 = pd.Series([1, 3, 5], index=['b', 'a', 'c'])
```

```
In [110]: ser1
```

```
Out[110]:
```

```
a 1
```

```
b 2
```

```
c 3
```

```
dtype: int64
```

```
In [111]: ser2
```

```
Out[111]:
```

```
b 1
```

```
a 3
```

```
c 5
```

```
dtype: int64
```

```
In [112]: np.remainder(ser1, ser2)
```

```
Out[112]:
```

```
a 1
```

```
b 0
```

```
c 3
```

```
dtype: int64
```

一般来说，Pandas 提取两个索引的并集，不重叠的值用缺失值填充。

```
In [113]: ser3 = pd.Series([2, 4, 6], index=['b', 'c', 'd'])
```

```
In [114]: ser3
```

```
Out[114]:
```

```
b 2
```

```
c 4
```

```
d 6
```

```
dtype: int64
```

```
In [115]: np.remainder(ser1, ser3)
```

```
Out[115]:
```

```
a NaN
```

```
b 0.0
```

```
c 3.0
```

```
d NaN
```

```
dtype: float64
```

对 [Series](#) 和 [Index](#) 应用二进制 ufunc 时，优先执行 Series，并返回的结果也是 Series。

```
In [116]: ser = pd.Series([1, 2, 3])

In [117]: idx = pd.Index([4, 5, 6])

In [118]: np.maximum(ser, idx)
Out[118]:
0 4
1 5
2 6
dtype: int64
```

NumPy 通用函数可以安全地应用于非多维数组支持的 [Series](#)，例如，[SparseArray](#)（参见[稀疏计算](#)）。如有可能，应用 `ufunc` 而不把基础数据转换为多维数组。

## 控制台显示

控制台显示大型 DataFrame 时，会根据空间调整显示大小。[info\(\)](#) 函数可以查看 DataFrame 的信息摘要。下列代码读取 R 语言 **plyr** 包里的[棒球数据集](#) CSV 文件）：

```
In [119]: baseball = pd.read_csv('data/baseball.csv')
```

```
In [120]: print(baseball)
```

	id	player	year	stint	team	lg	g	ab	r	h	X2b	X3b	hr	rbi	sb	cs	bb	so
		ibb	hbp	sh	sf	gidp												
0	88641	womacto01	2006	2	CHN	NL	19	50	6	14	1	0	1	2.0	1.0	1.0	4	4.0
		0.0	0.0	3.0	0.0	0.0												
1	88643	schilcu01	2006	1	BOS	AL	31	2	0	1	0	0	0	0.0	0.0	0.0	0	1.0
		0.0	0.0	0.0	0.0	0.0												
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
98	89533	aloumo01	2007	1	NYN	NL	87	328	51	112	19	1	13	49.0	3.0	0.0	27	30.0
		5.0	2.0	0.0	3.0	13.0												
99	89534	alomasa02	2007	1	NYN	NL	8	22	1	3	1	0	0	0.0	0.0	0.0	0	3.0
		0.0	0.0	0.0	0.0	0.0												

```
[100 rows x 23 columns]
```

```
In [121]: baseball.info()
```

```
<class 'Pandas.core.frame.DataFrame'>
```

```
RangeIndex: 100 entries, 0 to 99
```

```
Data columns (total 23 columns):
```

id	100	non-null	int64
player	100	non-null	object
year	100	non-null	int64
stint	100	non-null	int64
team	100	non-null	object
lg	100	non-null	object
g	100	non-null	int64
ab	100	non-null	int64
r	100	non-null	int64
h	100	non-null	int64
X2b	100	non-null	int64
X3b	100	non-null	int64
hr	100	non-null	int64
rbi	100	non-null	float64
sb	100	non-null	float64
cs	100	non-null	float64
bb	100	non-null	int64
so	100	non-null	float64
ibb	100	non-null	float64
hbp	100	non-null	float64
sh	100	non-null	float64
sf	100	non-null	float64
gidp	100	non-null	float64

```
dtypes: float64(9), int64(11), object(3)
```

```
memory usage: 18.1+ KB
```

尽管 `to_string` 有时不匹配控制台的宽度，但还是可以用 `to_string` 以表格形式返回 DataFrame 的字符串表示形式：

```
In [122]: print(baseball.iloc[-20:, :12].to_string())
```

	id	player	year	stint	team	lg	g	ab	r	h	X2b	X3b
80	89474	finlest01	2007	1	COL	NL	43	94	9	17	3	0
81	89480	embreal01	2007	1	OAK	AL	4	0	0	0	0	0
82	89481	edmonji01	2007	1	SLN	NL	117	365	39	92	15	2
83	89482	easleda01	2007	1	NYN	NL	76	193	24	54	6	0
84	89489	delgaca01	2007	1	NYN	NL	139	538	71	139	30	0
85	89493	cormirh01	2007	1	CIN	NL	6	0	0	0	0	0
86	89494	coninje01	2007	2	NYN	NL	21	41	2	8	2	0
87	89495	coninje01	2007	1	CIN	NL	80	215	23	57	11	1
88	89497	clemero02	2007	1	NYA	AL	2	2	0	1	0	0
89	89498	claytro01	2007	2	BOS	AL	8	6	1	0	0	0
90	89499	claytro01	2007	1	TOR	AL	69	189	23	48	14	0
91	89501	cirilje01	2007	2	ARI	NL	28	40	6	8	4	0
92	89502	cirilje01	2007	1	MIN	AL	50	153	18	40	9	2
93	89521	bondsba01	2007	1	SFN	NL	126	340	75	94	14	0
94	89523	biggicr01	2007	1	HOU	NL	141	517	68	130	31	3
95	89525	benitar01	2007	2	FLO	NL	34	0	0	0	0	0
96	89526	benitar01	2007	1	SFN	NL	19	0	0	0	0	0
97	89530	ausmubr01	2007	1	HOU	NL	117	349	38	82	16	3
98	89533	aloumo01	2007	1	NYN	NL	87	328	51	112	19	1
99	89534	alomas02	2007	1	NYN	NL	8	22	1	3	1	0

默认情况下，过宽的 DataFrame 会跨多行输出：

```
In [123]: pd.DataFrame(np.random.randn(3, 12))
```

```
Out[123]:
```

	0	1	2	3	4	5	6	7	8	
0	-0.345352	1.314232	0.690579	0.995761	2.396780	0.014871	3.357427	-0.317441	-1.236269	0.896171
	-0.487602	-0.082240								
1	-2.182937	0.380396	0.084844	0.432390	1.519970	-0.493662	0.600178	0.274230	0.132885	-0.023688
	2.410179	1.450520								
2	0.206053	-0.251905	-2.213588	1.063327	1.266143	0.299368	-0.863838	0.408204	-1.048089	-0.025747
	-0.988387	0.094055								

`display.width` 选项可以更改单行输出的宽度：

```
In [124]: pd.set_option('display.width', 40) # 默认值为 80
```

```
In [125]: pd.DataFrame(np.random.randn(3, 12))
```

```
Out[125]:
```

	0	1	2	3	4	5	6	7	8	
0	1.262731	1.289997	0.082423	-0.055758	0.536580	-0.489682	0.369374	-0.034571	-2.484478	-0.281461
	0.030711	0.109121								
1	1.126203	-0.977349	1.474071	-0.064034	-1.282782	0.781836	-1.071357	0.441153	2.353925	0.583787
	0.221471	-0.744471								
2	0.758527	1.729689	-0.964980	-0.845696	-1.340896	1.846883	-1.328865	1.682706	-1.717693	0.888782
	0.228440	0.901805								

还可以用 `display.max_colwidth` 调整最大列宽。

```

In [126]: datafile = {'filename': ['filename_01', 'filename_02'],
.....: 'path': ["media/user_name/storage/folder_01/filename_01",
.....: "media/user_name/storage/folder_02/filename_02"]}
.....:

In [127]: pd.set_option('display.max_colwidth', 30)

In [128]: pd.DataFrame(datafile)
Out[128]:
 filename path
0 filename_01 media/user_name/storage/fo...
1 filename_02 media/user_name/storage/fo...

In [129]: pd.set_option('display.max_colwidth', 100)

In [130]: pd.DataFrame(datafile)
Out[130]:
 filename path
0 filename_01 media/user_name/storage/folder_01/filename_01
1 filename_02 media/user_name/storage/folder_02/filename_02

```

`expand_frame_repr` 选项可以禁用此功能，在一个区块里输出整个表格。

## DataFrame 列属性访问和 IPython 代码补全

DataFrame 列标签是有效的 Python 变量名时，可以像属性一样访问该列：

```

In [131]: df = pd.DataFrame({'foo1': np.random.randn(5),
.....: 'foo2': np.random.randn(5)})
.....:

In [132]: df
Out[132]:
 foo1 foo2
0 1.171216 -0.858447
1 0.520260 0.306996
2 -1.197071 -0.028665
3 -1.066969 0.384316
4 -0.303421 1.574159

In [133]: df.foo1
Out[133]:
0 1.171216
1 0.520260
2 -1.197071
3 -1.066969
4 -0.303421
Name: foo1, dtype: float64

```

IPython 支持补全功能，按 **tab** 键可以实现代码补全：

```
In [134]: df.fo<TAB> # 此时按 tab 键 会显示下列内容
df.foo1 df.foo2
```

---

## 与其他工具比较

### 与R/R库的比较

---

由于 `pandas` 旨在为人们提供可以替代R的大量数据操作和分析的功能，因此本章节会提供较为详细的R语言的介绍以及与相关的许多第三方库的对比说明，比如我们的 `pandas` 库。在与R和CRAN库的比较中，我们关注以下事项：

- **功能/灵活性**：每个工具可以/不可以做什么
- **性能**：操作速度有多快。硬性数字/基准是优选的
- **易于使用**：一种工具更容易/更难使用（您可能需要对此进行判断，并进行并排代码比较）

此页面还为这些R包的用户提供了一些翻译指南。

要将 `DataFrame` 对象从 `pandas` 转化为到 R 的数据类型，有一个选择是采用HDF5文件，请参阅[外部兼容性](#)示例。

### 快速参考

我们将从快速参考指南开始，将[dplyr](#)与pandas等效的一些常见R操作配对。



## 查询、过滤、采样

R	Pandas
dim(df)	df.shape
head(df)	df.head()
slice(df, 1:10)	df.iloc[:9]
filter(df, col1 == 1, col2 == 1)	df.query('col1 == 1 & col2 == 1')
df[df\$col1 == 1 & df\$col2 == 1,]	df[(df.col1 == 1) & (df.col2 == 1)]
select(df, col1, col2)	df[['col1', 'col2']]
select(df, col1:col3)	df.loc[:, 'col1':'col3']
select(df, -(col1:col3))	df.drop(cols_to_drop, axis=1)但是看 <a href="#">[1]</a>
distinct(select(df, col1))	df[['col1']].drop_duplicates()
distinct(select(df, col1, col2))	df[['col1', 'col2']].drop_duplicates()
sample_n(df, 10)	df.sample(n=10)
sample_frac(df, 0.01)	df.sample(frac=0.01)

∴ tip Note

R表示列的子集 (`select(df, col1: col3)` 的缩写更接近 Pandas 的写法, 如果您有列的列表, 例如 `df[cols[1:3]]` 或 `df.drop(cols[1:3])`), 按列名执行此操作可能会引起混乱。

∴

## 排序

R	Pandas
arrange(df, col1, col2)	df.sort_values(['col1', 'col2'])
arrange(df, desc(col1))	df.sort_values('col1', ascending=False)

## 变换

R	Pandas
select(df, col_one = col1)	df.rename(columns={'col1': 'col_one'})['col_one']
rename(df, col_one = col1)	df.rename(columns={'col1': 'col_one'})
mutate(df, c=a-b)	df.assign(c=df.a-df.b)

## 分组和组合

R	Pandas
summary(df)	df.describe()
gdf <- group_by(df, col1)	gdf = df.groupby('col1')
summarise(gdf, avg=mean(col1, na.rm=TRUE))	df.groupby('col1').agg({'col1': 'mean'})
summarise(gdf, total=sum(col1))	df.groupby('col1').sum()

## 基本的R用法

### 用Rc方法来进行切片操作

R使您可以轻松地按名称访问列（`data.frame`）

```
df <- data.frame(a=rnorm(5), b=rnorm(5), c=rnorm(5), d=rnorm(5), e=rnorm(5))
df[, c("a", "c", "e")]
```

或整数位置

```
df <- data.frame(matrix(rnorm(1000), ncol=100))
df[, c(1:10, 25:30, 40, 50:100)]
```

按名称选择多个 `pandas` 的列非常简单

```
In [1]: df = pd.DataFrame(np.random.randn(10, 3), columns=list('abc'))
```

```
In [2]: df[['a', 'c']]
```

```
Out[2]:
```

	a	c
0	0.469112	-1.509059
1	-1.135632	-0.173215
2	0.119209	-0.861849
3	-2.104569	1.071804
4	0.721555	-1.039575
5	0.271860	0.567020
6	0.276232	-0.673690
7	0.113648	0.524988
8	0.404705	-1.715002
9	-1.039268	-1.157892

```
In [3]: df.loc[:, ['a', 'c']]
```

```
Out[3]:
```

	a	c
0	0.469112	-1.509059
1	-1.135632	-0.173215
2	0.119209	-0.861849
3	-2.104569	1.071804
4	0.721555	-1.039575
5	0.271860	0.567020
6	0.276232	-0.673690
7	0.113648	0.524988
8	0.404705	-1.715002
9	-1.039268	-1.157892

通过整数位置选择多个不连续的列可以通过 `iloc` 索引器属性和 `numpy.r_` 的组合来实现。

```

In [4]: named = list('abcdefg')

In [5]: n = 30

In [6]: columns = named + np.arange(len(named), n).tolist()

In [7]: df = pd.DataFrame(np.random.randn(n, n), columns=columns)

In [8]: df.iloc[:, np.r_[:10, 24:30]]
Out[8]:
 a b c d e f g 7 8
 9 24 25 26 27 28 29
0 -1.344312 0.844885 1.075770 -0.109050 1.643563 -1.469388 0.357021 -0.674600 -1.776904 -0.968914
 -1.170299 -0.226169 0.410835 0.813850 0.132003 -0.827317
1 -0.076467 -1.187678 1.130127 -1.436737 -1.413681 1.607920 1.024180 0.569605 0.875906 -2.211372
 0.959726 -1.110336 -0.619976 0.149748 -0.732339 0.687738
2 0.176444 0.403310 -0.154951 0.301624 -2.179861 -1.369849 -0.954208 1.462696 -1.743161 -0.826591
 0.084844 0.432390 1.519970 -0.493662 0.600178 0.274230
3 0.132885 -0.023688 2.410179 1.450520 0.206053 -0.251905 -2.213588 1.063327 1.266143 0.299368
 -2.484478 -0.281461 0.030711 0.109121 1.126203 -0.977349
4 1.474071 -0.064034 -1.282782 0.781836 -1.071357 0.441153 2.353925 0.583787 0.221471 -0.744471
 -1.197071 -1.066969 -0.303421 -0.858447 0.306996 -0.028665
..
25 1.492125 -0.068190 0.681456 1.221829 -0.434352 1.204815 -0.195612 1.251683 -1.040389 -0.796211
 1.944517 0.042344 -0.307904 0.428572 0.880609 0.487645
26 0.725238 0.624607 -0.141185 -0.143948 -0.328162 2.095086 -0.608888 -0.926422 1.872601 -2.513465
 -0.846188 1.190624 0.778507 1.008500 1.424017 0.717110
27 1.262419 1.950057 0.301038 -0.933858 0.814946 0.181439 -0.110015 -2.364638 -1.584814 0.307941
 -1.341814 0.334281 -0.162227 1.007824 2.826008 1.458383
28 -1.585746 -0.899734 0.921494 -0.211762 -0.059182 0.058308 0.915377 -0.696321 0.150664 -3.060395
 0.403620 -0.026602 -0.240481 0.577223 -1.088417 0.326687
29 -0.986248 0.169729 -1.158091 1.019673 0.646039 0.917399 -0.010435 0.366366 0.922729 0.869610
 -1.209247 -0.671466 0.332872 -2.013086 -1.602549 0.333109

[30 rows x 16 columns]

```

## aggregate

在R中，您可能希望将数据分成几个子集，并计算每个子集的平均值。使用名为 `df` 的data.frame并将其分成组 `by1` 和 `by2`：

```

df <- data.frame(
 v1 = c(1,3,5,7,8,3,5,NA,4,5,7,9),
 v2 = c(11,33,55,77,88,33,55,NA,44,55,77,99),
 by1 = c("red", "blue", 1, 2, NA, "big", 1, 2, "red", 1, NA, 12),
 by2 = c("wet", "dry", 99, 95, NA, "damp", 95, 99, "red", 99, NA, NA))
aggregate(x=df[, c("v1", "v2")], by=list(mydf2$by1, mydf2$by2), FUN = mean)

```

该 `groupby()` 方法类似于基本R的 `aggregate` 函数。

```
In [9]: df = pd.DataFrame(
...: {'v1': [1, 3, 5, 7, 8, 3, 5, np.nan, 4, 5, 7, 9],
...: 'v2': [11, 33, 55, 77, 88, 33, 55, np.nan, 44, 55, 77, 99],
...: 'by1': ["red", "blue", 1, 2, np.nan, "big", 1, 2, "red", 1, np.nan, 12],
...: 'by2': ["wet", "dry", 99, 95, np.nan, "damp", 95, 99, "red", 99, np.nan,
...: np.nan]})
...:
```

```
In [10]: g = df.groupby(['by1', 'by2'])
```

```
In [11]: g[['v1', 'v2']].mean()
```

```
Out[11]:
```

		v1	v2
by1	by2		
1	95	5.0	55.0
	99	5.0	55.0
2	95	7.0	77.0
	99	NaN	NaN
big	damp	3.0	33.0
blue	dry	3.0	33.0
red	red	4.0	44.0
	wet	1.0	11.0

有关更多详细信息和示例，请参阅[groupby文档](#)。

## match/%in%

在R中选择数据的常用方法是使用 `%in%` 使用该函数定义的数据 `match`。运算符 `%in%` 用于返回指示是否存在匹配的逻辑向量：

```
s <- 0:4
s %in% c(2,4)
```

该 `isin()` 方法类似于R `%in%` 运算符：

```
In [12]: s = pd.Series(np.arange(5), dtype=np.float32)

In [13]: s.isin([2, 4])
Out[13]:
0 False
1 False
2 True
3 False
4 True
dtype: bool
```

该 `match` 函数返回其第二个参数匹配位置的向量：

```
s <- 0:4
match(s, c(2,4))
```

有关更多详细信息和示例，请参阅[重塑文档](#)。

## tapply

`tapply` 类似于 `aggregate`，但数据可以是一个参差不齐的数组，因为子类大小可能是不规则的。使用调用的 `data.frame baseball`，并根据数组检索信息 `team`：

```
baseball <-
 data.frame(team = gl(5, 5,
 labels = paste("Team", LETTERS[1:5])),
 player = sample(letters, 25),
 batting.average = runif(25, .200, .400))

tapply(baseball$batting.average, baseball$team,
 max)
```

在 `pandas` 我们可以使用 `pivot_table()` 方法来处理这个：

```
In [14]: import random

In [15]: import string

In [16]: baseball = pd.DataFrame(
 : {'team': ["team %d" % (x + 1) for x in range(5)] * 5,
 : 'player': random.sample(list(string.ascii_lowercase), 25),
 : 'batting avg': np.random.uniform(.200, .400, 25)})
 :

In [17]: baseball.pivot_table(values='batting avg', columns='team', aggfunc=np.max)
Out[17]:
team team 1 team 2 team 3 team 4 team 5
batting avg 0.352134 0.295327 0.397191 0.394457 0.396194
```

有关更多详细信息和示例，请参阅[重塑文档](#)。

## subset

该 `query()` 方法类似于基本 R `subset` 函数。在 R 中，您可能希望获取 `data.frame` 一列的值小于另一列的值的行：

```
df <- data.frame(a=rnorm(10), b=rnorm(10))
subset(df, a <= b)
df[df$a <= df$b,] # note the comma
```

在 `pandas`，有几种方法可以执行子集化。您可以使用 `query()` 或传递表达式，就像它是索引/切片以及标准布尔索引一样：

```
In [18]: df = pd.DataFrame({'a': np.random.randn(10), 'b': np.random.randn(10)})
```

```
In [19]: df.query('a <= b')
```

```
Out[19]:
```

	a	b
1	0.174950	0.552887
2	-0.023167	0.148084
3	-0.495291	-0.300218
4	-0.860736	0.197378
5	-1.134146	1.720780
7	-0.290098	0.083515
8	0.238636	0.946550

```
In [20]: df[df.a <= df.b]
```

```
Out[20]:
```

	a	b
1	0.174950	0.552887
2	-0.023167	0.148084
3	-0.495291	-0.300218
4	-0.860736	0.197378
5	-1.134146	1.720780
7	-0.290098	0.083515
8	0.238636	0.946550

```
In [21]: df.loc[df.a <= df.b]
```

```
Out[21]:
```

	a	b
1	0.174950	0.552887
2	-0.023167	0.148084
3	-0.495291	-0.300218
4	-0.860736	0.197378
5	-1.134146	1.720780
7	-0.290098	0.083515
8	0.238636	0.946550

有关更多详细信息和示例，请参阅[查询文档](#)。

## with

使用 `df` 带有列的R中调用的`data.frame`的表达式 `a`，`b` 将使用 `with` 如下方式进行求值：

```
df <- data.frame(a=rnorm(10), b=rnorm(10))
with(df, a + b)
df$a + df$b # same as the previous expression
```

在 `pandas` 等效表达式中，使用该 `eval()` 方法将是：

```

In [22]: df = pd.DataFrame({'a': np.random.randn(10), 'b': np.random.randn(10)})

In [23]: df.eval('a + b')
Out[23]:
0 -0.091430
1 -2.483890
2 -0.252728
3 -0.626444
4 -0.261740
5 2.149503
6 -0.332214
7 0.799331
8 -2.377245
9 2.104677
dtype: float64

In [24]: df.a + df.b # same as the previous expression
Out[24]:
0 -0.091430
1 -2.483890
2 -0.252728
3 -0.626444
4 -0.261740
5 2.149503
6 -0.332214
7 0.799331
8 -2.377245
9 2.104677
dtype: float64

```

在某些情况下，`eval()`将比纯Python中的评估快得多。有关更多详细信息和示例，请参阅[eval文档](#)。

## plyr

`plyr` 是用于数据分析的拆分应用组合策略的R库。这些函数围绕R， `a` for `arrays`， `l` for `Lists` 和 `d` for 中的三个数据结构 `data.frame`。下表显示了如何在Python中映射这些数据结构。

R	Python
array	list
lists	字典（dict）或对象列表（list of objects）
data.frame	dataframe

## ddply

在R中使用名为 `df` 的 `data.frame` 的表达式，比如您有一个希望按月汇总 `x` 的需求：



```
require(plyr)
df <- data.frame(
 x = runif(120, 1, 168),
 y = runif(120, 7, 334),
 z = runif(120, 1.7, 20.7),
 month = rep(c(5,6,7,8),30),
 week = sample(1:4, 120, TRUE)
)

ddply(df, .(month, week), summarize,
 mean = round(mean(x), 2),
 sd = round(sd(x), 2))
```

在 pandas 等效表达式中，使用该 `groupby()` 方法将是：

```
In [25]: df = pd.DataFrame({'x': np.random.uniform(1., 168., 120),
.....: 'y': np.random.uniform(7., 334., 120),
.....: 'z': np.random.uniform(1.7, 20.7, 120),
.....: 'month': [5, 6, 7, 8] * 30,
.....: 'week': np.random.randint(1, 4, 120)})

In [26]: grouped = df.groupby(['month', 'week'])

In [27]: grouped['x'].agg([np.mean, np.std])
Out[27]:
```

		mean	std
month	week		
5	1	63.653367	40.601965
	2	78.126605	53.342400
	3	92.091886	57.630110
6	1	81.747070	54.339218
	2	70.971205	54.687287
	3	100.968344	54.010081
7	1	61.576332	38.844274
	2	61.733510	48.209013
	3	71.688795	37.595638
8	1	62.741922	34.618153
	2	91.774627	49.790202
	3	73.936856	60.773900

有关更多详细信息和示例，请参阅[groupby文档](#)。

## 重塑/ reshape2

### `melt.array`

使用 `a` 在 R 中调用的 3 维数组的表达式，您希望将其融合到 `data.frame` 中：

```
a <- array(c(1:23, NA), c(2,3,4))
data.frame(melt(a))
```

在Python中，既然 `a` 是一个列表，你可以简单地使用列表理解。

```
In [28]: a = np.array(list(range(1, 24)) + [np.NaN]).reshape(2, 3, 4)

In [29]: pd.DataFrame([tuple(list(x) + [val]) for x, val in np.ndenumerate(a)])
Out[29]:
 0 1 2 3
0 0 0 0 1.0
1 0 0 1 2.0
2 0 0 2 3.0
3 0 0 3 4.0
4 0 1 0 5.0
..
19 1 1 3 20.0
20 1 2 0 21.0
21 1 2 1 22.0
22 1 2 2 23.0
23 1 2 3 NaN

[24 rows x 4 columns]
```

### `melt.list`

使用 `a` 在R中调用的列表的表达式，您希望将其融合到 `data.frame` 中：

```
a <- as.list(c(1:4, NA))
data.frame(melt(a))
```

在Python中，此列表将是元组列表，因此 `DataFrame()` 方法会根据需要将其转换为数据帧。

```
In [30]: a = list(enumerate(list(range(1, 5)) + [np.NaN]))

In [31]: pd.DataFrame(a)
Out[31]:
 0 1
0 0 1.0
1 1 2.0
2 2 3.0
3 3 4.0
4 4 NaN
```

有关更多详细信息和示例，请参阅 [“进入数据结构”文档](#)。

### `melt.data.frame`

使用 `cheese` 在R中调用的 `data.frame` 的表达式，您要在其中重新整形 `data.frame`：

```
cheese <- data.frame(
 first = c('John', 'Mary'),
 last = c('Doe', 'Bo'),
 height = c(5.5, 6.0),
 weight = c(130, 150)
)
melt(cheese, id=c("first", "last"))
```

在Python中，该melt()方法是R等价物：

```
In [32]: cheese = pd.DataFrame({'first': ['John', 'Mary'],
....: 'last': ['Doe', 'Bo'],
....: 'height': [5.5, 6.0],
....: 'weight': [130, 150]})
....:

In [33]: pd.melt(cheese, id_vars=['first', 'last'])
Out[33]:
```

	first	last	variable	value
0	John	Doe	height	5.5
1	Mary	Bo	height	6.0
2	John	Doe	weight	130.0
3	Mary	Bo	weight	150.0

```

In [34]: cheese.set_index(['first', 'last']).stack() # alternative way
Out[34]:
```

first	last	variable	value
John	Doe	height	5.5
		weight	130.0
Mary	Bo	height	6.0
		weight	150.0

```

dtype: float64
```

有关更多详细信息和示例，请参阅[重塑文档](#)。

## cast

在R中 `acast` 是一个表达式，它使用 `df` 在R中调用的data.frame 来转换为更高维的数组：

```
df <- data.frame(
 x = runif(12, 1, 168),
 y = runif(12, 7, 334),
 z = runif(12, 1.7, 20.7),
 month = rep(c(5,6,7),4),
 week = rep(c(1,2), 6)
)

mdf <- melt(df, id=c("month", "week"))
acast(mdf, week ~ month ~ variable, mean)
```

在Python中，最好的方法是使用[pivot\\_table\(\)](#)：

```

In [35]: df = pd.DataFrame({'x': np.random.uniform(1., 168., 12),
.....: 'y': np.random.uniform(7., 334., 12),
.....: 'z': np.random.uniform(1.7, 20.7, 12),
.....: 'month': [5, 6, 7] * 4,
.....: 'week': [1, 2] * 6})
.....:

In [36]: mdf = pd.melt(df, id_vars=['month', 'week'])

In [37]: pd.pivot_table(mdf, values='value', index=['variable', 'week'],
.....: columns=['month'], aggfunc=np.mean)
.....:
Out[37]:

```

		5	6	7
variable	week			
x	1	93.888747	98.762034	55.219673
	2	94.391427	38.112932	83.942781
y	1	94.306912	279.454811	227.840449
	2	87.392662	193.028166	173.899260
z	1	11.016009	10.079307	16.170549
	2	8.476111	17.638509	19.003494

类似地 `dcast`，使用 `df` R中调用的`data.frame` 来基于 `Animal` 和聚合信息 `FeedType`：

```

df <- data.frame(
 Animal = c('Animal1', 'Animal2', 'Animal3', 'Animal2', 'Animal1',
 'Animal2', 'Animal3'),
 FeedType = c('A', 'B', 'A', 'A', 'B', 'B', 'A'),
 Amount = c(10, 7, 4, 2, 5, 6, 2)
)

dcast(df, Animal ~ FeedType, sum, fill=NaN)
Alternative method using base R
with(df, tapply(Amount, list(Animal, FeedType), sum))

```

Python可以通过两种不同的方式处理它。首先，类似于上面使用`pivot_table()`：

```
In [38]: df = pd.DataFrame({
.....: 'Animal': ['Animal1', 'Animal2', 'Animal3', 'Animal2', 'Animal1',
.....: 'Animal2', 'Animal3'],
.....: 'FeedType': ['A', 'B', 'A', 'A', 'B', 'B', 'A'],
.....: 'Amount': [10, 7, 4, 2, 5, 6, 2],
.....: })
.....:

In [39]: df.pivot_table(values='Amount', index='Animal', columns='FeedType',
.....: aggfunc='sum')
.....:
Out[39]:
FeedType A B
Animal
Animal1 10.0 5.0
Animal2 2.0 13.0
Animal3 6.0 NaN
```

第二种方法是使用该[groupby\(\)](#)方法：

```
In [40]: df.groupby(['Animal', 'FeedType'])['Amount'].sum()
Out[40]:
Animal FeedType
Animal1 A 10
 B 5
Animal2 A 2
 B 13
Animal3 A 6
Name: Amount, dtype: int64
```

有关更多详细信息和示例，请参阅[重新整形文档](#)或[groupby文档](#)。

## factor

pandas具有分类数据的数据类型。

```
cut(c(1,2,3,4,5,6), 3)
factor(c(1,2,3,2,2,3))
```

在Pandas，这是完成与 `pd.cut` 和 `astype("category")`：

```

In [41]: pd.cut(pd.Series([1, 2, 3, 4, 5, 6]), 3)
Out[41]:
0 (0.995, 2.667]
1 (0.995, 2.667]
2 (2.667, 4.333]
3 (2.667, 4.333]
4 (4.333, 6.0]
5 (4.333, 6.0]
dtype: category
Categories (3, interval[float64]): [(0.995, 2.667] < (2.667, 4.333] < (4.333, 6.0]]

In [42]: pd.Series([1, 2, 3, 2, 2, 3]).astype("category")
Out[42]:
0 1
1 2
2 3
3 2
4 2
5 3
dtype: category
Categories (3, int64): [1, 2, 3]

```

有关更多详细信息和示例，请参阅[分类介绍](#)和 [API文档](#)。还有一个关于[R因子差异](#)的文档。

## 与SQL比较

由于许多潜在的 pandas 用户对SQL有一定的了解，因此本页面旨在提供一些使用pandas如何执行各种SQL操作的示例。

如果您是 pandas 的新手，您可能需要先阅读[十分钟入门Pandas](#)以熟悉本库。

按照惯例，我们按如下方式导入 pandas 和 NumPy：

```

In [1]: import pandas as pd

In [2]: import numpy as np

```

大多数示例将使用 `tips` pandas测试中找到的数据集。我们将数据读入名为tips的DataFrame中，并假设我们有一个具有相同名称和结构的数据库表。

```
In [3]: url = ('https://raw.githubusercontent.com/pandas-dev/pandas/master/pandas/tests/data/tips.csv')
...:
...:
```

```
In [4]: tips = pd.read_csv(url)
```

```
In [5]: tips.head()
```

```
Out[5]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

## SELECT

在SQL中，使用您要选择的以逗号分隔的列列表（或 \* 选择所有列）来完成选择：

```
SELECT total_bill, tip, smoker, time
FROM tips
LIMIT 5;
```

使用pandas，通过将列名列表传递给DataFrame来完成列选择：

```
In [6]: tips[['total_bill', 'tip', 'smoker', 'time']].head(5)
Out[6]:
```

	total_bill	tip	smoker	time
0	16.99	1.01	No	Dinner
1	10.34	1.66	No	Dinner
2	21.01	3.50	No	Dinner
3	23.68	3.31	No	Dinner
4	24.59	3.61	No	Dinner

在没有列名列表的情况下调用DataFrame将显示所有列（类似于SQL \*）。

## WHERE

SQL中的过滤是通过WHERE子句完成的。

```
SELECT *
FROM tips
WHERE time = 'Dinner'
LIMIT 5;
```

DataFrame可以通过多种方式进行过滤; 最直观的是使用 [布尔索引](#)。

```
In [7]: tips[tips['time'] == 'Dinner'].head(5)
Out[7]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

上面的语句只是将一个 `Series` 的 True / False 对象传递给 `DataFrame`，返回所有带有True的行。

```
In [8]: is_dinner = tips['time'] == 'Dinner'

In [9]: is_dinner.value_counts()
Out[9]:
```

	count
True	176
False	68

Name: time, dtype: int64

```
In [10]: tips[is_dinner].head(5)
Out[10]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

就像SQL的OR和AND一样，可以使用|将多个条件传递给`DataFrame`（OR）和&（AND）。

```
-- tips of more than $5.00 at Dinner meals
SELECT *
FROM tips
WHERE time = 'Dinner' AND tip > 5.00;
```



```
tips of more than $5.00 at Dinner meals
In [11]: tips[(tips['time'] == 'Dinner') & (tips['tip'] > 5.00)]
Out[11]:
```

	total_bill	tip	sex	smoker	day	time	size
23	39.42	7.58	Male	No	Sat	Dinner	4
44	30.40	5.60	Male	No	Sun	Dinner	4
47	32.40	6.00	Male	No	Sun	Dinner	4
52	34.81	5.20	Female	No	Sun	Dinner	4
59	48.27	6.73	Male	No	Sat	Dinner	4
116	29.93	5.07	Male	No	Sun	Dinner	4
155	29.85	5.14	Female	No	Sun	Dinner	5
170	50.81	10.00	Male	Yes	Sat	Dinner	3
172	7.25	5.15	Male	Yes	Sun	Dinner	2
181	23.33	5.65	Male	Yes	Sun	Dinner	2
183	23.17	6.50	Male	Yes	Sun	Dinner	4
211	25.89	5.16	Male	Yes	Sat	Dinner	4
212	48.33	9.00	Male	No	Sat	Dinner	4
214	28.17	6.50	Female	Yes	Sat	Dinner	3
239	29.03	5.92	Male	No	Sat	Dinner	3

```
-- tips by parties of at least 5 diners OR bill total was more than $45
SELECT *
FROM tips
WHERE size >= 5 OR total_bill > 45;
```

```
tips by parties of at least 5 diners OR bill total was more than $45
In [12]: tips[(tips['size'] >= 5) | (tips['total_bill'] > 45)]
Out[12]:
```

	total_bill	tip	sex	smoker	day	time	size
59	48.27	6.73	Male	No	Sat	Dinner	4
125	29.80	4.20	Female	No	Thur	Lunch	6
141	34.30	6.70	Male	No	Thur	Lunch	6
142	41.19	5.00	Male	No	Thur	Lunch	5
143	27.05	5.00	Female	No	Thur	Lunch	6
155	29.85	5.14	Female	No	Sun	Dinner	5
156	48.17	5.00	Male	No	Sun	Dinner	6
170	50.81	10.00	Male	Yes	Sat	Dinner	3
182	45.35	3.50	Male	Yes	Sun	Dinner	3
185	20.69	5.00	Male	No	Sun	Dinner	5
187	30.46	2.00	Male	Yes	Sun	Dinner	5
212	48.33	9.00	Male	No	Sat	Dinner	4
216	28.15	3.00	Male	Yes	Sat	Dinner	5

使用notna()和isna() 方法完成NULL检查。

```
In [13]: frame = pd.DataFrame({'col1': ['A', 'B', np.NaN, 'C', 'D'],
....: 'col2': ['F', np.NaN, 'G', 'H', 'I']})
....:
```

```
In [14]: frame
```

```
Out[14]:
 col1 col2
0 A F
1 B NaN
2 NaN G
3 C H
4 D I
```

假设我们有一个与上面的DataFrame结构相同的表。我们只能 col2 通过以下查询看到IS NULL 的记录：

```
SELECT *
FROM frame
WHERE col2 IS NULL;
```

```
In [15]: frame[frame['col2'].isna()]
```

```
Out[15]:
 col1 col2
1 B NaN
```

获取 col1 IS NOT NULL的项目可以完成notna()。

```
SELECT *
FROM frame
WHERE col1 IS NOT NULL;
```

```
In [16]: frame[frame['col1'].notna()]
```

```
Out[16]:
 col1 col2
0 A F
1 B NaN
3 C H
4 D I
```

## GROUP BY

在pandas中，SQL的GROUP BY操作使用类似命名的 `groupby()` 方法执行。`groupby()` 通常是指我们想要将数据集拆分成组，应用某些功能（通常是聚合），然后将这些组合在一起的过程。

常见的SQL操作是获取整个数据集中每个组中的记录数。例如，有一个需要向我们提供提示中的性别的数量的查询语句：

```

SELECT sex, count(*)
FROM tips
GROUP BY sex;
/*
Female 87
Male 157
*/

```

在 pandas 中可以这样：

```

In [17]: tips.groupby('sex').size()
Out[17]:
sex
Female 87
Male 157
dtype: int64

```

请注意，在我们使用的pandas代码中`size()`，没有 `count()`。这是因为 `count()` 将函数应用于每个列，返回每个列中的记录数。 `not null`

```

In [18]: tips.groupby('sex').count()
Out[18]:
 total_bill tip smoker day time size
sex
Female 87 87 87 87 87 87
Male 157 157 157 157 157 157

```

或者，我们可以将该`count()`方法应用于单个列：

```

In [19]: tips.groupby('sex')['total_bill'].count()
Out[19]:
sex
Female 87
Male 157
Name: total_bill, dtype: int64

```

也可以一次应用多个功能。例如，假设我们希望查看提示量与星期几的不同之处 - `agg()` 允许您将字典传递给分组的DataFrame，指示要应用于特定列的函数。

```

SELECT day, AVG(tip), COUNT(*)
FROM tips
GROUP BY day;
/*
Fri 2.734737 19
Sat 2.993103 87
Sun 3.255132 76
Thur 2.771452 62
*/

```

```
In [20]: tips.groupby('day').agg({'tip': np.mean, 'day': np.size})
Out[20]:
```

	tip	day
day		
Fri	2.734737	19
Sat	2.993103	87
Sun	3.255132	76
Thur	2.771452	62

通过将列列表传递给`groupby()`方法来完成多个列的分组。

```
SELECT smoker, day, COUNT(*), AVG(tip)
FROM tips
GROUP BY smoker, day;
/*
smoker day
No Fri 4 2.812500
 Sat 45 3.102889
 Sun 57 3.167895
 Thur 45 2.673778
Yes Fri 15 2.714000
 Sat 42 2.875476
 Sun 19 3.516842
 Thur 17 3.030000
*/
```

```
In [21]: tips.groupby(['smoker', 'day']).agg({'tip': [np.size, np.mean]})
Out[21]:
```

		tip	
		size	mean
smoker	day		
No	Fri	4.0	2.812500
	Sat	45.0	3.102889
	Sun	57.0	3.167895
	Thur	45.0	2.673778
Yes	Fri	15.0	2.714000
	Sat	42.0	2.875476
	Sun	19.0	3.516842
	Thur	17.0	3.030000

## JOIN

可以使用`join()`或执行`JOIN merge()`。默认情况下，`join()`将在其索引上加入DataFrame。每个方法都有参数，允许您指定要执行的连接类型（LEFT，RIGHT，INNER，FULL）或要连接的列（列名称或索引）。

```
In [22]: df1 = pd.DataFrame({'key': ['A', 'B', 'C', 'D'],
....: 'value': np.random.randn(4)})
....:

In [23]: df2 = pd.DataFrame({'key': ['B', 'D', 'D', 'E'],
....: 'value': np.random.randn(4)})
....:
```

假设我们有两个与DataFrames名称和结构相同的数据库表。

现在让我们来看看各种类型的JOIN。

## INNER JOIN

```
SELECT *
FROM df1
INNER JOIN df2
ON df1.key = df2.key;
```

```
merge performs an INNER JOIN by default
In [24]: pd.merge(df1, df2, on='key')
Out[24]:
 key value_x value_y
0 B -0.282863 1.212112
1 D -1.135632 -0.173215
2 D -1.135632 0.119209
```

`merge()` 当您想要将一个DataFrame列与另一个DataFrame索引连接时，还会为这些情况提供参数。

```
In [25]: indexed_df2 = df2.set_index('key')

In [26]: pd.merge(df1, indexed_df2, left_on='key', right_index=True)
Out[26]:
 key value_x value_y
1 B -0.282863 1.212112
3 D -1.135632 -0.173215
3 D -1.135632 0.119209
```

## LEFT OUTER JOIN

```
-- show all records from df1
SELECT *
FROM df1
LEFT OUTER JOIN df2
ON df1.key = df2.key;
```

```
show all records from df1
In [27]: pd.merge(df1, df2, on='key', how='left')
Out[27]:
```

	key	value_x	value_y
0	A	0.469112	NaN
1	B	-0.282863	1.212112
2	C	-1.509059	NaN
3	D	-1.135632	-0.173215
4	D	-1.135632	0.119209

## RIGHT JOIN

```
-- show all records from df2
SELECT *
FROM df1
RIGHT OUTER JOIN df2
ON df1.key = df2.key;
```

```
show all records from df2
In [28]: pd.merge(df1, df2, on='key', how='right')
Out[28]:
```

	key	value_x	value_y
0	B	-0.282863	1.212112
1	D	-1.135632	-0.173215
2	D	-1.135632	0.119209
3	E	NaN	-1.044236

## FULL JOIN

pandas还允许显示数据集两侧的FULL JOIN，无论连接列是否找到匹配项。在编写时，所有RDBMS（MySQL）都不支持FULL JOIN。

```
-- show all records from both tables
SELECT *
FROM df1
FULL OUTER JOIN df2
ON df1.key = df2.key;
```

```
show all records from both frames
In [29]: pd.merge(df1, df2, on='key', how='outer')
Out[29]:
```

	key	value_x	value_y
0	A	0.469112	NaN
1	B	-0.282863	1.212112
2	C	-1.509059	NaN
3	D	-1.135632	-0.173215
4	D	-1.135632	0.119209
5	E	NaN	-1.044236

## UNION

UNION ALL可以使用`concat()`。

```
In [30]: df1 = pd.DataFrame({'city': ['Chicago', 'San Francisco', 'New York City'],
.....: 'rank': range(1, 4)})
.....:

In [31]: df2 = pd.DataFrame({'city': ['Chicago', 'Boston', 'Los Angeles'],
.....: 'rank': [1, 4, 5]})
.....:
```

```
SELECT city, rank
FROM df1
UNION ALL
SELECT city, rank
FROM df2;
/*
 city rank
Chicago 1
San Francisco 2
New York City 3
 Chicago 1
 Boston 4
 Los Angeles 5
*/
```

```
In [32]: pd.concat([df1, df2])
Out[32]:
 city rank
0 Chicago 1
1 San Francisco 2
2 New York City 3
0 Chicago 1
1 Boston 4
2 Los Angeles 5
```

SQL的UNION类似于UNION ALL，但是UNION将删除重复的行。

```

SELECT city, rank
FROM df1
UNION
SELECT city, rank
FROM df2;
-- notice that there is only one Chicago record this time
/*
 city rank
 Chicago 1
San Francisco 2
New York City 3
 Boston 4
 Los Angeles 5
*/

```

在 pandas 中，您可以 `concat()` 结合使用 `drop_duplicates()`。

```

In [33]: pd.concat([df1, df2]).drop_duplicates()
Out[33]:
 city rank
0 Chicago 1
1 San Francisco 2
2 New York City 3
1 Boston 4
2 Los Angeles 5

```

## Pandas 等同于某些 SQL 分析和聚合函数

### 带有偏移量的前 N 行

```

-- MySQL
SELECT * FROM tips
ORDER BY tip DESC
LIMIT 10 OFFSET 5;

```

```

In [34]: tips.nlargest(10 + 5, columns='tip').tail(10)
Out[34]:
 total_bill tip sex smoker day time size
183 23.17 6.50 Male Yes Sun Dinner 4
214 28.17 6.50 Female Yes Sat Dinner 3
47 32.40 6.00 Male No Sun Dinner 4
239 29.03 5.92 Male No Sat Dinner 3
88 24.71 5.85 Male No Thur Lunch 2
181 23.33 5.65 Male Yes Sun Dinner 2
44 30.40 5.60 Male No Sun Dinner 4
52 34.81 5.20 Female No Sun Dinner 4
85 34.83 5.17 Female No Thur Lunch 4
211 25.89 5.16 Male Yes Sat Dinner 4

```



## 每组前N行

```
-- Oracle's ROW_NUMBER() analytic function
SELECT * FROM (
 SELECT
 t.*,
 ROW_NUMBER() OVER(PARTITION BY day ORDER BY total_bill DESC) AS rn
 FROM tips t
)
WHERE rn < 3
ORDER BY day, rn;
```

```
In [35]: (tips.assign(rn=tips.sort_values(['total_bill'], ascending=False)
....: .groupby(['day'])
....: .cumcount() + 1)
....: .query('rn < 3')
....: .sort_values(['day', 'rn']))
Out[35]:
```

	total_bill	tip	sex	smoker	day	time	size	rn
95	40.17	4.73	Male	Yes	Fri	Dinner	4	1
90	28.97	3.00	Male	Yes	Fri	Dinner	2	2
170	50.81	10.00	Male	Yes	Sat	Dinner	3	1
212	48.33	9.00	Male	No	Sat	Dinner	4	2
156	48.17	5.00	Male	No	Sun	Dinner	6	1
182	45.35	3.50	Male	Yes	Sun	Dinner	3	2
197	43.11	5.00	Female	Yes	Thur	Lunch	4	1
142	41.19	5.00	Male	No	Thur	Lunch	5	2

同样使用 rank (method='first') 函数

```
In [36]: (tips.assign(rnk=tips.groupby(['day'])['total_bill']
....: .rank(method='first', ascending=False))
....: .query('rnk < 3')
....: .sort_values(['day', 'rnk']))
Out[36]:
```

	total_bill	tip	sex	smoker	day	time	size	rnk
95	40.17	4.73	Male	Yes	Fri	Dinner	4	1.0
90	28.97	3.00	Male	Yes	Fri	Dinner	2	2.0
170	50.81	10.00	Male	Yes	Sat	Dinner	3	1.0
212	48.33	9.00	Male	No	Sat	Dinner	4	2.0
156	48.17	5.00	Male	No	Sun	Dinner	6	1.0
182	45.35	3.50	Male	Yes	Sun	Dinner	3	2.0
197	43.11	5.00	Female	Yes	Thur	Lunch	4	1.0
142	41.19	5.00	Male	No	Thur	Lunch	5	2.0

```
-- Oracle's RANK() analytic function
SELECT * FROM (
 SELECT
 t.*,
 RANK() OVER(PARTITION BY sex ORDER BY tip) AS rnk
 FROM tips t
 WHERE tip < 2
)
WHERE rnk < 3
ORDER BY sex, rnk;
```

让我们找到每个性别组（等级<3）的提示（提示<2）。请注意，使用 `rank(method='min')` 函数时 `rnk_min` 对于相同的提示保持不变（如Oracle的RANK（）函数）

```
In [37]: (tips[tips['tip'] < 2]
....: .assign(rnk_min=tips.groupby(['sex'])['tip']
....: .rank(method='min'))
....: .query('rnk_min < 3')
....: .sort_values(['sex', 'rnk_min']))
....:
Out[37]:
```

	total_bill	tip	sex	smoker	day	time	size	rnk_min
67	3.07	1.00	Female	Yes	Sat	Dinner	1	1.0
92	5.75	1.00	Female	Yes	Fri	Dinner	2	1.0
111	7.25	1.00	Female	No	Sat	Dinner	1	1.0
236	12.60	1.00	Male	Yes	Sat	Dinner	2	1.0
237	32.83	1.17	Male	Yes	Sat	Dinner	2	2.0

## 更新（UPDATE）

```
UPDATE tips
SET tip = tip*2
WHERE tip < 2;
```

```
In [38]: tips.loc[tips['tip'] < 2, 'tip'] *= 2
```

## 删除（DELETE）

```
DELETE FROM tips
WHERE tip > 9;
```

在pandas中，我们选择应保留的行，而不是删除它们

```
In [39]: tips = tips.loc[tips['tip'] <= 9]
```

## 与SAS的比较

对于来自 [SAS](#) 的潜在用户，本节旨在演示如何在 pandas 中做各种类似SAS的操作。

由于许多潜在的 pandas 用户对[SQL](#)有一定的了解，因此本页面旨在提供一些使用 pandas 如何执行各种 SQL操作的示例。

如果您是 pandas 的新手，您可能需要先阅读[十分钟入门Pandas](#) 以熟悉本库。

按照惯例，我们按如下方式导入 pandas 和 NumPy：

```
In [1]: import pandas as pd

In [2]: import numpy as np
```

⋮ tip 注意

在本教程中，`DataFrame` 将通过调用显示 `pandas df.head()`，它将显示该行的前N行（默认为5行）`DataFrame`。这通常用于交互式工作（例如[Jupyter笔记本](#)或终端） - SAS中的等价物将是：

```
proc print data=df(obs=5);
run;
```

⋮

## 数据结构

### 一般术语对照表

Pandas	SAS
DataFrame	数据集（data set）
column	变量（variable）
row	观察（observation）
groupby	BY-group
NaN	.

### DataFrame/ Series

A `DataFrame` pandas类似于SAS数据集 - 具有标记列的二维数据源，可以是不同类型的。如本文档所示，几乎所有可以使用SAS `DATA` 步骤应用于数据集的操作也可以在pandas中完成。

A `Series` 是表示a的一列的数据结构 `DataFrame`。SAS没有针对单个列的单独数据结构，但通常，使用a `Series` 类似于在 `DATA` 步骤中引用列。

## Index

每一个 `DataFrame` 和 `Series` 有一个 `Index` -这是对标签 的行数据。SAS没有完全类似的概念。除了在 `DATA step` (`_N_`) 期间可以访问的隐式整数索引之外，数据集的行基本上是无标记的。

在pandas中，如果未指定索引，则默认情况下也使用整数索引（第一行= 0，第二行= 1，依此类推）。虽然使用标记 `Index` 或 `MultiIndex` 可以启用复杂的分析，并且最终是 Pandas 理解的重要部分，但是对于这种比较，我们基本上会忽略它，`Index` 并且只是将其 `DataFrame` 视为列的集合。有关如何有效使用的更多信息，请参阅[索引文档](#) `Index`。

## 数据输入/输出

### 从值构造DataFrame

通过将数据放在 `datalines` 语句之后并指定列名，可以从指定值构建SAS数据集。

```
data df;
 input x y;
 datalines;
1 2
3 4
5 6
;
run;
```

`DataFrame` 可以用许多不同的方式构造一个pandas，但是对于少量的值，通常很方便将它指定为Python字典，其中键是列名，值是数据。

```
In [3]: df = pd.DataFrame({'x': [1, 3, 5], 'y': [2, 4, 6]})

In [4]: df
Out[4]:
 x y
0 1 2
1 3 4
2 5 6
```

### 读取外部数据

与SAS一样，pandas提供了从多种格式读取数据的实用程序。`tips` 在pandas测试 ([csv](#)) 中找到的数据集将用于以下许多示例中。

SAS提供将csv数据读入数据集。 `PROC IMPORT`

```
proc import datafile='tips.csv' dbms=csv out=tips replace;
 getnames=yes;
run;
```

Pandas 方法是`read_csv()`类似的。

```
In [5]: url = ('https://raw.githubusercontent.com/pandas-dev/
...: 'pandas/master/pandas/tests/data/tips.csv')
...:

In [6]: tips = pd.read_csv(url)

In [7]: tips.head()
Out[7]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

比如，可以使用许多参数来指定数据应该如何解析。例如，如果数据是由制表符分隔的，并且没有列名，那么pandas命令将是：`PROC IMPORT` ``read\_csv`

```
tips = pd.read_csv('tips.csv', sep='\t', header=None)

alternatively, read_table is an alias to read_csv with tab delimiter
tips = pd.read_table('tips.csv', header=None)
```

除了text / csv之外，pandas还支持各种其他数据格式，如Excel，HDF5和SQL数据库。这些都是通过`pd.read_*` 函数读取的。有关更多详细信息，请参阅[IO文档](#)。

## 导出数据

在SAS中 `proc` 导入 相反就是 `proc` 导出

```
proc export data=tips outfile='tips2.csv' dbms=csv;
run;
```

类似地，在 Pandas ，相反 `read_csv` 是`to_csv()`，与其他的数据格式遵循类似的API。

```
tips.to_csv('tips2.csv')
```

## 数据操作

### 列上的操作

在该 DATA 步骤中，可以在新列或现有列上使用任意数学表达式。

```
data tips;
 set tips;
 total_bill = total_bill - 2;
 new_bill = total_bill / 2;
run;
```

pandas 通过指定个体提供了类似的矢量化操作 Series 中 DataFrame。可以以相同的方式分配新列。

```
In [8]: tips['total_bill'] = tips['total_bill'] - 2

In [9]: tips['new_bill'] = tips['total_bill'] / 2.0

In [10]: tips.head()
Out[10]:
```

	total_bill	tip	sex	smoker	day	time	size	new_bill
0	14.99	1.01	Female	No	Sun	Dinner	2	7.495
1	8.34	1.66	Male	No	Sun	Dinner	3	4.170
2	19.01	3.50	Male	No	Sun	Dinner	3	9.505
3	21.68	3.31	Male	No	Sun	Dinner	2	10.840
4	22.59	3.61	Female	No	Sun	Dinner	4	11.295

## 过滤

SAS中的过滤是通过一个或多个列上的 if 或 where 语句完成的。

```
data tips;
 set tips;
 if total_bill > 10;
run;

data tips;
 set tips;
 where total_bill > 10;
 /* equivalent in this case - where happens before the
 DATA step begins and can also be used in PROC statements */
run;
```

DataFrame可以通过多种方式进行过滤; 最直观的是使用 [布尔索引](#)

```
In [11]: tips[tips['total_bill'] > 10].head()
Out[11]:
```

	total_bill	tip	sex	smoker	day	time	size
0	14.99	1.01	Female	No	Sun	Dinner	2
2	19.01	3.50	Male	No	Sun	Dinner	3
3	21.68	3.31	Male	No	Sun	Dinner	2
4	22.59	3.61	Female	No	Sun	Dinner	4
5	23.29	4.71	Male	No	Sun	Dinner	4

## 如果/那么逻辑

在SAS中，if / then逻辑可用于创建新列。

```
data tips;
 set tips;
 format bucket $4.;

 if total_bill < 10 then bucket = 'low';
 else bucket = 'high';
run;
```

Pandas 中的相同操作可以使用 `where` 来自的方法来完成 `numpy`。

```
In [12]: tips['bucket'] = np.where(tips['total_bill'] < 10, 'low', 'high')

In [13]: tips.head()
Out[13]:
```

	total_bill	tip	sex	smoker	day	time	size	bucket
0	14.99	1.01	Female	No	Sun	Dinner	2	high
1	8.34	1.66	Male	No	Sun	Dinner	3	low
2	19.01	3.50	Male	No	Sun	Dinner	3	high
3	21.68	3.31	Male	No	Sun	Dinner	2	high
4	22.59	3.61	Female	No	Sun	Dinner	4	high

## 日期功能

SAS提供了各种功能来对日期/日期时间列进行操作。

```
data tips;
 set tips;
 format date1 date2 date1_plusmonth mmddyy10.;
 date1 = mdy(1, 15, 2013);
 date2 = mdy(2, 15, 2015);
 date1_year = year(date1);
 date2_month = month(date2);
 * shift date to beginning of next interval;
 date1_next = intnx('MONTH', date1, 1);
 * count intervals between dates;
 months_between = intck('MONTH', date1, date2);
run;
```

等效的pandas操作如下所示。除了这些功能外，pandas还支持Base SAS中不具备的其他时间序列功能（例如重新采样和自定义偏移） - 有关详细信息，请参阅[时间序列文档](#)。

```

In [14]: tips['date1'] = pd.Timestamp('2013-01-15')

In [15]: tips['date2'] = pd.Timestamp('2015-02-15')

In [16]: tips['date1_year'] = tips['date1'].dt.year

In [17]: tips['date2_month'] = tips['date2'].dt.month

In [18]: tips['date1_next'] = tips['date1'] + pd.offsets.MonthBegin()

In [19]: tips['months_between'] = (
.....: tips['date2'].dt.to_period('M') - tips['date1'].dt.to_period('M'))
.....:

In [20]: tips[['date1', 'date2', 'date1_year', 'date2_month',
.....: 'date1_next', 'months_between']].head()
.....:
Out[20]:
 date1 date2 date1_year date2_month date1_next months_between
0 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>
1 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>
2 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>
3 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>
4 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>

```

## 列的选择

SAS在 DATA 步骤中提供关键字以选择，删除和重命名列。

```

data tips;
 set tips;
 keep sex total_bill tip;
run;

data tips;
 set tips;
 drop sex;
run;

data tips;
 set tips;
 rename total_bill=total_bill_2;
run;

```

下面的 Pandas 表示相同的操作。



```
keep
In [21]: tips[['sex', 'total_bill', 'tip']].head()
Out[21]:
 sex total_bill tip
0 Female 14.99 1.01
1 Male 8.34 1.66
2 Male 19.01 3.50
3 Male 21.68 3.31
4 Female 22.59 3.61

drop
In [22]: tips.drop('sex', axis=1).head()
Out[22]:
 total_bill tip smoker day time size
0 14.99 1.01 No Sun Dinner 2
1 8.34 1.66 No Sun Dinner 3
2 19.01 3.50 No Sun Dinner 3
3 21.68 3.31 No Sun Dinner 2
4 22.59 3.61 No Sun Dinner 4

rename
In [23]: tips.rename(columns={'total_bill': 'total_bill_2'}).head()
Out[23]:
 total_bill_2 tip sex smoker day time size
0 14.99 1.01 Female No Sun Dinner 2
1 8.34 1.66 Male No Sun Dinner 3
2 19.01 3.50 Male No Sun Dinner 3
3 21.68 3.31 Male No Sun Dinner 2
4 22.59 3.61 Female No Sun Dinner 4
```

## 按值排序

SAS中的排序是通过 PROC SORT

```
proc sort data=tips;
 by sex total_bill;
run;
```

pandas对象有一个`sort_values()`方法，它采用列表进行排序。

```
In [24]: tips = tips.sort_values(['sex', 'total_bill'])

In [25]: tips.head()
Out[25]:
 total_bill tip sex smoker day time size
67 1.07 1.00 Female Yes Sat Dinner 1
92 3.75 1.00 Female Yes Fri Dinner 2
111 5.25 1.00 Female No Sat Dinner 1
145 6.35 1.50 Female No Thur Lunch 2
135 6.51 1.25 Female No Thur Lunch 2
```

## 字符串处理

### 长度

SAS使用`LENGTHN` 和`LENGTHC` 函数确定字符串的长度。 `LENGTHN` 排除尾随空白并 `LENGTHC` 包括尾随空白。

```
data _null_;
set tips;
put(LENGTHN(time));
put(LENGTHC(time));
run;
```

Python使用该 `len` 函数确定字符串的长度。 `len` 包括尾随空白。使用 `len` 和 `rstrip` 排除尾随空格。

```
In [26]: tips['time'].str.len().head()
Out[26]:
67 6
92 6
111 6
145 5
135 5
Name: time, dtype: int64

In [27]: tips['time'].str.rstrip().str.len().head()
Out[27]:
67 6
92 6
111 6
145 5
135 5
Name: time, dtype: int64
```

### 查找 (Find)

SAS使用`FINDW`函数确定字符串中字符的位置。 `FINDW` 获取第一个参数定义的字符串，并搜索您提供的子字符串的第一个位置作为第二个参数。

```
data _null_;
set tips;
put(FINDW(sex, 'ale'));
run;
```

Python使用 `find` 函数确定字符串中字符的位置。 `find` 搜索子字符串的第一个位置。如果找到子字符串，则该函数返回其位置。请记住，Python索引是从零开始的，如果找不到子串，函数将返回-1。

```
In [28]: tips['sex'].str.find("ale").head()
Out[28]:
67 3
92 3
111 3
145 3
135 3
Name: sex, dtype: int64
```

## 字符串提取 (Substring)

SAS使用SUBSTR函数根据其位置从字符串中提取子字符串。

```
data _null_;
set tips;
put(substr(sex,1,1));
run;
```

使用pandas，您可以使用 [] 符号从位置位置提取字符串中的子字符串。请记住，Python索引是从零开始的。

```
In [29]: tips['sex'].str[0:1].head()
Out[29]:
67 F
92 F
111 F
145 F
135 F
Name: sex, dtype: object
```

## SCAN

SAS SCAN 函数返回字符串中的第n个字。第一个参数是要解析的字符串，第二个参数指定要提取的字。

```
data firstlast;
input String $60.;
First_Name = scan(string, 1);
Last_Name = scan(string, -1);
datalines2;
John Smith;
Jane Cook;
;;;
run;
```

Python使用正则表达式根据文本从字符串中提取子字符串。有更强大的方法，但这只是一个简单的方法。

```
In [30]: firstlast = pd.DataFrame({'String': ['John Smith', 'Jane Cook']})

In [31]: firstlast['First_Name'] = firstlast['String'].str.split(" ", expand=True)[0]

In [32]: firstlast['Last_Name'] = firstlast['String'].str.rsplit(" ", expand=True)[0]

In [33]: firstlast
Out[33]:
 String First_Name Last_Name
0 John Smith John John
1 Jane Cook Jane Jane
```

## 大写，小写和特殊转换

SAS `UPCASE` `LOWCASE`和 `PROPCASE` 函数改变了参数的大小写。

```
data firstlast;
input String $60.;
string_up = UPCASE(string);
string_low = LOWCASE(string);
string_prop = PROPCASE(string);
datalines2;
John Smith;
Jane Cook;
;;;
run;
```

等效Python的功能 `upper` , `lower` 和 `title`。

```
In [34]: firstlast = pd.DataFrame({'String': ['John Smith', 'Jane Cook']})

In [35]: firstlast['string_up'] = firstlast['String'].str.upper()

In [36]: firstlast['string_low'] = firstlast['String'].str.lower()

In [37]: firstlast['string_prop'] = firstlast['String'].str.title()

In [38]: firstlast
Out[38]:
 String string_up string_low string_prop
0 John Smith JOHN SMITH john smith John Smith
1 Jane Cook JANE COOK jane cook Jane Cook
```

## 合并 (Merging)

合并示例中将使用以下表格

```
In [39]: df1 = pd.DataFrame({'key': ['A', 'B', 'C', 'D'],
.....: 'value': np.random.randn(4)})
.....:
```

```
In [40]: df1
Out[40]:
```

	key	value
0	A	0.469112
1	B	-0.282863
2	C	-1.509059
3	D	-1.135632

```
In [41]: df2 = pd.DataFrame({'key': ['B', 'D', 'D', 'E'],
.....: 'value': np.random.randn(4)})
.....:
```

```
In [42]: df2
```

```
Out[42]:
```

	key	value
0	B	1.212112
1	D	-0.173215
2	D	0.119209
3	E	-1.044236

在SAS中，必须在合并之前显式排序数据。使用 `in=` 虚拟变量来跟踪是否在一个或两个输入帧中找到匹配来完成不同类型的连接。

```
proc sort data=df1;
 by key;
run;

proc sort data=df2;
 by key;
run;

data left_join inner_join right_join outer_join;
 merge df1(in=a) df2(in=b);

 if a and b then output inner_join;
 if a then output left_join;
 if b then output right_join;
 if a or b then output outer_join;
run;
```

pandas DataFrames有一个`merge()`提供类似功能的方法。请注意，数据不必提前排序，并且通过 `how` 关键字可以实现不同的连接类型。

```
In [43]: inner_join = df1.merge(df2, on=['key'], how='inner')
```

```
In [44]: inner_join
```

```
Out[44]:
```

	key	value_x	value_y
0	B	-0.282863	1.212112
1	D	-1.135632	-0.173215
2	D	-1.135632	0.119209

```
In [45]: left_join = df1.merge(df2, on=['key'], how='left')
```

```
In [46]: left_join
```

```
Out[46]:
```

	key	value_x	value_y
0	A	0.469112	NaN
1	B	-0.282863	1.212112
2	C	-1.509059	NaN
3	D	-1.135632	-0.173215
4	D	-1.135632	0.119209

```
In [47]: right_join = df1.merge(df2, on=['key'], how='right')
```

```
In [48]: right_join
```

```
Out[48]:
```

	key	value_x	value_y
0	B	-0.282863	1.212112
1	D	-1.135632	-0.173215
2	D	-1.135632	0.119209
3	E	NaN	-1.044236

```
In [49]: outer_join = df1.merge(df2, on=['key'], how='outer')
```

```
In [50]: outer_join
```

```
Out[50]:
```

	key	value_x	value_y
0	A	0.469112	NaN
1	B	-0.282863	1.212112
2	C	-1.509059	NaN
3	D	-1.135632	-0.173215
4	D	-1.135632	0.119209
5	E	NaN	-1.044236

## 缺失数据 (Missing data)

与SAS一样，pandas具有丢失数据的表示 - 这是特殊浮点值 `NaN`（不是数字）。许多语义都是相同的，例如，丢失的数据通过数字操作传播，默认情况下会被聚合忽略。

```

In [51]: outer_join
Out[51]:
 key value_x value_y
0 A 0.469112 NaN
1 B -0.282863 1.212112
2 C -1.509059 NaN
3 D -1.135632 -0.173215
4 D -1.135632 0.119209
5 E NaN -1.044236

In [52]: outer_join['value_x'] + outer_join['value_y']
Out[52]:
0 NaN
1 0.929249
2 NaN
3 -1.308847
4 -1.016424
5 NaN
dtype: float64

In [53]: outer_join['value_x'].sum()
Out[53]: -3.5940742896293765

```

一个区别是丢失的数据无法与其哨兵值进行比较。例如，在SAS中，您可以执行此操作以过滤缺失值。

```

data outer_join_nulls;
 set outer_join;
 if value_x = .;
run;

data outer_join_no_nulls;
 set outer_join;
 if value_x ^= .;
run;

```

这在 Pandas 中不起作用。相反，应使用 `pd.isna` 或 `pd.notna` 函数进行比较。

```

In [54]: outer_join[pd.isna(outer_join['value_x'])]
Out[54]:
 key value_x value_y
5 E NaN -1.044236

In [55]: outer_join[pd.notna(outer_join['value_x'])]
Out[55]:
 key value_x value_y
0 A 0.469112 NaN
1 B -0.282863 1.212112
2 C -1.509059 NaN
3 D -1.135632 -0.173215
4 D -1.135632 0.119209

```

pandas还提供了各种方法来处理丢失的数据 - 其中一些方法在SAS中表达起来很有挑战性。例如，有一些方法可以删除具有任何缺失值的所有行，使用指定值替换缺失值，例如平均值或前一行的前向填充。看到[丢失的数据文件](#)为多。

```
In [56]: outer_join.dropna()
Out[56]:
 key value_x value_y
1 B -0.282863 1.212112
3 D -1.135632 -0.173215
4 D -1.135632 0.119209

In [57]: outer_join.fillna(method='ffill')
Out[57]:
 key value_x value_y
0 A 0.469112 NaN
1 B -0.282863 1.212112
2 C -1.509059 1.212112
3 D -1.135632 -0.173215
4 D -1.135632 0.119209
5 E -1.135632 -1.044236

In [58]: outer_join['value_x'].fillna(outer_join['value_x'].mean())
Out[58]:
0 0.469112
1 -0.282863
2 -1.509059
3 -1.135632
4 -1.135632
5 -0.718815
Name: value_x, dtype: float64
```

## GroupBy

### 聚合 (Aggregation)

SAS的PROC SUMMARY可用于按一个或多个关键变量进行分组，并计算数字列上的聚合。

```
proc summary data=tips nway;
 class sex smoker;
 var total_bill tip;
 output out=tips_summed sum=;
run;
```

pandas提供了一种 `groupby` 允许类似聚合的灵活机制。有关 更多详细信息和示例，请参阅[groupby文档](#)。



```
In [59]: tips_summed = tips.groupby(['sex', 'smoker'])['total_bill', 'tip'].sum()
```

```
In [60]: tips_summed.head()
```

```
Out[60]:
```

		total_bill	tip
sex	smoker		
Female	No	869.68	149.77
	Yes	527.27	96.74
Male	No	1725.75	302.00
	Yes	1217.07	183.07

## 转换 (Transformation)

在SAS中，如果组聚合需要与原始帧一起使用，则必须将它们合并在一起。例如，减去吸烟者组每次观察的平均值。

```
proc summary data=tips missing nway;
 class smoker;
 var total_bill;
 output out=smoker_means mean(total_bill)=group_bill;
run;

proc sort data=tips;
 by smoker;
run;

data tips;
 merge tips(in=a) smoker_means(in=b);
 by smoker;
 adj_total_bill = total_bill - group_bill;
 if a and b;
run;
```

pandas `groupby` 提供了一种 `transform` 机制，允许在一个操作中简洁地表达这些类型的操作。

```
In [61]: gb = tips.groupby('smoker')['total_bill']
```

```
In [62]: tips['adj_total_bill'] = tips['total_bill'] - gb.transform('mean')
```

```
In [63]: tips.head()
```

```
Out[63]:
```

	total_bill	tip	sex	smoker	day	time	size	adj_total_bill
67	1.07	1.00	Female	Yes	Sat	Dinner	1	-17.686344
92	3.75	1.00	Female	Yes	Fri	Dinner	2	-15.006344
111	5.25	1.00	Female	No	Sat	Dinner	1	-11.938278
145	6.35	1.50	Female	No	Thur	Lunch	2	-10.838278
135	6.51	1.25	Female	No	Thur	Lunch	2	-10.678278

## 按组处理

除了聚合之外，`groupby` 还可以使用pandas 通过SAS的组处理来复制大多数其他pandas 。例如，此 DATA 步骤按性别/吸烟者组读取数据，并过滤到每个的第一个条目。

```
proc sort data=tips;
 by sex smoker;
run;

data tips_first;
 set tips;
 by sex smoker;
 if FIRST.sex or FIRST.smoker then output;
run;
```

在 Pandas 中，这将写成：

```
In [64]: tips.groupby(['sex', 'smoker']).first()
Out[64]:
```

		total_bill	tip	day	time	size	adj_total_bill
sex	smoker						
Female	No	5.25	1.00	Sat	Dinner	1	-11.938278
	Yes	1.07	1.00	Sat	Dinner	1	-17.686344
Male	No	5.51	2.00	Thur	Lunch	2	-11.678278
	Yes	5.25	5.15	Sun	Dinner	2	-13.506344

## 其他注意事项

### 磁盘与内存

pandas仅在内存中运行，其中SAS数据集存在于磁盘上。这意味着可以在pandas中加载的数据大小受机器内存的限制，但对数据的操作可能更快。

如果需要进行核心处理，一种可能性是 `dask.dataframe` 库（目前正在开发中），它为磁盘上的pandas功能提供了一个子集 `DataFrame`

### 数据互操作

pandas提供了一种`read_sas()`方法，可以读取以XPORT或SAS7BDAT二进制格式保存的SAS数据。

```
libname xportout xport 'transport-file.xpt';
data xportout.tips;
 set tips(rename=(total_bill=tbill));
 * xport variable names limited to 6 characters;
run;
```

```
df = pd.read_sas('transport-file.xpt')
df = pd.read_sas('binary-file.sas7bdat')
```

您也可以直接指定文件格式。默认情况下，pandas将尝试根据其扩展名推断文件格式。

```
df = pd.read_sas('transport-file.xpt', format='xport')
df = pd.read_sas('binary-file.sas7bdat', format='sas7bdat')
```

XPORT是一种相对有限的格式，它的解析并不像其他一些pandas读者那样优化。在SAS和pandas之间交换数据的另一种方法是序列化为csv。

```
version 0.17, 10M rows

In [8]: %time df = pd.read_sas('big.xpt')
Wall time: 14.6 s

In [9]: %time df = pd.read_csv('big.csv')
Wall time: 4.86 s
```

## 与Stata的比较

对于来自 [Stata](#) 的潜在用户，本节旨在演示如何在 pandas 中做各种类似Stata的操作。

如果您是 pandas 的新手，您可能需要先阅读[十分钟入门Pandas](#)以熟悉本库。

按照惯例，我们按如下方式导入 pandas 和 NumPy：

```
In [1]: import pandas as pd

In [2]: import numpy as np
```

⋮ tip 注意

在本教程中，`DataFrame` 将通过调用显示 pandas `df.head()`，它将显示该行的前N行（默认为5行）`DataFrame`。这通常用于交互式工作（例如[Jupyter笔记本](#)或终端） - Stata中的等价物将是：

```
list in 1/5
```

⋮

# 数据结构

## 一般术语对照表

Pandas	Stata
DataFrame	数据集（data set）
column	变量（variable）
row	观察（observation）
groupby	bysort
NaN	.

### DataFrame/ Series

pandas 中的 `DataFrame` 类似于 `Stata` 数据集-具有不同类型的标记列的二维数据源。如本文档所示，几乎任何可以应用于 `Stata` 中的数据集的操作也可以在 `pandas` 中完成。

`Series` 是表示 `DataFrame` 的一列的数据结构。`Stata` 对于单个列没有单独的数据结构，但是通常，使用 `Series` 类似于引用 `Stata` 中的数据集的列。

### Index

每个 `DataFrame` 和 `Series` 在数据行上都有一个叫 `Index-label` 的标签。在 `Stata` 中没有相似的概念。在 `Stata` 中，数据集的行基本上是无标签的，除了可以用 `_n` 访问的隐式整数索引。

在 `pandas` 中，如果未指定索引，则默认情况下也使用整数索引（第一行= 0，第二行= 1，依此类推）。虽然使用标记 `Index` 或 `MultiIndex` 可以启用复杂的分析，并且最终是 `pandas` 理解的重要部分，但是对于这种比较，我们基本上会忽略它，`Index` 并且只是将其 `DataFrame` 视为列的集合。有关如何有效使用的更多信息，请参阅[索引文档](#) `Index`。

# 数据输入/输出

## 从价值观构建数据帧

通过将数据放在 `input` 语句之后并指定列名，可以从指定值构建 `Stata` 数据集。

```
input x y
1 2
3 4
5 6
end
```

`pandas` 的 `DataFrame` 可以用许多不同的方式构建，但对于少量的值，通常可以方便地将其指定为 `Python` 字典，其中键是列名，值是数据。

```
In [3]: df = pd.DataFrame({'x': [1, 3, 5], 'y': [2, 4, 6]})

In [4]: df
Out[4]:
 x y
0 1 2
1 3 4
2 5 6
```

## 读取外部数据

与Stata一样，pandas提供了从多种格式读取数据的实用程序。tips 在pandas测试 (csv) 中找到的数据集将用于以下许多示例中。

Stata提供将csv数据读入内存中的数据集。如果文件在当前工作目录中，我们可以按如下方式导入它。

```
import delimited ``tips.csv
```

```
import delimited tips.csv
```

pandas 方法是read\_csv()类似的。此外，如果提供了网址，它将自动下载数据集。

```
In [5]: url = ('https://raw.githubusercontent.com/pandas-dev/pandas/master/pandas/tests/data/tips.csv')
...:
...:

In [6]: tips = pd.read_csv(url)

In [7]: tips.head()
Out[7]:
 total_bill tip sex smoker day time size
0 16.99 1.01 Female No Sun Dinner 2
1 10.34 1.66 Male No Sun Dinner 3
2 21.01 3.50 Male No Sun Dinner 3
3 23.68 3.31 Male No Sun Dinner 2
4 24.59 3.61 Female No Sun Dinner 4
```

比如，可以使用许多参数来指定数据应该如何解析。例如，如果数据是由制表符分隔的，没有列名，并且存在于当前工作目录中，则pandas命令将为：import delimited read\_csv()

```
tips = pd.read_csv('tips.csv', sep='\t', header=None)

alternatively, read_table is an alias to read_csv with tab delimiter
tips = pd.read_table('tips.csv', header=None)
```

pandas 还可以用于 .dta 的文件格式中。使用read\_stata()函数读取格式的Stata数据集。

```
df = pd.read_stata('data.dta')
```

除了text / csv和Stata文件之外，pandas还支持各种其他数据格式，如Excel，SAS，HDF5，Parquet和SQL数据库。这些都是通过 `pd.read_*` 函数读取的。有关更多详细信息，请参阅[IO文档](#)。

## 导出数据

stata 中 `import delimited` 的反向操作是 `export delimited`。

```
export delimited tips2.csv
```

类似地，在 pandas 中，`read_csv` 的反向操作是 `DataFrame.to_csv()`。

```
tips.to_csv('tips2.csv')
```

pandas 还可以使用 `DataFrame.to_stata()` 方法导出为 Stata 文件格式。

```
tips.to_stata('tips2.dta')
```

## 数据操作

### 列上的操作

在 Stata 中，任意数学表达式可以与新列或现有列上的 `generate` 和 `replace` 命令一起使用。该 `drop` 命令从数据集中删除列。

```
replace total_bill = total_bill - 2
generate new_bill = total_bill / 2
drop new_bill
```

pandas 通过指定个体提供了类似的矢量化操作 `Series` 中 `DataFrame`。可以以相同的方式分配新列。该 `DataFrame.drop()` 方法从中删除一列 `DataFrame`。

```
In [8]: tips['total_bill'] = tips['total_bill'] - 2

In [9]: tips['new_bill'] = tips['total_bill'] / 2

In [10]: tips.head()
Out[10]:
```

	total_bill	tip	sex	smoker	day	time	size	new_bill
0	14.99	1.01	Female	No	Sun	Dinner	2	7.495
1	8.34	1.66	Male	No	Sun	Dinner	3	4.170
2	19.01	3.50	Male	No	Sun	Dinner	3	9.505
3	21.68	3.31	Male	No	Sun	Dinner	2	10.840
4	22.59	3.61	Female	No	Sun	Dinner	4	11.295

```
In [11]: tips = tips.drop('new_bill', axis=1)
```

## 过滤

在Stata中过滤是通过 `if` 一个或多个列上的子句完成的。

```
list if total_bill > 10
```

DataFrame可以通过多种方式进行过滤; 最直观的是使用 [布尔索引](#)。

```
In [12]: tips[tips['total_bill'] > 10].head()
Out[12]:
```

	total_bill	tip	sex	smoker	day	time	size
0	14.99	1.01	Female	No	Sun	Dinner	2
2	19.01	3.50	Male	No	Sun	Dinner	3
3	21.68	3.31	Male	No	Sun	Dinner	2
4	22.59	3.61	Female	No	Sun	Dinner	4
5	23.29	4.71	Male	No	Sun	Dinner	4

## 如果/那么逻辑

在Stata中, `if` 子句也可用于创建新列。

```
generate bucket = "low" if total_bill < 10
replace bucket = "high" if total_bill >= 10
```

使用 `numpy` 的 `where` 方法可以在 `pandas` 中完成相同的操作。

```
In [13]: tips['bucket'] = np.where(tips['total_bill'] < 10, 'low', 'high')

In [14]: tips.head()
Out[14]:
```

	total_bill	tip	sex	smoker	day	time	size	bucket
0	14.99	1.01	Female	No	Sun	Dinner	2	high
1	8.34	1.66	Male	No	Sun	Dinner	3	low
2	19.01	3.50	Male	No	Sun	Dinner	3	high
3	21.68	3.31	Male	No	Sun	Dinner	2	high
4	22.59	3.61	Female	No	Sun	Dinner	4	high

## 日期功能

Stata提供了各种函数来对date / datetime列进行操作。

```

generate date1 = mdy(1, 15, 2013)
generate date2 = date("Feb152015", "MDY")

generate date1_year = year(date1)
generate date2_month = month(date2)

* shift date to beginning of next month
generate date1_next = mdy(month(date1) + 1, 1, year(date1)) if month(date1) != 12
replace date1_next = mdy(1, 1, year(date1) + 1) if month(date1) == 12
generate months_between = mofd(date2) - mofd(date1)

list date1 date2 date1_year date2_month date1_next months_between

```

等效的 pandas 操作如下所示。除了这些功能外，pandas 还支持 Stata 中不具备的其他时间序列功能（例如时区处理和自定义偏移） - 有关详细信息，请参阅[时间序列文档](#)。

```

In [15]: tips['date1'] = pd.Timestamp('2013-01-15')

In [16]: tips['date2'] = pd.Timestamp('2015-02-15')

In [17]: tips['date1_year'] = tips['date1'].dt.year

In [18]: tips['date2_month'] = tips['date2'].dt.month

In [19]: tips['date1_next'] = tips['date1'] + pd.offsets.MonthBegin()

In [20]: tips['months_between'] = (tips['date2'].dt.to_period('M')
....: - tips['date1'].dt.to_period('M'))
....:

In [21]: tips[['date1', 'date2', 'date1_year', 'date2_month', 'date1_next',
....: 'months_between']].head()
....:
Out[21]:
 date1 date2 date1_year date2_month date1_next months_between
0 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>
1 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>
2 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>
3 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>
4 2013-01-15 2015-02-15 2013 2 2013-02-01 <25 * MonthEnds>

```

## 列的选择

Stata 提供了选择，删除和重命名列的关键字。

```

keep sex total_bill tip

drop sex

rename total_bill total_bill_2

```



下面的 pandas 表示相同的操作。请注意，与 Stata 相比，这些操作不会发生。要使这些更改保持不变，请将操作分配回变量。

```
keep
In [22]: tips[['sex', 'total_bill', 'tip']].head()
Out[22]:
```

	sex	total_bill	tip
0	Female	14.99	1.01
1	Male	8.34	1.66
2	Male	19.01	3.50
3	Male	21.68	3.31
4	Female	22.59	3.61

```
drop
In [23]: tips.drop('sex', axis=1).head()
Out[23]:
```

	total_bill	tip	smoker	day	time	size
0	14.99	1.01	No	Sun	Dinner	2
1	8.34	1.66	No	Sun	Dinner	3
2	19.01	3.50	No	Sun	Dinner	3
3	21.68	3.31	No	Sun	Dinner	2
4	22.59	3.61	No	Sun	Dinner	4

```
rename
In [24]: tips.rename(columns={'total_bill': 'total_bill_2'}).head()
Out[24]:
```

	total_bill_2	tip	sex	smoker	day	time	size
0	14.99	1.01	Female	No	Sun	Dinner	2
1	8.34	1.66	Male	No	Sun	Dinner	3
2	19.01	3.50	Male	No	Sun	Dinner	3
3	21.68	3.31	Male	No	Sun	Dinner	2
4	22.59	3.61	Female	No	Sun	Dinner	4

## 按值排序

Stata中的排序是通过 `sort`

```
sort sex total_bill
```

pandas 对象有一个 `DataFrame.sort_values()` 方法，它采用列表进行排序。

```
In [25]: tips = tips.sort_values(['sex', 'total_bill'])

In [26]: tips.head()
Out[26]:
```

	total_bill	tip	sex	smoker	day	time	size
67	1.07	1.00	Female	Yes	Sat	Dinner	1
92	3.75	1.00	Female	Yes	Fri	Dinner	2
111	5.25	1.00	Female	No	Sat	Dinner	1
145	6.35	1.50	Female	No	Thur	Lunch	2
135	6.51	1.25	Female	No	Thur	Lunch	2

## 字符串处理

### 查找字符串的长度

Stata 分别使用ASCII和Unicode字符串 `strlen()` 和 `ustrlen()` 函数确定字符串的长度。

```
generate strlen_time = strlen(time)
generate ustrlen_time = ustrlen(time)
```

Python 使用该 `len` 函数确定字符串的长度。在Python 3中，所有字符串都是Unicode字符串。`len` 包括尾随空白。使用 `len` 和 `rstrip` 排除尾随空格。

```
In [27]: tips['time'].str.len().head()
Out[27]:
67 6
92 6
111 6
145 5
135 5
Name: time, dtype: int64

In [28]: tips['time'].str.rstrip().str.len().head()
Out[28]:
67 6
92 6
111 6
145 5
135 5
Name: time, dtype: int64
```

### 找到字符串的位置

Stata使用该 `strpos()` 函数确定字符串中字符的位置。这将获取第一个参数定义的字符串，并搜索您提供的子字符串的第一个位置作为第二个参数。

```
generate str_position = strpos(sex, "ale")
```

Python使用 `find()` 函数确定字符串中字符的位置。`find` 搜索子字符串的第一个位置。如果找到子字符串，则该函数返回其位置。请记住，Python索引是从零开始的，如果找不到子串，函数将返回-1。

```
In [29]: tips['sex'].str.find("ale").head()
Out[29]:
67 3
92 3
111 3
145 3
135 3
Name: sex, dtype: int64
```

## 按位置提取字符串

Stata根据 `substr()` 函数的位置从字符串中提取字符串。

```
generate short_sex = substr(sex, 1, 1)
```

使用pandas，您可以使用 `[]` 符号从位置位置提取字符串中的子字符串。请记住，Python索引是从零开始的。

```
In [30]: tips['sex'].str[0:1].head()
Out[30]:
67 F
92 F
111 F
145 F
135 F
Name: sex, dtype: object
```

## 提取第n个字符

Stata `word()` 函数返回字符串中的第n个单词。第一个参数是要解析的字符串，第二个参数指定要提取的字。

```
clear
input str20 string
"John Smith"
"Jane Cook"
end

generate first_name = word(name, 1)
generate last_name = word(name, -1)
```

Python使用正则表达式根据文本从字符串中提取字符串。有更强大的方法，但这只是一个简单的方法。

```
In [31]: firstlast = pd.DataFrame({'string': ['John Smith', 'Jane Cook']})

In [32]: firstlast['First_Name'] = firstlast['string'].str.split(" ", expand=True)[0]

In [33]: firstlast['Last_Name'] = firstlast['string'].str.rsplit(" ", expand=True)[0]

In [34]: firstlast
Out[34]:
 string First_Name Last_Name
0 John Smith John John
1 Jane Cook Jane Jane
```

## 改变案例

所述的Stata `strupper()`，`strlower()`，`strproper()`，`ustrupper()`，`ustrlower()`，和 `ustrtitle()` 功能分别改变ASCII和Unicode字符串的情况下，。

```
clear
input str20 string
"John Smith"
"Jane Cook"
end

generate upper = strupper(string)
generate lower = strlower(string)
generate title = strproper(string)
list
```

等效Python的功能 `upper`，`lower` 和 `title`。

```
In [35]: firstlast = pd.DataFrame({'string': ['John Smith', 'Jane Cook']})

In [36]: firstlast['upper'] = firstlast['string'].str.upper()

In [37]: firstlast['lower'] = firstlast['string'].str.lower()

In [38]: firstlast['title'] = firstlast['string'].str.title()

In [39]: firstlast
Out[39]:
```

	string	upper	lower	title
0	John Smith	JOHN SMITH	john smith	John Smith
1	Jane Cook	JANE COOK	jane cook	Jane Cook

## 合并

合并示例中将使用以下表格

```

In [40]: df1 = pd.DataFrame({'key': ['A', 'B', 'C', 'D'],
....: 'value': np.random.randn(4)})
....:

In [41]: df1
Out[41]:
 key value
0 A 0.469112
1 B -0.282863
2 C -1.509059
3 D -1.135632

In [42]: df2 = pd.DataFrame({'key': ['B', 'D', 'D', 'E'],
....: 'value': np.random.randn(4)})
....:

In [43]: df2
Out[43]:
 key value
0 B 1.212112
1 D -0.173215
2 D 0.119209
3 E -1.044236

```

在Stata中，要执行合并，一个数据集必须在内存中，另一个必须作为磁盘上的文件名引用。相比之下，Python必须 `DataFrames` 已经在内存中。

默认情况下，Stata执行外部联接，其中两个数据集的所有观察值在合并后都保留在内存中。通过使用在 `_merge` 变量中创建的值，可以仅保留来自初始数据集，合并数据集或两者的交集的观察。

```

* First create df2 and save to disk
clear
input str1 key
B
D
D
E
end
generate value = rnormal()
save df2.dta

* Now create df1 in memory
clear
input str1 key
A
B
C
D
end
generate value = rnormal()

preserve

* Left join
merge 1:n key using df2.dta
keep if _merge == 1

* Right join
restore, preserve
merge 1:n key using df2.dta
keep if _merge == 2

* Inner join
restore, preserve
merge 1:n key using df2.dta
keep if _merge == 3

* Outer join
restore
merge 1:n key using df2.dta

```

pandas 的 DataFrames 有一个 `DataFrame.merge()` 提供类似功能的方法。请注意，通过 `how` 关键字可以实现不同的连接类型。

```
In [44]: inner_join = df1.merge(df2, on=['key'], how='inner')
```

```
In [45]: inner_join
```

```
Out[45]:
```

	key	value_x	value_y
0	B	-0.282863	1.212112
1	D	-1.135632	-0.173215
2	D	-1.135632	0.119209

```
In [46]: left_join = df1.merge(df2, on=['key'], how='left')
```

```
In [47]: left_join
```

```
Out[47]:
```

	key	value_x	value_y
0	A	0.469112	NaN
1	B	-0.282863	1.212112
2	C	-1.509059	NaN
3	D	-1.135632	-0.173215
4	D	-1.135632	0.119209

```
In [48]: right_join = df1.merge(df2, on=['key'], how='right')
```

```
In [49]: right_join
```

```
Out[49]:
```

	key	value_x	value_y
0	B	-0.282863	1.212112
1	D	-1.135632	-0.173215
2	D	-1.135632	0.119209
3	E	NaN	-1.044236

```
In [50]: outer_join = df1.merge(df2, on=['key'], how='outer')
```

```
In [51]: outer_join
```

```
Out[51]:
```

	key	value_x	value_y
0	A	0.469112	NaN
1	B	-0.282863	1.212112
2	C	-1.509059	NaN
3	D	-1.135632	-0.173215
4	D	-1.135632	0.119209
5	E	NaN	-1.044236

## 缺少数据

像Stata一样，pandas 有缺失数据的表示 - 特殊浮点值 `NaN`（不是数字）。许多语义都是一样的; 例如，丢失的数据通过数字操作传播，默认情况下会被聚合忽略。

```

In [52]: outer_join
Out[52]:
 key value_x value_y
0 A 0.469112 NaN
1 B -0.282863 1.212112
2 C -1.509059 NaN
3 D -1.135632 -0.173215
4 D -1.135632 0.119209
5 E NaN -1.044236

In [53]: outer_join['value_x'] + outer_join['value_y']
Out[53]:
0 NaN
1 0.929249
2 NaN
3 -1.308847
4 -1.016424
5 NaN
dtype: float64

In [54]: outer_join['value_x'].sum()
Out[54]: -3.5940742896293765

```

一个区别是丢失的数据无法与其哨兵值进行比较。例如，在 Stata 中，您可以执行此操作以过滤缺失值。

```

* Keep missing values
list if value_x == .
* Keep non-missing values
list if value_x != .

```

这在 pandas 中不起作用。相反，应使用 `pd.isna()` 或 `pd.notna()` 函数进行比较。

```

In [55]: outer_join[pd.isna(outer_join['value_x'])]
Out[55]:
 key value_x value_y
5 E NaN -1.044236

In [56]: outer_join[pd.notna(outer_join['value_x'])]
Out[56]:
 key value_x value_y
0 A 0.469112 NaN
1 B -0.282863 1.212112
2 C -1.509059 NaN
3 D -1.135632 -0.173215
4 D -1.135632 0.119209

```

pandas 还提供了多种处理丢失数据的方法，其中一些方法在 Stata 中表达起来很有挑战性。例如，有一些方法可以删除具有任何缺失值的所有行，用指定值(如平均值)替换缺失值，或从前一行向前填充。有关详细信息，请参阅[缺失数据文档](#)。



```
Drop rows with any missing value
In [57]: outer_join.dropna()
Out[57]:
 key value_x value_y
1 B -0.282863 1.212112
3 D -1.135632 -0.173215
4 D -1.135632 0.119209

Fill forwards
In [58]: outer_join.fillna(method='ffill')
Out[58]:
 key value_x value_y
0 A 0.469112 NaN
1 B -0.282863 1.212112
2 C -1.509059 1.212112
3 D -1.135632 -0.173215
4 D -1.135632 0.119209
5 E -1.135632 -1.044236

Impute missing values with the mean
In [59]: outer_join['value_x'].fillna(outer_join['value_x'].mean())
Out[59]:
0 0.469112
1 -0.282863
2 -1.509059
3 -1.135632
4 -1.135632
5 -0.718815
Name: value_x, dtype: float64
```

## 的GroupBy

### 聚合

Stata `collapse` 可用于按一个或多个关键变量进行分组，并计算数字列上的聚合。

```
collapse (sum) total_bill tip, by(sex smoker)
```

pandas提供了一种 `groupby` 允许类似聚合的灵活机制。有关 更多详细信息和示例，请参阅[groupby文档](#)。

```
In [60]: tips_summed = tips.groupby(['sex', 'smoker'])['total_bill', 'tip'].sum()

In [61]: tips_summed.head()
Out[61]:
 sex smoker total_bill tip
0 Female No 869.68 149.77
1 Female Yes 527.27 96.74
2 Male No 1725.75 302.00
3 Male Yes 1217.07 183.07
```

## 转换

在Stata中，如果组聚合需要与原始数据集一起使用 `bysort`，通常会使用 `egen()`。例如，减去吸烟者组每次观察的平均值。

```
bysort sex smoker: egen group_bill = mean(total_bill)
generate adj_total_bill = total_bill - group_bill
```

`pandas groupby` 提供了一种 `transform` 机制，允许在一个操作中简洁地表达这些类型的操作。

```
In [62]: gb = tips.groupby('smoker')['total_bill']

In [63]: tips['adj_total_bill'] = tips['total_bill'] - gb.transform('mean')

In [64]: tips.head()
Out[64]:
```

	total_bill	tip	sex	smoker	day	time	size	adj_total_bill
67	1.07	1.00	Female	Yes	Sat	Dinner	1	-17.686344
92	3.75	1.00	Female	Yes	Fri	Dinner	2	-15.006344
111	5.25	1.00	Female	No	Sat	Dinner	1	-11.938278
145	6.35	1.50	Female	No	Thur	Lunch	2	-10.838278
135	6.51	1.25	Female	No	Thur	Lunch	2	-10.678278

## 按组处理

除聚合外，`pandas groupby` 还可用于复制 `bysort` Stata中的大多数其他处理。例如，以下示例按性别/吸烟者组列出当前排序顺序中的第一个观察结果。

```
bysort sex smoker: list if _n == 1
```

在 `pandas` 中，这将写成：

```
In [65]: tips.groupby(['sex', 'smoker']).first()
Out[65]:
```

		total_bill	tip	day	time	size	adj_total_bill
sex	smoker						
Female	No	5.25	1.00	Sat	Dinner	1	-11.938278
	Yes	1.07	1.00	Sat	Dinner	1	-17.686344
Male	No	5.51	2.00	Thur	Lunch	2	-11.678278
	Yes	5.25	5.15	Sun	Dinner	2	-13.506344

## 其他注意事项

### 磁盘与内存

`pandas` 和 `Stata` 都只在内存中运行。这意味着能够在 `pandas` 中加载的数据大小受机器内存的限制。如果需要核心处理，则有一种可能性是 `dask.dataframe` 库，它为磁盘上的 `pandas` 功能提供了一个子集 `DataFrame`。

---

# 教程资料

为方便新用户上手 Pandas，本节收录了众多 Pandas 教程。

## 官方指南

---

[十分钟入门 Pandas](#)，Pandas 团队出品。

[Cookbook](#)，Pandas 实用案例。

[Pandas 速查表](#)，案头必备。

## 社区指南

---

### 《Pandas Cookbook》Julia Evans 著

[Julia Evans](#) 2015 年编著的《Pandas Cookbook》包含了很多 Pandas 实战例子，这些例子大多基于实战数据，涵盖了绝大多数新手小白遇到的实际问题。代码请参阅 Pandas-cookbook 的 [GitHub 仓库](#)。

### 《Learn Pandas》Hernan Rojas 著

面对新用户介绍 Pandas 学习经验：<https://bitbucket.org/hrojas/learn-Pandas>

## Python 数据分析实战

该指南介绍了用 Python 数据生态系统对开源数据集进行数据分析的过程。涵盖了[数据整理](#)、[数据聚合](#)、[数据可视化](#)和[时间序列](#)。

## 新手习题

用真实数据集与习题，锻炼 Pandas 运用能力。更多资源，请参阅[这个仓库](#)。

## 现代 Pandas

[Tom Augspurger](#) 2016 年编写的系列教程。源文件在 GitHub 存储库 [TomAugspurger/effective-Pandas](#)。

- [现代 Pandas](#)
- [方法链](#)
- [索引](#)
- [性能](#)
- [清洗数据](#)
- [可视化](#)
- [时间序列](#)

## 用 Pandas、vincent 和 xlsxwriter 绘制 Excel 图

[用 Pandas、vincent 和 xlsxwriter 绘制 Excel 图](#)

### 视频教程

- [Pandas 零基础](#) (2015) (2:24) [GitHub repo](#)
- [Pandas 简介](#) (2016) (1:28) [GitHub repo](#)
- [Pandas: 从 head\(\) 到 tail\(\)](#) (2016) (1:26) [GitHub repo](#)
- [Pandas 数据分析](#) (2016-2018) [GitHub repo](#) 与 [Jupyter Notebook](#)
- [Pandas 最佳实践](#) (2018) [GitHub repo](#) 与 [Jupyter Notebook](#)

### 其它教程

- [Wes McKinney \(Pandas 仁慈的终身独裁者\) 博客](#)
- [轻松上手 Python 统计分析 - SciPy 与 Pandas](#), Randal Olson
- [Python 统计数据分析](#), Christopher Fonnesbeck, SciPy 2013
- [Python 金融分析](#), Thomas Wiecki
- [Pandas 数据结构](#), Greg Reda
- [Pandas 与 Python: Top 10](#), Manish Amde
- [Pandas DataFrames 教程](#), Karlijn Willems
- [实战案例简明教程](#)

---

## 用户指南

---

### Pandas 用户指南目录

“用户指南”按主题划分区域涵盖了几乎所有Pandas的功能。每个小节都介绍了一个主题（例如“处理缺失的数据”），并讨论了Pandas如何解决问题，其中包含许多示例。

刚开始接触Pandas的同学应该从[十分钟入门Pandas](#)开始看起。

有关任何特定方法的更多信息，请[参阅API参考](#)。

- [IO工具](#) (文本, CSV, HDF5, ...)
  - [CSV & text files](#)
  - [JSON](#)
  - [HTML](#)
  - [Excel files](#)
  - [OpenDocument Spreadsheets](#)
  - [Clipboard](#)
  - [Pickling](#)
  - [msgpack](#)

- HDF5 (PyTables)
- Feather
- Parquet
- SQL queries
- Google BigQuery
- Stata format
- SAS formats
- Other file formats
- Performance considerations
- 索引和数据选择器
  - Different choices for indexing
  - Basics
  - Attribute access
  - Slicing ranges
  - Selection by label
  - Selection by position
  - Selection by callable
  - IX indexer is deprecated
  - Indexing with list with missing labels is deprecated
  - Selecting random samples
  - Setting with enlargement
  - Fast scalar value getting and setting
  - Boolean indexing
  - Indexing with isin
  - The `where()` Method and Masking
  - The `query()` Method
  - Duplicate data
  - Dictionary-like `get()` method
  - The `lookup()` method
  - Index objects
  - Set / reset index
  - Returning a view versus a copy
- 多索引/高级索引
  - Hierarchical indexing (MultiIndex)
  - Advanced indexing with hierarchical index
  - Sorting a MultiIndex
  - Take methods
  - Index types
  - Miscellaneous indexing FAQ
- 合并、联接和连接
  - Concatenating objects
  - Database-style DataFrame or named Series joining/merging

- Timeseries friendly merging
- 重塑和数据透视表
  - Reshaping by pivoting DataFrame objects
  - Reshaping by stacking and unstacking
  - Reshaping by Melt
  - Combining with stats and GroupBy
  - Pivot tables
  - Cross tabulations
  - Tiling
  - Computing indicator / dummy variables
  - Factorizing values
  - Examples
  - Exploding a list-like column
- 处理文本字符串
  - Splitting and replacing strings
  - Concatenation
  - Indexing with .str
  - Extracting substrings
  - Testing for Strings that match or contain a pattern
  - Creating indicator variables
  - Method summary
- 处理丢失的数据
  - Values considered “missing”
  - Sum/prod of empties/nans
  - NA values in GroupBy
  - Filling missing values: fillna
  - Filling with a PandasObject
  - Dropping axis labels with missing data: dropna
  - Interpolation
  - Replacing generic values
  - String/regular expression replacement
  - Numeric replacement
- 分类数据
  - Object creation
  - CategoricalDtype
  - Description
  - Working with categories
  - Sorting and order
  - Comparisons
  - Operations
  - Data munging
  - Getting data in/out

- Missing data
- Differences to R's factor
- Gotchas
- Nullable整型数据类型
- 可视化
  - Basic plotting: plot
  - Other plots
  - Plotting with missing data
  - Plotting Tools
  - Plot Formatting
  - Plotting directly with matplotlib
  - Trellis plotting interface
- 计算工具
  - Statistical functions
  - Window Functions
  - Aggregation
  - Expanding windows
  - Exponentially weighted windows
- 组操作: 拆分-应用-组合
  - Splitting an object into groups
  - Iterating through groups
  - Selecting a group
  - Aggregation
  - Transformation
  - Filtration
  - Dispatching to instance methods
  - Flexible apply
  - Other useful features
  - Examples
- 时间序列/日期方法
  - Overview
  - Timestamps vs. Time Spans
  - Converting to timestamps
  - Generating ranges of timestamps
  - Timestamp limitations
  - Indexing
  - Time/date components
  - DateOffset objects
  - Time Series-Related Instance Methods
  - Resampling
  - Time span representation
  - Converting between representations

- Representing out-of-bounds spans
- Time zone handling
- 时间增量
  - Parsing
  - Operations
  - Reductions
  - Frequency conversion
  - Attributes
  - TimedeltaIndex
  - Resampling
- 样式
  - Building styles
  - Finer control: slicing
  - Finer Control: Display Values
  - Builtin styles
  - Sharing styles
  - Other Options
  - Fun stuff
  - Export to Excel
  - Extensibility
- 选项和设置
  - Overview
  - Getting and setting options
  - Setting startup options in Python/IPython environment
  - Frequently Used Options
  - Available options
  - Number formatting
  - Unicode formatting
  - Table schema display
- 提高性能
  - Cython (writing C extensions for pandas)
  - Using Numba
  - Expression evaluation via `>eval()`
- 稀疏数据结构
  - SparseArray
  - SparseDtype
  - Sparse accessor
  - Sparse calculation
  - Migrating
  - Interaction with `scipy.sparse`
  - Sparse subclasses



- [常见问题\(FAQ\)](#)
  - [DataFrame memory usage](#)
  - [Using if/truth statements with pandas](#)
  - [NaN, Integer NA values and NA type promotions](#)
  - [Differences with NumPy](#)
  - [Thread-safety](#)
  - [Byte-Ordering issues](#)
- [烹饪指南](#)
  - [Idioms](#)
  - [Selection](#)
  - [MultiIndexing](#)
  - [Missing data](#)
  - [Grouping](#)
  - [Timeseries](#)
  - [Merge](#)
  - [Plotting](#)
  - [Data In/Out](#)
  - [Computation](#)
  - [Timedeltas](#)
  - [Aliasing axis names](#)
  - [Creating example data](#)

---

## IO工具（文本，CSV，HDF5，...）

pandas的I/O API是一组 `read` 函数，比如 `pandas.read_csv()` 函数。这类函数可以返回pandas对象。相应的 `write` 函数是像 `DataFrame.to_csv()` 一样的对象方法。下面是一个方法列表，包含了这里面的所有 `readers` 函数和 `writer` 函数。

Format Type	Data Description	Reader	Writer
text	CSV	<code>read_csv</code>	<code>to_csv</code>
text	JSON	<code>read_json</code>	<code>to_json</code>
text	HTML	<code>read_html</code>	<code>to_html</code>
text	Local clipboard	<code>read_clipboard</code>	<code>to_clipboard</code>
binary	MS Excel	<code>read_excel</code>	<code>to_excel</code>
binary	OpenDocument	<code>read_excel</code>	
binary	HDF5 Format	<code>read_hdf</code>	<code>to_hdf</code>
binary	Feather Format	<code>read_feather</code>	<code>to_feather</code>
binary	Parquet Format	<code>read_parquet</code>	<code>to_parquet</code>
binary	Msgpack	<code>read_msgpack</code>	<code>to_msgpack</code>
binary	Stata	<code>read_stata</code>	<code>to_stata</code>
binary	SAS	<code>read_sas</code>	
binary	Python Pickle Format	<code>read_pickle</code>	<code>to_pickle</code>
SQL	SQL	<code>read_sql</code>	<code>to_sql</code>
SQL	Google Big Query	<code>read_gbq</code>	<code>to_gbq</code>

[Here](#) is an informal performance comparison for some of these IO methods.

⋮ tip 注意

比如在使用 `StringIO` 类时, 请先确定python的版本信息。也就是说, 是使用python2的

`from StringIO import StringIO` 还是python3的 `from io import StringIO`。

⋮

## CSV & 文本文件

读文本文件 (a.k.a. flat files)的主要方法 is `read_csv()`. 关于一些更高级的用法请参阅[cookbook](#)。

### 方法解析(Parsing options)

`read_csv()` 可接受以下常用参数:

#### 基础

`filepath_or_buffer`: various

- 文件路径 (a `str`, `pathlib.Path`, or `py._path.local.LocalPath`), URL (including http, ftp, and S3 locations), 或者具有 `read()` 方法的任何对象 (such as an open file or `StringIO`).

sep : str, 默认 `read_csv()` 分隔符为 ',', `read_table()` 方法, 分隔符为 \t

- 分隔符的使用. 如果分隔符为 `None`, 虽然C不能解析, 但python解析引擎可解析, 这意味着python将被使用, 通过内置的sniffer tool自动检测分隔符, `csv.Sniffer`. 除此之外, 字符长度超过 1 并且不同于 's+' 的将被视为正则表达式, 并且将强制使用python解析引擎. 需要注意的是, 正则表达式易于忽略引用数据 (主要注意转义字符的使用) 例如: '\\r\\t'.

delimiter : str, default None

- sep的替代参数.

delim\_whitespace : boolean, default False

- 指定是否将空格 (e.g. ' ' or '\t') 当作delimiter. 等价于设置 `sep='\s+'`. 如果这个选项被设置为 `True`, 就不要给 `delimiter` 传参了.

version 0.18.1: 支持Python解析器.

## 列、索引、名称

header : int or list of ints, default 'infer'

- 当选择默认值或 `header=0` 时, 将首行设为列名. 如果列名被传入明确值就令 `header=None`. 注意, 当 `header=0` 时, 即使列名被传参也会被覆盖.
- 标题可以是指定列上的MultiIndex的行位置的整数列表, 例如 `[0,1,3]`. 在列名指定时, 若某列未被指定, 读取时将跳过该列 (例如 在下面的例子中第二列将被跳过). 注意, 如果 `skip_blank_lines=True`, 此参数将忽略空行和注释行, 因此 `header=0` 表示第一行数据而非文件的第一行.

names : array-like, default None

- 列名列表的使用. 如果文件不包含列名, 那么应该设置 `header=None`. 列名列表中不允许有重复值.

index\_col : int, str, sequence of int / str, or False, default None

- `DataFrame` 的行索引列表, 既可以是字符串名称也可以是列索引. 如果传入一个字符串序列或者整数序列, 那么一定要使用多级索引 (MultiIndex).
- 注意: 当 `index_col=False`, pandas不再使用首列作为索引. 例如, 当你的文件是一个每行末尾都带有一个分割符的格式错误的文件时.

usecols : list-like or callable, default None

- 返回列名列表的子集. 如果该参数为列表形式, 那么所有元素应全为位置 (即文档列中的整数索引) 或者 全为相应列的列名字符串 (这些列名字符串为names参数给出的或者文档的 header 行内容). 例如, 一个有效的列表型参数 usecols 将会是 `[0, 1, 2]` 或者 `['foo', 'bar', 'baz']`.

- 元素顺序可忽略，因此 `usecols=[0, 1]` 等价于 `[1, 0]`。如果想实例化一个自定义列顺序的 `DataFrame`，请使用 `pd.read_csv(data, usecols=['foo', 'bar'])[['foo', 'bar']]`，这样列的顺序为 `['foo', 'bar']`。如果设置 `pd.read_csv(data, usecols=['foo', 'bar'])[['bar', 'foo']]` 那么列的顺序为 `['bar', 'foo']`。
- 如果使用callable的方式, 可调用函数将根据列名计算，返回可调用函数计算结果为True的名称:

```
In [1]: from io import StringIO, BytesIO

In [2]: data = ('col1,col2,col3\n'
...: 'a,b,1\n'
...: 'a,b,2\n'
...: 'c,d,3')
...:

In [3]: pd.read_csv(StringIO(data))
Out[3]:
 col1 col2 col3
0 a b 1
1 a b 2
2 c d 3

In [4]: pd.read_csv(StringIO(data), usecols=lambda x: x.upper() in ['COL1', 'COL3'])
Out[4]:
 col1 col3
0 a 1
1 a 2
2 c 3
```

使用此参数可以大大加快解析时间并降低内存使用率。

`squeeze`: boolean, default False

- 如果解析的数据仅包含一个列，那么结果将以 `Series` 的形式返回。

`prefix`: str, default None

- 当没有header时，可通过该参数为数字列名添加前缀, e.g. 'X' for X0, X1, ...

`mangle_dupe_cols`: boolean, default True

- 当列名有重复时，解析列名将变为 'X', 'X.1' ... 'X.N' 而不是 'X' ... 'X'。如果该参数为 `False`，那么当列名中有重复时，前列将会被后列覆盖。

## 常规解析配置

`dtype`: Type name or dict of column -> type, default None

- 指定某列或整体数据的数据类型。E.g. `{ 'a': np.float64, 'b': np.int32 }` (不支持 `engine='python'`)。将str或object与合适的设置一起使用以保留和不解释dtype。

- New in version 0.20.0: 支持python解析器.

engine : {'c', 'python'}

- 解析引擎的使用。 尽管C引擎速度更快，但是目前python引擎功能更加完美。

converters : dict, default None

- Dict of functions for converting values in certain columns. Keys can either be integers or column labels.

true\_values : list, default None

- Values to consider as `True` .

false\_values : list, default None

- Values to consider as `False` .

skipinitialspace : boolean, default False

- Skip spaces after delimiter.

skiprows : list-like or integer, default None

- Line numbers to skip (0-indexed) or number of lines to skip (int) at the start of the file.
- If callable, the callable function will be evaluated against the row indices, returning True if the row should be skipped and False otherwise:

```
In [5]: data = ('col1,col2,col3\n'
...: 'a,b,1\n'
...: 'a,b,2\n'
...: 'c,d,3')
...:

In [6]: pd.read_csv(StringIO(data))
Out[6]:
 col1 col2 col3
0 a b 1
1 a b 2
2 c d 3

In [7]: pd.read_csv(StringIO(data), skiprows=lambda x: x % 2 != 0)
Out[7]:
 col1 col2 col3
0 a b 2
```

skipfooter : int, default 0

- Number of lines at bottom of file to skip (unsupported with engine='c' ).

nrows : int, default None

- Number of rows of file to read. Useful for reading pieces of large files.

low\_memory : boolean, default True

- Internally process the file in chunks, resulting in lower memory use while parsing, but possibly mixed type inference. To ensure no mixed types either set `False`, or specify the type with the `dtype` parameter. Note that the entire file is read into a single `DataFrame` regardless, use the `chunksize` or `iterator` parameter to return the data in chunks. (Only valid with C parser)

memory\_map : boolean, default False

- If a filepath is provided for `filepath_or_buffer`, map the file object directly onto memory and access the data directly from there. Using this option can improve performance because there is no longer any I/O overhead.

## NA and missing data handling

na\_values : scalar, str, list-like, or dict, default None

- Additional strings to recognize as NA/NaN. If dict passed, specific per-column NA values. See [na values const](#) below for a list of the values interpreted as NaN by default.

keep\_default\_na : boolean, default True

- Whether or not to include the default NaN values when parsing the data. Depending on whether `na_values` is passed in, the behavior is as follows:
  - If `keep_default_na` is `True`, and `na_values` are specified, `na_values` is appended to the default NaN values used for parsing.
  - If `keep_default_na` is `True`, and `na_values` are not specified, only the default NaN values are used for parsing.
  - If `keep_default_na` is `False`, and `na_values` are specified, only the NaN values specified `na_values` are used for parsing.
  - If `keep_default_na` is `False`, and `na_values` are not specified, no strings will be parsed as NaN.

Note that if `na_filter` is passed in as `False`, the `keep_default_na` and `na_values` parameters will be ignored.

na\_filter : boolean, default True

- Detect missing value markers (empty strings and the value of `na_values`). In data without any NAs, passing `na_filter=False` can improve the performance of reading a large file.

`verbose` : boolean, default `False`

- Indicate number of NA values placed in non-numeric columns.

`skip_blank_lines` : boolean, default `True`

- If `True`, skip over blank lines rather than interpreting as NaN values.

## Datetime handling

`parse_dates` : boolean or list of ints or names or list of lists or dict, default `False`.

- If `True` -> try parsing the index.
- If `[1, 2, 3]` -> try parsing columns 1, 2, 3 each as a separate date column.
- If `[[1, 3]]` -> combine columns 1 and 3 and parse as a single date column.
- If `{'foo': [1, 3]}` -> parse columns 1, 3 as date and call result `'foo'`. A fast-path exists for iso8601-formatted dates.

`infer_datetime_format` : boolean, default `False`

- If `True` and `parse_dates` is enabled for a column, attempt to infer the datetime format to speed up the processing.

`keep_date_col` : boolean, default `False`

- If `True` and `parse_dates` specifies combining multiple columns then keep the original columns.

`date_parser` : function, default `None`

- Function to use for converting a sequence of string columns to an array of datetime instances. The default uses `dateutil.parser.parser` to do the conversion. pandas will try to call `date_parser` in three different ways, advancing to the next if an exception occurs: 1) Pass one or more arrays (as defined by `parse_dates`) as arguments; 2) concatenate (row-wise) the string values from the columns defined by `parse_dates` into a single array and pass that; and 3) call `date_parser` once for each row using one or more strings (corresponding to the columns defined by `parse_dates`) as arguments.

`dayfirst` : boolean, default `False`

- DD/MM format dates, international and European format.

`cache_dates` : boolean, default `True`

- If `True`, use a cache of unique, converted dates to apply the datetime conversion. May produce significant speed-up when parsing duplicate date strings, especially ones with timezone offsets.

New in version 0.25.0.

## Iteration

iterator : boolean, default False

- Return TextFileReader object for iteration or getting chunks with `get_chunk()` .

chunksize : int, default None

- Return TextFileReader object for iteration. See [iterating and chunking](#) below.

## Quoting, compression, and file format

compression : {'infer', 'gzip', 'bz2', 'zip', 'xz', None}, default 'infer'

- For on-the-fly decompression of on-disk data. If 'infer' , then use gzip, bz2, zip, or xz if filepath\_or\_buffer is a string ending in '.gz' , '.bz2' , '.zip' , or '.xz' , respectively, and no decompression otherwise. If using 'zip' , the ZIP file must contain only one data file to be read in. Set to None for no decompression.

New in version 0.18.1: support for 'zip' and 'xz' compression.

Changed in version 0.24.0: 'infer' option added and set to default.

thousands : str, default None

- Thousands separator.

decimal : str, default '.'

- Character to recognize as decimal point. E.g. use ',' for European data.

float\_precision : string, default None

- Specifies which converter the C engine should use for floating-point values. The options are None for the ordinary converter, high for the high-precision converter, and round\_trip for the round-trip converter.

lineterminator : str (length 1), default None

- Character to break file into lines. Only valid with C parser.

quotechar : str (length 1)

- The character used to denote the start and end of a quoted item. Quoted items can include the delimiter and it will be ignored.



quoting : int or `csv.QUOTE_*` instance, default 0

- Control field quoting behavior per `csv.QUOTE_*` constants. Use one of `QUOTE_MINIMAL` (0), `QUOTE_ALL` (1), `QUOTE_NONNUMERIC` (2) or `QUOTE_NONE` (3).

doublequote : boolean, default True

- When `quotechar` is specified and `quoting` is not `QUOTE_NONE`, indicate whether or not to interpret two consecutive `quotechar` elements **inside** a field as a single `quotechar` element.

escapechar : str (length 1), default None

- One-character string used to escape delimiter when quoting is `QUOTE_NONE`.

comment : str, default None

- Indicates remainder of line should not be parsed. If found at the beginning of a line, the line will be ignored altogether. This parameter must be a single character. Like empty lines (as long as `skip_blank_lines=True`), fully commented lines are ignored by the parameter header but not by `skiprows`. For example, if `comment='#'`, parsing `'#empty a,b,c 1,2,3'` with `header=0` will result in `'a,b,c'` being treated as the header.

encoding : str, default None

- Encoding to use for UTF when reading/writing (e.g. `'utf-8'`). [List of Python standard encodings](#).

dialect : str or `csv.Dialect` instance, default None

- If provided, this parameter will override values (default or not) for the following parameters: `delimiter`, `doublequote`, `escapechar`, `skipinitialspace`, `quotechar`, and `quoting`. If it is necessary to override values, a `ParserWarning` will be issued. See [csv.Dialect](#) documentation for more details.

## Error handling

error\_bad\_lines : boolean, default True

- Lines with too many fields (e.g. a csv line with too many commas) will by default cause an exception to be raised, and no `DataFrame` will be returned. If `False`, then these “bad lines” will be dropped from the `DataFrame` that is returned. See [bad lines](#) below.

warn\_bad\_lines : boolean, default True

- If `error_bad_lines` is `False`, and `warn_bad_lines` is `True`, a warning for each “bad line” will be output.

## Specifying column data types

You can indicate the data type for the whole `DataFrame` or individual columns:

```
In [8]: data = ('a,b,c,d\n'
...: '1,2,3,4\n'
...: '5,6,7,8\n'
...: '9,10,11')
...:

In [9]: print(data)
a,b,c,d
1,2,3,4
5,6,7,8
9,10,11

In [10]: df = pd.read_csv(StringIO(data), dtype=object)

In [11]: df
Out[11]:
 a b c d
0 1 2 3 4
1 5 6 7 8
2 9 10 11 NaN

In [12]: df['a'][0]
Out[12]: '1'

In [13]: df = pd.read_csv(StringIO(data),
...: dtype={'b': object, 'c': np.float64, 'd': 'Int64'})
...:
...:

In [14]: df.dtypes
Out[14]:
a int64
b object
c float64
d Int64
dtype: object
```

Fortunately, pandas offers more than one way to ensure that your column(s) contain only one `dtype`. If you're unfamiliar with these concepts, you can see [here](#) to learn more about dtypes, and [here](#) to learn more about `object` conversion in pandas.

For instance, you can use the `converters` argument of `read_csv()`:

```

In [15]: data = ("col_1\n"
....: "1\n"
....: "2\n"
....: "'A'\n"
....: "4.22")
....:

In [16]: df = pd.read_csv(StringIO(data), converters={'col_1': str})

In [17]: df
Out[17]:
 col_1
0 1
1 2
2 'A'
3 4.22

In [18]: df['col_1'].apply(type).value_counts()
Out[18]:
<class 'str'> 4
Name: col_1, dtype: int64

```

Or you can use the `to_numeric()` function to coerce the dtypes after reading in the data,

```

In [19]: df2 = pd.read_csv(StringIO(data))

In [20]: df2['col_1'] = pd.to_numeric(df2['col_1'], errors='coerce')

In [21]: df2
Out[21]:
 col_1
0 1.00
1 2.00
2 NaN
3 4.22

In [22]: df2['col_1'].apply(type).value_counts()
Out[22]:
<class 'float'> 4
Name: col_1, dtype: int64

```

which will convert all valid parsing to floats, leaving the invalid parsing as `NaN`.

Ultimately, how you deal with reading in columns containing mixed dtypes depends on your specific needs. In the case above, if you wanted to `NaN` out the data anomalies, then `to_numeric()` is probably your best option. However, if you wanted for all the data to be coerced, no matter the type, then using the `converters` argument of `read_csv()` would certainly be worth trying.

New in version 0.20.0: support for the Python parser.

The `dtype` option is supported by the ‘python’ engine.

### ::: tip Note

In some cases, reading in abnormal data with columns containing mixed dtypes will result in an inconsistent dataset. If you rely on pandas to infer the dtypes of your columns, the parsing engine will go and infer the dtypes for different chunks of the data, rather than the whole dataset at once. Consequently, you can end up with column(s) with mixed dtypes. For example,

```
In [23]: col_1 = list(range(500000)) + ['a', 'b'] + list(range(500000))

In [24]: df = pd.DataFrame({'col_1': col_1})

In [25]: df.to_csv('foo.csv')

In [26]: mixed_df = pd.read_csv('foo.csv')

In [27]: mixed_df['col_1'].apply(type).value_counts()
Out[27]:
<class 'int'> 737858
<class 'str'> 262144
Name: col_1, dtype: int64

In [28]: mixed_df['col_1'].dtype
Out[28]: dtype('O')
```

will result with `mixed_df` containing an `int` dtype for certain chunks of the column, and `str` for others due to the mixed dtypes from the data that was read in. It is important to note that the overall column will be marked with a dtype of `object`, which is used for columns with mixed dtypes.

:::

## Specifying categorical dtype

New in version 0.19.0.

Categorical columns can be parsed directly by specifying `dtype='category'` or `dtype=CategoricalDtype(categories, ordered)`.

```

In [29]: data = ('col1,col2,col3\n'
....: 'a,b,1\n'
....: 'a,b,2\n'
....: 'c,d,3')
....:

In [30]: pd.read_csv(StringIO(data))
Out[30]:
 col1 col2 col3
0 a b 1
1 a b 2
2 c d 3

In [31]: pd.read_csv(StringIO(data)).dtypes
Out[31]:
col1 object
col2 object
col3 int64
dtype: object

In [32]: pd.read_csv(StringIO(data), dtype='category').dtypes
Out[32]:
col1 category
col2 category
col3 category
dtype: object

```

Individual columns can be parsed as a `Categorical` using a dict specification:

```

In [33]: pd.read_csv(StringIO(data), dtype={'col1': 'category'}).dtypes
Out[33]:
col1 category
col2 object
col3 int64
dtype: object

```

New in version 0.21.0.

Specifying `dtype='category'` will result in an unordered `Categorical` whose categories are the unique values observed in the data. For more control on the categories and order, create a `CategoricalDtype` ahead of time, and pass that for that column's `dtype`.

```
In [34]: from pandas.api.types import CategoricalDtype

In [35]: dtype = CategoricalDtype(['d', 'c', 'b', 'a'], ordered=True)

In [36]: pd.read_csv(StringIO(data), dtype={'col1': dtype}).dtypes
Out[36]:
col1 category
col2 object
col3 int64
dtype: object
```

When using `dtype=CategoricalDtype`, “unexpected” values outside of `dtype.categories` are treated as missing values.

```
In [37]: dtype = CategoricalDtype(['a', 'b', 'd']) # No 'c'

In [38]: pd.read_csv(StringIO(data), dtype={'col1': dtype}).col1
Out[38]:
0 a
1 a
2 NaN
Name: col1, dtype: category
Categories (3, object): [a, b, d]
```

This matches the behavior of `Categorical.set_categories()`.

::: tip Note

With `dtype='category'`, the resulting categories will always be parsed as strings (object dtype). If the categories are numeric they can be converted using the `to_numeric()` function, or as appropriate, another converter such as `to_datetime()`.

When `dtype` is a `CategoricalDtype` with homogeneous `categories` (all numeric, all datetimes, etc.), the conversion is done automatically.

```

In [39]: df = pd.read_csv(StringIO(data), dtype='category')

In [40]: df.dtypes
Out[40]:
col1 category
col2 category
col3 category
dtype: object

In [41]: df['col3']
Out[41]:
0 1
1 2
2 3
Name: col3, dtype: category
Categories (3, object): [1, 2, 3]

In [42]: df['col3'].cat.categories = pd.to_numeric(df['col3'].cat.categories)

In [43]: df['col3']
Out[43]:
0 1
1 2
2 3
Name: col3, dtype: category
Categories (3, int64): [1, 2, 3]

```

...

## Naming and using columns

### Handling column names

A file may or may not have a header row. pandas assumes the first row should be used as the column names:

```
In [44]: data = ('a,b,c\n'
.....: '1,2,3\n'
.....: '4,5,6\n'
.....: '7,8,9')
.....:

In [45]: print(data)
a,b,c
1,2,3
4,5,6
7,8,9

In [46]: pd.read_csv(StringIO(data))
Out[46]:
 a b c
0 1 2 3
1 4 5 6
2 7 8 9
```

By specifying the `names` argument in conjunction with `header` you can indicate other names to use and whether or not to throw away the header row (if any):

```
In [47]: print(data)
a,b,c
1,2,3
4,5,6
7,8,9

In [48]: pd.read_csv(StringIO(data), names=['foo', 'bar', 'baz'], header=0)
Out[48]:
 foo bar baz
0 1 2 3
1 4 5 6
2 7 8 9

In [49]: pd.read_csv(StringIO(data), names=['foo', 'bar', 'baz'], header=None)
Out[49]:
 foo bar baz
0 a b c
1 1 2 3
2 4 5 6
3 7 8 9
```

If the header is in a row other than the first, pass the row number to `header`. This will skip the preceding rows:



```
In [50]: data = ('skip this skip it\n'
....: 'a,b,c\n'
....: '1,2,3\n'
....: '4,5,6\n'
....: '7,8,9')
....:

In [51]: pd.read_csv(StringIO(data), header=1)
Out[51]:
 a b c
0 1 2 3
1 4 5 6
2 7 8 9
```

⋮ tip Note

Default behavior is to infer the column names: if no names are passed the behavior is identical to `header=0` and column names are inferred from the first non-blank line of the file, if column names are passed explicitly then the behavior is identical to `header=None`.

⋮

## Duplicate names parsing

If the file or header contains duplicate names, pandas will by default distinguish between them so as to prevent overwriting data:

```
In [52]: data = ('a,b,a\n'
....: '0,1,2\n'
....: '3,4,5')
....:

In [53]: pd.read_csv(StringIO(data))
Out[53]:
 a b a.1
0 0 1 2
1 3 4 5
```

There is no more duplicate data because `mangle_dupe_cols=True` by default, which modifies a series of duplicate columns 'X' , ..., 'X' to become 'X' , 'X.1' , ..., 'X.N' . If `mangle_dupe_cols=False` , duplicate data can arise:

```
In [2]: data = 'a,b,a\n0,1,2\n3,4,5'
In [3]: pd.read_csv(StringIO(data), mangle_dupe_cols=False)
Out[3]:
 a b a
0 2 1 2
1 5 4 5
```

To prevent users from encountering this problem with duplicate data, a `ValueError` exception is raised if `mangle_dupe_cols != True`:

```
In [2]: data = 'a,b,a\n0,1,2\n3,4,5'
In [3]: pd.read_csv(StringIO(data), mangle_dupe_cols=False)
...
ValueError: Setting mangle_dupe_cols=False is not supported yet
```

## Filtering columns (usecols)

The `usecols` argument allows you to select any subset of the columns in a file, either using the column names, position numbers or a callable:

New in version 0.20.0: support for callable usecols arguments

```
In [54]: data = 'a,b,c,d\n1,2,3,foo\n4,5,6,bar\n7,8,9,baz'

In [55]: pd.read_csv(StringIO(data))
Out[55]:
 a b c d
0 1 2 3 foo
1 4 5 6 bar
2 7 8 9 baz

In [56]: pd.read_csv(StringIO(data), usecols=['b', 'd'])
Out[56]:
 b d
0 2 foo
1 5 bar
2 8 baz

In [57]: pd.read_csv(StringIO(data), usecols=[0, 2, 3])
Out[57]:
 a c d
0 1 3 foo
1 4 6 bar
2 7 9 baz

In [58]: pd.read_csv(StringIO(data), usecols=lambda x: x.upper() in ['A', 'C'])
Out[58]:
 a c
0 1 3
1 4 6
2 7 9
```

The `usecols` argument can also be used to specify which columns not to use in the final result:

```
In [59]: pd.read_csv(StringIO(data), usecols=lambda x: x not in ['a', 'c'])
Out[59]:
 b d
0 2 foo
1 5 bar
2 8 baz
```

In this case, the callable is specifying that we exclude the “a” and “c” columns from the output.

## Comments and empty lines

### Ignoring line comments and empty lines

If the `comment` parameter is specified, then completely commented lines will be ignored. By default, completely blank lines will be ignored as well.

```
In [60]: data = ('\n'
....: 'a,b,c\n'
....: ' \n'
....: '# commented line\n'
....: '1,2,3\n'
....: '\n'
....: '4,5,6')
....:

In [61]: print(data)

a,b,c

commented line
1,2,3

4,5,6

In [62]: pd.read_csv(StringIO(data), comment='#')
Out[62]:
 a b c
0 1 2 3
1 4 5 6
```

If `skip_blank_lines=False`, then `read_csv` will not ignore blank lines:

```
In [63]: data = ('a,b,c\n'
....: '\n'
....: '1,2,3\n'
....: '\n'
....: '\n'
....: '4,5,6')
....:

In [64]: pd.read_csv(StringIO(data), skip_blank_lines=False)
Out[64]:
```

	a	b	c
0	NaN	NaN	NaN
1	1.0	2.0	3.0
2	NaN	NaN	NaN
3	NaN	NaN	NaN
4	4.0	5.0	6.0

::: danger Warning

The presence of ignored lines might create ambiguities involving line numbers; the parameter `header` uses row numbers (ignoring commented/empty lines), while `skiprows` uses line numbers (including commented/empty lines):

```
In [65]: data = ('#comment\n'
....: 'a,b,c\n'
....: 'A,B,C\n'
....: '1,2,3')
....:

In [66]: pd.read_csv(StringIO(data), comment='#', header=1)
Out[66]:
```

	A	B	C
0	1	2	3

```
In [67]: data = ('A,B,C\n'
....: '#comment\n'
....: 'a,b,c\n'
....: '1,2,3')
....:

In [68]: pd.read_csv(StringIO(data), comment='#', skiprows=2)
Out[68]:
```

	a	b	c
0	1	2	3

If both `header` and `skiprows` are specified, `header` will be relative to the end of `skiprows`. For example:

:::

```
In [69]: data = ('# empty\n'
....: '# second empty line\n'
....: '# third emptyline\n'
....: 'X,Y,Z\n'
....: '1,2,3\n'
....: 'A,B,C\n'
....: '1,2.,4.\n'
....: '5.,NaN,10.0\n')
....:

In [70]: print(data)
empty
second empty line
third emptyline
X,Y,Z
1,2,3
A,B,C
1,2.,4.
5.,NaN,10.0

In [71]: pd.read_csv(StringIO(data), comment='#', skiprows=4, header=1)
Out[71]:
 A B C
0 1.0 2.0 4.0
1 5.0 NaN 10.0
```

## Comments

Sometimes comments or meta data may be included in a file:

```
In [72]: print(open('tmp.csv').read())
ID,level,category
Patient1,123000,x # really unpleasant
Patient2,23000,y # wouldn't take his medicine
Patient3,1234018,z # awesome
```

By default, the parser includes the comments in the output:

```
In [73]: df = pd.read_csv('tmp.csv')

In [74]: df
Out[74]:
 ID level category
0 Patient1 123000 x # really unpleasant
1 Patient2 23000 y # wouldn't take his medicine
2 Patient3 1234018 z # awesome
```

We can suppress the comments using the `comment` keyword:

```
In [75]: df = pd.read_csv('tmp.csv', comment='#')
```

```
In [76]: df
```

```
Out[76]:
```

	ID	level	category
0	Patient1	123000	x
1	Patient2	23000	y
2	Patient3	1234018	z

## Dealing with Unicode data

The `encoding` argument should be used for encoded unicode data, which will result in byte strings being decoded to unicode in the result:

```
In [77]: data = (b'word,length\n'
```

```
.....: b'Tr\xc3\xa4umen,7\n'
```

```
.....: b'Gr\xc3\xbc\xc3\x9fe,5')
```

```
.....:
```

```
In [78]: data = data.decode('utf8').encode('latin-1')
```

```
In [79]: df = pd.read_csv(BytesIO(data), encoding='latin-1')
```

```
In [80]: df
```

```
Out[80]:
```

	word	length
0	Träumen	7
1	Grüße	5

```
In [81]: df['word'][1]
```

```
Out[81]: 'Grüße'
```

Some formats which encode all characters as multiple bytes, like UTF-16, won't parse correctly at all without specifying the encoding. [Full list of Python standard encodings](#).

## Index columns and trailing delimiters

If a file has one more column of data than the number of column names, the first column will be used as the `DataFrame`'s row names:

```
In [82]: data = ('a,b,c\n'
```

```
.....: '4,apple,bat,5.7\n'
```

```
.....: '8,orange,cow,10')
```

```
.....:
```

```
In [83]: pd.read_csv(StringIO(data))
```

```
Out[83]:
```

	a	b	c
4	apple	bat	5.7
8	orange	cow	10.0

```
In [84]: data = ('index,a,b,c\n'
....: '4,apple,bat,5.7\n'
....: '8,orange,cow,10')
....:

In [85]: pd.read_csv(StringIO(data), index_col=0)
Out[85]:
```

	a	b	c
index			
4	apple	bat	5.7
8	orange	cow	10.0

Ordinarily, you can achieve this behavior using the `index_col` option.

There are some exception cases when a file has been prepared with delimiters at the end of each data line, confusing the parser. To explicitly disable the index column inference and discard the last column, pass `index_col=False`:

```
In [86]: data = ('a,b,c\n'
....: '4,apple,bat,\n'
....: '8,orange,cow,')
....:

In [87]: print(data)
a,b,c
4,apple,bat,
8,orange,cow,

In [88]: pd.read_csv(StringIO(data))
Out[88]:
```

	a	b	c
4	apple	bat	NaN
8	orange	cow	NaN

```
In [89]: pd.read_csv(StringIO(data), index_col=False)
Out[89]:
```

	a	b	c
0	4	apple	bat
1	8	orange	cow

If a subset of data is being parsed using the `usecols` option, the `index_col` specification is based on that subset, not the original data.

```

In [90]: data = ('a,b,c\n'
....: '4,apple,bat,\n'
....: '8,orange,cow,')
....:

In [91]: print(data)
a,b,c
4,apple,bat,
8,orange,cow,

In [92]: pd.read_csv(StringIO(data), usecols=['b', 'c'])
Out[92]:
 b c
4 bat NaN
8 cow NaN

In [93]: pd.read_csv(StringIO(data), usecols=['b', 'c'], index_col=0)
Out[93]:
 b c
4 bat NaN
8 cow NaN

```

## Date Handling

### Specifying date columns

To better facilitate working with datetime data, `read_csv()` uses the keyword arguments `parse_dates` and `date_parser` to allow users to specify a variety of columns and date/time formats to turn the input text data into `datetime` objects.

The simplest case is to just pass in `parse_dates=True`:

```

Use a column as an index, and parse it as dates.
In [94]: df = pd.read_csv('foo.csv', index_col=0, parse_dates=True)

In [95]: df
Out[95]:
 A B C
date
2009-01-01 a 1 2
2009-01-02 b 3 4
2009-01-03 c 4 5

These are Python datetime objects
In [96]: df.index
Out[96]: DatetimeIndex(['2009-01-01', '2009-01-02', '2009-01-03'], dtype='datetime64[ns]', name='date',
 freq=None)

```

It is often the case that we may want to store date and time data separately, or store various date fields separately. the `parse_dates` keyword can be used to specify a combination of columns to parse the dates and/or times from.



You can specify a list of column lists to `parse_dates`, the resulting date columns will be prepended to the output (so as to not affect the existing column order) and the new column names will be the concatenation of the component column names:

```
In [97]: print(open('tmp.csv').read())
KORD,19990127, 19:00:00, 18:56:00, 0.8100
KORD,19990127, 20:00:00, 19:56:00, 0.0100
KORD,19990127, 21:00:00, 20:56:00, -0.5900
KORD,19990127, 21:00:00, 21:18:00, -0.9900
KORD,19990127, 22:00:00, 21:56:00, -0.5900
KORD,19990127, 23:00:00, 22:56:00, -0.5900

In [98]: df = pd.read_csv('tmp.csv', header=None, parse_dates=[[1, 2], [1, 3]])

In [99]: df
Out[99]:
```

	1_2	1_3	0	4
0	1999-01-27 19:00:00	1999-01-27 18:56:00	KORD	0.81
1	1999-01-27 20:00:00	1999-01-27 19:56:00	KORD	0.01
2	1999-01-27 21:00:00	1999-01-27 20:56:00	KORD	-0.59
3	1999-01-27 21:00:00	1999-01-27 21:18:00	KORD	-0.99
4	1999-01-27 22:00:00	1999-01-27 21:56:00	KORD	-0.59
5	1999-01-27 23:00:00	1999-01-27 22:56:00	KORD	-0.59

By default the parser removes the component date columns, but you can choose to retain them via the `keep_date_col` keyword:

```
In [100]: df = pd.read_csv('tmp.csv', header=None, parse_dates=[[1, 2], [1, 3]],
.....: keep_date_col=True)
.....:

In [101]: df
Out[101]:
```

	1_2	1_3	0	1	2	3	4
0	1999-01-27 19:00:00	1999-01-27 18:56:00	KORD	19990127	19:00:00	18:56:00	0.81
1	1999-01-27 20:00:00	1999-01-27 19:56:00	KORD	19990127	20:00:00	19:56:00	0.01
2	1999-01-27 21:00:00	1999-01-27 20:56:00	KORD	19990127	21:00:00	20:56:00	-0.59
3	1999-01-27 21:00:00	1999-01-27 21:18:00	KORD	19990127	21:00:00	21:18:00	-0.99
4	1999-01-27 22:00:00	1999-01-27 21:56:00	KORD	19990127	22:00:00	21:56:00	-0.59
5	1999-01-27 23:00:00	1999-01-27 22:56:00	KORD	19990127	23:00:00	22:56:00	-0.59

Note that if you wish to combine multiple columns into a single date column, a nested list must be used. In other words, `parse_dates=[1, 2]` indicates that the second and third columns should each be parsed as separate date columns while `parse_dates=[[1, 2]]` means the two columns should be parsed into a single column.

You can also use a dict to specify custom name columns:

```
In [102]: date_spec = {'nominal': [1, 2], 'actual': [1, 3]}

In [103]: df = pd.read_csv('tmp.csv', header=None, parse_dates=date_spec)

In [104]: df
Out[104]:
```

	nominal	actual	0	4
0	1999-01-27 19:00:00	1999-01-27 18:56:00	KORD	0.81
1	1999-01-27 20:00:00	1999-01-27 19:56:00	KORD	0.01
2	1999-01-27 21:00:00	1999-01-27 20:56:00	KORD	-0.59
3	1999-01-27 21:00:00	1999-01-27 21:18:00	KORD	-0.99
4	1999-01-27 22:00:00	1999-01-27 21:56:00	KORD	-0.59
5	1999-01-27 23:00:00	1999-01-27 22:56:00	KORD	-0.59

It is important to remember that if multiple text columns are to be parsed into a single date column, then a new column is prepended to the data. The `index_col` specification is based off of this new set of columns rather than the original data columns:

```
In [105]: date_spec = {'nominal': [1, 2], 'actual': [1, 3]}

In [106]: df = pd.read_csv('tmp.csv', header=None, parse_dates=date_spec,
.....: index_col=0) # index is the nominal column
.....:

In [107]: df
Out[107]:
```

	nominal	actual	0	4
1999-01-27 19:00:00	1999-01-27 18:56:00	KORD	0.81	
1999-01-27 20:00:00	1999-01-27 19:56:00	KORD	0.01	
1999-01-27 21:00:00	1999-01-27 20:56:00	KORD	-0.59	
1999-01-27 21:00:00	1999-01-27 21:18:00	KORD	-0.99	
1999-01-27 22:00:00	1999-01-27 21:56:00	KORD	-0.59	
1999-01-27 23:00:00	1999-01-27 22:56:00	KORD	-0.59	

::: tip Note

If a column or index contains an unparseable date, the entire column or index will be returned unaltered as an object data type. For non-standard datetime parsing, use `to_datetime()` after `pd.read_csv`.

:::

::: tip Note

`read_csv` has a `fast_path` for parsing datetime strings in iso8601 format, e.g “2000-01-01T00:01:02+00:00” and similar variations. If you can arrange for your data to store datetimes in this format, load times will be significantly faster, ~20x has been observed.

:::

::: tip Note

When passing a dict as the `parse_dates` argument, the order of the columns prepended is not guaranteed, because dict objects do not impose an ordering on their keys. On Python 2.7+ you may use `collections.OrderedDict` instead of a regular dict if this matters to you. Because of this, when using a dict for `'parse_dates'` in conjunction with the `index_col` argument, it's best to specify `index_col` as a column label rather than as an index on the resulting frame.

:::

## Date parsing functions

Finally, the parser allows you to specify a custom `date_parser` function to take full advantage of the flexibility of the date parsing API:

```
In [108]: df = pd.read_csv('tmp.csv', header=None, parse_dates=date_spec,
.....: date_parser=pd.io.date_converters.parse_date_time)
.....:
```

```
In [109]: df
```

```
Out[109]:
```

	nominal	actual	0	4
0	1999-01-27 19:00:00	1999-01-27 18:56:00	KORD	0.81
1	1999-01-27 20:00:00	1999-01-27 19:56:00	KORD	0.01
2	1999-01-27 21:00:00	1999-01-27 20:56:00	KORD	-0.59
3	1999-01-27 21:00:00	1999-01-27 21:18:00	KORD	-0.99
4	1999-01-27 22:00:00	1999-01-27 21:56:00	KORD	-0.59
5	1999-01-27 23:00:00	1999-01-27 22:56:00	KORD	-0.59

Pandas will try to call the `date_parser` function in three different ways. If an exception is raised, the next one is tried:

1. `date_parser` is first called with one or more arrays as arguments, as defined using `parse_dates` (e.g., `date_parser(['2013', '2013'], ['1', '2'])`).
2. If #1 fails, `date_parser` is called with all the columns concatenated row-wise into a single array (e.g., `date_parser(['2013 1', '2013 2'])`).
3. If #2 fails, `date_parser` is called once for every row with one or more string arguments from the columns indicated with `parse_dates` (e.g., `date_parser('2013', '1')` for the first row, `date_parser('2013', '2')` for the second, etc.).

Note that performance-wise, you should try these methods of parsing dates in order:

1. Try to infer the format using `infer_datetime_format=True` (see section below).
2. If you know the format, use `pd.to_datetime(): date_parser=lambda x: pd.to_datetime(x, format=...)`.
3. If you have a really non-standard format, use a custom `date_parser` function. For optimal performance, this should be vectorized, i.e., it should accept arrays as arguments.

You can explore the date parsing functionality in [date\\_converters.py](#) and add your own. We would love to turn this module into a community supported set of date/time parsers. To get you started, `date_converters.py` contains functions to parse dual date and time columns, year/month/day columns, and year/month/day/hour/minute/second columns. It also contains a `generic_parser` function so you can curry it with a function that deals with a single date rather than the entire array.

## Parsing a CSV with mixed timezones

Pandas cannot natively represent a column or index with mixed timezones. If your CSV file contains columns with a mixture of timezones, the default result will be an object-dtype column with strings, even with `parse_dates`.

```
In [110]: content = """\
.....: a
.....: 2000-01-01T00:00:00+05:00
.....: 2000-01-01T00:00:00+06:00"""
.....:

In [111]: df = pd.read_csv(StringIO(content), parse_dates=['a'])

In [112]: df['a']
Out[112]:
0 2000-01-01 00:00:00+05:00
1 2000-01-01 00:00:00+06:00
Name: a, dtype: object
```

To parse the mixed-timezone values as a datetime column, pass a partially-applied `to_datetime()` with `utc=True` as the `date_parser`.

```
In [113]: df = pd.read_csv(StringIO(content), parse_dates=['a'],
.....: date_parser=lambda col: pd.to_datetime(col, utc=True))
.....:

In [114]: df['a']
Out[114]:
0 1999-12-31 19:00:00+00:00
1 1999-12-31 18:00:00+00:00
Name: a, dtype: datetime64[ns, UTC]
```

## Inferring datetime format

If you have `parse_dates` enabled for some or all of your columns, and your datetime strings are all formatted the same way, you may get a large speed up by setting `infer_datetime_format=True`. If set, pandas will attempt to guess the format of your datetime strings, and then use a faster means of parsing the strings. 5-10x parsing speeds have been observed. pandas will fallback to the usual parsing if either the format cannot be guessed or the format that was guessed cannot properly parse

the entire column of strings. So in general, `infer_datetime_format` should not have any negative consequences if enabled.

Here are some examples of datetime strings that can be guessed (All representing December 30th, 2011 at 00:00:00):

- “20111230”
- “2011/12/30”
- “20111230 00:00:00”
- “12/30/2011 00:00:00”
- “30/Dec/2011 00:00:00”
- “30/December/2011 00:00:00”

Note that `infer_datetime_format` is sensitive to `dayfirst`. With `dayfirst=True`, it will guess “01/12/2011” to be December 1st. With `dayfirst=False` (default) it will guess “01/12/2011” to be January 12th.

```
Try to infer the format for the index column
In [115]: df = pd.read_csv('foo.csv', index_col=0, parse_dates=True,
.....: infer_datetime_format=True)
.....:

In [116]: df
Out[116]:
```

	A	B	C
date			
2009-01-01	a	1	2
2009-01-02	b	3	4
2009-01-03	c	4	5

## International date formats

While US date formats tend to be MM/DD/YYYY, many international formats use DD/MM/YYYY instead. For convenience, a `dayfirst` keyword is provided:

```
In [117]: print(open('tmp.csv').read())
date,value,cat
1/6/2000,5,a
2/6/2000,10,b
3/6/2000,15,c

In [118]: pd.read_csv('tmp.csv', parse_dates=[0])
Out[118]:
 date value cat
0 2000-01-06 5 a
1 2000-02-06 10 b
2 2000-03-06 15 c

In [119]: pd.read_csv('tmp.csv', dayfirst=True, parse_dates=[0])
Out[119]:
 date value cat
0 2000-06-01 5 a
1 2000-06-02 10 b
2 2000-06-03 15 c
```

## Specifying method for floating-point conversion

The parameter `float_precision` can be specified in order to use a specific floating-point converter during parsing with the C engine. The options are the ordinary converter, the high-precision converter, and the round-trip converter (which is guaranteed to round-trip values after writing to a file). For example:

```
In [120]: val = '0.3066101993807095471566981359501369297504425048828125'

In [121]: data = 'a,b,c\n1,2,{0}'.format(val)

In [122]: abs(pd.read_csv(StringIO(data), engine='c',
.....: float_precision=None)['c'][0] - float(val))
.....:
Out[122]: 1.1102230246251565e-16

In [123]: abs(pd.read_csv(StringIO(data), engine='c',
.....: float_precision='high')['c'][0] - float(val))
.....:
Out[123]: 5.551115123125783e-17

In [124]: abs(pd.read_csv(StringIO(data), engine='c',
.....: float_precision='round_trip')['c'][0] - float(val))
.....:
Out[124]: 0.0
```

## Thousand separators

For large numbers that have been written with a thousands separator, you can set the `thousands` keyword to a string of length 1 so that integers will be parsed correctly:

By default, numbers with a thousands separator will be parsed as strings:

```
In [125]: print(open('tmp.csv').read())
ID|level|category
Patient1|123,000|x
Patient2|23,000|y
Patient3|1,234,018|z

In [126]: df = pd.read_csv('tmp.csv', sep='|')

In [127]: df
Out[127]:
 ID level category
0 Patient1 123,000 x
1 Patient2 23,000 y
2 Patient3 1,234,018 z

In [128]: df.level.dtype
Out[128]: dtype('O')
```

The `thousands` keyword allows integers to be parsed correctly:

```
In [129]: print(open('tmp.csv').read())
ID|level|category
Patient1|123,000|x
Patient2|23,000|y
Patient3|1,234,018|z

In [130]: df = pd.read_csv('tmp.csv', sep='|', thousands=',')

In [131]: df
Out[131]:
 ID level category
0 Patient1 123000 x
1 Patient2 23000 y
2 Patient3 1234018 z

In [132]: df.level.dtype
Out[132]: dtype('int64')
```

## NA values

To control which values are parsed as missing values (which are signified by `NaN`), specify a string in `na_values`. If you specify a list of strings, then all values in it are considered to be missing values. If you specify a number (a `float`, like `5.0` or an `integer` like `5`), the corresponding equivalent values will also imply a missing value (in this case effectively `[5.0, 5]` are recognized as `NaN`).

To completely override the default values that are recognized as missing, specify `keep_default_na=False`.

The default `NaN` recognized values are

```
['-1.#IND', '1.#QNAN', '1.#IND', '-1.#QNAN', '#N/A N/A', '#N/A', 'N/A', 'n/a', 'NA', '#NA', 'NULL', 'null', 'NaN', '-NaN', 'nan', '-nan', '']
```

Let us consider some examples:

```
pd.read_csv('path_to_file.csv', na_values=[5])
```

In the example above `5` and `5.0` will be recognized as `NaN`, in addition to the defaults. A string will first be interpreted as a numerical `5`, then as a `NaN`.

```
pd.read_csv('path_to_file.csv', keep_default_na=False, na_values=[""])
```

Above, only an empty field will be recognized as `NaN`.

```
pd.read_csv('path_to_file.csv', keep_default_na=False, na_values=["NA", "0"])
```

Above, both `NA` and `0` as strings are `NaN`.

```
pd.read_csv('path_to_file.csv', na_values=["Nope"])
```

The default values, in addition to the string `"Nope"` are recognized as `NaN`.

## Infinity

`inf` like values will be parsed as `np.inf` (positive infinity), and `-inf` as `-np.inf` (negative infinity). These will ignore the case of the value, meaning `Inf`, will also be parsed as `np.inf`.

## Returning Series

Using the `squeeze` keyword, the parser will return output with a single column as a `Series`:



```

In [133]: print(open('tmp.csv').read())
level
Patient1,123000
Patient2,23000
Patient3,1234018

In [134]: output = pd.read_csv('tmp.csv', squeeze=True)

In [135]: output
Out[135]:
Patient1 123000
Patient2 23000
Patient3 1234018
Name: level, dtype: int64

In [136]: type(output)
Out[136]: pandas.core.series.Series

```

## Boolean values

The common values `True`, `False`, `TRUE`, and `FALSE` are all recognized as boolean. Occasionally you might want to recognize other values as being boolean. To do this, use the `true_values` and `false_values` options as follows:

```

In [137]: data = ('a,b,c\n'
.....: '1,Yes,2\n'
.....: '3,No,4')
.....:

In [138]: print(data)
a,b,c
1,Yes,2
3,No,4

In [139]: pd.read_csv(StringIO(data))
Out[139]:
 a b c
0 1 Yes 2
1 3 No 4

In [140]: pd.read_csv(StringIO(data), true_values=['Yes'], false_values=['No'])
Out[140]:
 a b c
0 1 True 2
1 3 False 4

```

## Handling “bad” lines

Some files may have malformed lines with too few fields or too many. Lines with too few fields will have NA values filled in the trailing fields. Lines with too many fields will raise an error by default:

```
In [141]: data = ('a,b,c\n'
.....: '1,2,3\n'
.....: '4,5,6,7\n'
.....: '8,9,10')
.....:
```

```
In [142]: pd.read_csv(StringIO(data))
```

```

ParserError Traceback (most recent call last)
<ipython-input-142-6388c394e6b8> in <module>
----> 1 pd.read_csv(StringIO(data))

/pandas/pandas/io/parsers.py in parser_f(filepath_or_buffer, sep, delimiter, header, names, index_col,
 usecols, squeeze, prefix, mangle_dupe_cols, dtype, engine, converters, true_values,
 false_values, skipinitialspace, skiprows, skipfooter, nrows, na_values, keep_default_na,
 na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format, keep_date_col,
 date_parser, dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal,
 lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, dialect,
 error_bad_lines, warn_bad_lines, delim_whitespace, low_memory, memory_map, float_precision)
 683)
 684
--> 685 return _read(filepath_or_buffer, kwds)
 686
 687 parser_f.__name__ = name

/pandas/pandas/io/parsers.py in _read(filepath_or_buffer, kwds)
 461
 462 try:
--> 463 data = parser.read(nrows)
 464 finally:
 465 parser.close()

/pandas/pandas/io/parsers.py in read(self, nrows)
 1152 def read(self, nrows=None):
 1153 nrows = _validate_integer("nrows", nrows)
-> 1154 ret = self._engine.read(nrows)
 1155
 1156 # May alter columns / col_dict

/pandas/pandas/io/parsers.py in read(self, nrows)
 2046 def read(self, nrows=None):
 2047 try:
-> 2048 data = self._reader.read(nrows)
 2049 except StopIteration:
 2050 if self._first_chunk:

/pandas/pandas/_libs/parsers.pyx in pandas._libs.parsers.TextReader.read()

/pandas/pandas/_libs/parsers.pyx in pandas._libs.parsers.TextReader._read_low_memory()

/pandas/pandas/_libs/parsers.pyx in pandas._libs.parsers.TextReader._read_rows()

/pandas/pandas/_libs/parsers.pyx in pandas._libs.parsers.TextReader._tokenize_rows()

/pandas/pandas/_libs/parsers.pyx in pandas._libs.parsers.raise_parser_error()
```

```
ParserError: Error tokenizing data. C error: Expected 3 fields in line 3, saw 4
```

You can elect to skip bad lines:

```
In [29]: pd.read_csv(StringIO(data), error_bad_lines=False)
Skipping line 3: expected 3 fields, saw 4
```

```
Out[29]:
 a b c
0 1 2 3
1 8 9 10
```

You can also use the `usecols` parameter to eliminate extraneous column data that appear in some lines but not others:

```
In [30]: pd.read_csv(StringIO(data), usecols=[0, 1, 2])
```

```
Out[30]:
 a b c
0 1 2 3
1 4 5 6
2 8 9 10
```

## Dialect

The `dialect` keyword gives greater flexibility in specifying the file format. By default it uses the Excel dialect but you can specify either the dialect name or a `csv.Dialect` instance.

Suppose you had data with unenclosed quotes:

```
In [143]: print(data)
label1,label2,label3
index1,"a,c,e
index2,b,d,f
```

By default, `read_csv` uses the Excel dialect and treats the double quote as the quote character, which causes it to fail when it finds a newline before it finds the closing double quote.

We can get around this using `dialect`:

```
In [144]: import csv

In [145]: dia = csv.excel()

In [146]: dia.quoting = csv.QUOTE_NONE

In [147]: pd.read_csv(StringIO(data), dialect=dia)
Out[147]:
```

	label1	label2	label3
index1	"a	c	e
index2	b	d	f

All of the dialect options can be specified separately by keyword arguments:

```
In [148]: data = 'a,b,c~1,2,3~4,5,6'

In [149]: pd.read_csv(StringIO(data), lineterminator='~')
Out[149]:
```

	a	b	c
0	1	2	3
1	4	5	6

Another common dialect option is `skipinitialspace`, to skip any whitespace after a delimiter:

```
In [150]: data = 'a, b, c\n1, 2, 3\n4, 5, 6'

In [151]: print(data)
a, b, c
1, 2, 3
4, 5, 6

In [152]: pd.read_csv(StringIO(data), skipinitialspace=True)
Out[152]:
```

	a	b	c
0	1	2	3
1	4	5	6

The parsers make every attempt to “do the right thing” and not be fragile. Type inference is a pretty big deal. If a column can be coerced to integer dtype without altering the contents, the parser will do so. Any non-numeric columns will come through as object dtype as with the rest of pandas objects.

## Quoting and Escape Characters

Quotes (and other escape characters) in embedded fields can be handled in any number of ways. One way is to use backslashes; to properly parse this data, you should pass the `escapechar` option:

```
In [153]: data = 'a,b\n"hello, \\"Bob\\", nice to see you",5'

In [154]: print(data)
a,b
"hello, \"Bob\", nice to see you",5

In [155]: pd.read_csv(StringIO(data), escapechar='\\')
Out[155]:
```

	a	b
0	hello, "Bob", nice to see you	5

## Files with fixed width columns

While `read_csv()` reads delimited data, the `read_fwf()` function works with data files that have known and fixed column widths. The function parameters to `read_fwf` are largely the same as `read_csv` with two extra parameters, and a different usage of the `delimiter` parameter:

- `colspecs`: A list of pairs (tuples) giving the extents of the fixed-width fields of each line as half-open intervals (i.e., [from, to[ ). String value 'infer' can be used to instruct the parser to try detecting the column specifications from the first 100 rows of the data. Default behavior, if not specified, is to infer.
- `widths`: A list of field widths which can be used instead of 'colspecs' if the intervals are contiguous.
- `delimiter`: Characters to consider as filler characters in the fixed-width file. Can be used to specify the filler character of the fields if it is not spaces (e.g., '~' ).

Consider a typical fixed-width data file:

```
In [156]: print(open('bar.csv').read())
id8141 360.242940 149.910199 11950.7
id1594 444.953632 166.985655 11788.4
id1849 364.136849 183.628767 11806.2
id1230 413.836124 184.375703 11916.8
id1948 502.953953 173.237159 12468.3
```

In order to parse this file into a `DataFrame`, we simply need to supply the column specifications to the `read_fwf` function along with the file name:

```
Column specifications are a list of half-intervals
In [157]: colspecs = [(0, 6), (8, 20), (21, 33), (34, 43)]

In [158]: df = pd.read_fwf('bar.csv', colspecs=colspecs, header=None, index_col=0)

In [159]: df
Out[159]:
```

	1	2	3
0			
id8141	360.242940	149.910199	11950.7
id1594	444.953632	166.985655	11788.4
id1849	364.136849	183.628767	11806.2
id1230	413.836124	184.375703	11916.8
id1948	502.953953	173.237159	12468.3

Note how the parser automatically picks column names `X.<column number>` when `header=None` argument is specified. Alternatively, you can supply just the column widths for contiguous columns:

```
Widths are a list of integers
In [160]: widths = [6, 14, 13, 10]

In [161]: df = pd.read_fwf('bar.csv', widths=widths, header=None)

In [162]: df
Out[162]:
```

	0	1	2	3
0	id8141	360.242940	149.910199	11950.7
1	id1594	444.953632	166.985655	11788.4
2	id1849	364.136849	183.628767	11806.2
3	id1230	413.836124	184.375703	11916.8
4	id1948	502.953953	173.237159	12468.3

The parser will take care of extra white spaces around the columns so it's ok to have extra separation between the columns in the file.

By default, `read_fwf` will try to infer the file's `colspecs` by using the first 100 rows of the file. It can do it only in cases when the columns are aligned and correctly separated by the provided `delimiter` (default delimiter is whitespace).

```
In [163]: df = pd.read_fwf('bar.csv', header=None, index_col=0)

In [164]: df
Out[164]:
```

	1	2	3
0			
id8141	360.242940	149.910199	11950.7
id1594	444.953632	166.985655	11788.4
id1849	364.136849	183.628767	11806.2
id1230	413.836124	184.375703	11916.8
id1948	502.953953	173.237159	12468.3

New in version 0.20.0.

`read_fwf` supports the `dtype` parameter for specifying the types of parsed columns to be different from the inferred type.

```
In [165]: pd.read_fwf('bar.csv', header=None, index_col=0).dtypes
Out[165]:
1 float64
2 float64
3 float64
dtype: object

In [166]: pd.read_fwf('bar.csv', header=None, dtype={2: 'object'}).dtypes
Out[166]:
0 object
1 float64
2 object
3 float64
dtype: object
```

## Indexes

### Files with an “implicit” index column

Consider a file with one less entry in the header than the number of data column:

```
In [167]: print(open('foo.csv').read())
A,B,C
20090101,a,1,2
20090102,b,3,4
20090103,c,4,5
```

In this special case, `read_csv` assumes that the first column is to be used as the index of the `DataFrame`:

```
In [168]: pd.read_csv('foo.csv')
Out[168]:
 A B C
20090101 a 1 2
20090102 b 3 4
20090103 c 4 5
```

Note that the dates weren't automatically parsed. In that case you would need to do as before:

```
In [169]: df = pd.read_csv('foo.csv', parse_dates=True)

In [170]: df.index
Out[170]: DatetimeIndex(['2009-01-01', '2009-01-02', '2009-01-03'], dtype='datetime64[ns]', freq=None)
```

## Reading an index with a MultiIndex

Suppose you have data indexed by two columns:

```
In [171]: print(open('data/mindex_ex.csv').read())
year,indiv,zit,xit
1977,"A",1.2,.6
1977,"B",1.5,.5
1977,"C",1.7,.8
1978,"A",.2,.06
1978,"B",.7,.2
1978,"C",.8,.3
1978,"D",.9,.5
1978,"E",1.4,.9
1979,"C",.2,.15
1979,"D",.14,.05
1979,"E",.5,.15
1979,"F",1.2,.5
1979,"G",3.4,1.9
1979,"H",5.4,2.7
1979,"I",6.4,1.2
```

The `index_col` argument to `read_csv` can take a list of column numbers to turn multiple columns into a `MultiIndex` for the index of the returned object:



```
In [172]: df = pd.read_csv("data/minindex_ex.csv", index_col=[0, 1])
```

```
In [173]: df
```

```
Out[173]:
```

		zit	xit
year	indiv		
1977	A	1.20	0.60
	B	1.50	0.50
	C	1.70	0.80
1978	A	0.20	0.06
	B	0.70	0.20
	C	0.80	0.30
	D	0.90	0.50
	E	1.40	0.90
1979	C	0.20	0.15
	D	0.14	0.05
	E	0.50	0.15
	F	1.20	0.50
	G	3.40	1.90
	H	5.40	2.70
	I	6.40	1.20

```
In [174]: df.loc[1978]
```

```
Out[174]:
```

	zit	xit
indiv		
A	0.2	0.06
B	0.7	0.20
C	0.8	0.30
D	0.9	0.50
E	1.4	0.90

## Reading columns with a MultiIndex

By specifying list of row locations for the `header` argument, you can read in a `MultiIndex` for the columns. Specifying non-consecutive rows will skip the intervening rows.

```
In [175]: from pandas.util.testing import makeCustomDataframe as mkdf
```

```
In [176]: df = mkdf(5, 3, r_idx_nlevels=2, c_idx_nlevels=4)
```

```
In [177]: df.to_csv('mi.csv')
```

```
In [178]: print(open('mi.csv').read())
```

```
C0,,C_l0_g0,C_l0_g1,C_l0_g2
C1,,C_l1_g0,C_l1_g1,C_l1_g2
C2,,C_l2_g0,C_l2_g1,C_l2_g2
C3,,C_l3_g0,C_l3_g1,C_l3_g2
R0,R1,,,
R_l0_g0,R_l1_g0,R0C0,R0C1,R0C2
R_l0_g1,R_l1_g1,R1C0,R1C1,R1C2
R_l0_g2,R_l1_g2,R2C0,R2C1,R2C2
R_l0_g3,R_l1_g3,R3C0,R3C1,R3C2
R_l0_g4,R_l1_g4,R4C0,R4C1,R4C2
```

```
In [179]: pd.read_csv('mi.csv', header=[0, 1, 2, 3], index_col=[0, 1])
```

```
Out[179]:
```

C0		C_l0_g0	C_l0_g1	C_l0_g2
C1		C_l1_g0	C_l1_g1	C_l1_g2
C2		C_l2_g0	C_l2_g1	C_l2_g2
C3		C_l3_g0	C_l3_g1	C_l3_g2
R0	R1			
R_l0_g0	R_l1_g0	R0C0	R0C1	R0C2
R_l0_g1	R_l1_g1	R1C0	R1C1	R1C2
R_l0_g2	R_l1_g2	R2C0	R2C1	R2C2
R_l0_g3	R_l1_g3	R3C0	R3C1	R3C2
R_l0_g4	R_l1_g4	R4C0	R4C1	R4C2

`read_csv` is also able to interpret a more common format of multi-columns indices.

```
In [180]: print(open('mi2.csv').read())
```

```
,a,a,a,b,c,c
,q,r,s,t,u,v
one,1,2,3,4,5,6
two,7,8,9,10,11,12
```

```
In [181]: pd.read_csv('mi2.csv', header=[0, 1], index_col=0)
```

```
Out[181]:
```

	a		b	c	
	q	r	s	t	u v
one	1	2	3	4	5 6
two	7	8	9	10	11 12

Note: If an `index_col` is not specified (e.g. you don't have an index, or wrote it with `df.to_csv(..., index=False)`), then any `names` on the columns index will be lost.

## Automatically “sniffing” the delimiter

`read_csv` is capable of inferring delimited (not necessarily comma-separated) files, as pandas uses the `csv.Sniffer` class of the `csv` module. For this, you have to specify `sep=None`.

```
In [182]: print(open('tmp2.csv').read())
:0:1:2:3
0:0.4691122999071863:-0.2828633443286633:-1.5090585031735124:-1.1356323710171934
1:1.2121120250208506:-0.17321464905330858:0.11920871129693428:-1.0442359662799567
2:-0.8618489633477999:-2.1045692188948086:-0.4949292740687813:1.071803807037338
3:0.7215551622443669:-0.7067711336300845:-1.0395749851146963:0.27185988554282986
4:-0.42497232978883753:0.567020349793672:0.27623201927771873:-1.0874006912859915
5:-0.6736897080883706:0.1136484096888855:-1.4784265524372235:0.5249876671147047
6:0.4047052186802365:0.5770459859204836:-1.7150020161146375:-1.0392684835147725
7:-0.3706468582364464:-1.1578922506419993:-1.344311812731667:0.8448851414248841
8:1.0757697837155533:-0.10904997528022223:1.6435630703622064:-1.4693879595399115
9:0.35702056413309086:-0.6746001037299882:-1.776903716971867:-0.9689138124473498
```

```
In [183]: pd.read_csv('tmp2.csv', sep=None, engine='python')
```

```
Out[183]:
```

	Unnamed: 0	0	1	2	3
0	0	0.469112	-0.282863	-1.509059	-1.135632
1	1	1.212112	-0.173215	0.119209	-1.044236
2	2	-0.861849	-2.104569	-0.494929	1.071804
3	3	0.721555	-0.706771	-1.039575	0.271860
4	4	-0.424972	0.567020	0.276232	-1.087401
5	5	-0.673690	0.113648	-1.478427	0.524988
6	6	0.404705	0.577046	-1.715002	-1.039268
7	7	-0.370647	-1.157892	-1.344312	0.844885
8	8	1.075770	-0.109050	1.643563	-1.469388
9	9	0.357021	-0.674600	-1.776904	-0.968914

## Reading multiple files to create a single DataFrame

It's best to use `concat()` to combine multiple files. See the [cookbook](#) for an example.

## Iterating through files chunk by chunk

Suppose you wish to iterate through a (potentially very large) file lazily rather than reading the entire file into memory, such as the following:

```
In [184]: print(open('tmp.csv').read())
|0|1|2|3
0|0.4691122999071863|-0.2828633443286633|-1.5090585031735124|-1.1356323710171934
1|1.2121120250208506|-0.17321464905330858|0.11920871129693428|-1.0442359662799567
2|-0.8618489633477999|-2.1045692188948086|-0.4949292740687813|1.071803807037338
3|0.7215551622443669|-0.7067711336300845|-1.0395749851146963|0.27185988554282986
4|-0.42497232978883753|0.567020349793672|0.27623201927771873|-1.0874006912859915
5|-0.6736897080883706|0.1136484096888855|-1.4784265524372235|0.5249876671147047
6|0.4047052186802365|0.5770459859204836|-1.7150020161146375|-1.0392684835147725
7|-0.3706468582364464|-1.1578922506419993|-1.344311812731667|0.8448851414248841
8|1.0757697837155533|-0.10904997528022223|1.6435630703622064|-1.4693879595399115
9|0.35702056413309086|-0.6746001037299882|-1.776903716971867|-0.9689138124473498
```

```
In [185]: table = pd.read_csv('tmp.csv', sep='|')
```

```
In [186]: table
```

```
Out[186]:
```

```

 Unnamed: 0 0 1 2 3
0 0 0.469112 -0.282863 -1.509059 -1.135632
1 1 1.212112 -0.173215 0.119209 -1.044236
2 2 -0.861849 -2.104569 -0.494929 1.071804
3 3 0.721555 -0.706771 -1.039575 0.271860
4 4 -0.424972 0.567020 0.276232 -1.087401
5 5 -0.673690 0.113648 -1.478427 0.524988
6 6 0.404705 0.577046 -1.715002 -1.039268
7 7 -0.370647 -1.157892 -1.344312 0.844885
8 8 1.075770 -0.109050 1.643563 -1.469388
9 9 0.357021 -0.674600 -1.776904 -0.968914
```

By specifying a `chunksize` to `read_csv`, the return value will be an iterable object of type `TextFileReader`:

```
In [187]: reader = pd.read_csv('tmp.csv', sep='|', chunksize=4)
```

```
In [188]: reader
```

```
Out[188]: <pandas.io.parsers.TextFileReader at 0x7f65f17cf7f0>
```

```
In [189]: for chunk in reader:
```

```
.....: print(chunk)
```

```
.....:
```

```

 Unnamed: 0 0 1 2 3
0 0 0.469112 -0.282863 -1.509059 -1.135632
1 1 1.212112 -0.173215 0.119209 -1.044236
2 2 -0.861849 -2.104569 -0.494929 1.071804
3 3 0.721555 -0.706771 -1.039575 0.271860
 Unnamed: 0 0 1 2 3
4 4 -0.424972 0.567020 0.276232 -1.087401
5 5 -0.673690 0.113648 -1.478427 0.524988
6 6 0.404705 0.577046 -1.715002 -1.039268
7 7 -0.370647 -1.157892 -1.344312 0.844885
 Unnamed: 0 0 1 2 3
8 8 1.075770 -0.10905 1.643563 -1.469388
9 9 0.357021 -0.67460 -1.776904 -0.968914
```

Specifying `iterator=True` will also return the `TextFileReader` object:

```
In [190]: reader = pd.read_csv('tmp.csv', sep='|', iterator=True)
```

```
In [191]: reader.get_chunk(5)
```

```
Out[191]:
```

	Unnamed: 0	0	1	2	3
0	0	0.469112	-0.282863	-1.509059	-1.135632
1	1	1.212112	-0.173215	0.119209	-1.044236
2	2	-0.861849	-2.104569	-0.494929	1.071804
3	3	0.721555	-0.706771	-1.039575	0.271860
4	4	-0.424972	0.567020	0.276232	-1.087401

## Specifying the parser engine

Under the hood pandas uses a fast and efficient parser implemented in C as well as a Python implementation which is currently more feature-complete. Where possible pandas uses the C parser (specified as `engine='c'`), but may fall back to Python if C-unsupported options are specified. Currently, C-unsupported options include:

- `sep` other than a single character (e.g. regex separators)
- `skipfooter`
- `sep=None` with `delim_whitespace=False`

Specifying any of the above options will produce a `ParserWarning` unless the python engine is selected explicitly using `engine='python'`.

## Reading remote files

You can pass in a URL to a CSV file:

```
df = pd.read_csv('https://download.bls.gov/pub/time.series/cu/cu.item',
 sep='\t')
```

S3 URLs are handled as well but require installing the [S3Fs](#) library:

```
df = pd.read_csv('s3://pandas-test/tips.csv')
```

If your S3 bucket requires credentials you will need to set them as environment variables or in the `~/.aws/credentials` config file, refer to the [S3Fs documentation on credentials](#).

## Writing out data

### Writing to CSV format

The `Series` and `DataFrame` objects have an instance method `to_csv` which allows storing the contents of the object as a comma-separated-values file. The function takes a number of arguments. Only the first is required.

- `path_or_buf`: A string path to the file to write or a file object. If a file object it must be opened with `newline=''`
- `sep`: Field delimiter for the output file (default `","`)
- `na_rep`: A string representation of a missing value (default `" "`)
- `float_format`: Format string for floating point numbers
- `columns`: Columns to write (default `None`)
- `header`: Whether to write out the column names (default `True`)
- `index`: whether to write row (index) names (default `True`)
- `index_label`: Column label(s) for index column(s) if desired. If `None` (default), and `header` and `index` are `True`, then the index names are used. (A sequence should be given if the `DataFrame` uses `MultiIndex`).
- `mode`: Python write mode, default `'w'`
- `encoding`: a string representing the encoding to use if the contents are non-ASCII, for Python versions prior to 3
- `line_terminator`: Character sequence denoting line end (default `os.linesep`)
- `quoting`: Set quoting rules as in `csv` module (default `csv.QUOTE_MINIMAL`). Note that if you have set a `float_format` then floats are converted to strings and `csv.QUOTE_NONNUMERIC` will treat them as non-numeric
- `quotechar`: Character used to quote fields (default `"'"`)
- `doublequote`: Control quoting of `quotechar` in fields (default `True`)
- `escapechar`: Character used to escape `sep` and `quotechar` when appropriate (default `None`)
- `chunksize`: Number of rows to write at a time
- `date_format`: Format string for datetime objects

### Writing a formatted string

The `DataFrame` object has an instance method `to_string` which allows control over the string representation of the object. All arguments are optional:

- `buf` default `None`, for example a `StringIO` object
- `columns` default `None`, which columns to write
- `col_space` default `None`, minimum width of each column.
- `na_rep` default `NaN`, representation of NA value
- `formatters` default `None`, a dictionary (by column) of functions each of which takes a single argument and returns a formatted string

- `float_format` default None, a function which takes a single (float) argument and returns a formatted string; to be applied to floats in the `DataFrame` .
- `sparsify` default True, set to False for a `DataFrame` with a hierarchical index to print every MultiIndex key at each row.
- `index_names` default True, will print the names of the indices
- `index` default True, will print the index (ie, row labels)
- `header` default True, will print the column labels
- `justify` default `left` , will print column headers left- or right-justified

The `Series` object also has a `to_string` method, but with only the `buf` , `na_rep` , `float_format` arguments. There is also a `length` argument which, if set to `True` , will additionally output the length of the Series.

## JSON

读取和写入 JSON 格式的文本和字符串。

### Writing JSON

一个 `Series` 或 `DataFrame` 能转化成有效的 JSON 字符串。使用 `to_json` 同可选的参数：

- `path_or_buf` : 写入输出的路径名或缓存可以是 `None` , 在这种情况下会返回一个JSON字符串。
- `orient` :

`Series` :

- 默认是 `index` ;
- 允许的值可以是{ `split` , `records` , `index` }。

`DataFrame` :

- 默认是 `columns` ;
- 允许的值可以是{ `split` , `records` , `index` , `columns` , `values` , `table` }。

JSON字符串的格式：

<code>split</code>	dict like {index -> [index], columns -> [columns], data -> [values]}
<code>records</code>	list like [{column -> value}, ..., {column -> value}]
<code>index</code>	dict like {index -> {column -> value}}
<code>columns</code>	dict like {column -> {index -> value}}
<code>values</code>	just the values array

- `date_format` : 字符串，日期类型的转换，'epoch'是时间戳，'iso'是 ISO8601。
- `double_precision` : 当要编码的是浮点数值时使用的小数位数，默认是 10。

- `force_ascii` : 强制编码字符串为 ASCII , 默认是 True。
- `date_unit` : 时间单位被编码来管理时间戳 和 ISO8601 精度。 's', 'ms', 'us' 或 'ns' 中的一个分别为 秒, 毫秒, 微秒, 纳秒。默认是 'ms'。
- `default_handler` : 如果一个对象没有转换成一个恰当的 JSON 格式, 处理程序就会被调用。采用单个参数, 即要转换的对象, 并返回一个序列化的对象。
- `lines` : 如果面向 `records` , 就将每行写入记录为 json。

注意: `NaN` 'S, `NaT` 'S 和 `None` 将会被转换为 `null` , 并且 `datetime` 将会基于 `date_format` 和 `date_unit` 两个参数转换。

```
In [192]: dfj = pd.DataFrame(np.random.randn(5, 2), columns=list('AB'))

In [193]: json = dfj.to_json()

In [194]: json
Out[194]: '{"A":
{"0":-1.2945235903,"1":0.2766617129,"2":-0.0139597524,"3":-0.0061535699,"4":0.8957173022},"B":
{"0":0.4137381054,
"1":-0.472034511,"2":-0.3625429925,"3":-0.923060654,"4":0.8052440254}}'
```

## 面向选项 (Orient options)

要生成 JSON 文件/字符串, 这儿有很多可选的格式。如下面的 `DataFrame` 和 `Series` :

```
In [195]: dfjo = pd.DataFrame(dict(A=range(1, 4), B=range(4, 7), C=range(7, 10)),
.....: columns=list('ABC'), index=list('xyz'))
.....:

In [196]: dfjo
Out[196]:
 A B C
x 1 4 7
y 2 5 8
z 3 6 9

In [197]: sjo = pd.Series(dict(x=15, y=16, z=17), name='D')

In [198]: sjo
Out[198]:
x 15
y 16
z 17
Name: D, dtype: int64
```

**面向列** 序列化数据 (默认是 `DataFrame`) 来作为嵌套的 JSON 对象, 且列标签充当主索引:



```
In [199]: dfjo.to_json(orient="columns")
Out[199]: '{"A":{"x":1,"y":2,"z":3},"B":{"x":4,"y":5,"z":6},"C":{"x":7,"y":8,"z":9}}'

Not available for Series (不适用于 Series)
```

**面向索引**（默认是 `Series`）与面向列类似，但是索引标签是主键：

```
In [200]: dfjo.to_json(orient="index")
Out[200]: '{"x":{"A":1,"B":4,"C":7},"y":{"A":2,"B":5,"C":8},"z":{"A":3,"B":6,"C":9}}'

In [201]: sjo.to_json(orient="index")
Out[201]: '{"x":15,"y":16,"z":17}'
```

**面向记录** 序列化数据为一列JSON数组 -> 值的记录，索引标签不包括在内。这个在传递 `DataFrame` 数据到绘图库的时候很有用，例如JavaScript库 `d3.js`：

```
In [202]: dfjo.to_json(orient="records")
Out[202]: '[{"A":1,"B":4,"C":7},{"A":2,"B":5,"C":8},{"A":3,"B":6,"C":9}]'

In [203]: sjo.to_json(orient="records")
Out[203]: '[15,16,17]'
```

**面向值** 是一个概要的选项，它只序列化为嵌套的JSON数组值，列和索引标签不包括在内：

```
In [204]: dfjo.to_json(orient="values")
Out[204]: '[[1,4,7],[2,5,8],[3,6,9]]'

Not available for Series
```

**面向切分** 序列化成一个JSON对象，它包括单项的值、索引和列。`Series` 的命名也包括：

```
In [205]: dfjo.to_json(orient="split")
Out[205]: '{"columns":["A","B","C"],"index":["x","y","z"],"data":[[1,4,7],[2,5,8],[3,6,9]]}'

In [206]: sjo.to_json(orient="split")
Out[206]: '{"name":"D","index":["x","y","z"],"data":[15,16,17]}'
```

**面向表格** 序列化为JSON的 **表格模式（Table Schema）**，允许保存为元数据，包括但不限于 `dtypes` 和索引名称。

⋮ tip 注意

任何面向选项编码为一个JSON对象在转为序列化期间将不会保留索引和列标签的顺序。如果你想要保留标签的顺序，就使用 `split` 选项，因为它使用有序的容器。

⋮

## 日期处理 (Date handling)

用ISO日期格式来写入：

```
In [207]: dfd = pd.DataFrame(np.random.randn(5, 2), columns=list('AB'))

In [208]: dfd['date'] = pd.Timestamp('20130101')

In [209]: dfd = dfd.sort_index(1, ascending=False)

In [210]: json = dfd.to_json(date_format='iso')

In [211]: json
Out[211]: '{"date":
{"0": "2013-01-01T00:00:00.000Z", "1": "2013-01-01T00:00:00.000Z", "2": "2013-01-01T00:00:00.000Z", "
3": "2013-01-01T00:00:00.000Z", "4": "2013-01-01T00:00:00.000Z"}, "B":
{"0": 2.5656459463, "1": 1.3403088498, "2": -0.2261692849, "3": 0.8138502857, "4": -0.8273169356}, "A":
{"0": -1.2064117817, "1": 1.4312559863, "2": -1.1702987971, "3": 0.4108345112, "4": 0.1320031703}}'
```

以ISO日期格式的微妙单位写入：

```
In [212]: json = dfd.to_json(date_format='iso', date_unit='us')

In [213]: json
Out[213]: '{"date":
{"0": "2013-01-01T00:00:00.000000Z", "1": "2013-01-01T00:00:00.000000Z", "2": "2013-01-01T00:00:00.0
00000Z", "3": "2013-01-01T00:00:00.000000Z", "4": "2013-01-01T00:00:00.000000Z"}, "B":
{"0": 2.5656459463, "1": 1.3403088498, "2": -0.2261692849, "3": 0.8138502857, "4": -0.8273169356}, "A":
{"0": -1.2064117817, "1": 1.4312559863, "2": -1.1702987971, "3": 0.4108345112, "4": 0.1320031703}}'
```

时间戳的时间，以秒为单位：

```
In [214]: json = dfd.to_json(date_format='epoch', date_unit='s')

In [215]: json
Out[215]: '{"date": {"0": 1356998400, "1": 1356998400, "2": 1356998400, "3": 1356998400, "4": 1356998400}, "B":
{"0": 2.5656459463, "1": 1.3403088498, "2": -0.2261692849, "3": 0.8138502857, "4": -0.8273169356}, "A":
{"0": -1.2064117817, "1": 1.4312559863, "2": -1.1702987971, "3": 0.4108345112, "4": 0.1320031703}}'
```

写入文件，以日期索引和日期列格式：

```

In [216]: dfj2 = dfj.copy()

In [217]: dfj2['date'] = pd.Timestamp('20130101')

In [218]: dfj2['ints'] = list(range(5))

In [219]: dfj2['bools'] = True

In [220]: dfj2.index = pd.date_range('20130101', periods=5)

In [221]: dfj2.to_json('test.json')

In [222]: with open('test.json') as fh:
.....: print(fh.read())
.....:
{"A":{"1356998400000":-1.2945235903,"1357084800000":0.2766617129,"1357171200000":-0.0139597524,"1357257600000":-0.00615
{"1356998400000":0.4137381054,"1357084800000":-0.472034511,"1357171200000":-0.3625429925,"1357257600000":-0.9
{"1356998400000":1356998400000,"1357084800000":1356998400000,"1357171200000":1356998400000,"1357257600000":13
{"1356998400000":0,"1357084800000":1,"1357171200000":2,"1357257600000":3,"1357344000000":4},"bools":
{"1356998400000":true,"1357084800000":true,"1357171200000":true,"1357257600000":true,"1357344000000":true}}

```

## 回退行为 (Fallback behavior)

如果JSON序列不能直接处理容器的内容，他将会以下面的方式发生回退：

- 如果dtype是不被支持的（例如：`np.complex`），则将为每个值调用 `default_handler`（如果提供），否则引发异常。
- 如果对象不受支持，它将尝试以下操作：
  - 检查一下是否对象被定义为 `toDict` 的方法并调用它。`toDict` 的方法将返回一个 dict，它将会是序列化的JSON格式。
  - 如果提供了 `default_handler`，则调用它。
  - 通过遍历其内容将对象转换为 dict。但是，这通常会出现 `OverflowError` 而失败或抛出意外的结果。

通常，对于不被支持的对象或dtypes，处理的最佳方法是提供 `default_handler`。例如：

```

>>> DataFrame([1.0, 2.0, complex(1.0, 2.0)]).to_json() # raises
RuntimeError: Unhandled numpy dtype 15

```

可以通过指定一个简单 `default_handler` 来处理：

```

In [223]: pd.DataFrame([1.0, 2.0, complex(1.0, 2.0)]).to_json(default_handler=str)
Out[223]: '{"0":{"0":"(1+0j)","1":"(2+0j)","2":"(1+2j)"}}'

```

## JSON的读取 (Reading JSON)

把JSON字符串读取到pandas对象里会采用很多参数。如果 `typ` 没有提供或者为 `None`，解析器将尝试解析 `DataFrame`。要强制地进行 `Series` 解析，请传递参数如 `typ = series`。

- `filepath_or_buffer`：一个有效的JSON字符串或文件句柄/StringIO(在内存中读写字符串)。字符串可以是一个URL。有效的URL格式包括http，ftp，S3和文件。对于文件型的URL, 最好有个主机地址。例如一个本地文件可以是 `file://localhost/path/to/table.json` 这样的格式。
- `typ`：要恢复的对象类型（series或者frame），默认“frame”。
- `orient`：

Series:

- 默认是 `index`。
- 允许值为{ `split`, `records`, `index` }。

DataFrame:

- 默认是 `columns`。
- 允许值是{ `split`, `records`, `index`, `columns`, `values`, `table` }。

JSON字符串的格式：

split	dict like {index -> [index], columns -> [columns], data -> [values]}
records	list like [{column -> value}, ..., {column -> value}]
index	dict like {index -> {column -> value}}
columns	dict like {column -> {index -> value}}
values	just the values array
table	adhering to the JSON <a href="#">Table Schema</a>

- `dtype`：如果为True，推断dtypes，如果列为dtype的字典，则使用那些；如果为 `False`，则根本不推断dtypes，默认为True，仅适用于数据。
- `convert_axes`：布尔值，尝试将轴转换为正确的dtypes，默认为 `True`。
- `convert_dates`：一列列表要解析为日期; 如果为 `True`，则尝试解析类似日期的列，默认为 `True`。
- `keep_default_dates`：布尔值，默认为 `True`。如果解析日期，则解析默认的类型日期的列。
- `numpy`：直接解码为NumPy数组。默认为 `False`；虽然标签可能是非数字的，但仅支持数字数据。另请注意，如果 `numpy = True`，则每个术语的JSON顺序 **必须** 相同。
- `precise_float`：布尔值，默认为 `False`。当解码字符串为双值时，设置为能使用更高精度（`strtod`）函数。默认（`False`）快速使用但不精确的内置功能。

- `date_unit` : 字符串, 用于检测转换日期的时间戳单位。默认无。默认情况下, 将检测时间戳精度, 如果不需要, 则传递's', 'ms', 'us'或'ns'中的一个, 以强制时间戳精度分别为秒, 毫秒, 微秒或纳秒。
- `lines` : 读取文件每行作为一个JSON对象。
- `encoding` : 用于解码py3字节的编码。
- `chunksize` : 当与 `lines = True` 结合使用时, 返回一个Json读取器 (JSONReader) , 每次迭代读取 `chunksize` 行。

如果JSON不能解析, 解析器将抛出 `ValueError` / `TypeError` / `AssertionError` 中的一个错误。

如果在编码为JSON时使用非默认的 `orient` 方法, 请确保在此处传递相同的选项以便解码产生合理的结果, 请参阅 [Orient Options](#)以获取概述。

## 数据转换 (Data conversion)

`convert_axes = True` , `dtype = True` 和 `convert_dates = True` 的默认值将尝试解析轴, 并将所有数据解析为适当的类型, 包括日期。如果需要覆盖特定的dtypes, 请将字典传递给 `dtype` 。如果您需要在轴中保留类似字符串的数字 (例如 “1” , “2” ) , 则只应将 `convert_axes` 设置为 `False` 。

::: tip 注意

如果 `convert_dates = True` 并且数据和/或列标签显示为 “类似日期 ('date-like') ” , 则可以将大的整数值转换为日期。确切的标准取决于指定的 `date_unit` 。 'date-like'表示列标签符合以下标准之一:

- 结尾以 `'_at'`
- 结尾以 `'_time'`
- 开头以 `'timestamp'`
- 它是 `'modified'`
- 它是 `'date'`

:::

::: danger 警告

在读取JSON数据时, 自动强制转换为dtypes有一些不同寻常的地方:

- 索引可以按序列化的不同顺序重建, 也就是说, 返回的顺序不能保证与序列化之前的顺序相同
- 如果可以安全地, 那么一系列浮动 ( `float` ) 数据将被转换为一列整数( `integer` ), 例如 一系列 1
- 布尔列将在重建时转换为整数 ( `integer` )

因此, 有时你会有那样的时刻可能想通过 `dtype` 关键字参数指定特定的dtypes。

:::

读取JSON字符串：

```
In [224]: pd.read_json(json)
Out[224]:
```

	date	B	A
0	2013-01-01	2.565646	-1.206412
1	2013-01-01	1.340309	1.431256
2	2013-01-01	-0.226169	-1.170299
3	2013-01-01	0.813850	0.410835
4	2013-01-01	-0.827317	0.132003

读取文件：

```
In [225]: pd.read_json('test.json')
Out[225]:
```

	A	B	date	ints	bools
2013-01-01	-1.294524	0.413738	2013-01-01	0	True
2013-01-02	0.276662	-0.472035	2013-01-01	1	True
2013-01-03	-0.013960	-0.362543	2013-01-01	2	True
2013-01-04	-0.006154	-0.923061	2013-01-01	3	True
2013-01-05	0.895717	0.805244	2013-01-01	4	True

不要转换任何数据（但仍然转换轴和日期）：

```
In [226]: pd.read_json('test.json', dtype=object).dtypes
Out[226]:
```

A	object
B	object
date	object
ints	object
bools	object
dtype:	object

指定转换的dtypes：

```
In [227]: pd.read_json('test.json', dtype={'A': 'float32', 'bools': 'int8'}).dtypes
Out[227]:
```

A	float32
B	float64
date	datetime64[ns]
ints	int64
bools	int8
dtype:	object

保留字符串索引：

```

In [228]: si = pd.DataFrame(np.zeros((4, 4)), columns=list(range(4)),
.....: index=[str(i) for i in range(4)])
.....:

In [229]: si
Out[229]:
 0 1 2 3
0 0.0 0.0 0.0 0.0
1 0.0 0.0 0.0 0.0
2 0.0 0.0 0.0 0.0
3 0.0 0.0 0.0 0.0

In [230]: si.index
Out[230]: Index(['0', '1', '2', '3'], dtype='object')

In [231]: si.columns
Out[231]: Int64Index([0, 1, 2, 3], dtype='int64')

In [232]: json = si.to_json()

In [233]: sij = pd.read_json(json, convert_axes=False)

In [234]: sij
Out[234]:
 0 1 2 3
0 0 0 0 0
1 0 0 0 0
2 0 0 0 0
3 0 0 0 0

In [235]: sij.index
Out[235]: Index(['0', '1', '2', '3'], dtype='object')

In [236]: sij.columns
Out[236]: Index(['0', '1', '2', '3'], dtype='object')

```

以纳秒为单位的日期需要以纳秒为单位读回：

```
In [237]: json = dfj2.to_json(date_unit='ns')

Try to parse timestamps as milliseconds -> Won't Work
In [238]: dfju = pd.read_json(json, date_unit='ms')

In [239]: dfju
Out[239]:
```

	A	B	date	ints	bools
1356998400000000000	-1.294524	0.413738	1356998400000000000	0	True
1357084800000000000	0.276662	-0.472035	1356998400000000000	1	True
1357171200000000000	-0.013960	-0.362543	1356998400000000000	2	True
1357257600000000000	-0.006154	-0.923061	1356998400000000000	3	True
1357344000000000000	0.895717	0.805244	1356998400000000000	4	True

```
Let pandas detect the correct precision
In [240]: dfju = pd.read_json(json)

In [241]: dfju
Out[241]:
```

	A	B	date	ints	bools
2013-01-01	-1.294524	0.413738	2013-01-01	0	True
2013-01-02	0.276662	-0.472035	2013-01-01	1	True
2013-01-03	-0.013960	-0.362543	2013-01-01	2	True
2013-01-04	-0.006154	-0.923061	2013-01-01	3	True
2013-01-05	0.895717	0.805244	2013-01-01	4	True

```
Or specify that all timestamps are in nanoseconds
In [242]: dfju = pd.read_json(json, date_unit='ns')

In [243]: dfju
Out[243]:
```

	A	B	date	ints	bools
2013-01-01	-1.294524	0.413738	2013-01-01	0	True
2013-01-02	0.276662	-0.472035	2013-01-01	1	True
2013-01-03	-0.013960	-0.362543	2013-01-01	2	True
2013-01-04	-0.006154	-0.923061	2013-01-01	3	True
2013-01-05	0.895717	0.805244	2013-01-01	4	True

## Numpy 参数

::: tip 注意

这仅支持数值数据。索引和列标签可以是数字的，例如 字符串，日期等。

:::

如果将 `numpy = True` 传递给 `read_json`，则会在反序列化期间尝试找到适当的dtype，然后直接解码到NumPy数组，从而绕过对中间Python对象的需求。

如果要反序列化大量数值数据，这可以提供加速：



```
In [244]: randfloats = np.random.uniform(-100, 1000, 10000)

In [245]: randfloats.shape = (1000, 10)

In [246]: dffloats = pd.DataFrame(randfloats, columns=list('ABCDEFGHJIJ'))

In [247]: jsonfloats = dffloats.to_json()
```

```
In [248]: %timeit pd.read_json(jsonfloats)
12.4 ms +- 116 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

```
In [249]: %timeit pd.read_json(jsonfloats, numpy=True)
9.56 ms +- 82.8 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

对于较小的数据集，加速不太明显：

```
In [250]: jsonfloats = dffloats.head(100).to_json()
```

```
In [251]: %timeit pd.read_json(jsonfloats)
8.05 ms +- 120 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

```
In [252]: %timeit pd.read_json(jsonfloats, numpy=True)
7 ms +- 162 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

⋮ danger 警告

直接NumPy解码会产生许多假设并可能导致失败，或如果这些假设不满足，则产生意外地输出：

- 数据是数值。
- 数据是统一的。从解码的第一个值中找到dtype。可能会引发 `ValueError` 错误，或者如果这个条件不满足可能产生不正确的输出。
- 标签是有序的。标签仅从第一个容器读取，假设每个后续行/列已按相同顺序编码。如果使用 `to_json` 编码数据，则应该满足这一要求，但如果JSON来自其他来源，则可能不是这种情况。

⋮

## 标准化 (Normalization)

pandas提供了一个实用程序函数来获取一个字典或字典列表，并将这个半结构化数据规范化为一个平面表。

```
In [253]: from pandas.io.json import json_normalize

In [254]: data = [{'id': 1, 'name': {'first': 'Coleen', 'last': 'Volk'}},
.....: {'name': {'given': 'Mose', 'family': 'Regner'}},
.....: {'id': 2, 'name': 'Faye Raker'}]
.....:
```

```
In [255]: json_normalize(data)
Out[255]:
```

	id	name.first	name.last	name.given	name.family	name
0	1.0	Coleen	Volk	NaN	NaN	NaN
1	NaN	NaN	NaN	Mose	Regner	NaN
2	2.0	NaN	NaN	NaN	NaN	Faye Raker

```
In [256]: data = [{'state': 'Florida',
.....: 'shortname': 'FL',
.....: 'info': {'governor': 'Rick Scott'},
.....: 'counties': [{'name': 'Dade', 'population': 12345},
.....: {'name': 'Broward', 'population': 40000},
.....: {'name': 'Palm Beach', 'population': 60000}]},
.....: {'state': 'Ohio',
.....: 'shortname': 'OH',
.....: 'info': {'governor': 'John Kasich'},
.....: 'counties': [{'name': 'Summit', 'population': 1234},
.....: {'name': 'Cuyahoga', 'population': 1337}]}]
.....:
```

```
In [257]: json_normalize(data, 'counties', ['state', 'shortname', ['info', 'governor']])
Out[257]:
```

	name	population	state	shortname	info.governor
0	Dade	12345	Florida	FL	Rick Scott
1	Broward	40000	Florida	FL	Rick Scott
2	Palm Beach	60000	Florida	FL	Rick Scott
3	Summit	1234	Ohio	OH	John Kasich
4	Cuyahoga	1337	Ohio	OH	John Kasich

`max_level` 参数提供了对结束规范化的级别的更多控制。当 `max_level = 1` 时，以下代码段会标准化，直到提供了字典的第一个嵌套级别为止。

```
In [258]: data = [{'CreatedBy': {'Name': 'User001'},
.....: 'Lookup': {'TextField': 'Some text',
.....: 'UserField': {'Id': 'ID001',
.....: 'Name': 'Name001'}}},
.....: 'Image': {'a': 'b'}}]
.....:
```

```
In [259]: json_normalize(data, max_level=1)
Out[259]:
```

	CreatedBy.Name	Lookup.TextField	Lookup.UserField	Image.a
0	User001	Some text	{'Id': 'ID001', 'Name': 'Name001'}	b

## json的行分割（Line delimited json）

New in version 0.19.0.

pandas能够读取和写入行分隔的json文件通常是在用Hadoop或Spark进行数据处理的管道中。

New in version 0.21.0.

对于行分隔的json文件，pandas也可以返回一个迭代器，它能一次读取 `chunksize` 行。这对于大型文件或从数据流中读取非常有用。

```
In [260]: jsonl = '''
.....: {"a": 1, "b": 2}
.....: {"a": 3, "b": 4}
.....: '''
.....:

In [261]: df = pd.read_json(jsonl, lines=True)

In [262]: df
Out[262]:
 a b
0 1 2
1 3 4

In [263]: df.to_json(orient='records', lines=True)
Out[263]: '{"a":1,"b":2}\n{"a":3,"b":4}'

reader is an iterator that returns `chunksize` lines each iteration
In [264]: reader = pd.read_json(StringIO(jsonl), lines=True, chunksize=1)

In [265]: reader
Out[265]: <pandas.io.json._json.JsonReader at 0x7f65f15bac18>

In [266]: for chunk in reader:
.....: print(chunk)
.....:
Empty DataFrame
Columns: []
Index: []
 a b
0 1 2
 a b
1 3 4
```

## 表模式（Table schema）

New in version 0.20.0.

表模式（[Table schema](#)）是用于将表格数据集描述为JSON对象的一种规范。JSON包含有关字段名称，类型和其他属性的信息。你可以使用面向 `table` 来构建一个JSON字符串包含两个字段，`schema` 和 `data`。

```

In [267]: df = pd.DataFrame({'A': [1, 2, 3],
.....: 'B': ['a', 'b', 'c'],
.....: 'C': pd.date_range('2016-01-01', freq='d', periods=3)},
.....: index=pd.Index(range(3), name='idx'))

In [268]: df
Out[268]:
 A B C
idx
0 1 a 2016-01-01
1 2 b 2016-01-02
2 3 c 2016-01-03

In [269]: df.to_json(orient='table', date_format="iso")
Out[269]: '{"schema": {"fields": [{"name": "idx", "type": "integer"}, {"name": "A", "type": "integer"}, {"name": "B", "type": "string"}, {"name": "C", "type": "datetime"}], "primaryKey": ["idx"], "pandas_version": "0.20.0"}, "data": [{"idx": 0, "A": 1, "B": "a", "C": "2016-01-01T00:00:00.000Z"}, {"idx": 1, "A": 2, "B": "b", "C": "2016-01-02T00:00:00.000Z"}, {"idx": 2, "A": 3, "B": "c", "C": "2016-01-03T00:00:00.000Z"}]}'

```

schema 字段包含 fields 主键，它本身包含一个列名称到列对的列表，包括 Index 或 MultiIndex（请参阅下面的类型列表）。如果（多）索引是唯一的，则 schema 字段也包含一个 primaryKey 字段。

第二个字段 data 包含用面向 records 来序列化数据。索引是包括的，并且任何日期时间都是ISO 8601格式，正如表模式规范所要求的那样。

表模式规范中描述了所有支持的全部类型列表。此表显示了pandas类型的映射：

Pandas type	Table Schema type
int64	integer
float64	number
bool	boolean
datetime64[ns]	datetime
timedelta64[ns]	duration
categorical	any
object	str

关于生成的表模式的一些注意事项：

- schema 对象包含 pandas\_version 的字段。它包含模式的pandas方言版本，并将随每个修订增加。
- 序列化时，所有日期都转换为UTC。甚至是时区的初始值，也被视为UTC，偏移量为0。

```
In [270]: from pandas.io.json import build_table_schema

In [271]: s = pd.Series(pd.date_range('2016', periods=4))

In [272]: build_table_schema(s)
Out[272]:
{'fields': [{'name': 'index', 'type': 'integer'},
 {'name': 'values', 'type': 'datetime'}],
 'primaryKey': ['index'],
 'pandas_version': '0.20.0'}
```

- 具有时区的日期时间（在序列化之前），包括具有时区名称的附加字段 `tz`（例如：'US / Central'）。

```
In [273]: s_tz = pd.Series(pd.date_range('2016', periods=12,
.....: tz='US/Central'))
.....:

In [274]: build_table_schema(s_tz)
Out[274]:
{'fields': [{'name': 'index', 'type': 'integer'},
 {'name': 'values', 'type': 'datetime', 'tz': 'US/Central'}],
 'primaryKey': ['index'],
 'pandas_version': '0.20.0'}
```

- 时间段在序列化之前是转换为时间戳的，因此具有转换为UTC的相同方式。此外，时间段将包含具有时间段频率的附加字段 `freq`，例如：'A-DEC'。

```
In [275]: s_per = pd.Series(1, index=pd.period_range('2016', freq='A-DEC',
.....: periods=4))
.....:

In [276]: build_table_schema(s_per)
Out[276]:
{'fields': [{'name': 'index', 'type': 'datetime', 'freq': 'A-DEC'},
 {'name': 'values', 'type': 'integer'}],
 'primaryKey': ['index'],
 'pandas_version': '0.20.0'}
```

- 分类使用 `any` 类型和 `enum` 约束来列出可能值的集合。此外，还包括一个 `ordered` 字段：

```
In [277]: s_cat = pd.Series(pd.Categorical(['a', 'b', 'a']))
```

```
In [278]: build_table_schema(s_cat)
```

```
Out[278]:
```

```
{'fields': [{'name': 'index', 'type': 'integer'},
 {'name': 'values',
 'type': 'any',
 'constraints': {'enum': ['a', 'b']},
 'ordered': False}],
 'primaryKey': ['index'],
 'pandas_version': '0.20.0'}
```

- 如果索引是唯一的，则包含 `primaryKey` 字段，它包含了标签数组：

```
In [279]: s_dupe = pd.Series([1, 2], index=[1, 1])
```

```
In [280]: build_table_schema(s_dupe)
```

```
Out[280]:
```

```
{'fields': [{'name': 'index', 'type': 'integer'},
 {'name': 'values', 'type': 'integer'}],
 'pandas_version': '0.20.0'}
```

- `primaryKey` 的形式与多索引相同，但在这种情况下，`primaryKey` 是一个数组：

```
In [281]: s_multi = pd.Series(1, index=pd.MultiIndex.from_product([('a', 'b'),
 : (0, 1)]))
 :
```

```
In [282]: build_table_schema(s_multi)
```

```
Out[282]:
```

```
{'fields': [{'name': 'level_0', 'type': 'string'},
 {'name': 'level_1', 'type': 'integer'},
 {'name': 'values', 'type': 'integer'}],
 'primaryKey': FrozenList(['level_0', 'level_1']),
 'pandas_version': '0.20.0'}
```

- 默认命名大致遵循以下规则：
  - 对于 `series`，使用 `object.name`。如果没有，那么名称就是 `values`
  - 对于 `DataFrames`，使用列名称的字符串化版本
  - 对于 `Index`（不是 `MultiIndex`），使用 `index.name`，如果为 `None`，则使用回退 `index`。
  - 对于 `MultiIndex`，使用 `mi.names`。如果任何级别没有名称，则使用 `level_`。

New in version 0.23.0.

`read_json` 也接受 `orient='table'` 作为参数。这允许以可循环移动的方式保存诸如 `dtypes` 和索引名称之类的元数据。

```
In [283]: df = pd.DataFrame({'foo': [1, 2, 3, 4],
.....: 'bar': ['a', 'b', 'c', 'd'],
.....: 'baz': pd.date_range('2018-01-01', freq='d', periods=4),
.....: 'qux': pd.Categorical(['a', 'b', 'c', 'c'])
.....: }, index=pd.Index(range(4), name='idx'))
.....:
```

```
In [284]: df
```

```
Out[284]:
```

	foo	bar	baz	qux
idx				
0	1	a	2018-01-01	a
1	2	b	2018-01-02	b
2	3	c	2018-01-03	c
3	4	d	2018-01-04	c

```
In [285]: df.dtypes
```

```
Out[285]:
```

foo	int64
bar	object
baz	datetime64[ns]
qux	category

dtype: object

```
In [286]: df.to_json('test.json', orient='table')
```

```
In [287]: new_df = pd.read_json('test.json', orient='table')
```

```
In [288]: new_df
```

```
Out[288]:
```

	foo	bar	baz	qux
idx				
0	1	a	2018-01-01	a
1	2	b	2018-01-02	b
2	3	c	2018-01-03	c
3	4	d	2018-01-04	c

```
In [289]: new_df.dtypes
```

```
Out[289]:
```

foo	int64
bar	object
baz	datetime64[ns]
qux	category

dtype: object

请注意，作为 [Index](#) 名称的文字字符串 'index' 是不能循环移动的，也不能在 [MultiIndex](#) 中用以 'level\_' 开头的任何名称。这些默认情况下在 [DataFrame.to\\_json\(\)](#) 中用于指示缺失值和后续读取无法区分的目的。

```
In [290]: df.index.name = 'index'

In [291]: df.to_json('test.json', orient='table')

In [292]: new_df = pd.read_json('test.json', orient='table')

In [293]: print(new_df.index.name)
None
```

## HTML

---

### 读取HTML的内容

::: danger 警告:

我们**强烈建议**你阅读 [HTML Table Parsing gotchas](#)里面相关的围绕BeautifulSoup4/html5lib/lxml解析器部分的问题。

:::

顶级的 `read_html()` 函数能接受HTML字符串/文件/URL格式，并且能解析HTML 表格为pandas DataFrames 的列表，让我们看看下面的几个例子。

::: tip 注意:

`read_html` 返回的是一个 DataFrame 对象的 list，即便在HTML页面里只包含单个表格。

:::

读取一个没有选项的URL:



```
In [294]: url = 'https://www.fdic.gov/bank/individual/failed/banklist.html'
```

```
In [295]: dfs = pd.read_html(url)
```

```
In [296]: dfs
```

```
Out[296]:
```

```
[
 Institution Closing Date Bank Name City ST CERT Acquiring
0 Institution Closing Date Bank Name City ST CERT Acquiring
 0 Bank, N. A. May 31, 2019 The Enloe State Bank Cooper TX 10716 Legend
 1 Washington Federal Bank for Savings Chicago IL 30570 Royal
 Savings Bank December 15, 2017 February 1, 2019
 2 The Farmers and Merchants State Bank of Argonia Argonia KS 17719
 Conway Bank October 13, 2017 February 21, 2018
 3 Fayette County Bank Saint Elmo IL
 1802 United Fidelity Bank, fsb May 26, 2017 January 29, 2019
 4 Guaranty Bank, (d/b/a BestBank in Georgia & Mi... Milwaukee WI 30003 First-Citizens Bank &
 Trust Company May 5, 2017 March 22, 2018

551 Superior Bank, FSB Hinsdale IL 32646 Superior
 Federal, FSB July 27, 2001 August 19, 2014
552 Malta National Bank Malta OH 6629 North
 Valley Bank May 3, 2001 November 18, 2002
553 First Alliance Bank & Trust Co. Manchester NH 34264 Southern New Hampshire
 Bank & Trust February 2, 2001 February 18, 2003
554 National State Bank of Metropolis Metropolis IL
 3815 Banterra Bank of Marion December 14, 2000 March 17, 2005
555 Bank of Honolulu Honolulu HI 21029 Bank of
 the Orient October 13, 2000 March 17, 2005

[556 rows x 7 columns]]
```

∴ tip 注意：

上面的URL数据修改了每个周一以至于上面的数据结果跟下面的数据结果可能有轻微的不同。

∴

从上面的URL读取文件内容并且传递它给 read\_html 作为一个字符串：

```

In [297]: with open(file_path, 'r') as f:
.....: dfs = pd.read_html(f.read())
.....:

In [298]: dfs
Out[298]:
[
 Institution Closing Date Bank Name City ST CERT Acquiring
0 Banks of Wisconsin d/b/a Bank of Kenosha Kenosha WI 35386 North Shore Bank,
 FSB May 31, 2013 May 31, 2013
1 Bank May 14, 2013 May 20, 2013 Central Arizona Bank Scottsdale AZ 34527 Western State
2 Bank May 10, 2013 May 21, 2013 Sunrise Bank Valdosta GA 58185 Synovus
3 N.A. May 10, 2013 May 14, 2013 Pisgah Community Bank Asheville NC 58701 Capital Bank,
4 Bank April 26, 2013 May 16, 2013 Douglas County Bank Douglasville GA 21649 Hamilton State

500 FSB July 27, 2001 June 5, 2012 Superior Bank, FSB Hinsdale IL 32646 Superior Federal,
501 Bank May 3, 2001 November 18, 2002 Malta National Bank Malta OH 6629 North Valley
502 First Alliance Bank & Trust Co. Manchester NH 34264 Southern New Hampshire Bank &
 Trust February 2, 2001 February 18, 2003
503 National State Bank of Metropolis Metropolis IL 3815 Banterra Bank of
 Marion December 14, 2000 March 17, 2005
504 Bank of Honolulu Honolulu HI 21029 Bank of the
 Orient October 13, 2000 March 17, 2005

[505 rows x 7 columns]]

```

甚至如果你想，你还可以传递一个 StringIO 的实例：

```
In [299]: with open(file_path, 'r') as f:
.....: sio = StringIO(f.read())
.....:

In [300]: dfs = pd.read_html(sio)

In [301]: dfs
Out[301]:
```

	Institution	Closing Date	Bank Name	Updated Date	City	ST	CERT	Acquiring
0	Banks of Wisconsin d/b/a Bank of Kenosha FSB	May 31, 2013	Bank of Kenosha	May 31, 2013	Kenosha	WI	35386	North Shore Bank,
1	Bank	May 14, 2013	Central Arizona Bank	May 20, 2013	Scottsdale	AZ	34527	Western State
2	Bank	May 10, 2013	Sunrise Bank	May 21, 2013	Valdosta	GA	58185	Synovus
3	N.A.	May 10, 2013	Pisgah Community Bank	May 14, 2013	Asheville	NC	58701	Capital Bank,
4	Bank	April 26, 2013	Douglas County Bank	May 16, 2013	Douglasville	GA	21649	Hamilton State
..			...		...	..	...	...
500	FSB	July 27, 2001	Superior Bank, FSB	June 5, 2012	Hinsdale	IL	32646	Superior Federal,
501	Bank	May 3, 2001	Malta National Bank	November 18, 2002	Malta	OH	6629	North Valley
502	First Alliance Bank & Trust Co. Trust	February 2, 2001	First Alliance Bank & Trust Co.	February 18, 2003	Manchester	NH	34264	Southern New Hampshire Bank &
503	National State Bank of Metropolis Marion	December 14, 2000	National State Bank of Metropolis	March 17, 2005	Metropolis	IL	3815	Banterra Bank of
504	Orient	October 13, 2000	Bank of Honolulu	March 17, 2005	Honolulu	HI	21029	Bank of the

```
[505 rows x 9 columns]]
```

∴ tip 注意：

以下的例子在IPython的程序中不会运行，因为太多的网络接入函数减缓了文档的创建。如果你的程序报错或者例子不运行，请立即向 [pandas GitHub issues page](#) 上报。

∴

读取一个URL并匹配表格里面所包含的具体文本内容：

```
match = 'Metcalf Bank'
df_list = pd.read_html(url, match=match)
```

指定一个标题行（通过默认的<th>或者<td>定位并伴随一个<thead>被用来作为列的索引，如果是多行含有<thead>，则多索引就会被创建）；如果已经指定，标题行则从数据减去已解析的标题元素中获取（<th>元素）。

```
dfs = pd.read_html(url, header=0)
```

指定一个索引列：

```
dfs = pd.read_html(url, index_col=0)
```

指定跳过行的数量：

```
dfs = pd.read_html(url, skiprows=0)
```

指定使用列表来跳过行的数量（xrange (只在Python 2 中)也有效）：

```
dfs = pd.read_html(url, skiprows=range(2))
```

指定一个HTML属性：

```
dfs1 = pd.read_html(url, attrs={'id': 'table'})
dfs2 = pd.read_html(url, attrs={'class': 'sortable'})
print(np.array_equal(dfs1[0], dfs2[0])) # Should be True
```

指定值将会被转换为NaN(非数值)：

```
dfs = pd.read_html(url, na_values=['No Acquirer'])
```

New in version 0.19.

指定是否保持默认的NaN值的设置：

```
dfs = pd.read_html(url, keep_default_na=False)
```

New in version 0.19.

指定列的转换器。这对于有前置零的数字文本数据很有用。默认情况下，数值列会转换成数值类型且前置零会丢失。为了避免这种情况，我们能转换这些列为字符串。

```
url_mcc = 'https://en.wikipedia.org/wiki/Mobile_country_code'
dfs = pd.read_html(url_mcc, match='Telekom Albania', header=0,
 converters={'MNC': str})
```

New in version 0.19.

把上面的一些结合使用：

```
dfs = pd.read_html(url, match='Metcalfe Bank', index_col=0)
```

读取pandas to\_html 输出（同时一些精确的浮点会失去）：

```
df = pd.DataFrame(np.random.randn(2, 2))
s = df.to_html(float_format='{0:.40g}'.format)
dfin = pd.read_html(s, index_col=0)
```

如果 `lxml` 后端是你提供的唯一解析器，那么它将在解析失败时报错。如果你能提供的解析器只有一个就选字符串，但是传递一个字符串列表会是很好的训练，例如，这个函数期望是一个字符串序列。你可以这样使用：

```
dfs = pd.read_html(url, 'Metcalf Bank', index_col=0, flavor=['lxml'])
```

或者你可以传递 `flavor='lxml'` 而不要列表：

```
dfs = pd.read_html(url, 'Metcalf Bank', index_col=0, flavor='lxml')
```

然而，如果你已经安装了 `bs4` 和 `html5lib` 并且传递 `None` 或 `['lxml', 'bs4']`，那么极大可能会解析成功。注意一旦解析成功了，函数将会返回。

```
dfs = pd.read_html(url, 'Metcalf Bank', index_col=0, flavor=['lxml', 'bs4'])
```

## 写入HTML文件

`DataFrame` 对象具有实例的方法 `to_html`，它能渲染 `DataFrame` 的内容为HTML表格。这个函数的参数同上面的 `to_string` 方法的一样。

⋮ tip 注意：

为了简洁起见，这儿显示的不是所有的 `DataFrame.to_html` 可选项。所有的选项设置见 `to_html()`。

⋮

```
In [302]: df = pd.DataFrame(np.random.randn(2, 2))
```

```
In [303]: df
```

```
Out[303]:
```

```
 0 1
0 -0.184744 0.496971
1 -0.856240 1.857977
```

```
In [304]: print(df.to_html()) # raw html
```

```
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>0</th>
 <th>1</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>-0.184744</td>
 <td>0.496971</td>
 </tr>
 <tr>
 <th>1</th>
 <td>-0.856240</td>
 <td>1.857977</td>
 </tr>
 </tbody>
</table>
```

HTML:

-	0	1
0	-0.184744	0.496971
1	-0.856240	1.857977

```
In [302]: df = pd.DataFrame(np.random.randn(2, 2))
```

```
In [303]: df
```

```
Out[303]:
```

```
 0 1
0 -0.184744 0.496971
1 -0.856240 1.857977
```

```
In [304]: print(df.to_html()) # raw html
```

```
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>0</th>
 <th>1</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>-0.184744</td>
 <td>0.496971</td>
 </tr>
 <tr>
 <th>1</th>
 <td>-0.856240</td>
 <td>1.857977</td>
 </tr>
 </tbody>
</table>
```

`columns` 参数将限制列的显示:

```
In [305]: print(df.to_html(columns=[0]))
```

```
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>0</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>-0.184744</td>
 </tr>
 <tr>
 <th>1</th>
 <td>-0.856240</td>
 </tr>
 </tbody>
</table>
```

HTML:

-	0
0	-0.184744
1	-0.856240

`float_format` 采用可调用的 Python 来控制浮点值的精确度：

```
In [306]: print(df.to_html(float_format='{0:.10f}'.format))
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>0</th>
 <th>1</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>-0.1847438576</td>
 <td>0.4969711327</td>
 </tr>
 <tr>
 <th>1</th>
 <td>-0.8562396763</td>
 <td>1.8579766508</td>
 </tr>
 </tbody>
</table>
```

HTML：

-	0	1
0	-0.1847438576	0.4969711327
1	-0.8562396763	1.8579766508

默认情况下，`bold_rows` 可以加粗行标签，但是你可以关掉它：



In [307]: `print(df.to_html(bold_rows=False))`

```
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>0</th>
 <th>1</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <td>0</td>
 <td>-0.184744</td>
 <td>0.496971</td>
 </tr>
 <tr>
 <td>1</td>
 <td>-0.856240</td>
 <td>1.857977</td>
 </tr>
 </tbody>
</table>
```

-	0	1
0	-0.184744	0.496971
1	-0.856240	1.857977

`classes` 参数提供了能生成HTML表的CSS类的功能。注意这些类是已添加到现有的 'dataframe' 类中的。

In [308]: `print(df.to_html(classes=['awesome_table_class', 'even_more_awesome_class']))`

```
<table border="1" class="dataframe awesome_table_class even_more_awesome_class">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>0</th>
 <th>1</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>-0.184744</td>
 <td>0.496971</td>
 </tr>
 <tr>
 <th>1</th>
 <td>-0.856240</td>
 <td>1.857977</td>
 </tr>
 </tbody>
</table>
```

`render_links` 参数提供了向包含URL的单元格添加超链接的功能。

New in version 0.24.

```
In [309]: url_df = pd.DataFrame({
.....: 'name': ['Python', 'Pandas'],
.....: 'url': ['https://www.python.org/', 'http://pandas.pydata.org']}))
.....:

In [310]: print(url_df.to_html(render_links=True))
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>name</th>
 <th>url</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>Python</td>
 <td>https://www.python.org</td>
 </tr>
 <tr>
 <th>1</th>
 <td>Pandas</td>
 <td>http://pandas.pydata.org</td>
 </tr>
 </tbody>
</table>
```

HTML:

-	name	url
0	Python	<a href="https://www.python.org/">https://www.python.org/</a>
1	Pandas	<a href="http://pandas.pydata.org">http://pandas.pydata.org</a>

最后，`escape` 参数允许你控制是否对生成的 HTML 字符 “<”，“>” 和 “&” 进行转义（默认是 `True`）。因此，获取不转义的 HTML 字符就设置为 `escape=False`。

```
In [311]: df = pd.DataFrame({'a': list('&<>'), 'b': np.random.randn(3)})
```

转义的:

```
In [312]: print(df.to_html())
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>a</th>
 <th>b</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td><</td>
 <td>-0.474063</td>
 </tr>
 <tr>
 <th>1</th>
 <td><</td>
 <td>-0.230305</td>
 </tr>
 <tr>
 <th>2</th>
 <td>></td>
 <td>-0.400654</td>
 </tr>
 </tbody>
</table>
```

-	a	b
0	&	-0.474063
1	<	-0.230305
2	>	-0.400654

不转义的：

```
In [313]: print(df.to_html(escape=False))
<table border="1" class="dataframe">
 <thead>
 <tr style="text-align: right;">
 <th></th>
 <th>a</th>
 <th>b</th>
 </tr>
 </thead>
 <tbody>
 <tr>
 <th>0</th>
 <td>&</td>
 <td>-0.474063</td>
 </tr>
 <tr>
 <th>1</th>
 <td><</td>
 <td>-0.230305</td>
 </tr>
 <tr>
 <th>2</th>
 <td>></td>
 <td>-0.400654</td>
 </tr>
 </tbody>
</table>
```

-	a	b
0	&	-0.474063
1	<	-0.230305
2	>	-0.400654

::: tip 注意：

一些浏览器在渲染上面的两个HTML表格的时候可能看不出区别。

:::

## HTML表格解析陷阱

在使用顶级的pandas io函数 `read_html` 来解析HTML表格的时候，围绕这些库，存在一些版本的问题。

**lxml**问题:

- 优点：
  - **lxml** 是非常快的。
  - **lxml**要求Cython正确安装。

- 缺点：
  - `lxml` 不能保证它的解析结果除非使用[严格有效地标记](#)。
  - 鉴于上述情况，我们选择允许用户使用 `lxml` 作为后端，但是如果 `lxml` 解析失败，**这个后端将使用`html5lib`**。
  - 因此，强烈推荐你安装\*\*`BeautifulSoup4`和`html5lib`这两个库。这样即使`lxml`解析失败，你仍然能够得到一个有效的结果（前提是其他所有内容都有效）。

#### BeautifulSoup4使用lxml作为后端的问题：

- 以上问题仍然会存在因为\*\*`BeautifulSoup4`

#### BeautifulSoup4使用html5lib作为后端的问题：

- 优点：
  - `html5lib`比`lxml`宽容得多，所以会以一种更理智的方式处理现实生活中的标记，而不是仅仅，比如在未通知你的情况下删除元素。
  - `html5lib`能自动从无效标记中生成有效的 HTML5 标记。这在解析HTML表格的时候是相当重要的，因为它保证了它是有效的文件。然而这不意味着它是“正确的”，因为修复标记的过程没有一个定义。
  - `html5lib`是纯净的Python，除了它自己的安装步骤没有其他的步骤。
- 缺点：
  - 使用`html5lib`最大的缺点就是太慢了。但是考虑到网络上许多表格并不足以如解析算法运行时的那么重要，它更可能像是正在通过网络上的URL读取原始文本过程中的瓶颈。例如当IO（输入-输出）时，对于非常大的表，事实可能并非如此。

## Excel 文件

`read_excel()`方法使用Python的 `xlrd` 模块来读取Excel 2003 ( `.xls` )版的文件，而Excel 2007+ ( `.xlsx` )版本的是用 `xlrd` 或者 `openpyxl` 模块来读取的。`to_excel()`方法则是用来把 `DataFrame` 数据存储为Excel格式。一般来说，它的语法同使用`csv`数据是类似的，更多高级的用法可以参考[cookbook](#)。

### 读取 Excel 文件

在大多数基本的使用案例中，`read_excel` 会读取Excel文件通过一个路径，并且 `sheet_name` 会表明需要解析哪一张表格。

```
Returns a DataFrame
pd.read_excel('path_to_file.xls', sheet_name='Sheet1')
```

## ExcelFile 类

为了方便地读取同一个文件的多张表格，`ExcelFile` 类可用来打包文件并传递给 `read_excel`。因为仅需读取一次内存，所以这种方式读取一个文件的多张表格会有性能上的优势。

```
xlsx = pd.ExcelFile('path_to_file.xls')
df = pd.read_excel(xlsx, 'Sheet1')
```

`ExcelFile` 类也能用来作为上下文管理器。

```
with pd.ExcelFile('path_to_file.xls') as xls:
 df1 = pd.read_excel(xls, 'Sheet1')
 df2 = pd.read_excel(xls, 'Sheet2')
```

`sheet_names` 属性能将文件中的所有表格名字生成一组列表。

`ExcelFile` 一个主要的用法就是用来解析多张表格的不同参数：

```
data = {}
For when Sheet1's format differs from Sheet2
with pd.ExcelFile('path_to_file.xls') as xls:
 data['Sheet1'] = pd.read_excel(xls, 'Sheet1', index_col=None,
 na_values=['NA'])
 data['Sheet2'] = pd.read_excel(xls, 'Sheet2', index_col=1)
```

注意如果所有的表格解析同一个参数，那么这组表格名的列表能轻易地传递给 `read_excel` 且不会有性能上的损失。

```
using the ExcelFile class
data = {}
with pd.ExcelFile('path_to_file.xls') as xls:
 data['Sheet1'] = pd.read_excel(xls, 'Sheet1', index_col=None,
 na_values=['NA'])
 data['Sheet2'] = pd.read_excel(xls, 'Sheet2', index_col=None,
 na_values=['NA'])

equivalent using the read_excel function
data = pd.read_excel('path_to_file.xls', ['Sheet1', 'Sheet2'],
 index_col=None, na_values=['NA'])
```

`ExcelFile` 也能同 `xlrd.book.Book` 对象作为一个参数被调用。这种方法让用户可以控制Excel文件被如何读取。例如，表格可以根据需求加载通过调用 `xlrd.open_workbook()` 伴随 `on_demand=True`。

```
import xlrd
xlrd_book = xlrd.open_workbook('path_to_file.xls', on_demand=True)
with pd.ExcelFile(xlrd_book) as xls:
 df1 = pd.read_excel(xls, 'Sheet1')
 df2 = pd.read_excel(xls, 'Sheet2')
```

## 指定表格

::: tip 注意

第二个参数是 `sheet_name`，不要同 `ExcelFile.sheet_names` 搞混淆。

:::

::: tip 注意

`ExcelFile`'s 的属性 `sheet_names` 提供的是多张表格所生成的列表。

:::

- `sheet_name` 参数允许指定单张表格或多张表格被读取。
- `sheet_name` 的默认值是0，这表明读取的是第一张表格。
- 在工作簿里面，使用字符串指向特定的表格名称。
- 使用整数指向表格的索引，索引遵守Python的约定是从0开始的。
- 无论是使用一组字符串还是整数的列表，返回的都是指定表格的字典。
- 使用 `None` 值则会返回所有可用表格的一组字典。

```
Returns a DataFrame
pd.read_excel('path_to_file.xls', 'Sheet1', index_col=None, na_values=['NA'])
```

使用表格索引：

```
Returns a DataFrame
pd.read_excel('path_to_file.xls', 0, index_col=None, na_values=['NA'])
```

使用所有默认值：

```
Returns a DataFrame
pd.read_excel('path_to_file.xls')
```

使用None获取所有表格：

```
Returns a dictionary of DataFrames
pd.read_excel('path_to_file.xls', sheet_name=None)
```

使用列表获取多张表格：

```
Returns the 1st and 4th sheet, as a dictionary of DataFrames.
pd.read_excel('path_to_file.xls', sheet_name=['Sheet1', 3])
```

`read_excel` 能读取不止一张表格，通过 `sheet_name` 能设置为读取表格名称的列表，表格位置的列表，还能设置为 `None` 来读取所有表格。多张表格能通过表格索引或表格名称分别使用整数或字符串来指定读取。

## MultiIndex读取

`read_excel` 能用 `MultiIndex` 读取多个索引，通过 `index_col` 方法来传递列的列表和 `header` 将行的列表传递给 `MultiIndex` 的列。无论是 `index` 还是 `columns`，如果已经具有序列化的层级名称，则可以通过指定组成层级的行/列来读取它们。

例如，用 `MultiIndex` 读取没有名称的索引：

```
In [314]: df = pd.DataFrame({'a': [1, 2, 3, 4], 'b': [5, 6, 7, 8]},
.....: index=pd.MultiIndex.from_product(['a', 'b'], ['c', 'd']))
.....:

In [315]: df.to_excel('path_to_file.xlsx')

In [316]: df = pd.read_excel('path_to_file.xlsx', index_col=[0, 1])

In [317]: df
Out[317]:
 a b
a c 1 5
 d 2 6
b c 3 7
 d 4 8
```

如果索引具有层级名称，它们将使用相同的参数进行解析：

```
In [318]: df.index = df.index.set_names(['lvl1', 'lvl2'])

In [319]: df.to_excel('path_to_file.xlsx')

In [320]: df = pd.read_excel('path_to_file.xlsx', index_col=[0, 1])

In [321]: df
Out[321]:
 a b
lvl1 lvl2
a c 1 5
 d 2 6
b c 3 7
 d 4 8
```

如果源文件具有 `MultiIndex` 索引和多列，那么可以使用 `index_col` 和 `header` 指定列表的每个值。



```
In [322]: df.columns = pd.MultiIndex.from_product([['a'], ['b', 'd']],
.....: names=['c1', 'c2'])
.....:

In [323]: df.to_excel('path_to_file.xlsx')

In [324]: df = pd.read_excel('path_to_file.xlsx', index_col=[0, 1], header=[0, 1])

In [325]: df
Out[325]:
c1 a
c2 b d
lvl1 lvl2
a c 1 5
 d 2 6
b c 3 7
 d 4 8
```

## 解析特定的列

常常会有这样的情况，当用户想要插入几列数据到Excel表格里面作为临时计算，但是你不想要读取这些列的时候，`read_excel` 提供的 `usecols` 方法就派上用场了，它让你可以解析指定的列。

Deprecated since version 0.24.0.

不推荐 `usecols` 方法使用单个整数值，请在 `usecols` 中使用包括从0开始的整数列表。

如果 `usecols` 是一个整数，那么它将被认为是暗示解析最后一列。

```
pd.read_excel('path_to_file.xls', 'Sheet1', usecols=2)
```

你也可以将逗号分隔的一组Excel列和范围指定为字符串：

```
pd.read_excel('path_to_file.xls', 'Sheet1', usecols='A,C:E')
```

如果 `usecols` 是一组整数列，那么将认为是解析的文件列索引。

```
pd.read_excel('path_to_file.xls', 'Sheet1', usecols=[0, 2, 3])
```

元素的顺序是可以忽略的，因此 `usecols=[0, 1]` 是等价于 `[1, 0]` 的。

New in version 0.24.

如果 `usecols` 是字符串列表，那么可以认为每个字符串对应的就是表格的每一个列名，列名是由 `name` 中的用户提供或从文档标题行推断出来。这些字符串定义了那些列将要被解析：

```
pd.read_excel('path_to_file.xls', 'Sheet1', usecols=['foo', 'bar'])
```

元素的顺序同样被忽略，因此 `usecols=['baz', 'joe']` 等同于 `['joe', 'baz']`。

New in version 0.24.

如果 `usecols` 是可调用的，那么该调用函数将会根据列名来调用，也会返回根据可调函数为 `True` 的列名。

```
pd.read_excel('path_to_file.xls', 'Sheet1', usecols=lambda x: x.isalpha())
```

## 解析日期

当读取excel文件的时候，像日期时间的值通常会自动转换为恰当的dtype(数据类型)。但是如果你有一列字符串看起来很像日期（实际上并不是excel里面的日期格式），那么你就能使用 `parse_dates` 方法来解析这些字符串为日期：

```
pd.read_excel('path_to_file.xls', 'Sheet1', parse_dates=['date_strings'])
```

## 单元格转换

Excel里面的单元格内容是可以通过 `converters` 方法来进行转换的。例如，把一列转换为布尔值：

```
pd.read_excel('path_to_file.xls', 'Sheet1', converters={'MyBools': bool})
```

这个方法可以处理缺失值并且能对缺失的数据进行如期的转换。由于转换是在单元格之间发生而不是整列，因此不能保证dtype为数组。例如一列含有缺失值的整数是不能转换为具有整数dtype的数组，因为NaN严格的被认为是浮点数。你能够手动地标记缺失数据为恢复整数dtype：

```
def cfun(x):
 return int(x) if x else -1

pd.read_excel('path_to_file.xls', 'Sheet1', converters={'MyInts': cfun})
```

## 数据类型规范

New in version 0.20.

作为另一个种转换器，使用dtype能指定整列地类型，它能让字典映射列名为数据类型。使用 `str` 或 `object` 来转译不能判断类型的数据：

```
pd.read_excel('path_to_file.xls', dtype={'MyInts': 'int64', 'MyText': str})
```

## 写入Excel文件

### 写入Excel文件到磁盘

你可以使用 `to_excel` 方法把 `DataFrame` 对象写入到Excel文件的一张表格中。它的参数大部分同前面 `to_csv` 提到的相同，第一个参数是excel文件的名称，而可选的第二个参数是 `DataFrame` 应该写入的表格名称，例如：

```
df.to_excel('path_to_file.xlsx', sheet_name='Sheet1')
```

文件以 `.xls` 结尾的将用 `xlwt` 写入，而那些以 `.xlsx` 结尾的则使用 `xlsxwriter`（如果可用的话）或 `openpyxl` 来写入。

`DataFrame` 将尝试以模拟REPL（“读取-求值-输出”循环的简写）输出的方式写入。`index_label` 将代替第一行放置到第二行，你也能放置它到第一行通过在 `to_excel()` 里设置 `merge_cells` 选项为 `False`：

```
df.to_excel('path_to_file.xlsx', index_label='label', merge_cells=False)
```

为了把 `DataFrames` 数据分开写入Excel文件的不同表格中，可以使用 `ExcelWriter` 方法。

```
with pd.ExcelWriter('path_to_file.xlsx') as writer:
 df1.to_excel(writer, sheet_name='Sheet1')
 df2.to_excel(writer, sheet_name='Sheet2')
```

⋮ tip 注意

为了从 `read_excel` 内部获取更多点的性能，Excel存储所有数值型数据为浮点数。但这会产生意外的情况当读取数据的时候，如果没有损失信息的话（`1.0 --> 1`），pandas默认地转换整数为浮点数。你可以通过 `convert_float=False` 禁止这种行为，这可能会在性能上有轻微的优化。

⋮

### 写入Excel文件到内存

Pandas支持写入Excel文件到类缓存区对象如 `StringIO` 或 `BytesIO`，使用 `ExcelWriter` 方法。

```
Safe import for either Python 2.x or 3.x
try:
 from io import BytesIO
except ImportError:
 from cStringIO import StringIO as BytesIO

bio = BytesIO()

By setting the 'engine' in the ExcelWriter constructor.
writer = pd.ExcelWriter(bio, engine='xlsxwriter')
df.to_excel(writer, sheet_name='Sheet1')

Save the workbook
writer.save()

Seek to the beginning and read to copy the workbook to a variable in memory
bio.seek(0)
workbook = bio.read()
```

∴ tip 注意

虽然 `engine` 是可选方法，但是推荐使用。设置 `engine` 决定了工作簿生成的版本。设置 `engine='xlrd'` 将生成 Excel 2003 版的工作簿（xls）。而使用 `'openpyxl'` 或 `'xlsxwriter'` 将生成 Excel 2007 版的工作簿（xlsx）。如果省略，将直接生成 Excel 2007 版的。

∴

## Excel写入引擎

Pandas选择Excel写入有两种方式：

1. 使用 `engine` 参数
2. 文件名的扩展（通过默认的配置方式指定）

默认的，pandas使用 `XlsxWriter` 为 `.xlsx`，使用 `openpyxl` 为 `.xlsm`，并且使用 `xlwt` 为 `.xls` 文件。如果你安装了多个引擎，你可以通过 [setting the config options](#) `io.excel.xlsx.writer` 和 `io.excel.xls.writer` 方法设置默认引擎。如果 `XlsxWriter` 不可用，pandas将回退使用 `openpyxl` 为 `xlsx` 文件。

为了指定你想要使用的写入方式，你可以设置引擎的主要参数为 `to_excel` 和 `ExcelWriter`。内置引擎是：

- `openpyxl`：要求2.4或者更高的版本。
- `xlsxwriter`
- `xlwt`

```
By setting the 'engine' in the DataFrame 'to_excel()' methods.
df.to_excel('path_to_file.xlsx', sheet_name='Sheet1', engine='xlsxwriter')

By setting the 'engine' in the ExcelWriter constructor.
writer = pd.ExcelWriter('path_to_file.xlsx', engine='xlsxwriter')

Or via pandas configuration.
from pandas import options # noqa: E402
options.io.excel.xlsx.writer = 'xlsxwriter'

df.to_excel('path_to_file.xlsx', sheet_name='Sheet1')
```

## 样式

通过pandas产生的Excel工作表的样式可以使用 DataFrame 的 `to_excel` 方法的以下参数进行修改。

- `float_format`：格式化字符串用于浮点数（默认是 `None`）。
- `freeze_panes`：两个整数的元组，表示要固化的最底行和最右列。这些参数中的每个都是以1为底，因此(1, 1)将固化第一行和第一列（默认是 `None`）。

使用 `XlsxWriter` 引擎提供的多种方法来修改用 `to_excel` 方法创建的Excel工作表的样式。你能在 `XlsxWriter` 文档里面找到绝佳的例子：[https://xlsxwriter.readthedocs.io/working\\_with\\_pandas.html](https://xlsxwriter.readthedocs.io/working_with_pandas.html)

## OpenDocument 电子表格

New in version 0.25.

`read_excel`方法也能使用 `odfpy` 模块来读取OpenDocument电子表格。读取OpenDocument电子表格的语法和方法同使用 `engine='odf'` 来操作Excel files的方法类似。

```
Returns a DataFrame
pd.read_excel('path_to_file.ods', engine='odf')
```

⋮ tip 注意

目前pandas仅支持读取OpenDocument电子表格，写入是不行的。

⋮

## 剪贴板

使用 `read_clipboard()` 方法是一种便捷的获取数据的方式，通过把剪贴的内容暂存，然后传递给 `read_csv` 方法。例如，你可以复制以下文本来剪贴（在许多操作系统上是CTRL-C）：

```
A B C
x 1 4 p
y 2 5 q
z 3 6 r
```

接着直接使用 `DataFrame` 来导入数据：

```
>>> clipdf = pd.read_clipboard()
>>> clipdf
 A B C
x 1 4 p
y 2 5 q
z 3 6 r
```

`to_clipboard` 方法可以把 `DataFrame` 内容写入到剪贴板。使用下面的方法你可以粘贴剪贴板的内容到其他应用（在许多系统中用的是CTRL-V）。这里我们解释一下如何使用 `DataFrame` 把内容写入到剪贴板并读回。

```
>>> df = pd.DataFrame({'A': [1, 2, 3],
... 'B': [4, 5, 6],
... 'C': ['p', 'q', 'r']},
... index=['x', 'y', 'z'])
>>> df
 A B C
x 1 4 p
y 2 5 q
z 3 6 r
>>> df.to_clipboard()
>>> pd.read_clipboard()
 A B C
x 1 4 p
y 2 5 q
z 3 6 r
```

我们可以看到返回了同样的内容，那就是我们早先写入剪贴板的内容。

⋮ tip 注意

要使用上面的这些方法，你可能需要在Linux上面安装(带有PyQt5, PyQt4 or qtpy)的xclip或者xsel。

⋮

## 序列化(Pickling)

所有的pandas对象都具有 `to_pickle` 方法，该方法使用Python的 `cPickle` 模块以序列化格式存储数据结构到磁盘上。

```
In [326]: df
Out[326]:
c1 a
c2 b d
lvl1 lvl2
a c 1 5
 d 2 6
b c 3 7
 d 4 8

In [327]: df.to_pickle('foo.pkl')
```

在 pandas 中命名的 `read_pickle` 函数能够从文件中加载任意序列化的pandas对象（或者任何其他序列化对象）：

```
In [328]: pd.read_pickle('foo.pkl')
Out[328]:
c1 a
c2 b d
lvl1 lvl2
a c 1 5
 d 2 6
b c 3 7
 d 4 8
```

::: danger 警告

加载来自不信任来源的序列化数据是不安全的。参见：<https://docs.python.org/3/library/pickle.html>

:::

::: danger 警告

`read_pickle()`仅在pandas的0.20.3版本及以下版本兼容。

:::

## 压缩序列化文件

New in version 0.20.0.

`read_pickle()`，`DataFrame.to_pickle()`和`Series.to_pickle()`能够读取和写入压缩的序列化文件。读取和写入所支持的压缩文件类型有 `gzip`，`bz2`，`xz`。 `zip` 文件格式仅支持读取，并且必须仅包含一个要读取的数据文件。

压缩类型可以是显式参数，也可以从文件扩展名中推断出来。如果文件名是以 `'.gz'`，`'.bz2'`，`'.zip'`，或者 `'.xz'` 结尾的，那么可以推断出应分别使用 `gzip`，`bz2`，`zip`，或 `xz` 压缩类型。

```
In [329]: df = pd.DataFrame({
.....: 'A': np.random.randn(1000),
.....: 'B': 'foo',
.....: 'C': pd.date_range('20130101', periods=1000, freq='s')})
.....:
```

```
In [330]: df
Out[330]:
```

	A	B	C
0	-0.288267	foo	2013-01-01 00:00:00
1	-0.084905	foo	2013-01-01 00:00:01
2	0.004772	foo	2013-01-01 00:00:02
3	1.382989	foo	2013-01-01 00:00:03
4	0.343635	foo	2013-01-01 00:00:04
..	...	...	...
995	-0.220893	foo	2013-01-01 00:16:35
996	0.492996	foo	2013-01-01 00:16:36
997	-0.461625	foo	2013-01-01 00:16:37
998	1.361779	foo	2013-01-01 00:16:38
999	-1.197988	foo	2013-01-01 00:16:39

[1000 rows x 3 columns]

使用显式压缩类型：

```
In [331]: df.to_pickle("data.pkl.compress", compression="gzip")

In [332]: rt = pd.read_pickle("data.pkl.compress", compression="gzip")

In [333]: rt
Out[333]:
```

	A	B	C
0	-0.288267	foo	2013-01-01 00:00:00
1	-0.084905	foo	2013-01-01 00:00:01
2	0.004772	foo	2013-01-01 00:00:02
3	1.382989	foo	2013-01-01 00:00:03
4	0.343635	foo	2013-01-01 00:00:04
..	...	...	...
995	-0.220893	foo	2013-01-01 00:16:35
996	0.492996	foo	2013-01-01 00:16:36
997	-0.461625	foo	2013-01-01 00:16:37
998	1.361779	foo	2013-01-01 00:16:38
999	-1.197988	foo	2013-01-01 00:16:39

[1000 rows x 3 columns]

从扩展名推断压缩类型：



```
In [334]: df.to_pickle("data.pkl.xz", compression="infer")

In [335]: rt = pd.read_pickle("data.pkl.xz", compression="infer")
```

```
In [336]: rt
```

```
Out[336]:
```

	A	B	C
0	-0.288267	foo	2013-01-01 00:00:00
1	-0.084905	foo	2013-01-01 00:00:01
2	0.004772	foo	2013-01-01 00:00:02
3	1.382989	foo	2013-01-01 00:00:03
4	0.343635	foo	2013-01-01 00:00:04
...	...	...	...
995	-0.220893	foo	2013-01-01 00:16:35
996	0.492996	foo	2013-01-01 00:16:36
997	-0.461625	foo	2013-01-01 00:16:37
998	1.361779	foo	2013-01-01 00:16:38
999	-1.197988	foo	2013-01-01 00:16:39

```
[1000 rows x 3 columns]
```

默认是使用“推断”：

```

In [337]: df.to_pickle("data.pkl.gz")

In [338]: rt = pd.read_pickle("data.pkl.gz")

In [339]: rt
Out[339]:
 A B C
0 -0.288267 foo 2013-01-01 00:00:00
1 -0.084905 foo 2013-01-01 00:00:01
2 0.004772 foo 2013-01-01 00:00:02
3 1.382989 foo 2013-01-01 00:00:03
4 0.343635 foo 2013-01-01 00:00:04
...
995 -0.220893 foo 2013-01-01 00:16:35
996 0.492996 foo 2013-01-01 00:16:36
997 -0.461625 foo 2013-01-01 00:16:37
998 1.361779 foo 2013-01-01 00:16:38
999 -1.197988 foo 2013-01-01 00:16:39

[1000 rows x 3 columns]

In [340]: df["A"].to_pickle("s1.pkl.bz2")

In [341]: rt = pd.read_pickle("s1.pkl.bz2")

In [342]: rt
Out[342]:
0 -0.288267
1 -0.084905
2 0.004772
3 1.382989
4 0.343635
...
995 -0.220893
996 0.492996
997 -0.461625
998 1.361779
999 -1.197988
Name: A, Length: 1000, dtype: float64

```

## msgpack（一种二进制格式）

pandas支持 msgpack 格式的对象序列化。他是一种轻量级可移植的二进制格式，同二进制的JSON类似，具有高效的空间利用率以及不错的写入（序列化）和读取（反序列化）性能。

::: danger 警告

从0.25版本开始，不推荐使用msgpack格式，并且之后的版本也将删除它。推荐使用pyarrow对pandas对象进行在线的转换。

:::

::: danger 警告

`read_msgpack()` 仅在pandas的0.20.3版本及以下版本兼容。

:::

```
In [343]: df = pd.DataFrame(np.random.rand(5, 2), columns=list('AB'))

In [344]: df.to_msgpack('foo.msg')

In [345]: pd.read_msgpack('foo.msg')
Out[345]:
```

	A	B
0	0.275432	0.293583
1	0.842639	0.165381
2	0.608925	0.778891
3	0.136543	0.029703
4	0.318083	0.604870

```
In [346]: s = pd.Series(np.random.rand(5), index=pd.date_range('20130101', periods=5))
```

你可以传递一组对象列表并得到反序列化的结果。

```
In [347]: pd.to_msgpack('foo.msg', df, 'foo', np.array([1, 2, 3]), s)

In [348]: pd.read_msgpack('foo.msg')
Out[348]:
```

	A	B
0	0.275432	0.293583
1	0.842639	0.165381
2	0.608925	0.778891
3	0.136543	0.029703
4	0.318083	0.604870
2013-01-01	0.330824	
2013-01-02	0.790825	
2013-01-03	0.308468	
2013-01-04	0.092397	
2013-01-05	0.703091	

Freq: D, dtype: float64

你能传递 `iterator=True` 参数来迭代解压后的结果：

```
In [349]: for o in pd.read_msgpack('foo.msg', iterator=True):
.....: print(o)
.....:
 A B
0 0.275432 0.293583
1 0.842639 0.165381
2 0.608925 0.778891
3 0.136543 0.029703
4 0.318083 0.604870
foo
[1 2 3]
2013-01-01 0.330824
2013-01-02 0.790825
2013-01-03 0.308468
2013-01-04 0.092397
2013-01-05 0.703091
Freq: D, dtype: float64
```

你也能传递 `append=True` 参数，给现有的包添加写入：

```
In [350]: df.to_msgpack('foo.msg', append=True)

In [351]: pd.read_msgpack('foo.msg')
Out[351]:
[A B
0 0.275432 0.293583
1 0.842639 0.165381
2 0.608925 0.778891
3 0.136543 0.029703
4 0.318083 0.604870, 'foo', array([1, 2, 3]), 2013-01-01 0.330824
2013-01-02 0.790825
2013-01-03 0.308468
2013-01-04 0.092397
2013-01-05 0.703091
Freq: D, dtype: float64, A B
0 0.275432 0.293583
1 0.842639 0.165381
2 0.608925 0.778891
3 0.136543 0.029703
4 0.318083 0.604870]
```

不像其他io方法，`to_msgpack` 既可以基于每个对象使用 `df.to_msgpack()` 方法，也可以在混合pandas对象的时候使用顶层 `pd.to_msgpack(...)` 方法，该方法可以让你打包任意的Python列表、字典、标量的集合。

```
In [352]: pd.to_msgpack('foo2.msg', {'dict': [{'df': df}, {'string': 'foo'},
.....: {'scalar': 1.}, {'s': s}]))
.....:
```

```
In [353]: pd.read_msgpack('foo2.msg')
```

```
Out[353]:
{'dict': ({'df': A B
0 0.275432 0.293583
1 0.842639 0.165381
2 0.608925 0.778891
3 0.136543 0.029703
4 0.318083 0.604870},
{'string': 'foo'},
{'scalar': 1.0},
{'s': 2013-01-01 0.330824
2013-01-02 0.790825
2013-01-03 0.308468
2013-01-04 0.092397
2013-01-05 0.703091
Freq: D, dtype: float64}}}
```

## 读/写API

Msgpacks也能读写字符串。

```
In [354]: df.to_msgpack()
```

```
Out[354]:
b'\x84\xa3typ\xadblock_manager\xa5klass\xa9DataFrame\xa4axes\x92\x86\xa3typ\xa5index\xa5klass\xa5Index\xa4nam
\x84\xac\xa0\xd1?\x0f\xa4.\xb5\xe6\xf6\xea?\xb9\x85\x9aL0|\xe3?\xac\xf0\xd7\x81>z\xc1?\\xca\x97\ty[\xd4?\x9c
```

此外你可以连接字符串生成一个原始的对象列表。

```
In [355]: pd.read_msgpack(df.to_msgpack() + s.to_msgpack())
```

```
Out[355]:
[A B
0 0.275432 0.293583
1 0.842639 0.165381
2 0.608925 0.778891
3 0.136543 0.029703
4 0.318083 0.604870, 2013-01-01 0.330824
2013-01-02 0.790825
2013-01-03 0.308468
2013-01-04 0.092397
2013-01-05 0.703091
Freq: D, dtype: float64]
```

## HDF5(PyTables) (一种以.h5结尾的分层数据格式)

HDFStore 是一个能读写pandas的类似字典的对象，它能使用高性能的HDF5格式，该格式是用优秀的PyTables库写的。一些更高级的用法参考[cookbook](#)。

::: danger 警告

pandas要求使用的 PyTables 版本要  $\geq 3.0.0$ 。当使用索引来检索存储的时候，PyTables  $< 3.2$ 的版本会出现索引bug。如果返回一个结果的子集，那么你就需要升级 PyTables 的版本  $\geq 3.2$ 才行。先前创建的存储数据将会使用更新后的版本再次写入。

:::

```
In [356]: store = pd.HDFStore('store.h5')

In [357]: print(store)
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5
```

对象能够被写入文件就像成对的键-值添加到字典里面一样：

```
In [358]: index = pd.date_range('1/1/2000', periods=8)

In [359]: s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])

In [360]: df = pd.DataFrame(np.random.randn(8, 3), index=index,
.....: columns=['A', 'B', 'C'])
.....:

store.put('s', s) is an equivalent method
In [361]: store['s'] = s

In [362]: store['df'] = df

In [363]: store
Out[363]:
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5
```

在当前或者之后的Python会话中，你都能检索存储的对象：

```
store.get('df') is an equivalent method
In [364]: store['df']
Out[364]:
```

	A	B	C
2000-01-01	-0.426936	-1.780784	0.322691
2000-01-02	1.638174	-2.184251	0.049673
2000-01-03	-1.022803	0.889445	2.827717
2000-01-04	1.767446	-1.305266	-0.378355
2000-01-05	0.486743	0.954551	0.859671
2000-01-06	-1.170458	-1.211386	-0.852728
2000-01-07	-0.450781	1.064650	1.014927
2000-01-08	-0.810399	0.254343	-0.875526

```
dotted (attribute) access provides get as well
In [365]: store.df
Out[365]:
```

	A	B	C
2000-01-01	-0.426936	-1.780784	0.322691
2000-01-02	1.638174	-2.184251	0.049673
2000-01-03	-1.022803	0.889445	2.827717
2000-01-04	1.767446	-1.305266	-0.378355
2000-01-05	0.486743	0.954551	0.859671
2000-01-06	-1.170458	-1.211386	-0.852728
2000-01-07	-0.450781	1.064650	1.014927
2000-01-08	-0.810399	0.254343	-0.875526

使用键删除指定的对象：

```
store.remove('df') is an equivalent method
In [366]: del store['df']

In [367]: store
Out[367]:
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5
```

关闭存储对象并使用环境管理器：

```
In [368]: store.close()

In [369]: store
Out[369]:
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5

In [370]: store.is_open
Out[370]: False

Working with, and automatically closing the store using a context manager
In [371]: with pd.HDFStore('store.h5') as store:
.....: store.keys()
.....:
```

## 读/写 API

HDFStore 支持顶层的API，用 `read_hdf` 来读取，和使用 `to_hdf` 来写入，类似于 `read_csv` 和 `to_csv` 的用法。

```
In [372]: df_tl = pd.DataFrame({'A': list(range(5)), 'B': list(range(5))})

In [373]: df_tl.to_hdf('store_tl.h5', 'table', append=True)

In [374]: pd.read_hdf('store_tl.h5', 'table', where=['index>2'])
Out[374]:
 A B
3 3 3
4 4 4
```

HDFStore默认不会删除全是缺失值的行，但是通过设置 `dropna=True` 参数就能改变。

```
In [375]: df_with_missing = pd.DataFrame({'col1': [0, np.nan, 2],
.....: 'col2': [1, np.nan, np.nan]})
.....:

In [376]: df_with_missing
Out[376]:
 col1 col2
0 0.0 1.0
1 NaN NaN
2 2.0 NaN

In [377]: df_with_missing.to_hdf('file.h5', 'df_with_missing',
.....: format='table', mode='w')
.....:

In [378]: pd.read_hdf('file.h5', 'df_with_missing')
Out[378]:
 col1 col2
0 0.0 1.0
1 NaN NaN
2 2.0 NaN

In [379]: df_with_missing.to_hdf('file.h5', 'df_with_missing',
.....: format='table', mode='w', dropna=True)
.....:

In [380]: pd.read_hdf('file.h5', 'df_with_missing')
Out[380]:
 col1 col2
0 0.0 1.0
2 2.0 NaN
```

## 固定格式

上面的例子表明了使用 `put` 进行存储的情况，该存储将 HDF5 以固定数组格式写入 PyTables，这就是所谓的 `fixed` 格式。这些类型的存储一旦被写入后将**不能再添加数据**（虽然你能轻易地删除它们并再次写入），也



不能查询；必须全部检索它们。它们也不支持没有唯一列名的数据表。fixed 格式提供了非常快速的写入功能，并且比 table 存储在读取方面更快捷。默认的指定格式是使用 put 或者 to\_hdf 亦或通过 format='fixed' 或 format='f' 格式。

⚠ danger 警告

如果你尝试使用 where 来检索，fixed 格式将会报错 TypeError：

```
>>> pd.DataFrame(np.random.randn(10, 2)).to_hdf('test_fixed.h5', 'df')
>>> pd.read_hdf('test_fixed.h5', 'df', where='index>5')
TypeError: cannot pass a where specification when reading a fixed format.
this store must be selected in its entirety
```

⚠

## 表格格式

HDFStore 支持在磁盘上使用另一种 PyTables 格式，即 table 格式。从概念上来讲，table 在外形上同具有行和列的 DataFrame 极度相似。table 也能被添加到同样的或其他的会话中。此外，删除和查询操作也是支持的。通过指定格式为 format='table' 或 format='t' 到 append 方法或 put 或者 to\_hdf。

put/append/to\_hdf 方法中使用的格式也可以设置为可选 pd.set\_option('io.hdf.default\_format', 'table')，以默认的 table 格式存储。

```

In [381]: store = pd.HDFStore('store.h5')

In [382]: df1 = df[0:4]

In [383]: df2 = df[4:]

append data (creates a table automatically)
In [384]: store.append('df', df1)

In [385]: store.append('df', df2)

In [386]: store
Out[386]:
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5

select the entire object
In [387]: store.select('df')
Out[387]:
 A B C
2000-01-01 -0.426936 -1.780784 0.322691
2000-01-02 1.638174 -2.184251 0.049673
2000-01-03 -1.022803 0.889445 2.827717
2000-01-04 1.767446 -1.305266 -0.378355
2000-01-05 0.486743 0.954551 0.859671
2000-01-06 -1.170458 -1.211386 -0.852728
2000-01-07 -0.450781 1.064650 1.014927
2000-01-08 -0.810399 0.254343 -0.875526

the type of stored data
In [388]: store.root.df._v_attrs.pandas_type
Out[388]: 'frame_table'

```

::: tip 注意

你也可以通过创建 `table` 来传递 `format='table'` 或者 `format='t'` 到 `put` 操作。

:::

## 分层键

存储的键能够指定为字符串，这些分层的路径名就像这样的格式（例如： `foo/bar/bah` ）。它将生成子存储的层次结构（或者在PyTables中叫做 `Groups` ）。键可以不带前面的 `'/'` 指定而且总是单独的（例如： `'foo'` 指的就是 `'/foo'` ）。删除操作能够删除所有子存储及之后的数据，所以要小心该操作。

```
In [389]: store.put('foo/bar/bah', df)

In [390]: store.append('food/orange', df)

In [391]: store.append('food/apple', df)

In [392]: store
Out[392]:
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5

a list of keys are returned
In [393]: store.keys()
Out[393]: ['/df', '/food/apple', '/food/orange', '/foo/bar/bah']

remove all nodes under this level
In [394]: store.remove('food')

In [395]: store
Out[395]:
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5
```

你能遍历组层次结构使用 `walk` 方法，该方法将为每个组键及其内容的相对键生成一个元组。

New in version 0.24.0.

```
In [396]: for (path, subgroups, subkeys) in store.walk():
.....: for subgroup in subgroups:
.....: print('GROUP: {}/{}'.format(path, subgroup))
.....: for subkey in subkeys:
.....: key = '/'.join([path, subkey])
.....: print('KEY: {}'.format(key))
.....: print(store.get(key))
.....:
```

GROUP: /foo

KEY: /df

	A	B	C
2000-01-01	-0.426936	-1.780784	0.322691
2000-01-02	1.638174	-2.184251	0.049673
2000-01-03	-1.022803	0.889445	2.827717
2000-01-04	1.767446	-1.305266	-0.378355
2000-01-05	0.486743	0.954551	0.859671
2000-01-06	-1.170458	-1.211386	-0.852728
2000-01-07	-0.450781	1.064650	1.014927
2000-01-08	-0.810399	0.254343	-0.875526

GROUP: /foo/bar

KEY: /foo/bar/bah

	A	B	C
2000-01-01	-0.426936	-1.780784	0.322691
2000-01-02	1.638174	-2.184251	0.049673
2000-01-03	-1.022803	0.889445	2.827717
2000-01-04	1.767446	-1.305266	-0.378355
2000-01-05	0.486743	0.954551	0.859671
2000-01-06	-1.170458	-1.211386	-0.852728
2000-01-07	-0.450781	1.064650	1.014927
2000-01-08	-0.810399	0.254343	-0.875526

::: danger 警告

分层键对于存储在根节点下的项目，无法使用如上的方法将其作为点（属性）进行检索。

```
In [8]: store.foo.bar.bah
```

```
AttributeError: 'HDFStore' object has no attribute 'foo'
```

```
you can directly access the actual PyTables node but using the root node
```

```
In [9]: store.root.foo.bar.bah
```

```
Out[9]:
```

```
/foo/bar/bah (Group) ''
```

```
children := ['block0_items' (Array), 'block0_values' (Array), 'axis0' (Array), 'axis1' (Array)]
```

相反，使用基于显式字符串的键：

```
In [397]: store['foo/bar/bah']
Out[397]:
```

	A	B	C
2000-01-01	-0.426936	-1.780784	0.322691
2000-01-02	1.638174	-2.184251	0.049673
2000-01-03	-1.022803	0.889445	2.827717
2000-01-04	1.767446	-1.305266	-0.378355
2000-01-05	0.486743	0.954551	0.859671
2000-01-06	-1.170458	-1.211386	-0.852728
2000-01-07	-0.450781	1.064650	1.014927
2000-01-08	-0.810399	0.254343	-0.875526

...

## 存储类型

### 在表格中存储混合类型

支持混合数据类型存储。字符串使用添加列的最大尺寸以固定宽度进行存储。后面尝试添加更长的字符串将会报错 `ValueError`。

添加参数 `min_itemsize={'values': size}` 将给字符串列设置一个更大的最小值。目前支持的存储类型有 `floats`, `strings`, `ints`, `bools`, `datetime64`。对于字符串列，添加参数 `nan_rep = 'nan'` 将改变磁盘上默认的 `nan` 值（转变为 `np.nan`），原本默认是 `nan`。

```
In [398]: df_mixed = pd.DataFrame({'A': np.random.randn(8),
.....: 'B': np.random.randn(8),
.....: 'C': np.array(np.random.randn(8), dtype='float32'),
.....: 'string': 'string',
.....: 'int': 1,
.....: 'bool': True,
.....: 'datetime64': pd.Timestamp('20010102')},
.....: index=list(range(8)))
```

```
In [399]: df_mixed.loc[df_mixed.index[3:5],
.....: ['A', 'B', 'string', 'datetime64']] = np.nan
.....:
```

```
In [400]: store.append('df_mixed', df_mixed, min_itemsize={'values': 50})
```

```
In [401]: df_mixed1 = store.select('df_mixed')
```

```
In [402]: df_mixed1
```

```
Out[402]:
```

	A	B	C	string	int	bool	datetime64
0	-0.980856	0.298656	0.151508	string	1	True	2001-01-02
1	-0.906920	-1.294022	0.587939	string	1	True	2001-01-02
2	0.988185	-0.618845	0.043096	string	1	True	2001-01-02
3	NaN	NaN	0.362451	NaN	1	True	NaT
4	NaN	NaN	1.356269	NaN	1	True	NaT
5	-0.772889	-0.340872	1.798994	string	1	True	2001-01-02
6	-0.043509	-0.303900	0.567265	string	1	True	2001-01-02
7	0.768606	-0.871948	-0.044348	string	1	True	2001-01-02

```
In [403]: df_mixed1.dtypes.value_counts()
```

```
Out[403]:
float64 2
float32 1
datetime64[ns] 1
int64 1
bool 1
object 1
dtype: int64
```

```
we have provided a minimum string column size
```

```
In [404]: store.root.df_mixed.table
```

```
Out[404]:
/df_mixed/table (Table(8,)) ''
description := {
 "index": Int64Col(shape=(), dflt=0, pos=0),
 "values_block_0": Float64Col(shape=(2,), dflt=0.0, pos=1),
 "values_block_1": Float32Col(shape=(1,), dflt=0.0, pos=2),
 "values_block_2": Int64Col(shape=(1,), dflt=0, pos=3),
 "values_block_3": Int64Col(shape=(1,), dflt=0, pos=4),
 "values_block_4": BoolCol(shape=(1,), dflt=False, pos=5),
 "values_block_5": StringCol(itemsize=50, shape=(1,), dflt=b'', pos=6)}
byteorder := 'little'
chunkshape := (689,)
autoindex := True
```

```
colindexes := {
 "index": Index(6, medium, shuffle, zlib(1)).is_csi=False}
```

## 存储多层索引数据表

存储多层索引 DataFrames 为表格与从同类索引 DataFrames 中存储/选取是非常类似的。

```
In [405]: index = pd.MultiIndex(levels=[['foo', 'bar', 'baz', 'qux'],
.....: ['one', 'two', 'three']],
.....: codes=[[0, 0, 0, 1, 1, 2, 2, 3, 3, 3],
.....: [0, 1, 2, 0, 1, 1, 2, 0, 1, 2]],
.....: names=['foo', 'bar'])
.....:
```

```
In [406]: df_mi = pd.DataFrame(np.random.randn(10, 3), index=index,
.....: columns=['A', 'B', 'C'])
.....:
```

```
In [407]: df_mi
```

```
Out[407]:
```

	A	B	C
foo bar			
foo one	0.031885	0.641045	0.479460
two	-0.630652	-0.182400	-0.789979
three	-0.282700	-0.813404	1.252998
bar one	0.758552	0.384775	-1.133177
two	-1.002973	-1.644393	-0.311536
baz two	-0.615506	-0.084551	-1.318575
three	0.923929	-0.105981	0.429424
qux one	-1.034590	0.542245	-0.384429
two	0.170697	-0.200289	1.220322
three	-1.001273	0.162172	0.376816

```
In [408]: store.append('df_mi', df_mi)
```

```
In [409]: store.select('df_mi')
```

```
Out[409]:
```

	A	B	C
foo bar			
foo one	0.031885	0.641045	0.479460
two	-0.630652	-0.182400	-0.789979
three	-0.282700	-0.813404	1.252998
bar one	0.758552	0.384775	-1.133177
two	-1.002973	-1.644393	-0.311536
baz two	-0.615506	-0.084551	-1.318575
three	0.923929	-0.105981	0.429424
qux one	-1.034590	0.542245	-0.384429
two	0.170697	-0.200289	1.220322
three	-1.001273	0.162172	0.376816

```
the levels are automatically included as data columns
```

```
In [410]: store.select('df_mi', 'foo=bar')
```

```
Out[410]:
```

	A	B	C
foo bar			
bar one	0.758552	0.384775	-1.133177
two	-1.002973	-1.644393	-0.311536



## 查询

### 查询表格

`select` 和 `delete` 操作有一个可选项即能指定选择/删除仅有数据的子集。这允许用户拥有一个很大的磁盘表并仅检索一部分数据。

在底层里使用 `Term` 类指定查询为布尔表达式。

- 支持的 `DataFrames` 索引器有 `index` 和 `columns`。
- 如果指定为 `data_columns`，这些将作为额外的索引器。

有效的比较运算符有：

`=, ==, !=, >, >=, <, <=`

有效的布尔表达式包含如下几种：

- `|` : 选择
- `&` : 并列
- `(` 和 `)` : 用来分组

这些规则同在 `pandas` 的索引中使用布尔表达式是类似的。

⋮ tip 注意

- `=` 将自动扩展为比较运算符 `==`
- `~` 不是运算符，且只在有限的条件下使用
- 如果传递的表达式时列表/元组，他们将通过 `&` 符号合并

⋮

以下都是有效的表达式：

- `'index >= date'`
- `"columns = ['A', 'D']"`
- `"columns in ['A', 'D']"`
- `'columns = A'`
- `'columns == A'`
- `"~(columns = ['A', 'B'])"`
- `'index > df.index[3] & string = "bar"'`
- `'(index > df.index[3] & index <= df.index[6]) | string = "bar"'`
- `"ts >= Timestamp('2012-02-01')"`
- `"major_axis>=20130101"`

`indexers` 在子表达式的左边的有：`columns`, `major_axis`, `ts`

(在比较运算符后面) 子表达式可以是：

- 能被求值的函数，比如： `Timestamp('2012-02-01')`
- 字符串，比如： `"bar"`
- 类似日期，比如： `20130101` 或者 `"20130101"`
- 列表，比如： `"['A', 'B']"`
- 以本地命名空间定义的变量，比如： `date`

⋮ tip 注意

在查询表达式中插入字符串进行查询是不推荐的。如果将带有%的字符串分配给变量，然后在表达式中使用该变量。那么，这样做

```
string = "HolyMoly"
store.select('df', 'index == string')
```

来代替下面这样

```
string = "HolyMoly"
store.select('df', 'index == %s' % string)
```

因为后者将 **不会** 起作用并引起 `SyntaxError`。注意 `string` 变量的双引号里面有一个单引号。

如果你一定要插入，使用说明符格式 `'%r'`

```
store.select('df', 'index == %r' % string)
```

它将会引用变量 `string`。

⋮

这儿有一些例子：

```
In [411]: dfq = pd.DataFrame(np.random.randn(10, 4), columns=list('ABCD'),
.....: index=pd.date_range('20130101', periods=10))
.....:

In [412]: store.append('dfq', dfq, format='table', data_columns=True)
```

使用布尔表达式同内联求值函数。

```
In [413]: store.select('dfq', "index>pd.Timestamp('20130104') & columns=['A', 'B']")
Out[413]:
```

	A	B
2013-01-05	0.450263	0.755221
2013-01-06	0.019915	0.300003
2013-01-07	1.878479	-0.026513
2013-01-08	3.272320	0.077044
2013-01-09	-0.398346	0.507286
2013-01-10	0.516017	-0.501550

## 内联引用

```
In [414]: store.select('dfq', where="A>0 or C>0")
Out[414]:
```

	A	B	C	D
2013-01-01	-0.161614	-1.636805	0.835417	0.864817
2013-01-02	0.843452	-0.122918	-0.026122	-1.507533
2013-01-03	0.335303	-1.340566	-1.024989	1.125351
2013-01-05	0.450263	0.755221	-1.506656	0.808794
2013-01-06	0.019915	0.300003	-0.727093	-1.119363
2013-01-07	1.878479	-0.026513	0.573793	0.154237
2013-01-08	3.272320	0.077044	0.397034	-0.613983
2013-01-10	0.516017	-0.501550	0.138212	0.218366

关键词 `columns` 能用来筛选列字段并返回为列表，这等价于传递 '`columns=list_of_columns_to_filter`'：

```
In [415]: store.select('df', "columns=['A', 'B']")
Out[415]:
```

	A	B
2000-01-01	-0.426936	-1.780784
2000-01-02	1.638174	-2.184251
2000-01-03	-1.022803	0.889445
2000-01-04	1.767446	-1.305266
2000-01-05	0.486743	0.954551
2000-01-06	-1.170458	-1.211386
2000-01-07	-0.450781	1.064650
2000-01-08	-0.810399	0.254343

`start` and `stop` 参数能指定总的搜索范围。这些是根据表中的总行数得出来的。

## ::: tip 注意

如果查询表达式有未知的引用变量，那么 `select` 将会报错 `ValueError`。通常这就意味着你正在尝试选取的一列并不在当前数据列中。

如果查询表达式无效，那么 `select` 将会报错 `SyntaxError`。

:::

## timedelta64[ns]的用法

你能使用 `timedelta64[ns]` 进行存储和查询。使用 `()` 来指定查询的条目，浮点数可以带符号（和小数），`timedelta` 的单位可以是 `D,s,ms,us,ns`。看示例：

```
In [416]: from datetime import timedelta

In [417]: dftd = pd.DataFrame({'A': pd.Timestamp('20130101'),
.....: 'B': [pd.Timestamp('20130101') + timedelta(days=i,
.....: seconds=10)
.....: for i in range(10)]})

In [418]: dftd['C'] = dftd['A'] - dftd['B']

In [419]: dftd
Out[419]:
```

	A	B	C
0	2013-01-01 00:00:10	-1 days +23:59:50	
1	2013-01-01 00:00:10	-2 days +23:59:50	
2	2013-01-01 00:00:10	-3 days +23:59:50	
3	2013-01-01 00:00:10	-4 days +23:59:50	
4	2013-01-01 00:00:10	-5 days +23:59:50	
5	2013-01-01 00:00:10	-6 days +23:59:50	
6	2013-01-01 00:00:10	-7 days +23:59:50	
7	2013-01-01 00:00:10	-8 days +23:59:50	
8	2013-01-01 00:00:10	-9 days +23:59:50	
9	2013-01-01 00:00:10	-10 days +23:59:50	

```
In [420]: store.append('dftd', dftd, data_columns=True)

In [421]: store.select('dftd', "C<-3.5D")
Out[421]:
```

	A	B	C
4	2013-01-01 00:00:10	-5 days +23:59:50	
5	2013-01-01 00:00:10	-6 days +23:59:50	
6	2013-01-01 00:00:10	-7 days +23:59:50	
7	2013-01-01 00:00:10	-8 days +23:59:50	
8	2013-01-01 00:00:10	-9 days +23:59:50	
9	2013-01-01 00:00:10	-10 days +23:59:50	

## 索引

你能在表格中已经有数据的情况下（在 `append/put` 操作之后）使用 `create_table_index` 创建/修改表格的索引。给表格创建索引是**推荐**的操作。当你使用带有索引的 `select` 当作 `where` 查询条件的时候，这将极大的加快你的查询速度。

⋮ tip 注意

索引会自动创建在可索引对象和任意你指定的数据列。你可以传递 `index=False` 到 `append` 来关闭这个操作。

⋮

```

we have automagically already created an index (in the first section)
In [422]: i = store.root.df.table.cols.index.index

In [423]: i.optlevel, i.kind
Out[423]: (6, 'medium')

change an index by passing new parameters
In [424]: store.create_table_index('df', optlevel=9, kind='full')

In [425]: i = store.root.df.table.cols.index.index

In [426]: i.optlevel, i.kind
Out[426]: (9, 'full')

```

通常当有大量数据添加保存的时候，关闭添加列的索引创建，等结束后再创建是非常有效的。

```

In [427]: df_1 = pd.DataFrame(np.random.randn(10, 2), columns=list('AB'))

In [428]: df_2 = pd.DataFrame(np.random.randn(10, 2), columns=list('AB'))

In [429]: st = pd.HDFStore('appends.h5', mode='w')

In [430]: st.append('df', df_1, data_columns=['B'], index=False)

In [431]: st.append('df', df_2, data_columns=['B'], index=False)

In [432]: st.get_storer('df').table
Out[432]:
/df/table (Table(20,)) ''
 description := {
 "index": Int64Col(shape=(), dflt=0, pos=0),
 "values_block_0": Float64Col(shape=(1,), dflt=0.0, pos=1),
 "B": Float64Col(shape=(), dflt=0.0, pos=2)}
 byteorder := 'little'
 chunkshape := (2730,)

```

当完成添加后再创建索引。

```

In [433]: st.create_table_index('df', columns=['B'], optlevel=9, kind='full')

In [434]: st.get_storer('df').table
Out[434]:
/df/table (Table(20,)) ''
description := {
 "index": Int64Col(shape=(), dflt=0, pos=0),
 "values_block_0": Float64Col(shape=(1,), dflt=0.0, pos=1),
 "B": Float64Col(shape=(), dflt=0.0, pos=2)}
byteorder := 'little'
chunkshape := (2730,)
autoindex := True
colindexes := {
 "B": Index(9, full, shuffle, zlib(1)).is_csi=True}

In [435]: st.close()

```

看[这里](#)关于如何在现存的表格中创建完全分类索引（CSI）。

## 通过数据列查询

你可以指定（并建立索引）某些你希望能够执行查询的列（除了可索引的列，你始终可以查询这些列）。例如，假设你要在磁盘上执行此常规操作，仅返回与该查询匹配的帧。你可以指定 `data_columns = True` 来强制所有列为 `data_columns`。

```

In [436]: df_dc = df.copy()

In [437]: df_dc['string'] = 'foo'

In [438]: df_dc.loc[df_dc.index[4:6], 'string'] = np.nan

In [439]: df_dc.loc[df_dc.index[7:9], 'string'] = 'bar'

In [440]: df_dc['string2'] = 'cool'

In [441]: df_dc.loc[df_dc.index[1:3], ['B', 'C']] = 1.0

In [442]: df_dc
Out[442]:

```

	A	B	C	string	string2
2000-01-01	-0.426936	-1.780784	0.322691	foo	cool
2000-01-02	1.638174	1.000000	1.000000	foo	cool
2000-01-03	-1.022803	1.000000	1.000000	foo	cool
2000-01-04	1.767446	-1.305266	-0.378355	foo	cool
2000-01-05	0.486743	0.954551	0.859671	NaN	cool
2000-01-06	-1.170458	-1.211386	-0.852728	NaN	cool
2000-01-07	-0.450781	1.064650	1.014927	foo	cool
2000-01-08	-0.810399	0.254343	-0.875526	bar	cool

```

on-disk operations
In [443]: store.append('df_dc', df_dc, data_columns=['B', 'C', 'string', 'string2'])

In [444]: store.select('df_dc', where='B > 0')
Out[444]:

```

	A	B	C	string	string2
2000-01-02	1.638174	1.000000	1.000000	foo	cool
2000-01-03	-1.022803	1.000000	1.000000	foo	cool
2000-01-05	0.486743	0.954551	0.859671	NaN	cool
2000-01-07	-0.450781	1.064650	1.014927	foo	cool
2000-01-08	-0.810399	0.254343	-0.875526	bar	cool

```

getting creative
In [445]: store.select('df_dc', 'B > 0 & C > 0 & string == foo')
Out[445]:

```

	A	B	C	string	string2
2000-01-02	1.638174	1.000000	1.000000	foo	cool
2000-01-03	-1.022803	1.000000	1.000000	foo	cool
2000-01-07	-0.450781	1.064650	1.014927	foo	cool

```

this is in-memory version of this type of selection
In [446]: df_dc[(df_dc.B > 0) & (df_dc.C > 0) & (df_dc.string == 'foo')]
Out[446]:

```

	A	B	C	string	string2
2000-01-02	1.638174	1.000000	1.000000	foo	cool
2000-01-03	-1.022803	1.000000	1.000000	foo	cool
2000-01-07	-0.450781	1.064650	1.014927	foo	cool

```

we have automagically created this index and the B/C/string/string2
columns are stored separately as ``PyTables`` columns
In [447]: store.root.df_dc.table

```

```

Out[447]:
/df_dc/table (Table(8,)) ''
description := {
 "index": Int64Col(shape=(), dflt=0, pos=0),
 "values_block_0": Float64Col(shape=(1,), dflt=0.0, pos=1),
 "B": Float64Col(shape=(), dflt=0.0, pos=2),
 "C": Float64Col(shape=(), dflt=0.0, pos=3),
 "string": StringCol(itemsz=3, shape=(), dflt=b'', pos=4),
 "string2": StringCol(itemsz=4, shape=(), dflt=b'', pos=5)}
byteorder := 'little'
chunkshape := (1680,)
autoindex := True
colindexes := {
 "index": Index(6, medium, shuffle, zlib(1)).is_csi=False,
 "B": Index(6, medium, shuffle, zlib(1)).is_csi=False,
 "C": Index(6, medium, shuffle, zlib(1)).is_csi=False,
 "string": Index(6, medium, shuffle, zlib(1)).is_csi=False,
 "string2": Index(6, medium, shuffle, zlib(1)).is_csi=False}

```

把很多列变成数据列会存在性能下降的情况，因此它取决于用户。此外，在第一次添加/插入操作后你不能改变数据列（也不能索引）（当然，你能读取数据和创建一个新表！）。

## 迭代器

你能传递 `iterator=True` 或者 `chunksize=number_in_a_chunk` 给 `select` 和 `select_as_multiple`，然后在结果中返回一个迭代器。默认一个块返回50,000行。

```

In [448]: for df in store.select('df', chunksize=3):
.....: print(df)
.....:
 A B C
2000-01-01 -0.426936 -1.780784 0.322691
2000-01-02 1.638174 -2.184251 0.049673
2000-01-03 -1.022803 0.889445 2.827717
 A B C
2000-01-04 1.767446 -1.305266 -0.378355
2000-01-05 0.486743 0.954551 0.859671
2000-01-06 -1.170458 -1.211386 -0.852728
 A B C
2000-01-07 -0.450781 1.064650 1.014927
2000-01-08 -0.810399 0.254343 -0.875526

```

⋮ tip 注意

你也能使用 `read_hdf` 打开迭代器，然后迭代结束会自动关闭保存。

```

for df in pd.read_hdf('store.h5', 'df', chunksize=3):
 print(df)

```

⋮



注意，`chunksize`主要适用于源行。因此，如果你正在进行查询，`chunksize`将细分表中的总行和应用的查询，并在大小可能不相等的块上返回一个迭代器。

这是生成查询并使用它创建大小相等的返回块的方法。

```
In [449]: dfreq = pd.DataFrame({'number': np.arange(1, 11)})

In [450]: dfreq
Out[450]:
 number
0 1
1 2
2 3
3 4
4 5
5 6
6 7
7 8
8 9
9 10

In [451]: store.append('dfreq', dfreq, data_columns=['number'])

In [452]: def chunks(l, n):
.....: return [l[i:i + n] for i in range(0, len(l), n)]
.....:

In [453]: evens = [2, 4, 6, 8, 10]

In [454]: coordinates = store.select_as_coordinates('dfreq', 'number=evens')

In [455]: for c in chunks(coordinates, 2):
.....: print(store.select('dfreq', where=c))
.....:
 number
1 2
3 4
 number
5 6
7 8
 number
9 10
```

## 高级查询

### 选取单列

使用 `select_column` 方法可以找到单个可索引列或数据列。例如，这将让你非常快速地得到索引。它会返回一个由行号索引的 `Series` 结果。目前不接受 `where` 选择器。

```
In [456]: store.select_column('df_dc', 'index')
Out[456]:
0 2000-01-01
1 2000-01-02
2 2000-01-03
3 2000-01-04
4 2000-01-05
5 2000-01-06
6 2000-01-07
7 2000-01-08
Name: index, dtype: datetime64[ns]

In [457]: store.select_column('df_dc', 'string')
Out[457]:
0 foo
1 foo
2 foo
3 foo
4 NaN
5 NaN
6 foo
7 bar
Name: string, dtype: object
```

### 选取坐标

有时候你想要得到查询的坐标（又叫做 索引的定位），使用 `Int64Index` 将返回结果的定位。这些坐标也可以传递给之后的 `where` 操作。

```

In [458]: df_coord = pd.DataFrame(np.random.randn(1000, 2),
.....: index=pd.date_range('20000101', periods=1000))
.....:

In [459]: store.append('df_coord', df_coord)

In [460]: c = store.select_as_coordinates('df_coord', 'index > 20020101')

In [461]: c
Out[461]:
Int64Index([732, 733, 734, 735, 736, 737, 738, 739, 740, 741,
...
 990, 991, 992, 993, 994, 995, 996, 997, 998, 999],
 dtype='int64', length=268)

In [462]: store.select('df_coord', where=c)
Out[462]:
 0 1
2002-01-02 0.440865 -0.151651
2002-01-03 -1.195089 0.285093
2002-01-04 -0.925046 0.386081
2002-01-05 -1.942756 0.277699
2002-01-06 0.811776 0.528965
...
2002-09-22 1.061729 0.618085
2002-09-23 -0.209744 0.677197
2002-09-24 -1.808184 0.185667
2002-09-25 -0.208629 0.928603
2002-09-26 1.579717 -1.259530

[268 rows x 2 columns]

```

### 使用位置遮罩选取

有时你的查询可能涉及到创建要选择的行列表。通常这个 `mask` 将得到索引操作的 `index` 结果。下面这个例子显示了选取日期索引的月份等于5的操作。

```

In [463]: df_mask = pd.DataFrame(np.random.randn(1000, 2),
.....: index=pd.date_range('20000101', periods=1000))
.....:

In [464]: store.append('df_mask', df_mask)

In [465]: c = store.select_column('df_mask', 'index')

In [466]: where = c[pd.DatetimeIndex(c).month == 5].index

In [467]: store.select('df_mask', where=where)
Out[467]:
 0 1
2000-05-01 -1.199892 1.073701
2000-05-02 -1.058552 0.658487
2000-05-03 -0.015418 0.452879
2000-05-04 1.737818 0.426356
2000-05-05 -0.711668 -0.021266
...
2002-05-27 0.656196 0.993383
2002-05-28 -0.035399 -0.269286
2002-05-29 0.704503 2.574402
2002-05-30 -1.301443 2.770770
2002-05-31 -0.807599 0.420431

[93 rows x 2 columns]

```

## 存储对象

如果你想检查存储对象，可以通过 `get_storer` 找到。你能使用这种编程方法获得一个对象的行数。

```

In [468]: store.get_storer('df_dc').nrows
Out[468]: 8

```

## 多表查询

`append_to_multiple` 和 `select_as_multiple` 方法能一次性执行多表的添加/选取操作。这个方法是让一个表（称为选择器表）索引大多数/所有列，并执行查询。其他表是带索引的数据表，它会匹配选择器表的索引。然后，你能在选择器表执行非常快速的查询并返回大量数据。这个方法类似于有个非常宽的表，但能高效的查询。

`append_to_multiple` 方法根据 `d` 把单个 `DataFrame` 划分为多个表，这里的 `d` 指的是字典，即将表名映射到该表中所需的“列”列表。如果使用 `None` 代替列表，则该表将具有给定 `DataFrame` 的其余未指定列。`selector` 参数定义了哪张表是选择器表（即你可以从中执行查询的）。`dropna` 参数将删除输入 `DataFrame` 的行来确保表格是同步的。这意味着如果其中一张表写入的一行全是 `np.NaN`，那么将从所有表中删除这行。

如果 `dropna` 是 `False`，则**用户负责同步表**。记住全是 `np.NaN` 的行是不会写入 `HDFStore`，因此如果你选择 `dropna=False`，一些表会比其他表具有更多行，而且 `select_as_multiple` 将不会有作用或返回意外的结果。

```

In [469]: df_mt = pd.DataFrame(np.random.randn(8, 6),
.....: index=pd.date_range('1/1/2000', periods=8),
.....: columns=['A', 'B', 'C', 'D', 'E', 'F'])
.....:

In [470]: df_mt['foo'] = 'bar'

In [471]: df_mt.loc[df_mt.index[1], ('A', 'B')] = np.nan

you can also create the tables individually
In [472]: store.append_to_multiple({'df1_mt': ['A', 'B'], 'df2_mt': None},
.....: df_mt, selector='df1_mt')
.....:

In [473]: store
Out[473]:
<class 'pandas.io.pytables.HDFStore'>
File path: store.h5

individual tables were created
In [474]: store.select('df1_mt')
Out[474]:
 A B
2000-01-01 0.475158 0.427905
2000-01-02 NaN NaN
2000-01-03 -0.201829 0.651656
2000-01-04 -0.766427 -1.852010
2000-01-05 1.642910 -0.055583
2000-01-06 0.187880 1.536245
2000-01-07 -1.801014 0.244721
2000-01-08 3.055033 -0.683085

In [475]: store.select('df2_mt')
Out[475]:
 C D E F foo
2000-01-01 1.846285 -0.044826 0.074867 0.156213 bar
2000-01-02 0.446978 -0.323516 0.311549 -0.661368 bar
2000-01-03 -2.657254 0.649636 1.520717 1.604905 bar
2000-01-04 -0.201100 -2.107934 -0.450691 -0.748581 bar
2000-01-05 0.543779 0.111444 0.616259 -0.679614 bar
2000-01-06 0.831475 -0.566063 1.130163 -1.004539 bar
2000-01-07 0.745984 1.532560 0.229376 0.526671 bar
2000-01-08 -0.922301 2.760888 0.515474 -0.129319 bar

as a multiple
In [476]: store.select_as_multiple(['df1_mt', 'df2_mt'], where=['A>0', 'B>0'],
.....: selector='df1_mt')
.....:
Out[476]:
 A B C D E F foo
2000-01-01 0.475158 0.427905 1.846285 -0.044826 0.074867 0.156213 bar
2000-01-06 0.187880 1.536245 0.831475 -0.566063 1.130163 -1.004539 bar

```

## 从表中删除

你能够通过 `where` 指定有选择地从表中删除数据。删除行，重点理解 `PyTables` 删除行是通过先抹去行，接着删除后面的数据。因此，根据数据的方向来删除会是非常耗时的操作。所以为了获得最佳性能，首先让要删除的数据维度可索引是很有必要的。

数据根据（在磁盘上）可索引项来排序，这里有个简单的用例。你能存储面板数据（也叫时间序列-截面数据），在 `major_axis` 中存储日期，而 `minor_axis` 中存储ids。数据像下面这样交错：

- `date_1`
  - `id_1`
  - `id_2`
  - `.`
  - `id_n`
- `date_2`
  - `id_1`
  - `.`
  - `id_n`

应该清楚的是在 `major_axis` 上的删除操作将非常快，正如数据块被删除，接着后面的数据也会移动。另一方面，在 `minor_axis` 的操作将非常耗时。在这种情况下，几乎可以肯定使用 `where` 操作来选取所有除开缺失数据的列重写表格会更快。

::: danger 警告

请注意HDF5**不会自动回收空间**在h5文件中。于是重复地删除（或者移除节点）再添加操作**将会导致文件体积增大**。

要重新打包和清理文件，请使用[ptrepack](#)。

:::

## 注意事项

### 压缩

`PyTables` 允许存储地数据被压缩。这适用于所有类型地存储，不仅仅是表格。这两个参数 `complevel` 和 `complib` 可用来控制压缩。

`complevel` 指定数据会以何种方式压缩。

`complib` 指定要使用的压缩库。如果没有指定，那将使用默认的 `zlib` 库。压缩库通常会从压缩率或速度两方面来优化，而结果取决于数据类型。选择哪种压缩类型取决于你的具体需求和数据。下面是支持的压缩库列表：

- [zlib](#): 默认的压缩库。经典的压缩方式，能获得好的压缩率但是速度有点慢。

- `lzo`: 快速地压缩和解压。
- `bzip2`: 不错的压缩率。
- `blosc`: 快速地压缩和解压。

New in version 0.20.2: 支持另一种压缩机:

- `blosc:blosclz` 这是默认地 `blosc` 压缩机
- `blosc:lz4`: 一款紧凑、快速且流行的压缩机。
- `blosc:lz4hc`: 调整后的LZ4版本可产生更好的压缩比，但会牺牲速度。
- `blosc:snappy`: 一款在很多地方使用的流行压缩机。
- `blosc:zlib`: 经典款；虽然比前一款速度慢，但是可实现更好的压缩比。
- `blosc:zstd`: 极其平衡的编解码器；它是以上所有压缩机中提供最佳压缩比的，且速度相当快。

如果 `complib` 定义为其他的，不在上表中的库，那么就会出现 `ValueError`。

::: tip 注意

如果你的平台上缺失指定的 `complib` 库，压缩机会使用默认 `zlib` 库。

:::

文件中所有的对象都可以启用压缩:

```
store_compressed = pd.HDFStore('store_compressed.h5', complevel=9,
 complib='blosc:blosclz')
```

或者在未启用压缩的存储中进行即时压缩（这仅适用于表格）:

```
store.append('df', df, complib='zlib', complevel=5)
```

## ptrepack

`PyTables` 不是在一开始的时候开启压缩，而是在表被写入后再压缩，这提供了更好的写入性能。你能使用 `PyTables` 提供的实用程序 `ptrepack` 实现。此外，事实上在 `ptrepack` 之后会改变压缩等级。

```
ptrepack --chunkshape=auto --propindexes --complevel=9 --complib=blosc in.h5 out.h5
```

另外，`ptrepack in.h5 out.h5` 将重新打包文件让你可以重用之前删除的空间。或者，它能简单的删除文件并再次写入亦或使用 `copy` 方法。

## 注意事项

::: danger 警告

`HDFStore` **不是一个安全的写入线程**. `PyTables` 的底层仅支持（通过线程或进程的）并发读取。如果你要同时读取和写入，那么你需要单个进程的单个线程里序列化这些操作，否则会破坏你的数据。更多信息参见 [\(GH2397\)](#)。

...

- 如果你用锁来管理多个进程间的写入，那么你可能要在释放写锁之前使用 `fsync()`。方便起见，你能用 `store.flush(fsync=True)` 操作。
- 一旦 `table` 创建的列 (`DataFrame`)固定了; 那只有相同的列才可以添加数据。
- 注意时区 (例如, `pytz.timezone('US/Eastern')`)在不同的时区版本间不相等。因此，如果使用时区库的一个版本将数据本地化到`HDFStore`中的特定时区，并且使用另一个版本更新该数据，则由于这些时区不相等，因此数据将转换为UTC。使用相同版本的时区库或在更新的时区定义中使用 `tz_convert`。

::: danger 警告

如果列名没能用作属性选择器，那么 `PyTables` 将显示 `NaturalNameWarning` 。自然标识符仅包括字母、数字和下划线，且不能以数字开头。其他标识符不能用 `where` 从句，这通常不是个好主意。

...

### 数据类型

`HDFStore` 将对象数据类型映射到 `PyTables` 的底层数据类型。这意味着以下的已知类型都有效：

Type	Represents missing values
floating : float64, float32, float16	np.nan
integer : int64, int32, int8, uint64,uint32, uint8	
boolean	
datetime64[ns]	NaT
timedelta64[ns]	NaT
categorical : see the section below	
object : strings	np.nan

不支持 `unicode` 列，这会出现 **映射失败**。

### 数据类别

你可以写入含 `category` 类型的数据到 `HDFStore`。如果它是对象数组，那查询方式是一样的。然而，含 `category` 的数据会以更高效的方式存储。



```
In [477]: dfcat = pd.DataFrame({'A': pd.Series(list('aabbcdab')).astype('category'),
.....: 'B': np.random.randn(8)})
.....:
```

```
In [478]: dfcat
```

```
Out[478]:
```

	A	B
0	a	1.706605
1	a	1.373485
2	b	-0.758424
3	b	-0.116984
4	c	-0.959461
5	d	-1.517439
6	b	-0.453150
7	a	-0.827739

```
In [479]: dfcat.dtypes
```

```
Out[479]:
```

```
A category
B float64
dtype: object
```

```
In [480]: cstore = pd.HDFStore('cats.h5', mode='w')
```

```
In [481]: cstore.append('dfcat', dfcat, format='table', data_columns=['A'])
```

```
In [482]: result = cstore.select('dfcat', where="A in ['b', 'c']")
```

```
In [483]: result
```

```
Out[483]:
```

	A	B
2	b	-0.758424
3	b	-0.116984
4	c	-0.959461
6	b	-0.453150

```
In [484]: result.dtypes
```

```
Out[484]:
```

```
A category
B float64
dtype: object
```

## 字符串列

### min\_itemsize

对于字符串列，`HDFStore` 的底层使用的固定列宽（列的大小）。字符串列大小的计算方式是：**在第一个添加的时候**，传递给 `HDFStore`（该列）数据长度的最大值。随后的添加可能会引入**更大**一系列字符串，这超过了该列所能容纳的内容，这将引发异常（不然，你可以悄悄地截断这些列，让信息丢失）。之后将放松这一点，允许用户指定截断。

在第一个表创建的时候，传递 `min_itemsize` 将优先指定特定字符串列的最小长度。`min_itemsize` 可以是整数或将列名映射为整数的字典。你可以将 `values` 作为键传递，以允许所有可索引对象或 `data_columns` 具有此 `min_itemsize`。

传递 `min_itemsize` 字典将导致所有可传递列自动创建 `data_columns`。

::: tip 注意

如果你没有传递任意 `data_columns`，那么 `min_itemsize` 将会传递任意字符串的最大长度。

:::

```
In [485]: dfs = pd.DataFrame({'A': 'foo', 'B': 'bar'}, index=list(range(5)))
```

```
In [486]: dfs
```

```
Out[486]:
```

```
 A B
0 foo bar
1 foo bar
2 foo bar
3 foo bar
4 foo bar
```

```
A and B have a size of 30
```

```
In [487]: store.append('dfs', dfs, min_itemsize=30)
```

```
In [488]: store.get_storer('dfs').table
```

```
Out[488]:
```

```
/dfs/table (Table(5,)) ''
description := {
 "index": Int64Col(shape=(), dflt=0, pos=0),
 "values_block_0": StringCol(itemsize=30, shape=(2,), dflt=b'', pos=1)}
byteorder := 'little'
chunkshape := (963,)
autoindex := True
colindexes := {
 "index": Index(6, medium, shuffle, zlib(1)).is_csi=False}
```

```
A is created as a data_column with a size of 30
```

```
B is size is calculated
```

```
In [489]: store.append('dfs2', dfs, min_itemsize={'A': 30})
```

```
In [490]: store.get_storer('dfs2').table
```

```
Out[490]:
```

```
/dfs2/table (Table(5,)) ''
description := {
 "index": Int64Col(shape=(), dflt=0, pos=0),
 "values_block_0": StringCol(itemsize=3, shape=(1,), dflt=b'', pos=1),
 "A": StringCol(itemsize=30, shape=(), dflt=b'', pos=2)}
byteorder := 'little'
chunkshape := (1598,)
autoindex := True
colindexes := {
 "index": Index(6, medium, shuffle, zlib(1)).is_csi=False,
 "A": Index(6, medium, shuffle, zlib(1)).is_csi=False}
```

## nan\_rep

字符串列将序列化 `np.nan`（缺失值）以 `nan_rep` 的字符串形式。默认的字符串值为 `nan`。你可能会无意中  
将实际的 `nan` 值转换为缺失值。

```

In [491]: dfss = pd.DataFrame({'A': ['foo', 'bar', 'nan']})

In [492]: dfss
Out[492]:
 A
0 foo
1 bar
2 nan

In [493]: store.append('dfss', dfss)

In [494]: store.select('dfss')
Out[494]:
 A
0 foo
1 bar
2 NaN

here you need to specify a different nan rep
In [495]: store.append('dfss2', dfss, nan_rep='_nan_')

In [496]: store.select('dfss2')
Out[496]:
 A
0 foo
1 bar
2 nan

```

## 外部兼容性

`HDFStore` 以特定格式写入 `table` 对象，这些格式适用于产生无损往返的 `pandas` 对象。对于外部兼容性，`HDFStore` 能读取本地的 `PyTables` 格式表格。

可以编写一个 `HDFStore` 对象，该对象可以使用 `rhdf5` 库 ([Package website](#)) 轻松导入到 R 中。创建表格存储可以像这样：

```

In [497]: df_for_r = pd.DataFrame({"first": np.random.rand(100),
.....: "second": np.random.rand(100),
.....: "class": np.random.randint(0, 2, (100,))},
.....: index=range(100))
.....:

In [498]: df_for_r.head()
Out[498]:
 first second class
0 0.366979 0.794525 0
1 0.296639 0.635178 1
2 0.395751 0.359693 0
3 0.484648 0.970016 1
4 0.810047 0.332303 0

In [499]: store_export = pd.HDFStore('export.h5')

In [500]: store_export.append('df_for_r', df_for_r, data_columns=df_dc.columns)

In [501]: store_export
Out[501]:
<class 'pandas.io.pytables.HDFStore'>
File path: export.h5

```

在这个R文件中使用 `rhdf5` 库能读入数据到 `data.frame` 对象中。下面这个示例函数从值中读取相应的列名和数据值，再组合它们到 `data.frame` 中：

```

Load values and column names for all datasets from corresponding nodes and
insert them into one data.frame object.

library(rhdf5)

loadhdf5data <- function(h5File) {

 listing <- h5ls(h5File)
 # Find all data nodes, values are stored in *_values and corresponding column
 # titles in *_items
 data_nodes <- grep("_values", listing$name)
 name_nodes <- grep("_items", listing$name)
 data_paths = paste(listing$group[data_nodes], listing$name[data_nodes], sep = "/")
 name_paths = paste(listing$group[name_nodes], listing$name[name_nodes], sep = "/")
 columns = list()
 for (idx in seq(data_paths)) {
 # NOTE: matrices returned by h5read have to be transposed to obtain
 # required Fortran order!
 data <- data.frame(t(h5read(h5File, data_paths[idx])))
 names <- t(h5read(h5File, name_paths[idx]))
 entry <- data.frame(data)
 colnames(entry) <- names
 columns <- append(columns, entry)
 }

 data <- data.frame(columns)

 return(data)
}

```

现在你能导入 DataFrame 到R中:

```

> data = loadhdf5data("transfer.hdf5")
> head(data)
 first second class
1 0.4170220047 0.3266449 0
2 0.7203244934 0.5270581 0
3 0.0001143748 0.8859421 1
4 0.3023325726 0.3572698 1
5 0.1467558908 0.9085352 1
6 0.0923385948 0.6233601 1

```

::: tip 注意

R函数列出了整个HDF5文件的内容，并从所有匹配的节点组合了 data.frame 对象，因此，如果你已将多个 DataFrame 对象存储到单个HDF5文件中，那么只能用它作为起点。

:::

## 性能

- 同 `fixed` 存储相比较，`tables` 格式会有写入的性能损失。这样的好处就是便于（大量的数据）能添加/删除和查询。与常规存储相比，写入时间通常更长。但是查询的时间就相当快，特别是在有索引的轴上。
- 你可以传递 `chunksizes` 给 `append`，指定写入块的大小（默认是50000）。这将极大地降低写入时地内存使用情况。
- 你可以将 `expectedrows` 传递给第一个 `append` 来设置 PyTables 将预期的总行数。这将优化读取/写入性能。
- 重复的行可以写入表格，但是在选取的时候会进行筛选（会选最后一项；然后表在主要、次要对上唯一的）。
- 如果你企图存储已经序列化的PyTables类型数据（而不是存储为本地数据），那将引发 `PerformanceWarning`。更多信息和解决办法参见[Here](#)。

## Feather

---

New in version 0.20.0.

Feather provides binary columnar serialization for data frames. It is designed to make reading and writing data frames efficient, and to make sharing data across data analysis languages easy.

Feather is designed to faithfully serialize and de-serialize DataFrames, supporting all of the pandas dtypes, including extension dtypes such as categorical and datetime with tz.

Several caveats.

- This is a newer library, and the format, though stable, is not guaranteed to be backward compatible to the earlier versions.
- The format will NOT write an `Index`, or `MultiIndex` for the `DataFrame` and will raise an error if a non-default one is provided. You can `.reset_index()` to store the index or `.reset_index(drop=True)` to ignore it.
- Duplicate column names and non-string columns names are not supported
- Non supported types include `Period` and actual Python object types. These will raise a helpful error message on an attempt at serialization.

See the [Full Documentation](#).

```

In [502]: df = pd.DataFrame({'a': list('abc'),
.....: 'b': list(range(1, 4)),
.....: 'c': np.arange(3, 6).astype('u1'),
.....: 'd': np.arange(4.0, 7.0, dtype='float64'),
.....: 'e': [True, False, True],
.....: 'f': pd.Categorical(list('abc')),
.....: 'g': pd.date_range('20130101', periods=3),
.....: 'h': pd.date_range('20130101', periods=3, tz='US/Eastern'),
.....: 'i': pd.date_range('20130101', periods=3, freq='ns')})

In [503]: df
Out[503]:
 a b c d e f g h i
0 a 1 3 4.0 True a 2013-01-01 2013-01-01 00:00:00-05:00 2013-01-01 00:00:00.000000000
1 b 2 4 5.0 False b 2013-01-02 2013-01-02 00:00:00-05:00 2013-01-01 00:00:00.000000001
2 c 3 5 6.0 True c 2013-01-03 2013-01-03 00:00:00-05:00 2013-01-01 00:00:00.000000002

In [504]: df.dtypes
Out[504]:
a object
b int64
c uint8
d float64
e bool
f category
g datetime64[ns]
h datetime64[ns, US/Eastern]
i datetime64[ns]
dtype: object

```

Write to a feather file.

```
In [505]: df.to_feather('example.feather')
```

Read from a feather file.



```

In [506]: result = pd.read_feather('example.feather')

In [507]: result
Out[507]:
 a b c d e f g h i
0 a 1 3 4.0 True a 2013-01-01 2013-01-01 00:00:00-05:00 2013-01-01 00:00:00.000000000
1 b 2 4 5.0 False b 2013-01-02 2013-01-02 00:00:00-05:00 2013-01-01 00:00:00.000000001
2 c 3 5 6.0 True c 2013-01-03 2013-01-03 00:00:00-05:00 2013-01-01 00:00:00.000000002

we preserve dtypes
In [508]: result.dtypes
Out[508]:
a object
b int64
c uint8
d float64
e bool
f category
g datetime64[ns]
h datetime64[ns, US/Eastern]
i datetime64[ns]
dtype: object

```

## Parquet

New in version 0.21.0.

[Apache Parquet](#) provides a partitioned binary columnar serialization for data frames. It is designed to make reading and writing data frames efficient, and to make sharing data across data analysis languages easy. Parquet can use a variety of compression techniques to shrink the file size as much as possible while still maintaining good read performance.

Parquet is designed to faithfully serialize and de-serialize `DataFrame` s, supporting all of the pandas dtypes, including extension dtypes such as datetime with tz.

Several caveats.

- Duplicate column names and non-string columns names are not supported.
- The `pyarrow` engine always writes the index to the output, but `fastparquet` only writes non-default indexes. This extra column can cause problems for non-Pandas consumers that are not expecting it. You can force including or omitting indexes with the `index` argument, regardless of the underlying engine.
- Index level names, if specified, must be strings.
- Categorical dtypes can be serialized to parquet, but will de-serialize as `object` dtype.
- Non supported types include `Period` and actual Python object types. These will raise a helpful error message on an attempt at serialization.

You can specify an `engine` to direct the serialization. This can be one of `pyarrow`, or `fastparquet`, or `auto`. If the engine is NOT specified, then the `pd.options.io.parquet.engine` option is checked; if this is also `auto`, then `pyarrow` is tried, and falling back to `fastparquet`.

See the documentation for [pyarrow](#) and [fastparquet](#).

⋮ tip Note

These engines are very similar and should read/write nearly identical parquet format files. Currently `pyarrow` does not support timedelta data, `fastparquet` >= 0.1.4 supports timezone aware datetimes. These libraries differ by having different underlying dependencies (`fastparquet` by using `numba`, while `pyarrow` uses a c-library).

⋮

```
In [509]: df = pd.DataFrame({'a': list('abc'),
.....: 'b': list(range(1, 4)),
.....: 'c': np.arange(3, 6).astype('u1'),
.....: 'd': np.arange(4.0, 7.0, dtype='float64'),
.....: 'e': [True, False, True],
.....: 'f': pd.date_range('20130101', periods=3),
.....: 'g': pd.date_range('20130101', periods=3, tz='US/Eastern')})
.....:
```

```
In [510]: df
```

```
Out[510]:
```

	a	b	c	d	e	f	g
0	a	1	3	4.0	True	2013-01-01	2013-01-01 00:00:00-05:00
1	b	2	4	5.0	False	2013-01-02	2013-01-02 00:00:00-05:00
2	c	3	5	6.0	True	2013-01-03	2013-01-03 00:00:00-05:00

```
In [511]: df.dtypes
```

```
Out[511]:
```

a	object
b	int64
c	uint8
d	float64
e	bool
f	datetime64[ns]
g	datetime64[ns, US/Eastern]

```
dtype: object
```

Write to a parquet file.

```
In [512]: df.to_parquet('example_pa.parquet', engine='pyarrow')
```

```
In [513]: df.to_parquet('example_fp.parquet', engine='fastparquet')
```

Read from a parquet file.

```

In [514]: result = pd.read_parquet('example_fp.parquet', engine='fastparquet')

In [515]: result = pd.read_parquet('example_pa.parquet', engine='pyarrow')

In [516]: result.dtypes
Out[516]:
a object
b int64
c uint8
d float64
e bool
f datetime64[ns]
g datetime64[ns, US/Eastern]
dtype: object

```

Read only certain columns of a parquet file.

```

In [517]: result = pd.read_parquet('example_fp.parquet',
.....: engine='fastparquet', columns=['a', 'b'])
.....:

In [518]: result = pd.read_parquet('example_pa.parquet',
.....: engine='pyarrow', columns=['a', 'b'])
.....:

In [519]: result.dtypes
Out[519]:
a object
b int64
dtype: object

```

## Handling indexes

Serializing a `DataFrame` to parquet may include the implicit index as one or more columns in the output file. Thus, this code:

```

In [520]: df = pd.DataFrame({'a': [1, 2], 'b': [3, 4]})

In [521]: df.to_parquet('test.parquet', engine='pyarrow')

```

creates a parquet file with three columns if you use `pyarrow` for serialization: `a`, `b`, and `__index_level_0__`. If you're using `fastparquet`, the index [may or may not](#) be written to the file.

This unexpected extra column causes some databases like Amazon Redshift to reject the file, because that column doesn't exist in the target table.

If you want to omit a dataframe's indexes when writing, pass `index=False` to `to_parquet()`:

```

In [522]: df.to_parquet('test.parquet', index=False)

```

This creates a parquet file with just the two expected columns, `a` and `b`. If your `DataFrame` has a custom index, you won't get it back when you load this file into a `DataFrame`.

Passing `index=True` will always write the index, even if that's not the underlying engine's default behavior.

## Partitioning Parquet files

New in version 0.24.0.

Parquet supports partitioning of data based on the values of one or more columns.

```
In [523]: df = pd.DataFrame({'a': [0, 0, 1, 1], 'b': [0, 1, 0, 1]})

In [524]: df.to_parquet(fname='test', engine='pyarrow',
.....: partition_cols=['a'], compression=None)
.....:
```

The `fname` specifies the parent directory to which data will be saved. The `partition_cols` are the column names by which the dataset will be partitioned. Columns are partitioned in the order they are given. The partition splits are determined by the unique values in the partition columns. The above example creates a partitioned dataset that may look like:

```
test
├── a=0
│ ├── 0bac803e32dc42ae83fddfd029cbdebc.parquet
│ └── ...
└── a=1
 ├── e6ab24a4f45147b49b54a662f0c412a3.parquet
 └── ...
```

## SQL queries

The `pandas.io.sql` module provides a collection of query wrappers to both facilitate data retrieval and to reduce dependency on DB-specific API. Database abstraction is provided by `SQLAlchemy` if installed. In addition you will need a driver library for your database. Examples of such drivers are [psycopg2](#) for PostgreSQL or [pymysql](#) for MySQL. For [SQLite](#) this is included in Python's standard library by default. You can find an overview of supported drivers for each SQL dialect in the [SQLAlchemy docs](#).

If `SQLAlchemy` is not installed, a fallback is only provided for `sqlite` (and for `mysql` for backwards compatibility, but this is deprecated and will be removed in a future version). This mode requires a Python database adapter which respect the [Python DB-API](#).

See also some [cookbook examples](#) for some advanced strategies.

The key functions are:

Method	Description
<code>read_sql_table(table_name, con[, schema, ...])</code>	Read SQL database table into a DataFrame.
<code>read_sql_query(sql, con[, index_col, ...])</code>	Read SQL query into a DataFrame.
<code>read_sql(sql, con[, index_col, ...])</code>	Read SQL query or database table into a DataFrame.
<code>DataFrame.to_sql(self, name, con[, schema, ...])</code>	Write records stored in a DataFrame to a SQL database.

::: tip Note

The function `read_sql()` is a convenience wrapper around `read_sql_table()` and `read_sql_query()` (and for backward compatibility) and will delegate to specific function depending on the provided input (database table name or sql query). Table names do not need to be quoted if they have special characters.

:::

In the following example, we use the [SQLite](#) SQL database engine. You can use a temporary SQLite database where data are stored in “memory” .

To connect with SQLAlchemy you use the `create_engine()` function to create an engine object from database URI. You only need to create the engine once per database you are connecting to. For more information on `create_engine()` and the URI formatting, see the examples below and the [SQLAlchemy documentation](#)

```
In [525]: from sqlalchemy import create_engine

Create your engine.
In [526]: engine = create_engine('sqlite:///memory:')
```

If you want to manage your own connections you can pass one of those instead:

```
with engine.connect() as conn, conn.begin():
 data = pd.read_sql_table('data', conn)
```

## Writing DataFrames

Assuming the following data is in a DataFrame `data` , we can insert it into the database using `to_sql()`.

id	Date	Col_1	Col_2	Col_3
26	2012-10-18	X	25.7	True
42	2012-10-19	Y	-12.4	False
63	2012-10-20	Z	5.73	True

```
In [527]: data
Out[527]:
 id Date Col_1 Col_2 Col_3
0 26 2010-10-18 X 27.50 True
1 42 2010-10-19 Y -12.50 False
2 63 2010-10-20 Z 5.73 True

In [528]: data.to_sql('data', engine)
```

With some databases, writing large DataFrames can result in errors due to packet size limitations being exceeded. This can be avoided by setting the `chunksize` parameter when calling `to_sql`. For example, the following writes `data` to the database in batches of 1000 rows at a time:

```
In [529]: data.to_sql('data_chunked', engine, chunksize=1000)
```

## SQL data types

`to_sql()` will try to map your data to an appropriate SQL data type based on the dtype of the data. When you have columns of dtype `object`, pandas will try to infer the data type.

You can always override the default type by specifying the desired SQL type of any of the columns by using the `dtype` argument. This argument needs a dictionary mapping column names to SQLAlchemy types (or strings for the sqlite3 fallback mode). For example, specifying to use the sqlalchemy `String` type instead of the default `Text` type for string columns:

```
In [530]: from sqlalchemy.types import String

In [531]: data.to_sql('data_dtype', engine, dtype={'Col_1': String})
```

::: tip Note

Due to the limited support for `timedelta`'s in the different database flavors, columns with type `timedelta64` will be written as integer values as nanoseconds to the database and a warning will be raised.

:::

::: tip Note

Columns of `category` dtype will be converted to the dense representation as you would get with `np.asarray(categorical)` (e.g. for string categories this gives an array of strings). Because of this, reading the database table back in does **not** generate a categorical.

:::

## Datetime data types

Using SQLAlchemy, `to_sql()` is capable of writing datetime data that is timezone naive or timezone aware. However, the resulting data stored in the database ultimately depends on the supported data type for datetime data of the database system being used.

The following table lists supported data types for datetime data for some common databases. Other database dialects may have different data types for datetime data.

Database	SQL Datetime Types	Timezone Support
SQLite	TEXT	No
MySQL	TIMESTAMP or DATETIME	No
PostgreSQL	TIMESTAMP or TIMESTAMP WITH TIME ZONE	Yes

When writing timezone aware data to databases that do not support timezones, the data will be written as timezone naive timestamps that are in local time with respect to the timezone.

`read_sql_table()` is also capable of reading datetime data that is timezone aware or naive. When reading `TIMESTAMP WITH TIME ZONE` types, pandas will convert the data to UTC.

## Insertion method

New in version 0.24.0.

The parameter `method` controls the SQL insertion clause used. Possible values are:

- `None` : Uses standard SQL `INSERT` clause (one per row).
- `'multi'` : Pass multiple values in a single `INSERT` clause. It uses a special SQL syntax not supported by all backends. This usually provides better performance for analytic databases like Presto and Redshift, but has worse performance for traditional SQL backend if the table contains many columns. For more information check the SQLAlchemy [documentation](#).
- callable with signature `(pd_table, conn, keys, data_iter)` : This can be used to implement a more performant insertion method based on specific backend dialect features.

Example of a callable using PostgreSQL [COPY clause](#):

```
Alternative to_sql() *method* for DBs that support COPY FROM
import csv
from io import StringIO

def psql_insert_copy(table, conn, keys, data_iter):
 # gets a DBAPI connection that can provide a cursor
 dbapi_conn = conn.connection
 with dbapi_conn.cursor() as cur:
 s_buf = StringIO()
 writer = csv.writer(s_buf)
 writer.writerows(data_iter)
 s_buf.seek(0)

 columns = ', '.join("{} {}".format(k, t) for k, t in keys)
 if table.schema:
 table_name = '{}.{}'.format(table.schema, table.name)
 else:
 table_name = table.name

 sql = 'COPY {} ({} FROM STDIN WITH CSV'.format(
 table_name, columns)
 cur.copy_expert(sql=sql, file=s_buf)
```

## Reading tables

`read_sql_table()` will read a database table given the table name and optionally a subset of columns to read.

::: tip Note

In order to use `read_sql_table()`, you **must** have the SQLAlchemy optional dependency installed.

:::

```
In [532]: pd.read_sql_table('data', engine)
Out[532]:
```

	index	id	Date	Col_1	Col_2	Col_3
0	0	26	2010-10-18	X	27.50	True
1	1	42	2010-10-19	Y	-12.50	False
2	2	63	2010-10-20	Z	5.73	True

You can also specify the name of the column as the `DataFrame` index, and specify a subset of columns to be read.



```
In [533]: pd.read_sql_table('data', engine, index_col='id')
Out[533]:
```

	index	Date	Col_1	Col_2	Col_3
id					
26	0	2010-10-18	X	27.50	True
42	1	2010-10-19	Y	-12.50	False
63	2	2010-10-20	Z	5.73	True

```
In [534]: pd.read_sql_table('data', engine, columns=['Col_1', 'Col_2'])
Out[534]:
```

	Col_1	Col_2
0	X	27.50
1	Y	-12.50
2	Z	5.73

And you can explicitly force columns to be parsed as dates:

```
In [535]: pd.read_sql_table('data', engine, parse_dates=['Date'])
Out[535]:
```

	index	id	Date	Col_1	Col_2	Col_3
0	0	26	2010-10-18	X	27.50	True
1	1	42	2010-10-19	Y	-12.50	False
2	2	63	2010-10-20	Z	5.73	True

If needed you can explicitly specify a format string, or a dict of arguments to pass to `pandas.to_datetime()`:

```
pd.read_sql_table('data', engine, parse_dates={'Date': '%Y-%m-%d'})
pd.read_sql_table('data', engine,
 parse_dates={'Date': {'format': '%Y-%m-%d %H:%M:%S'}})
```

You can check if a table exists using `has_table()`

## Schema support

Reading from and writing to different schema's is supported through the `schema` keyword in the `read_sql_table()` and `to_sql()` functions. Note however that this depends on the database flavor (sqlite does not have schema's). For example:

```
df.to_sql('table', engine, schema='other_schema')
pd.read_sql_table('table', engine, schema='other_schema')
```

## Querying

You can query using raw SQL in the `read_sql_query()` function. In this case you must use the SQL variant appropriate for your database. When using SQLAlchemy, you can also pass SQLAlchemy Expression language constructs, which are database-agnostic.

```
In [536]: pd.read_sql_query('SELECT * FROM data', engine)
Out[536]:
```

	index	id	Date	Col_1	Col_2	Col_3
0	0	26	2010-10-18 00:00:00.000000	X	27.50	1
1	1	42	2010-10-19 00:00:00.000000	Y	-12.50	0
2	2	63	2010-10-20 00:00:00.000000	Z	5.73	1

Of course, you can specify a more “complex” query.

```
In [537]: pd.read_sql_query("SELECT id, Col_1, Col_2 FROM data WHERE id = 42;", engine)
Out[537]:
```

	id	Col_1	Col_2
0	42	Y	-12.5

The `read_sql_query()` function supports a `chunksize` argument. Specifying this will return an iterator through chunks of the query result:

```
In [538]: df = pd.DataFrame(np.random.randn(20, 3), columns=list('abc'))

In [539]: df.to_sql('data_chunks', engine, index=False)
```

```
In [540]: for chunk in pd.read_sql_query("SELECT * FROM data_chunks",
.....: engine, chunksize=5):
.....: print(chunk)
.....:
```

	a	b	c
0	-0.900850	-0.323746	0.037100
1	0.057533	-0.032842	0.550902
2	1.026623	1.035455	-0.965140
3	-0.252405	-1.255987	0.639156
4	1.076701	-0.309155	-0.800182

	a	b	c
0	-0.206623	0.496077	-0.219935
1	0.631362	-1.166743	1.808368
2	0.023531	0.987573	0.471400
3	-0.982250	-0.192482	1.195452
4	-1.758855	0.477551	1.412567

	a	b	c
0	-1.120570	1.232764	0.417814
1	1.688089	-0.037645	-0.269582
2	0.646823	-0.603366	1.592966
3	0.724019	-0.515606	-0.180920
4	0.038244	-2.292866	-0.114634

	a	b	c
0	-0.970230	-0.963257	-0.128304
1	0.498621	-1.496506	0.701471
2	-0.272608	-0.119424	-0.882023
3	-0.253477	0.714395	0.664179
4	0.897140	0.455791	1.549590

You can also run a plain query without creating a `DataFrame` with `execute()`. This is useful for queries that don't return values, such as `INSERT`. This is functionally equivalent to calling `execute` on the SQLAlchemy engine or db connection object. Again, you must use the SQL syntax variant appropriate for your database.

```
from pandas.io import sql
sql.execute('SELECT * FROM table_name', engine)
sql.execute('INSERT INTO table_name VALUES(?, ?, ?)', engine,
 params=[('id', 1, 12.2, True)])
```

## Engine connection examples

To connect with SQLAlchemy you use the `create_engine()` function to create an engine object from database URI. You only need to create the engine once per database you are connecting to.

```
from sqlalchemy import create_engine

engine = create_engine('postgresql://scott:tiger@localhost:5432/mydatabase')

engine = create_engine('mysql+mysqldb://scott:tiger@localhost/foo')

engine = create_engine('oracle://scott:tiger@127.0.0.1:1521/sidname')

engine = create_engine('mssql+pyodbc://mydsn')

sqlite://<nohostname>/<path>
where <path> is relative:
engine = create_engine('sqlite:///foo.db')

or absolute, starting with a slash:
engine = create_engine('sqlite:///absolute/path/to/foo.db')
```

For more information see the examples the SQLAlchemy [documentation](#)

## Advanced SQLAlchemy queries

You can use SQLAlchemy constructs to describe your query.

Use `sqlalchemy.text()` to specify query parameters in a backend-neutral way

```
In [541]: import sqlalchemy as sa

In [542]: pd.read_sql(sa.text('SELECT * FROM data where Col_1=:col1'),
.....: engine, params={'col1': 'X'})
.....:
Out[542]:
 index id Date Col_1 Col_2 Col_3
0 0 26 2010-10-18 00:00:00.000000 X 27.5 1
```

If you have an SQLAlchemy description of your database you can express where conditions using SQLAlchemy expressions

```
In [543]: metadata = sa.MetaData()

In [544]: data_table = sa.Table('data', metadata,
.....: sa.Column('index', sa.Integer),
.....: sa.Column('Date', sa.DateTime),
.....: sa.Column('Col_1', sa.String),
.....: sa.Column('Col_2', sa.Float),
.....: sa.Column('Col_3', sa.Boolean),
.....:)
.....:

In [545]: pd.read_sql(sa.select([data_table]).where(data_table.c.Col_3 is True), engine)
Out[545]:
Empty DataFrame
Columns: [index, Date, Col_1, Col_2, Col_3]
Index: []
```

You can combine SQLAlchemy expressions with parameters passed to `read_sql()` using `sqlalchemy.bindparam()`

```
In [546]: import datetime as dt

In [547]: expr = sa.select([data_table]).where(data_table.c.Date > sa.bindparam('date'))

In [548]: pd.read_sql(expr, engine, params={'date': dt.datetime(2010, 10, 18)})
Out[548]:
 index Date Col_1 Col_2 Col_3
0 1 2010-10-19 Y -12.50 False
1 2 2010-10-20 Z 5.73 True
```

## Sqlite fallback

The use of sqlite is supported without using SQLAlchemy. This mode requires a Python database adapter which respect the [Python DB-API](#).

You can create connections like so:

```
import sqlite3
con = sqlite3.connect(':memory:')
```

And then issue the following queries:

```
data.to_sql('data', con)
pd.read_sql_query("SELECT * FROM data", con)
```

## Google BigQuery

---

::: danger Warning

Starting in 0.20.0, pandas has split off Google BigQuery support into the separate package `pandas-gbq`. You can `pip install pandas-gbq` to get it.

:::

The `pandas-gbq` package provides functionality to read/write from Google BigQuery.

pandas integrates with this external package. if `pandas-gbq` is installed, you can use the pandas methods `pd.read_gbq` and `DataFrame.to_gbq`, which will call the respective functions from `pandas-gbq`.

Full documentation can be found [here](#).

## Stata format

---

### Writing to stata format

The method `to_stata()` will write a DataFrame into a `.dta` file. The format version of this file is always 115 (Stata 12).

```
In [549]: df = pd.DataFrame(np.random.randn(10, 2), columns=list('AB'))

In [550]: df.to_stata('stata.dta')
```

Stata data files have limited data type support; only strings with 244 or fewer characters, `int8`, `int16`, `int32`, `float32` and `float64` can be stored in `.dta` files. Additionally, Stata reserves certain values to represent missing data. Exporting a non-missing value that is outside of the permitted range in Stata for a particular data type will retype the variable to the next larger size. For example, `int8` values are restricted to lie between -127 and 100 in Stata, and so variables with values above 100 will trigger a conversion to `int16`. `nan` values in floating points data types are stored as the basic missing data type (`.` in Stata).

::: tip Note

It is not possible to export missing data values for integer data types.

:::

The Stata writer gracefully handles other data types including `int64`, `bool`, `uint8`, `uint16`, `uint32` by casting to the smallest supported type that can represent the data. For example, data with a type of `uint8` will be cast to `int8` if all values are less than 100 (the upper bound for non-missing `int8` data in Stata), or, if values are outside of this range, the variable is cast to `int16`.

::: danger Warning

Conversion from `int64` to `float64` may result in a loss of precision if `int64` values are larger than  $2^{53}$ .

:::

::: danger Warning

`StataWriter` and `to_stata()` only support fixed width strings containing up to 244 characters, a limitation imposed by the version 115 dta file format. Attempting to write Stata dta files with strings longer than 244 characters raises a `ValueError`.

:::

## Reading from Stata format

The top-level function `read_stata` will read a dta file and return either a `DataFrame` or a `StataReader` that can be used to read the file incrementally.

```
In [551]: pd.read_stata('stata.dta')
```

```
Out[551]:
```

	index	A	B
0	0	1.031231	0.196447
1	1	0.190188	0.619078
2	2	0.036658	-0.100501
3	3	0.201772	1.763002
4	4	0.454977	-1.958922
5	5	-0.628529	0.133171
6	6	-1.274374	2.518925
7	7	-0.517547	-0.360773
8	8	0.877961	-1.881598
9	9	-0.699067	-1.566913

Specifying a `chunksize` yields a `StataReader` instance that can be used to read `chunksize` lines from the file at a time. The `StataReader` object can be used as an iterator.

```
In [552]: reader = pd.read_stata('stata.dta', chunksize=3)
```

```
In [553]: for df in reader:
```

```
.....: print(df.shape)
```

```
.....:
```

```
(3, 3)
```

```
(3, 3)
```

```
(3, 3)
```

```
(1, 3)
```

For more fine-grained control, use `iterator=True` and specify `chunksize` with each call to `read()`.

```
In [554]: reader = pd.read_stata('stata.dta', iterator=True)

In [555]: chunk1 = reader.read(5)

In [556]: chunk2 = reader.read(5)
```

Currently the `index` is retrieved as a column.

The parameter `convert_categoricals` indicates whether value labels should be read and used to create a `Categorical` variable from them. Value labels can also be retrieved by the function `value_labels`, which requires `read()` to be called before use.

The parameter `convert_missing` indicates whether missing value representations in Stata should be preserved. If `False` (the default), missing values are represented as `np.nan`. If `True`, missing values are represented using `StataMissingValue` objects, and columns containing missing values will have `object` data type.

::: tip Note

`read_stata()` and `StataReader` support .dta formats 113-115 (Stata 10-12), 117 (Stata 13), and 118 (Stata 14).

:::

::: tip Note

Setting `preserve_dtypes=False` will upcast to the standard pandas data types: `int64` for all integer types and `float64` for floating point data. By default, the Stata data types are preserved when importing.

:::

## Categorical data

`Categorical` data can be exported to Stata data files as value labeled data. The exported data consists of the underlying category codes as integer data values and the categories as value labels. Stata does not have an explicit equivalent to a `Categorical` and information about whether the variable is ordered is lost when exporting.

::: danger Warning

Stata only supports string value labels, and so `str` is called on the categories when exporting data. Exporting `Categorical` variables with non-string categories produces a warning, and can result a loss of information if the `str` representations of the categories are not unique.

:::

Labeled data can similarly be imported from Stata data files as `Categorical` variables using the keyword argument `convert_categoricals` ( `True` by default). The keyword argument `order_categoricals` ( `True` by default) determines whether imported `Categorical` variables are ordered.

::: tip Note

When importing categorical data, the values of the variables in the Stata data file are not preserved since `Categorical` variables always use integer data types between `-1` and `n-1` where `n` is the number of categories. If the original values in the Stata data file are required, these can be imported by setting `convert_categoricals=False`, which will import original data (but not the variable labels). The original values can be matched to the imported categorical data since there is a simple mapping between the original Stata data values and the category codes of imported `Categorical` variables: missing values are assigned code `-1`, and the smallest original value is assigned `0`, the second smallest is assigned `1` and so on until the largest original value is assigned the code `n-1`.

:::

::: tip Note

Stata supports partially labeled series. These series have value labels for some but not all data values. Importing a partially labeled series will produce a `Categorical` with string categories for the values that are labeled and numeric categories for values with no label.

:::

## SAS formats

---

The top-level function `read_sas()` can read (but not write) SAS xport (.XPT) and (since v0.18.0) SAS7BDAT (.sas7bdat) format files.

SAS files only contain two value types: ASCII text and floating point values (usually 8 bytes but sometimes truncated). For xport files, there is no automatic type conversion to integers, dates, or categoricals. For SAS7BDAT files, the format codes may allow date variables to be automatically converted to dates. By default the whole file is read and returned as a `DataFrame`.

Specify a `chunksize` or use `iterator=True` to obtain reader objects ( `XportReader` or `SAS7BDATReader` ) for incrementally reading the file. The reader objects also have attributes that contain additional information about the file and its variables.

Read a SAS7BDAT file:

```
df = pd.read_sas('sas_data.sas7bdat')
```

Obtain an iterator and read an XPORT file 100,000 lines at a time:



```
def do_something(chunk):
 pass

rdr = pd.read_sas('sas_xport.xpt', chunk=100000)
for chunk in rdr:
 do_something(chunk)
```

The [specification](#) for the xport file format is available from the SAS web site.

No official documentation is available for the SAS7BDAT format.

## Other file formats

---

pandas itself only supports IO with a limited set of file formats that map cleanly to its tabular data model. For reading and writing other file formats into and from pandas, we recommend these packages from the broader community.

### netCDF

[xarray](#) provides data structures inspired by the pandas `DataFrame` for working with multi-dimensional datasets, with a focus on the netCDF file format and easy conversion to and from pandas.

## Performance considerations

---

This is an informal comparison of various IO methods, using pandas 0.20.3. Timings are machine dependent and small differences should be ignored.

```
In [1]: sz = 1000000
In [2]: df = pd.DataFrame({'A': np.random.randn(sz), 'B': [1] * sz})

In [3]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 2 columns):
A 1000000 non-null float64
B 1000000 non-null int64
dtypes: float64(1), int64(1)
memory usage: 15.3 MB
```

Given the next test set:

```

from numpy.random import randn

sz = 1000000
df = pd.DataFrame({'A': randn(sz), 'B': [1] * sz})

def test_sql_write(df):
 if os.path.exists('test.sql'):
 os.remove('test.sql')
 sql_db = sqlite3.connect('test.sql')
 df.to_sql(name='test_table', con=sql_db)
 sql_db.close()

def test_sql_read():
 sql_db = sqlite3.connect('test.sql')
 pd.read_sql_query("select * from test_table", sql_db)
 sql_db.close()

def test_hdf_fixed_write(df):
 df.to_hdf('test_fixed.hdf', 'test', mode='w')

def test_hdf_fixed_read():
 pd.read_hdf('test_fixed.hdf', 'test')

def test_hdf_fixed_write_compress(df):
 df.to_hdf('test_fixed_compress.hdf', 'test', mode='w', complib='blosc')

def test_hdf_fixed_read_compress():
 pd.read_hdf('test_fixed_compress.hdf', 'test')

def test_hdf_table_write(df):
 df.to_hdf('test_table.hdf', 'test', mode='w', format='table')

def test_hdf_table_read():
 pd.read_hdf('test_table.hdf', 'test')

def test_hdf_table_write_compress(df):
 df.to_hdf('test_table_compress.hdf', 'test', mode='w',
 complib='blosc', format='table')

def test_hdf_table_read_compress():
 pd.read_hdf('test_table_compress.hdf', 'test')

def test_csv_write(df):
 df.to_csv('test.csv', mode='w')

```

```
def test_csv_read():
 pd.read_csv('test.csv', index_col=0)

def test_feather_write(df):
 df.to_feather('test.feather')

def test_feather_read():
 pd.read_feather('test.feather')

def test_pickle_write(df):
 df.to_pickle('test.pkl')

def test_pickle_read():
 pd.read_pickle('test.pkl')

def test_pickle_write_compress(df):
 df.to_pickle('test.pkl.compress', compression='xz')

def test_pickle_read_compress():
 pd.read_pickle('test.pkl.compress', compression='xz')
```

When writing, the top-three functions in terms of speed are `test_pickle_write`, `test_feather_write` and `test_hdf_fixed_write_compress`.

```
In [14]: %timeit test_sql_write(df)
2.37 s ± 36.6 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

In [15]: %timeit test_hdf_fixed_write(df)
194 ms ± 65.9 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [26]: %timeit test_hdf_fixed_write_compress(df)
119 ms ± 2.15 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [16]: %timeit test_hdf_table_write(df)
623 ms ± 125 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

In [27]: %timeit test_hdf_table_write_compress(df)
563 ms ± 23.7 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

In [17]: %timeit test_csv_write(df)
3.13 s ± 49.9 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

In [30]: %timeit test_feather_write(df)
103 ms ± 5.88 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [31]: %timeit test_pickle_write(df)
109 ms ± 3.72 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [32]: %timeit test_pickle_write_compress(df)
3.33 s ± 55.2 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

When reading, the top three are `test_feather_read`, `test_pickle_read` and `test_hdf_fixed_read`.

```
In [18]: %timeit test_sql_read()
1.35 s ± 14.7 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

In [19]: %timeit test_hdf_fixed_read()
14.3 ms ± 438 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

In [28]: %timeit test_hdf_fixed_read_compress()
23.5 ms ± 672 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [20]: %timeit test_hdf_table_read()
35.4 ms ± 314 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [29]: %timeit test_hdf_table_read_compress()
42.6 ms ± 2.1 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [22]: %timeit test_csv_read()
516 ms ± 27.1 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

In [33]: %timeit test_feather_read()
4.06 ms ± 115 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

In [34]: %timeit test_pickle_read()
6.5 ms ± 172 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

In [35]: %timeit test_pickle_read_compress()
588 ms ± 3.57 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Space on disk (in bytes)

```
34816000 Aug 21 18:00 test.sql
24009240 Aug 21 18:00 test_fixed.hdf
 7919610 Aug 21 18:00 test_fixed_compress.hdf
24458892 Aug 21 18:00 test_table.hdf
 8657116 Aug 21 18:00 test_table_compress.hdf
28520770 Aug 21 18:00 test.csv
16000248 Aug 21 18:00 test.feather
16000848 Aug 21 18:00 test.pkl
 7554108 Aug 21 18:00 test.pkl.compress
```

---

## 索引和数据选择器

Pandas对象中的轴标记信息有多种用途：

- 使用已知指标识别数据（即提供元数据），这对于分析，可视化和交互式控制台显示非常重要。
- 启用自动和显式数据对齐。
- 允许直观地获取和设置数据集的子集。

在本节中，我们将重点关注最后一点：即如何切片，切块，以及通常获取和设置pandas对象的子集。主要关注的是Series和DataFrame，因为他们在这个领域受到了更多的开发关注。

::: tip 注意

Python和NumPy索引运算符 `[]` 和属性运算符 `.` 可以在各种用例中快速轻松地访问pandas数据结构。这使得交互式工作变得直观，因为如果您已经知道如何处理Python字典和NumPy数组，那么几乎没有新的东西需要学习。但是，由于预先不知道要访问的数据类型，因此直接使用标准运算符会有一些优化限制。对于生产代码，我们建议您利用本章中介绍的优化的pandas数据访问方法。

...

... danger 警告

是否为设置操作返回副本或引用可能取决于上下文。这有时被称为应该避免。请参阅[返回视图与复制](#)。

chained assignment

...

... danger 警告

使用浮点数对基于整数的索引进行索引已在0.18.0中进行了说明，有关更改的摘要，请参见[此处](#)。

...

见[多指标/高级索引](#)的 MultiIndex 和更先进的索引文件。

有关一些高级策略，请参阅[食谱](#)。

## 索引的不同选择

对象选择已经有许多用户请求的添加，以支持更明确的基于位置的索引。Pandas现在支持三种类型的多轴索引。

- `.loc` 主要是基于标签的，但也可以与布尔数组一起使用。当找不到物品时 `.loc` 会提高 `KeyError`。允许的输入是：
  - 单个标签，例如 `5` 或 `'a'`（注意，它 `5` 被解释为索引的 标签。此用法**不是**索引的整数位置。）。
  - 列表或标签数组。 `['a', 'b', 'c']`
  - 带标签的切片对象 `'a':'f'`（注意，相反普通的Python片，**都**开始和停止都包括在内，当存在于索引中！见[有标签切片](#)和[端点都包括在内](#)。）
  - 布尔数组
  - 一个 callable 带有一个参数的函数（调用Series或DataFrame）并返回有效的索引输出（上面的一个）。

版本0.18.1中的新功能。

在[标签选择](#)中查看更多信息。

- `.iloc` 是基于主要的整数位置（从 0 到 `length-1` 所述轴的），但也可以用布尔阵列使用。如果请求的索引器超出范围，`.iloc` 则会引发 `IndexError`，但允许越界索引的切片索引器除外。（这符合 Python / NumPy 切片 语义）。允许的输入是：
  - 一个整数，例如 5。
  - 整数列表或数组。 `[4, 3, 0]`
  - 带有整数的切片对象 `1:7`。
  - 布尔数组。
  - 一个 callable 带有一个参数的函数（调用 `Series` 或 `DataFrame`）并返回有效的索引输出（上面的一个）。

版本 0.18.1 中的新功能。

有关详细信息，请参阅[按位置选择](#)，[高级索引](#)和[高级层次结构](#)。

- `.loc`，`.iloc` 以及 `[]` 索引也可以接受一个 callable 索引器。在[Select By Callable](#)中查看更多信息。

从具有多轴选择的对象获取值使用以下表示法（使用 `.loc` 作为示例，但以下也适用 `.iloc`）。任何轴访问器可以是空切片 `:`。假设超出规范的轴是 `:`，例如 `p.loc['a']` 相当于 `p.loc['a', :, :]`

对象类型	索引
系列	<code>s.loc[indexer]</code>
数据帧	<code>df.loc[row_indexer,column_indexer]</code>

## 基础知识

正如在[上一节中](#)介绍数据结构时所提到的，索引的主要功能 `[]`（也就是 `__getitem__` 那些熟悉在 Python 中实现类行为的人）是选择低维切片。下表显示了使用以下方法索引 pandas 对象时的返回类型值 `[]`：

对象类型	选择	返回值类型
系列	<code>series[label]</code>	标量值
数据帧	<code>frame[colname]</code>	<code>Series</code> 对应于 <code>colname</code>

在这里，我们构建一个简单的时间序列数据集，用于说明索引功能：

```
In [1]: dates = pd.date_range('1/1/2000', periods=8)

In [2]: df = pd.DataFrame(np.random.randn(8, 4),
...: index=dates, columns=['A', 'B', 'C', 'D'])
...:

In [3]: df
Out[3]:
```

	A	B	C	D
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632
2000-01-02	1.212112	-0.173215	0.119209	-1.044236
2000-01-03	-0.861849	-2.104569	-0.494929	1.071804
2000-01-04	0.721555	-0.706771	-1.039575	0.271860
2000-01-05	-0.424972	0.567020	0.276232	-1.087401
2000-01-06	-0.673690	0.113648	-1.478427	0.524988
2000-01-07	0.404705	0.577046	-1.715002	-1.039268
2000-01-08	-0.370647	-1.157892	-1.344312	0.844885

∴ tip 注意

除非特别说明，否则索引功能都不是时间序列特定的。

∴

因此，如上所述，我们使用最基本的索引 []：

```
In [4]: s = df['A']

In [5]: s[dates[5]]
Out[5]: -0.6736897080883706
```

您可以传递列表列表 [] 以按该顺序选择列。如果DataFrame中未包含列，则会引发异常。也可以这种方式设置多列：



```
In [6]: df
Out[6]:
```

	A	B	C	D
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632
2000-01-02	1.212112	-0.173215	0.119209	-1.044236
2000-01-03	-0.861849	-2.104569	-0.494929	1.071804
2000-01-04	0.721555	-0.706771	-1.039575	0.271860
2000-01-05	-0.424972	0.567020	0.276232	-1.087401
2000-01-06	-0.673690	0.113648	-1.478427	0.524988
2000-01-07	0.404705	0.577046	-1.715002	-1.039268
2000-01-08	-0.370647	-1.157892	-1.344312	0.844885

```
In [7]: df[['B', 'A']] = df[['A', 'B']]
```

```
In [8]: df
Out[8]:
```

	A	B	C	D
2000-01-01	-0.282863	0.469112	-1.509059	-1.135632
2000-01-02	-0.173215	1.212112	0.119209	-1.044236
2000-01-03	-2.104569	-0.861849	-0.494929	1.071804
2000-01-04	-0.706771	0.721555	-1.039575	0.271860
2000-01-05	0.567020	-0.424972	0.276232	-1.087401
2000-01-06	0.113648	-0.673690	-1.478427	0.524988
2000-01-07	0.577046	0.404705	-1.715002	-1.039268
2000-01-08	-1.157892	-0.370647	-1.344312	0.844885

您可能会发现这对于将变换（就地）应用于列的子集非常有用。

::: danger 警告

pandas在设置 Series 和 DataFrame 来自 `.loc` 和时对齐所有AXES `.iloc`。

这**不会**修改，`df` 因为列对齐在赋值之前。

```
In [9]: df[['A', 'B']]
Out[9]:
```

	A	B
2000-01-01	-0.282863	0.469112
2000-01-02	-0.173215	1.212112
2000-01-03	-2.104569	-0.861849
2000-01-04	-0.706771	0.721555
2000-01-05	0.567020	-0.424972
2000-01-06	0.113648	-0.673690
2000-01-07	0.577046	0.404705
2000-01-08	-1.157892	-0.370647

```
In [10]: df.loc[:, ['B', 'A']] = df[['A', 'B']]

In [11]: df[['A', 'B']]
Out[11]:
```

	A	B
2000-01-01	-0.282863	0.469112
2000-01-02	-0.173215	1.212112
2000-01-03	-2.104569	-0.861849
2000-01-04	-0.706771	0.721555
2000-01-05	0.567020	-0.424972
2000-01-06	0.113648	-0.673690
2000-01-07	0.577046	0.404705
2000-01-08	-1.157892	-0.370647

交换列值的正确方法是使用原始值：

```
In [12]: df.loc[:, ['B', 'A']] = df[['A', 'B']].to_numpy()

In [13]: df[['A', 'B']]
Out[13]:
```

	A	B
2000-01-01	0.469112	-0.282863
2000-01-02	1.212112	-0.173215
2000-01-03	-0.861849	-2.104569
2000-01-04	0.721555	-0.706771
2000-01-05	-0.424972	0.567020
2000-01-06	-0.673690	0.113648
2000-01-07	0.404705	0.577046
2000-01-08	-0.370647	-1.157892

...

## 属性访问

您可以直接访问某个 Series 或列上的索引 DataFrame 作为属性：

```
In [14]: sa = pd.Series([1, 2, 3], index=list('abc'))

In [15]: dfa = df.copy()
```

```

In [16]: sa.b
Out[16]: 2

In [17]: dfa.A
Out[17]:
2000-01-01 0.469112
2000-01-02 1.212112
2000-01-03 -0.861849
2000-01-04 0.721555
2000-01-05 -0.424972
2000-01-06 -0.673690
2000-01-07 0.404705
2000-01-08 -0.370647
Freq: D, Name: A, dtype: float64

```

```

In [18]: sa.a = 5

In [19]: sa
Out[19]:
a 5
b 2
c 3
dtype: int64

```

```

In [20]: dfa.A = list(range(len(dfa.index))) # ok if A already exists

```

```

In [21]: dfa
Out[21]:
 A B C D
2000-01-01 0 -0.282863 -1.509059 -1.135632
2000-01-02 1 -0.173215 0.119209 -1.044236
2000-01-03 2 -2.104569 -0.494929 1.071804
2000-01-04 3 -0.706771 -1.039575 0.271860
2000-01-05 4 0.567020 0.276232 -1.087401
2000-01-06 5 0.113648 -1.478427 0.524988
2000-01-07 6 0.577046 -1.715002 -1.039268
2000-01-08 7 -1.157892 -1.344312 0.844885

```

```

In [22]: dfa['A'] = list(range(len(dfa.index))) # use this form to create a new column

```

```

In [23]: dfa
Out[23]:
 A B C D
2000-01-01 0 -0.282863 -1.509059 -1.135632
2000-01-02 1 -0.173215 0.119209 -1.044236
2000-01-03 2 -2.104569 -0.494929 1.071804
2000-01-04 3 -0.706771 -1.039575 0.271860
2000-01-05 4 0.567020 0.276232 -1.087401
2000-01-06 5 0.113648 -1.478427 0.524988
2000-01-07 6 0.577046 -1.715002 -1.039268
2000-01-08 7 -1.157892 -1.344312 0.844885

```

⋮ danger 警告

- 仅当index元素是有效的Python标识符时才可以使用此访问权限，例如 `s.1`，不允许。有关[有效标识符的说明](#)，请参见[此处](#)。
- 如果该属性与现有方法名称冲突，则该属性将不可用，例如 `s.min`，不允许。
- 同样的，如果它与任何下面的列表冲突的属性将不可用：`index`，`major_axis`，`minor_axis`，`items`。
- 在任何一种情况下，标准索引仍然可以工作，例如 `s['1']`，`s['min']` 和 `s['index']` 将访问相应的元素或列。

⋮

如果您使用的是IPython环境，则还可以使用tab-completion来查看这些可访问的属性。

您还可以将a分配 dict 给一行 DataFrame：

```
In [24]: x = pd.DataFrame({'x': [1, 2, 3], 'y': [3, 4, 5]})

In [25]: x.iloc[1] = {'x': 9, 'y': 99}

In [26]: x
Out[26]:
 x y
0 1 3
1 9 99
2 3 5
```

您可以使用属性访问来修改DataFrame的Series或列的现有元素，但要小心；如果您尝试使用属性访问权来创建新列，则会创建新属性而不是新列。在0.21.0及更高版本中，这将引发 `UserWarning`：

```
In [1]: df = pd.DataFrame({'one': [1., 2., 3.]})
In [2]: df.two = [4, 5, 6]
UserWarning: Pandas doesn't allow Series to be assigned into nonexistent columns - see https://pandas.pydata.org/pandas-docs/stable/indexing.html#attribute_access

In [3]: df
Out[3]:
 one
0 1.0
1 2.0
2 3.0
```

## 切片范围

沿着任意轴切割范围的最稳健和一致的方法在详细说明该方法的“[按位置选择](#)”部分中描述 `.iloc`。现在，我们解释使用 `[]` 运算符切片的语义。

使用Series，语法与ndarray完全一样，返回值的一部分和相应的标签：

```

In [27]: s[:5]
Out[27]:
2000-01-01 0.469112
2000-01-02 1.212112
2000-01-03 -0.861849
2000-01-04 0.721555
2000-01-05 -0.424972
Freq: D, Name: A, dtype: float64

In [28]: s[:2]
Out[28]:
2000-01-01 0.469112
2000-01-03 -0.861849
2000-01-05 -0.424972
2000-01-07 0.404705
Freq: 2D, Name: A, dtype: float64

In [29]: s[::-1]
Out[29]:
2000-01-08 -0.370647
2000-01-07 0.404705
2000-01-06 -0.673690
2000-01-05 -0.424972
2000-01-04 0.721555
2000-01-03 -0.861849
2000-01-02 1.212112
2000-01-01 0.469112
Freq: -1D, Name: A, dtype: float64

```

请注意，设置也适用：

```

In [30]: s2 = s.copy()

In [31]: s2[:5] = 0

In [32]: s2
Out[32]:
2000-01-01 0.000000
2000-01-02 0.000000
2000-01-03 0.000000
2000-01-04 0.000000
2000-01-05 0.000000
2000-01-06 -0.673690
2000-01-07 0.404705
2000-01-08 -0.370647
Freq: D, Name: A, dtype: float64

```

使用DataFrame，切片内部 [] 切片。这主要是为了方便而提供的，因为它是如此常见的操作。

```
In [33]: df[:3]
Out[33]:
```

	A	B	C	D
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632
2000-01-02	1.212112	-0.173215	0.119209	-1.044236
2000-01-03	-0.861849	-2.104569	-0.494929	1.071804

```
In [34]: df[::-1]
Out[34]:
```

	A	B	C	D
2000-01-08	-0.370647	-1.157892	-1.344312	0.844885
2000-01-07	0.404705	0.577046	-1.715002	-1.039268
2000-01-06	-0.673690	0.113648	-1.478427	0.524988
2000-01-05	-0.424972	0.567020	0.276232	-1.087401
2000-01-04	0.721555	-0.706771	-1.039575	0.271860
2000-01-03	-0.861849	-2.104569	-0.494929	1.071804
2000-01-02	1.212112	-0.173215	0.119209	-1.044236
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632

## 按标签选择

::: danger 警告

是否为设置操作返回副本或引用可能取决于上下文。这有时被称为应该避免。请参阅[返回视图与复制](#)。

chained assignment

:::

::: danger 警告

```
In [35]: dfl = pd.DataFrame(np.random.randn(5, 4),
.....: columns=list('ABCD'),
.....: index=pd.date_range('20130101', periods=5))
.....:
```

```
In [36]: dfl
Out[36]:
```

	A	B	C	D
2013-01-01	1.075770	-0.109050	1.643563	-1.469388
2013-01-02	0.357021	-0.674600	-1.776904	-0.968914
2013-01-03	-1.294524	0.413738	0.276662	-0.472035
2013-01-04	-0.013960	-0.362543	-0.006154	-0.923061
2013-01-05	0.895717	0.805244	-1.206412	2.565646

```
In [4]: dfl.loc[2:3]
TypeError: cannot do slice indexing on <class 'pandas.tseries.index.DatetimeIndex'> with these indexers
[2] of <type 'int'>
```

切片中的字符串喜欢可以转换为索引的类型并导致自然切片。

```
In [37]: dfl.loc['20130102':'20130104']
Out[37]:
```

	A	B	C	D
2013-01-02	0.357021	-0.674600	-1.776904	-0.968914
2013-01-03	-1.294524	0.413738	0.276662	-0.472035
2013-01-04	-0.013960	-0.362543	-0.006154	-0.923061

...

::: danger 警告

从0.21.0开始，pandas将显示 `FutureWarning` 带有缺少标签的列表的if索引。将来这会提高一个 `KeyError`。请参阅[list-like使用列表中缺少键的loc是不推荐使用](#)。

...

pandas提供了一套方法，以便拥有**纯粹基于标签的索引**。这是一个严格的包含协议。要求的每个标签必须在索引中，否则 `KeyError` 将被提出。切片时，如果索引中存在，则包括起始绑定**和**停止边界。整数是有效标签，但它们是指标签**而不是位置**。\*\*\*\*\*

该 `.loc` 属性是主要访问方法。以下是有效输入：

- 单个标签，例如 5 或 'a'（注意，它 5 被解释为索引的标签。此用法**不是**索引的整数位置。）。
- 列表或标签数组。 ['a', 'b', 'c']
- 带有标签的切片对象 'a':'f'（注意，与通常的python切片相反，**包括**起始和停止，当存在于索引中时！请参见[切片标签](#)。
- 布尔数组。
- A callable，参见[按可调用选择](#)。

```
In [38]: s1 = pd.Series(np.random.randn(6), index=list('abcdef'))
```

```
In [39]: s1
```

```
Out[39]:
```

```
a 1.431256
```

```
b 1.340309
```

```
c -1.170299
```

```
d -0.226169
```

```
e 0.410835
```

```
f 0.813850
```

```
dtype: float64
```

```
In [40]: s1.loc['c:']
```

```
Out[40]:
```

```
c -1.170299
```

```
d -0.226169
```

```
e 0.410835
```

```
f 0.813850
```

```
dtype: float64
```

```
In [41]: s1.loc['b']
```

```
Out[41]: 1.3403088497993827
```

请注意，设置也适用：

```
In [42]: s1.loc['c:'] = 0
```

```
In [43]: s1
```

```
Out[43]:
```

```
a 1.431256
```

```
b 1.340309
```

```
c 0.000000
```

```
d 0.000000
```

```
e 0.000000
```

```
f 0.000000
```

```
dtype: float64
```

使用DataFrame：



```
In [44]: df1 = pd.DataFrame(np.random.randn(6, 4),
.....: index=list('abcdef'),
.....: columns=list('ABCD'))
.....:
```

```
In [45]: df1
```

```
Out[45]:
```

	A	B	C	D
a	0.132003	-0.827317	-0.076467	-1.187678
b	1.130127	-1.436737	-1.413681	1.607920
c	1.024180	0.569605	0.875906	-2.211372
d	0.974466	-2.006747	-0.410001	-0.078638
e	0.545952	-1.219217	-1.226825	0.769804
f	-1.281247	-0.727707	-0.121306	-0.097883

```
In [46]: df1.loc[['a', 'b', 'd'], :]
```

```
Out[46]:
```

	A	B	C	D
a	0.132003	-0.827317	-0.076467	-1.187678
b	1.130127	-1.436737	-1.413681	1.607920
d	0.974466	-2.006747	-0.410001	-0.078638

通过标签切片访问：

```
In [47]: df1.loc['d:', 'A':'C']
```

```
Out[47]:
```

	A	B	C
d	0.974466	-2.006747	-0.410001
e	0.545952	-1.219217	-1.226825
f	-1.281247	-0.727707	-0.121306

使用标签获取横截面（相当于 `df.xs('a')`）：

```
In [48]: df1.loc['a']
```

```
Out[48]:
```

	A	B	C	D
a	0.132003	-0.827317	-0.076467	-1.187678

Name: a, dtype: float64

要使用布尔数组获取值：

```

In [49]: df1.loc['a'] > 0
Out[49]:
A True
B False
C False
D False
Name: a, dtype: bool

In [50]: df1.loc[:, df1.loc['a'] > 0]
Out[50]:
 A
a 0.132003
b 1.130127
c 1.024180
d 0.974466
e 0.545952
f -1.281247

```

要明确获取值（相当于已弃用 `df.get_value('a', 'A')`）：

```

this is also equivalent to ``df1.at['a', 'A']``
In [51]: df1.loc['a', 'A']
Out[51]: 0.13200317033032932

```

## 用标签切片

使用 `.loc` 切片时，如果索引中存在开始和停止标签，则返回位于两者之间的元素（包括它们）：

```

In [52]: s = pd.Series(list('abcde'), index=[0, 3, 2, 5, 4])

In [53]: s.loc[3:5]
Out[53]:
3 b
2 c
5 d
dtype: object

```

如果两个中至少有一个不存在，但索引已排序，并且可以与开始和停止标签进行比较，那么通过选择在两者之间排名的标签，切片仍将按预期工作：

```
In [54]: s.sort_index()
Out[54]:
0 a
2 c
3 b
4 e
5 d
dtype: object

In [55]: s.sort_index().loc[1:6]
Out[55]:
2 c
3 b
4 e
5 d
dtype: object
```

然而，如果两个中的至少一个不存在并且索引未被排序，则将引发错误（因为否则将是计算上昂贵的，并且对于混合类型索引可能是模糊的）。例如，在上面的例子中，`s.loc[1:6]` 会提高 `KeyError`。

有关此行为背后的基本原理，请参阅 [端点包含](#)。

## 按位置选择

---

::: danger 警告

是否为设置操作返回副本或引用可能取决于上下文。这有时被称为应该避免。请参阅[返回视图与复制](#)。

chained assignment

:::

Pandas提供了一套方法，以获得**纯粹基于整数的索引**。语义紧跟Python和NumPy切片。这些是 0-based 索引。切片时，所结合的开始被包括，而上限是排除。尝试使用非整数，甚至是**有效的**标签都会引发一个问题 `IndexError`。

该 `.iloc` 属性是主要访问方法。以下是有效输入：

- 一个整数，例如 5。
- 整数列表或数组。 `[4, 3, 0]`
- 带有整数的切片对象 `1:7`。
- 布尔数组。
- A callable，参见[按可调用选择](#)。

```

In [56]: s1 = pd.Series(np.random.randn(5), index=list(range(0, 10, 2)))

In [57]: s1
Out[57]:
0 0.695775
2 0.341734
4 0.959726
6 -1.110336
8 -0.619976
dtype: float64

In [58]: s1.iloc[:3]
Out[58]:
0 0.695775
2 0.341734
4 0.959726
dtype: float64

In [59]: s1.iloc[3]
Out[59]: -1.110336102891167

```

请注意，设置也适用：

```

In [60]: s1.iloc[:3] = 0

In [61]: s1
Out[61]:
0 0.000000
2 0.000000
4 0.000000
6 -1.110336
8 -0.619976
dtype: float64

```

使用DataFrame：

```

In [62]: df1 = pd.DataFrame(np.random.randn(6, 4),
.....: index=list(range(0, 12, 2)),
.....: columns=list(range(0, 8, 2)))
.....:

In [63]: df1
Out[63]:
 0 2 4 6
0 0.149748 -0.732339 0.687738 0.176444
2 0.403310 -0.154951 0.301624 -2.179861
4 -1.369849 -0.954208 1.462696 -1.743161
6 -0.826591 -0.345352 1.314232 0.690579
8 0.995761 2.396780 0.014871 3.357427
10 -0.317441 -1.236269 0.896171 -0.487602

```

通过整数切片选择：

```
In [64]: df1.iloc[:3]
Out[64]:
```

	0	2	4	6
0	0.149748	-0.732339	0.687738	0.176444
2	0.403310	-0.154951	0.301624	-2.179861
4	-1.369849	-0.954208	1.462696	-1.743161

```
In [65]: df1.iloc[1:5, 2:4]
Out[65]:
```

	4	6
2	0.301624	-2.179861
4	1.462696	-1.743161
6	1.314232	0.690579
8	0.014871	3.357427

通过整数列表选择：

```
In [66]: df1.iloc[[1, 3, 5], [1, 3]]
Out[66]:
```

	2	6
2	-0.154951	-2.179861
6	-0.345352	0.690579
10	-1.236269	-0.487602

```
In [67]: df1.iloc[1:3, :]
Out[67]:
```

	0	2	4	6
2	0.403310	-0.154951	0.301624	-2.179861
4	-1.369849	-0.954208	1.462696	-1.743161

```
In [68]: df1.iloc[:, 1:3]
Out[68]:
```

	2	4
0	-0.732339	0.687738
2	-0.154951	0.301624
4	-0.954208	1.462696
6	-0.345352	1.314232
8	2.396780	0.014871
10	-1.236269	0.896171

```
this is also equivalent to ``df1.iat[1,1]``
In [69]: df1.iloc[1, 1]
Out[69]: -0.1549507744249032
```

使用整数位置（等效 `df.xs(1)`）得到横截面：

```
In [70]: df1.iloc[1]
Out[70]:
0 0.403310
2 -0.154951
4 0.301624
6 -2.179861
Name: 2, dtype: float64
```

超出范围的切片索引正如Python / Numpy中一样优雅地处理。

```
these are allowed in python/numpy.
In [71]: x = list('abcdef')

In [72]: x
Out[72]: ['a', 'b', 'c', 'd', 'e', 'f']

In [73]: x[4:10]
Out[73]: ['e', 'f']

In [74]: x[8:10]
Out[74]: []

In [75]: s = pd.Series(x)

In [76]: s
Out[76]:
0 a
1 b
2 c
3 d
4 e
5 f
dtype: object

In [77]: s.iloc[4:10]
Out[77]:
4 e
5 f
dtype: object

In [78]: s.iloc[8:10]
Out[78]: Series([], dtype: object)
```

请注意，使用超出边界的切片可能会导致空轴（例如，返回一个空的DataFrame）。

```
In [79]: dfl = pd.DataFrame(np.random.randn(5, 2), columns=list('AB'))
```

```
In [80]: dfl
```

```
Out[80]:
```

	A	B
0	-0.082240	-2.182937
1	0.380396	0.084844
2	0.432390	1.519970
3	-0.493662	0.600178
4	0.274230	0.132885

```
In [81]: dfl.iloc[:, 2:3]
```

```
Out[81]:
```

```
Empty DataFrame
```

```
Columns: []
```

```
Index: [0, 1, 2, 3, 4]
```

```
In [82]: dfl.iloc[:, 1:3]
```

```
Out[82]:
```

	B
0	-2.182937
1	0.084844
2	1.519970
3	0.600178
4	0.132885

```
In [83]: dfl.iloc[4:6]
```

```
Out[83]:
```

	A	B
4	0.27423	0.132885

一个超出范围的索引器会引发一个 `IndexError`。任何元素超出范围的索引器列表都会引发 `IndexError`。

```
>>> dfl.iloc[[4, 5, 6]]
```

```
IndexError: positional indexers are out-of-bounds
```

```
>>> dfl.iloc[:, 4]
```

```
IndexError: single positional indexer is out-of-bounds
```

## 通过可调用选择

版本0.18.1中的新功能。

`.loc`，`.iloc` 以及 `[]` 索引也可以接受一个 callable 索引器。在 callable 必须与一个参数（调用系列或数据帧）返回的有效输出索引功能。

```
In [84]: df1 = pd.DataFrame(np.random.randn(6, 4),
.....: index=list('abcdef'),
.....: columns=list('ABCD'))
.....:
```

```
In [85]: df1
```

```
Out[85]:
```

	A	B	C	D
a	-0.023688	2.410179	1.450520	0.206053
b	-0.251905	-2.213588	1.063327	1.266143
c	0.299368	-0.863838	0.408204	-1.048089
d	-0.025747	-0.988387	0.094055	1.262731
e	1.289997	0.082423	-0.055758	0.536580
f	-0.489682	0.369374	-0.034571	-2.484478

```
In [86]: df1.loc[lambda df: df.A > 0, :]
```

```
Out[86]:
```

	A	B	C	D
c	0.299368	-0.863838	0.408204	-1.048089
e	1.289997	0.082423	-0.055758	0.536580

```
In [87]: df1.loc[:, lambda df: ['A', 'B']]
```

```
Out[87]:
```

	A	B
a	-0.023688	2.410179
b	-0.251905	-2.213588
c	0.299368	-0.863838
d	-0.025747	-0.988387
e	1.289997	0.082423
f	-0.489682	0.369374

```
In [88]: df1.iloc[:, lambda df: [0, 1]]
```

```
Out[88]:
```

	A	B
a	-0.023688	2.410179
b	-0.251905	-2.213588
c	0.299368	-0.863838
d	-0.025747	-0.988387
e	1.289997	0.082423
f	-0.489682	0.369374

```
In [89]: df1[lambda df: df.columns[0]]
```

```
Out[89]:
```

a	-0.023688
b	-0.251905
c	0.299368
d	-0.025747
e	1.289997
f	-0.489682

Name: A, dtype: float64

您可以使用可调用索引 Series。



```
In [90]: df1.A.loc[lambda s: s > 0]
Out[90]:
c 0.299368
e 1.289997
Name: A, dtype: float64
```

使用这些方法/索引器，您可以在不使用临时变量的情况下链接数据选择操作。

```
In [91]: bb = pd.read_csv('data/baseball.csv', index_col='id')

In [92]: (bb.groupby(['year', 'team']).sum()
....: .loc[lambda df: df.r > 100])
....:
Out[92]:
```

	stint	g	ab	r	h	X2b	X3b	hr	rbi	sb	cs	bb	so	ibb	hbp	sh
	sf	gidp														
year team																
2007 CIN	6	379	745	101	203	35	2	36	125.0	10.0	1.0	105	127.0	14.0	1.0	1.0
	15.0	18.0														
DET	5	301	1062	162	283	54	4	37	144.0	24.0	7.0	97	176.0	3.0	10.0	4.0
	8.0	28.0														
HOU	4	311	926	109	218	47	6	14	77.0	10.0	4.0	60	212.0	3.0	9.0	16.0
	6.0	17.0														
LAN	11	413	1021	153	293	61	3	36	154.0	7.0	5.0	114	141.0	8.0	9.0	3.0
	8.0	29.0														
NYN	13	622	1854	240	509	101	3	61	243.0	22.0	4.0	174	310.0	24.0	23.0	18.0
	15.0	48.0														
SFN	5	482	1305	198	337	67	6	40	171.0	26.0	7.0	235	188.0	51.0	8.0	16.0
	6.0	41.0														
TEX	2	198	729	115	200	40	4	28	115.0	21.0	4.0	73	140.0	4.0	5.0	2.0
	8.0	16.0														
TOR	4	459	1408	187	378	96	2	58	223.0	4.0	2.0	190	265.0	16.0	12.0	4.0
	16.0	38.0														

## 不推荐使用IX索引器

::: danger 警告

在0.20.0开始，`.ix` 索引器已被弃用，赞成更加严格 `.i loc` 和 `.loc` 索引。

:::

`.ix` 在推断用户想要做的事情上提供了很多魔力。也就是说，`.ix` 可以根据索引的数据类型决定按位置或通过标签进行索引。多年来，这引起了相当多的用户混淆。

建议的索引方法是：

- `.loc` 如果你想标记索引。
- `.i loc` 如果你想要定位索引。

```
In [93]: dfd = pd.DataFrame({'A': [1, 2, 3],
.....: 'B': [4, 5, 6]},
.....: index=list('abc'))

In [94]: dfd
Out[94]:
 A B
a 1 4
b 2 5
c 3 6
```

以前的行为，您希望从“A”列中获取索引中的第0个和第2个元素。

```
In [3]: dfd.ix[[0, 2], 'A']
Out[3]:
a 1
c 3
Name: A, dtype: int64
```

用 `.loc`。这里我们将从索引中选择适当的索引，然后使用标签索引。

```
In [95]: dfd.loc[dfd.index[[0, 2], 'A']]
Out[95]:
a 1
c 3
Name: A, dtype: int64
```

这也可以 `.iloc` 通过在索引器上显式获取位置，并使用位置索引来选择事物来表达。

```
In [96]: dfd.iloc[[0, 2], dfd.columns.get_loc('A')]
Out[96]:
a 1
c 3
Name: A, dtype: int64
```

要获得多个索引器，请使用 `.get_indexer`：

```
In [97]: dfd.iloc[[0, 2], dfd.columns.get_indexer(['A', 'B'])]
Out[97]:
 A B
a 1 4
c 3 6
```

## 不推荐使用缺少标签的列表进行索引

::: danger 警告

从0.21.0开始，使用 `.loc` 或 `[]` 包含一个或多个缺少标签的列表，不赞成使用 `.reindex`。

...

在以前的版本中，`.loc[list-of-labels]` 只要找到至少1个密钥，使用就可以工作（否则会引起a `KeyError`）。不推荐使用此行为，并将显示指向此部分的警告消息。推荐的替代方案是使用 `.reindex()`。

例如。

```
In [98]: s = pd.Series([1, 2, 3])

In [99]: s
Out[99]:
0 1
1 2
2 3
dtype: int64
```

找到所有键的选择保持不变。

```
In [100]: s.loc[[1, 2]]
Out[100]:
1 2
2 3
dtype: int64
```

以前的行为

```
In [4]: s.loc[[1, 2, 3]]
Out[4]:
1 2.0
2 3.0
3 NaN
dtype: float64
```

目前的行为

```
In [4]: s.loc[[1, 2, 3]]
Passing list-likes to .loc with any non-matching elements will raise
KeyError in the future, you can use .reindex() as an alternative.

See the documentation here:
http://pandas.pydata.org/pandas-docs/stable/indexing.html#deprecate-loc-reindex-listlike

Out[4]:
1 2.0
2 3.0
3 NaN
dtype: float64
```

## 重新索引

实现选择潜在的未找到元素的惯用方法是通过 `.reindex()`。另请参阅[重建索引](#)部分。

```
In [101]: s.reindex([1, 2, 3])
Out[101]:
1 2.0
2 3.0
3 NaN
dtype: float64
```

或者，如果您只想选择有效的密钥，则以下是惯用且有效的; 保证保留选择的dtype。

```
In [102]: labels = [1, 2, 3]

In [103]: s.loc[s.index.intersection(labels)]
Out[103]:
1 2
2 3
dtype: int64
```

拥有重复索引会引发 `.reindex()`：

```
In [104]: s = pd.Series(np.arange(4), index=['a', 'a', 'b', 'c'])

In [105]: labels = ['c', 'd']
```

```
In [17]: s.reindex(labels)
ValueError: cannot reindex from a duplicate axis
```

通常，您可以将所需标签与当前轴相交，然后重新索引。

```
In [106]: s.loc[s.index.intersection(labels)].reindex(labels)
Out[106]:
c 3.0
d NaN
dtype: float64
```

但是，如果生成的索引重复，这仍然会提高。

```
In [41]: labels = ['a', 'd']

In [42]: s.loc[s.index.intersection(labels)].reindex(labels)
ValueError: cannot reindex from a duplicate axis
```

## 选择随机样本

使用该`sample()`方法随机选择Series或DataFrame中的行或列。默认情况下，该方法将对行进行采样，并接受要返回的特定行数/列数或一小部分行。

```
In [107]: s = pd.Series([0, 1, 2, 3, 4, 5])

When no arguments are passed, returns 1 row.
In [108]: s.sample()
Out[108]:
4 4
dtype: int64

One may specify either a number of rows:
In [109]: s.sample(n=3)
Out[109]:
0 0
4 4
1 1
dtype: int64

Or a fraction of the rows:
In [110]: s.sample(frac=0.5)
Out[110]:
5 5
3 3
1 1
dtype: int64
```

默认情况下，`sample` 最多会返回每行一次，但也可以使用以下 `replace` 选项进行替换：

```
In [111]: s = pd.Series([0, 1, 2, 3, 4, 5])
```

```
Without replacement (default):
```

```
In [112]: s.sample(n=6, replace=False)
```

```
Out[112]:
```

```
0 0
1 1
5 5
3 3
2 2
4 4
```

```
dtype: int64
```

```
With replacement:
```

```
In [113]: s.sample(n=6, replace=True)
```

```
Out[113]:
```

```
0 0
4 4
3 3
2 2
4 4
4 4
```

```
dtype: int64
```

默认情况下，每行具有相同的选择概率，但如果您希望行具有不同的概率，则可以将 `sample` 函数采样权重作为 `weights`。这些权重可以是列表，NumPy数组或系列，但它们的长度必须与您采样的对象的长度相同。缺失的值将被视为零的权重，并且不允许使用inf值。如果权重不总和为1，则通过将所有权重除以权重之和来对它们进行重新规范化。例如：

```
In [114]: s = pd.Series([0, 1, 2, 3, 4, 5])
```

```
In [115]: example_weights = [0, 0, 0.2, 0.2, 0.2, 0.4]
```

```
In [116]: s.sample(n=3, weights=example_weights)
```

```
Out[116]:
```

```
5 5
4 4
3 3
```

```
dtype: int64
```

```
Weights will be re-normalized automatically
```

```
In [117]: example_weights2 = [0.5, 0, 0, 0, 0, 0]
```

```
In [118]: s.sample(n=1, weights=example_weights2)
```

```
Out[118]:
```

```
0 0
```

```
dtype: int64
```

应用于DataFrame时，只需将列的名称作为字符串传递，就可以使用DataFrame的列作为采样权重（假设您要对行而不是列进行采样）。

```
In [119]: df2 = pd.DataFrame({'col1': [9, 8, 7, 6],
.....: 'weight_column': [0.5, 0.4, 0.1, 0]})
.....:

In [120]: df2.sample(n=3, weights='weight_column')
Out[120]:
```

	col1	weight_column
1	8	0.4
0	9	0.5
2	7	0.1

`sample` 还允许用户使用 `axis` 参数对列而不是行进行采样。

```
In [121]: df3 = pd.DataFrame({'col1': [1, 2, 3], 'col2': [2, 3, 4]})

In [122]: df3.sample(n=1, axis=1)
Out[122]:
```

	col1
0	1
1	2
2	3

最后，还可以 `sample` 使用 `random_state` 参数为随机数生成器设置种子，该参数将接受整数（作为种子）或 NumPy RandomState 对象。

```
In [123]: df4 = pd.DataFrame({'col1': [1, 2, 3], 'col2': [2, 3, 4]})

With a given seed, the sample will always draw the same rows.
In [124]: df4.sample(n=2, random_state=2)
Out[124]:
```

	col1	col2
2	3	4
1	2	3

```
In [125]: df4.sample(n=2, random_state=2)
Out[125]:
```

	col1	col2
2	3	4
1	2	3

## 用放大设定

`.loc[]` 当为该轴设置不存在的键时，操作可以执行放大。

在这种 Series 情况下，这实际上是一种附加操作。

```
In [126]: se = pd.Series([1, 2, 3])
```

```
In [127]: se
```

```
Out[127]:
```

```
0 1
1 2
2 3
dtype: int64
```

```
In [128]: se[5] = 5.
```

```
In [129]: se
```

```
Out[129]:
```

```
0 1.0
1 2.0
2 3.0
5 5.0
dtype: float64
```

A `DataFrame` 可以在任一轴上放大 `.loc`。

```
In [130]: dfi = pd.DataFrame(np.arange(6).reshape(3, 2),
.....: columns=['A', 'B'])
.....:
```

```
In [131]: dfi
```

```
Out[131]:
```

```
 A B
0 0 1
1 2 3
2 4 5
```

```
In [132]: dfi.loc[:, 'C'] = dfi.loc[:, 'A']
```

```
In [133]: dfi
```

```
Out[133]:
```

```
 A B C
0 0 1 0
1 2 3 2
2 4 5 4
```

这就像是一个 `append` 操作 `DataFrame`。

```
In [134]: dfi.loc[3] = 5
```

```
In [135]: dfi
```

```
Out[135]:
```

```
 A B C
0 0 1 0
1 2 3 2
2 4 5 4
3 5 5 5
```



## 快速标量值获取和设置

因为索引 [] 必须处理很多情况（单标签访问，切片，布尔索引等），所以它有一些开销以便弄清楚你要求的是什麼。如果您只想访问标量值，最快的方法是使用在所有数据结构上实现的 `at` 和 `iat` 方法。

与之类似 `loc`，`at` 提供基于**标签**的标量查找，同时 `iat` 提供类似于基于**整数**的查找 `iloc`

```
In [136]: s.iat[5]
Out[136]: 5

In [137]: df.at[dates[5], 'A']
Out[137]: -0.6736897080883706

In [138]: df.iat[3, 0]
Out[138]: 0.7215551622443669
```

您也可以使用这些相同的索引器进行设置。

```
In [139]: df.at[dates[5], 'E'] = 7

In [140]: df.iat[3, 0] = 7
```

`at` 如果索引器丢失，可以如上所述放大对象。

```
In [141]: df.at[dates[-1] + pd.Timedelta('1 day'), 0] = 7

In [142]: df
Out[142]:
```

	A	B	C	D	E	0
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632	NaN	NaN
2000-01-02	1.212112	-0.173215	0.119209	-1.044236	NaN	NaN
2000-01-03	-0.861849	-2.104569	-0.494929	1.071804	NaN	NaN
2000-01-04	7.000000	-0.706771	-1.039575	0.271860	NaN	NaN
2000-01-05	-0.424972	0.567020	0.276232	-1.087401	NaN	NaN
2000-01-06	-0.673690	0.113648	-1.478427	0.524988	7.0	NaN
2000-01-07	0.404705	0.577046	-1.715002	-1.039268	NaN	NaN
2000-01-08	-0.370647	-1.157892	-1.344312	0.844885	NaN	NaN
2000-01-09	NaN	NaN	NaN	NaN	NaN	7.0

## 布尔索引

另一种常见操作是使用布尔向量来过滤数据。运营商是：`|` for `or`，`&` for `and` 和 `~` for `not`。必须使用括号对这些进行分组，因为默认情况下，Python将评估表达式，例如 `as`，而期望的评估顺序是。`df.A > 2 & df.B < 3`  
`df.A > (2 & df.B < 3)`

使用布尔向量索引系列的工作方式与NumPy ndarray完全相同：

```
In [143]: s = pd.Series(range(-3, 4))
```

```
In [144]: s
```

```
Out[144]:
```

```
0 -3
1 -2
2 -1
3 0
4 1
5 2
6 3
```

```
dtype: int64
```

```
In [145]: s[s > 0]
```

```
Out[145]:
```

```
4 1
5 2
6 3
```

```
dtype: int64
```

```
In [146]: s[(s < -1) | (s > 0.5)]
```

```
Out[146]:
```

```
0 -3
1 -2
4 1
5 2
6 3
```

```
dtype: int64
```

```
In [147]: s[~(s < 0)]
```

```
Out[147]:
```

```
3 0
4 1
5 2
6 3
```

```
dtype: int64
```

您可以使用与DataFrame索引长度相同的布尔向量从DataFrame中选择行（例如，从DataFrame的其中一列派生的东西）：

```
In [148]: df[df['A'] > 0]
```

```
Out[148]:
```

	A	B	C	D	E	0
2000-01-01	0.469112	-0.282863	-1.509059	-1.135632	NaN	NaN
2000-01-02	1.212112	-0.173215	0.119209	-1.044236	NaN	NaN
2000-01-04	7.000000	-0.706771	-1.039575	0.271860	NaN	NaN
2000-01-07	0.404705	0.577046	-1.715002	-1.039268	NaN	NaN

列表推导和 `map` 系列方法也可用于产生更复杂的标准：

```
In [149]: df2 = pd.DataFrame({'a': ['one', 'one', 'two', 'three', 'two', 'one', 'six'],
.....: 'b': ['x', 'y', 'y', 'x', 'y', 'x', 'x'],
.....: 'c': np.random.randn(7)})
.....:
```

# only want 'two' or 'three'

```
In [150]: criterion = df2['a'].map(lambda x: x.startswith('t'))
```

```
In [151]: df2[criterion]
```

```
Out[151]:
```

	a	b	c
2	two	y	0.041290
3	three	x	0.361719
4	two	y	-0.238075

# equivalent but slower

```
In [152]: df2[[x.startswith('t') for x in df2['a']]]
```

```
Out[152]:
```

	a	b	c
2	two	y	0.041290
3	three	x	0.361719
4	two	y	-0.238075

# Multiple criteria

```
In [153]: df2[criterion & (df2['b'] == 'x')]
```

```
Out[153]:
```

	a	b	c
3	three	x	0.361719

随着选择方法[通过标签选择](#)，[通过位置选择](#)和[高级索引](#)，你可以沿着使用布尔向量与其他索引表达式中组合选择多个轴。

```
In [154]: df2.loc[criterion & (df2['b'] == 'x'), 'b':'c']
Out[154]:
```

	b	c
3	x	0.361719

## 使用isin进行索引

考虑一下`isin()`方法 `Series`，该方法返回一个布尔向量，只要 `Series` 元素存在于传递列表中，该向量就为真。这允许您选择一行或多列具有所需值的行：

```
In [155]: s = pd.Series(np.arange(5), index=np.arange(5)[::-1], dtype='int64')
```

```
In [156]: s
```

```
Out[156]:
```

```
4 0
```

```
3 1
```

```
2 2
```

```
1 3
```

```
0 4
```

```
dtype: int64
```

```
In [157]: s.isin([2, 4, 6])
```

```
Out[157]:
```

```
4 False
```

```
3 False
```

```
2 True
```

```
1 False
```

```
0 True
```

```
dtype: bool
```

```
In [158]: s[s.isin([2, 4, 6])]
```

```
Out[158]:
```

```
2 2
```

```
0 4
```

```
dtype: int64
```

`Index` 对象可以使用相同的方法，当您不知道哪些搜索标签实际存在时，它们非常有用：

```
In [159]: s[s.index.isin([2, 4, 6])]
```

```
Out[159]:
```

```
4 0
```

```
2 2
```

```
dtype: int64
```

```
compare it to the following
```

```
In [160]: s.reindex([2, 4, 6])
```

```
Out[160]:
```

```
2 2.0
```

```
4 0.0
```

```
6 NaN
```

```
dtype: float64
```

除此之外，还 `MultiIndex` 允许选择在成员资格检查中使用的单独级别：

```

In [161]: s_mi = pd.Series(np.arange(6),
.....: index=pd.MultiIndex.from_product([[0, 1], ['a', 'b', 'c']]))
.....:

In [162]: s_mi
Out[162]:
0 a 0
 b 1
 c 2
1 a 3
 b 4
 c 5
dtype: int64

In [163]: s_mi.iloc[s_mi.index.isin([(1, 'a'), (2, 'b'), (0, 'c')])]
Out[163]:
0 c 2
1 a 3
dtype: int64

In [164]: s_mi.iloc[s_mi.index.isin(['a', 'c', 'e'], level=1)]
Out[164]:
0 a 0
 c 2
1 a 3
 c 5
dtype: int64

```

DataFrame也有一个`isin()`方法。调用时 `isin`，将一组值作为数组或字典传递。如果`values`是一个数组，则 `isin` 返回与原始DataFrame形状相同的布尔数据框，并在元素序列中的任何位置使用True。

```

In [165]: df = pd.DataFrame({'vals': [1, 2, 3, 4], 'ids': ['a', 'b', 'f', 'n'],
.....: 'ids2': ['a', 'n', 'c', 'n']})
.....:

In [166]: values = ['a', 'b', 1, 3]

In [167]: df.isin(values)
Out[167]:
 vals ids ids2
0 True True True
1 False True False
2 True False False
3 False False False

```

通常，您需要将某些值与某些列匹配。只需将值设置 `dict` 为键为列的位置，值即为要检查的项目列表。

```
In [168]: values = {'ids': ['a', 'b'], 'vals': [1, 3]}
```

```
In [169]: df.isin(values)
```

```
Out[169]:
```

	vals	ids	ids2
0	True	True	False
1	False	True	False
2	True	False	False
3	False	False	False

结合数据帧的 `isin` 同 `any()` 和 `all()` 方法来快速选择符合给定的标准对数据子集。要选择每列符合其自己标准的行：

```
In [170]: values = {'ids': ['a', 'b'], 'ids2': ['a', 'c'], 'vals': [1, 3]}
```

```
In [171]: row_mask = df.isin(values).all(1)
```

```
In [172]: df[row_mask]
```

```
Out[172]:
```

	vals	ids	ids2
0	1	a	a

## 该 `where()` 方法和屏蔽

从具有布尔向量的 Series 中选择值通常会返回数据的子集。为了保证选择输出与原始数据具有相同的形状，您可以 `where` 在 Series 和中使用该方法 DataFrame。

仅返回选定的行：

```
In [173]: s[s > 0]
```

```
Out[173]:
```

```
3 1
2 2
1 3
0 4
dtype: int64
```

要返回与原始形状相同的系列：

```
In [174]: s.where(s > 0)
```

```
Out[174]:
```

```
4 NaN
3 1.0
2 2.0
1 3.0
0 4.0
dtype: float64
```

现在，使用布尔标准从DataFrame中选择值也可以保留输入数据形状。 `where` 在引擎盖下用作实现。下面的代码相当于。 `df.where(df < 0)`

```
In [175]: df[df < 0]
Out[175]:
```

	A	B	C	D
2000-01-01	-2.104139	-1.309525	NaN	NaN
2000-01-02	-0.352480	NaN	-1.192319	NaN
2000-01-03	-0.864883	NaN	-0.227870	NaN
2000-01-04	NaN	-1.222082	NaN	-1.233203
2000-01-05	NaN	-0.605656	-1.169184	NaN
2000-01-06	NaN	-0.948458	NaN	-0.684718
2000-01-07	-2.670153	-0.114722	NaN	-0.048048
2000-01-08	NaN	NaN	-0.048788	-0.808838

此外，在返回的副本中， `where` 使用可选 `other` 参数替换条件为False的值。

```
In [176]: df.where(df < 0, -df)
Out[176]:
```

	A	B	C	D
2000-01-01	-2.104139	-1.309525	-0.485855	-0.245166
2000-01-02	-0.352480	-0.390389	-1.192319	-1.655824
2000-01-03	-0.864883	-0.299674	-0.227870	-0.281059
2000-01-04	-0.846958	-1.222082	-0.600705	-1.233203
2000-01-05	-0.669692	-0.605656	-1.169184	-0.342416
2000-01-06	-0.868584	-0.948458	-2.297780	-0.684718
2000-01-07	-2.670153	-0.114722	-0.168904	-0.048048
2000-01-08	-0.801196	-1.392071	-0.048788	-0.808838

您可能希望根据某些布尔条件设置值。这可以直观地完成，如下所示：

```

In [177]: s2 = s.copy()

In [178]: s2[s2 < 0] = 0

In [179]: s2
Out[179]:
4 0
3 1
2 2
1 3
0 4
dtype: int64

In [180]: df2 = df.copy()

In [181]: df2[df2 < 0] = 0

In [182]: df2
Out[182]:

```

	A	B	C	D
2000-01-01	0.000000	0.000000	0.485855	0.245166
2000-01-02	0.000000	0.390389	0.000000	1.655824
2000-01-03	0.000000	0.299674	0.000000	0.281059
2000-01-04	0.846958	0.000000	0.600705	0.000000
2000-01-05	0.669692	0.000000	0.000000	0.342416
2000-01-06	0.868584	0.000000	2.297780	0.000000
2000-01-07	0.000000	0.000000	0.168904	0.000000
2000-01-08	0.801196	1.392071	0.000000	0.000000

默认情况下，`where` 返回数据的修改副本。有一个可选参数，`inplace` 以便可以在不创建副本的情况下修改原始数据：

```

In [183]: df_orig = df.copy()

In [184]: df_orig.where(df > 0, -df, inplace=True)

In [185]: df_orig
Out[185]:

```

	A	B	C	D
2000-01-01	2.104139	1.309525	0.485855	0.245166
2000-01-02	0.352480	0.390389	1.192319	1.655824
2000-01-03	0.864883	0.299674	0.227870	0.281059
2000-01-04	0.846958	1.222082	0.600705	1.233203
2000-01-05	0.669692	0.605656	1.169184	0.342416
2000-01-06	0.868584	0.948458	2.297780	0.684718
2000-01-07	2.670153	0.114722	0.168904	0.048048
2000-01-08	0.801196	1.392071	0.048788	0.808838

::: tip 注意

签名 `DataFrame.where()` 不同于 `numpy.where()`。大致相当于。 `df1.where(m, df2)``np.where(m, df1, df2)`



```
In [186]: df.where(df < 0, -df) == np.where(df < 0, df, -df)
Out[186]:
```

	A	B	C	D
2000-01-01	True	True	True	True
2000-01-02	True	True	True	True
2000-01-03	True	True	True	True
2000-01-04	True	True	True	True
2000-01-05	True	True	True	True
2000-01-06	True	True	True	True
2000-01-07	True	True	True	True
2000-01-08	True	True	True	True

...

## 对准

此外，`where` 对齐输入布尔条件（`ndarray`或`DataFrame`），以便可以使用设置进行部分选择。这类似于部分设置通过 `.loc`（但是在内容而不是轴标签上）。

```
In [187]: df2 = df.copy()

In [188]: df2[df2[1:4] > 0] = 3

In [189]: df2
Out[189]:
```

	A	B	C	D
2000-01-01	-2.104139	-1.309525	0.485855	0.245166
2000-01-02	-0.352480	3.000000	-1.192319	3.000000
2000-01-03	-0.864883	3.000000	-0.227870	3.000000
2000-01-04	3.000000	-1.222082	3.000000	-1.233203
2000-01-05	0.669692	-0.605656	-1.169184	0.342416
2000-01-06	0.868584	-0.948458	2.297780	-0.684718
2000-01-07	-2.670153	-0.114722	0.168904	-0.048048
2000-01-08	0.801196	1.392071	-0.048788	-0.808838

哪里也可以接受 `axis` 和 `level` 参数在执行时对齐输入 `where`。

```
In [190]: df2 = df.copy()

In [191]: df2.where(df2 > 0, df2['A'], axis='index')
Out[191]:
```

	A	B	C	D
2000-01-01	-2.104139	-2.104139	0.485855	0.245166
2000-01-02	-0.352480	0.390389	-0.352480	1.655824
2000-01-03	-0.864883	0.299674	-0.864883	0.281059
2000-01-04	0.846958	0.846958	0.600705	0.846958
2000-01-05	0.669692	0.669692	0.669692	0.342416
2000-01-06	0.868584	0.868584	2.297780	0.868584
2000-01-07	-2.670153	-2.670153	0.168904	-2.670153
2000-01-08	0.801196	1.392071	0.801196	0.801196

这相当于（但快于）以下内容。

```
In [192]: df2 = df.copy()

In [193]: df.apply(lambda x, y: x.where(x > 0, y), y=df['A'])
Out[193]:
```

	A	B	C	D
2000-01-01	-2.104139	-2.104139	0.485855	0.245166
2000-01-02	-0.352480	0.390389	-0.352480	1.655824
2000-01-03	-0.864883	0.299674	-0.864883	0.281059
2000-01-04	0.846958	0.846958	0.600705	0.846958
2000-01-05	0.669692	0.669692	0.669692	0.342416
2000-01-06	0.868584	0.868584	2.297780	0.868584
2000-01-07	-2.670153	-2.670153	0.168904	-2.670153
2000-01-08	0.801196	1.392071	0.801196	0.801196

版本0.18.1中的新功能。

哪里可以接受一个可调用的条件和 `other` 参数。该函数必须带有一个参数（调用Series或DataFrame），并返回有效的输出作为条件和 `other` 参数。

```
In [194]: df3 = pd.DataFrame({'A': [1, 2, 3],
.....: 'B': [4, 5, 6],
.....: 'C': [7, 8, 9]})
.....:

In [195]: df3.where(lambda x: x > 4, lambda x: x + 10)
Out[195]:
```

	A	B	C
0	11	14	7
1	12	5	8
2	13	6	9

## 面具

`mask()` 是的反布尔运算 `where`。

```

In [196]: s.mask(s >= 0)
Out[196]:
4 NaN
3 NaN
2 NaN
1 NaN
0 NaN
dtype: float64

In [197]: df.mask(df >= 0)
Out[197]:

```

	A	B	C	D
2000-01-01	-2.104139	-1.309525	NaN	NaN
2000-01-02	-0.352480	NaN	-1.192319	NaN
2000-01-03	-0.864883	NaN	-0.227870	NaN
2000-01-04	NaN	-1.222082	NaN	-1.233203
2000-01-05	NaN	-0.605656	-1.169184	NaN
2000-01-06	NaN	-0.948458	NaN	-0.684718
2000-01-07	-2.670153	-0.114722	NaN	-0.048048
2000-01-08	NaN	NaN	-0.048788	-0.808838

## 该query()方法

`DataFrame`对象有一个`query()` 允许使用表达式进行选择的方法。

您可以获取列的值，其中列 `b` 具有列值 `a` 和值之间的值 `c`。例如：

```

In [198]: n = 10

In [199]: df = pd.DataFrame(np.random.rand(n, 3), columns=list('abc'))

In [200]: df
Out[200]:
 a b c
0 0.438921 0.118680 0.863670
1 0.138138 0.577363 0.686602
2 0.595307 0.564592 0.520630
3 0.913052 0.926075 0.616184
4 0.078718 0.854477 0.898725
5 0.076404 0.523211 0.591538
6 0.792342 0.216974 0.564056
7 0.397890 0.454131 0.915716
8 0.074315 0.437913 0.019794
9 0.559209 0.502065 0.026437

pure python
In [201]: df[(df.a < df.b) & (df.b < df.c)]
Out[201]:
 a b c
1 0.138138 0.577363 0.686602
4 0.078718 0.854477 0.898725
5 0.076404 0.523211 0.591538
7 0.397890 0.454131 0.915716

query
In [202]: df.query('(a < b) & (b < c)')
Out[202]:
 a b c
1 0.138138 0.577363 0.686602
4 0.078718 0.854477 0.898725
5 0.076404 0.523211 0.591538
7 0.397890 0.454131 0.915716

```

如果没有名称的列，则执行相同的操作但返回命名索引 `a`。

```

In [203]: df = pd.DataFrame(np.random.randint(n / 2, size=(n, 2)), columns=list('bc'))

In [204]: df.index.name = 'a'

In [205]: df
Out[205]:
 b c
a
0 0 4
1 0 1
2 3 4
3 4 3
4 1 4
5 0 3
6 0 1
7 3 4
8 2 3
9 1 1

In [206]: df.query('a < b and b < c')
Out[206]:
 b c
a
2 3 4

```

如果您不希望或不能命名索引，则可以 `index` 在查询表达式中使用该名称：

```

In [207]: df = pd.DataFrame(np.random.randint(n, size=(n, 2)), columns=list('bc'))

In [208]: df
Out[208]:
 b c
0 3 1
1 3 0
2 5 6
3 5 2
4 7 4
5 0 1
6 2 5
7 0 1
8 6 0
9 7 9

In [209]: df.query('index < b < c')
Out[209]:
 b c
2 5 6

```

::: tip 注意

如果索引的名称与列名称重叠，则列名称优先。例如，

```
In [210]: df = pd.DataFrame({'a': np.random.randint(5, size=5)})

In [211]: df.index.name = 'a'

In [212]: df.query('a > 2') # uses the column 'a', not the index
Out[212]:
 a
1 3
3 3
```

您仍然可以使用特殊标识符 'index' 在查询表达式中使用索引：

```
In [213]: df.query('index > 2')
Out[213]:
 a
3 3
4 2
```

如果由于某种原因你有一个名为列的列 `index`，那么你也可以引用索引 `ilvl_0`，但是此时你应该考虑将列重命名为不那么模糊的列。

...

### **MultiIndex query() 语法**

您还可以使用的水平 `DataFrame` 带 `MultiIndex`，好像他们是在框架柱：

```

In [214]: n = 10

In [215]: colors = np.random.choice(['red', 'green'], size=n)

In [216]: foods = np.random.choice(['eggs', 'ham'], size=n)

In [217]: colors
Out[217]:
array(['red', 'red', 'red', 'green', 'green', 'green', 'green', 'green',
 'green', 'green'], dtype='<U5')

In [218]: foods
Out[218]:
array(['ham', 'ham', 'eggs', 'eggs', 'eggs', 'ham', 'ham', 'eggs', 'eggs',
 'eggs'], dtype='<U4')

In [219]: index = pd.MultiIndex.from_arrays([colors, foods], names=['color', 'food'])

In [220]: df = pd.DataFrame(np.random.randn(n, 2), index=index)

In [221]: df
Out[221]:
```

		0	1
red	ham	0.194889	-0.381994
	ham	0.318587	2.089075
	eggs	-0.728293	-0.090255
green	eggs	-0.748199	1.318931
	eggs	-2.029766	0.792652
	ham	0.461007	-0.542749
	ham	-0.305384	-0.479195
	eggs	0.095031	-0.270099
	eggs	-0.707140	-0.773882
	eggs	0.229453	0.304418

```

In [222]: df.query('color == "red"')
Out[222]:
```

		0	1
red	ham	0.194889	-0.381994
	ham	0.318587	2.089075
	eggs	-0.728293	-0.090255

如果 `MultiIndex` 未命名的级别，您可以使用特殊名称引用它们：

```
In [223]: df.index.names = [None, None]
```

```
In [224]: df
```

```
Out[224]:
```

		0	1
red	ham	0.194889	-0.381994
	ham	0.318587	2.089075
	eggs	-0.728293	-0.090255
green	eggs	-0.748199	1.318931
	eggs	-2.029766	0.792652
	ham	0.461007	-0.542749
	ham	-0.305384	-0.479195
	eggs	0.095031	-0.270099
	eggs	-0.707140	-0.773882
	eggs	0.229453	0.304418

```
In [225]: df.query('i level_0 == "red"')
```

```
Out[225]:
```

		0	1
red	ham	0.194889	-0.381994
	ham	0.318587	2.089075
	eggs	-0.728293	-0.090255

约定是 `i level_0`，这意味着第0级的“索引级别0” `index`。

## query()用例

用例`query()`是当您拥有一组具有共同 `DataFrame`列名（或索引级别/名称）子集的对象时。您可以将相同的查询传递给两个帧，而无需指定您对查询感兴趣的帧



```
In [226]: df = pd.DataFrame(np.random.rand(n, 3), columns=list('abc'))
```

```
In [227]: df
```

```
Out[227]:
```

	a	b	c
0	0.224283	0.736107	0.139168
1	0.302827	0.657803	0.713897
2	0.611185	0.136624	0.984960
3	0.195246	0.123436	0.627712
4	0.618673	0.371660	0.047902
5	0.480088	0.062993	0.185760
6	0.568018	0.483467	0.445289
7	0.309040	0.274580	0.587101
8	0.258993	0.477769	0.370255
9	0.550459	0.840870	0.304611

```
In [228]: df2 = pd.DataFrame(np.random.rand(n + 2, 3), columns=df.columns)
```

```
In [229]: df2
```

```
Out[229]:
```

	a	b	c
0	0.357579	0.229800	0.596001
1	0.309059	0.957923	0.965663
2	0.123102	0.336914	0.318616
3	0.526506	0.323321	0.860813
4	0.518736	0.486514	0.384724
5	0.190804	0.505723	0.614533
6	0.891939	0.623977	0.676639
7	0.480559	0.378528	0.460858
8	0.420223	0.136404	0.141295
9	0.732206	0.419540	0.604675
10	0.604466	0.848974	0.896165
11	0.589168	0.920046	0.732716

```
In [230]: expr = '0.0 <= a <= c <= 0.5'
```

```
In [231]: map(lambda frame: frame.query(expr), [df, df2])
```

```
Out[231]: <map at 0x7f65f7952d30>
```

## query()Python与pandas语法比较

完全类似numpy的语法：

```
In [232]: df = pd.DataFrame(np.random.randint(n, size=(n, 3)), columns=list('abc'))
```

```
In [233]: df
```

```
Out[233]:
```

	a	b	c
0	7	8	9
1	1	0	7
2	2	7	2
3	6	2	2
4	2	6	3
5	3	8	2
6	1	7	2
7	5	1	5
8	9	8	0
9	1	5	0

```
In [234]: df.query('(a < b) & (b < c)')
```

```
Out[234]:
```

	a	b	c
0	7	8	9

```
In [235]: df[(df.a < df.b) & (df.b < df.c)]
```

```
Out[235]:
```

	a	b	c
0	7	8	9

通过删除括号略微更好（通过绑定使比较运算符绑定比 `&` 和更紧 `|`）。

```
In [236]: df.query('a < b & b < c')
```

```
Out[236]:
```

	a	b	c
0	7	8	9

使用英语而不是符号：

```
In [237]: df.query('a < b and b < c')
```

```
Out[237]:
```

	a	b	c
0	7	8	9

非常接近你如何在纸上写它：

```
In [238]: df.query('a < b < c')
```

```
Out[238]:
```

	a	b	c
0	7	8	9

## 在 `in` 与 运营商 `not in`

`query()` 还支持 Python `in` 和 比较运算符的特殊用法，为调用或的方法提供了简洁的语法。 `not`

`in` `isin` `Series` `DataFrame`

```

get all rows where columns "a" and "b" have overlapping values
In [239]: df = pd.DataFrame({'a': list('aabbccddeeff'), 'b': list('aaaabbbbcccc'),
.....: 'c': np.random.randint(5, size=12),
.....: 'd': np.random.randint(9, size=12)})
.....:

In [240]: df
Out[240]:
 a b c d
0 a a 2 6
1 a a 4 7
2 b a 1 6
3 b a 2 1
4 c b 3 6
5 c b 0 2
6 d b 3 3
7 d b 2 1
8 e c 4 3
9 e c 2 0
10 f c 0 6
11 f c 1 2

In [241]: df.query('a in b')
Out[241]:
 a b c d
0 a a 2 6
1 a a 4 7
2 b a 1 6
3 b a 2 1
4 c b 3 6
5 c b 0 2

How you'd do it in pure Python
In [242]: df[df.a.isin(df.b)]
Out[242]:
 a b c d
0 a a 2 6
1 a a 4 7
2 b a 1 6
3 b a 2 1
4 c b 3 6
5 c b 0 2

In [243]: df.query('a not in b')
Out[243]:
 a b c d
6 d b 3 3
7 d b 2 1
8 e c 4 3
9 e c 2 0
10 f c 0 6
11 f c 1 2

pure Python
In [244]: df[~df.a.isin(df.b)]

```

```
Out[244]:
 a b c d
6 d b 3 3
7 d b 2 1
8 e c 4 3
9 e c 2 0
10 f c 0 6
11 f c 1 2
```

您可以将此与其他表达式结合使用，以获得非常简洁的查询：

```
rows where cols a and b have overlapping values
and col c's values are less than col d's
In [245]: df.query('a in b and c < d')
Out[245]:
 a b c d
0 a a 2 6
1 a a 4 7
2 b a 1 6
4 c b 3 6
5 c b 0 2

pure Python
In [246]: df[df.b.isin(df.a) & (df.c < df.d)]
Out[246]:
 a b c d
0 a a 2 6
1 a a 4 7
2 b a 1 6
4 c b 3 6
5 c b 0 2
10 f c 0 6
11 f c 1 2
```

⋮ tip 注意

请注意 `in` 并在Python中进行评估，因为它没有相应的操作。但是，只有 `/ expression` 本身在vanilla Python中进行评估。例如，在表达式中 `not in` `numexpr` `in` `not in`

```
df.query('a in b + c + d')
```

`(b + c + d)` 通过评估 `numexpr` 和随后的 `in` 操作在普通的Python评价。通常，任何可以使用的评估操作 `numexpr` 都是。

⋮

## ==运算符与list对象的特殊用法

一个比较 `list` 值的使用列 `==` `!=` 工程，以类似 `in` `not in`

```
In [247]: df.query('b == ["a", "b", "c"]')
```

```
Out[247]:
```

	a	b	c	d
0	a	a	2	6
1	a	a	4	7
2	b	a	1	6
3	b	a	2	1
4	c	b	3	6
5	c	b	0	2
6	d	b	3	3
7	d	b	2	1
8	e	c	4	3
9	e	c	2	0
10	f	c	0	6
11	f	c	1	2

```
pure Python
```

```
In [248]: df[df.b.isin(["a", "b", "c"])]
```

```
Out[248]:
```

	a	b	c	d
0	a	a	2	6
1	a	a	4	7
2	b	a	1	6
3	b	a	2	1
4	c	b	3	6
5	c	b	0	2
6	d	b	3	3
7	d	b	2	1
8	e	c	4	3
9	e	c	2	0
10	f	c	0	6
11	f	c	1	2

```
In [249]: df.query('c == [1, 2]')
```

```
Out[249]:
```

	a	b	c	d
0	a	a	2	6
2	b	a	1	6
3	b	a	2	1
7	d	b	2	1
9	e	c	2	0
11	f	c	1	2

```
In [250]: df.query('c != [1, 2]')
```

```
Out[250]:
```

	a	b	c	d
1	a	a	4	7
4	c	b	3	6
5	c	b	0	2
6	d	b	3	3
8	e	c	4	3
10	f	c	0	6

```
using in/not in
```

```
In [251]: df.query('[1, 2] in c')
```

```
Out[251]:
 a b c d
0 a a 2 6
2 b a 1 6
3 b a 2 1
7 d b 2 1
9 e c 2 0
11 f c 1 2
```

```
In [252]: df.query('[1, 2] not in c')
```

```
Out[252]:
 a b c d
1 a a 4 7
4 c b 3 6
5 c b 0 2
6 d b 3 3
8 e c 4 3
10 f c 0 6
```

```
pure Python
```

```
In [253]: df[df.c.isin([1, 2])]
```

```
Out[253]:
 a b c d
0 a a 2 6
2 b a 1 6
3 b a 2 1
7 d b 2 1
9 e c 2 0
11 f c 1 2
```

## 布尔运算符

您可以使用单词 `not` 或 `~` 运算符否定布尔表达式。

```
In [254]: df = pd.DataFrame(np.random.rand(n, 3), columns=list('abc'))
```

```
In [255]: df['bools'] = np.random.rand(len(df)) > 0.5
```

```
In [256]: df.query('~bools')
```

```
Out[256]:
```

	a	b	c	bools
2	0.697753	0.212799	0.329209	False
7	0.275396	0.691034	0.826619	False
8	0.190649	0.558748	0.262467	False

```
In [257]: df.query('not bools')
```

```
Out[257]:
```

	a	b	c	bools
2	0.697753	0.212799	0.329209	False
7	0.275396	0.691034	0.826619	False
8	0.190649	0.558748	0.262467	False

```
In [258]: df.query('not bools') == df[~df.bools]
```

```
Out[258]:
```

	a	b	c	bools
2	True	True	True	True
7	True	True	True	True
8	True	True	True	True

当然，表达式也可以是任意复杂的：

```
short query syntax
```

```
In [259]: shorter = df.query('a < b < c and (not bools) or bools > 2')
```

```
equivalent in pure Python
```

```
In [260]: longer = df[(df.a < df.b) & (df.b < df.c) & (~df.bools) | (df.bools > 2)]
```

```
In [261]: shorter
```

```
Out[261]:
```

	a	b	c	bools
7	0.275396	0.691034	0.826619	False

```
In [262]: longer
```

```
Out[262]:
```

	a	b	c	bools
7	0.275396	0.691034	0.826619	False

```
In [263]: shorter == longer
```

```
Out[263]:
```

	a	b	c	bools
7	True	True	True	True

## 的表现 ¶ query()

`DataFrame.query()` 对于大型帧，使用比Python略快。

query-perf

⋮ tip 注意

如果您的框架超过大约200,000行，您将只看到使用 `numexpr` 引擎的性能优势 `DataFrame.query()`。

`query-perf-small`

⋮

此图是使用 `DataFrame` 3列创建的，每列包含使用生成的浮点值 `numpy.random.randn()`。

## 重复数据

---

如果要识别和删除`DataFrame`中的重复行，有两种方法可以提供帮助：`duplicated` 和 `drop_duplicates`。每个都将用于标识重复行的列作为参数。

- `duplicated` 返回一个布尔向量，其长度为行数，表示行是否重复。
- `drop_duplicates` 删除重复的行。

默认情况下，重复集的第一个观察行被认为是唯一的，但每个方法都有一个 `keep` 参数来指定要保留的目标。

- `keep='first'`（默认值）：标记/删除重复项，第一次出现除外。
- `keep='last'`：标记/删除重复项，除了最后一次出现。
- `keep=False`：标记/删除所有重复项。



```
In [264]: df2 = pd.DataFrame({'a': ['one', 'one', 'two', 'two', 'two', 'three', 'four'],
.....: 'b': ['x', 'y', 'x', 'y', 'x', 'x', 'x'],
.....: 'c': np.random.randn(7)})
.....:
```

```
In [265]: df2
```

```
Out[265]:
```

	a	b	c
0	one	x	-1.067137
1	one	y	0.309500
2	two	x	-0.211056
3	two	y	-1.842023
4	two	x	-0.390820
5	three	x	-1.964475
6	four	x	1.298329

```
In [266]: df2.duplicated('a')
```

```
Out[266]:
```

0	False
1	True
2	False
3	True
4	True
5	False
6	False

dtype: bool

```
In [267]: df2.duplicated('a', keep='last')
```

```
Out[267]:
```

0	True
1	False
2	True
3	True
4	False
5	False
6	False

dtype: bool

```
In [268]: df2.duplicated('a', keep=False)
```

```
Out[268]:
```

0	True
1	True
2	True
3	True
4	True
5	False
6	False

dtype: bool

```
In [269]: df2.drop_duplicates('a')
```

```
Out[269]:
```

	a	b	c
0	one	x	-1.067137
2	two	x	-0.211056
5	three	x	-1.964475

```

6 four x 1.298329

In [270]: df2.drop_duplicates('a', keep='last')
Out[270]:
 a b c
1 one y 0.309500
4 two x -0.390820
5 three x -1.964475
6 four x 1.298329

In [271]: df2.drop_duplicates('a', keep=False)
Out[271]:
 a b c
5 three x -1.964475
6 four x 1.298329

```

此外，您可以传递列表列表以识别重复。

```

In [272]: df2.duplicated(['a', 'b'])
Out[272]:
0 False
1 False
2 False
3 False
4 True
5 False
6 False
dtype: bool

In [273]: df2.drop_duplicates(['a', 'b'])
Out[273]:
 a b c
0 one x -1.067137
1 one y 0.309500
2 two x -0.211056
3 two y -1.842023
5 three x -1.964475
6 four x 1.298329

```

要按索引值删除重复项，请使用 `Index.duplicated` 然后执行切片。 `keep` 参数可以使用相同的选项集。

```
In [274]: df3 = pd.DataFrame({'a': np.arange(6),
.....: 'b': np.random.randn(6)},
.....: index=['a', 'a', 'b', 'c', 'b', 'a'])
.....:
```

```
In [275]: df3
```

```
Out[275]:
```

	a	b
a 0	1.440455	
a 1	2.456086	
b 2	1.038402	
c 3	-0.894409	
b 4	0.683536	
a 5	3.082764	

```
In [276]: df3.index.duplicated()
```

```
Out[276]: array([False, True, False, False, True, True])
```

```
In [277]: df3[~df3.index.duplicated()]
```

```
Out[277]:
```

	a	b
a 0	1.440455	
b 2	1.038402	
c 3	-0.894409	

```
In [278]: df3[~df3.index.duplicated(keep='last')]
```

```
Out[278]:
```

	a	b
c 3	-0.894409	
b 4	0.683536	
a 5	3.082764	

```
In [279]: df3[~df3.index.duplicated(keep=False)]
```

```
Out[279]:
```

	a	b
c 3	-0.894409	

## 类字典`get()`方法

Series或DataFrame中的每一个都有一个`get`可以返回默认值的方法。

```
In [280]: s = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
```

```
In [281]: s.get('a') # equivalent to s['a']
```

```
Out[281]: 1
```

```
In [282]: s.get('x', default=-1)
```

```
Out[282]: -1
```

## 该 `lookup()` 方法

有时，您希望在给定一系列行标签和列标签的情况下提取一组值，并且该 `lookup` 方法允许此操作并返回 NumPy 数组。例如：

```
In [283]: dflookup = pd.DataFrame(np.random.rand(20, 4), columns = ['A', 'B', 'C', 'D'])

In [284]: dflookup.lookup(list(range(0, 10, 2)), ['B', 'C', 'A', 'B', 'D'])
Out[284]: array([0.3506, 0.4779, 0.4825, 0.9197, 0.5019])
```

## 索引对象

pandas `Index` 类及其子类可以视为实现有序的多集合。允许重复。但是，如果您尝试将 `Index` 具有重复条目的对象转换为 `set`，则会引发异常。

`Index` 还提供了查找，数据对齐和重建索引所需的基础结构。`Index` 直接创建的最简单方法 是将一个 `list` 或其他序列传递给 `Index`：

```
In [285]: index = pd.Index(['e', 'd', 'a', 'b'])

In [286]: index
Out[286]: Index(['e', 'd', 'a', 'b'], dtype='object')

In [287]: 'd' in index
Out[287]: True
```

您还可以传递一个 `name` 存储在索引中：

```
In [288]: index = pd.Index(['e', 'd', 'a', 'b'], name='something')

In [289]: index.name
Out[289]: 'something'
```

名称（如果已设置）将显示在控制台显示中：

```

In [290]: index = pd.Index(list(range(5)), name='rows')

In [291]: columns = pd.Index(['A', 'B', 'C'], name='cols')

In [292]: df = pd.DataFrame(np.random.randn(5, 3), index=index, columns=columns)

In [293]: df
Out[293]:
cols A B C
rows
0 1.295989 0.185778 0.436259
1 0.678101 0.311369 -0.528378
2 -0.674808 -1.103529 -0.656157
3 1.889957 2.076651 -1.102192
4 -1.211795 -0.791746 0.634724

In [294]: df['A']
Out[294]:
rows
0 1.295989
1 0.678101
2 -0.674808
3 1.889957
4 -1.211795
Name: A, dtype: float64

```

## 设置元数据

索引是“不可改变的大多是”，但它可以设置和改变它们的元数据，如指数 `name`（或为 `MultiIndex`，`levels` 和 `codes`）。

您可以使用 `rename`，`set_names`，`set_levels`，和 `set_codes` 直接设置这些属性。他们默认返回一份副本；但是，您可以指定 `inplace=True` 使数据更改到位。

有关 `MultiIndexes` 的使用，请参阅[高级索引](#)。

```

In [295]: ind = pd.Index([1, 2, 3])

In [296]: ind.rename("apple")
Out[296]: Int64Index([1, 2, 3], dtype='int64', name='apple')

In [297]: ind
Out[297]: Int64Index([1, 2, 3], dtype='int64')

In [298]: ind.set_names(["apple"], inplace=True)

In [299]: ind.name = "bob"

In [300]: ind
Out[300]: Int64Index([1, 2, 3], dtype='int64', name='bob')

```

`set_names`，`set_levels` 并且 `set_codes` 还采用可选 `level` 参数

```

In [301]: index = pd.MultiIndex.from_product([range(3), ['one', 'two']], names=['first', 'second'])

In [302]: index
Out[302]:
MultiIndex([(0, 'one'),
 (0, 'two'),
 (1, 'one'),
 (1, 'two'),
 (2, 'one'),
 (2, 'two')],
 names=['first', 'second'])

In [303]: index.levels[1]
Out[303]: Index(['one', 'two'], dtype='object', name='second')

In [304]: index.set_levels(["a", "b"], level=1)
Out[304]:
MultiIndex([(0, 'a'),
 (0, 'b'),
 (1, 'a'),
 (1, 'b'),
 (2, 'a'),
 (2, 'b')],
 names=['first', 'second'])

```

## 在Index对象上设置操作

两个主要业务是和。这些可以直接称为实例方法，也可以通过重载运算符使用。通过该方法提供差异。

```
union (|) ``intersection (&)``.difference()
```

```

In [305]: a = pd.Index(['c', 'b', 'a'])

In [306]: b = pd.Index(['c', 'e', 'd'])

In [307]: a | b
Out[307]: Index(['a', 'b', 'c', 'd', 'e'], dtype='object')

In [308]: a & b
Out[308]: Index(['c'], dtype='object')

In [309]: a.difference(b)
Out[309]: Index(['a', 'b'], dtype='object')

```

同时还提供了操作，它返回出现在任一元件或，但不是在两者。这相当于创建的索引，删除了重复项。

```
symmetric_difference (^) ``idx1 ``idx2 ``idx1.difference(idx2).union(idx2.difference(idx1))
```

```
In [310]: idx1 = pd.Index([1, 2, 3, 4])

In [311]: idx2 = pd.Index([2, 3, 4, 5])

In [312]: idx1.symmetric_difference(idx2)
Out[312]: Int64Index([1, 5], dtype='int64')

In [313]: idx1 ^ idx2
Out[313]: Int64Index([1, 5], dtype='int64')
```

::: tip 注意

来自设置操作的结果索引将按升序排序。

:::

在`Index.union()`具有不同dtypes的索引之间执行时，必须将索引强制转换为公共dtype。通常，虽然并非总是如此，但这是对象dtype。例外是在整数和浮点数据之间执行联合。在这种情况下，整数值将转换为float

```
In [314]: idx1 = pd.Index([0, 1, 2])

In [315]: idx2 = pd.Index([0.5, 1.5])

In [316]: idx1 | idx2
Out[316]: Float64Index([0.0, 0.5, 1.0, 1.5, 2.0], dtype='float64')
```

## 缺少值

即使 `Index` 可以保存缺失值（`NaN`），但如果您不想要任何意外结果，也应该避免使用。例如，某些操作会隐式排除缺失值。

`Index.fillna` 使用指定的标量值填充缺失值。

```

In [317]: idx1 = pd.Index([1, np.nan, 3, 4])

In [318]: idx1
Out[318]: Float64Index([1.0, nan, 3.0, 4.0], dtype='float64')

In [319]: idx1.fillna(2)
Out[319]: Float64Index([1.0, 2.0, 3.0, 4.0], dtype='float64')

In [320]: idx2 = pd.DatetimeIndex([pd.Timestamp('2011-01-01'),
.....: pd.NaT,
.....: pd.Timestamp('2011-01-03')])
.....:

In [321]: idx2
Out[321]: DatetimeIndex(['2011-01-01', 'NaT', '2011-01-03'], dtype='datetime64[ns]', freq=None)

In [322]: idx2.fillna(pd.Timestamp('2011-01-02'))
Out[322]: DatetimeIndex(['2011-01-01', '2011-01-02', '2011-01-03'], dtype='datetime64[ns]', freq=None)

```

## 设置/重置索引

---

有时您会将数据集加载或创建到DataFrame中，并希望在您已经完成之后添加索引。有几种不同的方式。

### 设置索引

DataFrame有一个`set_index()`方法，它采用列名（对于常规 Index）或列名列表（对于a MultiIndex）。要创建新的重新索引的DataFrame：



```

In [323]: data
Out[323]:
 a b c d
0 bar one z 1.0
1 bar two y 2.0
2 foo one x 3.0
3 foo two w 4.0

In [324]: indexed1 = data.set_index('c')

In [325]: indexed1
Out[325]:
 a b d
c
z bar one 1.0
y bar two 2.0
x foo one 3.0
w foo two 4.0

In [326]: indexed2 = data.set_index(['a', 'b'])

In [327]: indexed2
Out[327]:
 c d
a b
bar one z 1.0
 two y 2.0
foo one x 3.0
 two w 4.0

```

该 `append` 关键字选项让你保持现有索引并追加给列一个多指标：

```

In [328]: frame = data.set_index('c', drop=False)

In [329]: frame = frame.set_index(['a', 'b'], append=True)

In [330]: frame
Out[330]:
 c d
c a b
z bar one z 1.0
y bar two y 2.0
x foo one x 3.0
w foo two w 4.0

```

其他选项 `set_index` 允许您不删除索引列或就地添加索引（不创建新对象）：

```

In [331]: data.set_index('c', drop=False)
Out[331]:
 a b c d
c
z bar one z 1.0
y bar two y 2.0
x foo one x 3.0
w foo two w 4.0

In [332]: data.set_index(['a', 'b'], inplace=True)

In [333]: data
Out[333]:
 c d
a b
bar one z 1.0
 two y 2.0
foo one x 3.0
 two w 4.0

```

## 重置索引

为方便起见，DataFrame上有一个新函数，它将 `reset_index()` 索引值传输到DataFrame的列中并设置一个简单的整数索引。这是反向操作`set_index()`。

```

In [334]: data
Out[334]:
 c d
a b
bar one z 1.0
 two y 2.0
foo one x 3.0
 two w 4.0

In [335]: data.reset_index()
Out[335]:
 a b c d
0 bar one z 1.0
1 bar two y 2.0
2 foo one x 3.0
3 foo two w 4.0

```

输出更类似于SQL表或记录数组。从索引派生的列的名称是存储在 `names` 属性中的名称。

您可以使用 `level` 关键字仅删除索引的一部分：

```

In [336]: frame
Out[336]:
 c d
c a b
z bar one z 1.0
y bar two y 2.0
x foo one x 3.0
w foo two w 4.0

In [337]: frame.reset_index(level=1)
Out[337]:
 a c d
c b
z one bar z 1.0
y two bar y 2.0
x one foo x 3.0
w two foo w 4.0

```

`reset_index` 采用一个可选参数 `drop`，如果为 `true`，则只丢弃索引，而不是将索引值放在 `DataFrame` 的列中。

## 添加ad hoc索引

如果您自己创建索引，则可以将其分配给 `index` 字段：

```
data.index = index
```

## 返回视图与副本

在 `pandas` 对象中设置值时，必须注意避免调用所谓的对象。这是一个例子。 `chained indexing`

```

In [338]: dfmi = pd.DataFrame([list('abcd'),
.....: list('efgh'),
.....: list('ijkl'),
.....: list('mnop')],
.....: columns=pd.MultiIndex.from_product([['one', 'two'],
.....: ['first', 'second']]))
.....:

In [339]: dfmi
Out[339]:
 one two
first second first second
0 a b c d
1 e f g h
2 i j k l
3 m n o p

```

比较这两种访问方法：

```
In [340]: dfmi['one']['second']
Out[340]:
0 b
1 f
2 j
3 n
Name: second, dtype: object
```

```
In [341]: dfmi.loc[:, ('one', 'second')]
Out[341]:
0 b
1 f
2 j
3 n
Name: (one, second), dtype: object
```

这两者都产生相同的结果，所以你应该使用哪个？理解这些操作的顺序以及为什么方法2（`.loc`）比方法1（链接 `[]`）更受欢迎是有益的。

`dfmi['one']` 选择列的第一级并返回单索引的DataFrame。然后另一个Python操作 `dfmi_with_one['second']` 选择索引的系列 `'second'`。这由变量指示，`dfmi_with_one` 因为pandas将这些操作视为单独的事件。例如，单独调用 `__getitem__`，因此它必须将它们视为线性操作，它们一个接一个地发生。

对比这个 `df.loc[:, ('one', 'second')]` 将一个嵌套的元组传递 (`slice(None), ('one', 'second')`) 给一个单独的调用 `__getitem__`。这允许pandas将其作为单个实体来处理。此外，这种操作顺序可以明显更快，并且如果需要，允许人们对两个轴进行索引。

## 使用链式索引时为什么分配失败？

上一节中的问题只是一个性能问题。这是怎么回事与 `SettingWithCopy` 警告？当你做一些可能花费几毫秒的事情时，我们通常不会发出警告！

但事实证明，分配链式索引的产品具有固有的不可预测的结果。要看到这一点，请考虑Python解释器如何执行此代码：

```
dfmi.loc[:, ('one', 'second')] = value
becomes
dfmi.loc.__setitem__((slice(None), ('one', 'second'))), value)
```

但是这个代码的处理方式不同：

```
dfmi['one']['second'] = value
becomes
dfmi.__getitem__('one').__setitem__('second', value)
```

看到 `__getitem__` 那里？除了简单的情况之外，很难预测它是否会返回一个视图或一个副本（它取决于数组的内存布局，关于哪些pandas不能保证），因此是否 `__setitem__` 会修改 `dfmi` 或者是一个临时对象之后立即抛出。那什么 `SettingWithCopy` 是警告你！

∴ tip 注意

您可能想知道我们是否应该关注 `loc` 第一个示例中的属性。但 `dfmi.loc` 保证 `dfmi` 本身具有修改的索引行为，因此 `dfmi.loc.__getitem__` / 直接 `dfmi.loc.__setitem__` 操作 `dfmi`。当然，`dfmi.loc.__getitem__(idx)` 可能是一个视图或副本 `dfmi`。

∴

有时 `SettingWithCopy`，当没有明显的链式索引时，会出现警告。这些 `SettingWithCopy` 是旨在捕获的错误！Pandas可能会试图警告你，你已经这样做了：

```
def do_something(df):
 foo = df[['bar', 'baz']] # Is foo a view? A copy? Nobody knows!
 # ... many lines here ...
 # We don't know whether this will modify df or not!
 foo['quux'] = value
 return foo
```

哎呀！

## 评估订单事项

使用链式索引时，索引操作的顺序和类型会部分确定结果是原始对象的切片还是切片的副本。

Pandas有，`SettingWithCopyWarning` 因为分配一个切片的副本通常不是故意的，而是由链式索引引起的错误返回一个预期切片的副本。

如果您希望pandas或多或少地信任链接索引表达式的赋值，则可以将选项设置 `mode.chained_assignment` 为以下值之一：

- 'warn'，默认值表示 `SettingWithCopyWarning` 打印。
- 'raise' 意味着大Pandas会提出 `SettingWithCopyException` 你必须处理的事情。
- None 将完全压制警告。

```
In [342]: dfb = pd.DataFrame({'a': ['one', 'one', 'two',
.....: 'three', 'two', 'one', 'six'],
.....: 'c': np.arange(7)})
.....:

This will show the SettingWithCopyWarning
but the frame values will be set
In [343]: dfb['c'][dfb.a.str.startswith('o')] = 42
```

然而，这是在副本上运行，不起作用。

```
>>> pd.set_option('mode.chained_assignment','warn')
>>> dfb[dfb.a.str.startswith('o')]['c'] = 42
Traceback (most recent call last)
...
SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_index,col_indexer] = value instead
```

链式分配也可以在混合dtype帧中进行设置。

::: tip 注意

这些设置规则适用于所有 `.loc/.iloc`。

:::

这是正确的访问方法：

```
In [344]: dfc = pd.DataFrame({'A': ['aaa', 'bbb', 'ccc'], 'B': [1, 2, 3]})

In [345]: dfc.loc[0, 'A'] = 11

In [346]: dfc
Out[346]:
 A B
0 11 1
1 bbb 2
2 ccc 3
```

这有时会起作用，但不能保证，因此应该避免：

```
In [347]: dfc = dfc.copy()

In [348]: dfc['A'][0] = 111

In [349]: dfc
Out[349]:
 A B
0 111 1
1 bbb 2
2 ccc 3
```

这根本不起作用，所以应该避免：

```
>>> pd.set_option('mode.chained_assignment','raise')
>>> dfc.loc[0]['A'] = 1111
Traceback (most recent call last)
...
SettingWithCopyException:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_index,col_indexer] = value instead
```

::: danger 警告

链式分配警告/异常旨在通知用户可能无效的分配。可能存在误报; 无意中报告链式作业的情况。

:::

---

## 多层级索引和高级索引

本章节包含了[使用多层级索引](#) 以及 [其他高级索引特性](#)。

请参阅 [索引与选择数据](#)来获得更多的通用索引方面的帮助文档

::: danger 警告

基于实际的使用场景不同，返回的内容也会不尽相同（返回一个数据的副本，或者返回数据的引用）。有时，这种情况被称作连锁赋值，但是这种情况应当被尽力避免。参见[返回视图or返回副本](#)。

:::

参见 [cookbook](#)，获取更多高级的使用技巧。

---

### 分层索引（多层级索引）

分层/多级索引在处理复杂的数据分析和数据操作方面为开发者奠定了基础，尤其是在处理高纬度数据处理上。本质上，它使您能够在较低维度的数据结构(如 `Series (1d)`和 `DataFrame (2d)`)中存储和操作任意维数的数据。

在本节中，我们将展示“层次”索引的确切含义，以及它如何与上面和前面部分描述的所有panda索引功能集成。稍后，在讨论[分组](#) 和[数据透视与重塑性数据](#)时，我们将展示一些重要的应用程序，以说明它如何帮助构建分析数据的结构。

请参阅[cookbook](#)，查看一些高级策略。

在0.24.0版本中的改变: `MultIndex.labels` 被更名为`MultiIndex.codes`，同时 `MultIndex.set_labels` 更名为 `MultiIndex.set_codes`。

### 创建多级索引和分层索引对象

`MultiIndex` 对象是[标准索引对象](#) 的分层模拟，标准索引对象通常将axis标签存储在panda对象中。您可以将 `MultiIndex` 看作一个元组数组，其中每个元组都是惟一的。可以从数组列表(使用 `MultiIndex.from_arrays()`)、元组数组(使用`MultiIndex.from_tuples()`)或交叉迭代器集(使用`MultiIndex.from_product()`)或者将一个 `DataFrame`(使用using `MultiIndex.from_frame()`)创建多索引。当传递一个元组列表时，索引构造函数将尝试返回一个 `MultiIndex`。下面的示例演示了初始化多索引的不同方法。

```
In [1]: arrays = [['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux'],
...: ['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two']]
...:
```

```
In [2]: tuples = list(zip(*arrays))
```

```
In [3]: tuples
```

```
Out[3]:
[('bar', 'one'),
 ('bar', 'two'),
 ('baz', 'one'),
 ('baz', 'two'),
 ('foo', 'one'),
 ('foo', 'two'),
 ('qux', 'one'),
 ('qux', 'two')]
```

```
In [4]: index = pd.MultiIndex.from_tuples(tuples, names=['first', 'second'])
```

```
In [5]: index
```

```
Out[5]:
MultiIndex([('bar', 'one'),
 ('bar', 'two'),
 ('baz', 'one'),
 ('baz', 'two'),
 ('foo', 'one'),
 ('foo', 'two'),
 ('qux', 'one'),
 ('qux', 'two')],
 names=['first', 'second'])
```

```
In [6]: s = pd.Series(np.random.randn(8), index=index)
```

```
In [7]: s
```

```
Out[7]:
first second
bar one 0.469112
 two -0.282863
baz one -1.509059
 two -1.135632
foo one 1.212112
 two -0.173215
qux one 0.119209
 two -1.044236
dtype: float64
```

当您想要在两个迭代器中对每个元素进行配对时，可以更容易地使用 `MultiIndex.from_product()` 方法:



```
In [8]: iterables = [['bar', 'baz', 'foo', 'qux'], ['one', 'two']]

In [9]: pd.MultiIndex.from_product(iterables, names=['first', 'second'])
Out[9]:
MultiIndex([('bar', 'one'),
 ('bar', 'two'),
 ('baz', 'one'),
 ('baz', 'two'),
 ('foo', 'one'),
 ('foo', 'two'),
 ('qux', 'one'),
 ('qux', 'two')],
 names=['first', 'second'])
```

还可以使用 `MultiIndex.from_frame()` 方法直接将一个 `DataFrame` 对象构造一个多索引。这是 `MultiIndex.to_frame()` 的一个补充方法。

0.24.0版本新增。

```
In [10]: df = pd.DataFrame([['bar', 'one'], ['bar', 'two'],
.....: ['foo', 'one'], ['foo', 'two']],
.....: columns=['first', 'second'])
.....:

In [11]: pd.MultiIndex.from_frame(df)
Out[11]:
MultiIndex([('bar', 'one'),
 ('bar', 'two'),
 ('foo', 'one'),
 ('foo', 'two')],
 names=['first', 'second'])
```

为了方便，您可以将数组列表直接传递到 `Series` 或 `DataFrame` 中，从而自动构造一个 `MultiIndex`：

```
In [12]: arrays = [np.array(['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux']),
....: np.array(['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two'])]
....:
```

```
In [13]: s = pd.Series(np.random.randn(8), index=arrays)
```

```
In [14]: s
```

```
Out[14]:
bar one -0.861849
 two -2.104569
baz one -0.494929
 two 1.071804
foo one 0.721555
 two -0.706771
qux one -1.039575
 two 0.271860
dtype: float64
```

```
In [15]: df = pd.DataFrame(np.random.randn(8, 4), index=arrays)
```

```
In [16]: df
```

```
Out[16]:
 0 1 2 3
bar one -0.424972 0.567020 0.276232 -1.087401
 two -0.673690 0.113648 -1.478427 0.524988
baz one 0.404705 0.577046 -1.715002 -1.039268
 two -0.370647 -1.157892 -1.344312 0.844885
foo one 1.075770 -0.109050 1.643563 -1.469388
 two 0.357021 -0.674600 -1.776904 -0.968914
qux one -1.294524 0.413738 0.276662 -0.472035
 two -0.013960 -0.362543 -0.006154 -0.923061
```

所 `MultiIndex` 构造函数都接受 `names` 参数，该参数存储级别本身的字符串名称。如果没有提供 `name` 属性，将分配 `None`：

```
In [17]: df.index.names
Out[17]: FrozenList([None, None])
```

此索引可以支持panda对象的任何轴，索引的**级别**由开发者决定：

```
In [18]: df = pd.DataFrame(np.random.randn(3, 8), index=['A', 'B', 'C'], columns=index)
```

```
In [19]: df
```

```
Out[19]:
```

	first	second	bar	one	two	baz	one	two	foo	one	two	qux	one	two
A			0.895717	0.805244	-1.206412	2.565646	1.431256	1.340309	-1.170299	-0.226169				
B			0.410835	0.813850	0.132003	-0.827317	-0.076467	-1.187678	1.130127	-1.436737				
C			-1.413681	1.607920	1.024180	0.569605	0.875906	-2.211372	0.974466	-2.006747				

```
In [20]: pd.DataFrame(np.random.randn(6, 6), index=index[:6], columns=index[:6])
```

```
Out[20]:
```

	first	second	bar	one	two	baz	one	two	foo	one	two
first	second										
bar	one	-0.410001	-0.078638	0.545952	-1.219217	-1.226825	0.769804				
	two	-1.281247	-0.727707	-0.121306	-0.097883	0.695775	0.341734				
baz	one	0.959726	-1.110336	-0.619976	0.149748	-0.732339	0.687738				
	two	0.176444	0.403310	-0.154951	0.301624	-2.179861	-1.369849				
foo	one	-0.954208	1.462696	-1.743161	-0.826591	-0.345352	1.314232				
	two	0.690579	0.995761	2.396780	0.014871	3.357427	-0.317441				

我们已经“稀疏化”了更高级别的索引，使控制台的输出更容易显示。注意，可以使用 `pandas.set_options()` 中的 `multi_sparse` 选项控制索引的显示方式：

```
In [21]: with pd.option_context('display.multi_sparse', False):
.....: df
.....:
```

值得记住的是，没有什么可以阻止您使用元组作为轴上的原子标签：

```
In [22]: pd.Series(np.random.randn(8), index=tuples)
```

```
Out[22]:
```

```
(bar, one) -1.236269
(bar, two) 0.896171
(baz, one) -0.487602
(baz, two) -0.082240
(foo, one) -2.182937
(foo, two) 0.380396
(qux, one) 0.084844
(qux, two) 0.432390
dtype: float64
```

`MultiIndex` 之所以重要，是因为它允许您进行分组、选择和重新构造操作，我们将在下面的文档和后续部分中进行描述。正如您将在后面的部分中看到的，您可以发现自己使用分层索引的数据，而不需要显式地创建一个 `MultiIndex`。然而，当从文件中加载数据时，您可能希望在准备数据集时生成自己的 `MultiIndex`。

## 重构层次标签

`get_level_values()` 方法将返回特定级别每个位置的标签向量：

```
In [23]: index.get_level_values(0)
Out[23]: Index(['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux'], dtype='object', name='first')

In [24]: index.get_level_values('second')
Out[24]: Index(['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two'], dtype='object', name='second')
```

## 基本索引轴上的多索引

层次索引的一个重要特性是，您可以通过 `partial` 标签来选择数据，该标签标识数据中的子组。**局部** 选择“降低”层次索引的级别，其结果完全类似于在常规数据 `Dataframe` 中选择列：

```
In [25]: df['bar']
Out[25]:
second one two
A 0.895717 0.805244
B 0.410835 0.813850
C -1.413681 1.607920

In [26]: df['bar', 'one']
Out[26]:
A 0.895717
B 0.410835
C -1.413681
Name: (bar, one), dtype: float64

In [27]: df['bar']['one']
Out[27]:
A 0.895717
B 0.410835
C -1.413681
Name: one, dtype: float64

In [28]: s['qux']
Out[28]:
one -1.039575
two 0.271860
dtype: float64
```

有关如何在更深层次上进行选择，请参见[具有层次索引的横切](#)。

## 定义不同层次索引

`MultiIndex` 保存了所有被定义的索引层级，即使它们实际上没有被使用。在切割索引时，您可能会注意到这一点。例如：

```
In [29]: df.columns.levels # original MultiIndex
Out[29]: FrozenList(['bar', 'baz', 'foo', 'qux'], ['one', 'two'])

In [30]: df[['foo', 'qux']].columns.levels # sliced
Out[30]: FrozenList(['bar', 'baz', 'foo', 'qux'], ['one', 'two'])
```

这样做是为了避免重新计算级别，从而使切片具有很高的性能。如果只想查看使用的级别，可以使用 `remove_unused_levels()` 方法。

```
In [31]: df[['foo', 'qux']].columns.to_numpy()
Out[31]:
array([('foo', 'one'), ('foo', 'two'), ('qux', 'one'), ('qux', 'two')],
 dtype=object)

for a specific level
In [32]: df[['foo', 'qux']].columns.get_level_values(0)
Out[32]: Index(['foo', 'foo', 'qux', 'qux'], dtype='object', name='first')
```

若要仅使用已使用的级别来重构 `MultiIndex`，可以使用 `remove_unused_levels()` 方法。

0.20.0中更新。

```
In [33]: new_mi = df[['foo', 'qux']].columns.remove_unused_levels()

In [34]: new_mi.levels
Out[34]: FrozenList(['foo', 'qux'], ['one', 'two'])
```

## 数据对齐和使用 `reindex`

在轴上具有 `MultiIndex` 的不同索引对象之间的操作将如您所期望的那样工作;数据对齐的工作原理与元组索引相同:

```
In [35]: s + s[:-2]
Out[35]:
bar one -1.723698
 two -4.209138
baz one -0.989859
 two 2.143608
foo one 1.443110
 two -1.413542
qux one NaN
 two NaN
dtype: float64

In [36]: s + s[:,2]
Out[36]:
bar one -1.723698
 two NaN
baz one -0.989859
 two NaN
foo one 1.443110
 two NaN
qux one -2.079150
 two NaN
dtype: float64
```

`Series/DataFrames` 对象的 `reindex()` 方法可以调用另一个 `MultiIndex`，甚至一个列表或数组元组:

```

In [37]: s.reindex(index[:3])
Out[37]:
first second
bar one -0.861849
 two -2.104569
baz one -0.494929
dtype: float64

In [38]: s.reindex([('foo', 'two'), ('bar', 'one'), ('qux', 'one'), ('baz', 'one')])
Out[38]:
foo two -0.706771
bar one -0.861849
qux one -1.039575
baz one -0.494929
dtype: float64

```

## 具有层次索引的高级索引方法

语法上，使用 `.loc` 方法，在高级索引中加入 `MultiIndex`（多层索引）是有一些挑战的，但是我们一直在尽力所能地去实现这个功能。简单来说，多层索引的索引键（keys）来自元组的格式。例如，下列代码将会按照你的期望工作：

```

In [39]: df = df.T
In [40]: df
Out[40]:
 A B C
first second
bar one 0.895717 0.410835 -1.413681
 two 0.805244 0.813850 1.607920
baz one -1.206412 0.132003 1.024180
 two 2.565646 -0.827317 0.569605
foo one 1.431256 -0.076467 0.875906
 two 1.340309 -1.187678 -2.211372
qux one -1.170299 1.130127 0.974466
 two -0.226169 -1.436737 -2.006747

In [41]: df.loc[('bar', 'two')]
Out[41]:
A 0.805244
B 0.813850
C 1.607920
Name: (bar, two), dtype: float64

```

注意 `df.loc['bar', 'two']` 也将会在这个用例中正常工作，但是这种便捷的简写方法总的来说是容易产生歧义的。

如果你也希望使用 `.loc` 对某个特定的列进行索引，你需要使用如下的元组样式：

```
In [42]: df.loc[('bar', 'two'), 'A']
Out[42]: 0.8052440253863785
```

你可以只输入元组的第一个元素，而不需要写出所有的多级索引的每一个层级。例如，你可以使用“局部”索引，来获得所有在第一层为 `bar` 的元素，参见下例：

```
df.loc['bar']
```

这种方式是对于更为冗长的方式 `df.loc[('bar',),,]` 的一个简写（在本例中，等同于 `df.loc['bar',,]`）

您也可以类似地使用“局部”切片。

```
In [43]: df.loc['baz':'foo']
Out[43]:
```

		A	B	C
first	second			
baz	one	-1.206412	0.132003	1.024180
	two	2.565646	-0.827317	0.569605
foo	one	1.431256	-0.076467	0.875906
	two	1.340309	-1.187678	-2.211372

您可以通过使用一个元组的切片，提供一个值的范围(a ‘range’ of values),来进行切片

```
In [44]: df.loc[('baz', 'two'):('qux', 'one')]
Out[44]:
```

		A	B	C
first	second			
baz	two	2.565646	-0.827317	0.569605
foo	one	1.431256	-0.076467	0.875906
	two	1.340309	-1.187678	-2.211372
qux	one	-1.170299	1.130127	0.974466

```
In [45]: df.loc[('baz', 'two'):'foo']
Out[45]:
```

		A	B	C
first	second			
baz	two	2.565646	-0.827317	0.569605
foo	one	1.431256	-0.076467	0.875906
	two	1.340309	-1.187678	-2.211372

类似于重命名索引 (reindexing)，您可以通过输入一个标签的元组来实现：

```
In [46]: df.loc[[('bar', 'two'), ('qux', 'one')]]
Out[46]:
```

		A	B	C
first	second			
bar	two	0.805244	0.813850	1.607920
qux	one	-1.170299	1.130127	0.974466

::: tip 小技巧

在pandas中，元组和列表，在索引时，是有区别的。一个元组会被识别为一个多层级的索引值（key），而列表被用于表明多个不同的索引值（several keys）。换句话说，元组是按照横向展开的，即水平层级（traversing levels），而列表是纵向的，即扫描层级（scanning levels）。

...

注意，一个元组构成的列表提供的是完整的多级索引，而一个列表构成的元组提供的是同一个级别中的多个值：

```
In [47]: s = pd.Series([1, 2, 3, 4, 5, 6],
....: index=pd.MultiIndex.from_product([["A", "B"], ["c", "d", "e"]]))
....:

In [48]: s.loc[[("A", "c"), ("B", "d")]] # list of tuples
Out[48]:
A c 1
B d 5
dtype: int64

In [49]: s.loc[(["A", "B"], ["c", "d"])] # tuple of lists
Out[49]:
A c 1
 d 2
B c 4
 d 5
dtype: int64
```

## 使用切片器

你可以使用多级索引器来切片一个``多级索引

你可以提供任意的选择器，就仿佛你按照标签索引一样，参见[按照标签索引](#)，包含切片，标签构成的列表，标签，和布尔值索引器。

你可以使用 `slice(None)` 来选择所有的该级别的内容。你不需要指明所有的深层级别，他们将按照 `slice(None)` 的方式来做默认推测。

一如既往，切片器的**两侧**都会被包含进来，因为这是按照标签索引的方式进行的。

::: danger 警告 你需要在 `.loc` 中声明所有的维度，这意味着同时包含**行索引**以及**列索引**。在一些情况下，索引器中的数据有可能会被错误地识别为在两个维度同时进行索引，而不是只对行进行多层级索引。

建议使用下列的方式：

```
df.loc[(slice('A1', 'A3'), ...), :] # noqa: E999
```

**不建议**使用下列的方式：

```
df.loc[(slice('A1', 'A3'), ...)] # noqa: E999
```



...

```
In [50]: def mklbl(prefix, n):
....: return ["%s%s" % (prefix, i) for i in range(n)]
....:

In [51]: miindex = pd.MultiIndex.from_product([mklbl('A', 4),
....: mklbl('B', 2),
....: mklbl('C', 4),
....: mklbl('D', 2)])
....:

In [52]: micolumns = pd.MultiIndex.from_tuples([('a', 'foo'), ('a', 'bar'),
....: ('b', 'foo'), ('b', 'bah')],
....: names=['lvl0', 'lvl1'])
....:

In [53]: dfmi = pd.DataFrame(np.arange(len(miindex) * len(micolumns))
....: .reshape((len(miindex), len(micolumns))),
....: index=miindex,
....: columns=micolumns).sort_index().sort_index(axis=1)
....:

In [54]: dfmi
Out[54]:
lvl0 a b
lvl1 bar foo bah foo
A0 B0 C0 D0 1 0 3 2
 D1 5 4 7 6
 C1 D0 9 8 11 10
 D1 13 12 15 14
 C2 D0 17 16 19 18
...
A3 B1 C1 D1 237 236 239 238
 C2 D0 241 240 243 242
 D1 245 244 247 246
 C3 D0 249 248 251 250
 D1 253 252 255 254

[64 rows x 4 columns]
```

使用切片，列表和标签来进行简单的多层级切片

```
In [55]: dfmi.loc[(slice('A1', 'A3'), slice(None), ['C1', 'C3']), :]
Out[55]:
lvl0 a b
lvl1 bar foo bah foo
A1 B0 C1 D0 73 72 75 74
 D1 77 76 79 78
 C3 D0 89 88 91 90
 D1 93 92 95 94
 B1 C1 D0 105 104 107 106
...
A3 B0 C3 D1 221 220 223 222
 B1 C1 D0 233 232 235 234
 D1 237 236 239 238
 C3 D0 249 248 251 250
 D1 253 252 255 254

[24 rows x 4 columns]
```

你可以使用 `pandas.IndexSlice`，即使用 `:`，一个更为符合习惯的语法，而不是使用 `slice(None)`。

```
In [56]: idx = pd.IndexSlice

In [57]: dfmi.loc[idx[:, :, ['C1', 'C3']], idx[:, 'foo']]
Out[57]:
lvl0 a b
lvl1 foo foo
A0 B0 C1 D0 8 10
 D1 12 14
 C3 D0 24 26
 D1 28 30
 B1 C1 D0 40 42
...
A3 B0 C3 D1 220 222
 B1 C1 D0 232 234
 D1 236 238
 C3 D0 248 250
 D1 252 254

[32 rows x 2 columns]
```

您可以使用这种方法在两个维度上同时实现非常复杂的选择。

```
In [58]: dfmi.loc['A1', (slice(None), 'foo')]
Out[58]:
```

lvl0		a	b	
lvl1		foo	foo	
B0	C0	D0	64	66
		D1	68	70
	C1	D0	72	74
		D1	76	78
	C2	D0	80	82
...		...	...	...
B1	C1	D1	108	110
	C2	D0	112	114
		D1	116	118
	C3	D0	120	122
		D1	124	126

[16 rows x 2 columns]

```
In [59]: dfmi.loc[idx[:, :, ['C1', 'C3']], idx[:, 'foo']]
Out[59]:
```

lvl0			a	b	
lvl1			foo	foo	
A0	B0	C1	D0	8	10
			D1	12	14
		C3	D0	24	26
			D1	28	30
	B1	C1	D0	40	42
...			...	...	...
A3	B0	C3	D1	220	222
	B1	C1	D0	232	234
			D1	236	238
		C3	D0	248	250
			D1	252	254

[32 rows x 2 columns]

使用布尔索引器，您可以对数值进行选择。

```
In [60]: mask = dfmi[('a', 'foo')] > 200

In [61]: dfmi.loc[idx[mask, :, ['C1', 'C3']], idx[:, 'foo']]
Out[61]:
```

lvl0			a	b	
lvl1			foo	foo	
A3	B0	C1	D1	204	206
		C3	D0	216	218
			D1	220	222
	B1	C1	D0	232	234
			D1	236	238
		C3	D0	248	250
			D1	252	254

您也可以使用 `.loc` 来明确您所希望的 维度 (axis)，从而只在一个维度上来进行切片。

```
In [62]: dfmi.loc(axis=0)[: , :, ['C1', 'C3']]
```

```
Out[62]:
```

lvl0				a		b	
lvl1				bar	foo	bah	foo
A0	B0	C1	D0	9	8	11	10
			D1	13	12	15	14
		C3	D0	25	24	27	26
			D1	29	28	31	30
	B1	C1	D0	41	40	43	42
...				...	...	...	...
A3	B0	C3	D1	221	220	223	222
	B1	C1	D0	233	232	235	234
			D1	237	236	239	238
		C3	D0	249	248	251	250
			D1	253	252	255	254

```
[32 rows x 4 columns]
```

进一步，您可以使用下列的方式来赋值

```
In [63]: df2 = dfmi.copy()
```

```
In [64]: df2.loc(axis=0)[: , :, ['C1', 'C3']] = -10
```

```
In [65]: df2
```

```
Out[65]:
```

lvl0				a		b	
lvl1				bar	foo	bah	foo
A0	B0	C0	D0	1	0	3	2
			D1	5	4	7	6
		C1	D0	-10	-10	-10	-10
			D1	-10	-10	-10	-10
		C2	D0	17	16	19	18
...				...	...	...	...
A3	B1	C1	D1	-10	-10	-10	-10
		C2	D0	241	240	243	242
			D1	245	244	247	246
		C3	D0	-10	-10	-10	-10
			D1	-10	-10	-10	-10

```
[64 rows x 4 columns]
```

您也可以在等号的右侧使用一个可以被“重命名”的对象来赋值

```
In [66]: df2 = dfmi.copy()

In [67]: df2.loc[idx[:, :, ['C1', 'C3']], :] = df2 * 1000

In [68]: df2
Out[68]:
```

lvl0				a		b	
lvl1				bar	foo	bah	foo
A0	B0	C0	D0	1	0	3	2
		D1	5	4	7	6	
		C1	D0	9000	8000	11000	10000
		D1	13000	12000	15000	14000	
C2	D0			17	16	19	18
...				...	...	...	...
A3	B1	C1	D1	237000	236000	239000	238000
		C2	D0	241	240	243	242
		D1		245	244	247	246
C3	D0			249000	248000	251000	250000
		D1		253000	252000	255000	254000

```
[64 rows x 4 columns]
```

## 交叉选择

`DataFrame` 的 `xs()` 方法接受一个额外的参数，从而可以简便地在某个特定的 多级索引 中的某一个层级进行数据的选取。

```
In [69]: df
Out[69]:
```

		A	B	C
first	second			
bar	one	0.895717	0.410835	-1.413681
	two	0.805244	0.813850	1.607920
baz	one	-1.206412	0.132003	1.024180
	two	2.565646	-0.827317	0.569605
foo	one	1.431256	-0.076467	0.875906
	two	1.340309	-1.187678	-2.211372
qux	one	-1.170299	1.130127	0.974466
	two	-0.226169	-1.436737	-2.006747

```
In [70]: df.xs('one', level='second')
Out[70]:
```

	A	B	C
first			
bar	0.895717	0.410835	-1.413681
baz	-1.206412	0.132003	1.024180
foo	1.431256	-0.076467	0.875906
qux	-1.170299	1.130127	0.974466

```
using the slicers
In [71]: df.loc[(slice(None), 'one'), :]
Out[71]:
```

		A	B	C
first	second			
bar	one	0.895717	0.410835	-1.413681
baz	one	-1.206412	0.132003	1.024180
foo	one	1.431256	-0.076467	0.875906
qux	one	-1.170299	1.130127	0.974466

您也可以使用 `xs()` 并填写坐标参数来选择列。

```
In [72]: df = df.T

In [73]: df.xs('one', level='second', axis=1)
Out[73]:
```

first	bar	baz	foo	qux
A	0.895717	-1.206412	1.431256	-1.170299
B	0.410835	0.132003	-0.076467	1.130127
C	-1.413681	1.024180	0.875906	0.974466

```
using the slicers
In [74]: df.loc[:, (slice(None), 'one')]
Out[74]:
```

first	bar	baz	foo	qux
second	one	one	one	one
A	0.895717	-1.206412	1.431256	-1.170299
B	0.410835	0.132003	-0.076467	1.130127
C	-1.413681	1.024180	0.875906	0.974466

`xs` 也接受多个键 (keys) 来进行选取

```
In [75]: df.xs(('one', 'bar'), level=('second', 'first'), axis=1)
Out[75]:
```

first	bar
second	one
A	0.895717
B	0.410835
C	-1.413681

```
using the slicers
In [76]: df.loc[:, ('bar', 'one')]
Out[76]:
```

A	0.895717
B	0.410835
C	-1.413681

Name: (bar, one), dtype: float64

您可以向 `xs` 传入 `drop_level=False` 来保留那些已经选取的层级。

```
In [77]: df.xs('one', level='second', axis=1, drop_level=False)
Out[77]:
```

first	bar	baz	foo	qux
second	one	one	one	one
A	0.895717	-1.206412	1.431256	-1.170299
B	0.410835	0.132003	-0.076467	1.130127
C	-1.413681	1.024180	0.875906	0.974466

请比较上面，使用 `drop_level=True` (默认值)的结果。

```
In [78]: df.xs('one', level='second', axis=1, drop_level=True)
Out[78]:
```

first	bar	baz	foo	qux
A	0.895717	-1.206412	1.431256	-1.170299
B	0.410835	0.132003	-0.076467	1.130127
C	-1.413681	1.024180	0.875906	0.974466

## 高级重命名索引及对齐

`level` 参数已经被加入到pandas对象中的 `reindex()` 和 `align()` 方法中。这将有助于沿着一个层级来广播值 (broadcast values) 。例如：

```
In [79]: midx = pd.MultiIndex(levels=[['zero', 'one'], ['x', 'y']],
....: codes=[[1, 1, 0, 0], [1, 0, 1, 0]])
....:
```

```
In [80]: df = pd.DataFrame(np.random.randn(4, 2), index=midx)
```

```
In [81]: df
```

```
Out[81]:
```

		0	1
one	y	1.519970	-0.493662
	x	0.600178	0.274230
zero	y	0.132885	-0.023688
	x	2.410179	1.450520

```
In [82]: df2 = df.mean(level=0)
```

```
In [83]: df2
```

```
Out[83]:
```

	0	1
one	1.060074	-0.109716
zero	1.271532	0.713416

```
In [84]: df2.reindex(df.index, level=0)
```

```
Out[84]:
```

		0	1
one	y	1.060074	-0.109716
	x	1.060074	-0.109716
zero	y	1.271532	0.713416
	x	1.271532	0.713416

```
aligning
```

```
In [85]: df_aligned, df2_aligned = df.align(df2, level=0)
```

```
In [86]: df_aligned
```

```
Out[86]:
```

		0	1
one	y	1.519970	-0.493662
	x	0.600178	0.274230
zero	y	0.132885	-0.023688
	x	2.410179	1.450520

```
In [87]: df2_aligned
```

```
Out[87]:
```

		0	1
one	y	1.060074	-0.109716
	x	1.060074	-0.109716
zero	y	1.271532	0.713416
	x	1.271532	0.713416

## 使用swapLevel来交换层级

swapLevel()函数可以用来交换两个层级



```
In [88]: df[:5]
Out[88]:
```

		0	1
one	y	1.519970	-0.493662
	x	0.600178	0.274230
zero	y	0.132885	-0.023688
	x	2.410179	1.450520

```
In [89]: df[:5].swaplevel(0, 1, axis=0)
Out[89]:
```

		0	1
y	one	1.519970	-0.493662
x	one	0.600178	0.274230
y	zero	0.132885	-0.023688
x	zero	2.410179	1.450520

## 使用reorder\_levels来进行层级重排序

`reorder_levels()`是一个更一般化的 `swaplevel` 方法，允许您用简单的一步来重排列索引的层级：

```
In [90]: df[:5].reorder_levels([1, 0], axis=0)
Out[90]:
```

		0	1
y	one	1.519970	-0.493662
x	one	0.600178	0.274230
y	zero	0.132885	-0.023688
x	zero	2.410179	1.450520

## 对索引和多层索引进行重命名

`rename()`方法可以用来重命名 多层索引，并且他经常被用于 DataFrame 的列名重命名。`renames` 的 `columns` 参数可以接受一个字典，从而仅仅重命名你希望更改名字的列。

```
In [91]: df.rename(columns={0: "col0", 1: "col1"})
Out[91]:
```

		col0	col1
one	y	1.519970	-0.493662
	x	0.600178	0.274230
zero	y	0.132885	-0.023688
	x	2.410179	1.450520

该方法也可以被用于重命名一些 DataFrame 的特定主索引的名称。

```
In [92]: df.rename(index={"one": "two", "y": "z"})
Out[92]:
```

		0	1
two	z	1.519970	-0.493662
	x	0.600178	0.274230
zero	z	0.132885	-0.023688
	x	2.410179	1.450520

`rename_axis()` 方法可以用于对 `Index` 或者 `MultiIndex` 进行重命名。尤其的，你可以明确 `MultiIndex` 中的不同层级的名称，这可以被用于在之后使用 `reset_index()`，把多层级索引的值转换为一个列

```
In [93]: df.rename_axis(index=['abc', 'def'])
Out[93]:
```

		0	1
abc	def		
one	y	1.519970	-0.493662
	x	0.600178	0.274230
zero	y	0.132885	-0.023688
	x	2.410179	1.450520

注意，`DataFrame` 的列也是一个索引，因此在 `rename_axis` 中使用 `columns` 参数，将会改变那个索引的名称

```
In [94]: df.rename_axis(columns="Cols").columns
Out[94]: RangeIndex(start=0, stop=2, step=1, name='Cols')
```

`rename` 和 `rename_axis` 都支持一个明确的字典，`Series` 或者一个映射函数，将标签，名称映射为新的值

## 对多索引进行排序

对于拥有多层级索引的对象来说，你可以通过排序来是的索引或切片更为高效。就如同其他任何的索引操作一样，你可以使用 `sort_index` 方法来实现。

```

In [95]: import random

In [96]: random.shuffle(tuples)

In [97]: s = pd.Series(np.random.randn(8), index=pd.MultiIndex.from_tuples(tuples))

In [98]: s
Out[98]:
baz one 0.206053
foo two -0.251905
qux one -2.213588
foo one 1.063327
bar two 1.266143
baz two 0.299368
bar one -0.863838
qux two 0.408204
dtype: float64

In [99]: s.sort_index()
Out[99]:
bar one -0.863838
 two 1.266143
baz one 0.206053
 two 0.299368
foo one 1.063327
 two -0.251905
qux one -2.213588
 two 0.408204
dtype: float64

In [100]: s.sort_index(level=0)
Out[100]:
bar one -0.863838
 two 1.266143
baz one 0.206053
 two 0.299368
foo one 1.063327
 two -0.251905
qux one -2.213588
 two 0.408204
dtype: float64

In [101]: s.sort_index(level=1)
Out[101]:
bar one -0.863838
baz one 0.206053
foo one 1.063327
qux one -2.213588
bar two 1.266143
baz two 0.299368
foo two -0.251905
qux two 0.408204
dtype: float64

```

如果你的多层级索引都被命名了的话，你也可以向 `sort_index` 传入一个层级名称。

```
In [102]: s.index.set_names(['L1', 'L2'], inplace=True)
```

```
In [103]: s.sort_index(level='L1')
```

```
Out[103]:
```

```
L1 L2
bar one -0.863838
 two 1.266143
baz one 0.206053
 two 0.299368
foo one 1.063327
 two -0.251905
qux one -2.213588
 two 0.408204
dtype: float64
```

```
In [104]: s.sort_index(level='L2')
```

```
Out[104]:
```

```
L1 L2
bar one -0.863838
baz one 0.206053
foo one 1.063327
qux one -2.213588
bar two 1.266143
baz two 0.299368
foo two -0.251905
qux two 0.408204
dtype: float64
```

对于多维度的对象来说，你也可以对任意的维度来进行索引，只要他们是具有 多层次索引 的：

```
In [105]: df.T.sort_index(level=1, axis=1)
```

```
Out[105]:
```

```
 one zero one zero
 x x y y
0 0.600178 2.410179 1.519970 0.132885
1 0.274230 1.450520 -0.493662 -0.023688
```

即便数据没有排序，你仍然可以对他们进行索引，但是索引的效率会极大降低，并且也会抛出 `PerformanceWarning` 警告。而且，这将返回一个数据的副本而非一个数据的视图：

```
In [106]: dfm = pd.DataFrame({'jim': [0, 0, 1, 1],
.....: 'joe': ['x', 'x', 'z', 'y'],
.....: 'jolie': np.random.rand(4)})
.....:
```

```
In [107]: dfm = dfm.set_index(['jim', 'joe'])
```

```
In [108]: dfm
```

```
Out[108]:
```

		jolie
jim	joe	
0	x	0.490671
	x	0.120248
1	z	0.537020
	y	0.110968

```
In [4]: dfm.loc[(1, 'z')]
```

PerformanceWarning: indexing past lexsort depth may impact performance.

```
Out[4]:
```

		jolie
jim	joe	
1	z	0.64094

另外，如果你试图索引一个没有完全lexsorted的对象，你将会碰到如下的错误：

```
In [5]: dfm.loc[(0, 'y'):(1, 'z')]
```

UnsortedIndexError: 'Key length (2) was greater than MultiIndex lexsort depth (1)'

在 `MultiIndex` 上使用 `is_lexsorted()` 方法，你可以查看这个索引是否已经被排序。而使用 `lexsort_depth` 属性则可以返回排序的深度

```
In [109]: dfm.index.is_lexsorted()
```

```
Out[109]: False
```

```
In [110]: dfm.index.lexsort_depth
```

```
Out[110]: 1
```

```

In [111]: dfm = dfm.sort_index()

In [112]: dfm
Out[112]:
 jolie
jim joe
0 x 0.490671
 x 0.120248
1 y 0.110968
 z 0.537020

In [113]: dfm.index.is_lexsorted()
Out[113]: True

In [114]: dfm.index.lexsort_depth
Out[114]: 2

```

现在，你的选择就可以正常工作了。

```

In [115]: dfm.loc[(0, 'y'):(1, 'z')]
Out[115]:
 jolie
jim joe
1 y 0.110968
 z 0.537020

```

## Take方法

与 NumPy 的 `ndarrays` 相似，pandas 的 `Index`，`Series`，和 `DataFrame` 也提供 `take()` 方法。他可以沿着某个维度，按照给定的索引取回所有的元素。这个给定的索引必须要是一个由整数组成的列表或者 `ndarray`，用以指明在索引中的位置。`take` 也可以接受负整数，作为相对于结尾的相对位置。

```

In [116]: index = pd.Index(np.random.randint(0, 1000, 10))

In [117]: index
Out[117]: Int64Index([214, 502, 712, 567, 786, 175, 993, 133, 758, 329], dtype='int64')

In [118]: positions = [0, 9, 3]

In [119]: index[positions]
Out[119]: Int64Index([214, 329, 567], dtype='int64')

In [120]: index.take(positions)
Out[120]: Int64Index([214, 329, 567], dtype='int64')

In [121]: ser = pd.Series(np.random.randn(10))

In [122]: ser.iloc[positions]
Out[122]:
0 -0.179666
9 1.824375
3 0.392149
dtype: float64

In [123]: ser.take(positions)
Out[123]:
0 -0.179666
9 1.824375
3 0.392149
dtype: float64

```

对于 DataFrames 来说，这个给定的索引应当是一个一维列表或者 ndarray，用于指明行或者列的位置。

```

In [124]: frm = pd.DataFrame(np.random.randn(5, 3))

In [125]: frm.take([1, 4, 3])
Out[125]:
 0 1 2
1 -1.237881 0.106854 -1.276829
4 0.629675 -1.425966 1.857704
3 0.979542 -1.633678 0.615855

In [126]: frm.take([0, 2], axis=1)
Out[126]:
 0 2
0 0.595974 0.601544
1 -1.237881 -1.276829
2 -0.767101 1.499591
3 0.979542 0.615855
4 0.629675 1.857704

```

需要注意的是，pandas 对象的 `take` 方法并不会正常地工作在布尔索引上，并且有可能会返回一切意外的结果。

```

In [127]: arr = np.random.randn(10)

In [128]: arr.take([False, False, True, True])
Out[128]: array([-1.1935, -1.1935, 0.6775, 0.6775])

In [129]: arr[[0, 1]]
Out[129]: array([-1.1935, 0.6775])

In [130]: ser = pd.Series(np.random.randn(10))

In [131]: ser.take([False, False, True, True])
Out[131]:
0 0.233141
0 0.233141
1 -0.223540
1 -0.223540
dtype: float64

In [132]: ser.iloc[[0, 1]]
Out[132]:
0 0.233141
1 -0.223540
dtype: float64

```

最后，关于性能方面的小建议，因为 `take` 方法处理的是一个范围更窄的输入，因此会比话实索引（fancy indexing）的速度快很多。

```

In [133]: arr = np.random.randn(10000, 5)

In [134]: indexer = np.arange(10000)

In [135]: random.shuffle(indexer)

In [136]: %timeit arr[indexer]
.....: %timeit arr.take(indexer, axis=0)
.....:
152 us +- 988 ns per loop (mean +- std. dev. of 7 runs, 10000 loops each)
41.7 us +- 204 ns per loop (mean +- std. dev. of 7 runs, 10000 loops each)

```

```

In [137]: ser = pd.Series(arr[:, 0])

In [138]: %timeit ser.iloc[indexer]
.....: %timeit ser.take(indexer)
.....:
120 us +- 1.05 us per loop (mean +- std. dev. of 7 runs, 10000 loops each)
110 us +- 795 ns per loop (mean +- std. dev. of 7 runs, 10000 loops each)

```

## 索引类型

我们在前面已经较为深入地探讨过了多层索引。你可以在 [这里](#)，可以找到关于 `DatetimeIndex` 和 `PeriodIndex` 的说明文件。在 [这里](#)，你可以找到关于 `TimedeltaIndex` 的说明。



In the following sub-sections we will highlight some other index types.

下面的一个子章节，我们将会着重探讨另外的一些索引的类型。

## 分类索引

`CategoricalIndex` 分类索引 这种索引类型非常适合有重复的索引。这是一个围绕 `Categorical` 而创建的容器。这可以非常高效地存储和索引的具有大量重复元素的索引。

```
In [139]: from pandas.api.types import CategoricalDtype

In [140]: df = pd.DataFrame({'A': np.arange(6),
.....: 'B': list('aabbca')})
.....:

In [141]: df['B'] = df['B'].astype(CategoricalDtype(list('cab')))

In [142]: df
Out[142]:
 A B
0 0 a
1 1 a
2 2 b
3 3 b
4 4 c
5 5 a

In [143]: df.dtypes
Out[143]:
A int64
B category
dtype: object

In [144]: df.B.cat.categories
Out[144]: Index(['c', 'a', 'b'], dtype='object')
```

通过设置索引将会建立一个 `CategoricalIndex` 分类索引。

```
In [145]: df2 = df.set_index('B')

In [146]: df2.index
Out[146]: CategoricalIndex(['a', 'a', 'b', 'b', 'c', 'a'], categories=['c', 'a', 'b'], ordered=False,
 name='B', dtype='category')
```

使用 `__getitem__/.iloc/.loc` 进行索引，在含有重复值的索引上的工作原理相似。索引值**必须**在一个分类中，否则将会引发 `KeyError` 错误。

```
In [147]: df2.loc['a']
Out[147]:
A
B
a 0
a 1
a 5
```

`CategoricalIndex` 在索引之后也会被**保留**:

```
In [148]: df2.loc['a'].index
Out[148]: CategoricalIndex(['a', 'a', 'a'], categories=['c', 'a', 'b'], ordered=False, name='B',
 dtype='category')
```

索引排序将会按照类别清单中的顺序进行（我们已经基于 `CategoricalDtype(list('cab'))` 建立了一个索引，因此排序的顺序是 `cab`）

```
In [149]: df2.sort_index()
Out[149]:
A
B
c 4
a 0
a 1
a 5
b 2
b 3
```

分组操作（`Groupby`）也会保留索引的全部信息。

```
In [150]: df2.groupby(level=0).sum()
Out[150]:
A
B
c 4
a 6
b 5

In [151]: df2.groupby(level=0).sum().index
Out[151]: CategoricalIndex(['c', 'a', 'b'], categories=['c', 'a', 'b'], ordered=False, name='B',
 dtype='category')
```

重设索引的操作将会根据输入的索引值返回一个索引。传入一个列表，将会返回一个最普通的 `Index`；如果使用类别对象 `Categorical`，则会返回一个分类索引 `CategoricalIndex`，按照其中**传入的**类别值 `Categorical` `dtype`来进行索引。正如同你可以对**任意**pandas的索引进行重新索引一样，这将允许你随意索引任意的索引值，即便它们并**不存在**在你的类别对象中。

```

In [152]: df2.reindex(['a', 'e'])
Out[152]:
 A
B
a 0.0
a 1.0
a 5.0
e NaN

In [153]: df2.reindex(['a', 'e']).index
Out[153]: Index(['a', 'a', 'a', 'e'], dtype='object', name='B')

In [154]: df2.reindex(pd.Categorical(['a', 'e'], categories=list('abcde'))))
Out[154]:
 A
B
a 0.0
a 1.0
a 5.0
e NaN

In [155]: df2.reindex(pd.Categorical(['a', 'e'], categories=list('abcde'))).index
Out[155]: CategoricalIndex(['a', 'a', 'a', 'e'], categories=['a', 'b', 'c', 'd', 'e'], ordered=False,
 name='B', dtype='category')

```

⋮ danger 警告

对于一个 `分类索引` 的对象进行变形或者比较操作，一定要确保他们的索引包含相同的列别，否则将会出发类型错误 `TypeError`

```

In [9]: df3 = pd.DataFrame({'A': np.arange(6), 'B': pd.Series(list('aabbca')).astype('category')})

In [11]: df3 = df3.set_index('B')

In [11]: df3.index
Out[11]: CategoricalIndex(['a', 'a', 'b', 'b', 'c', 'a'], categories=['a', 'b', 'c'], ordered=False,
 name='B', dtype='category')

In [12]: pd.concat([df2, df3])
TypeError: categories must match existing categories when appending

```

⋮

## 64位整型索引和范围索引

⋮ danger 警告

使用浮点数进行基于数值的索引已经再0.18.0的版本中进行了声明。想查看更改的汇总，请参见 [这里](#)。 ⋮

**Int64Index** 64位整型索引是pandas中的一种非常基本的索引操作。这是一个不可变的数组组成的一个有序的，可切片的集合。再0.18.0之前， `Int64Index` 是会为所有 `NDFrame` 对象提供默认的索引。

`RangeIndex` 范围索引是 64 位整型索引的子集，在 v0.18.0 版本加入。现在由范围索引来为所有的 `NDFrame` 对象提供默认索引。`RangeIndex` 是一个对于 `Int64Index` 的优化版本，能够提供一个有序且严格单调的集合。这个索引与 python 的 `range types` 是相似的

## 64 位浮点索引

默认情况下，当传入浮点数、或者浮点整型混合数的时候，一个 64 位浮点索引 `Float64Index` 将会自动被建立。这样将能够确保一个存粹而统一的基于标签的索引切片行为，这样 `[], ix, loc` 对于标量索引和切片的工作行为将会完全一致。

```
In [156]: indexf = pd.Index([1.5, 2, 3, 4.5, 5])

In [157]: indexf
Out[157]: Float64Index([1.5, 2.0, 3.0, 4.5, 5.0], dtype='float64')

In [158]: sf = pd.Series(range(5), index=indexf)

In [159]: sf
Out[159]:
1.5 0
2.0 1
3.0 2
4.5 3
5.0 4
dtype: int64
```

标量选择对于 `[], .loc` 永远都是基于标签的。一个整型将会自动匹配一个浮点标签（例如，`3` 等于 `3.0`）

```
In [160]: sf[3]
Out[160]: 2

In [161]: sf[3.0]
Out[161]: 2

In [162]: sf.loc[3]
Out[162]: 2

In [163]: sf.loc[3.0]
Out[163]: 2
```

唯一能够通过位置进行索引的方式是通过 `iloc` 方法。

```
In [164]: sf.iloc[3]
Out[164]: 3
```

一个找不到的标量索引会触发一个 `KeyError` 错误。当使用 `[], ix, loc` 是，切片操作优先会选择索引的值，但是 `iloc` 永远都会按位置索引。唯一的例外是使用布尔索引，此时将始终按位置选择。

```

In [165]: sf[2:4]
Out[165]:
2.0 1
3.0 2
dtype: int64

In [166]: sf.loc[2:4]
Out[166]:
2.0 1
3.0 2
dtype: int64

In [167]: sf.iloc[2:4]
Out[167]:
3.0 2
4.5 3
dtype: int64

```

如果你使用的是浮点数索引，那么使用浮点数切片也是可以执行的。

```

In [168]: sf[2.1:4.6]
Out[168]:
3.0 2
4.5 3
dtype: int64

In [169]: sf.loc[2.1:4.6]
Out[169]:
3.0 2
4.5 3
dtype: int64

```

在非浮点数中，如果使用浮点索引，将会触发 `TypeError` 错误。

```

In [1]: pd.Series(range(5))[3.5]
TypeError: the label [3.5] is not a proper indexer for this index type (Int64Index)

In [1]: pd.Series(range(5))[3.5:4.5]
TypeError: the slice start [3.5] is not a proper indexer for this index type (Int64Index)

```

::: danger 警告

从0.18.0开始，`.iloc` 将不能够使用标量浮点数进行索引，因此下列操作将触发 `TypeError` 错误。

```

In [3]: pd.Series(range(5)).iloc[3.0]
TypeError: cannot do positional indexing on <class 'pandas.indexes.range.RangeIndex'>
 with these indexers [3.0] of <type 'float'>

```

:::

这里有一个典型的场景来使用这种类型的索引方式。设想你有一个不规范的类timedelta的索引方案，但是日期是按照浮点数的方式记录的。这将会导致（例如）毫秒级的延迟。

```
In [170]: dfir = pd.concat([pd.DataFrame(np.random.randn(5, 2),
.....: index=np.arange(5) * 250.0,
.....: columns=list('AB')),
.....: pd.DataFrame(np.random.randn(6, 2),
.....: index=np.arange(4, 10) * 250.1,
.....: columns=list('AB'))])
```

```
In [171]: dfir
```

```
Out[171]:
```

	A	B
0.0	-0.435772	-1.188928
250.0	-0.808286	-0.284634
500.0	-1.815703	1.347213
750.0	-0.243487	0.514704
1000.0	1.162969	-0.287725
1000.4	-0.179734	0.993962
1250.5	-0.212673	0.909872
1500.6	-0.733333	-0.349893
1750.7	0.456434	-0.306735
2000.8	0.553396	0.166221
2250.9	-0.101684	-0.734907

因此选择操作将总是按照值来进行所有的选择工作，

```
In [172]: dfir[0:1000.4]
```

```
Out[172]:
```

	A	B
0.0	-0.435772	-1.188928
250.0	-0.808286	-0.284634
500.0	-1.815703	1.347213
750.0	-0.243487	0.514704
1000.0	1.162969	-0.287725
1000.4	-0.179734	0.993962

```
In [173]: dfir.loc[0:1001, 'A']
```

```
Out[173]:
```

0.0	-0.435772
250.0	-0.808286
500.0	-1.815703
750.0	-0.243487
1000.0	1.162969
1000.4	-0.179734

Name: A, dtype: float64

```
In [174]: dfir.loc[1000.4]
```

```
Out[174]:
```

A	-0.179734
B	0.993962

Name: 1000.4, dtype: float64

你可以返回第一秒（1000毫秒）的数据：

```
In [175]: dfir[0:1000]
Out[175]:
```

	A	B
0.0	-0.435772	-1.188928
250.0	-0.808286	-0.284634
500.0	-1.815703	1.347213
750.0	-0.243487	0.514704
1000.0	1.162969	-0.287725

如果你想要使用基于整型的选择，你应该使用 `iloc`：

```
In [176]: dfir.iloc[0:5]
Out[176]:
```

	A	B
0.0	-0.435772	-1.188928
250.0	-0.808286	-0.284634
500.0	-1.815703	1.347213
750.0	-0.243487	0.514704
1000.0	1.162969	-0.287725

## 间隔索引

0.20.0中新加入 `IntervalIndex`和它自己特有的 `IntervalDtype` 以及 `Interval` 标量类型，在pandas中，间隔数据是获得头等支持的。

`IntervalIndex` 间隔索引允许一些唯一的索引，并且也是 `cut()` 和 `qcut()`的返回类型

### 使用间隔索引来进行数据索引

```
In [177]: df = pd.DataFrame({'A': [1, 2, 3, 4]},
.....: index=pd.IntervalIndex.from_breaks([0, 1, 2, 3, 4]))
.....:

In [178]: df
Out[178]:
```

A
(0, 1] 1
(1, 2] 2
(2, 3] 3
(3, 4] 4

在间隔序列上使用基于标签的索引 `.loc` ，正如你所预料到的，将会选择那个特定的间隔

```
In [179]: df.loc[2]
Out[179]:
A 2
Name: (1, 2], dtype: int64
```

```
In [180]: df.loc[[2, 3]]
Out[180]:
 A
(1, 2] 2
(2, 3] 3
```

如果你选取了一个标签，被包含在间隔当中，这个间隔也将会被选择

```
In [181]: df.loc[2.5]
Out[181]:
A 3
Name: (2, 3], dtype: int64
```

```
In [182]: df.loc[[2.5, 3.5]]
Out[182]:
 A
(2, 3] 3
(3, 4] 4
```

使用 `Interval` 来选择，将只返回严格匹配（从pandas0.25.0开始）。

```
In [183]: df.loc[pd.Interval(1, 2)]
Out[183]:
A 2
Name: (1, 2], dtype: int64
```

试图选择一个没有被严格包含在 `IntervalIndex` 内的区间 `Interval`，将会出发 `KeyError` 错误。

```
In [7]: df.loc[pd.Interval(0.5, 2.5)]

KeyError: Interval(0.5, 2.5, closed='right')
```

可以使用`overlaps()`来创建一个布尔选择器，来选中所有与 给定区间 (`Interval`) 重复的所有区间。

```
In [184]: idxr = df.index.overlaps(pd.Interval(0.5, 2.5))

In [185]: idxr
Out[185]: array([True, True, True, False])

In [186]: df[idxr]
Out[186]:
 A
(0, 1] 1
(1, 2] 2
(2, 3] 3
```



## 使用 cut 和 qcut来为数据分块

`cut()` 和 `qcut()` 都将返回一个分类 `Categorical` 对象，并且每个分块区域都会以 分类索引 `IntervalIndex` 的方式被创建并保存在它的 `.categories` 属性中。

```
In [187]: c = pd.cut(range(4), bins=2)

In [188]: c
Out[188]:
[(-0.003, 1.5], (-0.003, 1.5], (1.5, 3.0], (1.5, 3.0]]
Categories (2, interval[float64]): [(-0.003, 1.5] < (1.5, 3.0]]

In [189]: c.categories
Out[189]:
IntervalIndex([(-0.003, 1.5], (1.5, 3.0]],
 closed='right',
 dtype='interval[float64]')
```

`cut()` 也可以接受一个 `IntervalIndex` 作为他的 `bins` 参数，这样可以使用一个非常有用的pandas的写法。首先，我们调用 `cut()` 在一些数据上面，并且将 `bins` 设置为某一个固定的数，从而生成bins。

随后，我们可以在其他的数据上调用 `cut()`，并传入 `.categories` 的值，作为 `bins` 参数。这样新的数据就也将会被分配到同样的bins里面

```
In [190]: pd.cut([0, 3, 5, 1], bins=c.categories)
Out[190]:
[(-0.003, 1.5], (1.5, 3.0], NaN, (-0.003, 1.5]]
Categories (2, interval[float64]): [(-0.003, 1.5] < (1.5, 3.0]]
```

任何落在bins之外的数据都将会被设为 `NaN`

## 生成一定区间内的间隔

如果我们需要经常地使用步进区间，我们可以使用 `interval_range()` 函数，结合 `start`, `end`, 和 `periods` 来建立一个 `IntervalIndex` 对于数值型的间隔，默认的 `interval_range` 间隔频率是1，对于datetime类型的间隔则是日历日。

```

In [191]: pd.interval_range(start=0, end=5)
Out[191]:
IntervalIndex([(0, 1], (1, 2], (2, 3], (3, 4], (4, 5]],
 closed='right',
 dtype='interval[int64]')

In [192]: pd.interval_range(start=pd.Timestamp('2017-01-01'), periods=4)
Out[192]:
IntervalIndex([(2017-01-01, 2017-01-02], (2017-01-02, 2017-01-03], (2017-01-03, 2017-01-04],
 (2017-01-04, 2017-01-05]],
 closed='right',
 dtype='interval[datetime64[ns]]')

In [193]: pd.interval_range(end=pd.Timedelta('3 days'), periods=3)
Out[193]:
IntervalIndex([(0 days 00:00:00, 1 days 00:00:00], (1 days 00:00:00, 2 days 00:00:00], (2 days 00:00:00,
 3 days 00:00:00]],
 closed='right',
 dtype='interval[timedelta64[ns]]')

```

`freq` 参数可以被用来明确非默认的频率，并且可以充分地利用各种各样的 [frequency aliases](#) datetime 类型的时间间隔。

```

In [194]: pd.interval_range(start=0, periods=5, freq=1.5)
Out[194]:
IntervalIndex([(0.0, 1.5], (1.5, 3.0], (3.0, 4.5], (4.5, 6.0], (6.0, 7.5]],
 closed='right',
 dtype='interval[float64]')

In [195]: pd.interval_range(start=pd.Timestamp('2017-01-01'), periods=4, freq='W')
Out[195]:
IntervalIndex([(2017-01-01, 2017-01-08], (2017-01-08, 2017-01-15], (2017-01-15, 2017-01-22],
 (2017-01-22, 2017-01-29]],
 closed='right',
 dtype='interval[datetime64[ns]]')

In [196]: pd.interval_range(start=pd.Timedelta('0 days'), periods=3, freq='9H')
Out[196]:
IntervalIndex([(0 days 00:00:00, 0 days 09:00:00], (0 days 09:00:00, 0 days 18:00:00], (0 days 18:00:00,
 1 days 03:00:00]],
 closed='right',
 dtype='interval[timedelta64[ns]]')

```

此外，`closed` 参数可以用来声明哪个边界是包含的。默认情况下，间隔的右界是包含的。

```
In [197]: pd.interval_range(start=0, end=4, closed='both')
Out[197]:
IntervalIndex([[0, 1], [1, 2], [2, 3], [3, 4]],
 closed='both',
 dtype='interval[int64]')

In [198]: pd.interval_range(start=0, end=4, closed='neither')
Out[198]:
IntervalIndex([(0, 1), (1, 2), (2, 3), (3, 4)],
 closed='neither',
 dtype='interval[int64]')
```

## v0.23.0新加入

使用 `start`, `end`, 和 `periods` 可以从 `start` 到 `end`（包含）生成一个平均分配的间隔，在返回 `IntervalIndex` 中生成 `periods` 这么多的元素（译者：区间）。

```
In [199]: pd.interval_range(start=0, end=6, periods=4)
Out[199]:
IntervalIndex([(0.0, 1.5], (1.5, 3.0], (3.0, 4.5], (4.5, 6.0]],
 closed='right',
 dtype='interval[float64]')

In [200]: pd.interval_range(pd.Timestamp('2018-01-01'),
.....: pd.Timestamp('2018-02-28'), periods=3)
.....:
Out[200]:
IntervalIndex([(2018-01-01, 2018-01-20 08:00:00], (2018-01-20 08:00:00, 2018-02-08 16:00:00],
 (2018-02-08 16:00:00, 2018-02-28]],
 closed='right',
 dtype='interval[datetime64[ns]]')
```

## 其他索引常见问题

### 数值索引

使用数值作为各维度的标签，再基于标签进行索引是一个非常痛苦的话题。在Scientific Python社区的邮件列表中，进行着剧烈的争论。在Pandas中，我们一般性的观点是，标签比实际的（用数值表示的）位置更为重要。因此，对于使用数值作为标签的对象来说，只有基于标签的索引才可以在标准工具，例如 `.loc` 方法，中正常使用。下面的代码将引发错误：

```
In [201]: s = pd.Series(range(5))
```

```
In [202]: s[-1]
```

```

KeyError Traceback (most recent call last)
<ipython-input-202-76c3dce40054> in <module>
----> 1 s[-1]

/pandas/pandas/core/series.py in __getitem__(self, key)
 1062 key = com.apply_if_callable(key, self)
 1063 try:
-> 1064 result = self.index.get_value(self, key)
 1065
 1066 if not is_scalar(result):

/pandas/pandas/core/indexes/base.py in get_value(self, series, key)
 4721 k = self._convert_scalar_indexer(k, kind="getitem")
 4722 try:
-> 4723 return self._engine.get_value(s, k, tz=getattr(series.dtype, "tz", None))
 4724 except KeyError as e1:
 4725 if len(self) > 0 and (self.holds_integer() or self.is_boolean()):

/pandas/pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_value()

/pandas/pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_value()

/pandas/pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()

/pandas/pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.Int64HashTable.get_item()

/pandas/pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.Int64HashTable.get_item()
```

```
KeyError: -1
```

```
In [203]: df = pd.DataFrame(np.random.randn(5, 4))
```

```
In [204]: df
```

```
Out[204]:
```

	0	1	2	3
0	-0.130121	-0.476046	0.759104	0.213379
1	-0.082641	0.448008	0.656420	-1.051443
2	0.594956	-0.151360	-0.069303	1.221431
3	-0.182832	0.791235	0.042745	2.069775
4	1.446552	0.019814	-1.389212	-0.702312

```
In [205]: df.loc[-2:]
```

```
Out[205]:
```

	0	1	2	3
0	-0.130121	-0.476046	0.759104	0.213379
1	-0.082641	0.448008	0.656420	-1.051443
2	0.594956	-0.151360	-0.069303	1.221431
3	-0.182832	0.791235	0.042745	2.069775
4	1.446552	0.019814	-1.389212	-0.702312

我们特意地做了这样的设计，是为了阻止歧义性以及一些小bug（当我们修改了函数，从而阻止了“滚回”到基于位置的索引方式以后，许多用户报告说，他们发现了bug）。

## 非单调索引要求严格匹配

如果一个 序列 或者 数据表 是单调递增或递减的，那么基于标签的切片行为的边界是可以超出索引的，这与普通的python 列表 的索引切片非常相似。索引的单调性可以使用 `is_monotonic_increasing()` 和 `is_monotonic_decreasing()` 属性来检查

```
In [206]: df = pd.DataFrame(index=[2, 3, 3, 4, 5], columns=['data'], data=list(range(5)))

In [207]: df.index.is_monotonic_increasing
Out[207]: True

no rows 0 or 1, but still returns rows 2, 3 (both of them), and 4:
In [208]: df.loc[0:4, :]
Out[208]:
 data
2 0
3 1
3 2
4 3

slice is are outside the index, so empty DataFrame is returned
In [209]: df.loc[13:15, :]
Out[209]:
Empty DataFrame
Columns: [data]
Index: []
```

另一方面，如果索引不是单调的，那么切片的两侧边界都必须是索引值中的唯一值。

```
In [210]: df = pd.DataFrame(index=[2, 3, 1, 4, 3, 5],
.....: columns=['data'], data=list(range(6)))
.....:

In [211]: df.index.is_monotonic_increasing
Out[211]: False

OK because 2 and 4 are in the index
In [212]: df.loc[2:4, :]
Out[212]:
 data
2 0
3 1
1 2
4 3
```

```
0 is not in the index
In [9]: df.loc[0:4, :]
KeyError: 0

3 is not a unique label
In [11]: df.loc[2:3, :]
KeyError: 'Cannot get right slice bound for non-unique label: 3'
```

`Index.is_monotonic_increasing` 和 `Index.is_monotonic_decreasing` 方法只能进行弱单调性的检查。要进行严格的单调性检查，你可以配合 `is_unique()` 方法一起使用。

```
In [213]: weakly_monotonic = pd.Index(['a', 'b', 'c', 'c'])

In [214]: weakly_monotonic
Out[214]: Index(['a', 'b', 'c', 'c'], dtype='object')

In [215]: weakly_monotonic.is_monotonic_increasing
Out[215]: True

In [216]: weakly_monotonic.is_monotonic_increasing & weakly_monotonic.is_unique
Out[216]: False
```

## 终止点被包含在内

与表中的python序列切片中，终止点不被包含不同，基于标签的切片在Pandas中，终止点是被**包含在内**的。最主要的原因是因为，我们很难准确地确定在索引中的“下一个”标签“到底是什么。例如下面这个序列：

```
In [217]: s = pd.Series(np.random.randn(6), index=list('abcdef'))

In [218]: s
Out[218]:
a 0.301379
b 1.240445
c -0.846068
d -0.043312
e -1.658747
f -0.819549
dtype: float64
```

如果我们希望从 c 选取到 e，如果我们使用基于数值的索引，那将会由如下操作：

```
In [219]: s[2:5]
Out[219]:
c -0.846068
d -0.043312
e -1.658747
dtype: float64
```

然而，如果你只有 `c` 和 `e`，确定下一个索引中的元素将会是比较困难的。例如，下面的这种方法完全是行不通的：

```
s.loc['c':'e' + 1]
```

一个非常常见的用例是限制一个时间序列的起始和终止日期。为了能够便于操作，我们决定在基于标签的切片行为中包含两个端点：

```
In [220]: s.loc['c':'e']
Out[220]:
c -0.846068
d -0.043312
e -1.658747
dtype: float64
```

这是一个非常典型的“显示战胜理想”的情况，但是如果你仅仅是想当然的认为基于标签的索引应该会和标准python中的整数型索引有着相同的行为时，你也确实需要多加留意。

## 索引会潜在地改变序列的dtype

不同的索引操作有可能会潜在地改变一个序列的dtypes

```
In [221]: series1 = pd.Series([1, 2, 3])

In [222]: series1.dtype
Out[222]: dtype('int64')

In [223]: res = series1.reindex([0, 4])

In [224]: res.dtype
Out[224]: dtype('float64')

In [225]: res
Out[225]:
0 1.0
4 NaN
dtype: float64
```

```
In [226]: series2 = pd.Series([True])

In [227]: series2.dtype
Out[227]: dtype('bool')

In [228]: res = series2.reindex_like(series1)

In [229]: res.dtype
Out[229]: dtype('O')

In [230]: res
Out[230]:
0 True
1 NaN
2 NaN
dtype: object
```

这是因为上述（重新）索引的操作悄悄地插入了 `NaNs`，因此 `dtype` 也就随之发生改变。如果你在使用一些 `numpy` 的 `ufuncs`，如 `numpy.logical_and` 时，将会导致一些问题。

参见 [this old issue](#) 了解更详细的讨论过程

---

## Merge, join, and concatenate

pandas provides various facilities for easily combining together Series or DataFrame with various kinds of set logic for the indexes and relational algebra functionality in the case of join / merge-type operations.

### Concatenating objects

---

The `concat()` function (in the main pandas namespace) does all of the heavy lifting of performing concatenation operations along an axis while performing optional set logic (union or intersection) of the indexes (if any) on the other axes. Note that I say “if any” because there is only a single possible axis of concatenation for Series.

Before diving into all of the details of `concat` and what it can do, here is a simple example:



```

In [1]: df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
...: 'B': ['B0', 'B1', 'B2', 'B3'],
...: 'C': ['C0', 'C1', 'C2', 'C3'],
...: 'D': ['D0', 'D1', 'D2', 'D3']},
...: index=[0, 1, 2, 3])
...:

In [2]: df2 = pd.DataFrame({'A': ['A4', 'A5', 'A6', 'A7'],
...: 'B': ['B4', 'B5', 'B6', 'B7'],
...: 'C': ['C4', 'C5', 'C6', 'C7'],
...: 'D': ['D4', 'D5', 'D6', 'D7']},
...: index=[4, 5, 6, 7])
...:

In [3]: df3 = pd.DataFrame({'A': ['A8', 'A9', 'A10', 'A11'],
...: 'B': ['B8', 'B9', 'B10', 'B11'],
...: 'C': ['C8', 'C9', 'C10', 'C11'],
...: 'D': ['D8', 'D9', 'D10', 'D11']},
...: index=[8, 9, 10, 11])
...:

In [4]: frames = [df1, df2, df3]

In [5]: result = pd.concat(frames)

```

## merging concat basic

Like its sibling function on ndarrays, `numpy.concatenate`, `pandas.concat` takes a list or dict of homogeneously-typed objects and concatenates them with some configurable handling of “what to do with the other axes” :

```

pd.concat(objs, axis=0, join='outer', ignore_index=False, keys=None,
 levels=None, names=None, verify_integrity=False, copy=True)

```

- `objs` : a sequence or mapping of Series or DataFrame objects. If a dict is passed, the sorted keys will be used as the keys argument, unless it is passed, in which case the values will be selected (see below). Any None objects will be dropped silently unless they are all None in which case a `ValueError` will be raised.
- `axis` : {0, 1, ...}, default 0. The axis to concatenate along.
- `join` : { 'inner' , 'outer' }, default 'outer' . How to handle indexes on other axis(es). Outer for union and inner for intersection.
- `ignore_index` : boolean, default False. If True, do not use the index values on the concatenation axis. The resulting axis will be labeled 0, ..., n - 1. This is useful if you are concatenating objects where the concatenation axis does not have meaningful indexing information. Note the index values on the other axes are still respected in the join.
- `keys` : sequence, default None. Construct hierarchical index using the passed keys as the outermost level. If multiple levels passed, should contain tuples.

- `levels` : list of sequences, default None. Specific levels (unique values) to use for constructing a MultiIndex. Otherwise they will be inferred from the keys.
- `names` : list, default None. Names for the levels in the resulting hierarchical index.
- `verify_integrity` : boolean, default False. Check whether the new concatenated axis contains duplicates. This can be very expensive relative to the actual data concatenation.
- `copy` : boolean, default True. If False, do not copy data unnecessarily.

Without a little bit of context many of these arguments don't make much sense. Let's revisit the above example. Suppose we wanted to associate specific keys with each of the pieces of the chopped up DataFrame. We can do this using the `keys` argument:

```
In [6]: result = pd.concat(frames, keys=['x', 'y', 'z'])
```

merging\_concat\_keys

As you can see (if you've read the rest of the documentation), the resulting object's index has a [hierarchical index](#). This means that we can now select out each chunk by key:

```
In [7]: result.loc['y']
Out[7]:
 A B C D
4 A4 B4 C4 D4
5 A5 B5 C5 D5
6 A6 B6 C6 D6
7 A7 B7 C7 D7
```

It's not a stretch to see how this can be very useful. More detail on this functionality below.

::: tip Note

It is worth noting that `concat()` (and therefore `append()`) makes a full copy of the data, and that constantly reusing this function can create a significant performance hit. If you need to use the operation over several datasets, use a list comprehension.

:::

```
frames = [process_your_file(f) for f in files]
result = pd.concat(frames)
```

## Set logic on the other axes

When gluing together multiple DataFrames, you have a choice of how to handle the other axes (other than the one being concatenated). This can be done in the following two ways:

- Take the union of them all, `join='outer'`. This is the default option as it results in zero information loss.
- Take the intersection, `join='inner'`.

Here is an example of each of these methods. First, the default `join='outer'` behavior:

```
In [8]: df4 = pd.DataFrame({'B': ['B2', 'B3', 'B6', 'B7'],
...: 'D': ['D2', 'D3', 'D6', 'D7'],
...: 'F': ['F2', 'F3', 'F6', 'F7']},
...: index=[2, 3, 6, 7])
...:
```

```
In [9]: result = pd.concat([df1, df4], axis=1, sort=False)
```

merging\_concat\_axis1

::: danger Warning

Changed in version 0.23.0.

The default behavior with `join='outer'` is to sort the other axis (columns in this case). In a future version of pandas, the default will be to not sort. We specified `sort=False` to opt in to the new behavior now.

:::

Here is the same thing with `join='inner'`:

```
In [10]: result = pd.concat([df1, df4], axis=1, join='inner')
```

merging\_concat\_axis1\_inner

Lastly, suppose we just wanted to reuse the exact index from the original DataFrame:

```
In [11]: result = pd.concat([df1, df4], axis=1).reindex(df1.index)
```

Similarly, we could index before the concatenation:

```
In [12]: pd.concat([df1, df4.reindex(df1.index)], axis=1)
Out[12]:
```

	A	B	C	D	B	D	F
0	A0	B0	C0	D0	NaN	NaN	NaN
1	A1	B1	C1	D1	NaN	NaN	NaN
2	A2	B2	C2	D2	B2	D2	F2
3	A3	B3	C3	D3	B3	D3	F3

merging\_concat\_axis1\_join\_axes

## Concatenating using `append`

A useful shortcut to `concat()` are the `append()` instance methods on `Series` and `DataFrame`. These methods actually predated `concat`. They concatenate along `axis=0`, namely the index:

```
In [13]: result = df1.append(df2)
```

merging\_append1

In the case of `DataFrame`, the indexes must be disjoint but the columns do not need to be:

```
In [14]: result = df1.append(df4, sort=False)
```

merging\_append2

`append` may take multiple objects to concatenate:

```
In [15]: result = df1.append([df2, df3])
```

merging\_append3

::: tip Note

Unlike the `append()` method, which appends to the original list and returns `None`, `append()` here **does not** modify `df1` and returns its copy with `df2` appended.

:::

## Ignoring indexes on the concatenation axis

For `DataFrame` objects which don't have a meaningful index, you may wish to append them and ignore the fact that they may have overlapping indexes. To do this, use the `ignore_index` argument:

```
In [16]: result = pd.concat([df1, df4], ignore_index=True, sort=False)
```

merging\_concat\_ignore\_index

This is also a valid argument to `DataFrame.append()`:

```
In [17]: result = df1.append(df4, ignore_index=True, sort=False)
```

merging\_append\_ignore\_index

## Concatenating with mixed ndims

You can concatenate a mix of `Series` and `DataFrame` objects. The `Series` will be transformed to `DataFrame` with the column name as the name of the `Series`.

```
In [18]: s1 = pd.Series(['X0', 'X1', 'X2', 'X3'], name='X')
```

```
In [19]: result = pd.concat([df1, s1], axis=1)
```

merging\_concat\_mixed\_ndim

::: tip Note

Since we're concatenating a `Series` to a `DataFrame`, we could have achieved the same result with `DataFrame.assign()`. To concatenate an arbitrary number of pandas objects (`DataFrame` or `Series`), use `concat`.

:::

If unnamed `Series` are passed they will be numbered consecutively.

```
In [20]: s2 = pd.Series(['_0', '_1', '_2', '_3'])

In [21]: result = pd.concat([df1, s2, s2, s2], axis=1)
```

merging\_concat\_unnamed\_series

Passing `ignore_index=True` will drop all name references.

```
In [22]: result = pd.concat([df1, s1], axis=1, ignore_index=True)
```

merging\_concat\_series\_ignore\_index

## More concatenating with group keys

A fairly common use of the `keys` argument is to override the column names when creating a new `DataFrame` based on existing `Series`. Notice how the default behaviour consists on letting the resulting `DataFrame` inherit the parent `Series`' name, when these existed.

```
In [23]: s3 = pd.Series([0, 1, 2, 3], name='foo')

In [24]: s4 = pd.Series([0, 1, 2, 3])

In [25]: s5 = pd.Series([0, 1, 4, 5])

In [26]: pd.concat([s3, s4, s5], axis=1)
Out[26]:
 foo 0 1
0 0 0 0
1 1 1 1
2 2 2 4
3 3 3 5
```

Through the `keys` argument we can override the existing column names.

```
In [27]: pd.concat([s3, s4, s5], axis=1, keys=['red', 'blue', 'yellow'])
Out[27]:
```

	red	blue	yellow
0	0	0	0
1	1	1	1
2	2	2	4
3	3	3	5

Let's consider a variation of the very first example presented:

```
In [28]: result = pd.concat(frames, keys=['x', 'y', 'z'])
```

merging\_concat\_group\_keys2

You can also pass a dict to `concat` in which case the dict keys will be used for the `keys` argument (unless other keys are specified):

```
In [29]: pieces = {'x': df1, 'y': df2, 'z': df3}
In [30]: result = pd.concat(pieces)
```

merging\_concat\_dict

```
In [31]: result = pd.concat(pieces, keys=['z', 'y'])
```

merging\_concat\_dict\_keys

The MultiIndex created has levels that are constructed from the passed keys and the index of the DataFrame `pieces`:

```
In [32]: result.index.levels
Out[32]: FrozenList([['z', 'y'], [4, 5, 6, 7, 8, 9, 10, 11]])
```

If you wish to specify other levels (as will occasionally be the case), you can do so using the `levels` argument:

```
In [33]: result = pd.concat(pieces, keys=['x', 'y', 'z'],
.....: levels=[['z', 'y', 'x', 'w']],
.....: names=['group_key'])
.....:
```

merging\_concat\_dict\_keys\_names

```
In [34]: result.index.levels
Out[34]: FrozenList([['z', 'y', 'x', 'w'], [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]])
```

This is fairly esoteric, but it is actually necessary for implementing things like `GroupBy` where the order of a categorical variable is meaningful.

## Appending rows to a DataFrame

While not especially efficient (since a new object must be created), you can append a single row to a `DataFrame` by passing a `Series` or dict to `append`, which returns a new `DataFrame` as above.

```
In [35]: s2 = pd.Series(['X0', 'X1', 'X2', 'X3'], index=['A', 'B', 'C', 'D'])

In [36]: result = df1.append(s2, ignore_index=True)
```

merging append series as row

You should use `ignore_index` with this method to instruct `DataFrame` to discard its index. If you wish to preserve the index, you should construct an appropriately-indexed `DataFrame` and append or concatenate those objects.

You can also pass a list of dicts or `Series`:

```
In [37]: dicts = [{'A': 1, 'B': 2, 'C': 3, 'X': 4},
....: {'A': 5, 'B': 6, 'C': 7, 'Y': 8}]
....:

In [38]: result = df1.append(dicts, ignore_index=True, sort=False)
```

merging\_append\_dits

## Database-style DataFrame or named Series joining/merging

pandas has full-featured, **high performance** in-memory join operations idiomatically very similar to relational databases like SQL. These methods perform significantly better (in some cases well over an order of magnitude better) than other open source implementations (like `base::merge.data.frame` in R). The reason for this is careful algorithmic design and the internal layout of the data in `DataFrame`.

See the [cookbook](#) for some advanced strategies.

Users who are familiar with SQL but new to pandas might be interested in a [comparison with SQL](#).

pandas provides a single function, `merge()`, as the entry point for all standard database join operations between `DataFrame` or named `Series` objects:

```
pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None,
 left_index=False, right_index=False, sort=True,
 suffixes=('_x', '_y'), copy=True, indicator=False,
 validate=None)
```

- `left`: A DataFrame or named Series object.
- `right`: Another DataFrame or named Series object.
- `on`: Column or index level names to join on. Must be found in both the left and right DataFrame and/or Series objects. If not passed and `left_index` and `right_index` are `False`, the intersection of the columns in the DataFrames and/or Series will be inferred to be the join keys.
- `left_on`: Columns or index levels from the left DataFrame or Series to use as keys. Can either be column names, index level names, or arrays with length equal to the length of the DataFrame or Series.
- `right_on`: Columns or index levels from the right DataFrame or Series to use as keys. Can either be column names, index level names, or arrays with length equal to the length of the DataFrame or Series.
- `left_index`: If `True`, use the index (row labels) from the left DataFrame or Series as its join key(s). In the case of a DataFrame or Series with a MultiIndex (hierarchical), the number of levels must match the number of join keys from the right DataFrame or Series.
- `right_index`: Same usage as `left_index` for the right DataFrame or Series
- `how`: One of `'left'`, `'right'`, `'outer'`, `'inner'`. Defaults to `inner`. See below for more detailed description of each method.
- `sort`: Sort the result DataFrame by the join keys in lexicographical order. Defaults to `True`, setting to `False` will improve performance substantially in many cases.
- `suffixes`: A tuple of string suffixes to apply to overlapping columns. Defaults to `('_x', '_y')`.
- `copy`: Always copy data (default `True`) from the passed DataFrame or named Series objects, even when reindexing is not necessary. Cannot be avoided in many cases but may improve performance / memory usage. The cases where copying can be avoided are somewhat pathological but this option is provided nonetheless.
- `indicator`: Add a column to the output DataFrame called `_merge` with information on the source of each row. `_merge` is Categorical-type and takes on a value of `left_only` for observations whose merge key only appears in `'left'` DataFrame or Series, `right_only` for observations whose merge key only appears in `'right'` DataFrame or Series, and `both` if the observation's merge key is found in both.



- `validate` : string, default None. If specified, checks if merge is of specified type.
  - “one\_to\_one” or “1:1” : checks if merge keys are unique in both left and right datasets.
  - “one\_to\_many” or “1:m” : checks if merge keys are unique in left dataset.
  - “many\_to\_one” or “m:1” : checks if merge keys are unique in right dataset.
  - “many\_to\_many” or “m:m” : allowed, but does not result in checks.

New in version 0.21.0.

::: tip Note

Support for specifying index levels as the `on`, `left_on`, and `right_on` parameters was added in version 0.23.0. Support for merging named `Series` objects was added in version 0.24.0.

:::

The return type will be the same as `left`. If `left` is a `DataFrame` or named `Series` and `right` is a subclass of `DataFrame`, the return type will still be `DataFrame`.

`merge` is a function in the pandas namespace, and it is also available as a `DataFrame` instance method `merge()`, with the calling `DataFrame` being implicitly considered the left object in the join.

The related `join()` method, uses `merge` internally for the index-on-index (by default) and column(s)-on-index join. If you are joining on index only, you may wish to use `DataFrame.join` to save yourself some typing.

## Brief primer on merge methods (relational algebra)

Experienced users of relational databases like SQL will be familiar with the terminology used to describe join operations between two SQL-table like structures (`DataFrame` objects). There are several cases to consider which are very important to understand:

- **one-to-one** joins: for example when joining two `DataFrame` objects on their indexes (which must contain unique values).
- **many-to-one** joins: for example when joining an index (unique) to one or more columns in a different `DataFrame`.
- **many-to-many** joins: joining columns on columns.

::: tip Note

When joining columns on columns (potentially a many-to-many join), any indexes on the passed `DataFrame` objects **will be discarded**.

:::

It is worth spending some time understanding the result of the **many-to-many** join case. In SQL / standard relational algebra, if a key combination appears more than once in both tables, the resulting table will have the **Cartesian product** of the associated data. Here is a very basic example with one unique key combination:

```
In [39]: left = pd.DataFrame({'key': ['K0', 'K1', 'K2', 'K3'],
....: 'A': ['A0', 'A1', 'A2', 'A3'],
....: 'B': ['B0', 'B1', 'B2', 'B3']})
....:

In [40]: right = pd.DataFrame({'key': ['K0', 'K1', 'K2', 'K3'],
....: 'C': ['C0', 'C1', 'C2', 'C3'],
....: 'D': ['D0', 'D1', 'D2', 'D3']})
....:

In [41]: result = pd.merge(left, right, on='key')
```

### merging\_merge\_on\_key

Here is a more complicated example with multiple join keys. Only the keys appearing in `left` and `right` are present (the intersection), since `how='inner'` by default.

```
In [42]: left = pd.DataFrame({'key1': ['K0', 'K0', 'K1', 'K2'],
....: 'key2': ['K0', 'K1', 'K0', 'K1'],
....: 'A': ['A0', 'A1', 'A2', 'A3'],
....: 'B': ['B0', 'B1', 'B2', 'B3']})
....:

In [43]: right = pd.DataFrame({'key1': ['K0', 'K1', 'K1', 'K2'],
....: 'key2': ['K0', 'K0', 'K0', 'K0'],
....: 'C': ['C0', 'C1', 'C2', 'C3'],
....: 'D': ['D0', 'D1', 'D2', 'D3']})
....:

In [44]: result = pd.merge(left, right, on=['key1', 'key2'])
```

### merging merge on key multiple

The `how` argument to `merge` specifies how to determine which keys are to be included in the resulting table. If a key combination **does not appear** in either the left or right tables, the values in the joined table will be `NA`. Here is a summary of the `how` options and their SQL equivalent names:

Merge method	SQL Join Name	Description
left	LEFT OUTER JOIN	Use keys from left frame only
right	RIGHT OUTER JOIN	Use keys from right frame only
outer	FULL OUTER JOIN	Use union of keys from both frames
inner	INNER JOIN	Use intersection of keys from both frames

```
In [45]: result = pd.merge(left, right, how='left', on=['key1', 'key2'])
```

merging\_merge\_on\_key\_left

```
In [46]: result = pd.merge(left, right, how='right', on=['key1', 'key2'])
```

merging\_merge\_on\_key\_right

```
In [47]: result = pd.merge(left, right, how='outer', on=['key1', 'key2'])
```

merging\_merge\_on\_key\_outer

```
In [48]: result = pd.merge(left, right, how='inner', on=['key1', 'key2'])
```

merging merge on key inner

Here is another example with duplicate join keys in DataFrames:

```
In [49]: left = pd.DataFrame({'A': [1, 2], 'B': [2, 2]})
```

```
In [50]: right = pd.DataFrame({'A': [4, 5, 6], 'B': [2, 2, 2]})
```

```
In [51]: result = pd.merge(left, right, on='B', how='outer')
```

merging merge on key dup

::: danger Warning

Joining / merging on duplicate keys can cause a returned frame that is the multiplication of the row dimensions, which may result in memory overflow. It is the user's responsibility to manage duplicate values in keys before joining large DataFrames.

:::

## Checking for duplicate keys

New in version 0.21.0.

Users can use the `validate` argument to automatically check whether there are unexpected duplicates in their merge keys. Key uniqueness is checked before merge operations and so should protect against memory overflows. Checking key uniqueness is also a good way to ensure user data structures are as expected.

In the following example, there are duplicate values of `B` in the right `DataFrame`. As this is not a one-to-one merge – as specified in the `validate` argument – an exception will be raised.

```
In [52]: left = pd.DataFrame({'A' : [1,2], 'B' : [1, 2]})
```

```
In [53]: right = pd.DataFrame({'A' : [4,5,6], 'B': [2, 2, 2]})
```

```
In [53]: result = pd.merge(left, right, on='B', how='outer', validate="one_to_one")
```

```
...
```

```
MergeError: Merge keys are not unique in right dataset; not a one-to-one merge
```

If the user is aware of the duplicates in the right `DataFrame` but wants to ensure there are no duplicates in the left `DataFrame`, one can use the `validate='one_to_many'` argument instead, which will not raise an exception.

```
In [54]: pd.merge(left, right, on='B', how='outer', validate="one_to_many")
```

```
Out[54]:
```

```
 A_x B A_y
0 1 1 NaN
1 2 2 4.0
2 2 2 5.0
3 2 2 6.0
```

## The merge indicator

`merge()` accepts the argument `indicator`. If `True`, a Categorical-type column called `_merge` will be added to the output object that takes on values:

Observation Origin	<code>_merge</code> value
Merge key only in 'left' frame	<code>left_only</code>
Merge key only in 'right' frame	<code>right_only</code>
Merge key in both frames	<code>both</code>

```
In [55]: df1 = pd.DataFrame({'col1': [0, 1], 'col_left': ['a', 'b']})
```

```
In [56]: df2 = pd.DataFrame({'col1': [1, 2, 2], 'col_right': [2, 2, 2]})
```

```
In [57]: pd.merge(df1, df2, on='col1', how='outer', indicator=True)
```

```
Out[57]:
```

```
 col1 col_left col_right _merge
0 0 a NaN left_only
1 1 b 2.0 both
2 2 NaN 2.0 right_only
3 2 NaN 2.0 right_only
```

The `indicator` argument will also accept string arguments, in which case the indicator function will use the value of the passed string as the name for the indicator column.

```
In [58]: pd.merge(df1, df2, on='col1', how='outer', indicator='indicator_column')
Out[58]:
```

	col1	col_left	col_right	indicator_column
0	0	a	NaN	left_only
1	1	b	2.0	both
2	2	NaN	2.0	right_only
3	2	NaN	2.0	right_only

## Merge dtypes

New in version 0.19.0.

Merging will preserve the dtype of the join keys.

```
In [59]: left = pd.DataFrame({'key': [1], 'v1': [10]})

In [60]: left
Out[60]:
```

	key	v1
0	1	10

```
In [61]: right = pd.DataFrame({'key': [1, 2], 'v1': [20, 30]})

In [62]: right
Out[62]:
```

	key	v1
0	1	20
1	2	30

We are able to preserve the join keys:

```
In [63]: pd.merge(left, right, how='outer')
Out[63]:
```

	key	v1
0	1	10
1	1	20
2	2	30

```
In [64]: pd.merge(left, right, how='outer').dtypes
Out[64]:
```

	key	v1
	int64	int64

```
dtype: object
```

Of course if you have missing values that are introduced, then the resulting dtype will be upcast.

```

In [65]: pd.merge(left, right, how='outer', on='key')
Out[65]:
 key v1_x v1_y
0 1 10.0 20
1 2 NaN 30

In [66]: pd.merge(left, right, how='outer', on='key').dtypes
Out[66]:
key int64
v1_x float64
v1_y int64
dtype: object

```

New in version 0.20.0.

Merging will preserve `category` dtypes of the mergands. See also the section on [categoricals](#).

The left frame.

```

In [67]: from pandas.api.types import CategoricalDtype

In [68]: X = pd.Series(np.random.choice(['foo', 'bar'], size=(10,)))

In [69]: X = X.astype(CategoricalDtype(categories=['foo', 'bar']))

In [70]: left = pd.DataFrame({'X': X,
.....: 'Y': np.random.choice(['one', 'two', 'three'],
.....: size=(10,))})
.....:

In [71]: left
Out[71]:
 X Y
0 bar one
1 foo one
2 foo three
3 bar three
4 foo one
5 bar one
6 bar three
7 bar three
8 bar three
9 foo three

In [72]: left.dtypes
Out[72]:
X category
Y object
dtype: object

```

The right frame.

```

In [73]: right = pd.DataFrame({'X': pd.Series(['foo', 'bar'],
.....: dtype=CategoricalDtype(['foo', 'bar']))},
.....: 'Z': [1, 2])
.....:

In [74]: right
Out[74]:
 X Z
0 foo 1
1 bar 2

In [75]: right.dtypes
Out[75]:
X category
Z int64
dtype: object

```

The merged result:

```

In [76]: result = pd.merge(left, right, how='outer')

In [77]: result
Out[77]:
 X Y Z
0 bar one 2
1 bar three 2
2 bar one 2
3 bar three 2
4 bar three 2
5 bar three 2
6 foo one 1
7 foo three 1
8 foo one 1
9 foo three 1

In [78]: result.dtypes
Out[78]:
X category
Y object
Z int64
dtype: object

```

::: tip Note

The category dtypes must be exactly the same, meaning the same categories and the ordered attribute. Otherwise the result will coerce to `object` dtype.

:::

::: tip Note

Merging on `category` dtypes that are the same can be quite performant compared to `object` dtype merging.

...

## Joining on index

`DataFrame.join()` is a convenient method for combining the columns of two potentially differently-indexed `DataFrames` into a single result `DataFrame`. Here is a very basic example:

```
In [79]: left = pd.DataFrame({'A': ['A0', 'A1', 'A2'],
.....: 'B': ['B0', 'B1', 'B2']},
.....: index=['K0', 'K1', 'K2'])
.....:

In [80]: right = pd.DataFrame({'C': ['C0', 'C2', 'C3'],
.....: 'D': ['D0', 'D2', 'D3']},
.....: index=['K0', 'K2', 'K3'])
.....:

In [81]: result = left.join(right)
```

merging join

```
In [82]: result = left.join(right, how='outer')
```

merging join outer

The same as above, but with `how='inner'`.

```
In [83]: result = left.join(right, how='inner')
```

merging\_join\_inner

The data alignment here is on the indexes (row labels). This same behavior can be achieved using `merge` plus additional arguments instructing it to use the indexes:

```
In [84]: result = pd.merge(left, right, left_index=True, right_index=True, how='outer')
```

merging merge index outer

```
In [85]: result = pd.merge(left, right, left_index=True, right_index=True, how='inner');
```

merging merge index inner



## Joining key columns on an index

`join()` takes an optional `on` argument which may be a column or multiple column names, which specifies that the passed `DataFrame` is to be aligned on that column in the `DataFrame`. These two function calls are completely equivalent:

```
left.join(right, on=key_or_keys)
pd.merge(left, right, left_on=key_or_keys, right_index=True,
 how='left', sort=False)
```

Obviously you can choose whichever form you find more convenient. For many-to-one joins (where one of the `DataFrame`'s is already indexed by the join key), using `join` may be more convenient. Here is a simple example:

```
In [86]: left = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
.....: 'B': ['B0', 'B1', 'B2', 'B3'],
.....: 'key': ['K0', 'K1', 'K0', 'K1']})
.....:

In [87]: right = pd.DataFrame({'C': ['C0', 'C1'],
.....: 'D': ['D0', 'D1']},
.....: index=['K0', 'K1'])
.....:

In [88]: result = left.join(right, on='key')
```

merging join key columns

```
In [89]: result = pd.merge(left, right, left_on='key', right_index=True,
.....: how='left', sort=False);
.....:
```

merging\_merge\_key\_columns

To join on multiple keys, the passed `DataFrame` must have a `MultiIndex`:

```

In [90]: left = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
....: 'B': ['B0', 'B1', 'B2', 'B3'],
....: 'key1': ['K0', 'K0', 'K1', 'K2'],
....: 'key2': ['K0', 'K1', 'K0', 'K1']})
....:

In [91]: index = pd.MultiIndex.from_tuples([('K0', 'K0'), ('K1', 'K0'),
....: ('K2', 'K0'), ('K2', 'K1')])
....:

In [92]: right = pd.DataFrame({'C': ['C0', 'C1', 'C2', 'C3'],
....: 'D': ['D0', 'D1', 'D2', 'D3']},
....: index=index)
....:

```

Now this can be joined by passing the two key column names:

```

In [93]: result = left.join(right, on=['key1', 'key2'])

```

merging join multikeys

The default for `DataFrame.join` is to perform a left join (essentially a “VLOOKUP” operation, for Excel users), which uses only the keys found in the calling `DataFrame`. Other join types, for example inner join, can be just as easily performed:

```

In [94]: result = left.join(right, on=['key1', 'key2'], how='inner')

```

merging join multikeys inner

As you can see, this drops any rows where there was no match.

## Joining a single Index to a MultiIndex

You can join a singly-indexed `DataFrame` with a level of a MultiIndexed `DataFrame`. The level will match on the name of the index of the singly-indexed frame against a level name of the MultiIndexed frame.

```

In [95]: left = pd.DataFrame({'A': ['A0', 'A1', 'A2'],
.....: 'B': ['B0', 'B1', 'B2']},
.....: index=pd.Index(['K0', 'K1', 'K2'], name='key'))
.....:

In [96]: index = pd.MultiIndex.from_tuples([('K0', 'Y0'), ('K1', 'Y1'),
.....: ('K2', 'Y2'), ('K2', 'Y3')],
.....: names=['key', 'Y'])
.....:

In [97]: right = pd.DataFrame({'C': ['C0', 'C1', 'C2', 'C3'],
.....: 'D': ['D0', 'D1', 'D2', 'D3']},
.....: index=index)
.....:

In [98]: result = left.join(right, how='inner')

```

merging join multiindex inner

This is equivalent but less verbose and more memory efficient / faster than this.

```

In [99]: result = pd.merge(left.reset_index(), right.reset_index(),
.....: on=['key'], how='inner').set_index(['key', 'Y'])
.....:

```

merging\_merge\_multiindex\_alternative

## Joining with two MultiIndexes

This is supported in a limited way, provided that the index for the right argument is completely used in the join, and is a subset of the indices in the left argument, as in this example:

```
In [100]: leftindex = pd.MultiIndex.from_product([list('abc'), list('xy'), [1, 2]],
.....: names=['abc', 'xy', 'num'])
.....:
```

```
In [101]: left = pd.DataFrame({'v1': range(12)}, index=leftindex)
```

```
In [102]: left
```

```
Out[102]:
```

			v1
abc	xy	num	
a	x	1	0
		2	1
	y	1	2
		2	3
b	x	1	4
		2	5
	y	1	6
		2	7
c	x	1	8
		2	9
	y	1	10
		2	11

```
In [103]: rightindex = pd.MultiIndex.from_product([list('abc'), list('xy')],
.....: names=['abc', 'xy'])
.....:
```

```
In [104]: right = pd.DataFrame({'v2': [100 * i for i in range(1, 7)]}, index=rightindex)
```

```
In [105]: right
```

```
Out[105]:
```

		v2
abc	xy	
a	x	100
	y	200
b	x	300
	y	400
c	x	500
	y	600

```
In [106]: left.join(right, on=['abc', 'xy'], how='inner')
```

```
Out[106]:
```

			v1	v2
abc	xy	num		
a	x	1	0	100
		2	1	100
	y	1	2	200
		2	3	200
b	x	1	4	300
		2	5	300
	y	1	6	400
		2	7	400
c	x	1	8	500
		2	9	500

```
y 1 10 600
 2 11 600
```

If that condition is not satisfied, a join with two multi-indexes can be done using the following code.

```
In [107]: leftindex = pd.MultiIndex.from_tuples([('K0', 'X0'), ('K0', 'X1'),
.....: ('K1', 'X2')],
.....: names=['key', 'X'])
.....:

In [108]: left = pd.DataFrame({'A': ['A0', 'A1', 'A2'],
.....: 'B': ['B0', 'B1', 'B2']},
.....: index=leftindex)
.....:

In [109]: rightindex = pd.MultiIndex.from_tuples([('K0', 'Y0'), ('K1', 'Y1'),
.....: ('K2', 'Y2'), ('K2', 'Y3')],
.....: names=['key', 'Y'])
.....:

In [110]: right = pd.DataFrame({'C': ['C0', 'C1', 'C2', 'C3'],
.....: 'D': ['D0', 'D1', 'D2', 'D3']},
.....: index=rightindex)
.....:

In [111]: result = pd.merge(left.reset_index(), right.reset_index(),
.....: on=['key'], how='inner').set_index(['key', 'X', 'Y'])
.....:
```

merging merge two multiindex

## Merging on a combination of columns and index levels

New in version 0.23.

Strings passed as the `on`, `left_on`, and `right_on` parameters may refer to either column names or index level names. This enables merging `DataFrame` instances on a combination of index levels and columns without resetting indexes.

```

In [112]: left_index = pd.Index(['K0', 'K0', 'K1', 'K2'], name='key1')

In [113]: left = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
.....: 'B': ['B0', 'B1', 'B2', 'B3'],
.....: 'key2': ['K0', 'K1', 'K0', 'K1']},
.....: index=left_index)
.....:

In [114]: right_index = pd.Index(['K0', 'K1', 'K2', 'K2'], name='key1')

In [115]: right = pd.DataFrame({'C': ['C0', 'C1', 'C2', 'C3'],
.....: 'D': ['D0', 'D1', 'D2', 'D3'],
.....: 'key2': ['K0', 'K0', 'K0', 'K1']},
.....: index=right_index)
.....:

In [116]: result = left.merge(right, on=['key1', 'key2'])

```

merge\_on\_index\_and\_column

::: tip Note

When DataFrames are merged on a string that matches an index level in both frames, the index level is preserved as an index level in the resulting DataFrame.

:::

::: tip Note

When DataFrames are merged using only some of the levels of a MultiIndex, the extra levels will be dropped from the resulting merge. In order to preserve those levels, use `reset_index` on those level names to move those levels to columns prior to doing the merge.

:::

::: tip Note

If a string matches both a column name and an index level name, then a warning is issued and the column takes precedence. This will result in an ambiguity error in a future version.

:::

## Overlapping value columns

The merge `suffixes` argument takes a tuple of list of strings to append to overlapping column names in the input `DataFrame`s to disambiguate the result columns:

```
In [117]: left = pd.DataFrame({'k': ['K0', 'K1', 'K2'], 'v': [1, 2, 3]})
In [118]: right = pd.DataFrame({'k': ['K0', 'K0', 'K3'], 'v': [4, 5, 6]})
In [119]: result = pd.merge(left, right, on='k')
```

merging merge overlapped

```
In [120]: result = pd.merge(left, right, on='k', suffixes=['_l', '_r'])
```

merging merge overlapped suffix

`DataFrame.join()` has `lsuffix` and `rsuffix` arguments which behave similarly.

```
In [121]: left = left.set_index('k')
In [122]: right = right.set_index('k')
In [123]: result = left.join(right, lsuffix='_l', rsuffix='_r')
```

merging merge overlapped multi suffix

## Joining multiple DataFrames

A list or tuple of `DataFrames` can also be passed to `join()` to join them together on their indexes.

```
In [124]: right2 = pd.DataFrame({'v': [7, 8, 9]}, index=['K1', 'K1', 'K2'])
In [125]: result = left.join([right, right2])
```

merging\_join\_multi\_df

## Merging together values within Series or DataFrame columns

Another fairly common situation is to have two like-indexed (or similarly indexed) `Series` or `DataFrame` objects and wanting to “patch” values in one object from values for matching indices in the other. Here is an example:

```
In [126]: df1 = pd.DataFrame([[np.nan, 3., 5.], [-4.6, np.nan, np.nan],
.....: [np.nan, 7., np.nan]])
.....:

In [127]: df2 = pd.DataFrame([[-42.6, np.nan, -8.2], [-5., 1.6, 4]],
.....: index=[1, 2])
.....:
```

For this, use the `combine_first()` method:

```
In [128]: result = df1.combine_first(df2)
```

merging\_combine\_first

Note that this method only takes values from the right `DataFrame` if they are missing in the left `DataFrame`. A related method, `update()`, alters non-NA values in place:

```
In [129]: df1.update(df2)
```

merging\_update

## Timeseries friendly merging

### Merging ordered data

A `merge_ordered()` function allows combining time series and other ordered data. In particular it has an optional `fill_method` keyword to fill/interpolate missing data:

```
In [130]: left = pd.DataFrame({'k': ['K0', 'K1', 'K1', 'K2'],
.....: 'lv': [1, 2, 3, 4],
.....: 's': ['a', 'b', 'c', 'd']})
.....:

In [131]: right = pd.DataFrame({'k': ['K1', 'K2', 'K4'],
.....: 'rv': [1, 2, 3]})
.....:

In [132]: pd.merge_ordered(left, right, fill_method='ffill', left_by='s')
Out[132]:
```

	k	lv	s	rv
0	K0	1.0	a	NaN
1	K1	1.0	a	1.0
2	K2	1.0	a	2.0
3	K4	1.0	a	3.0
4	K1	2.0	b	1.0
5	K2	2.0	b	2.0
6	K4	2.0	b	3.0
7	K1	3.0	c	1.0
8	K2	3.0	c	2.0
9	K4	3.0	c	3.0
10	K1	NaN	d	1.0
11	K2	4.0	d	2.0
12	K4	4.0	d	3.0

### Merging asof

New in version 0.19.0.



A `merge_asof()` is similar to an ordered left-join except that we match on nearest key rather than equal keys. For each row in the `left` `DataFrame`, we select the last row in the `right` `DataFrame` whose `on` key is less than the left's key. Both `DataFrames` must be sorted by the key.

Optionally an asof merge can perform a group-wise merge. This matches the `by` key equally, in addition to the nearest match on the `on` key.

For example; we might have `trades` and `quotes` and we want to `asof` merge them.

```
In [133]: trades = pd.DataFrame({
.....: 'time': pd.to_datetime(['20160525 13:30:00.023',
.....: '20160525 13:30:00.038',
.....: '20160525 13:30:00.048',
.....: '20160525 13:30:00.048',
.....: '20160525 13:30:00.048']),
.....: 'ticker': ['MSFT', 'MSFT',
.....: 'GOOG', 'GOOG', 'AAPL'],
.....: 'price': [51.95, 51.95,
.....: 720.77, 720.92, 98.00],
.....: 'quantity': [75, 155,
.....: 100, 100, 100]},
.....: columns=['time', 'ticker', 'price', 'quantity'])
.....:

In [134]: quotes = pd.DataFrame({
.....: 'time': pd.to_datetime(['20160525 13:30:00.023',
.....: '20160525 13:30:00.023',
.....: '20160525 13:30:00.030',
.....: '20160525 13:30:00.041',
.....: '20160525 13:30:00.048',
.....: '20160525 13:30:00.049',
.....: '20160525 13:30:00.072',
.....: '20160525 13:30:00.075']),
.....: 'ticker': ['GOOG', 'MSFT', 'MSFT',
.....: 'MSFT', 'GOOG', 'AAPL', 'GOOG',
.....: 'MSFT'],
.....: 'bid': [720.50, 51.95, 51.97, 51.99,
.....: 720.50, 97.99, 720.50, 52.01],
.....: 'ask': [720.93, 51.96, 51.98, 52.00,
.....: 720.93, 98.01, 720.88, 52.03]},
.....: columns=['time', 'ticker', 'bid', 'ask'])
.....:
```

```
In [135]: trades
Out[135]:
```

		time	ticker	price	quantity
0	2016-05-25	13:30:00.023	MSFT	51.95	75
1	2016-05-25	13:30:00.038	MSFT	51.95	155
2	2016-05-25	13:30:00.048	GOOG	720.77	100
3	2016-05-25	13:30:00.048	GOOG	720.92	100
4	2016-05-25	13:30:00.048	AAPL	98.00	100

```
In [136]: quotes
Out[136]:
```

		time	ticker	bid	ask
0	2016-05-25	13:30:00.023	GOOG	720.50	720.93
1	2016-05-25	13:30:00.023	MSFT	51.95	51.96
2	2016-05-25	13:30:00.030	MSFT	51.97	51.98
3	2016-05-25	13:30:00.041	MSFT	51.99	52.00
4	2016-05-25	13:30:00.048	GOOG	720.50	720.93
5	2016-05-25	13:30:00.049	AAPL	97.99	98.01
6	2016-05-25	13:30:00.072	GOOG	720.50	720.88
7	2016-05-25	13:30:00.075	MSFT	52.01	52.03

By default we are taking the asof of the quotes.

```
In [137]: pd.merge_asof(trades, quotes,
.....: on='time',
.....: by='ticker')
.....:
Out[137]:
```

		time	ticker	price	quantity	bid	ask
0	2016-05-25	13:30:00.023	MSFT	51.95	75	51.95	51.96
1	2016-05-25	13:30:00.038	MSFT	51.95	155	51.97	51.98
2	2016-05-25	13:30:00.048	GOOG	720.77	100	720.50	720.93
3	2016-05-25	13:30:00.048	GOOG	720.92	100	720.50	720.93
4	2016-05-25	13:30:00.048	AAPL	98.00	100	NaN	NaN

We only asof within 2ms between the quote time and the trade time.

```
In [138]: pd.merge_asof(trades, quotes,
.....: on='time',
.....: by='ticker',
.....: tolerance=pd.Timedelta('2ms'))
.....:
Out[138]:
```

		time	ticker	price	quantity	bid	ask
0	2016-05-25	13:30:00.023	MSFT	51.95	75	51.95	51.96
1	2016-05-25	13:30:00.038	MSFT	51.95	155	NaN	NaN
2	2016-05-25	13:30:00.048	GOOG	720.77	100	720.50	720.93
3	2016-05-25	13:30:00.048	GOOG	720.92	100	720.50	720.93
4	2016-05-25	13:30:00.048	AAPL	98.00	100	NaN	NaN

We only asof within 10ms between the quote time and the trade time and we exclude exact matches on time. Note that though we exclude the exact matches (of the quotes), prior quotes **do** propagate to that point in time.

```
In [139]: pd.merge_asof(trades, quotes,
.....: on='time',
.....: by='ticker',
.....: tolerance=pd.Timedelta('10ms'),
.....: allow_exact_matches=False)
.....:
Out[139]:
```

		time	ticker	price	quantity	bid	ask
0	2016-05-25	13:30:00.023	MSFT	51.95	75	NaN	NaN
1	2016-05-25	13:30:00.038	MSFT	51.95	155	51.97	51.98
2	2016-05-25	13:30:00.048	GOOG	720.77	100	NaN	NaN
3	2016-05-25	13:30:00.048	GOOG	720.92	100	NaN	NaN
4	2016-05-25	13:30:00.048	AAPL	98.00	100	NaN	NaN

---

## Reshaping and pivot tables

### Reshaping by pivoting DataFrame objects

---

reshaping pivot

Data is often stored in so-called “stacked” or “record” format:

```
In [1]: df
Out[1]:
```

	date	variable	value
0	2000-01-03	A	0.469112
1	2000-01-04	A	-0.282863
2	2000-01-05	A	-1.509059
3	2000-01-03	B	-1.135632
4	2000-01-04	B	1.212112
5	2000-01-05	B	-0.173215
6	2000-01-03	C	0.119209
7	2000-01-04	C	-1.044236
8	2000-01-05	C	-0.861849
9	2000-01-03	D	-2.104569
10	2000-01-04	D	-0.494929
11	2000-01-05	D	1.071804

For the curious here is how the above DataFrame was created:

```
import pandas.util.testing as tm

tm.N = 3

def unpivot(frame):
 N, K = frame.shape
 data = {'value': frame.to_numpy().ravel('F'),
 'variable': np.asarray(frame.columns).repeat(N),
 'date': np.tile(np.asarray(frame.index), K)}
 return pd.DataFrame(data, columns=['date', 'variable', 'value'])

df = unpivot(tm.makeTimeDataFrame())
```

To select out everything for variable `A` we could do:

```
In [2]: df[df['variable'] == 'A']
Out[2]:
```

	date	variable	value
0	2000-01-03	A	0.469112
1	2000-01-04	A	-0.282863
2	2000-01-05	A	-1.509059

But suppose we wish to do time series operations with the variables. A better representation would be where the `columns` are the unique variables and an `index` of dates identifies individual observations. To reshape the data into this form, we use the `DataFrame.pivot()` method (also implemented as a top level function `pivot()`):

```
In [3]: df.pivot(index='date', columns='variable', values='value')
Out[3]:
```

variable	A	B	C	D
date				
2000-01-03	0.469112	-1.135632	0.119209	-2.104569
2000-01-04	-0.282863	1.212112	-1.044236	-0.494929
2000-01-05	-1.509059	-0.173215	-0.861849	1.071804

If the `values` argument is omitted, and the input `DataFrame` has more than one column of values which are not used as column or index inputs to `pivot`, then the resulting “pivoted” `DataFrame` will have [hierarchical columns](#) whose topmost level indicates the respective value column:

```
In [4]: df['value2'] = df['value'] * 2

In [5]: pivoted = df.pivot(index='date', columns='variable')

In [6]: pivoted
Out[6]:
```

	value				value2			
variable	A	B	C	D	A	B	C	D
date								
2000-01-03	0.469112	-1.135632	0.119209	-2.104569	0.938225	-2.271265	0.238417	-4.209138
2000-01-04	-0.282863	1.212112	-1.044236	-0.494929	-0.565727	2.424224	-2.088472	-0.989859
2000-01-05	-1.509059	-0.173215	-0.861849	1.071804	-3.018117	-0.346429	-1.723698	2.143608

You can then select subsets from the pivoted DataFrame :

```
In [7]: pivoted['value2']
Out[7]:
```

variable	A	B	C	D
date				
2000-01-03	0.938225	-2.271265	0.238417	-4.209138
2000-01-04	-0.565727	2.424224	-2.088472	-0.989859
2000-01-05	-3.018117	-0.346429	-1.723698	2.143608

Note that this returns a view on the underlying data in the case where the data are homogeneously-typed.

::: tip Note

`pivot()` will error with a `ValueError: Index contains duplicate entries, cannot reshape` if the index/column pair is not unique. In this case, consider using `pivot_table()` which is a generalization of `pivot` that can handle duplicate values for one index/column pair.

:::

## Reshaping by stacking and unstacking

reshaping stack

Closely related to the `pivot()` method are the related `stack()` and `unstack()` methods available on `Series` and `DataFrame`. These methods are designed to work together with `MultiIndex` objects (see the section on [hierarchical indexing](#)). Here are essentially what these methods do:

- `stack`: “pivot” a level of the (possibly hierarchical) column labels, returning a `DataFrame` with an index with a new inner-most level of row labels.
- `unstack`: (inverse operation of `stack`) “pivot” a level of the (possibly hierarchical) row index to the column axis, producing a reshaped `DataFrame` with a new inner-most level of column labels.

reshaping unstack

The clearest way to explain is by example. Let's take a prior example data set from the hierarchical indexing section:

```
In [8]: tuples = list(zip(*[['bar', 'bar', 'baz', 'baz',
...: 'foo', 'foo', 'qux', 'qux'],
...: ['one', 'two', 'one', 'two',
...: 'one', 'two', 'one', 'two']]))
...:

In [9]: index = pd.MultiIndex.from_tuples(tuples, names=['first', 'second'])

In [10]: df = pd.DataFrame(np.random.randn(8, 2), index=index, columns=['A', 'B'])

In [11]: df2 = df[:4]

In [12]: df2
Out[12]:
```

		A	B
bar	one	0.721555	-0.706771
	two	-1.039575	0.271860
baz	one	-0.424972	0.567020
	two	0.276232	-1.087401

The `stack` function “compresses” a level in the `DataFrame`’s columns to produce either:

- A `Series`, in the case of a simple column Index.
- A `DataFrame`, in the case of a `MultiIndex` in the columns.

If the columns have a `MultiIndex`, you can choose which level to stack. The stacked level becomes the new lowest level in a `MultiIndex` on the columns:

```
In [13]: stacked = df2.stack()

In [14]: stacked
Out[14]:
```

first	second		
bar	one	A	0.721555
		B	-0.706771
	two	A	-1.039575
		B	0.271860
baz	one	A	-0.424972
		B	0.567020
	two	A	0.276232
		B	-1.087401

dtype: float64

With a “stacked” `DataFrame` or `Series` (having a `MultiIndex` as the `index`), the inverse operation of `stack` is `unstack`, which by default unstacks the **last level**:

```
In [15]: stacked.unstack()
Out[15]:
```

		A	B
first			
bar	one	0.721555	-0.706771
	two	-1.039575	0.271860
baz	one	-0.424972	0.567020
	two	0.276232	-1.087401

```
In [16]: stacked.unstack(1)
Out[16]:
```

		one	two
second			
first			
bar	A	0.721555	-1.039575
	B	-0.706771	0.271860
baz	A	-0.424972	0.276232
	B	0.567020	-1.087401

```
In [17]: stacked.unstack(0)
Out[17]:
```

		bar	baz
first			
second			
one	A	0.721555	-0.424972
	B	-0.706771	0.567020
two	A	-1.039575	0.276232
	B	0.271860	-1.087401

reshaping unstack 1

If the indexes have names, you can use the level names instead of specifying the level numbers:

```
In [18]: stacked.unstack('second')
Out[18]:
```

		one	two
first			
bar	A	0.721555	-1.039575
	B	-0.706771	0.271860
baz	A	-0.424972	0.276232
	B	0.567020	-1.087401

reshaping unstack 0

Notice that the `stack` and `unstack` methods implicitly sort the index levels involved. Hence a call to `stack` and then `unstack`, or vice versa, will result in a **sorted** copy of the original `DataFrame` or `Series`:

```
In [19]: index = pd.MultiIndex.from_product([[2, 1], ['a', 'b']])

In [20]: df = pd.DataFrame(np.random.randn(4), index=index, columns=['A'])

In [21]: df
Out[21]:
```

		A
2	a	-0.370647
	b	-1.157892
1	a	-1.344312
	b	0.844885

```
In [22]: all(df.unstack().stack() == df.sort_index())
Out[22]: True
```

The above code will raise a `TypeError` if the call to `sort_index` is removed.

## Multiple levels

You may also stack or unstack more than one level at a time by passing a list of levels, in which case the end result is as if each level in the list were processed individually.



```
In [23]: columns = pd.MultiIndex.from_tuples([
.....: ('A', 'cat', 'long'), ('B', 'cat', 'long'),
.....: ('A', 'dog', 'short'), ('B', 'dog', 'short')],
.....: names=['exp', 'animal', 'hair_length']
.....:)
.....:
```

```
In [24]: df = pd.DataFrame(np.random.randn(4, 4), columns=columns)
```

```
In [25]: df
```

```
Out[25]:
```

exp	A	B	A	B
animal	cat	cat	dog	dog
hair_length	long	long	short	short
0	1.075770	-0.109050	1.643563	-1.469388
1	0.357021	-0.674600	-1.776904	-0.968914
2	-1.294524	0.413738	0.276662	-0.472035
3	-0.013960	-0.362543	-0.006154	-0.923061

```
In [26]: df.stack(level=['animal', 'hair_length'])
```

```
Out[26]:
```

exp	A	B
animal hair_length		
0 cat long	1.075770	-0.109050
dog short	1.643563	-1.469388
1 cat long	0.357021	-0.674600
dog short	-1.776904	-0.968914
2 cat long	-1.294524	0.413738
dog short	0.276662	-0.472035
3 cat long	-0.013960	-0.362543
dog short	-0.006154	-0.923061

The list of levels can contain either level names or level numbers (but not a mixture of the two).

```
df.stack(level=['animal', 'hair_length'])
```

```
from above is equivalent to:
```

```
In [27]: df.stack(level=[1, 2])
```

```
Out[27]:
```

exp	A	B
animal hair_length		
0 cat long	1.075770	-0.109050
dog short	1.643563	-1.469388
1 cat long	0.357021	-0.674600
dog short	-1.776904	-0.968914
2 cat long	-1.294524	0.413738
dog short	0.276662	-0.472035
3 cat long	-0.013960	-0.362543
dog short	-0.006154	-0.923061

## Missing data

These functions are intelligent about handling missing data and do not expect each subgroup within the hierarchical index to have the same set of labels. They also can handle the index being unsorted (but you can make it sorted by calling `sort_index`, of course). Here is a more complex example:

```
In [28]: columns = pd.MultiIndex.from_tuples([('A', 'cat'), ('B', 'dog'),
.....: ('B', 'cat'), ('A', 'dog')],
.....: names=['exp', 'animal'])
.....:

In [29]: index = pd.MultiIndex.from_product([('bar', 'baz', 'foo', 'qux'),
.....: ('one', 'two')],
.....: names=['first', 'second'])
.....:

In [30]: df = pd.DataFrame(np.random.randn(8, 4), index=index, columns=columns)

In [31]: df2 = df.iloc[[0, 1, 2, 4, 5, 7]]

In [32]: df2
Out[32]:
```

		exp		animal	
		A	B	cat	dog
first	second	cat	dog	cat	dog
bar	one	0.895717	0.805244	-1.206412	2.565646
	two	1.431256	1.340309	-1.170299	-0.226169
baz	one	0.410835	0.813850	0.132003	-0.827317
foo	one	-1.413681	1.607920	1.024180	0.569605
	two	0.875906	-2.211372	0.974466	-2.006747
qux	two	-1.226825	0.769804	-1.281247	-0.727707

As mentioned above, `stack` can be called with a `level` argument to select which level in the columns to stack:

```
In [33]: df2.stack('exp')
Out[33]:
animal cat dog
first second exp
bar one A 0.895717 2.565646
 one B -1.206412 0.805244
 two A 1.431256 -0.226169
 two B -1.170299 1.340309
baz one A 0.410835 -0.827317
 one B 0.132003 0.813850
foo one A -1.413681 0.569605
 one B 1.024180 1.607920
 two A 0.875906 -2.006747
 two B 0.974466 -2.211372
qux two A -1.226825 -0.727707
 two B -1.281247 0.769804
```

```
In [34]: df2.stack('animal')
Out[34]:
exp A B
first second animal
bar one cat 0.895717 -1.206412
 one dog 2.565646 0.805244
 two cat 1.431256 -1.170299
 two dog -0.226169 1.340309
baz one cat 0.410835 0.132003
 one dog -0.827317 0.813850
foo one cat -1.413681 1.024180
 one dog 0.569605 1.607920
 two cat 0.875906 0.974466
 two dog -2.006747 -2.211372
qux two cat -1.226825 -1.281247
 two dog -0.727707 0.769804
```

Unstacking can result in missing values if subgroups do not have the same set of labels. By default, missing values will be replaced with the default fill value for that data type, `NaN` for float, `NaT` for datetimelike, etc. For integer types, by default data will be converted to float and missing values will be set to `NaN`.

```
In [35]: df3 = df.iloc[[0, 1, 4, 7], [1, 2]]
```

```
In [36]: df3
```

```
Out[36]:
```

		B	
exp		dog	cat
animal			
first	second		
bar	one	0.805244	-1.206412
	two	1.340309	-1.170299
foo	one	1.607920	1.024180
qux	two	0.769804	-1.281247

```
In [37]: df3.unstack()
```

```
Out[37]:
```

		B			
exp		dog		cat	
animal		one	two	one	two
second	first				
bar	one	0.805244	1.340309	-1.206412	-1.170299
	two				
foo	one	1.607920	NaN	1.024180	NaN
qux	two	NaN	0.769804	NaN	-1.281247

New in version 0.18.0.

Alternatively, `unstack` takes an optional `fill_value` argument, for specifying the value of missing data.

```
In [38]: df3.unstack(fill_value=-1e9)
```

```
Out[38]:
```

		B			
exp		dog		cat	
animal		one	two	one	two
second	first				
bar	one	8.052440e-01	1.340309e+00	-1.206412e+00	-1.170299e+00
	two				
foo	one	1.607920e+00	-1.000000e+09	1.024180e+00	-1.000000e+09
qux	two	-1.000000e+09	7.698036e-01	-1.000000e+09	-1.281247e+00

## With a MultiIndex

Unstacking when the columns are a `MultiIndex` is also careful about doing the right thing:

```
In [39]: df[:3].unstack(0)
Out[39]:
```

exp	A			B			A		
animal	cat			dog			cat		dog
first	bar	baz		bar	baz		bar	baz	bar
second									
one	0.895717	0.410835		0.805244	0.81385	-1.206412	0.132003	2.565646	-0.827317
two	1.431256	NaN		1.340309	NaN	-1.170299	NaN	-0.226169	NaN

```
In [40]: df2.unstack(1)
Out[40]:
```

exp	A		B				A	
animal	cat		dog		cat		dog	
second	one	two	one	two	one	two	one	two
first								
bar	0.895717	1.431256	0.805244	1.340309	-1.206412	-1.170299	2.565646	-0.226169
baz	0.410835	NaN	0.813850	NaN	0.132003	NaN	-0.827317	NaN
foo	-1.413681	0.875906	1.607920	-2.211372	1.024180	0.974466	0.569605	-2.006747
qux	NaN	-1.226825	NaN	0.769804	NaN	-1.281247	NaN	-0.727707

## Reshaping by Melt

reshaping melt

The top-level `melt()` function and the corresponding `DataFrame.melt()` are useful to massage a `DataFrame` into a format where one or more columns are identifier variables, while all other columns, considered measured variables, are “unpivoted” to the row axis, leaving just two non-identifier columns, “variable” and “value”. The names of those columns can be customized by supplying the `var_name` and `value_name` parameters.

For instance,

```

In [41]: cheese = pd.DataFrame({'first': ['John', 'Mary'],
.....: 'last': ['Doe', 'Bo'],
.....: 'height': [5.5, 6.0],
.....: 'weight': [130, 150]})
.....:

In [42]: cheese
Out[42]:
 first last height weight
0 John Doe 5.5 130
1 Mary Bo 6.0 150

In [43]: cheese.melt(id_vars=['first', 'last'])
Out[43]:
 first last variable value
0 John Doe height 5.5
1 Mary Bo height 6.0
2 John Doe weight 130.0
3 Mary Bo weight 150.0

In [44]: cheese.melt(id_vars=['first', 'last'], var_name='quantity')
Out[44]:
 first last quantity value
0 John Doe height 5.5
1 Mary Bo height 6.0
2 John Doe weight 130.0
3 Mary Bo weight 150.0

```

Another way to transform is to use the `wide_to_long()` panel data convenience function. It is less flexible than `melt()`, but more user-friendly.

```
In [45]: dft = pd.DataFrame({"A1970": {0: "a", 1: "b", 2: "c"},
.....: "A1980": {0: "d", 1: "e", 2: "f"},
.....: "B1970": {0: 2.5, 1: 1.2, 2: .7},
.....: "B1980": {0: 3.2, 1: 1.3, 2: .1},
.....: "X": dict(zip(range(3), np.random.randn(3)))
.....: })
```

```
In [46]: dft["id"] = dft.index
```

```
In [47]: dft
```

```
Out[47]:
```

	A1970	A1980	B1970	B1980	X	id
0	a	d	2.5	3.2	-0.121306	0
1	b	e	1.2	1.3	-0.097883	1
2	c	f	0.7	0.1	0.695775	2

```
In [48]: pd.wide_to_long(dft, ["A", "B"], i="id", j="year")
```

```
Out[48]:
```

		X	A	B
id	year			
0	1970	-0.121306	a	2.5
1	1970	-0.097883	b	1.2
2	1970	0.695775	c	0.7
0	1980	-0.121306	d	3.2
1	1980	-0.097883	e	1.3
2	1980	0.695775	f	0.1

## Combining with stats and GroupBy

It should be no shock that combining `pivot / stack / unstack` with `GroupBy` and the basic `Series` and `DataFrame` statistical functions can produce some very expressive and fast data manipulations.

```
In [49]: df
Out[49]:
```

exp		A	B		A
animal		cat	dog	cat	dog
	first second				
bar	one	0.895717	0.805244	-1.206412	2.565646
	two	1.431256	1.340309	-1.170299	-0.226169
baz	one	0.410835	0.813850	0.132003	-0.827317
	two	-0.076467	-1.187678	1.130127	-1.436737
foo	one	-1.413681	1.607920	1.024180	0.569605
	two	0.875906	-2.211372	0.974466	-2.006747
qux	one	-0.410001	-0.078638	0.545952	-1.219217
	two	-1.226825	0.769804	-1.281247	-0.727707

```
In [50]: df.stack().mean(1).unstack()
Out[50]:
```

animal		cat	dog
	first second		
bar	one	-0.155347	1.685445
	two	0.130479	0.557070
baz	one	0.271419	-0.006733
	two	0.526830	-1.312207
foo	one	-0.194750	1.088763
	two	0.925186	-2.109060
qux	one	0.067976	-0.648927
	two	-1.254036	0.021048

# same result, another way

```
In [51]: df.groupby(level=1, axis=1).mean()
Out[51]:
```

animal		cat	dog
	first second		
bar	one	-0.155347	1.685445
	two	0.130479	0.557070
baz	one	0.271419	-0.006733
	two	0.526830	-1.312207
foo	one	-0.194750	1.088763
	two	0.925186	-2.109060
qux	one	0.067976	-0.648927
	two	-1.254036	0.021048

```
In [52]: df.stack().groupby(level=1).mean()
Out[52]:
```

exp		A	B
	second		
one		0.071448	0.455513
two		-0.424186	-0.204486

```
In [53]: df.mean().unstack(0)
Out[53]:
```

exp		A	B
animal			
cat		0.060843	0.018596
dog		-0.413580	0.232430



## Pivot tables

While `pivot()` provides general purpose pivoting with various data types (strings, numerics, etc.), pandas also provides `pivot_table()` for pivoting with aggregation of numeric data.

The function `pivot_table()` can be used to create spreadsheet-style pivot tables. See the [cookbook](#) for some advanced strategies.

It takes a number of arguments:

- `data`: a DataFrame object.
- `values`: a column or a list of columns to aggregate.
- `index`: a column, Grouper, array which has the same length as data, or list of them. Keys to group by on the pivot table index. If an array is passed, it is being used as the same manner as column values.
- `columns`: a column, Grouper, array which has the same length as data, or list of them. Keys to group by on the pivot table column. If an array is passed, it is being used as the same manner as column values.
- `aggfunc`: function to use for aggregation, defaulting to `numpy.mean`.

Consider a data set like this:

```
In [54]: import datetime
```

```
In [55]: df = pd.DataFrame({'A': ['one', 'one', 'two', 'three'] * 6,
.....: 'B': ['A', 'B', 'C'] * 8,
.....: 'C': ['foo', 'foo', 'foo', 'bar', 'bar', 'bar'] * 4,
.....: 'D': np.random.randn(24),
.....: 'E': np.random.randn(24),
.....: 'F': [datetime.datetime(2013, i, 1) for i in range(1, 13)]
.....: + [datetime.datetime(2013, i, 15) for i in range(1, 13)]})
.....:
```

```
In [56]: df
```

```
Out[56]:
```

	A	B	C	D	E	F
0	one	A	foo	0.341734	-0.317441	2013-01-01
1	one	B	foo	0.959726	-1.236269	2013-02-01
2	two	C	foo	-1.110336	0.896171	2013-03-01
3	three	A	bar	-0.619976	-0.487602	2013-04-01
4	one	B	bar	0.149748	-0.082240	2013-05-01
..	...	...	...	...	...	...
19	three	B	foo	0.690579	-2.213588	2013-08-15
20	one	C	foo	0.995761	1.063327	2013-09-15
21	one	A	bar	2.396780	1.266143	2013-10-15
22	two	B	bar	0.014871	0.299368	2013-11-15
23	three	C	bar	3.357427	-0.863838	2013-12-15

```
[24 rows x 6 columns]
```

We can produce pivot tables from this data very easily:

```
In [57]: pd.pivot_table(df, values='D', index=['A', 'B'], columns=['C'])
```

```
Out[57]:
```

		C	bar	foo
one	A	1.120915	-0.514058	
	B	-0.338421	0.002759	
	C	-0.538846	0.699535	
three	A	-1.181568	NaN	
	B	NaN	0.433512	
	C	0.588783	NaN	
two	A	NaN	1.000985	
	B	0.158248	NaN	
	C	NaN	0.176180	

```
In [58]: pd.pivot_table(df, values='D', index=['B'], columns=['A', 'C'], aggfunc=np.sum)
```

```
Out[58]:
```

		A	one	three	two		
	C	bar	foo	bar	foo		
B	A	2.241830	-1.028115	-2.363137	NaN	NaN	2.001971
	B	-0.676843	0.005518	NaN	0.867024	0.316495	NaN
	C	-1.077692	1.399070	1.177566	NaN	NaN	0.352360

```
In [59]: pd.pivot_table(df, values=['D', 'E'], index=['B'], columns=['A', 'C'],
```

```
.....: aggfunc=np.sum)
```

```
.....:
```

```
Out[59]:
```

		D	E								
A	C	one	three	two	one						
		bar	foo	bar	foo	bar	foo	bar	foo	bar	
	B	foo	bar	foo							
B	A	2.241830	-1.028115	-2.363137	NaN	NaN	2.001971	2.786113	-0.043211	1.922577	
		NaN	NaN	0.128491							
	B	-0.676843	0.005518	NaN	0.867024	0.316495	NaN	1.368280	-1.103384	NaN	-2.128743
C		-0.194294	NaN								
	A	-1.077692	1.399070	1.177566	NaN	NaN	0.352360	-1.976883	1.495717	-0.263660	
		NaN	NaN	0.872482							

The result object is a `DataFrame` having potentially hierarchical indexes on the rows and columns. If the `values` column name is not given, the pivot table will include all of the data that can be aggregated in an additional level of hierarchy in the columns:

```
In [60]: pd.pivot_table(df, index=['A', 'B'], columns=['C'])
```

```
Out[60]:
```

		D		E	
C		bar	foo	bar	foo
one	A	1.120915	-0.514058	1.393057	-0.021605
	B	-0.338421	0.002759	0.684140	-0.551692
	C	-0.538846	0.699535	-0.988442	0.747859
three	A	-1.181568	NaN	0.961289	NaN
	B	NaN	0.433512	NaN	-1.064372
	C	0.588783	NaN	-0.131830	NaN
two	A	NaN	1.000985	NaN	0.064245
	B	0.158248	NaN	-0.097147	NaN
	C	NaN	0.176180	NaN	0.436241

Also, you can use `Grouper` for `index` and `columns` keywords. For detail of `Grouper`, see [Grouping with a Grouper specification](#).

```
In [61]: pd.pivot_table(df, values='D', index=pd.Grouper(freq='M', key='F'),
```

```
.....: columns='C')
```

```
.....:
```

```
Out[61]:
```

C	bar	foo
F		
2013-01-31	NaN	-0.514058
2013-02-28	NaN	0.002759
2013-03-31	NaN	0.176180
2013-04-30	-1.181568	NaN
2013-05-31	-0.338421	NaN
2013-06-30	-0.538846	NaN
2013-07-31	NaN	1.000985
2013-08-31	NaN	0.433512
2013-09-30	NaN	0.699535
2013-10-31	1.120915	NaN
2013-11-30	0.158248	NaN
2013-12-31	0.588783	NaN

You can render a nice output of the table omitting the missing values by calling `to_string` if you wish:

```
In [62]: table = pd.pivot_table(df, index=['A', 'B'], columns=['C'])
```

```
In [63]: print(table.to_string(na_rep=''))
```

		D		E	
C		bar	foo	bar	foo
one	A	1.120915	-0.514058	1.393057	-0.021605
	B	-0.338421	0.002759	0.684140	-0.551692
	C	-0.538846	0.699535	-0.988442	0.747859
three	A	-1.181568		0.961289	
	B		0.433512		-1.064372
	C	0.588783		-0.131830	
two	A		1.000985		0.064245
	B	0.158248		-0.097147	
	C		0.176180		0.436241

Note that `pivot_table` is also available as an instance method on `DataFrame`,

## Adding margins

If you pass `margins=True` to `pivot_table`, special `ALL` columns and rows will be added with partial group aggregates across the categories on the rows and columns:

```
In [64]: df.pivot_table(index=['A', 'B'], columns='C', margins=True, aggfunc=np.std)
```

```
Out[64]:
```

		D			E		
C		bar	foo	ALL	bar	foo	ALL
one	A	1.804346	1.210272	1.569879	0.179483	0.418374	0.858005
	B	0.690376	1.353355	0.898998	1.083825	0.968138	1.101401
	C	0.273641	0.418926	0.771139	1.689271	0.446140	1.422136
three	A	0.794212	NaN	0.794212	2.049040	NaN	2.049040
	B	NaN	0.363548	0.363548	NaN	1.625237	1.625237
	C	3.915454	NaN	3.915454	1.035215	NaN	1.035215
two	A	NaN	0.442998	0.442998	NaN	0.447104	0.447104
	B	0.202765	NaN	0.202765	0.560757	NaN	0.560757
	C	NaN	1.819408	1.819408	NaN	0.650439	0.650439
ALL		1.556686	0.952552	1.246608	1.250924	0.899904	1.059389

## Cross tabulations

Use `crosstab()` to compute a cross-tabulation of two (or more) factors. By default `crosstab` computes a frequency table of the factors unless an array of values and an aggregation function are passed.

It takes a number of arguments

- `index`: array-like, values to group by in the rows.
- `columns`: array-like, values to group by in the columns.
- `values`: array-like, optional, array of values to aggregate according to the factors.
- `aggfunc`: function, optional, If no values array is passed, computes a frequency table.

- `rownames`: sequence, default `None`, must match number of row arrays passed.
- `colnames`: sequence, default `None`, if passed, must match number of column arrays passed.
- `margins`: boolean, default `False`, Add row/column margins (subtotals)
- `normalize`: boolean, { 'all' , 'index' , 'columns' }, or {0,1}, default `False`. Normalize by dividing all values by the sum of values.

Any `Series` passed will have their name attributes used unless row or column names for the cross-tabulation are specified

For example:

```
In [65]: foo, bar, dull, shiny, one, two = 'foo', 'bar', 'dull', 'shiny', 'one', 'two'

In [66]: a = np.array([foo, foo, bar, bar, foo, foo], dtype=object)

In [67]: b = np.array([one, one, two, one, two, one], dtype=object)

In [68]: c = np.array([dull, dull, shiny, dull, dull, shiny], dtype=object)

In [69]: pd.crosstab(a, [b, c], rownames=['a'], colnames=['b', 'c'])
Out[69]:
b one two
c dull shiny dull shiny
a
bar 1 0 0 1
foo 2 1 1 0
```

If `crosstab` receives only two Series, it will provide a frequency table.

```
In [70]: df = pd.DataFrame({'A': [1, 2, 2, 2, 2], 'B': [3, 3, 4, 4, 4],
.....: 'C': [1, 1, np.nan, 1, 1]})
.....:

In [71]: df
Out[71]:
 A B C
0 1 3 1.0
1 2 3 1.0
2 2 4 NaN
3 2 4 1.0
4 2 4 1.0

In [72]: pd.crosstab(df.A, df.B)
Out[72]:
B 3 4
A
1 1 0
2 1 3
```

Any input passed containing `Categorical` data will have **all** of its categories included in the cross-tabulation, even if the actual data does not contain any instances of a particular category.

```
In [73]: foo = pd.Categorical(['a', 'b'], categories=['a', 'b', 'c'])

In [74]: bar = pd.Categorical(['d', 'e'], categories=['d', 'e', 'f'])

In [75]: pd.crosstab(foo, bar)
Out[75]:
col_0 d e
row_0
a 1 0
b 0 1
```

## Normalization

New in version 0.18.1.

Frequency tables can also be normalized to show percentages rather than counts using the `normalize` argument:

```
In [76]: pd.crosstab(df.A, df.B, normalize=True)
Out[76]:
B 3 4
A
1 0.2 0.0
2 0.2 0.6
```

`normalize` can also normalize values within each row or within each column:

```
In [77]: pd.crosstab(df.A, df.B, normalize='columns')
Out[77]:
B 3 4
A
1 0.5 0.0
2 0.5 1.0
```

`crosstab` can also be passed a third `Series` and an aggregation function (`aggfunc`) that will be applied to the values of the third `Series` within each group defined by the first two `Series`:

```
In [78]: pd.crosstab(df.A, df.B, values=df.C, aggfunc=np.sum)
Out[78]:
B 3 4
A
1 1.0 NaN
2 1.0 2.0
```

## Adding margins

Finally, one can also add margins or normalize this output.

```
In [79]: pd.crosstab(df.A, df.B, values=df.C, aggfunc=np.sum, normalize=True,
.....: margins=True)
.....:
Out[79]:
B 3 4 All
A
1 0.25 0.0 0.25
2 0.25 0.5 0.75
All 0.50 0.5 1.00
```

## Tiling

The `cut()` function computes groupings for the values of the input array and is often used to transform continuous variables to discrete or categorical variables:

```
In [80]: ages = np.array([10, 15, 13, 12, 23, 25, 28, 59, 60])

In [81]: pd.cut(ages, bins=3)
Out[81]:
[(9.95, 26.667], (9.95, 26.667], (9.95, 26.667], (9.95, 26.667], (9.95, 26.667], (9.95, 26.667],
 (26.667, 43.333], (43.333, 60.0], (43.333, 60.0]]
Categories (3, interval[float64]): [(9.95, 26.667] < (26.667, 43.333] < (43.333, 60.0]]
```

If the `bins` keyword is an integer, then equal-width bins are formed. Alternatively we can specify custom bin-edges:

```
In [82]: c = pd.cut(ages, bins=[0, 18, 35, 70])

In [83]: c
Out[83]:
[(0, 18], (0, 18], (0, 18], (0, 18], (18, 35], (18, 35], (18, 35], (35, 70], (35, 70]]
Categories (3, interval[int64]): [(0, 18] < (18, 35] < (35, 70]]
```

New in version 0.20.0.

If the `bins` keyword is an `IntervalIndex`, then these will be used to bin the passed data.:

```
pd.cut([25, 20, 50], bins=c.categories)
```

## Computing indicator / dummy variables

To convert a categorical variable into a “dummy” or “indicator” `DataFrame`, for example a column in a `DataFrame` (a `Series`) which has `k` distinct values, can derive a `DataFrame` containing `k` columns of 1s and 0s using `get_dummies()`:

```
In [84]: df = pd.DataFrame({'key': list('bbacab'), 'data1': range(6)})
```

```
In [85]: pd.get_dummies(df['key'])
```

```
Out[85]:
```

	a	b	c
0	0	1	0
1	0	1	0
2	1	0	0
3	0	0	1
4	1	0	0
5	0	1	0

Sometimes it's useful to prefix the column names, for example when merging the result with the original DataFrame:

```
In [86]: dummies = pd.get_dummies(df['key'], prefix='key')
```

```
In [87]: dummies
```

```
Out[87]:
```

	key_a	key_b	key_c
0	0	1	0
1	0	1	0
2	1	0	0
3	0	0	1
4	1	0	0
5	0	1	0

```
In [88]: df[['data1']].join(dummies)
```

```
Out[88]:
```

	data1	key_a	key_b	key_c
0	0	0	1	0
1	1	0	1	0
2	2	1	0	0
3	3	0	0	1
4	4	1	0	0
5	5	0	1	0

This function is often used along with discretization functions like `cut`:



```
In [89]: values = np.random.randn(10)

In [90]: values
Out[90]:
array([0.4082, -1.0481, -0.0257, -0.9884, 0.0941, 1.2627, 1.29 ,
 0.0824, -0.0558, 0.5366])

In [91]: bins = [0, 0.2, 0.4, 0.6, 0.8, 1]

In [92]: pd.get_dummies(pd.cut(values, bins))
Out[92]:
```

	(0.0, 0.2]	(0.2, 0.4]	(0.4, 0.6]	(0.6, 0.8]	(0.8, 1.0]
0	0	0	1	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	1	0	0	0	0
5	0	0	0	0	0
6	0	0	0	0	0
7	1	0	0	0	0
8	0	0	0	0	0
9	0	0	1	0	0

See also [Series.str.get\\_dummies](#).

[get\\_dummies\(\)](#) also accepts a `DataFrame`. By default all categorical variables (categorical in the statistical sense, those with object or categorical dtype) are encoded as dummy variables.

```
In [93]: df = pd.DataFrame({'A': ['a', 'b', 'a'], 'B': ['c', 'c', 'b'],
.....: 'C': [1, 2, 3]})
.....:

In [94]: pd.get_dummies(df)
Out[94]:
```

	C	A_a	A_b	B_b	B_c
0	1	1	0	0	1
1	2	0	1	0	1
2	3	1	0	1	0

All non-object columns are included untouched in the output. You can control the columns that are encoded with the `columns` keyword.

```
In [95]: pd.get_dummies(df, columns=['A'])
Out[95]:
```

	B	C	A_a	A_b
0	c	1	1	0
1	c	2	0	1
2	b	3	1	0

Notice that the `B` column is still included in the output, it just hasn't been encoded. You can drop `B` before calling `get_dummies` if you don't want to include it in the output.

As with the `Series` version, you can pass values for the `prefix` and `prefix_sep`. By default the column name is used as the prefix, and `'_'` as the prefix separator. You can specify `prefix` and `prefix_sep` in 3 ways:

- string: Use the same value for `prefix` or `prefix_sep` for each column to be encoded.
- list: Must be the same length as the number of columns being encoded.
- dict: Mapping column name to prefix.

```
In [96]: simple = pd.get_dummies(df, prefix='new_prefix')
```

```
In [97]: simple
```

```
Out[97]:
```

	C	new_prefix_a	new_prefix_b	new_prefix_b	new_prefix_c
0	1	1	0	0	1
1	2	0	1	0	1
2	3	1	0	1	0

```
In [98]: from_list = pd.get_dummies(df, prefix=['from_A', 'from_B'])
```

```
In [99]: from_list
```

```
Out[99]:
```

	C	from_A_a	from_A_b	from_B_b	from_B_c
0	1	1	0	0	1
1	2	0	1	0	1
2	3	1	0	1	0

```
In [100]: from_dict = pd.get_dummies(df, prefix={'B': 'from_B', 'A': 'from_A'})
```

```
In [101]: from_dict
```

```
Out[101]:
```

	C	from_A_a	from_A_b	from_B_b	from_B_c
0	1	1	0	0	1
1	2	0	1	0	1
2	3	1	0	1	0

New in version 0.18.0.

Sometimes it will be useful to only keep  $k-1$  levels of a categorical variable to avoid collinearity when feeding the result to statistical models. You can switch to this mode by turn on `drop_first`.

```
In [102]: s = pd.Series(list('abcaa'))

In [103]: pd.get_dummies(s)
Out[103]:
```

	a	b	c
0	1	0	0
1	0	1	0
2	0	0	1
3	1	0	0
4	1	0	0

```

In [104]: pd.get_dummies(s, drop_first=True)
Out[104]:
```

	b	c
0	0	0
1	1	0
2	0	1
3	0	0
4	0	0

When a column contains only one level, it will be omitted in the result.

```
In [105]: df = pd.DataFrame({'A': list('aaaaa'), 'B': list('ababc')})

In [106]: pd.get_dummies(df)
Out[106]:
```

	A_a	B_a	B_b	B_c
0	1	1	0	0
1	1	0	1	0
2	1	1	0	0
3	1	0	1	0
4	1	0	0	1

```

In [107]: pd.get_dummies(df, drop_first=True)
Out[107]:
```

	B_b	B_c
0	0	0
1	1	0
2	0	0
3	1	0
4	0	1

By default new columns will have `np.uint8` dtype. To choose another dtype, use the `dtype` argument:

```
In [108]: df = pd.DataFrame({'A': list('abc'), 'B': [1.1, 2.2, 3.3]})

In [109]: pd.get_dummies(df, dtype=bool).dtypes
Out[109]:
B float64
A_a bool
A_b bool
A_c bool
dtype: object
```

New in version 0.23.0.

## Factorizing values

To encode 1-d values as an enumerated type use `factorize()`:

```
In [110]: x = pd.Series(['A', 'A', np.nan, 'B', 3.14, np.inf])

In [111]: x
Out[111]:
0 A
1 A
2 NaN
3 B
4 3.14
5 inf
dtype: object

In [112]: labels, uniques = pd.factorize(x)

In [113]: labels
Out[113]: array([0, 0, -1, 1, 2, 3])

In [114]: uniques
Out[114]: Index(['A', 'B', 3.14, inf], dtype='object')
```

Note that `factorize` is similar to `numpy.unique`, but differs in its handling of NaN:

::: tip Note

The following `numpy.unique` will fail under Python 3 with a `TypeError` because of an ordering bug. See also [here](#).

:::

```
In [1]: x = pd.Series(['A', 'A', np.nan, 'B', 3.14, np.inf])
In [2]: pd.factorize(x, sort=True)
Out[2]:
(array([2, 2, -1, 3, 0, 1]),
 Index([3.14, inf, 'A', 'B'], dtype='object'))

In [3]: np.unique(x, return_inverse=True)[:,-1]
Out[3]: (array([3, 3, 0, 4, 1, 2]), array([nan, 3.14, inf, 'A', 'B'], dtype=object))
```

⋮ tip Note

If you just want to handle one column as a categorical variable (like R's factor), you can use `df["cat_col"] = pd.Categorical(df["col"])` or `df["cat_col"] = df["col"].astype("category")`. For full docs on [Categorical](#), see the [Categorical introduction](#) and the [API documentation](#).

⋮

## Examples

---

In this section, we will review frequently asked questions and examples. The column names and relevant column values are named to correspond with how this DataFrame will be pivoted in the answers below.

```

In [115]: np.random.seed([3, 1415])

In [116]: n = 20

In [117]: cols = np.array(['key', 'row', 'item', 'col'])

In [118]: df = cols + pd.DataFrame((np.random.randint(5, size=(n, 4))
.....: // [2, 1, 2, 1]).astype(str))
.....:

In [119]: df.columns = cols

In [120]: df = df.join(pd.DataFrame(np.random.rand(n, 2).round(2)).add_prefix('val'))

In [121]: df
Out[121]:
 key row item col val0 val1
0 key0 row3 item1 col3 0.81 0.04
1 key1 row2 item1 col2 0.44 0.07
2 key1 row0 item1 col0 0.77 0.01
3 key0 row4 item0 col2 0.15 0.59
4 key1 row0 item2 col1 0.81 0.64
..
15 key0 row3 item1 col1 0.31 0.23
16 key0 row0 item2 col3 0.86 0.01
17 key0 row4 item0 col3 0.64 0.21
18 key2 row2 item2 col0 0.13 0.45
19 key0 row2 item0 col4 0.37 0.70

[20 rows x 6 columns]

```

## Pivoting with single aggregations

Suppose we wanted to pivot `df` such that the `col` values are columns, `row` values are the index, and the mean of `val0` are the values? In particular, the resulting DataFrame should look like:

::: tip Note

```

col col0 col1 col2 col3 col4 row row0 0.77 0.605 NaN 0.860 0.65 row2 0.13 NaN 0.395 0.500 0.25 row3
NaN 0.310 NaN 0.545 NaN row4 NaN 0.100 0.395 0.760 0.24

```

:::

This solution uses `pivot_table()`. Also note that `aggfunc='mean'` is the default. It is included here to be explicit.

```
In [122]: df.pivot_table(
.....: values='val0', index='row', columns='col', aggfunc='mean')
.....:
Out[122]:
```

col	col0	col1	col2	col3	col4
row					
row0	0.77	0.605	NaN	0.860	0.65
row2	0.13	NaN	0.395	0.500	0.25
row3	NaN	0.310	NaN	0.545	NaN
row4	NaN	0.100	0.395	0.760	0.24

Note that we can also replace the missing values by using the `fill_value` parameter.

```
In [123]: df.pivot_table(
.....: values='val0', index='row', columns='col', aggfunc='mean', fill_value=0)
.....:
Out[123]:
```

col	col0	col1	col2	col3	col4
row					
row0	0.77	0.605	0.000	0.860	0.65
row2	0.13	0.000	0.395	0.500	0.25
row3	0.00	0.310	0.000	0.545	0.00
row4	0.00	0.100	0.395	0.760	0.24

Also note that we can pass in other aggregation functions as well. For example, we can also pass in `sum`.

```
In [124]: df.pivot_table(
.....: values='val0', index='row', columns='col', aggfunc='sum', fill_value=0)
.....:
Out[124]:
```

col	col0	col1	col2	col3	col4
row					
row0	0.77	1.21	0.00	0.86	0.65
row2	0.13	0.00	0.79	0.50	0.50
row3	0.00	0.31	0.00	1.09	0.00
row4	0.00	0.10	0.79	1.52	0.24

Another aggregation we can do is calculate the frequency in which the columns and rows occur together a.k.a. “cross tabulation”. To do this, we can pass `size` to the `aggfunc` parameter.

```
In [125]: df.pivot_table(index='row', columns='col', fill_value=0, aggfunc='size')
Out[125]:
```

col	col0	col1	col2	col3	col4
row					
row0	1	2	0	1	1
row2	1	0	2	1	2
row3	0	1	0	2	0
row4	0	1	2	2	1

## Pivoting with multiple aggregations

We can also perform multiple aggregations. For example, to perform both a `sum` and `mean`, we can pass in a list to the `aggfunc` argument.

```
In [126]: df.pivot_table(
.....: values='val0', index='row', columns='col', aggfunc=['mean', 'sum'])
.....:
Out[126]:
```

	mean					sum				
col	col0	col1	col2	col3	col4	col0	col1	col2	col3	col4
row										
row0	0.77	0.605	NaN	0.860	0.65	0.77	1.21	NaN	0.86	0.65
row2	0.13	NaN	0.395	0.500	0.25	0.13	NaN	0.79	0.50	0.50
row3	NaN	0.310	NaN	0.545	NaN	NaN	0.31	NaN	1.09	NaN
row4	NaN	0.100	0.395	0.760	0.24	NaN	0.10	0.79	1.52	0.24

Note to aggregate over multiple value columns, we can pass in a list to the `values` parameter.

```
In [127]: df.pivot_table(
.....: values=['val0', 'val1'], index='row', columns='col', aggfunc=['mean'])
.....:
Out[127]:
```

	mean					val1				
col	col0	col1	col2	col3	col4	col0	col1	col2	col3	col4
row										
row0	0.77	0.605	NaN	0.860	0.65	0.01	0.745	NaN	0.010	0.02
row2	0.13	NaN	0.395	0.500	0.25	0.45	NaN	0.34	0.440	0.79
row3	NaN	0.310	NaN	0.545	NaN	NaN	0.230	NaN	0.075	NaN
row4	NaN	0.100	0.395	0.760	0.24	NaN	0.070	0.42	0.300	0.46

Note to subdivide over multiple columns we can pass in a list to the `columns` parameter.

```
In [128]: df.pivot_table(
.....: values=['val0'], index='row', columns=['item', 'col'], aggfunc=['mean'])
.....:
Out[128]:
```

	mean				val0				item2			
item	item0	item1			col2	col3	col4	col0	col1	col3	col4	
col	col2	col3	col4	col0	col1	col2	col3	col4	col0	col1	col3	col4
row												
row0	NaN	NaN	NaN	0.77	NaN	NaN	NaN	NaN	NaN	0.605	0.86	0.65
row2	0.35	NaN	0.37	NaN	NaN	0.44	NaN	NaN	0.13	NaN	0.50	0.13
row3	NaN	NaN	NaN	NaN	0.31	NaN	0.81	NaN	NaN	NaN	0.28	NaN
row4	0.15	0.64	NaN	NaN	0.10	0.64	0.88	0.24	NaN	NaN	NaN	NaN



## Exploding a list-like column

New in version 0.25.0.

Sometimes the values in a column are list-like.

```
In [129]: keys = ['panda1', 'panda2', 'panda3']

In [130]: values = [['eats', 'shoots'], ['shoots', 'leaves'], ['eats', 'leaves']]

In [131]: df = pd.DataFrame({'keys': keys, 'values': values})

In [132]: df
Out[132]:
```

	keys	values
0	panda1	[eats, shoots]
1	panda2	[shoots, leaves]
2	panda3	[eats, leaves]

We can ‘explode’ the `values` column, transforming each list-like to a separate row, by using `explode()`. This will replicate the index values from the original row:

```
In [133]: df['values'].explode()
Out[133]:
```

0	eats
0	shoots
1	shoots
1	leaves
2	eats
2	leaves

Name: values, dtype: object

You can also explode the column in the `DataFrame`.

```
In [134]: df.explode('values')
Out[134]:
```

	keys	values
0	panda1	eats
0	panda1	shoots
1	panda2	shoots
1	panda2	leaves
2	panda3	eats
2	panda3	leaves

`Series.explode()` will replace empty lists with `np.nan` and preserve scalar entries. The dtype of the resulting `Series` is always `object`.

```
In [135]: s = pd.Series([[1, 2, 3], 'foo', [], ['a', 'b']])
```

```
In [136]: s
Out[136]:
0 [1, 2, 3]
1 foo
2 []
3 [a, b]
dtype: object
```

```
In [137]: s.explode()
Out[137]:
0 1
0 2
0 3
1 foo
2 NaN
3 a
3 b
dtype: object
```

Here is a typical usecase. You have comma separated strings in a column and want to expand this.

```
In [138]: df = pd.DataFrame([{'var1': 'a,b,c', 'var2': 1},
.....: {'var1': 'd,e,f', 'var2': 2}])
.....:
```

```
In [139]: df
Out[139]:
 var1 var2
0 a,b,c 1
1 d,e,f 2
```

Creating a long form DataFrame is now straightforward using explode and chained operations

```
In [140]: df.assign(var1=df.var1.str.split(',')).explode('var1')
Out[140]:
 var1 var2
0 a 1
0 b 1
0 c 1
1 d 2
1 e 2
1 f 2
```

---

## Pandas 处理文本字符串

序列和索引包含一系列的字符操作方法，这可以使我们轻易操作数组中的各个元素。最重要的是，这些方法可以自动跳过 缺失/NA 值。这些方法可以在 `str` 属性中访问到，并且基本上和python内建的（标量）字符串方法同名：

```
In [1]: s = pd.Series(['A', 'B', 'C', 'Aaba', 'Baca', np.nan, 'CABA', 'dog', 'cat'])
```

```
In [2]: s.str.lower()
```

```
Out[2]:
```

```
0 a
1 b
2 c
3 aaba
4 baca
5 NaN
6 caba
7 dog
8 cat
```

```
dtype: object
```

```
In [3]: s.str.upper()
```

```
Out[3]:
```

```
0 A
1 B
2 C
3 AABA
4 BACA
5 NaN
6 CABA
7 DOG
8 CAT
```

```
dtype: object
```

```
In [4]: s.str.len()
```

```
Out[4]:
```

```
0 1.0
1 1.0
2 1.0
3 4.0
4 4.0
5 NaN
6 4.0
7 3.0
8 3.0
```

```
dtype: float64
```

```
In [5]: idx = pd.Index([' jack', 'jill ', ' jesse ', 'frank'])
```

```
In [6]: idx.str.strip()
```

```
Out[6]: Index(['jack', 'jill', 'jesse', 'frank'], dtype='object')
```

```
In [7]: idx.str.lstrip()
```

```
Out[7]: Index(['jack', 'jill ', 'jesse ', 'frank'], dtype='object')
```

```
In [8]: idx.str.rstrip()
```

```
Out[8]: Index([' jack', 'jill', ' jesse', 'frank'], dtype='object')
```

索引的字符串方法在清理或者转换数据表列的时候非常有用。例如，你的列中或许会包含首位的白空格：

```
In [9]: df = pd.DataFrame(np.random.randn(3, 2),
...: columns=[' Column A ', ' Column B '], index=range(3))
...:

In [10]: df
Out[10]:
```

	Column A	Column B
0	0.469112	-0.282863
1	-1.509059	-1.135632
2	1.212112	-0.173215

Since `df.columns` is an Index object, we can use the `.str` accessor

```
In [11]: df.columns.str.strip()
Out[11]: Index(['Column A', 'Column B'], dtype='object')

In [12]: df.columns.str.lower()
Out[12]: Index(['column a ', 'column b '], dtype='object')
```

这些字符串方法可以被用来清理需要的列。这里，我们想清理开头和结尾的白空格，将所有的名称都换为小写，并且将其余的空格都替换为下划线：

```
In [13]: df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

In [14]: df
Out[14]:
```

	column_a	column_b
0	0.469112	-0.282863
1	-1.509059	-1.135632
2	1.212112	-0.173215

### ::: tip 小贴士

如果你有一个序列，里面有很多重复的值（即，序列中唯一元素的数量远小于序列的长度），将原有序列转换为一种分类类型，然后使用 `.str` 或者 `.dt` 方法，则会获得更快的速度。速度的差异来源于，在分类类型的序列中，字符操作只是在 `categories` 中完成的，而不是针对序列中的每一个元素。

请注意，相比于字符串类型的序列，带 `.categories` 类型的分类类别的序列有一些限制（例如，你不能像其中的元素追加其他的字符串：`s + " " + s` 将不能正确工作，如果 `s` 是一个分类类型的序列。并且，`.str` 中，那些可以对列表（list）类型的元素进行操作的方法，在分类序列中也无法使用。

:::

### ::: danger 警告

v.0.25.0版以前，`.str` 访问器只会进行最基本的类型检查。从v.0.25.0起，序列的类型会被自动推断出来，并且会更为激进地使用恰当的类型。

一般来说 `.str` 访问器只倾向于针对字符串类型工作。只有在个别的情况下，才能对非字符串类型工作，但是这也将会在未来的版本中被逐步修正 :::

## 拆分和替换字符串

类似 `split` 的方法返回一个列表类型的序列：

```
In [15]: s2 = pd.Series(['a_b_c', 'c_d_e', np.nan, 'f_g_h'])

In [16]: s2.str.split('_')
Out[16]:
0 [a, b, c]
1 [c, d, e]
2 NaN
3 [f, g, h]
dtype: object
```

切分后的列表中的元素可以通过 `get` 方法或者 `[]` 方法进行读取：

```
In [17]: s2.str.split('_').str.get(1)
Out[17]:
0 b
1 d
2 NaN
3 g
dtype: object

In [18]: s2.str.split('_').str[1]
Out[18]:
0 b
1 d
2 NaN
3 g
dtype: object
```

使用 `expand` 方法可以轻易地将这种返回展开为一个数据表。

```
In [19]: s2.str.split('_', expand=True)
Out[19]:
 0 1 2
0 a b c
1 c d e
2 NaN NaN NaN
3 f g h
```

同样，我们也可以限制切分的次数：

```
In [20]: s2.str.split('_', expand=True, n=1)
Out[20]:
 0 1
0 a b_c
1 c d_e
2 NaN NaN
3 f g_h
```

`rsplit` 与 `split` 相似，不同的是，这个切分的方向是反的。即，从字串的尾端向首段切分：

```
In [21]: s2.str.rsplit('_', expand=True, n=1)
Out[21]:
 0 1
0 a_b c
1 c_d e
2 NaN NaN
3 f_g h
```

`replace` 方法默认使用 [正则表达式](#)：

```
In [22]: s3 = pd.Series(['A', 'B', 'C', 'Aaba', 'Baca',
.....: '', np.nan, 'CABA', 'dog', 'cat'])
.....:

In [23]: s3
Out[23]:
0 A
1 B
2 C
3 Aaba
4 Baca
5
6 NaN
7 CABA
8 dog
9 cat
dtype: object

In [24]: s3.str.replace('^.a|dog', 'XX-XX ', case=False)
Out[24]:
0 A
1 B
2 C
3 XX-XX ba
4 XX-XX ca
5
6 NaN
7 XX-XX BA
8 XX-XX
9 XX-XX t
dtype: object
```

一定要时时记得，是正则表达式，因此要格外小心。例如，因为正则表达式中的\*\$\$符号,下列代码将会导致一些麻烦：

```
Consider the following badly formatted financial data
In [25]: dollars = pd.Series(['12', '-$10', '$10,000'])

This does what you'd naively expect:
In [26]: dollars.str.replace('$', '')
Out[26]:
0 12
1 -10
2 10,000
dtype: object

But this doesn't:
In [27]: dollars.str.replace('-$', '-')
Out[27]:
0 12
1 -$10
2 $10,000
dtype: object

We need to escape the special character (for >1 len patterns)
In [28]: dollars.str.replace(r'\$', '-')
Out[28]:
0 12
1 -10
2 $10,000
dtype: object
```

#### v0.23.0. 新加入

如果你只是向单纯地替换字符 (等价于python中的 `str.replace()`)，你可以将可选参数 `regex` 设置为 `False`，而不是傻傻地转义所有符号。这种情况下，`pat` 和 `repl` 就都将作为普通字符对待：

```
These lines are equivalent
In [29]: dollars.str.replace(r'\$', '-')
Out[29]:
0 12
1 -10
2 $10,000
dtype: object

In [30]: dollars.str.replace('-$', '-', regex=False)
Out[30]:
0 12
1 -10
2 $10,000
dtype: object
```

#### v0.20.0. 新加入

`replace` 方法也可以传入一个可调用对象作为替换值。它针对每一个 `pat` 通过 `re.sub()` 来调用。可调用对象应只具有一个形参（一个正则表达式对象）并且返回一个字符串。

```
Reverse every lowercase alphabetic word
In [31]: pat = r'[a-z]+'

In [32]: def repl(m):
....: return m.group(0)[::-1]
....:

In [33]: pd.Series(['foo 123', 'bar baz', np.nan]).str.replace(pat, repl)
Out[33]:
0 oof 123
1 rab zab
2 NaN
dtype: object

Using regex groups
In [34]: pat = r"(?P<one>\w+) (?P<two>\w+) (?P<three>\w+)"

In [35]: def repl(m):
....: return m.group('two').swapcase()
....:

In [36]: pd.Series(['Foo Bar Baz', np.nan]).str.replace(pat, repl)
Out[36]:
0 bAR
1 NaN
dtype: object
```

## v0.20.0. 新加入

`replace` 方法也可以接受一个来自 `re.compile()` 编译过的正则表达式对象，来做为 表达式。所有的标记都应该被包含在这个已经编译好的正则表达式对象中。

```
In [37]: import re

In [38]: regex_pat = re.compile(r'^.a|dog', flags=re.IGNORECASE)

In [39]: s3.str.replace(regex_pat, 'XX-XX ')
Out[39]:
0 A
1 B
2 C
3 XX-XX ba
4 XX-XX ca
5
6 NaN
7 XX-XX BA
8 XX-XX
9 XX-XX t
dtype: object
```



如果在已经使用编译的正则对象中继续传入 `flags` 参数，并进行替换，将会导致 `ValueError`。

```
In [40]: s3.str.replace(regex_pat, 'XX-XX ', flags=re.IGNORECASE)

ValueError: case and flags cannot be set when pat is a compiled regex
```

## 拼接

Pandas提供了不同的方法将 序列 或 索引 与他们自己或者其他的对象进行拼接，所有的方法都是基于各自的 `cat()`, resp. `Index.str.cat`。

### 将单个序列拼接为一个完整字符串

序列 或 索引 的内容可以进行拼接：

```
In [41]: s = pd.Series(['a', 'b', 'c', 'd'])

In [42]: s.str.cat(sep=',')
Out[42]: 'a,b,c,d'
```

如果没有额外声明，`sep` 即分隔符默认为空字符串，即 `sep=''`：

```
In [43]: s.str.cat()
Out[43]: 'abcd'
```

默认情况下，缺失值会被忽略。使用 `na_rep` 参数，可以对缺失值进行赋值：

```
In [44]: t = pd.Series(['a', 'b', np.nan, 'd'])

In [45]: t.str.cat(sep=',')
Out[45]: 'a,b,d'

In [46]: t.str.cat(sep=',', na_rep='-')
Out[46]: 'a,b,-,d'
```

### 拼接序列和其他类列表型对象为新的序列

`cat()` 的第一个参数为类列表对象，但必须要确保长度与 序列 或 索引 相同。

```
In [47]: s.str.cat(['A', 'B', 'C', 'D'])
Out[47]:
0 aA
1 bB
2 cC
3 dD
dtype: object
```

任何一端的缺失值都会导致之中结果为缺失值，除非使用 `na_rep`：

```
In [48]: s.str.cat(t)
Out[48]:
0 aa
1 bb
2 NaN
3 dd
dtype: object

In [49]: s.str.cat(t, na_rep='-')
Out[49]:
0 aa
1 bb
2 c-
3 dd
dtype: object
```

## 拼接序列与类数组对象为新的序列

v0.23.0. 新加入

`others` 参数可以是二维的。此时，行数需要与 `序列` 或 `索引` 的长度相同。

```
In [50]: d = pd.concat([t, s], axis=1)

In [51]: s
Out[51]:
0 a
1 b
2 c
3 d
dtype: object

In [52]: d
Out[52]:
 0 1
0 a a
1 b b
2 NaN c
3 d d

In [53]: s.str.cat(d, na_rep='-')
Out[53]:
0 aaa
1 bbb
2 c-c
3 ddd
dtype: object
```

## 对齐拼接序列与带索引的对象成为新的序列

v0.23.0.新加入

对于拼接 序列 或者 数据表，我们可以使用 `join` 关键字来对齐索引。

```
In [54]: u = pd.Series(['b', 'd', 'a', 'c'], index=[1, 3, 0, 2])
```

```
In [55]: s
```

```
Out[55]:
```

```
0 a
1 b
2 c
3 d
dtype: object
```

```
In [56]: u
```

```
Out[56]:
```

```
1 b
3 d
0 a
2 c
dtype: object
```

```
In [57]: s.str.cat(u)
```

```
Out[57]:
```

```
0 ab
1 bd
2 ca
3 dc
dtype: object
```

```
In [58]: s.str.cat(u, join='left')
```

```
Out[58]:
```

```
0 aa
1 bb
2 cc
3 dd
dtype: object
```

⚠ danger 警告

如果不使用 `join` 关键字，`cat()` 方法将会滚回到0.23.0版之前，即（无对齐）模式。但如果任何的索引不一致时，将会抛出一个 `FutureWarning` 警告，因为在未来的版本中，默认行为将改为`join='left'`。

⋮

`join` 的选项为（`'left'`，`'outer'`，`'inner'`，`'right'`）中的一个。特别的，对齐操作使得两个对象可以是不同的长度。

```
In [59]: v = pd.Series(['z', 'a', 'b', 'd', 'e'], index=[-1, 0, 1, 3, 4])
```

```
In [60]: s
```

```
Out[60]:
```

```
0 a
```

```
1 b
```

```
2 c
```

```
3 d
```

```
dtype: object
```

```
In [61]: v
```

```
Out[61]:
```

```
-1 z
```

```
0 a
```

```
1 b
```

```
3 d
```

```
4 e
```

```
dtype: object
```

```
In [62]: s.str.cat(v, join='left', na_rep='-')
```

```
Out[62]:
```

```
0 aa
```

```
1 bb
```

```
2 c-
```

```
3 dd
```

```
dtype: object
```

```
In [63]: s.str.cat(v, join='outer', na_rep='-')
```

```
Out[63]:
```

```
-1 -z
```

```
0 aa
```

```
1 bb
```

```
2 c-
```

```
3 dd
```

```
4 -e
```

```
dtype: object
```

当 `others` 是一个数据表时，也可以执行相同的对齐操作：

```

In [64]: f = d.loc[[3, 2, 1, 0], :]

In [65]: s
Out[65]:
0 a
1 b
2 c
3 d
dtype: object

In [66]: f
Out[66]:
 0 1
3 d d
2 NaN c
1 b b
0 a a

In [67]: s.str.cat(f, join='left', na_rep='-')
Out[67]:
0 aaa
1 bbb
2 c-c
3 ddd
dtype: object

```

## 将一个序列与多个对象拼接为一个新的序列

所有的一维，类列表对象都可以任意组合进一个类列表的容器（包括迭代器，dict-视图等）：

```

In [68]: s
Out[68]:
0 a
1 b
2 c
3 d
dtype: object

In [69]: u
Out[69]:
1 b
3 d
0 a
2 c
dtype: object

In [70]: s.str.cat([u, u.to_numpy()], join='left')
Out[70]:
0 aab
1 bbd
2 cca
3 ddc
dtype: object

```

除了那些有索引的，所有传入没有索引的元素（如 `np.ndarray`）必须与 序列 或 索引 有相同的长度。但是，只要禁用对齐 `join=None`，那么 序列 或 索引 就可以是任意长度。

```
In [71]: v
Out[71]:
-1 z
 0 a
 1 b
 3 d
 4 e
dtype: object

In [72]: s.str.cat([v, u, u.to_numpy()], join='outer', na_rep='-')
Out[72]:
-1 -z--
 0 aaab
 1 bbbd
 2 c-ca
 3 dddc
 4 -e--
dtype: object
```

如果在一个包含不同的索引的 `others` 列表上使用 `join='right'`，所有索引的并集将会被作为最终拼接的基础：

```
In [73]: u.loc[[3]]
Out[73]:
3 d
dtype: object

In [74]: v.loc[[-1, 0]]
Out[74]:
-1 z
 0 a
dtype: object

In [75]: s.str.cat([u.loc[[3]], v.loc[[-1, 0]]], join='right', na_rep='-')
Out[75]:
-1 --z
 0 a-a
 3 dd-
dtype: object
```

## 使用.str进行索引

你可以使用 `[]` 方法来直接索引定位。如果你的索引超过了字符串的结尾，将返回 `NaN`。

```
In [76]: s = pd.Series(['A', 'B', 'C', 'Aaba', 'Baca', np.nan,
.....: 'CABA', 'dog', 'cat'])
.....:
```

```
In [77]: s.str[0]
```

```
Out[77]:
```

```
0 A
1 B
2 C
3 A
4 B
5 NaN
6 C
7 d
8 c
```

```
dtype: object
```

```
In [78]: s.str[1]
```

```
Out[78]:
```

```
0 NaN
1 NaN
2 NaN
3 a
4 a
5 NaN
6 A
7 o
8 a
```

```
dtype: object
```

## 提取子字符串

### 提取第一个匹配的对象 (extract)

::: danger 警告

在 0.18.0 中，`extract` 拥有了 `expand` 参数。当 `expand=False` 时，将返回一个序列，索引或者数据表，这取决于原对象和正则表达式（之前的版本也是如此）。当 `expand=True` 时，它则总是返回一个 `DataFrame`，这样可以更加一致，并且减少用户的混淆。`Expand=True` 从 0.23.0 版本之后成为默认值。

:::

`extract` 方法接受一个至少含有一个捕获组的 [正则表达式](#)。

使用超过一个捕获组的正则表达式则会提取并返回一个数据表，每一列为一个捕获组。

```
In [79]: pd.Series(['a1', 'b2', 'c3']).str.extract(r'([ab])(\d)', expand=False)
Out[79]:
 0 1
0 a 1
1 b 2
2 NaN NaN
```

没有成功匹配的元素将会返回一行 NaN。因此，一个序列的混乱的字符串可以被‘转换’为一个类似索引的序列或数据表。返回的内容会更为清爽，而且不需要使用 `get()` 方法来访问元组中的成员或者 `re.match` 对象。返回的类型将总是 `object` 类，即使匹配失败，返回的全是 NaN。

有名称的捕获组，如：

```
In [80]: pd.Series(['a1', 'b2', 'c3']).str.extract(r'(?P<letter>[ab])(?P<digit>\d)',
.....: expand=False)
.....:
Out[80]:
 letter digit
0 a 1
1 b 2
2 NaN NaN
```

可选组类似，如：

```
In [81]: pd.Series(['a1', 'b2', '3']).str.extract(r'([ab])?(\d)', expand=False)
Out[81]:
 0 1
0 a 1
1 b 2
2 NaN 3
```

也可以被使用。注意，任何有名称的捕获组，其名称都会被用做列名，否则将会直接使用数字。

如果仅使用正则表达式捕获一个组，而 `expand=True`，那么仍然将返回一个数据表。

```
In [82]: pd.Series(['a1', 'b2', 'c3']).str.extract(r'[ab](\d)', expand=True)
Out[82]:
 0
0 1
1 2
2 NaN
```

如果 `expand=False`，则会返回一个序列。



```
In [83]: pd.Series(['a1', 'b2', 'c3']).str.extract(r'[ab](\d)', expand=False)
Out[83]:
0 1
1 2
2 NaN
dtype: object
```

在索引上使用正则表达式，并且仅捕获一个组时，将会返回一个数据表，如果 `expand=True`。

```
In [84]: s = pd.Series(["a1", "b2", "c3"], ["A11", "B22", "C33"])

In [85]: s
Out[85]:
A11 a1
B22 b2
C33 c3
dtype: object

In [86]: s.index.str.extract("(?P<letter>[a-zA-Z])", expand=True)
Out[86]:
 letter
0 A
1 B
2 C
```

如果 `expand=False`，则返回一个 Index。

```
In [87]: s.index.str.extract("(?P<letter>[a-zA-Z])", expand=False)
Out[87]: Index(['A', 'B', 'C'], dtype='object', name='letter')
```

如果在索引上使用正则并捕获多个组，则返回一个数据表，如果 `expand=True`。

```
In [88]: s.index.str.extract("(?P<letter>[a-zA-Z])([0-9]+)", expand=True)
Out[88]:
 letter 1
0 A 11
1 B 22
2 C 33
```

如果 `expand=False`，则抛出 `ValueError`。

```
>>> s.index.str.extract("(?P<letter>[a-zA-Z])([0-9]+)", expand=False)
ValueError: only one regex group is supported with Index
```

下面的表格总结了 `extract (expand=False)` 时的行为（输入对象在第一列，捕获组的数量在第一行）

	1 group	>1 group
Index	Index	ValueError
Series	Series	DataFrame

## 提取所有的匹配 (extractall)

v0.18.0. 新加入

不同于 `extract` (只返回第一个匹配) ,

```
In [89]: s = pd.Series(["a1a2", "b1", "c1"], index=["A", "B", "C"])

In [90]: s
Out[90]:
A a1a2
B b1
C c1
dtype: object

In [91]: two_groups = '(?P<letter>[a-z])(?P<digit>[0-9])'

In [92]: s.str.extract(two_groups, expand=True)
Out[92]:
 letter digit
A a 1
B b 1
C c 1
```

`extractall` 方法返回所有的匹配。`extractall` 总是返回一个带有行多重索引的数据表，最后一级多重索引被命名为 `match`，它指出匹配的顺序

```
In [93]: s.str.extractall(two_groups)
Out[93]:
 letter digit
match
A 0 a 1
 1 a 2
B 0 b 1
C 0 c 1
```

当所有的对象字符串都只有一个匹配时，

```
In [94]: s = pd.Series(['a3', 'b3', 'c2'])

In [95]: s
Out[95]:
0 a3
1 b3
2 c2
dtype: object
```

`extractall(pat).xs(0, level='match')` 的返回与 `extract(pat)` 相同。

```
In [96]: extract_result = s.str.extract(two_groups, expand=True)
```

```
In [97]: extract_result
```

```
Out[97]:
```

	letter	digit
0	a	3
1	b	3
2	c	2

```
In [98]: extractall_result = s.str.extractall(two_groups)
```

```
In [99]: extractall_result
```

```
Out[99]:
```

		letter	digit
	match		
0	0	a	3
1	0	b	3
2	0	c	2

```
In [100]: extractall_result.xs(0, level="match")
```

```
Out[100]:
```

	letter	digit
0	a	3
1	b	3
2	c	2

索引也支持 `.str.extractall`。它返回一个数据表，其中包含与 `Series.str.extractall` 相同的结果，使用默认索引（从0开始）

v0.19.0. 新加入

```
In [101]: pd.Index(["a1a2", "b1", "c1"]).str.extractall(two_groups)
```

```
Out[101]:
```

		letter	digit
	match		
0	0	a	1
	1	a	2
1	0	b	1
2	0	c	1

```
In [102]: pd.Series(["a1a2", "b1", "c1"]).str.extractall(two_groups)
```

```
Out[102]:
```

		letter	digit
	match		
0	0	a	1
	1	a	2
1	0	b	1
2	0	c	1

## 测试匹配或包含模式的字符串

你可以检查是否一个元素包含一个可以匹配到的正则表达式：

```
In [103]: pattern = r'[0-9][a-z]'

In [104]: pd.Series(['1', '2', '3a', '3b', '03c']).str.contains(pattern)
Out[104]:
0 False
1 False
2 True
3 True
4 True
dtype: bool
```

或者是否元素完整匹配一个正则表达式

```
In [105]: pd.Series(['1', '2', '3a', '3b', '03c']).str.match(pattern)
Out[105]:
0 False
1 False
2 True
3 True
4 False
dtype: bool
```

`match` 和 `contains` 的区别是是否严格匹配。`match` 严格基于 `re.match`，而 `contains` 基于 `re.search`。

类似 `match`，`contains`，`startswith` 和 `endswith` 可以传入一个额外的 `na` 参数，因此，因此缺失值在匹配时可以被认为是 `True` 或者 `False`：

```
In [106]: s4 = pd.Series(['A', 'B', 'C', 'Aaba', 'Baca', np.nan, 'CABA', 'dog', 'cat'])

In [107]: s4.str.contains('A', na=False)
Out[107]:
0 True
1 False
2 False
3 True
4 False
5 False
6 True
7 False
8 False
dtype: bool
```

## 建立一个指示变量

你从字符串列可以抽出一个哑变量。例如，是否他们由 `|` 分割：

```
In [108]: s = pd.Series(['a', 'a|b', np.nan, 'a|c'])
```

```
In [109]: s.str.get_dummies(sep='|')
```

```
Out[109]:
```

	a	b	c
0	1	0	0
1	1	1	0
2	0	0	0
3	1	0	1

索引也支持 `get_dummies`，它返回一个多重索引：

v0.18.1. 新加入

```
In [110]: idx = pd.Index(['a', 'a|b', np.nan, 'a|c'])
```

```
In [111]: idx.str.get_dummies(sep='|')
```

```
Out[111]:
```

```
MultiIndex([(1, 0, 0),
 (1, 1, 0),
 (0, 0, 0),
 (1, 0, 1)],
 names=['a', 'b', 'c'])
```

参见 `get_dummies()`。



方法	描述
<code>cat()</code>	拼接字符串
<code>split()</code>	基于分隔符切分字符串
<code>rsplit()</code>	基于分隔符，逆向切分字符串
<code>get()</code>	索引每一个元素（返回第i个元素）
<code>join()</code>	使用传入的分隔符依次拼接每一个元素
<code>get_dummies()</code>	用分隔符切分字符串，并返回一个含有哑变量的数据表
<code>contains()</code>	返回一个布尔矩阵表明是每个元素包含字符串或正则表达式
<code>replace()</code>	将匹配到的子串或正则表达式替换为另外的字符串，或者一个可调对象的返回值
<code>repeat()</code>	值复制（ <code>s.str.repeat(3)</code> 等价于 <code>x * 3</code> ）
<code>pad()</code>	将白空格插入到字符串的左、右或者两端
<code>center()</code>	等价于 <code>str.center</code>
<code>ljust()</code>	等价于 <code>str.ljust</code>
<code>rjust()</code>	等价于 <code>str.rjust</code>
<code>zfill()</code>	等价于 <code>str.zfill</code>
<code>wrap()</code>	将长字符串转换为不长于给定长度的行
<code>slice()</code>	将序列中的每一个字符串切片
<code>slice_replace()</code>	用传入的值替换每一个字符串中的切片
<code>count()</code>	对出现符合的规则进行计数
<code>startswith()</code>	等价于 <code>str.startswith(pat)</code>
<code>endswith()</code>	等价于 <code>str.endswith(pat)</code>
<code>findall()</code>	返回每一个字符串中出现的所有满足样式或正则的匹配
<code>match()</code>	素调用 <code>re.match</code> ，并以列表形式返回匹配到的组
<code>extract()</code>	Call 对每一个元素调用 <code>re.search</code> ，并以数据表的形式返回。行对应原有的一个元素，列对应所有捕获的组
<code>extractall()</code>	一个元素调用 <code>re.findall</code> ，并以数据表的形式返回。行对应原有的一个元素，列对应所有捕获的组
<code>len()</code>	计算字符串长度
<code>strip()</code>	等价于 <code>str.strip</code>
<code>rstrip()</code>	等价于 <code>str.rstrip</code>
<code>lstrip()</code>	等价于 <code>str.lstrip</code>
<code>partition()</code>	等价于 <code>str.partition</code>
<code>rpartition()</code>	等价于 <code>str.rpartition</code>

方法	描述
<code>lower()</code>	等价于 <code>str.lower</code>
<code>casefold()</code>	等价于 <code>str.casefold</code>
<code>upper()</code>	等价于 <code>str.upper</code>
<code>find()</code>	等价于 <code>str.find</code>
<code>rfind()</code>	等价于 <code>str.rfind</code>
<code>index()</code>	等价于 <code>str.index</code>
<code>rindex()</code>	等价于 <code>str.rindex</code>
<code>capitalize()</code>	等价于 <code>str.capitalize</code>
<code>swapcase()</code>	等价于 <code>str.swapcase</code>
<code>normalize()</code>	返回Unicode 标注格式。等价于 <code>unicodedata.normalize</code>
<code>translate()</code>	等价于 <code>str.translate</code>
<code>isalnum()</code>	等价于 <code>str.isalnum</code>
<code>isalpha()</code>	等价于 <code>str.isalpha</code>
<code>isdigit()</code>	等价于 <code>str.isdigit</code>
<code>isspace()</code>	等价于 <code>str.isspace</code>
<code>islower()</code>	等价于 <code>str.islower</code>
<code>isupper()</code>	等价于 <code>str.isupper</code>
<code>istitle()</code>	等价于 <code>str.istitle</code>
<code>isnumeric()</code>	等价于 <code>str.isnumeric</code>
<code>isdecimal()</code>	等价于 <code>str.isdecimal</code>

## Working with missing data

In this section, we will discuss missing (also referred to as NA) values in pandas.

::: tip Note

The choice of using `NaN` internally to denote missing data was largely for simplicity and performance reasons. It differs from the `MaskedArray` approach of, for example, `scikits.timeseries`. We are hopeful that NumPy will soon be able to provide a native NA type solution (similar to R) performant enough to be used in pandas.

:::

See the [cookbook](#) for some advanced strategies.



## Values considered “missing”

As data comes in many shapes and forms, pandas aims to be flexible with regard to handling missing data. While `NaN` is the default missing value marker for reasons of computational speed and convenience, we need to be able to easily detect this value with data of different types: floating point, integer, boolean, and general object. In many cases, however, the Python `None` will arise and we wish to also consider that “missing” or “not available” or “NA” .

... tip Note

If you want to consider `inf` and `-inf` to be “NA” in computations, you can set

```
pandas.options.mode.use_inf_as_na = True .
```

...

```
In [1]: df = pd.DataFrame(np.random.randn(5, 3), index=['a', 'c', 'e', 'f', 'h'],
...: columns=['one', 'two', 'three'])
...:
```

```
In [2]: df['four'] = 'bar'
```

```
In [3]: df['five'] = df['one'] > 0
```

```
In [4]: df
```

```
Out[4]:
```

	one	two	three	four	five
a	0.469112	-0.282863	-1.509059	bar	True
c	-1.135632	1.212112	-0.173215	bar	False
e	0.119209	-1.044236	-0.861849	bar	True
f	-2.104569	-0.494929	1.071804	bar	False
h	0.721555	-0.706771	-1.039575	bar	True

```
In [5]: df2 = df.reindex(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'])
```

```
In [6]: df2
```

```
Out[6]:
```

	one	two	three	four	five
a	0.469112	-0.282863	-1.509059	bar	True
b	NaN	NaN	NaN	NaN	NaN
c	-1.135632	1.212112	-0.173215	bar	False
d	NaN	NaN	NaN	NaN	NaN
e	0.119209	-1.044236	-0.861849	bar	True
f	-2.104569	-0.494929	1.071804	bar	False
g	NaN	NaN	NaN	NaN	NaN
h	0.721555	-0.706771	-1.039575	bar	True

To make detecting missing values easier (and across different array dtypes), pandas provides the `isna()` and `notna()` functions, which are also methods on Series and DataFrame objects:

```
In [7]: df2['one']
Out[7]:
a 0.469112
b NaN
c -1.135632
d NaN
e 0.119209
f -2.104569
g NaN
h 0.721555
Name: one, dtype: float64
```

```
In [8]: pd.isna(df2['one'])
Out[8]:
a False
b True
c False
d True
e False
f False
g True
h False
Name: one, dtype: bool
```

```
In [9]: df2['four'].notna()
Out[9]:
a True
b False
c True
d False
e True
f True
g False
h True
Name: four, dtype: bool
```

```
In [10]: df2.isna()
Out[10]:
```

	one	two	three	four	five
a	False	False	False	False	False
b	True	True	True	True	True
c	False	False	False	False	False
d	True	True	True	True	True
e	False	False	False	False	False
f	False	False	False	False	False
g	True	True	True	True	True
h	False	False	False	False	False

::: danger Warning

One has to be mindful that in Python (and NumPy), the `nan`'s don't compare equal, but `None`'s **do**. Note that pandas/NumPy uses the fact that `np.nan != np.nan`, and treats `None` like `np.nan`.

```
In [11]: None == None
Out[11]: True

In [12]: np.nan == np.nan
Out[12]: False
```

# noqa: E711

So as compared to above, a scalar equality comparison versus a `None/np.nan` doesn't provide useful information.

```
In [13]: df2['one'] == np.nan
Out[13]:
a False
b False
c False
d False
e False
f False
g False
h False
Name: one, dtype: bool
```

...

## Integer dtypes and missing data

Because `NaN` is a float, a column of integers with even one missing values is cast to floating-point dtype (see [Support for integer NA](#) for more). Pandas provides a nullable integer array, which can be used by explicitly requesting the dtype:

```
In [14]: pd.Series([1, 2, np.nan, 4], dtype=pd.Int64Dtype())
Out[14]:
0 1
1 2
2 NaN
3 4
dtype: Int64
```

Alternatively, the string alias `dtype='Int64'` (note the capital "I") can be used.

See [Nullable integer data type](#) for more.

## Datetimes

For `datetime64[ns]` types, `NaT` represents missing values. This is a pseudo-native sentinel value that can be represented by NumPy in a singular dtype (`datetime64[ns]`). pandas objects provide compatibility between `NaT` and `NaN`.

```

In [15]: df2 = df.copy()

In [16]: df2['timestamp'] = pd.Timestamp('20120101')

In [17]: df2
Out[17]:
 one two three four five timestamp
a 0.469112 -0.282863 -1.509059 bar True 2012-01-01
c -1.135632 1.212112 -0.173215 bar False 2012-01-01
e 0.119209 -1.044236 -0.861849 bar True 2012-01-01
f -2.104569 -0.494929 1.071804 bar False 2012-01-01
h 0.721555 -0.706771 -1.039575 bar True 2012-01-01

In [18]: df2.loc[['a', 'c', 'h'], ['one', 'timestamp']] = np.nan

In [19]: df2
Out[19]:
 one two three four five timestamp
a NaN -0.282863 -1.509059 bar True NaT
c NaN 1.212112 -0.173215 bar False NaT
e 0.119209 -1.044236 -0.861849 bar True 2012-01-01
f -2.104569 -0.494929 1.071804 bar False 2012-01-01
h NaN -0.706771 -1.039575 bar True NaT

In [20]: df2.dtypes.value_counts()
Out[20]:
float64 3
bool 1
datetime64[ns] 1
object 1
dtype: int64

```

## Inserting missing data

You can insert missing values by simply assigning to containers. The actual missing value used will be chosen based on the dtype.

For example, numeric containers will always use `NaN` regardless of the missing value type chosen:

```

In [21]: s = pd.Series([1, 2, 3])

In [22]: s.loc[0] = None

In [23]: s
Out[23]:
0 NaN
1 2.0
2 3.0
dtype: float64

```

Likewise, datetime containers will always use `NaT`.

For object containers, pandas will use the value given:

```
In [24]: s = pd.Series(["a", "b", "c"])
```

```
In [25]: s.loc[0] = None
```

```
In [26]: s.loc[1] = np.nan
```

```
In [27]: s
```

```
Out[27]:
```

```
0 None
```

```
1 NaN
```

```
2 c
```

```
dtype: object
```

## Calculations with missing data

Missing values propagate naturally through arithmetic operations between pandas objects.

```
In [28]: a
```

```
Out[28]:
```

	one	two
a	NaN	-0.282863
c	NaN	1.212112
e	0.119209	-1.044236
f	-2.104569	-0.494929
h	-2.104569	-0.706771

```
In [29]: b
```

```
Out[29]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	0.119209	-1.044236	-0.861849
f	-2.104569	-0.494929	1.071804
h	NaN	-0.706771	-1.039575

```
In [30]: a + b
```

```
Out[30]:
```

	one	three	two
a	NaN	NaN	-0.565727
c	NaN	NaN	2.424224
e	0.238417	NaN	-2.088472
f	-4.209138	NaN	-0.989859
h	NaN	NaN	-1.413542

The descriptive statistics and computational methods discussed in the [data structure overview](#) (and listed [here](#) and [here](#)) are all written to account for missing data. For example:

- When summing data, NA (missing) values will be treated as zero.
- If the data are all NA, the result will be 0.

- Cumulative methods like `cumsum()` and `cumprod()` ignore NA values by default, but preserve them in the resulting arrays. To override this behaviour and include NA values, use `skipna=False`.

```
In [31]: df
Out[31]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	0.119209	-1.044236	-0.861849
f	-2.104569	-0.494929	1.071804
h	NaN	-0.706771	-1.039575

```
In [32]: df['one'].sum()
Out[32]: -1.9853605075978744
```

```
In [33]: df.mean(1)
Out[33]:
```

a	-0.895961
c	0.519449
e	-0.595625
f	-0.509232
h	-0.873173

dtype: float64

```
In [34]: df.cumsum()
Out[34]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	0.929249	-1.682273
e	0.119209	-0.114987	-2.544122
f	-1.985361	-0.609917	-1.472318
h	NaN	-1.316688	-2.511893

```
In [35]: df.cumsum(skipna=False)
Out[35]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	0.929249	-1.682273
e	NaN	-0.114987	-2.544122
f	NaN	-0.609917	-1.472318
h	NaN	-1.316688	-2.511893

## Sum/prod of empties/nans

::: danger Warning

This behavior is now standard as of v0.22.0 and is consistent with the default in `numpy`; previously sum/prod of all-NA or empty Series/DataFrames would return NaN. See [v0.22.0 whatsnew](#) for more.

:::

The sum of an empty or all-NA Series or column of a DataFrame is 0.

```
In [36]: pd.Series([np.nan]).sum()
Out[36]: 0.0

In [37]: pd.Series([]).sum()
Out[37]: 0.0
```

The product of an empty or all-NA Series or column of a DataFrame is 1.

```
In [38]: pd.Series([np.nan]).prod()
Out[38]: 1.0

In [39]: pd.Series([]).prod()
Out[39]: 1.0
```

## NA values in GroupBy

NA groups in GroupBy are automatically excluded. This behavior is consistent with R, for example:

```
In [40]: df
Out[40]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	0.119209	-1.044236	-0.861849
f	-2.104569	-0.494929	1.071804
h	NaN	-0.706771	-1.039575

```
In [41]: df.groupby('one').mean()
Out[41]:
```

	two	three
one		
-2.104569	-0.494929	1.071804
0.119209	-1.044236	-0.861849

See the groupby section [here](#) for more information.

## Cleaning / filling missing data

pandas objects are equipped with various data manipulation methods for dealing with missing data.

## Filling missing values: fillna

`fillna()` can “fill in” NA values with non-NA data in a couple of ways, which we illustrate:

### Replace NA with a scalar value

```
In [42]: df2
Out[42]:
```

	one	two	three	four	five	timestamp
a	NaN	-0.282863	-1.509059	bar	True	NaT
c	NaN	1.212112	-0.173215	bar	False	NaT
e	0.119209	-1.044236	-0.861849	bar	True	2012-01-01
f	-2.104569	-0.494929	1.071804	bar	False	2012-01-01
h	NaN	-0.706771	-1.039575	bar	True	NaT

```
In [43]: df2.fillna(0)
Out[43]:
```

	one	two	three	four	five	timestamp
a	0.000000	-0.282863	-1.509059	bar	True	0
c	0.000000	1.212112	-0.173215	bar	False	0
e	0.119209	-1.044236	-0.861849	bar	True	2012-01-01 00:00:00
f	-2.104569	-0.494929	1.071804	bar	False	2012-01-01 00:00:00
h	0.000000	-0.706771	-1.039575	bar	True	0

```
In [44]: df2['one'].fillna('missing')
Out[44]:
```

a	missing
c	missing
e	0.119209
f	-2.10457
h	missing

Name: one, dtype: object

## Fill gaps forward or backward

Using the same filling arguments as [reindexing](#), we can propagate non-NA values forward or backward:

```
In [45]: df
Out[45]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	0.119209	-1.044236	-0.861849
f	-2.104569	-0.494929	1.071804
h	NaN	-0.706771	-1.039575

```
In [46]: df.fillna(method='pad')
Out[46]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	0.119209	-1.044236	-0.861849
f	-2.104569	-0.494929	1.071804
h	-2.104569	-0.706771	-1.039575

## Limit the amount of filling



If we only want consecutive gaps filled up to a certain number of data points, we can use the limit keyword:

```
In [47]: df
Out[47]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	NaN	NaN	NaN
f	NaN	NaN	NaN
h	NaN	-0.706771	-1.039575

```
In [48]: df.fillna(method='pad', limit=1)
Out[48]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	NaN	1.212112	-0.173215
f	NaN	NaN	NaN
h	NaN	-0.706771	-1.039575

To remind you, these are the available filling methods:

Method	Action
pad / ffill	Fill values forward
bfill / backfill	Fill values backward

With time series data, using pad/ffill is extremely common so that the “last known value” is available at every time point.

`ffill()` is equivalent to `fillna(method='ffill')` and `bfill()` is equivalent to `fillna(method='bfill')`

## Filling with a PandasObject

You can also fillna using a dict or Series that is alignable. The labels of the dict or index of the Series must match the columns of the frame you wish to fill. The use case of this is to fill a DataFrame with the mean of that column.

```
In [49]: dff = pd.DataFrame(np.random.randn(10, 3), columns=list('ABC'))
```

```
In [50]: dff.iloc[3:5, 0] = np.nan
```

```
In [51]: dff.iloc[4:6, 1] = np.nan
```

```
In [52]: dff.iloc[5:8, 2] = np.nan
```

```
In [53]: dff
```

```
Out[53]:
```

	A	B	C
0	0.271860	-0.424972	0.567020
1	0.276232	-1.087401	-0.673690
2	0.113648	-1.478427	0.524988
3	NaN	0.577046	-1.715002
4	NaN	NaN	-1.157892
5	-1.344312	NaN	NaN
6	-0.109050	1.643563	NaN
7	0.357021	-0.674600	NaN
8	-0.968914	-1.294524	0.413738
9	0.276662	-0.472035	-0.013960

```
In [54]: dff.fillna(dff.mean())
```

```
Out[54]:
```

	A	B	C
0	0.271860	-0.424972	0.567020
1	0.276232	-1.087401	-0.673690
2	0.113648	-1.478427	0.524988
3	-0.140857	0.577046	-1.715002
4	-0.140857	-0.401419	-1.157892
5	-1.344312	-0.401419	-0.293543
6	-0.109050	1.643563	-0.293543
7	0.357021	-0.674600	-0.293543
8	-0.968914	-1.294524	0.413738
9	0.276662	-0.472035	-0.013960

```
In [55]: dff.fillna(dff.mean()['B':'C'])
```

```
Out[55]:
```

	A	B	C
0	0.271860	-0.424972	0.567020
1	0.276232	-1.087401	-0.673690
2	0.113648	-1.478427	0.524988
3	NaN	0.577046	-1.715002
4	NaN	-0.401419	-1.157892
5	-1.344312	-0.401419	-0.293543
6	-0.109050	1.643563	-0.293543
7	0.357021	-0.674600	-0.293543
8	-0.968914	-1.294524	0.413738
9	0.276662	-0.472035	-0.013960

Same result as above, but is aligning the 'fill' value which is a Series in this case.

```
In [56]: dff.where(pd.notna(dff), dff.mean(), axis='columns')
```

```
Out[56]:
```

	A	B	C
0	0.271860	-0.424972	0.567020
1	0.276232	-1.087401	-0.673690
2	0.113648	-1.478427	0.524988
3	-0.140857	0.577046	-1.715002
4	-0.140857	-0.401419	-1.157892
5	-1.344312	-0.401419	-0.293543
6	-0.109050	1.643563	-0.293543
7	0.357021	-0.674600	-0.293543
8	-0.968914	-1.294524	0.413738
9	0.276662	-0.472035	-0.013960

## Dropping axis labels with missing data: dropna

You may wish to simply exclude labels from a data set which refer to missing data. To do this, use `dropna()`:

```
In [57]: df
```

```
Out[57]:
```

	one	two	three
a	NaN	-0.282863	-1.509059
c	NaN	1.212112	-0.173215
e	NaN	0.000000	0.000000
f	NaN	0.000000	0.000000
h	NaN	-0.706771	-1.039575

```
In [58]: df.dropna(axis=0)
```

```
Out[58]:
```

```
Empty DataFrame
Columns: [one, two, three]
Index: []
```

```
In [59]: df.dropna(axis=1)
```

```
Out[59]:
```

	two	three
a	-0.282863	-1.509059
c	1.212112	-0.173215
e	0.000000	0.000000
f	0.000000	0.000000
h	-0.706771	-1.039575

```
In [60]: df['one'].dropna()
```

```
Out[60]: Series([], Name: one, dtype: float64)
```

An equivalent `dropna()` is available for Series. `DataFrame.dropna` has considerably more options than `Series.dropna`, which can be examined [in the API](#).

# Interpolation

New in version 0.23.0: The `limit_area` keyword argument was added.

Both Series and DataFrame objects have `interpolate()` that, by default, performs linear interpolation at missing data points.

```
In [61]: ts
Out[61]:
2000-01-31 0.469112
2000-02-29 NaN
2000-03-31 NaN
2000-04-28 NaN
2000-05-31 NaN
...
2007-12-31 -6.950267
2008-01-31 -7.904475
2008-02-29 -6.441779
2008-03-31 -8.184940
2008-04-30 -9.011531
Freq: BM, Length: 100, dtype: float64

In [62]: ts.count()
Out[62]: 66

In [63]: ts.plot()
Out[63]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d8ac0eb8>
```

series before interpolate

```
In [64]: ts.interpolate()
Out[64]:
2000-01-31 0.469112
2000-02-29 0.434469
2000-03-31 0.399826
2000-04-28 0.365184
2000-05-31 0.330541
...
2007-12-31 -6.950267
2008-01-31 -7.904475
2008-02-29 -6.441779
2008-03-31 -8.184940
2008-04-30 -9.011531
Freq: BM, Length: 100, dtype: float64

In [65]: ts.interpolate().count()
Out[65]: 100

In [66]: ts.interpolate().plot()
Out[66]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d8adfeb8>
```

series\_interpolate

Index aware interpolation is available via the `method` keyword:

```
In [67]: ts2
Out[67]:
2000-01-31 0.469112
2000-02-29 NaN
2002-07-31 -5.785037
2005-01-31 NaN
2008-04-30 -9.011531
dtype: float64

In [68]: ts2.interpolate()
Out[68]:
2000-01-31 0.469112
2000-02-29 -2.657962
2002-07-31 -5.785037
2005-01-31 -7.398284
2008-04-30 -9.011531
dtype: float64

In [69]: ts2.interpolate(method='time')
Out[69]:
2000-01-31 0.469112
2000-02-29 0.270241
2002-07-31 -5.785037
2005-01-31 -7.190866
2008-04-30 -9.011531
dtype: float64
```

For a floating-point index, use `method='values'` :

```
In [70]: ser
Out[70]:
0.0 0.0
1.0 NaN
10.0 10.0
dtype: float64

In [71]: ser.interpolate()
Out[71]:
0.0 0.0
1.0 5.0
10.0 10.0
dtype: float64

In [72]: ser.interpolate(method='values')
Out[72]:
0.0 0.0
1.0 1.0
10.0 10.0
dtype: float64
```

You can also interpolate with a DataFrame:

```
In [73]: df = pd.DataFrame({'A': [1, 2.1, np.nan, 4.7, 5.6, 6.8],
....: 'B': [.25, np.nan, np.nan, 4, 12.2, 14.4]})
....:
```

```
In [74]: df
Out[74]:
```

	A	B
0	1.0	0.25
1	2.1	NaN
2	NaN	NaN
3	4.7	4.00
4	5.6	12.20
5	6.8	14.40

```
In [75]: df.interpolate()
Out[75]:
```

	A	B
0	1.0	0.25
1	2.1	1.50
2	3.4	2.75
3	4.7	4.00
4	5.6	12.20
5	6.8	14.40

The `method` argument gives access to fancier interpolation methods. If you have [scipy](#) installed, you can pass the name of a 1-d interpolation routine to `method`. You'll want to consult the full [scipy interpolation documentation](#) and reference [guide](#) for details. The appropriate interpolation method will depend on the type of data you are working with.

- If you are dealing with a time series that is growing at an increasing rate, `method='quadratic'` may be appropriate.
- If you have values approximating a cumulative distribution function, then `method='pchip'` should work well.
- To fill missing values with goal of smooth plotting, consider `method='akima'`.

::: danger Warning

These methods require `scipy`.

:::

```
In [76]: df.interpolate(method='barycentric')
```

```
Out[76]:
```

	A	B
0	1.00	0.250
1	2.10	-7.660
2	3.53	-4.515
3	4.70	4.000
4	5.60	12.200
5	6.80	14.400

```
In [77]: df.interpolate(method='pchip')
```

```
Out[77]:
```

	A	B
0	1.00000	0.250000
1	2.10000	0.672808
2	3.43454	1.928950
3	4.70000	4.000000
4	5.60000	12.200000
5	6.80000	14.400000

```
In [78]: df.interpolate(method='akima')
```

```
Out[78]:
```

	A	B
0	1.000000	0.250000
1	2.100000	-0.873316
2	3.406667	0.320034
3	4.700000	4.000000
4	5.600000	12.200000
5	6.800000	14.400000

When interpolating via a polynomial or spline approximation, you must also specify the degree or order of the approximation:

```
In [79]: df.interpolate(method='spline', order=2)
```

```
Out[79]:
```

	A	B
0	1.000000	0.250000
1	2.100000	-0.428598
2	3.404545	1.206900
3	4.700000	4.000000
4	5.600000	12.200000
5	6.800000	14.400000

```
In [80]: df.interpolate(method='polynomial', order=2)
```

```
Out[80]:
```

	A	B
0	1.000000	0.250000
1	2.100000	-2.703846
2	3.451351	-1.453846
3	4.700000	4.000000
4	5.600000	12.200000
5	6.800000	14.400000

Compare several methods:

```
In [81]: np.random.seed(2)

In [82]: ser = pd.Series(np.arange(1, 10.1, .25) ** 2 + np.random.randn(37))

In [83]: missing = np.array([4, 13, 14, 15, 16, 17, 18, 20, 29])

In [84]: ser[missing] = np.nan

In [85]: methods = ['linear', 'quadratic', 'cubic']

In [86]: df = pd.DataFrame({m: ser.interpolate(method=m) for m in methods})

In [87]: df.plot()
Out[87]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d8a196a0>
```

compare interpolations

Another use case is interpolation at new values. Suppose you have 100 observations from some distribution. And let's suppose that you're particularly interested in what's happening around the middle. You can mix pandas' `reindex` and `interpolate` methods to interpolate at the new values.

```
In [88]: ser = pd.Series(np.sort(np.random.uniform(size=100)))

interpolate at new_index
In [89]: new_index = ser.index | pd.Index([49.25, 49.5, 49.75, 50.25, 50.5, 50.75])

In [90]: interp_s = ser.reindex(new_index).interpolate(method='pchip')

In [91]: interp_s[49:51]
Out[91]:
49.00 0.471410
49.25 0.476841
49.50 0.481780
49.75 0.485998
50.00 0.489266
50.25 0.491814
50.50 0.493995
50.75 0.495763
51.00 0.497074
dtype: float64
```

## Interpolation limits

Like other pandas fill methods, `interpolate()` accepts a `limit` keyword argument. Use this argument to limit the number of consecutive `NaN` values filled since the last valid observation:



```
In [92]: ser = pd.Series([np.nan, np.nan, 5, np.nan, np.nan,
.....: np.nan, 13, np.nan, np.nan])
.....:
```

```
In [93]: ser
```

```
Out[93]:
```

```
0 NaN
1 NaN
2 5.0
3 NaN
4 NaN
5 NaN
6 13.0
7 NaN
8 NaN
```

```
dtype: float64
```

```
fill all consecutive values in a forward direction
```

```
In [94]: ser.interpolate()
```

```
Out[94]:
```

```
0 NaN
1 NaN
2 5.0
3 7.0
4 9.0
5 11.0
6 13.0
7 13.0
8 13.0
```

```
dtype: float64
```

```
fill one consecutive value in a forward direction
```

```
In [95]: ser.interpolate(limit=1)
```

```
Out[95]:
```

```
0 NaN
1 NaN
2 5.0
3 7.0
4 NaN
5 NaN
6 13.0
7 13.0
8 NaN
```

```
dtype: float64
```

By default, NaN values are filled in a forward direction. Use `limit_direction` parameter to fill backward or from both directions.

```

fill one consecutive value backwards
In [96]: ser.interpolate(limit=1, limit_direction='backward')
Out[96]:
0 NaN
1 5.0
2 5.0
3 NaN
4 NaN
5 11.0
6 13.0
7 NaN
8 NaN
dtype: float64

fill one consecutive value in both directions
In [97]: ser.interpolate(limit=1, limit_direction='both')
Out[97]:
0 NaN
1 5.0
2 5.0
3 7.0
4 NaN
5 11.0
6 13.0
7 13.0
8 NaN
dtype: float64

fill all consecutive values in both directions
In [98]: ser.interpolate(limit_direction='both')
Out[98]:
0 5.0
1 5.0
2 5.0
3 7.0
4 9.0
5 11.0
6 13.0
7 13.0
8 13.0
dtype: float64

```

By default, `NaN` values are filled whether they are inside (surrounded by) existing valid values, or outside existing valid values. Introduced in v0.23 the `limit_area` parameter restricts filling to either inside or outside values.

```

fill one consecutive inside value in both directions
In [99]: ser.interpolate(limit_direction='both', limit_area='inside', limit=1)
Out[99]:
0 NaN
1 NaN
2 5.0
3 7.0
4 NaN
5 11.0
6 13.0
7 NaN
8 NaN
dtype: float64

fill all consecutive outside values backward
In [100]: ser.interpolate(limit_direction='backward', limit_area='outside')
Out[100]:
0 5.0
1 5.0
2 5.0
3 NaN
4 NaN
5 NaN
6 13.0
7 NaN
8 NaN
dtype: float64

fill all consecutive outside values in both directions
In [101]: ser.interpolate(limit_direction='both', limit_area='outside')
Out[101]:
0 5.0
1 5.0
2 5.0
3 NaN
4 NaN
5 NaN
6 13.0
7 13.0
8 13.0
dtype: float64

```

## Replacing generic values

---

Often times we want to replace arbitrary values with other values.

`replace()` in Series and `replace()` in DataFrame provides an efficient yet flexible way to perform such replacements.

For a Series, you can replace a single value or a list of values by another value:

```
In [102]: ser = pd.Series([0., 1., 2., 3., 4.])
```

```
In [103]: ser.replace(0, 5)
```

```
Out[103]:
```

```
0 5.0
```

```
1 1.0
```

```
2 2.0
```

```
3 3.0
```

```
4 4.0
```

```
dtype: float64
```

You can replace a list of values by a list of other values:

```
In [104]: ser.replace([0, 1, 2, 3, 4], [4, 3, 2, 1, 0])
```

```
Out[104]:
```

```
0 4.0
```

```
1 3.0
```

```
2 2.0
```

```
3 1.0
```

```
4 0.0
```

```
dtype: float64
```

You can also specify a mapping dict:

```
In [105]: ser.replace({0: 10, 1: 100})
```

```
Out[105]:
```

```
0 10.0
```

```
1 100.0
```

```
2 2.0
```

```
3 3.0
```

```
4 4.0
```

```
dtype: float64
```

For a DataFrame, you can specify individual values by column:

```
In [106]: df = pd.DataFrame({'a': [0, 1, 2, 3, 4], 'b': [5, 6, 7, 8, 9]})
```

```
In [107]: df.replace({'a': 0, 'b': 5}, 100)
```

```
Out[107]:
```

```
 a b
```

```
0 100 100
```

```
1 1 6
```

```
2 2 7
```

```
3 3 8
```

```
4 4 9
```

Instead of replacing with specified values, you can treat all given values as missing and interpolate over them:

```
In [108]: ser.replace([1, 2, 3], method='pad')
Out[108]:
0 0.0
1 0.0
2 0.0
3 0.0
4 4.0
dtype: float64
```

## String/regular expression replacement

::: tip Note

Python strings prefixed with the `r` character such as `r'hello world'` are so-called “raw” strings. They have different semantics regarding backslashes than strings without this prefix. Backslashes in raw strings will be interpreted as an escaped backslash, e.g., `r'\'` == `'\\'`. You should [read about them](#) if this is unclear.

:::

Replace the `'.'` with `NaN` (str -> str):

```
In [109]: d = {'a': list(range(4)), 'b': list('ab..'), 'c': ['a', 'b', np.nan, 'd']}
In [110]: df = pd.DataFrame(d)
In [111]: df.replace('.', np.nan)
Out[111]:
 a b c
0 0 a a
1 1 b b
2 2 NaN NaN
3 3 NaN d
```

Now do it with a regular expression that removes surrounding whitespace (regex -> regex):

```
In [112]: df.replace(r'\s*\.\s*', np.nan, regex=True)
Out[112]:
 a b c
0 0 a a
1 1 b b
2 2 NaN NaN
3 3 NaN d
```

Replace a few different values (list -> list):

```
In [113]: df.replace(['a', '.'], ['b', np.nan])
Out[113]:
```

	a	b	c
0	0	b	b
1	1	b	b
2	2	NaN	NaN
3	3	NaN	d

list of regex -> list of regex:

```
In [114]: df.replace([r'\.', r'(a)'], ['dot', r'\1stuff'], regex=True)
Out[114]:
```

	a	b	c
0	0	astuff	astuff
1	1	b	b
2	2	dot	NaN
3	3	dot	d

Only search in column 'b' (dict -> dict):

```
In [115]: df.replace({'b': '.'}, {'b': np.nan})
Out[115]:
```

	a	b	c
0	0	a	a
1	1	b	b
2	2	NaN	NaN
3	3	NaN	d

Same as the previous example, but use a regular expression for searching instead (dict of regex -> dict):

```
In [116]: df.replace({'b': r'\s*\.\s*'}, {'b': np.nan}, regex=True)
Out[116]:
```

	a	b	c
0	0	a	a
1	1	b	b
2	2	NaN	NaN
3	3	NaN	d

You can pass nested dictionaries of regular expressions that use `regex=True`:

```
In [117]: df.replace({'b': {'b': r'.'}}, regex=True)
Out[117]:
```

	a	b	c
0	0	a	a
1	1	b	b
2	2	.	NaN
3	3	.	d

Alternatively, you can pass the nested dictionary like so:

```
In [118]: df.replace(regex={'b': {r'\s*\.\s*': np.nan}})
Out[118]:
```

	a	b	c
0	0	a	a
1	1	b	b
2	2	NaN	NaN
3	3	NaN	d

You can also use the group of a regular expression match when replacing (dict of regex -> dict of regex), this works for lists as well.

```
In [119]: df.replace({'b': r'\s*(\.)\s*'}, {'b': r'\1ty'}, regex=True)
Out[119]:
```

	a	b	c
0	0	a	a
1	1	b	b
2	2	.ty	NaN
3	3	.ty	d

You can pass a list of regular expressions, of which those that match will be replaced with a scalar (list of regex -> regex).

```
In [120]: df.replace([r'\s*\.\s*', r'a|b'], np.nan, regex=True)
Out[120]:
```

	a	b	c
0	0	NaN	NaN
1	1	NaN	NaN
2	2	NaN	NaN
3	3	NaN	d

All of the regular expression examples can also be passed with the `to_replace` argument as the `regex` argument. In this case the `value` argument must be passed explicitly by name or `regex` must be a nested dictionary. The previous example, in this case, would then be:

```
In [121]: df.replace(regex=[r'\s*\.\s*', r'a|b'], value=np.nan)
Out[121]:
```

	a	b	c
0	0	NaN	NaN
1	1	NaN	NaN
2	2	NaN	NaN
3	3	NaN	d

This can be convenient if you do not want to pass `regex=True` every time you want to use a regular expression.

::: tip Note

Anywhere in the above `replace` examples that you see a regular expression a compiled regular expression is valid as well.

...

## Numeric replacement

`replace()` is similar to `fillna()`.

```
In [122]: df = pd.DataFrame(np.random.randn(10, 2))

In [123]: df[np.random.rand(df.shape[0]) > 0.5] = 1.5

In [124]: df.replace(1.5, np.nan)
Out[124]:
```

	0	1
0	-0.844214	-1.021415
1	0.432396	-0.323580
2	0.423825	0.799180
3	1.262614	0.751965
4	NaN	NaN
5	NaN	NaN
6	-0.498174	-1.060799
7	0.591667	-0.183257
8	1.019855	-1.482465
9	NaN	NaN

Replacing more than one value is possible by passing a list.

```
In [125]: df00 = df.iloc[0, 0]

In [126]: df.replace([1.5, df00], [np.nan, 'a'])
Out[126]:
```

	0	1
0	a	-1.02141
1	0.432396	-0.32358
2	0.423825	0.79918
3	1.26261	0.751965
4	NaN	NaN
5	NaN	NaN
6	-0.498174	-1.0608
7	0.591667	-0.183257
8	1.01985	-1.48247
9	NaN	NaN

```
In [127]: df[1].dtype
Out[127]: dtype('float64')
```

You can also operate on the DataFrame in place:

```
In [128]: df.replace(1.5, np.nan, inplace=True)
```

::: danger Warning



When replacing multiple `bool` or `datetime64` objects, the first argument to `replace` (`to_replace`) must match the type of the value being replaced. For example,

```
>>> s = pd.Series([True, False, True])
>>> s.replace({'a string': 'new value', True: False}) # raises
TypeError: Cannot compare types 'ndarray(dtype=bool)' and 'str'
```

will raise a `TypeError` because one of the `dict` keys is not of the correct type for replacement.

However, when replacing a single object such as,

```
In [129]: s = pd.Series([True, False, True])

In [130]: s.replace('a string', 'another string')
Out[130]:
0 True
1 False
2 True
dtype: bool
```

the original `NDFrame` object will be returned untouched. We’re working on unifying this API, but for backwards compatibility reasons we cannot break the latter behavior. See [GH6354](#) for more details.

...

## Missing data casting rules and indexing

While pandas supports storing arrays of integer and boolean type, these types are not capable of storing missing data. Until we can switch to using a native NA type in NumPy, we’ve established some “casting rules”. When a reindexing operation introduces missing data, the Series will be cast according to the rules introduced in the table below.

data type	Cast to
integer	float
boolean	object
float	no cast
object	no cast

For example:

```
In [131]: s = pd.Series(np.random.randn(5), index=[0, 2, 4, 6, 7])
```

```
In [132]: s > 0
```

```
Out[132]:
```

```
0 True
```

```
2 True
```

```
4 True
```

```
6 True
```

```
7 True
```

```
dtype: bool
```

```
In [133]: (s > 0).dtype
```

```
Out[133]: dtype('bool')
```

```
In [134]: crit = (s > 0).reindex(list(range(8)))
```

```
In [135]: crit
```

```
Out[135]:
```

```
0 True
```

```
1 NaN
```

```
2 True
```

```
3 NaN
```

```
4 True
```

```
5 NaN
```

```
6 True
```

```
7 True
```

```
dtype: object
```

```
In [136]: crit.dtype
```

```
Out[136]: dtype('O')
```

Ordinarily NumPy will complain if you try to use an object array (even if it contains boolean values) instead of a boolean array to get or set values from an ndarray (e.g. selecting values based on some criteria). If a boolean vector contains NAs, an exception will be generated:

```

In [137]: reindexed = s.reindex(list(range(8))).fillna(0)

In [138]: reindexed[crit]

ValueError Traceback (most recent call last)
<ipython-input-138-0dac417a4890> in <module>
----> 1 reindexed[crit]

/pandas/pandas/core/series.py in __getitem__(self, key)
 1101 key = list(key)
 1102
-> 1103 if com.is_bool_indexer(key):
 1104 key = check_bool_indexer(self.index, key)
 1105

/pandas/pandas/core/common.py in is_bool_indexer(key)
 128 if not lib.is_bool_array(key):
 129 if isna(key).any():
--> 130 raise ValueError(na_msg)
 131 return False
 132 return True

ValueError: cannot index with vector containing NA / NaN values

```

However, these can be filled in using `fillna()` and it will work fine:

```

In [139]: reindexed[crit.fillna(False)]
Out[139]:
0 0.126504
2 0.696198
4 0.697416
6 0.601516
7 0.003659
dtype: float64

In [140]: reindexed[crit.fillna(True)]
Out[140]:
0 0.126504
1 0.000000
2 0.696198
3 0.000000
4 0.697416
5 0.000000
6 0.601516
7 0.003659
dtype: float64

```

Pandas provides a nullable integer dtype, but you must explicitly request it when creating the series or column. Notice that we use a capital “I” in the `dtype="Int64"`.

```
In [141]: s = pd.Series([0, 1, np.nan, 3, 4], dtype="Int64")
```

```
In [142]: s
```

```
Out[142]:
```

```
0 0
```

```
1 1
```

```
2 NaN
```

```
3 3
```

```
4 4
```

```
dtype: Int64
```

See [Nullable integer data type](#) for more.

---

## Categorical data

This is an introduction to pandas categorical data type, including a short comparison with R's `factor`.

Categoricals are a pandas data type corresponding to categorical variables in statistics. A categorical variable takes on a limited, and usually fixed, number of possible values (categories; levels in R). Examples are gender, social class, blood type, country affiliation, observation time or rating via Likert scales.

In contrast to statistical categorical variables, categorical data might have an order (e.g. 'strongly agree' vs 'agree' or 'first observation' vs. 'second observation'), but numerical operations (additions, divisions, ...) are not possible.

All values of categorical data are either in categories or `np.nan`. Order is defined by the order of categories, not lexical order of the values. Internally, the data structure consists of a categories array and an integer array of codes which point to the real value in the categories array.

The categorical data type is useful in the following cases:

- A string variable consisting of only a few different values. Converting such a string variable to a categorical variable will save some memory, see [here](#).
- The lexical order of a variable is not the same as the logical order ( "one" , "two" , "three" ). By converting to a categorical and specifying an order on the categories, sorting and min/max will use the logical order instead of the lexical order, see [here](#).
- As a signal to other Python libraries that this column should be treated as a categorical variable (e.g. to use suitable statistical methods or plot types).

See also the [API docs on categoricals](#).

# Object creation

---

## Series creation

Categorical `Series` or columns in a `DataFrame` can be created in several ways:

By specifying `dtype="category"` when constructing a `Series`:

```
In [1]: s = pd.Series(["a", "b", "c", "a"], dtype="category")

In [2]: s
Out[2]:
0 a
1 b
2 c
3 a
dtype: category
Categories (3, object): [a, b, c]
```

By converting an existing `Series` or column to a `category` dtype:

```
In [3]: df = pd.DataFrame({"A": ["a", "b", "c", "a"]})

In [4]: df["B"] = df["A"].astype('category')

In [5]: df
Out[5]:
 A B
0 a a
1 b b
2 c c
3 a a
```

By using special functions, such as `cut()`, which groups data into discrete bins. See the [example on tiling](#) in the docs.

```

In [6]: df = pd.DataFrame({'value': np.random.randint(0, 100, 20)})

In [7]: labels = ["{0} - {1}".format(i, i + 9) for i in range(0, 100, 10)]

In [8]: df['group'] = pd.cut(df.value, range(0, 105, 10), right=False, labels=labels)

In [9]: df.head(10)
Out[9]:
 value group
0 65 60 - 69
1 49 40 - 49
2 56 50 - 59
3 43 40 - 49
4 43 40 - 49
5 91 90 - 99
6 32 30 - 39
7 87 80 - 89
8 36 30 - 39
9 8 0 - 9

```

By passing a `pandas.Categorical` object to a `Series` or assigning it to a `DataFrame`.

```

In [10]: raw_cat = pd.Categorical(["a", "b", "c", "a"], categories=["b", "c", "d"],
....: ordered=False)
....:

In [11]: s = pd.Series(raw_cat)

In [12]: s
Out[12]:
0 NaN
1 b
2 c
3 NaN
dtype: category
Categories (3, object): [b, c, d]

In [13]: df = pd.DataFrame({"A": ["a", "b", "c", "a"]})

In [14]: df["B"] = raw_cat

In [15]: df
Out[15]:
 A B
0 a NaN
1 b b
2 c c
3 a NaN

```

Categorical data has a specific `category` `dtype`:

```
In [16]: df.dtypes
Out[16]:
A object
B category
dtype: object
```

## DataFrame creation

Similar to the previous section where a single column was converted to categorical, all columns in a `DataFrame` can be batch converted to categorical either during or after construction.

This can be done during construction by specifying `dtype="category"` in the `DataFrame` constructor:

```
In [17]: df = pd.DataFrame({'A': list('abca'), 'B': list('bccd')}, dtype="category")

In [18]: df.dtypes
Out[18]:
A category
B category
dtype: object
```

Note that the categories present in each column differ; the conversion is done column by column, so only labels present in a given column are categories:

```
In [19]: df['A']
Out[19]:
0 a
1 b
2 c
3 a
Name: A, dtype: category
Categories (3, object): [a, b, c]

In [20]: df['B']
Out[20]:
0 b
1 c
2 c
3 d
Name: B, dtype: category
Categories (3, object): [b, c, d]
```

New in version 0.23.0.

Analogously, all columns in an existing `DataFrame` can be batch converted using `DataFrame.astype()`:

```
In [21]: df = pd.DataFrame({'A': list('abca'), 'B': list('bccd')})

In [22]: df_cat = df.astype('category')

In [23]: df_cat.dtypes
Out[23]:
A category
B category
dtype: object
```

This conversion is likewise done column by column:

```
In [24]: df_cat['A']
Out[24]:
0 a
1 b
2 c
3 a
Name: A, dtype: category
Categories (3, object): [a, b, c]

In [25]: df_cat['B']
Out[25]:
0 b
1 c
2 c
3 d
Name: B, dtype: category
Categories (3, object): [b, c, d]
```

## Controlling behavior

In the examples above where we passed `dtype='category'`, we used the default behavior:

1. Categories are inferred from the data.
2. Categories are unordered.

To control those behaviors, instead of passing `'category'`, use an instance of `CategoricalDtype`.



```

In [26]: from pandas.api.types import CategoricalDtype

In [27]: s = pd.Series(["a", "b", "c", "a"])

In [28]: cat_type = CategoricalDtype(categories=["b", "c", "d"],
.....: ordered=True)
.....:

In [29]: s_cat = s.astype(cat_type)

In [30]: s_cat
Out[30]:
0 NaN
1 b
2 c
3 NaN
dtype: category
Categories (3, object): [b < c < d]

```

Similarly, a `CategoricalDtype` can be used with a `DataFrame` to ensure that categories are consistent among all columns.

```

In [31]: from pandas.api.types import CategoricalDtype

In [32]: df = pd.DataFrame({'A': list('abca'), 'B': list('bccd')})

In [33]: cat_type = CategoricalDtype(categories=list('abcd'),
.....: ordered=True)
.....:

In [34]: df_cat = df.astype(cat_type)

In [35]: df_cat['A']
Out[35]:
0 a
1 b
2 c
3 a
Name: A, dtype: category
Categories (4, object): [a < b < c < d]

In [36]: df_cat['B']
Out[36]:
0 b
1 c
2 c
3 d
Name: B, dtype: category
Categories (4, object): [a < b < c < d]

```

::: tip Note

To perform table-wise conversion, where all labels in the entire `DataFrame` are used as categories for each column, the `categories` parameter can be determined programmatically by `categories = pd.unique(df.to_numpy().ravel())`.

...

If you already have `codes` and `categories`, you can use the `from_codes()` constructor to save the factorize step during normal constructor mode:

```
In [37]: splitter = np.random.choice([0, 1], 5, p=[0.5, 0.5])

In [38]: s = pd.Series(pd.Categorical.from_codes(splitter,
....: categories=["train", "test"]))
....:
```

## Regaining original data

To get back to the original `Series` or NumPy array, use `Series.astype(original_dtype)` or `np.asarray(categorical)`:

```
In [39]: s = pd.Series(["a", "b", "c", "a"])

In [40]: s
Out[40]:
0 a
1 b
2 c
3 a
dtype: object

In [41]: s2 = s.astype('category')

In [42]: s2
Out[42]:
0 a
1 b
2 c
3 a
dtype: category
Categories (3, object): [a, b, c]

In [43]: s2.astype(str)
Out[43]:
0 a
1 b
2 c
3 a
dtype: object

In [44]: np.asarray(s2)
Out[44]: array(['a', 'b', 'c', 'a'], dtype=object)
```

::: tip Note

In contrast to R's factor function, categorical data is not converting input values to strings; categories will end up the same data type as the original values.

:::

::: tip Note

In contrast to R's factor function, there is currently no way to assign/change labels at creation time. Use categories to change the categories after creation time.

:::

## CategoricalDtype

---

Changed in version 0.21.0.

A categorical's type is fully described by

1. `categories`: a sequence of unique values and no missing values
2. `ordered`: a boolean

This information can be stored in a `CategoricalDtype`. The `categories` argument is optional, which implies that the actual categories should be inferred from whatever is present in the data when the `pandas.Categorical` is created. The categories are assumed to be unordered by default.

```
In [45]: from pandas.api.types import CategoricalDtype

In [46]: CategoricalDtype(['a', 'b', 'c'])
Out[46]: CategoricalDtype(categories=['a', 'b', 'c'], ordered=None)

In [47]: CategoricalDtype(['a', 'b', 'c'], ordered=True)
Out[47]: CategoricalDtype(categories=['a', 'b', 'c'], ordered=True)

In [48]: CategoricalDtype()
Out[48]: CategoricalDtype(categories=None, ordered=None)
```

A `CategoricalDtype` can be used in any place pandas expects a dtype. For example `pandas.read_csv()`, `pandas.DataFrame.astype()`, or in the `Series` constructor.

::: tip Note

As a convenience, you can use the string `'category'` in place of a `CategoricalDtype` when you want the default behavior of the categories being unordered, and equal to the set values present in the array. In other words, `dtype='category'` is equivalent to `dtype=CategoricalDtype()`.

:::

## Equality semantics

Two instances of `CategoricalDtype` compare equal whenever they have the same categories and order. When comparing two unordered categoricals, the order of the `categories` is not considered.

```
In [49]: c1 = CategoricalDtype(['a', 'b', 'c'], ordered=False)

Equal, since order is not considered when ordered=False
In [50]: c1 == CategoricalDtype(['b', 'c', 'a'], ordered=False)
Out[50]: True

Unequal, since the second CategoricalDtype is ordered
In [51]: c1 == CategoricalDtype(['a', 'b', 'c'], ordered=True)
Out[51]: False
```

All instances of `CategoricalDtype` compare equal to the string `'category'`.

```
In [52]: c1 == 'category'
Out[52]: True
```

::: danger Warning

Since `dtype='category'` is essentially `CategoricalDtype(None, False)`, and since all instances `CategoricalDtype` compare equal to `'category'`, all instances of `CategoricalDtype` compare equal to a `CategoricalDtype(None, False)`, regardless of `categories` or `ordered`.

:::

## Description

---

Using `describe()` on categorical data will produce similar output to a `Series` or `DataFrame` of type `string`.

```
In [53]: cat = pd.Categorical(["a", "c", "c", np.nan], categories=["b", "a", "c"])
```

```
In [54]: df = pd.DataFrame({"cat": cat, "s": ["a", "c", "c", np.nan]})
```

```
In [55]: df.describe()
```

```
Out[55]:
```

	cat	s
count	3	3
unique	2	2
top	c	c
freq	2	2

```
In [56]: df["cat"].describe()
```

```
Out[56]:
```

count	3
unique	2
top	c
freq	2

Name: cat, dtype: object

## Working with categories

Categorical data has a `categories` and an `ordered` property, which list their possible values and whether the ordering matters or not. These properties are exposed as `s.cat.categories` and `s.cat.ordered`. If you don't manually specify categories and ordering, they are inferred from the passed arguments.

```
In [57]: s = pd.Series(["a", "b", "c", "a"], dtype="category")
```

```
In [58]: s.cat.categories
```

```
Out[58]: Index(['a', 'b', 'c'], dtype='object')
```

```
In [59]: s.cat.ordered
```

```
Out[59]: False
```

It's also possible to pass in the categories in a specific order:

```
In [60]: s = pd.Series(pd.Categorical(["a", "b", "c", "a"],
....: categories=["c", "b", "a"]))
....:
```

```
In [61]: s.cat.categories
```

```
Out[61]: Index(['c', 'b', 'a'], dtype='object')
```

```
In [62]: s.cat.ordered
```

```
Out[62]: False
```

::: tip Note

New categorical data are **not** automatically ordered. You must explicitly pass `ordered=True` to indicate an ordered `Categorical`.

...

... tip Note

The result of `unique()` is not always the same as `Series.cat.categories`, because `Series.unique()` has a couple of guarantees, namely that it returns categories in the order of appearance, and it only includes values that are actually present.

```
In [63]: s = pd.Series(list('babc')).astype(CategoricalDtype(list('abcd')))

In [64]: s
Out[64]:
0 b
1 a
2 b
3 c
dtype: category
Categories (4, object): [a, b, c, d]

categories
In [65]: s.cat.categories
Out[65]: Index(['a', 'b', 'c', 'd'], dtype='object')

uniques
In [66]: s.unique()
Out[66]:
[b, a, c]
Categories (3, object): [b, a, c]
```

...

## Renaming categories

Renaming categories is done by assigning new values to the `Series.cat.categories` property or by using the `rename_categories()` method:

```

In [67]: s = pd.Series(["a", "b", "c", "a"], dtype="category")

In [68]: s
Out[68]:
0 a
1 b
2 c
3 a
dtype: category
Categories (3, object): [a, b, c]

In [69]: s.cat.categories = ["Group %s" % g for g in s.cat.categories]

In [70]: s
Out[70]:
0 Group a
1 Group b
2 Group c
3 Group a
dtype: category
Categories (3, object): [Group a, Group b, Group c]

In [71]: s = s.cat.rename_categories([1, 2, 3])

In [72]: s
Out[72]:
0 1
1 2
2 3
3 1
dtype: category
Categories (3, int64): [1, 2, 3]

You can also pass a dict-like object to map the renaming
In [73]: s = s.cat.rename_categories({1: 'x', 2: 'y', 3: 'z'})

In [74]: s
Out[74]:
0 x
1 y
2 z
3 x
dtype: category
Categories (3, object): [x, y, z]

```

::: tip Note

In contrast to R' s factor, categorical data can have categories of other types than string.

:::

::: tip Note

Be aware that assigning new categories is an inplace operation, while most other operations under `Series.cat` per default return a new `Series` of dtype category.

...

Categories must be unique or a `ValueError` is raised:

```
In [75]: try:
....: s.cat.categories = [1, 1, 1]
....: except ValueError as e:
....: print("ValueError:", str(e))
....:
ValueError: Categorical categories must be unique
```

Categories must also not be `NaN` or a `ValueError` is raised:

```
In [76]: try:
....: s.cat.categories = [1, 2, np.nan]
....: except ValueError as e:
....: print("ValueError:", str(e))
....:
ValueError: Categorical categories cannot be null
```

## Appending new categories

Appending categories can be done by using the `add_categories()` method:

```
In [77]: s = s.cat.add_categories([4])

In [78]: s.cat.categories
Out[78]: Index(['x', 'y', 'z', 4], dtype='object')

In [79]: s
Out[79]:
0 x
1 y
2 z
3 x
dtype: category
Categories (4, object): [x, y, z, 4]
```

## Removing categories

Removing categories can be done by using the `remove_categories()` method. Values which are removed are replaced by `np.nan` .:



```
In [80]: s = s.cat.remove_categories([4])
```

```
In [81]: s
```

```
Out[81]:
```

```
0 x
```

```
1 y
```

```
2 z
```

```
3 x
```

```
dtype: category
```

```
Categories (3, object): [x, y, z]
```

## Removing unused categories

Removing unused categories can also be done:

```
In [82]: s = pd.Series(pd.Categorical(["a", "b", "a"],
....: categories=["a", "b", "c", "d"]))
....:
```

```
In [83]: s
```

```
Out[83]:
```

```
0 a
```

```
1 b
```

```
2 a
```

```
dtype: category
```

```
Categories (4, object): [a, b, c, d]
```

```
In [84]: s.cat.remove_unused_categories()
```

```
Out[84]:
```

```
0 a
```

```
1 b
```

```
2 a
```

```
dtype: category
```

```
Categories (2, object): [a, b]
```

## Setting categories

If you want to do remove and add new categories in one step (which has some speed advantage), or simply set the categories to a predefined scale, use `set_categories()`.

```
In [85]: s = pd.Series(["one", "two", "four", "-"], dtype="category")
```

```
In [86]: s
```

```
Out[86]:
```

```
0 one
1 two
2 four
3 -
```

```
dtype: category
```

```
Categories (4, object): [-, four, one, two]
```

```
In [87]: s = s.cat.set_categories(["one", "two", "three", "four"])
```

```
In [88]: s
```

```
Out[88]:
```

```
0 one
1 two
2 four
3 NaN
```

```
dtype: category
```

```
Categories (4, object): [one, two, three, four]
```

::: tip Note

Be aware that `Categorical.set_categories()` cannot know whether some category is omitted intentionally or because it is misspelled or (under Python3) due to a type difference (e.g., NumPy S1 dtype and Python strings). This can result in surprising behaviour!

:::

## Sorting and order

---

If categorical data is ordered (`s.cat.ordered == True`), then the order of the categories has a meaning and certain operations are possible. If the categorical is unordered, `.min()/ .max()` will raise a `TypeError`.

```

In [89]: s = pd.Series(pd.Categorical(["a", "b", "c", "a"], ordered=False))

In [90]: s.sort_values(inplace=True)

In [91]: s = pd.Series(["a", "b", "c", "a"]).astype(
.....: CategoricalDtype(ordered=True)
.....:)
.....:

In [92]: s.sort_values(inplace=True)

In [93]: s
Out[93]:
0 a
3 a
1 b
2 c
dtype: category
Categories (3, object): [a < b < c]

In [94]: s.min(), s.max()
Out[94]: ('a', 'c')

```

You can set categorical data to be ordered by using `as_ordered()` or unordered by using `as_unordered()`. These will by default return a new object.

```

In [95]: s.cat.as_ordered()
Out[95]:
0 a
3 a
1 b
2 c
dtype: category
Categories (3, object): [a < b < c]

In [96]: s.cat.as_unordered()
Out[96]:
0 a
3 a
1 b
2 c
dtype: category
Categories (3, object): [a, b, c]

```

Sorting will use the order defined by categories, not any lexical order present on the data type. This is even true for strings and numeric data:

```

In [97]: s = pd.Series([1, 2, 3, 1], dtype="category")

In [98]: s = s.cat.set_categories([2, 3, 1], ordered=True)

In [99]: s
Out[99]:
0 1
1 2
2 3
3 1
dtype: category
Categories (3, int64): [2 < 3 < 1]

In [100]: s.sort_values(inplace=True)

In [101]: s
Out[101]:
1 2
2 3
0 1
3 1
dtype: category
Categories (3, int64): [2 < 3 < 1]

In [102]: s.min(), s.max()
Out[102]: (2, 1)

```

## Reordering

Reordering the categories is possible via the `Categorical.reorder_categories()` and the `Categorical.set_categories()` methods. For `Categorical.reorder_categories()`, all old categories must be included in the new categories and no new categories are allowed. This will necessarily make the sort order the same as the categories order.

```

In [103]: s = pd.Series([1, 2, 3, 1], dtype="category")

In [104]: s = s.cat.reorder_categories([2, 3, 1], ordered=True)

In [105]: s
Out[105]:
0 1
1 2
2 3
3 1
dtype: category
Categories (3, int64): [2 < 3 < 1]

In [106]: s.sort_values(inplace=True)

In [107]: s
Out[107]:
1 2
2 3
0 1
3 1
dtype: category
Categories (3, int64): [2 < 3 < 1]

In [108]: s.min(), s.max()
Out[108]: (2, 1)

```

::: tip Note

Note the difference between assigning new categories and reordering the categories: the first renames categories and therefore the individual values in the `Series`, but if the first position was sorted last, the renamed value will still be sorted last. Reordering means that the way values are sorted is different afterwards, but not that individual values in the `Series` are changed.

:::

::: tip Note

If the `Categorical` is not ordered, `Series.min()` and `Series.max()` will raise `TypeError`. Numeric operations like `+`, `-`, `*`, `/` and operations based on them (e.g. `Series.median()`, which would need to compute the mean between two values if the length of an array is even) do not work and raise a `TypeError`.

:::

## Multi column sorting

A categorical dtyped column will participate in a multi-column sort in a similar manner to other columns. The ordering of the categorical is determined by the `categories` of that column.

```
In [109]: dfs = pd.DataFrame({'A': pd.Categorical(list('bbeebbaa'),
.....: categories=['e', 'a', 'b'],
.....: ordered=True),
.....: 'B': [1, 2, 1, 2, 2, 1, 2, 1]})
.....:
```

```
In [110]: dfs.sort_values(by=['A', 'B'])
```

```
Out[110]:
```

```
 A B
2 e 1
3 e 2
7 a 1
6 a 2
0 b 1
5 b 1
1 b 2
4 b 2
```

Reordering the `categories` changes a future sort.

```
In [111]: dfs['A'] = dfs['A'].cat.reorder_categories(['a', 'b', 'e'])
```

```
In [112]: dfs.sort_values(by=['A', 'B'])
```

```
Out[112]:
```

```
 A B
7 a 1
6 a 2
0 b 1
5 b 1
1 b 2
4 b 2
2 e 1
3 e 2
```

## Comparisons

Comparing categorical data with other objects is possible in three cases:

- Comparing equality (`==` and `!=`) to a list-like object (list, Series, array, ...) of the same length as the categorical data.
- All comparisons (`==`, `!=`, `>`, `>=`, `<`, and `<=`) of categorical data to another categorical Series, when `ordered==True` and the categories are the same.
- All comparisons of a categorical data to a scalar.

All other comparisons, especially “non-equality” comparisons of two categoricals with different categories or a categorical with any list-like object, will raise a `TypeError`.

::: tip Note

Any “non-equality” comparisons of categorical data with a `Series`, `np.array`, `List` or categorical data with different categories or ordering will raise a `TypeError` because custom categories ordering could be interpreted in two ways: one with taking into account the ordering and one without.

...

```
In [113]: cat = pd.Series([1, 2, 3]).astype(
.....: CategoricalDtype([3, 2, 1], ordered=True)
.....:)
.....:
```

```
In [114]: cat_base = pd.Series([2, 2, 2]).astype(
.....: CategoricalDtype([3, 2, 1], ordered=True)
.....:)
.....:
```

```
In [115]: cat_base2 = pd.Series([2, 2, 2]).astype(
.....: CategoricalDtype(ordered=True)
.....:)
.....:
```

```
In [116]: cat
Out[116]:
0 1
1 2
2 3
dtype: category
Categories (3, int64): [3 < 2 < 1]
```

```
In [117]: cat_base
Out[117]:
0 2
1 2
2 2
dtype: category
Categories (3, int64): [3 < 2 < 1]
```

```
In [118]: cat_base2
Out[118]:
0 2
1 2
2 2
dtype: category
Categories (1, int64): [2]
```

Comparing to a categorical with the same categories and ordering or to a scalar works:

```
In [119]: cat > cat_base
Out[119]:
0 True
1 False
2 False
dtype: bool
```

```
In [120]: cat > 2
Out[120]:
0 True
1 False
2 False
dtype: bool
```

Equality comparisons work with any list-like object of same length and scalars:

```
In [121]: cat == cat_base
Out[121]:
0 False
1 True
2 False
dtype: bool
```

```
In [122]: cat == np.array([1, 2, 3])
Out[122]:
0 True
1 True
2 True
dtype: bool
```

```
In [123]: cat == 2
Out[123]:
0 False
1 True
2 False
dtype: bool
```

This doesn't work because the categories are not the same:

```
In [124]: try:
.....: cat > cat_base2
.....: except TypeError as e:
.....: print("TypeError:", str(e))
.....:
TypeError: Categoricals can only be compared if 'categories' are the same. Categories are different lengths
```

If you want to do a “non-equality” comparison of a categorical series with a list-like object which is not categorical data, you need to be explicit and convert the categorical data back to the original values:



```

In [125]: base = np.array([1, 2, 3])

In [126]: try:
.....: cat > base
.....: except TypeError as e:
.....: print("TypeError:", str(e))
.....:
TypeError: Cannot compare a Categorical for op __gt__ with type <class 'numpy.ndarray'>.
If you want to compare values, use 'np.asarray(cat) <op> other'.

In [127]: np.asarray(cat) > base
Out[127]: array([False, False, False])

```

When you compare two unordered categoricals with the same categories, the order is not considered:

```

In [128]: c1 = pd.Categorical(['a', 'b'], categories=['a', 'b'], ordered=False)

In [129]: c2 = pd.Categorical(['a', 'b'], categories=['b', 'a'], ordered=False)

In [130]: c1 == c2
Out[130]: array([True, True])

```

## Operations

Apart from `Series.min()`, `Series.max()` and `Series.mode()`, the following operations are possible with categorical data:

`Series` methods like `Series.value_counts()` will use all categories, even if some categories are not present in the data:

```

In [131]: s = pd.Series(pd.Categorical(["a", "b", "c", "c"],
.....: categories=["c", "a", "b", "d"]))
.....:

In [132]: s.value_counts()
Out[132]:
c 2
b 1
a 1
d 0
dtype: int64

```

Groupby will also show “unused” categories:

```

In [133]: cats = pd.Categorical(["a", "b", "b", "b", "c", "c", "c"],
.....: categories=["a", "b", "c", "d"])
.....:

In [134]: df = pd.DataFrame({"cats": cats, "values": [1, 2, 2, 2, 3, 4, 5]})

In [135]: df.groupby("cats").mean()
Out[135]:
 values
cats
a 1.0
b 2.0
c 4.0
d NaN

In [136]: cats2 = pd.Categorical(["a", "a", "b", "b"], categories=["a", "b", "c"])

In [137]: df2 = pd.DataFrame({"cats": cats2,
.....: "B": ["c", "d", "c", "d"],
.....: "values": [1, 2, 3, 4]})
.....:

In [138]: df2.groupby(["cats", "B"]).mean()
Out[138]:
 values
cats B
a c 1.0
 d 2.0
b c 3.0
 d 4.0
c c NaN
 d NaN

```

## Pivot tables:

```

In [139]: raw_cat = pd.Categorical(["a", "a", "b", "b"], categories=["a", "b", "c"])

In [140]: df = pd.DataFrame({"A": raw_cat,
.....: "B": ["c", "d", "c", "d"],
.....: "values": [1, 2, 3, 4]})
.....:

In [141]: pd.pivot_table(df, values='values', index=['A', 'B'])
Out[141]:
 values
A B
a c 1
 d 2
b c 3
 d 4

```

## Data munging

The optimized pandas data access methods `.loc`, `.iloc`, `.at`, and `.iat`, work as normal. The only difference is the return type (for getting) and that only values already in categories can be assigned.

### Getting

If the slicing operation returns either a `DataFrame` or a column of type `Series`, the `category` dtype is preserved.

```
In [142]: idx = pd.Index(["h", "i", "j", "k", "l", "m", "n"])

In [143]: cats = pd.Series(["a", "b", "b", "b", "c", "c", "c"],
.....: dtype="category", index=idx)
.....:

In [144]: values = [1, 2, 2, 2, 3, 4, 5]

In [145]: df = pd.DataFrame({"cats": cats, "values": values}, index=idx)

In [146]: df.iloc[2:4, :]
Out[146]:
 cats values
j b 2
k b 2

In [147]: df.iloc[2:4, :].dtypes
Out[147]:
cats category
values int64
dtype: object

In [148]: df.loc["h":"j", "cats"]
Out[148]:
h a
i b
j b
Name: cats, dtype: category
Categories (3, object): [a, b, c]

In [149]: df[df["cats"] == "b"]
Out[149]:
 cats values
i b 2
j b 2
k b 2
```

An example where the category type is not preserved is if you take one single row: the resulting `Series` is of dtype `object`:

```
get the complete "h" row as a Series
In [150]: df.loc["h", :]
Out[150]:
cats a
values 1
Name: h, dtype: object
```

Returning a single item from categorical data will also return the value, not a categorical of length “1” .

```
In [151]: df.iat[0, 0]
Out[151]: 'a'

In [152]: df["cats"].cat.categories = ["x", "y", "z"]

In [153]: df.at["h", "cats"] # returns a string
Out[153]: 'x'
```

::: tip Note

The is in contrast to R’ s factor function, where `factor(c(1,2,3))[1]` returns a single value factor.

:::

To get a single value `Series` of type `category` , you pass in a list with a single value:

```
In [154]: df.loc[["h"], "cats"]
Out[154]:
h x
Name: cats, dtype: category
Categories (3, object): [x, y, z]
```

## String and datetime accessors

The accessors `.dt` and `.str` will work if the `s.cat.categories` are of an appropriate type:

```

In [155]: str_s = pd.Series(list('aabb'))

In [156]: str_cat = str_s.astype('category')

In [157]: str_cat
Out[157]:
0 a
1 a
2 b
3 b
dtype: category
Categories (2, object): [a, b]

In [158]: str_cat.str.contains("a")
Out[158]:
0 True
1 True
2 False
3 False
dtype: bool

In [159]: date_s = pd.Series(pd.date_range('1/1/2015', periods=5))

In [160]: date_cat = date_s.astype('category')

In [161]: date_cat
Out[161]:
0 2015-01-01
1 2015-01-02
2 2015-01-03
3 2015-01-04
4 2015-01-05
dtype: category
Categories (5, datetime64[ns]): [2015-01-01, 2015-01-02, 2015-01-03, 2015-01-04, 2015-01-05]

In [162]: date_cat.dt.day
Out[162]:
0 1
1 2
2 3
3 4
4 5
dtype: int64

```

::: tip Note

The returned `Series` (or `DataFrame`) is of the same type as if you used the `.str.` / `.dt.` on a `Series` of that type (and not of type `category`!).

:::

That means, that the returned values from methods and properties on the accessors of a `Series` and the returned values from methods and properties on the accessors of this `Series` transformed to one of type `category` will be equal:

```
In [163]: ret_s = str_s.str.contains("a")

In [164]: ret_cat = str_cat.str.contains("a")

In [165]: ret_s.dtype == ret_cat.dtype
Out[165]: True

In [166]: ret_s == ret_cat
Out[166]:
0 True
1 True
2 True
3 True
dtype: bool
```

::: tip Note

The work is done on the `categories` and then a new `Series` is constructed. This has some performance implication if you have a `Series` of type string, where lots of elements are repeated (i.e. the number of unique elements in the `Series` is a lot smaller than the length of the `Series`). In this case it can be faster to convert the original `Series` to one of type `category` and use `.str.` or `.dt.` on that.

:::

## Setting

Setting values in a categorical column (or `Series`) works as long as the value is included in the categories:

```

In [167]: idx = pd.Index(["h", "i", "j", "k", "l", "m", "n"])

In [168]: cats = pd.Categorical(["a", "a", "a", "a", "a", "a", "a"],
.....: categories=["a", "b"])
.....:

In [169]: values = [1, 1, 1, 1, 1, 1, 1]

In [170]: df = pd.DataFrame({"cats": cats, "values": values}, index=idx)

In [171]: df.iloc[2:4, :] = [["b", 2], ["b", 2]]

In [172]: df
Out[172]:
 cats values
h a 1
i a 1
j b 2
k b 2
l a 1
m a 1
n a 1

In [173]: try:
.....: df.iloc[2:4, :] = [["c", 3], ["c", 3]]
.....: except ValueError as e:
.....: print("ValueError:", str(e))
.....:
ValueError: Cannot setitem on a Categorical with a new category, set the categories first

```

Setting values by assigning categorical data will also check that the categories match:

```

In [174]: df.loc["j":"k", "cats"] = pd.Categorical(["a", "a"], categories=["a", "b"])

In [175]: df
Out[175]:
 cats values
h a 1
i a 1
j a 2
k a 2
l a 1
m a 1
n a 1

In [176]: try:
.....: df.loc["j":"k", "cats"] = pd.Categorical(["b", "b"],
.....: categories=["a", "b", "c"])
.....: except ValueError as e:
.....: print("ValueError:", str(e))
.....:
ValueError: Cannot set a Categorical with another, without identical categories

```

Assigning a `Categorical` to parts of a column of other types will use the values:

```

In [177]: df = pd.DataFrame({"a": [1, 1, 1, 1, 1], "b": ["a", "a", "a", "a", "a"]})

In [178]: df.loc[1:2, "a"] = pd.Categorical(["b", "b"], categories=["a", "b"])

In [179]: df.loc[2:3, "b"] = pd.Categorical(["b", "b"], categories=["a", "b"])

In [180]: df
Out[180]:
 a b
0 1 a
1 b a
2 b b
3 1 b
4 1 a

In [181]: df.dtypes
Out[181]:
a object
b object
dtype: object

```

## Merging

You can concat two `DataFrames` containing categorical data together, but the categories of these categoricals need to be the same:

```

In [182]: cat = pd.Series(["a", "b"], dtype="category")

In [183]: vals = [1, 2]

In [184]: df = pd.DataFrame({"cats": cat, "vals": vals})

In [185]: res = pd.concat([df, df])

In [186]: res
Out[186]:
 cats vals
0 a 1
1 b 2
0 a 1
1 b 2

In [187]: res.dtypes
Out[187]:
cats category
vals int64
dtype: object

```

In this case the categories are not the same, and therefore an error is raised:



```
In [188]: df_different = df.copy()

In [189]: df_different["cats"].cat.categories = ["c", "d"]

In [190]: try:
.....: pd.concat([df, df_different])
.....: except ValueError as e:
.....: print("ValueError:", str(e))
.....:
```

The same applies to `df.append(df_different)`.

See also the section on [merge dtypes](#) for notes about preserving merge dtypes and performance.

## Unioning

New in version 0.19.0.

If you want to combine categoricals that do not necessarily have the same categories, the `union_categoricals()` function will combine a list-like of categoricals. The new categories will be the union of the categories being combined.

```
In [191]: from pandas.api.types import union_categoricals

In [192]: a = pd.Categorical(["b", "c"])

In [193]: b = pd.Categorical(["a", "b"])

In [194]: union_categoricals([a, b])
Out[194]:
[b, c, a, b]
Categories (3, object): [b, c, a]
```

By default, the resulting categories will be ordered as they appear in the data. If you want the categories to be lexicographically sorted, use `sort_categories=True` argument.

```
In [195]: union_categoricals([a, b], sort_categories=True)
Out[195]:
[b, c, a, b]
Categories (3, object): [a, b, c]
```

`union_categoricals` also works with the “easy” case of combining two categoricals of the same categories and order information (e.g. what you could also `append` for).

```
In [196]: a = pd.Categorical(["a", "b"], ordered=True)

In [197]: b = pd.Categorical(["a", "b", "a"], ordered=True)

In [198]: union_categoricals([a, b])
Out[198]:
[a, b, a, b, a]
Categories (2, object): [a < b]
```

The below raises `TypeError` because the categories are ordered and not identical.

```
In [1]: a = pd.Categorical(["a", "b"], ordered=True)
In [2]: b = pd.Categorical(["a", "b", "c"], ordered=True)
In [3]: union_categoricals([a, b])
Out[3]:
TypeError: to union ordered Categoricals, all categories must be the same
```

New in version 0.20.0.

Ordered categoricals with different categories or orderings can be combined by using the `ignore_ordered=True` argument.

```
In [199]: a = pd.Categorical(["a", "b", "c"], ordered=True)

In [200]: b = pd.Categorical(["c", "b", "a"], ordered=True)

In [201]: union_categoricals([a, b], ignore_order=True)
Out[201]:
[a, b, c, c, b, a]
Categories (3, object): [a, b, c]
```

`union_categoricals()` also works with a `CategoricalIndex`, or `Series` containing categorical data, but note that the resulting array will always be a plain `Categorical`:

```
In [202]: a = pd.Series(["b", "c"], dtype='category')

In [203]: b = pd.Series(["a", "b"], dtype='category')

In [204]: union_categoricals([a, b])
Out[204]:
[b, c, a, b]
Categories (3, object): [b, c, a]
```

::: tip Note

`union_categoricals` may recode the integer codes for categories when combining categoricals. This is likely what you want, but if you are relying on the exact numbering of the categories, be aware.

```

In [205]: c1 = pd.Categorical(["b", "c"])

In [206]: c2 = pd.Categorical(["a", "b"])

In [207]: c1
Out[207]:
[b, c]
Categories (2, object): [b, c]

"b" is coded to 0
In [208]: c1.codes
Out[208]: array([0, 1], dtype=int8)

In [209]: c2
Out[209]:
[a, b]
Categories (2, object): [a, b]

"b" is coded to 1
In [210]: c2.codes
Out[210]: array([0, 1], dtype=int8)

In [211]: c = union_categoricals([c1, c2])

In [212]: c
Out[212]:
[b, c, a, b]
Categories (3, object): [b, c, a]

"b" is coded to 0 throughout, same as c1, different from c2
In [213]: c.codes
Out[213]: array([0, 1, 2, 0], dtype=int8)

```

...

## Concatenation

This section describes concatenations specific to `category` dtype. See [Concatenating objects](#) for general description.

By default, `Series` or `DataFrame` concatenation which contains the same categories results in `category` dtype, otherwise results in `object` dtype. Use `.astype` or `union_categoricals` to get `category` result.

```

same categories
In [214]: s1 = pd.Series(['a', 'b'], dtype='category')

In [215]: s2 = pd.Series(['a', 'b', 'a'], dtype='category')

In [216]: pd.concat([s1, s2])
Out[216]:
0 a
1 b
0 a
1 b
2 a
dtype: category
Categories (2, object): [a, b]

different categories
In [217]: s3 = pd.Series(['b', 'c'], dtype='category')

In [218]: pd.concat([s1, s3])
Out[218]:
0 a
1 b
0 b
1 c
dtype: object

In [219]: pd.concat([s1, s3]).astype('category')
Out[219]:
0 a
1 b
0 b
1 c
dtype: category
Categories (3, object): [a, b, c]

In [220]: union_categoricals([s1.array, s3.array])
Out[220]:
[a, b, b, c]
Categories (3, object): [a, b, c]

```

Following table summarizes the results of `Categoricals` related concatenations.

arg1	arg2	result
category	category (identical categories)	category
category	category (different categories, both not ordered)	object (dtype is inferred)
category	category (different categories, either one is ordered)	object (dtype is inferred)
category	not category	object (dtype is inferred)

## Getting data in/out

---

You can write data that contains `category` dtypes to a `HDFStore`. See [here](#) for an example and caveats.

It is also possible to write data to and reading data from Stata format files. See [here](#) for an example and caveats.

Writing to a CSV file will convert the data, effectively removing any information about the categorical (categories and ordering). So if you read back the CSV file you have to convert the relevant columns back to category and assign the right categories and categories ordering.

```

In [221]: import io

In [222]: s = pd.Series(pd.Categorical(['a', 'b', 'b', 'a', 'a', 'd']))

rename the categories
In [223]: s.cat.categories = ["very good", "good", "bad"]

reorder the categories and add missing categories
In [224]: s = s.cat.set_categories(["very bad", "bad", "medium", "good", "very good"])

In [225]: df = pd.DataFrame({"cats": s, "vals": [1, 2, 3, 4, 5, 6]})

In [226]: csv = io.StringIO()

In [227]: df.to_csv(csv)

In [228]: df2 = pd.read_csv(io.StringIO(csv.getvalue()))

In [229]: df2.dtypes
Out[229]:
Unnamed: 0 int64
cats object
vals int64
dtype: object

In [230]: df2["cats"]
Out[230]:
0 very good
1 good
2 good
3 very good
4 very good
5 bad
Name: cats, dtype: object

Redo the category
In [231]: df2["cats"] = df2["cats"].astype("category")

In [232]: df2["cats"].cat.set_categories(["very bad", "bad", "medium",
.....: "good", "very good"],
.....: inplace=True)
.....:

In [233]: df2.dtypes
Out[233]:
Unnamed: 0 int64
cats category
vals int64
dtype: object

In [234]: df2["cats"]
Out[234]:
0 very good
1 good
2 good

```

```
3 very good
4 very good
5 bad
Name: cats, dtype: category
Categories (5, object): [very bad, bad, medium, good, very good]
```

The same holds for writing to a SQL database with `to_sql`.

## Missing data

pandas primarily uses the value `np.nan` to represent missing data. It is by default not included in computations. See the [Missing Data section](#).

Missing values should **not** be included in the Categorical's `categories`, only in the `values`. Instead, it is understood that `NaN` is different, and is always a possibility. When working with the Categorical's `codes`, missing values will always have a code of `-1`.

```
In [235]: s = pd.Series(["a", "b", np.nan, "a"], dtype="category")

only two categories
In [236]: s
Out[236]:
0 a
1 b
2 NaN
3 a
dtype: category
Categories (2, object): [a, b]

In [237]: s.cat.codes
Out[237]:
0 0
1 1
2 -1
3 0
dtype: int8
```

Methods for working with missing data, e.g. `isna()`, `fillna()`, `dropna()`, all work normally:

```
In [238]: s = pd.Series(["a", "b", np.nan], dtype="category")
```

```
In [239]: s
```

```
Out[239]:
```

```
0 a
```

```
1 b
```

```
2 NaN
```

```
dtype: category
```

```
Categories (2, object): [a, b]
```

```
In [240]: pd.isna(s)
```

```
Out[240]:
```

```
0 False
```

```
1 False
```

```
2 True
```

```
dtype: bool
```

```
In [241]: s.fillna("a")
```

```
Out[241]:
```

```
0 a
```

```
1 b
```

```
2 a
```

```
dtype: category
```

```
Categories (2, object): [a, b]
```

## Differences to R' s factor

---

The following differences to R' s factor functions can be observed:

- R' s levels are named categories.
- R' s levels are always of type string, while categories in pandas can be of any dtype.
- It' s not possible to specify labels at creation time. Use `s.cat.rename_categories(new_labels)` afterwards.
- In contrast to R' s factor function, using categorical data as the sole input to create a new categorical series will not remove unused categories but create a new categorical series which is equal to the passed in one!
- R allows for missing values to be included in its levels (pandas' categories). Pandas does not allow NaN categories, but missing values can still be in the values.

## Gotchas

---

### Memory usage

The memory usage of a `Categorical` is proportional to the number of categories plus the length of the data. In contrast, an `object` dtype is a constant times the length of the data.



```
In [242]: s = pd.Series(['foo', 'bar'] * 1000)

object dtype
In [243]: s.nbytes
Out[243]: 16000

category dtype
In [244]: s.astype('category').nbytes
Out[244]: 2016
```

::: tip Note

If the number of categories approaches the length of the data, the `Categorical` will use nearly the same or more memory than an equivalent `object` dtype representation.

```
In [245]: s = pd.Series(['foo%04d' % i for i in range(2000)])

object dtype
In [246]: s.nbytes
Out[246]: 16000

category dtype
In [247]: s.astype('category').nbytes
Out[247]: 20000
```

:::

## Categorical is not a numpy array

Currently, categorical data and the underlying `Categorical` is implemented as a Python object and not as a low-level NumPy array dtype. This leads to some problems.

NumPy itself doesn't know about the new dtype:

```
In [248]: try:
.....: np.dtype("category")
.....: except TypeError as e:
.....: print("TypeError:", str(e))
.....:
TypeError: data type "category" not understood

In [249]: dtype = pd.Categorical(["a"]).dtype

In [250]: try:
.....: np.dtype(dtype)
.....: except TypeError as e:
.....: print("TypeError:", str(e))
.....:
TypeError: data type not understood
```

Dtype comparisons work:

```
In [251]: dtype == np.str_
Out[251]: False

In [252]: np.str_ == dtype
Out[252]: False
```

To check if a Series contains Categorical data, use `hasattr(s, 'cat')`:

```
In [253]: hasattr(pd.Series(['a'], dtype='category'), 'cat')
Out[253]: True

In [254]: hasattr(pd.Series(['a']), 'cat')
Out[254]: False
```

Using NumPy functions on a Series of type `category` should not work as Categoricals are not numeric data (even in the case that `.categories` is numeric).

```
In [255]: s = pd.Series(pd.Categorical([1, 2, 3, 4]))

In [256]: try:
.....: np.sum(s)
.....: except TypeError as e:
.....: print("TypeError:", str(e))
.....:
TypeError: Categorical cannot perform the operation sum
```

::: tip Note

If such a function works, please file a bug at <https://github.com/pandas-dev/pandas>!

:::

## dtype in apply

Pandas currently does not preserve the dtype in apply functions: If you apply along rows you get a Series of `object` dtype (same as getting a row -> getting one element will return a basic type) and applying along columns will also convert to object. `NaN` values are unaffected. You can use `fillna` to handle missing values before applying a function.

```
In [257]: df = pd.DataFrame({"a": [1, 2, 3, 4],
.....: "b": ["a", "b", "c", "d"],
.....: "cats": pd.Categorical([1, 2, 3, 2])})
.....:
```

```
In [258]: df.apply(lambda row: type(row["cats"]), axis=1)
```

```
Out[258]:
0 <class 'int'>
1 <class 'int'>
2 <class 'int'>
3 <class 'int'>
dtype: object
```

```
In [259]: df.apply(lambda col: col.dtype, axis=0)
```

```
Out[259]:
a int64
b object
cats category
dtype: object
```

## Categorical index

`CategoricalIndex` is a type of index that is useful for supporting indexing with duplicates. This is a container around a `Categorical` and allows efficient indexing and storage of an index with a large number of duplicated elements. See the [advanced indexing docs](#) for a more detailed explanation.

Setting the index will create a `CategoricalIndex`:

```
In [260]: cats = pd.Categorical([1, 2, 3, 4], categories=[4, 2, 3, 1])
```

```
In [261]: strings = ["a", "b", "c", "d"]
```

```
In [262]: values = [4, 2, 3, 1]
```

```
In [263]: df = pd.DataFrame({"strings": strings, "values": values}, index=cats)
```

```
In [264]: df.index
```

```
Out[264]: CategoricalIndex([1, 2, 3, 4], categories=[4, 2, 3, 1], ordered=False, dtype='category')
```

```
This now sorts by the categories order
```

```
In [265]: df.sort_index()
```

```
Out[265]:
 strings values
4 d 1
2 b 2
3 c 3
1 a 4
```

## Side effects

Constructing a `Series` from a `Categorical` will not copy the input `Categorical`. This means that changes to the `Series` will in most cases change the original `Categorical`:

```
In [266]: cat = pd.Categorical([1, 2, 3, 10], categories=[1, 2, 3, 4, 10])

In [267]: s = pd.Series(cat, name="cat")

In [268]: cat
Out[268]:
[1, 2, 3, 10]
Categories (5, int64): [1, 2, 3, 4, 10]

In [269]: s.iloc[0:2] = 10

In [270]: cat
Out[270]:
[10, 10, 3, 10]
Categories (5, int64): [1, 2, 3, 4, 10]

In [271]: df = pd.DataFrame(s)

In [272]: df["cat"].cat.categories = [1, 2, 3, 4, 5]

In [273]: cat
Out[273]:
[5, 5, 3, 5]
Categories (5, int64): [1, 2, 3, 4, 5]
```

Use `copy=True` to prevent such a behaviour or simply don't reuse `Categoricals`:

```
In [274]: cat = pd.Categorical([1, 2, 3, 10], categories=[1, 2, 3, 4, 10])

In [275]: s = pd.Series(cat, name="cat", copy=True)

In [276]: cat
Out[276]:
[1, 2, 3, 10]
Categories (5, int64): [1, 2, 3, 4, 10]

In [277]: s.iloc[0:2] = 10

In [278]: cat
Out[278]:
[1, 2, 3, 10]
Categories (5, int64): [1, 2, 3, 4, 10]
```

::: tip Note

This also happens in some cases when you supply a NumPy array instead of a `Categorical`: using an int array (e.g. `np.array([1,2,3,4])`) will exhibit the same behavior, while using a string array (e.g. `np.array(["a","b","c","a"])`) will not.

...

---

## Nullable整型数据类型

在0.24.0版本中新引入

::: tip 小贴士

`IntegerArray`目前属于实验性阶段，因此他的API或者使用方式可能会在没有提示的情况下更改。

...

在 [处理丢失的数据](#)部分, 我们知道pandas主要使用 `NaN` 来代表丢失数据。因为 `NaN` 属于浮点型数据，这强制有缺失值的整型array强制转换成浮点型。在某些情况下，这可能不会有太大影响，但是如果你的整型数据恰好是标识符，数据类型的转换可能会存在隐患。同时，某些整数无法使用浮点型来表示。

Pandas能够将可能存在缺失值的整型数据使用`arrays.IntegerArray`来表示。这是pandas中内置的 [扩展方式](#)。它并不是整型数据组成array对象的默认方式，并且并不会被pandas直接使用。因此，如果你希望生成这种数据类型，你需要在生成`array()` 或者 `Series`时，在 `dtype` 变量中直接指定。

```
In [1]: arr = pd.array([1, 2, np.nan], dtype=pd.Int64Dtype())
```

```
In [2]: arr
Out[2]:
<IntegerArray>
[1, 2, NaN]
Length: 3, dtype: Int64
```

或者使用字符串 `"Int64"`（注意此处的 `"I"` 需要大写，以此和NumPy中的 `'int64'` 数据类型作出区别）：

```
In [3]: pd.array([1, 2, np.nan], dtype="Int64")
```

```
Out[3]:
<IntegerArray>
[1, 2, NaN]
Length: 3, dtype: Int64
```

这样的array对象与NumPy的array对象类似，可以被存放在[DataFrame](#) 或 `Series`中。

```
In [4]: pd.Series(arr)
Out[4]:
0 1
1 2
2 NaN
dtype: Int64
```

你也可以直接将列表形式的数据直接传入Series中，并指明 dtype。

```
In [5]: s = pd.Series([1, 2, np.nan], dtype="Int64")

In [6]: s
Out[6]:
0 1
1 2
2 NaN
dtype: Int64
```

默认情况下（如果你不指明 dtype），则会使用NumPy来构建这个数据，最终你会得到 float64 类型的 Series：

```
In [7]: pd.Series([1, 2, np.nan])
Out[7]:
0 1.0
1 2.0
2 NaN
dtype: float64
```

对使用了整型array的操作与对NumPy中array的操作类似，缺失值会被继承并保留原本的数据类型，但在必要的情况下，数据类型也会发生转变。

```

运算
In [8]: s + 1
Out[8]:
0 2
1 3
2 NaN
dtype: Int64

比较
In [9]: s == 1
Out[9]:
0 True
1 False
2 False
dtype: bool

索引
In [10]: s.iloc[1:3]
Out[10]:
1 2
2 NaN
dtype: Int64

和其他数据类型联合使用
In [11]: s + s.iloc[1:3].astype('Int8')
Out[11]:
0 NaN
1 4
2 NaN
dtype: Int64

在必要情况下，数据类型发生转变
In [12]: s + 0.01
Out[12]:
0 1.01
1 2.01
2 NaN
dtype: float64

```

这种数据类型可以作为 `DataFrame` 的一部分进行使用。

```

In [13]: df = pd.DataFrame({'A': s, 'B': [1, 1, 3], 'C': list('aab')})

In [14]: df
Out[14]:
 A B C
0 1 1 a
1 2 1 a
2 NaN 3 b

In [15]: df.dtypes
Out[15]:
A Int64
B int64
C object
dtype: object

```

这种数据类型也可以在合并（merge）、重构（reshape）和类型转换（cast）。

```

In [16]: pd.concat([df[['A']], df[['B', 'C']]], axis=1).dtypes
Out[16]:
A Int64
B int64
C object
dtype: object

In [17]: df['A'].astype(float)
Out[17]:
0 1.0
1 2.0
2 NaN
Name: A, dtype: float64

```

类似于求和的降维和分组操作也能正常使用。

```

In [18]: df.sum()
Out[18]:
A 3
B 5
C aab
dtype: object

In [19]: df.groupby('B').A.sum()
Out[19]:
B
1 3
3 0
Name: A, dtype: Int64

```

## Visualization

We use the standard convention for referencing the matplotlib API:



```
In [1]: import matplotlib.pyplot as plt
```

```
In [2]: plt.close('all')
```

We provide the basics in pandas to easily create decent looking plots. See the [ecosystem](#) section for visualization libraries that go beyond the basics documented here.

::: tip Note

All calls to `np.random` are seeded with 123456.

:::

## Basic plotting: `plot`

---

We will demonstrate the basics, see the [cookbook](#) for some advanced strategies.

The `plot` method on Series and DataFrame is just a simple wrapper around `plt.plot()`:

```
In [3]: ts = pd.Series(np.random.randn(1000),
...: index=pd.date_range('1/1/2000', periods=1000))
...:
```

```
In [4]: ts = ts.cumsum()
```

```
In [5]: ts.plot()
```

```
Out[5]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d8c0ac50>
```

### series\_plot\_basic

If the index consists of dates, it calls `gcf().autofmt_xdate()` to try to format the x-axis nicely as per above.

On DataFrame, `plot()` is a convenience to plot all of the columns with labels:

```
In [6]: df = pd.DataFrame(np.random.randn(1000, 4),
...: index=ts.index, columns=List('ABCD'))
...:
```

```
In [7]: df = df.cumsum()
```

```
In [8]: plt.figure();
```

```
In [9]: df.plot();
```

### frame plot basic

You can plot one column versus another using the `x` and `y` keywords in `plot()`:

```
In [10]: df3 = pd.DataFrame(np.random.randn(1000, 2), columns=['B', 'C']).cumsum()

In [11]: df3['A'] = pd.Series(List(range(len(df))))

In [12]: df3.plot(x='A', y='B')
Out[12]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d97c1668>
```

df\_plot\_xy

::: tip Note

For more formatting and styling options, see [formatting](#) below.

:::

## Other plots

Plotting methods allow for a handful of plot styles other than the default line plot. These methods can be provided as the `kind` keyword argument to `plot()`, and include:

- `'bar'` or `'barh'` for bar plots
- `'hist'` for histogram
- `'box'` for boxplot
- `'kde'` or `'density'` for density plots
- `'area'` for area plots
- `'scatter'` for scatter plots
- `'hexbin'` for hexagonal bin plots
- `'pie'` for pie plots

For example, a bar plot can be created the following way:

```
In [13]: plt.figure();

In [14]: df.iloc[5].plot(kind='bar');
```

bar\_plot\_ex

You can also create these other plots using the methods `DataFrame.plot.` instead of providing the `kind` keyword argument. This makes it easier to discover plot methods and the specific arguments they use:

```
In [15]: df = pd.DataFrame()

In [16]: df.plot.<TAB> # noqa: E225, E999
df.plot.area df.plot.barh df.plot.density df.plot.hist df.plot.line df.plot.scatter
df.plot.bar df.plot.box df.plot.hexbin df.plot.kde df.plot.pie
```

In addition to these `kind`s, there are the `DataFrame.hist()`, and `DataFrame.boxplot()` methods, which use a separate interface.

Finally, there are several [plotting functions](#) in `pandas.plotting` that take a `Series` or `DataFrame` as an argument. These include:

- [Scatter Matrix](#)
- [Andrews Curves](#)
- [Parallel Coordinates](#)
- [Lag Plot](#)
- [Autocorrelation Plot](#)
- [Bootstrap Plot](#)
- [RadViz](#)

Plots may also be adorned with [errorbars](#) or [tables](#).

## Bar plots

For labeled, non-time series data, you may wish to produce a bar plot:

```
In [17]: plt.figure();

In [18]: df.iloc[5].plot.bar()
Out[18]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da446a90>

In [19]: plt.axhline(0, color='k');
```

bar plot ex

Calling a `DataFrame`'s `plot.bar()` method produces a multiple bar plot:

```
In [20]: df2 = pd.DataFrame(np.random.rand(10, 4), columns=['a', 'b', 'c', 'd'])

In [21]: df2.plot.bar();
```

bar plot multi ex

To produce a stacked bar plot, pass `stacked=True`:

```
In [22]: df2.plot.bar(stacked=True);
```

bar plot stacked ex

To get horizontal bar plots, use the `barh` method:

```
In [23]: df2.plot.barh(stacked=True);
```

barh\_plot\_stacked\_ex

## Histograms

Histograms can be drawn by using the `DataFrame.plot.hist()` and `Series.plot.hist()` methods.

```
In [24]: df4 = pd.DataFrame({'a': np.random.randn(1000) + 1, 'b': np.random.randn(1000),
.....: 'c': np.random.randn(1000) - 1}, columns=['a', 'b', 'c'])
.....:

In [25]: plt.figure();

In [26]: df4.plot.hist(alpha=0.5)
Out[26]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da345e48>
```

hist new

A histogram can be stacked using `stacked=True`. Bin size can be changed using the `bins` keyword.

```
In [27]: plt.figure();

In [28]: df4.plot.hist(stacked=True, bins=20)
Out[28]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da30b9b0>
```

hist new stacked

You can pass other keywords supported by matplotlib `hist`. For example, horizontal and cumulative histograms can be drawn by `orientation='horizontal'` and `cumulative=True`.

```
In [29]: plt.figure();

In [30]: df4['a'].plot.hist(orientation='horizontal', cumulative=True)
Out[30]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da69fd68>
```

hist new kwargs

See the [hist](#) method and the [matplotlib hist documentation](#) for more.

The existing interface `DataFrame.hist` to plot histogram still can be used.

```
In [31]: plt.figure();

In [32]: df['A'].diff().hist()
Out[32]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65dac9d240>
```

hist plot ex

`DataFrame.hist()` plots the histograms of the columns on multiple subplots:

```
In [33]: plt.figure()
Out[33]: <Figure size 640x480 with 0 Axes>

In [34]: df.diff().hist(color='k', alpha=0.5, bins=50)
Out[34]:
array([[<matplotlib.axes._subplots.AxesSubplot object at 0x7f6601550cc0>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f66079a9400>],
 [<matplotlib.axes._subplots.AxesSubplot object at 0x7f65f87ac828>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f6604bd6b70>]],
 dtype=object)
```

frame hist ex

The `by` keyword can be specified to plot grouped histograms:

```
In [35]: data = pd.Series(np.random.randn(1000))

In [36]: data.hist(by=np.random.randint(0, 4, 1000), figsize=(6, 4))
Out[36]:
array([[<matplotlib.axes._subplots.AxesSubplot object at 0x7f6601550ef0>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d9b82438>],
 [<matplotlib.axes._subplots.AxesSubplot object at 0x7f65dc30ba58>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65f63c2320>]],
 dtype=object)
```

grouped\_hist

## Box plots

Boxplot can be drawn calling `Series.plot.box()` and `DataFrame.plot.box()`, or `DataFrame.boxplot()` to visualize the distribution of values within each column.

For instance, here is a boxplot representing five trials of 10 observations of a uniform random variable on [0,1).

```
In [37]: df = pd.DataFrame(np.random.rand(10, 5), columns=['A', 'B', 'C', 'D', 'E'])

In [38]: df.plot.box()
Out[38]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da17f898>
```

box\_plot\_new

Boxplot can be colorized by passing `color` keyword. You can pass a `dict` whose keys are `boxes`, `whiskers`, `medians` and `caps`. If some keys are missing in the `dict`, default colors are used for the corresponding artists. Also, boxplot has `sym` keyword to specify fliers style.

When you pass other type of arguments via `color` keyword, it will be directly passed to matplotlib for all the `boxes`, `whiskers`, `medians` and `caps` colorization.

The colors are applied to every boxes to be drawn. If you want more complicated colorization, you can get each drawn artists by passing `return_type`.

```
In [39]: color = {'boxes': 'DarkGreen', 'whiskers': 'DarkOrange',
....: 'medians': 'DarkBlue', 'caps': 'Gray'}
....:

In [40]: df.plot.box(color=color, sym='r+')
Out[40]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da880b00>
```

box new colorize

Also, you can pass other keywords supported by matplotlib `boxplot`. For example, horizontal and custom-positioned boxplot can be drawn by `vert=False` and `positions` keywords.

```
In [41]: df.plot.box(vert=False, positions=[1, 4, 5, 6, 8])
Out[41]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65db18ffd0>
```

box new kwargs

See the [boxplot](#) method and the [matplotlib boxplot documentation](#) for more.

The existing interface `DataFrame.boxplot` to plot boxplot still can be used.

```
In [42]: df = pd.DataFrame(np.random.rand(10, 5))

In [43]: plt.figure();

In [44]: bp = df.boxplot()
```

box plot ex

You can create a stratified boxplot using the `by` keyword argument to create groupings. For instance,

```
In [45]: df = pd.DataFrame(np.random.rand(10, 2), columns=['Col1', 'Col2'])

In [46]: df['X'] = pd.Series(['A', 'A', 'A', 'A', 'A', 'B', 'B', 'B', 'B', 'B'])

In [47]: plt.figure();

In [48]: bp = df.boxplot(by='X')
```

box plot ex2

You can also pass a subset of columns to plot, as well as group by multiple columns:

```
In [49]: df = pd.DataFrame(np.random.rand(10, 3), columns=['Col1', 'Col2', 'Col3'])

In [50]: df['X'] = pd.Series(['A', 'A', 'A', 'A', 'A', 'B', 'B', 'B', 'B', 'B'])

In [51]: df['Y'] = pd.Series(['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B', 'A', 'B'])

In [52]: plt.figure();

In [53]: bp = df.boxplot(column=['Col1', 'Col2'], by=['X', 'Y'])
```

box plot ex3

::: danger Warning

The default changed from 'dict' to 'axes' in version 0.19.0.

:::

In `boxplot`, the return type can be controlled by the `return_type`, keyword. The valid choices are `{"axes", "dict", "both", None}`. Faceting, created by `DataFrame.boxplot` with the `by` keyword, will affect the output type as well:

return_type=	Faceted	Output type
None	No	axes
None	Yes	2-D ndarray of axes
'axes'	No	axes
'axes'	Yes	Series of axes
'dict'	No	dict of artists
'dict'	Yes	Series of dicts of artists
'both'	No	namedtuple
'both'	Yes	Series of namedtuples

`Groupby.boxplot` always returns a `Series` of `return_type`.

```
In [54]: np.random.seed(1234)

In [55]: df_box = pd.DataFrame(np.random.randn(50, 2))

In [56]: df_box['g'] = np.random.choice(['A', 'B'], size=50)

In [57]: df_box.loc[df_box['g'] == 'B', 1] += 3

In [58]: bp = df_box.boxplot(by='g')
```

boxplot\_groupby

The subplots above are split by the numeric columns first, then the value of the `g` column. Below the subplots are first split by the value of `g`, then by the numeric columns.

```
In [59]: bp = df_box.groupby('g').boxplot()
```

groupby boxplot vis

## Area plot

You can create area plots with `Series.plot.area()` and `DataFrame.plot.area()`. Area plots are stacked by default. To produce stacked area plot, each column must be either all positive or all negative values.

When input data contains NaN, it will be automatically filled by 0. If you want to drop or fill by different values, use `dataframe.dropna()` or `dataframe.fillna()` before calling plot.

```
In [60]: df = pd.DataFrame(np.random.rand(10, 4), columns=['a', 'b', 'c', 'd'])
```

```
In [61]: df.plot.area();
```

area plot stacked

To produce an unstacked plot, pass `stacked=False`. Alpha value is set to 0.5 unless otherwise specified:

```
In [62]: df.plot.area(stacked=False);
```

area plot unstacked

## Scatter plot

Scatter plot can be drawn by using the `DataFrame.plot.scatter()` method. Scatter plot requires numeric columns for the x and y axes. These can be specified by the `x` and `y` keywords.

```
In [63]: df = pd.DataFrame(np.random.rand(50, 4), columns=['a', 'b', 'c', 'd'])
```

```
In [64]: df.plot.scatter(x='a', y='b');
```

scatter plot

To plot multiple column groups in a single axes, repeat `plot` method specifying target `ax`. It is recommended to specify `color` and `label` keywords to distinguish each groups.

```
In [65]: ax = df.plot.scatter(x='a', y='b', color='DarkBlue', label='Group 1');
```

```
In [66]: df.plot.scatter(x='c', y='d', color='DarkGreen', label='Group 2', ax=ax);
```

scatter plot repeated



The keyword `c` may be given as the name of a column to provide colors for each point:

```
In [67]: df.plot.scatter(x='a', y='b', c='c', s=50);
```

scatter plot colored

You can pass other keywords supported by matplotlib [scatter](#). The example below shows a bubble chart using a column of the `DataFrame` as the bubble size.

```
In [68]: df.plot.scatter(x='a', y='b', s=df['c'] * 200);
```

scatter plot bubble

See the [scatter](#) method and the [matplotlib scatter documentation](#) for more.

## Hexagonal bin plot

You can create hexagonal bin plots with `DataFrame.plot.hexbin()`. Hexbin plots can be a useful alternative to scatter plots if your data are too dense to plot each point individually.

```
In [69]: df = pd.DataFrame(np.random.randn(1000, 2), columns=['a', 'b'])
```

```
In [70]: df['b'] = df['b'] + np.arange(1000)
```

```
In [71]: df.plot.hexbin(x='a', y='b', gridsize=25)
```

```
Out[71]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d96e4fd0>
```

hexbin\_plot

A useful keyword argument is `gridsize`; it controls the number of hexagons in the x-direction, and defaults to 100. A larger `gridsize` means more, smaller bins.

By default, a histogram of the counts around each `(x, y)` point is computed. You can specify alternative aggregations by passing values to the `C` and `reduce_C_function` arguments. `C` specifies the value at each `(x, y)` point and `reduce_C_function` is a function of one argument that reduces all the values in a bin to a single number (e.g. `mean`, `max`, `sum`, `std`). In this example the positions are given by columns `a` and `b`, while the value is given by column `z`. The bins are aggregated with NumPy's `max` function.

```
In [72]: df = pd.DataFrame(np.random.randn(1000, 2), columns=['a', 'b'])
```

```
In [73]: df['b'] = df['b'] = df['b'] + np.arange(1000)
```

```
In [74]: df['z'] = np.random.uniform(0, 3, 1000)
```

```
In [75]: df.plot.hexbin(x='a', y='b', C='z', reduce_C_function=np.max, gridsize=25)
```

```
Out[75]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d98ea390>
```

hexbin\_plot\_agg

See the [hexbin](#) method and the [matplotlib hexbin documentation](#) for more.

## Pie plot

You can create a pie plot with `DataFrame.plot.pie()` or `Series.plot.pie()`. If your data includes any `NaN`, they will be automatically filled with 0. A `ValueError` will be raised if there are any negative values in your data.

```
In [76]: series = pd.Series(3 * np.random.rand(4),
....: index=['a', 'b', 'c', 'd'], name='series')
....:

In [77]: series.plot.pie(figsize=(6, 6))
Out[77]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da5ff278>
```

series pie plot

For pie plots it's best to use square figures, i.e. a figure aspect ratio 1. You can create the figure with equal width and height, or force the aspect ratio to be equal after plotting by calling `ax.set_aspect('equal')` on the returned `axes` object.

Note that pie plot with `DataFrame` requires that you either specify a target column by the `y` argument or `subplots=True`. When `y` is specified, pie plot of selected column will be drawn. If `subplots=True` is specified, pie plots for each column are drawn as subplots. A legend will be drawn in each pie plots by default; specify `legend=False` to hide it.

```
In [78]: df = pd.DataFrame(3 * np.random.rand(4, 2),
....: index=['a', 'b', 'c', 'd'], columns=['x', 'y'])
....:

In [79]: df.plot.pie(subplots=True, figsize=(8, 4))
Out[79]:
array([<matplotlib.axes._subplots.AxesSubplot object at 0x7f65d915b0b8>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d9493a90>],
 dtype=object)
```

df pie plot

You can use the `labels` and `colors` keywords to specify the labels and colors of each wedge.

::: danger Warning

Most pandas plots use the `label` and `color` arguments (note the lack of “s” on those). To be consistent with `matplotlib.pyplot.pie()` you must use `labels` and `colors`.

:::

If you want to hide wedge labels, specify `labels=None`. If `fontsize` is specified, the value will be applied to wedge labels. Also, other keywords supported by `matplotlib.pyplot.pie()` can be used.

```
In [80]: series.plot.pie(labels=['AA', 'BB', 'CC', 'DD'], colors=['r', 'g', 'b', 'c'],
....: autopct='%2f', fontsize=20, figsize=(6, 6))
....:
Out[80]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da0be4a8>
```

series pie plot options

If you pass values whose sum total is less than 1.0, matplotlib draws a semicircle.

```
In [81]: series = pd.Series([0.1] * 4, index=['a', 'b', 'c', 'd'], name='series2')

In [82]: series.plot.pie(figsize=(6, 6))
Out[82]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d92543c8>
```

series pie plot semi

See the [matplotlib pie documentation](#) for more.

## Plotting with missing data

Pandas tries to be pragmatic about plotting `DataFrames` or `Series` that contain missing data. Missing values are dropped, left out, or filled depending on the plot type.

Plot Type	NaN Handling
Line	Leave gaps at NaNs
Line (stacked)	Fill 0's
Bar	Fill 0's
Scatter	Drop NaNs
Histogram	Drop NaNs (column-wise)
Box	Drop NaNs (column-wise)
Area	Fill 0's
KDE	Drop NaNs (column-wise)
Hexbin	Drop NaNs
Pie	Fill 0's

If any of these defaults are not what you want, or if you want to be explicit about how missing values are handled, consider using `fillna()` or `dropna()` before plotting.

## Plotting Tools

These functions can be imported from `pandas.plotting` and take a [Series](#) or [DataFrame](#) as an argument.

### Scatter matrix plot

You can create a scatter plot matrix using the `scatter_matrix` method in `pandas.plotting`:

```
In [83]: from pandas.plotting import scatter_matrix

In [84]: df = pd.DataFrame(np.random.randn(1000, 4), columns=['a', 'b', 'c', 'd'])

In [85]: scatter_matrix(df, alpha=0.2, figsize=(6, 6), diagonal='kde')
Out[85]:
array([[<matplotlib.axes._subplots.AxesSubplot object at 0x7f65dc209da0>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d9df9588>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d8fb4b38>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d9834128>],
 [<matplotlib.axes._subplots.AxesSubplot object at 0x7f65db04d6d8>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d8cdcc88>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d94e8278>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d9a67860>],
 [<matplotlib.axes._subplots.AxesSubplot object at 0x7f65d9a67898>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65da9f43c8>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65dacb7978>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65daddaf28>],
 [<matplotlib.axes._subplots.AxesSubplot object at 0x7f65dbe47518>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d9016ac8>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d99540b8>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65d9f84668>]],
 dtype=object)
```

`scatter_matrix_kde`

### Density plot

You can create density plots using the `Series.plot.kde()` and `DataFrame.plot.kde()` methods.

```
In [86]: ser = pd.Series(np.random.randn(1000))

In [87]: ser.plot.kde()
Out[87]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d909c828>
```

`kde_plot`

### Andrews curves

Andrews curves allow one to plot multivariate data as a large number of curves that are created using the attributes of samples as coefficients for Fourier series, see the [Wikipedia entry](#) for more

information. By coloring these curves differently for each class it is possible to visualize data clustering. Curves belonging to samples of the same class will usually be closer together and form larger structures.

**Note:** The “Iris” dataset is available [here](#).

```
In [88]: from pandas.plotting import andrews_curves

In [89]: data = pd.read_csv('data/iris.data')

In [90]: plt.figure()
Out[90]: <Figure size 640x480 with 0 Axes>

In [91]: andrews_curves(data, 'Name')
Out[91]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da6e5518>
```

andrews curves

## Parallel coordinates

Parallel coordinates is a plotting technique for plotting multivariate data, see the [Wikipedia entry](#) for an introduction. Parallel coordinates allows one to see clusters in data and to estimate other statistics visually. Using parallel coordinates points are represented as connected line segments. Each vertical line represents one attribute. One set of connected line segments represents one data point. Points that tend to cluster will appear closer together.

```
In [92]: from pandas.plotting import parallel_coordinates

In [93]: data = pd.read_csv('data/iris.data')

In [94]: plt.figure()
Out[94]: <Figure size 640x480 with 0 Axes>

In [95]: parallel_coordinates(data, 'Name')
Out[95]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d96fbc88>
```

parallel coordinates

## Lag plot

Lag plots are used to check if a data set or time series is random. Random data should not exhibit any structure in the lag plot. Non-random structure implies that the underlying data are not random. The `lag` argument may be passed, and when `lag=1` the plot is essentially `data[:-1]` vs. `data[1:]`.

```
In [96]: from pandas.plotting import lag_plot

In [97]: plt.figure()
Out[97]: <Figure size 640x480 with 0 Axes>

In [98]: spacing = np.linspace(-99 * np.pi, 99 * np.pi, num=1000)

In [99]: data = pd.Series(0.1 * np.random.rand(1000) + 0.9 * np.sin(spacing))

In [100]: lag_plot(data)
Out[100]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65da8b5e10>
```

lag\_plot

## Autocorrelation plot

Autocorrelation plots are often used for checking randomness in time series. This is done by computing autocorrelations for data values at varying time lags. If time series is random, such autocorrelations should be near zero for any and all time-lag separations. If time series is non-random then one or more of the autocorrelations will be significantly non-zero. The horizontal lines displayed in the plot correspond to 95% and 99% confidence bands. The dashed line is 99% confidence band. See the [Wikipedia entry](#) for more about autocorrelation plots.

```
In [101]: from pandas.plotting import autocorrelation_plot

In [102]: plt.figure()
Out[102]: <Figure size 640x480 with 0 Axes>

In [103]: spacing = np.linspace(-9 * np.pi, 9 * np.pi, num=1000)

In [104]: data = pd.Series(0.7 * np.random.rand(1000) + 0.3 * np.sin(spacing))

In [105]: autocorrelation_plot(data)
Out[105]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d19556d8>
```

autocorrelation\_plot

## Bootstrap plot

Bootstrap plots are used to visually assess the uncertainty of a statistic, such as mean, median, midrange, etc. A random subset of a specified size is selected from a data set, the statistic in question is computed for this subset and the process is repeated a specified number of times. Resulting plots and histograms are what constitutes the bootstrap plot.

```
In [106]: from pandas.plotting import bootstrap_plot

In [107]: data = pd.Series(np.random.rand(1000))

In [108]: bootstrap_plot(data, size=50, samples=500, color='grey')
Out[108]: <Figure size 640x480 with 6 Axes>
```

bootstrap\_plot

## RadViz

RadViz is a way of visualizing multi-variate data. It is based on a simple spring tension minimization algorithm. Basically you set up a bunch of points in a plane. In our case they are equally spaced on a unit circle. Each point represents a single attribute. You then pretend that each sample in the data set is attached to each of these points by a spring, the stiffness of which is proportional to the numerical value of that attribute (they are normalized to unit interval). The point in the plane, where our sample settles to (where the forces acting on our sample are at an equilibrium) is where a dot representing our sample will be drawn. Depending on which class that sample belongs to it will be colored differently. See the R package [Radviz](#) for more information.

**Note:** The “Iris” dataset is available [here](#).

```
In [109]: from pandas.plotting import radviz

In [110]: data = pd.read_csv('data/iris.data')

In [111]: plt.figure()
Out[111]: <Figure size 640x480 with 0 Axes>

In [112]: radviz(data, 'Name')
Out[112]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f66bd630>
```

radviz

## Plot Formatting

### Setting the plot style

From version 1.5 and up, matplotlib offers a range of pre-configured plotting styles. Setting the style can be used to easily give plots the general look that you want. Setting the style is as easy as calling `matplotlib.style.use(my_plot_style)` before creating your plot. For example you could write `matplotlib.style.use('ggplot')` for ggplot-style plots.

You can see the various available style names at `matplotlib.style.available` and it's very easy to try them out.

## General plot style arguments

Most plotting methods have a set of keyword arguments that control the layout and formatting of the returned plot:

```
In [113]: plt.figure();

In [114]: ts.plot(style='k--', label='Series');
```

series plot basic2

For each kind of plot (e.g. line, bar, scatter) any additional arguments keywords are passed along to the corresponding matplotlib function (`ax.plot()`, `ax.bar()`, `ax.scatter()`). These can be used to control additional styling, beyond what pandas provides.

## Controlling the legend

You may set the `legend` argument to `False` to hide the legend, which is shown by default.

```
In [115]: df = pd.DataFrame(np.random.randn(1000, 4),
.....: index=ts.index, columns=list('ABCD'))
.....:

In [116]: df = df.cumsum()

In [117]: df.plot(legend=False)
Out[117]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65dbdbc0f0>
```

frame plot basic noleg

## Scales

You may pass `logy` to get a log-scale Y axis.

```
In [118]: ts = pd.Series(np.random.randn(1000),
.....: index=pd.date_range('1/1/2000', periods=1000))
.....:

In [119]: ts = np.exp(ts.cumsum())

In [120]: ts.plot(logy=True)
Out[120]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65dbefdf98>
```

series plot logy

See also the `logx` and `loglog` keyword arguments.



## Plotting on a secondary y-axis

To plot data on a secondary y-axis, use the `secondary_y` keyword:

```
In [121]: df.A.plot()
Out[121]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f8ef6b00>

In [122]: df.B.plot(secondary_y=True, style='g')
Out[122]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f6297780>
```

series plot secondary y

To plot some columns in a `DataFrame`, give the column names to the `secondary_y` keyword:

```
In [123]: plt.figure()
Out[123]: <Figure size 640x480 with 0 Axes>

In [124]: ax = df.plot(secondary_y=['A', 'B'])

In [125]: ax.set_ylabel('CD scale')
Out[125]: Text(0, 0.5, 'CD scale')

In [126]: ax.right_ax.set_ylabel('AB scale')
Out[126]: Text(0, 0.5, 'AB scale')
```

frame plot secondary y

Note that the columns plotted on the secondary y-axis is automatically marked with “(right)” in the legend. To turn off the automatic marking, use the `mark_right=False` keyword:

```
In [127]: plt.figure()
Out[127]: <Figure size 640x480 with 0 Axes>

In [128]: df.plot(secondary_y=['A', 'B'], mark_right=False)
Out[128]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f6102390>
```

frame plot secondary y no right

## Suppressing tick resolution adjustment

pandas includes automatic tick resolution adjustment for regular frequency time-series data. For limited cases where pandas cannot infer the frequency information (e.g., in an externally created `twinx`), you can choose to suppress this behavior for alignment purposes.

Here is the default behavior, notice how the x-axis tick labeling is performed:

```
In [129]: plt.figure()
Out[129]: <Figure size 640x480 with 0 Axes>

In [130]: df.A.plot()
Out[130]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65dc39f978>
```

ser plot suppress

Using the `x_compat` parameter, you can suppress this behavior:

```
In [131]: plt.figure()
Out[131]: <Figure size 640x480 with 0 Axes>

In [132]: df.A.plot(x_compat=True)
Out[132]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65dc39f1d0>
```

ser plot suppress parm

If you have more than one plot that needs to be suppressed, the `use` method in `pandas.plotting.plot_params` can be used in a with statement:

```
In [133]: plt.figure()
Out[133]: <Figure size 640x480 with 0 Axes>

In [134]: with pd.plotting.plot_params.use('x_compat', True):
.....: df.A.plot(color='r')
.....: df.B.plot(color='g')
.....: df.C.plot(color='b')
.....:
```

ser\_plot\_suppress\_context

## Automatic date tick adjustment

New in version 0.20.0.

`TimedeltaIndex` now uses the native matplotlib tick locator methods, it is useful to call the automatic date tick adjustment from matplotlib for figures whose ticklabels overlap.

See the `autofmt_xdate` method and the [matplotlib documentation](#) for more.

## Subplots

Each `Series` in a `DataFrame` can be plotted on a different axis with the `subplots` keyword:

```
In [135]: df.plot(subplots=True, figsize=(6, 6));
```

frame\_plot\_subplots

## Using layout and targeting multiple axes

The layout of subplots can be specified by the `layout` keyword. It can accept `(rows, columns)`. The `layout` keyword can be used in `hist` and `boxplot` also. If the input is invalid, a `ValueError` will be raised.

The number of axes which can be contained by rows x columns specified by `layout` must be larger than the number of required subplots. If layout can contain more axes than required, blank axes are not drawn. Similar to a NumPy array's `reshape` method, you can use `-1` for one dimension to automatically calculate the number of rows or columns needed, given the other.

```
In [136]: df.plot(subplots=True, layout=(2, 3), figsize=(6, 6), sharex=False);
```

frame plot subplots layout

The above example is identical to using:

```
In [137]: df.plot(subplots=True, layout=(2, -1), figsize=(6, 6), sharex=False);
```

The required number of columns (3) is inferred from the number of series to plot and the given number of rows (2).

You can pass multiple axes created beforehand as list-like via `ax` keyword. This allows more complicated layouts. The passed axes must be the same number as the subplots being drawn.

When multiple axes are passed via the `ax` keyword, `layout`, `sharex` and `sharey` keywords don't affect to the output. You should explicitly pass `sharex=False` and `sharey=False`, otherwise you will see a warning.

```
In [138]: fig, axes = plt.subplots(4, 4, figsize=(6, 6))

In [139]: plt.subplots_adjust(wspace=0.5, hspace=0.5)

In [140]: target1 = [axes[0][0], axes[1][1], axes[2][2], axes[3][3]]

In [141]: target2 = [axes[3][0], axes[2][1], axes[1][2], axes[0][3]]

In [142]: df.plot(subplots=True, ax=target1, legend=False, sharex=False, sharey=False);

In [143]: (-df).plot(subplots=True, ax=target2, legend=False,
.....: sharex=False, sharey=False);
.....:
```

frame plot subplots multi ax

Another option is passing an `ax` argument to `Series.plot()` to plot on a particular axis:

```

In [144]: fig, axes = plt.subplots(nrows=2, ncols=2)

In [145]: df['A'].plot(ax=axes[0, 0]);

In [146]: axes[0, 0].set_title('A');

In [147]: df['B'].plot(ax=axes[0, 1]);

In [148]: axes[0, 1].set_title('B');

In [149]: df['C'].plot(ax=axes[1, 0]);

In [150]: axes[1, 0].set_title('C');

In [151]: df['D'].plot(ax=axes[1, 1]);

In [152]: axes[1, 1].set_title('D');

```

series\_plot\_multi

## Plotting with error bars

Plotting with error bars is supported in `DataFrame.plot()` and `Series.plot()`.

Horizontal and vertical error bars can be supplied to the `xerr` and `yerr` keyword arguments to `plot()`. The error values can be specified using a variety of formats:

- As a `DataFrame` or `dict` of errors with column names matching the `columns` attribute of the plotting `DataFrame` or matching the `name` attribute of the `Series`.
- As a `str` indicating which of the columns of plotting `DataFrame` contain the error values.
- As raw values ( `list`, `tuple`, or `np.ndarray` ). Must be the same length as the plotting `DataFrame`/`Series`.

Asymmetrical error bars are also supported, however raw error values must be provided in this case. For a `M` length `Series`, a `Mx2` array should be provided indicating lower and upper (or left and right) errors. For a `MxN` `DataFrame`, asymmetrical errors should be in a `Mx2xN` array.

Here is an example of one way to easily plot group means with standard deviations from the raw data.

```

Generate the data
In [153]: ix3 = pd.MultiIndex.from_arrays([
.....: ['a', 'a', 'a', 'a', 'b', 'b', 'b', 'b'],
.....: ['foo', 'foo', 'bar', 'bar', 'foo', 'foo', 'bar', 'bar']],
.....: names=['letter', 'word'])
.....:

In [154]: df3 = pd.DataFrame({'data1': [3, 2, 4, 3, 2, 4, 3, 2],
.....: 'data2': [6, 5, 7, 5, 4, 5, 6, 5]}, index=ix3)
.....:

Group by index labels and take the means and standard deviations
for each group
In [155]: gp3 = df3.groupby(level=('letter', 'word'))

In [156]: means = gp3.mean()

In [157]: errors = gp3.std()

In [158]: means
Out[158]:
 data1 data2
letter word
a bar 3.5 6.0
 foo 2.5 5.5
b bar 2.5 5.5
 foo 3.0 4.5

In [159]: errors
Out[159]:
 data1 data2
letter word
a bar 0.707107 1.414214
 foo 0.707107 0.707107
b bar 0.707107 0.707107
 foo 1.414214 0.707107

Plot
In [160]: fig, ax = plt.subplots()

In [161]: means.plot.bar(yerr=errors, ax=ax, capsize=4)
Out[161]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d1048400>

```

errorbar example

## Plotting tables

Plotting with matplotlib table is now supported in `DataFrame.plot()` and `Series.plot()` with a `table` keyword. The `table` keyword can accept `bool`, `DataFrame` or `Series`. The simple way to draw a table is to specify `table=True`. Data will be transposed to meet matplotlib's default layout.

```
In [162]: fig, ax = plt.subplots(1, 1)

In [163]: df = pd.DataFrame(np.random.rand(5, 3), columns=['a', 'b', 'c'])

In [164]: ax.get_xaxis().set_visible(False) # Hide Ticks

In [165]: df.plot(table=True, ax=ax)
Out[165]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d0ff7550>
```

line plot table true

Also, you can pass a different `DataFrame` or `Series` to the `table` keyword. The data will be drawn as displayed in print method (not transposed automatically). If required, it should be transposed manually as seen in the example below.

```
In [166]: fig, ax = plt.subplots(1, 1)

In [167]: ax.get_xaxis().set_visible(False) # Hide Ticks

In [168]: df.plot(table=np.round(df.T, 2), ax=ax)
Out[168]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d0efdcc0>
```

line plot table data

There also exists a helper function `pandas.plotting.table`, which creates a table from `DataFrame` or `Series`, and adds it to an `matplotlib.Axes` instance. This function can accept keywords which the `matplotlib table` has.

```
In [169]: from pandas.plotting import table

In [170]: fig, ax = plt.subplots(1, 1)

In [171]: table(ax, np.round(df.describe(), 2),
.....: loc='upper right', colWidths=[0.2, 0.2, 0.2])
.....:
Out[171]: <matplotlib.table.Table at 0x7f65d0e61b38>

In [172]: df.plot(ax=ax, ylim=(0, 2), legend=None)
Out[172]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d0eab358>
```

line plot table describe

**Note:** You can get table instances on the axes using `axes.tables` property for further decorations. See the [matplotlib table documentation](#) for more.

## Colormaps

A potential issue when plotting a large number of columns is that it can be difficult to distinguish some series due to repetition in the default colors. To remedy this, `DataFrame` plotting supports the

use of the `colormap` argument, which accepts either a Matplotlib [colormap](#) or a string that is a name of a colormap registered with Matplotlib. A visualization of the default matplotlib colormaps is available [here](#).

As matplotlib does not directly support colormaps for line-based plots, the colors are selected based on an even spacing determined by the number of columns in the `DataFrame`. There is no consideration made for background color, so some colormaps will produce lines that are not easily visible.

To use the `cubehelix` colormap, we can pass `colormap='cubehelix'`.

```
In [173]: df = pd.DataFrame(np.random.randn(1000, 10), index=ts.index)

In [174]: df = df.cumsum()

In [175]: plt.figure()
Out[175]: <Figure size 640x480 with 0 Axes>

In [176]: df.plot(colormap='cubehelix')
Out[176]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d0defdd8>
```

`cubehelix`

Alternatively, we can pass the colormap itself:

```
In [177]: from matplotlib import cm

In [178]: plt.figure()
Out[178]: <Figure size 640x480 with 0 Axes>

In [179]: df.plot(colormap=cm.cubehelix)
Out[179]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d0c22a90>
```

`cubehelix cm`

Colormaps can also be used other plot types, like bar charts:

```
In [180]: dd = pd.DataFrame(np.random.randn(10, 10)).applymap(abs)

In [181]: dd = dd.cumsum()

In [182]: plt.figure()
Out[182]: <Figure size 640x480 with 0 Axes>

In [183]: dd.plot.bar(colormap='Greens')
Out[183]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d0a5de10>
```

`greens`

Parallel coordinates charts:

```
In [184]: plt.figure()
Out[184]: <Figure size 640x480 with 0 Axes>

In [185]: parallel_coordinates(data, 'Name', colormap='gist_rainbow')
Out[185]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d08e5eb8>
```

parallel gist rainbow

Andrews curves charts:

```
In [186]: plt.figure()
Out[186]: <Figure size 640x480 with 0 Axes>

In [187]: andrews_curves(data, 'Name', colormap='winter')
Out[187]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65d06b6518>
```

andrews curve winter

## Plotting directly with matplotlib

---

In some situations it may still be preferable or necessary to prepare plots directly with matplotlib, for instance when a certain type of plot or customization is not (yet) supported by pandas. `Series` and `DataFrame` objects behave like arrays and can therefore be passed directly to matplotlib functions without explicit casts.

pandas also automatically registers formatters and locators that recognize date indices, thereby extending date and time support to practically all plot types available in matplotlib. Although this formatting does not provide the same level of refinement you would get when plotting via pandas, it can be faster when plotting a large number of points.



```

In [188]: price = pd.Series(np.random.randn(150).cumsum(),
.....: index=pd.date_range('2000-1-1', periods=150, freq='B'))
.....:

In [189]: ma = price.rolling(20).mean()

In [190]: mstd = price.rolling(20).std()

In [191]: plt.figure()
Out[191]: <Figure size 640x480 with 0 Axes>

In [192]: plt.plot(price.index, price, 'k')
Out[192]: [<matplotlib.lines.Line2D at 0x7f65da5f8710>]

In [193]: plt.plot(ma.index, ma, 'b')
Out[193]: [<matplotlib.lines.Line2D at 0x7f65d9ab9518>]

In [194]: plt.fill_between(mstd.index, ma - 2 * mstd, ma + 2 * mstd,
.....: color='b', alpha=0.2)
.....:
Out[194]: <matplotlib.collections.PolyCollection at 0x7f65d9ab9128>

```

bollinger

## Trellis plotting interface

---

::: danger Warning

The `rplot` trellis plotting interface has been **removed**. Please use external packages like [seaborn](#) for similar but more refined functionality and refer to our 0.18.1 documentation [here](#) for how to convert to using it.

:::

---

# Computational tools

## Statistical functions

---

### Percent change

`Series` and `DataFrame` have a method `pct_change()` to compute the percent change over a given number of periods (using `fill_method` to fill NA/null values before computing the percent change).

```
In [1]: ser = pd.Series(np.random.randn(8))
```

```
In [2]: ser.pct_change()
```

```
Out[2]:
```

```
0 NaN
1 -1.602976
2 4.334938
3 -0.247456
4 -2.067345
5 -1.142903
6 -1.688214
7 -9.759729
dtype: float64
```

```
In [3]: df = pd.DataFrame(np.random.randn(10, 4))
```

```
In [4]: df.pct_change(periods=3)
```

```
Out[4]:
```

```
 0 1 2 3
0 NaN NaN NaN NaN
1 NaN NaN NaN NaN
2 NaN NaN NaN NaN
3 -0.218320 -1.054001 1.987147 -0.510183
4 -0.439121 -1.816454 0.649715 -4.822809
5 -0.127833 -3.042065 -5.866604 -1.776977
6 -2.596833 -1.959538 -2.111697 -3.798900
7 -0.117826 -2.169058 0.036094 -0.067696
8 2.492606 -1.357320 -1.205802 -1.558697
9 -1.012977 2.324558 -1.003744 -0.371806
```

## Covariance

`Series.cov()` can be used to compute covariance between series (excluding missing values).

```
In [5]: s1 = pd.Series(np.random.randn(1000))
```

```
In [6]: s2 = pd.Series(np.random.randn(1000))
```

```
In [7]: s1.cov(s2)
```

```
Out[7]: 0.000680108817431082
```

Analogously, `DataFrame.cov()` to compute pairwise covariances among the series in the DataFrame, also excluding NA/null values.

::: tip Note

Assuming the missing data are missing at random this results in an estimate for the covariance matrix which is unbiased. However, for many applications this estimate may not be acceptable because the estimated covariance matrix is not guaranteed to be positive semi-definite. This could lead to

estimated correlations having absolute values which are greater than one, and/or a non-invertible covariance matrix. See [Estimation of covariance matrices](#) for more details.

...

```
In [8]: frame = pd.DataFrame(np.random.randn(1000, 5),
...: columns=['a', 'b', 'c', 'd', 'e'])
...:
```

```
In [9]: frame.cov()
```

```
Out[9]:
```

	a	b	c	d	e
a	1.000882	-0.003177	-0.002698	-0.006889	0.031912
b	-0.003177	1.024721	0.000191	0.009212	0.000857
c	-0.002698	0.000191	0.950735	-0.031743	-0.005087
d	-0.006889	0.009212	-0.031743	1.002983	-0.047952
e	0.031912	0.000857	-0.005087	-0.047952	1.042487

`DataFrame.cov` also supports an optional `min_periods` keyword that specifies the required minimum number of observations for each column pair in order to have a valid result.

```
In [10]: frame = pd.DataFrame(np.random.randn(20, 3), columns=['a', 'b', 'c'])
```

```
In [11]: frame.loc[frame.index[:5], 'a'] = np.nan
```

```
In [12]: frame.loc[frame.index[5:10], 'b'] = np.nan
```

```
In [13]: frame.cov()
```

```
Out[13]:
```

	a	b	c
a	1.123670	-0.412851	0.018169
b	-0.412851	1.154141	0.305260
c	0.018169	0.305260	1.301149

```
In [14]: frame.cov(min_periods=12)
```

```
Out[14]:
```

	a	b	c
a	1.123670	NaN	0.018169
b	NaN	1.154141	0.305260
c	0.018169	0.305260	1.301149

## Correlation

Correlation may be computed using the `corr()` method. Using the `method` parameter, several methods for computing correlations are provided:

Method name	Description
pearson (default)	Standard correlation coefficient
kendall	Kendall Tau correlation coefficient
spearman	Spearman rank correlation coefficient

All of these are currently computed using pairwise complete observations. Wikipedia has articles covering the above correlation coefficients:

- [Pearson correlation coefficient](#)
- [Kendall rank correlation coefficient](#)
- [Spearman's rank correlation coefficient](#)

::: tip Note

Please see the [caveats](#) associated with this method of calculating correlation matrices in the [covariance section](#).

:::

```
In [15]: frame = pd.DataFrame(np.random.randn(1000, 5),
....: columns=['a', 'b', 'c', 'd', 'e'])
....:
```

```
In [16]: frame.iloc[:,2] = np.nan
```

```
Series with Series
```

```
In [17]: frame['a'].corr(frame['b'])
Out[17]: 0.01347904040098794
```

```
In [18]: frame['a'].corr(frame['b'], method='spearman')
Out[18]: -0.007289885159540637
```

```
Pairwise correlation of DataFrame columns
```

```
In [19]: frame.corr()
Out[19]:
```

```

 a b c d e
a 1.000000 0.013479 -0.049269 -0.042239 -0.028525
b 0.013479 1.000000 -0.020433 -0.011139 0.005654
c -0.049269 -0.020433 1.000000 0.018587 -0.054269
d -0.042239 -0.011139 0.018587 1.000000 -0.017060
e -0.028525 0.005654 -0.054269 -0.017060 1.000000
```

Note that non-numeric columns will be automatically excluded from the correlation calculation.

Like `cov`, `corr` also supports the optional `min_periods` keyword:

```
In [20]: frame = pd.DataFrame(np.random.randn(20, 3), columns=['a', 'b', 'c'])
```

```
In [21]: frame.loc[frame.index[:5], 'a'] = np.nan
```

```
In [22]: frame.loc[frame.index[5:10], 'b'] = np.nan
```

```
In [23]: frame.corr()
```

```
Out[23]:
```

	a	b	c
a	1.000000	-0.121111	0.069544
b	-0.121111	1.000000	0.051742
c	0.069544	0.051742	1.000000

```
In [24]: frame.corr(min_periods=12)
```

```
Out[24]:
```

	a	b	c
a	1.000000	NaN	0.069544
b	NaN	1.000000	0.051742
c	0.069544	0.051742	1.000000

New in version 0.24.0.

The `method` argument can also be a callable for a generic correlation calculation. In this case, it should be a single function that produces a single value from two ndarray inputs. Suppose we wanted to compute the correlation based on histogram intersection:

```
histogram intersection
```

```
In [25]: def histogram_intersection(a, b):
```

```
.....: return np.minimum(np.true_divide(a, a.sum()),
.....: np.true_divide(b, b.sum())).sum()
.....:
```

```
In [26]: frame.corr(method=histogram_intersection)
```

```
Out[26]:
```

	a	b	c
a	1.000000	-6.404882	-2.058431
b	-6.404882	1.000000	-19.255743
c	-2.058431	-19.255743	1.000000

A related method `corrwith()` is implemented on `DataFrame` to compute the correlation between like-labeled Series contained in different `DataFrame` objects.

```

In [27]: index = ['a', 'b', 'c', 'd', 'e']

In [28]: columns = ['one', 'two', 'three', 'four']

In [29]: df1 = pd.DataFrame(np.random.randn(5, 4), index=index, columns=columns)

In [30]: df2 = pd.DataFrame(np.random.randn(4, 4), index=index[:4], columns=columns)

In [31]: df1.corrwith(df2)
Out[31]:
one -0.125501
two -0.493244
three 0.344056
four 0.004183
dtype: float64

In [32]: df2.corrwith(df1, axis=1)
Out[32]:
a -0.675817
b 0.458296
c 0.190809
d -0.186275
e NaN
dtype: float64

```

## Data ranking

The `rank()` method produces a data ranking with ties being assigned the mean of the ranks (by default) for the group:

```

In [33]: s = pd.Series(np.random.randn(5), index=list('abcde'))

In [34]: s['d'] = s['b'] # so there's a tie

In [35]: s.rank()
Out[35]:
a 5.0
b 2.5
c 1.0
d 2.5
e 4.0
dtype: float64

```

`rank()` is also a DataFrame method and can rank either the rows (`axis=0`) or the columns (`axis=1`). NaN values are excluded from the ranking.

```
In [36]: df = pd.DataFrame(np.random.randn(10, 6))

In [37]: df[4] = df[2][:5] # some ties

In [38]: df
Out[38]:
```

	0	1	2	3	4	5
0	-0.904948	-1.163537	-1.457187	0.135463	-1.457187	0.294650
1	-0.976288	-0.244652	-0.748406	-0.999601	-0.748406	-0.800809
2	0.401965	1.460840	1.256057	1.308127	1.256057	0.876004
3	0.205954	0.369552	-0.669304	0.038378	-0.669304	1.140296
4	-0.477586	-0.730705	-1.129149	-0.601463	-1.129149	-0.211196
5	-1.092970	-0.689246	0.908114	0.204848	NaN	0.463347
6	0.376892	0.959292	0.095572	-0.593740	NaN	-0.069180
7	-1.002601	1.957794	-0.120708	0.094214	NaN	-1.467422
8	-0.547231	0.664402	-0.519424	-0.073254	NaN	-1.263544
9	-0.250277	-0.237428	-1.056443	0.419477	NaN	1.375064

```
In [39]: df.rank(1)
Out[39]:
```

	0	1	2	3	4	5
0	4.0	3.0	1.5	5.0	1.5	6.0
1	2.0	6.0	4.5	1.0	4.5	3.0
2	1.0	6.0	3.5	5.0	3.5	2.0
3	4.0	5.0	1.5	3.0	1.5	6.0
4	5.0	3.0	1.5	4.0	1.5	6.0
5	1.0	2.0	5.0	3.0	NaN	4.0
6	4.0	5.0	3.0	1.0	NaN	2.0
7	2.0	5.0	3.0	4.0	NaN	1.0
8	2.0	5.0	3.0	4.0	NaN	1.0
9	2.0	3.0	1.0	4.0	NaN	5.0

`rank` optionally takes a parameter `ascending` which by default is true; when false, data is reverse-ranked, with larger values assigned a smaller rank.

`rank` supports different tie-breaking methods, specified with the `method` parameter:

- `average` : average rank of tied group
- `min` : lowest rank in the group
- `max` : highest rank in the group
- `first` : ranks assigned in the order they appear in the array

## Window Functions

For working with data, a number of window functions are provided for computing common window or rolling statistics. Among these are count, sum, mean, median, correlation, variance, covariance, standard deviation, skewness, and kurtosis.

The `rolling()` and `expanding()` functions can be used directly from `DataFrameGroupBy` objects, see the [groupby docs](#).

::: tip Note

The API for window statistics is quite similar to the way one works with `GroupBy` objects, see the documentation [here](#).

:::

We work with `rolling`, `expanding` and `exponentially weighted` data through the corresponding objects, `Rolling`, `Expanding` and `EWM`.

```
In [40]: s = pd.Series(np.random.randn(1000),
....: index=pd.date_range('1/1/2000', periods=1000))
....:

In [41]: s = s.cumsum()

In [42]: s
Out[42]:
2000-01-01 -0.268824
2000-01-02 -1.771855
2000-01-03 -0.818003
2000-01-04 -0.659244
2000-01-05 -1.942133
...
2002-09-22 -67.457323
2002-09-23 -69.253182
2002-09-24 -70.296818
2002-09-25 -70.844674
2002-09-26 -72.475016
Freq: D, Length: 1000, dtype: float64
```

These are created from methods on `Series` and `DataFrame`.

```
In [43]: r = s.rolling(window=60)

In [44]: r
Out[44]: Rolling [window=60,center=False,axis=0]
```

These object provide tab-completion of the available methods and properties.

```
In [14]: r.<TAB> # noqa: E225, E999
r.agg r.apply r.count r.exclusions r.max r.median r.name
 r.skew r.sum
r.aggregate r.corr r.cov r.kurt r.mean r.min r.quantile
 r.std r.var
```

Generally these methods all have the same interface. They all accept the following arguments:

- `window`: size of moving window
- `min_periods`: threshold of non-null data points to require (otherwise result is NA)
- `center`: boolean, whether to set the labels at the center (default is False)



We can then call methods on these `rolling` objects. These return like-indexed objects:

```
In [45]: r.mean()
Out[45]:
2000-01-01 NaN
2000-01-02 NaN
2000-01-03 NaN
2000-01-04 NaN
2000-01-05 NaN
...
2002-09-22 -62.914971
2002-09-23 -63.061867
2002-09-24 -63.213876
2002-09-25 -63.375074
2002-09-26 -63.539734
Freq: D, Length: 1000, dtype: float64
```

```
In [46]: s.plot(style='k--')
Out[46]: <matplotlib.axes._subplots.AxesSubplot at 0x7f66048bdef0>

In [47]: r.mean().plot(style='k')
Out[47]: <matplotlib.axes._subplots.AxesSubplot at 0x7f66048bdef0>
```

`rolling_mean_ex`

They can also be applied to DataFrame objects. This is really just syntactic sugar for applying the moving window operator to all of the DataFrame's columns:

```
In [48]: df = pd.DataFrame(np.random.randn(1000, 4),
.....: index=pd.date_range('1/1/2000', periods=1000),
.....: columns=['A', 'B', 'C', 'D'])
.....:

In [49]: df = df.cumsum()

In [50]: df.rolling(window=60).sum().plot(subplots=True)
Out[50]:
array([<matplotlib.axes._subplots.AxesSubplot object at 0x7f66075a7f60>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65f79e55c0>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65f7998588>,
 <matplotlib.axes._subplots.AxesSubplot object at 0x7f65f794b550>],
 dtype=object)
```

`rolling_mean_frame`

## Method summary

We provide a number of common statistical functions:

Method	Description
<code>count()</code>	Number of non-null observations
<code>sum()</code>	Sum of values
<code>mean()</code>	Mean of values
<code>median()</code>	Arithmetic median of values
<code>min()</code>	Minimum
<code>max()</code>	Maximum
<code>std()</code>	Bessel-corrected sample standard deviation
<code>var()</code>	Unbiased variance
<code>skew()</code>	Sample skewness (3rd moment)
<code>kurt()</code>	Sample kurtosis (4th moment)
<code>quantile()</code>	Sample quantile (value at %)
<code>apply()</code>	Generic apply
<code>cov()</code>	Unbiased covariance (binary)
<code>corr()</code>	Correlation (binary)

The `apply()` function takes an extra `func` argument and performs generic rolling computations. The `func` argument should be a single function that produces a single value from an ndarray input. Suppose we wanted to compute the mean absolute deviation on a rolling basis:

```
In [51]: def mad(x):
...: return np.fabs(x - x.mean()).mean()
...:

In [52]: s.rolling(window=60).apply(mad, raw=True).plot(style='k')
Out[52]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f795fac8>
```

rolling\_apply\_ex

## Rolling windows

Passing `win_type` to `.rolling` generates a generic rolling window computation, that is weighted according to the `win_type`. The following methods are available:

Method	Description
<code>sum()</code>	Sum of values
<code>mean()</code>	Mean of values

The weights used in the window are specified by the `win_type` keyword. The list of recognized types are the [scipy.signal window functions](#):

- `boxcar`
- `triang`
- `blackman`
- `hamming`
- `bartlett`
- `parzen`
- `bohman`
- `blackmanharris`
- `nuttall`
- `barthann`
- `kaiser` (needs beta)
- `gaussian` (needs std)
- `general_gaussian` (needs power, width)
- `slepian` (needs width)
- `exponential` (needs tau).

```
In [53]: ser = pd.Series(np.random.randn(10),
....: index=pd.date_range('1/1/2000', periods=10))
....:
```

```
In [54]: ser.rolling(window=5, win_type='triang').mean()
```

```
Out[54]:
```

```
2000-01-01 NaN
2000-01-02 NaN
2000-01-03 NaN
2000-01-04 NaN
2000-01-05 -1.037870
2000-01-06 -0.767705
2000-01-07 -0.383197
2000-01-08 -0.395513
2000-01-09 -0.558440
2000-01-10 -0.672416
```

```
Freq: D, dtype: float64
```

Note that the `boxcar` window is equivalent to `mean()`.

```
In [55]: ser.rolling(window=5, win_type='boxcar').mean()
Out[55]:
2000-01-01 NaN
2000-01-02 NaN
2000-01-03 NaN
2000-01-04 NaN
2000-01-05 -0.841164
2000-01-06 -0.779948
2000-01-07 -0.565487
2000-01-08 -0.502815
2000-01-09 -0.553755
2000-01-10 -0.472211
Freq: D, dtype: float64
```

```
In [56]: ser.rolling(window=5).mean()
Out[56]:
2000-01-01 NaN
2000-01-02 NaN
2000-01-03 NaN
2000-01-04 NaN
2000-01-05 -0.841164
2000-01-06 -0.779948
2000-01-07 -0.565487
2000-01-08 -0.502815
2000-01-09 -0.553755
2000-01-10 -0.472211
Freq: D, dtype: float64
```

For some windowing functions, additional parameters must be specified:

```
In [57]: ser.rolling(window=5, win_type='gaussian').mean(std=0.1)
Out[57]:
2000-01-01 NaN
2000-01-02 NaN
2000-01-03 NaN
2000-01-04 NaN
2000-01-05 -1.309989
2000-01-06 -1.153000
2000-01-07 0.606382
2000-01-08 -0.681101
2000-01-09 -0.289724
2000-01-10 -0.996632
Freq: D, dtype: float64
```

### ::: tip Note

For `.sum()` with a `win_type`, there is no normalization done to the weights for the window. Passing custom weights of `[1, 1, 1]` will yield a different result than passing weights of `[2, 2, 2]`, for example. When passing a `win_type` instead of explicitly specifying the weights, the weights are already normalized so that the largest weight is 1.

In contrast, the nature of the `.mean()` calculation is such that the weights are normalized with respect to each other. Weights of `[1, 1, 1]` and `[2, 2, 2]` yield the same result.

...

## Time-aware rolling

New in version 0.19.0.

New in version 0.19.0 are the ability to pass an offset (or convertible) to a `.rolling()` method and have it produce variable sized windows based on the passed time window. For each time point, this includes all preceding values occurring within the indicated time delta.

This can be particularly useful for a non-regular time frequency index.

```
In [58]: dft = pd.DataFrame({'B': [0, 1, 2, np.nan, 4]},
....: index=pd.date_range('20130101 09:00:00',
....: periods=5,
....: freq='s'))
....:
```

```
In [59]: dft
```

```
Out[59]:
```

	B
2013-01-01 09:00:00	0.0
2013-01-01 09:00:01	1.0
2013-01-01 09:00:02	2.0
2013-01-01 09:00:03	NaN
2013-01-01 09:00:04	4.0

This is a regular frequency index. Using an integer window parameter works to roll along the window frequency.

```
In [60]: dft.rolling(2).sum()
Out[60]:
```

	B
2013-01-01 09:00:00	NaN
2013-01-01 09:00:01	1.0
2013-01-01 09:00:02	3.0
2013-01-01 09:00:03	NaN
2013-01-01 09:00:04	NaN

```
In [61]: dft.rolling(2, min_periods=1).sum()
```

```
Out[61]:
```

	B
2013-01-01 09:00:00	0.0
2013-01-01 09:00:01	1.0
2013-01-01 09:00:02	3.0
2013-01-01 09:00:03	2.0
2013-01-01 09:00:04	4.0

Specifying an offset allows a more intuitive specification of the rolling frequency.

```
In [62]: dft.rolling('2s').sum()
Out[62]:
```

	B
2013-01-01 09:00:00	0.0
2013-01-01 09:00:01	1.0
2013-01-01 09:00:02	3.0
2013-01-01 09:00:03	2.0
2013-01-01 09:00:04	4.0

Using a non-regular, but still monotonic index, rolling with an integer window does not impart any special calculation.

```
In [63]: dft = pd.DataFrame({'B': [0, 1, 2, np.nan, 4]},
....: index=pd.Index([pd.Timestamp('20130101 09:00:00'),
....: pd.Timestamp('20130101 09:00:02'),
....: pd.Timestamp('20130101 09:00:03'),
....: pd.Timestamp('20130101 09:00:05'),
....: pd.Timestamp('20130101 09:00:06')],
....: name='foo'))
....:
```

```
In [64]: dft
Out[64]:
```

	B
foo	
2013-01-01 09:00:00	0.0
2013-01-01 09:00:02	1.0
2013-01-01 09:00:03	2.0
2013-01-01 09:00:05	NaN
2013-01-01 09:00:06	4.0

```
In [65]: dft.rolling(2).sum()
Out[65]:
```

	B
foo	
2013-01-01 09:00:00	NaN
2013-01-01 09:00:02	1.0
2013-01-01 09:00:03	3.0
2013-01-01 09:00:05	NaN
2013-01-01 09:00:06	NaN

Using the time-specification generates variable windows for this sparse data.

```
In [66]: dft.rolling('2s').sum()
Out[66]:
```

```

 B
foo
2013-01-01 09:00:00 0.0
2013-01-01 09:00:02 1.0
2013-01-01 09:00:03 3.0
2013-01-01 09:00:05 NaN
2013-01-01 09:00:06 4.0

```

Furthermore, we now allow an optional `on` parameter to specify a column (rather than the default of the index) in a DataFrame.

```
In [67]: dft = dft.reset_index()
```

```
In [68]: dft
```

```
Out[68]:
 foo B
0 2013-01-01 09:00:00 0.0
1 2013-01-01 09:00:02 1.0
2 2013-01-01 09:00:03 2.0
3 2013-01-01 09:00:05 NaN
4 2013-01-01 09:00:06 4.0

```

```
In [69]: dft.rolling('2s', on='foo').sum()
```

```
Out[69]:
 foo B
0 2013-01-01 09:00:00 0.0
1 2013-01-01 09:00:02 1.0
2 2013-01-01 09:00:03 3.0
3 2013-01-01 09:00:05 NaN
4 2013-01-01 09:00:06 4.0

```

## Rolling window endpoints

New in version 0.20.0.

The inclusion of the interval endpoints in rolling window calculations can be specified with the `closed` parameter:

<b>closed</b>	<b>Description</b>	<b>Default for</b>
right	close right endpoint	time-based windows
left	close left endpoint	
both	close both endpoints	fixed windows
neither	open endpoints	

For example, having the right endpoint open is useful in many problems that require that there is no contamination from present information back to past information. This allows the rolling window to compute statistics “up to that point in time” , but not including that point in time.

```
In [70]: df = pd.DataFrame({'x': 1},
.....: index=[pd.Timestamp('20130101 09:00:01'),
.....: pd.Timestamp('20130101 09:00:02'),
.....: pd.Timestamp('20130101 09:00:03'),
.....: pd.Timestamp('20130101 09:00:04'),
.....: pd.Timestamp('20130101 09:00:06')])
.....:

In [71]: df["right"] = df.rolling('2s', closed='right').x.sum() # default

In [72]: df["both"] = df.rolling('2s', closed='both').x.sum()

In [73]: df["left"] = df.rolling('2s', closed='left').x.sum()

In [74]: df["neither"] = df.rolling('2s', closed='neither').x.sum()

In [75]: df
Out[75]:
```

	x	right	both	left	neither
2013-01-01 09:00:01	1	1.0	1.0	NaN	NaN
2013-01-01 09:00:02	1	2.0	2.0	1.0	1.0
2013-01-01 09:00:03	1	2.0	3.0	2.0	1.0
2013-01-01 09:00:04	1	2.0	3.0	2.0	1.0
2013-01-01 09:00:06	1	1.0	2.0	1.0	NaN

Currently, this feature is only implemented for time-based windows. For fixed windows, the closed parameter cannot be set and the rolling window will always have both endpoints closed.

## Time-aware rolling vs. resampling

Using `.rolling()` with a time-based index is quite similar to [resampling](#). They both operate and perform reductive operations on time-indexed pandas objects.

When using `.rolling()` with an offset. The offset is a time-delta. Take a backwards-in-time looking window, and aggregate all of the values in that window (including the end-point, but not the start-point). This is the new value at that point in the result. These are variable sized windows in time-space for each point of the input. You will get a same sized result as the input.

When using `.resample()` with an offset. Construct a new index that is the frequency of the offset. For each frequency bin, aggregate points from the input within a backwards-in-time looking window that fall in that bin. The result of this aggregation is the output for that frequency point. The windows are fixed size in the frequency space. Your result will have the shape of a regular frequency between the min and the max of the original input object.



To summarize, `.rolling()` is a time-based window operation, while `.resample()` is a frequency-based window operation.

## Centering windows

By default the labels are set to the right edge of the window, but a `center` keyword is available so the labels can be set at the center.

```
In [76]: ser.rolling(window=5).mean()
Out[76]:
2000-01-01 NaN
2000-01-02 NaN
2000-01-03 NaN
2000-01-04 NaN
2000-01-05 -0.841164
2000-01-06 -0.779948
2000-01-07 -0.565487
2000-01-08 -0.502815
2000-01-09 -0.553755
2000-01-10 -0.472211
Freq: D, dtype: float64

In [77]: ser.rolling(window=5, center=True).mean()
Out[77]:
2000-01-01 NaN
2000-01-02 NaN
2000-01-03 -0.841164
2000-01-04 -0.779948
2000-01-05 -0.565487
2000-01-06 -0.502815
2000-01-07 -0.553755
2000-01-08 -0.472211
2000-01-09 NaN
2000-01-10 NaN
Freq: D, dtype: float64
```

## Binary window functions

`cov()` and `corr()` can compute moving window statistics about two `Series` or any combination of `DataFrame/Series` or `DataFrame/DataFrame`. Here is the behavior in each case:

- two `Series`: compute the statistic for the pairing.
- `DataFrame/Series`: compute the statistics for each column of the `DataFrame` with the passed `Series`, thus returning a `DataFrame`.
- `DataFrame/DataFrame`: by default compute the statistic for matching column names, returning a `DataFrame`. If the keyword argument `pairwise=True` is passed then computes the statistic for each pair of columns, returning a `MultiIndexed DataFrame` whose `index` are the dates in question (see [the next section](#)).

For example:

```
In [78]: df = pd.DataFrame(np.random.randn(1000, 4),
.....: index=pd.date_range('1/1/2000', periods=1000),
.....: columns=['A', 'B', 'C', 'D'])
.....:
```

```
In [79]: df = df.cumsum()
```

```
In [80]: df2 = df[:20]
```

```
In [81]: df2.rolling(window=5).corr(df2['B'])
```

```
Out[81]:
```

	A	B	C	D
2000-01-01	NaN	NaN	NaN	NaN
2000-01-02	NaN	NaN	NaN	NaN
2000-01-03	NaN	NaN	NaN	NaN
2000-01-04	NaN	NaN	NaN	NaN
2000-01-05	0.768775	1.0	-0.977990	0.800252
...	...	...	...	...
2000-01-16	0.691078	1.0	0.807450	-0.939302
2000-01-17	0.274506	1.0	0.582601	-0.902954
2000-01-18	0.330459	1.0	0.515707	-0.545268
2000-01-19	0.046756	1.0	-0.104334	-0.419799
2000-01-20	-0.328241	1.0	-0.650974	-0.777777

```
[20 rows x 4 columns]
```

## Computing rolling pairwise covariances and correlations

In financial data analysis and other fields it's common to compute covariance and correlation matrices for a collection of time series. Often one is also interested in moving-window covariance and correlation matrices. This can be done by passing the `pairwise` keyword argument, which in the case of `DataFrame` inputs will yield a MultiIndexed `DataFrame` whose `index` are the dates in question. In the case of a single `DataFrame` argument the `pairwise` argument can even be omitted:

::: tip Note

Missing values are ignored and each entry is computed using the pairwise complete observations. Please see the [covariance section](#) for [caveats](#) associated with this method of calculating covariance and correlation matrices.

:::

```
In [82]: covs = (df[['B', 'C', 'D']].rolling(window=50)
.....: .cov(df[['A', 'B', 'C']], pairwise=True))
.....:
```

```
In [83]: covs.loc['2002-09-22:']
```

```
Out[83]:
```

		B	C	D
2002-09-22	A	1.367467	8.676734	-8.047366
	B	3.067315	0.865946	-1.052533
	C	0.865946	7.739761	-4.943924
2002-09-23	A	0.910343	8.669065	-8.443062
	B	2.625456	0.565152	-0.907654
	C	0.565152	7.825521	-5.367526
2002-09-24	A	0.463332	8.514509	-8.776514
	B	2.306695	0.267746	-0.732186
	C	0.267746	7.771425	-5.696962
2002-09-25	A	0.467976	8.198236	-9.162599
	B	2.307129	0.267287	-0.754080
	C	0.267287	7.466559	-5.822650
2002-09-26	A	0.545781	7.899084	-9.326238
	B	2.311058	0.322295	-0.844451
	C	0.322295	7.038237	-5.684445

```
In [84]: correls = df.rolling(window=50).corr()
```

```
In [85]: correls.loc['2002-09-22:']
```

```
Out[85]:
```

		A	B	C	D
2002-09-22	A	1.000000	0.186397	0.744551	-0.769767
	B	0.186397	1.000000	0.177725	-0.240802
	C	0.744551	0.177725	1.000000	-0.712051
	D	-0.769767	-0.240802	-0.712051	1.000000
2002-09-23	A	1.000000	0.134723	0.743113	-0.758758
...	...	...	...	...	...
2002-09-25	D	-0.739160	-0.164179	-0.704686	1.000000
2002-09-26	A	1.000000	0.087756	0.727792	-0.736562
	B	0.087756	1.000000	0.079913	-0.179477
	C	0.727792	0.079913	1.000000	-0.692303
	D	-0.736562	-0.179477	-0.692303	1.000000

```
[20 rows x 4 columns]
```

You can efficiently retrieve the time series of correlations between two columns by reshaping and indexing:

```
In [86]: correls.unstack(1)[('A', 'C')].plot()
```

```
Out[86]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f8dc79e8>
```

rolling corr pairwise ex

## Aggregation

---

Once the `Rolling`, `Expanding` or `EWM` objects have been created, several methods are available to perform multiple computations on the data. These operations are similar to the [aggregating API](#), [groupby API](#), and [resample API](#).

```
In [87]: dfa = pd.DataFrame(np.random.randn(1000, 3),
.....: index=pd.date_range('1/1/2000', periods=1000),
.....: columns=['A', 'B', 'C'])
.....:

In [88]: r = dfa.rolling(window=60, min_periods=1)

In [89]: r
Out[89]: Rolling [window=60,min_periods=1,center=False,axis=0]
```

We can aggregate by passing a function to the entire `DataFrame`, or select a `Series` (or multiple `Series`) via standard `__getitem__`.

```
In [90]: r.aggregate(np.sum)
Out[90]:
```

	A	B	C
2000-01-01	-0.289838	-0.370545	-1.284206
2000-01-02	-0.216612	-1.675528	-1.169415
2000-01-03	1.154661	-1.634017	-1.566620
2000-01-04	2.969393	-4.003274	-1.816179
2000-01-05	4.690630	-4.682017	-2.717209
...	...	...	...
2002-09-22	2.860036	-9.270337	6.415245
2002-09-23	3.510163	-8.151439	5.177219
2002-09-24	6.524983	-10.168078	5.792639
2002-09-25	6.409626	-9.956226	5.704050
2002-09-26	5.093787	-7.074515	6.905823

[1000 rows x 3 columns]

```
In [91]: r['A'].aggregate(np.sum)
```

```
Out[91]:
```

2000-01-01	-0.289838
2000-01-02	-0.216612
2000-01-03	1.154661
2000-01-04	2.969393
2000-01-05	4.690630
...	...
2002-09-22	2.860036
2002-09-23	3.510163
2002-09-24	6.524983
2002-09-25	6.409626
2002-09-26	5.093787

Freq: D, Name: A, Length: 1000, dtype: float64

```
In [92]: r[['A', 'B']].aggregate(np.sum)
```

```
Out[92]:
```

	A	B
2000-01-01	-0.289838	-0.370545
2000-01-02	-0.216612	-1.675528
2000-01-03	1.154661	-1.634017
2000-01-04	2.969393	-4.003274
2000-01-05	4.690630	-4.682017
...	...	...
2002-09-22	2.860036	-9.270337
2002-09-23	3.510163	-8.151439
2002-09-24	6.524983	-10.168078
2002-09-25	6.409626	-9.956226
2002-09-26	5.093787	-7.074515

[1000 rows x 2 columns]

As you can see, the result of the aggregation will have the selected columns, or all columns if none are selected.

## Applying multiple functions

With windowed `Series` you can also pass a list of functions to do aggregation with, outputting a `DataFrame`:

```
In [93]: r['A'].agg([np.sum, np.mean, np.std])
```

```
Out[93]:
```

	sum	mean	std
2000-01-01	-0.289838	-0.289838	NaN
2000-01-02	-0.216612	-0.108306	0.256725
2000-01-03	1.154661	0.384887	0.873311
2000-01-04	2.969393	0.742348	1.009734
2000-01-05	4.690630	0.938126	0.977914
...	...	...	...
2002-09-22	2.860036	0.047667	1.132051
2002-09-23	3.510163	0.058503	1.134296
2002-09-24	6.524983	0.108750	1.144204
2002-09-25	6.409626	0.106827	1.142913
2002-09-26	5.093787	0.084896	1.151416

```
[1000 rows x 3 columns]
```

On a windowed `DataFrame`, you can pass a list of functions to apply to each column, which produces an aggregated result with a hierarchical index:

```
In [94]: r.agg([np.sum, np.mean])
```

```
Out[94]:
```

	A		B		C	
	sum	mean	sum	mean	sum	mean
2000-01-01	-0.289838	-0.289838	-0.370545	-0.370545	-1.284206	-1.284206
2000-01-02	-0.216612	-0.108306	-1.675528	-0.837764	-1.169415	-0.584708
2000-01-03	1.154661	0.384887	-1.634017	-0.544672	-1.566620	-0.522207
2000-01-04	2.969393	0.742348	-4.003274	-1.000819	-1.816179	-0.454045
2000-01-05	4.690630	0.938126	-4.682017	-0.936403	-2.717209	-0.543442
...	...	...	...	...	...	...
2002-09-22	2.860036	0.047667	-9.270337	-0.154506	6.415245	0.106921
2002-09-23	3.510163	0.058503	-8.151439	-0.135857	5.177219	0.086287
2002-09-24	6.524983	0.108750	-10.168078	-0.169468	5.792639	0.096544
2002-09-25	6.409626	0.106827	-9.956226	-0.165937	5.704050	0.095068
2002-09-26	5.093787	0.084896	-7.074515	-0.117909	6.905823	0.115097

```
[1000 rows x 6 columns]
```

Passing a dict of functions has different behavior by default, see the next section.

## Applying different functions to DataFrame columns

By passing a dict to `aggregate` you can apply a different aggregation to the columns of a `DataFrame`:

```
In [95]: r.agg({'A': np.sum, 'B': lambda x: np.std(x, ddof=1)})
```

```
Out[95]:
```

	A	B
2000-01-01	-0.289838	NaN
2000-01-02	-0.216612	0.660747
2000-01-03	1.154661	0.689929
2000-01-04	2.969393	1.072199
2000-01-05	4.690630	0.939657
...	...	...
2002-09-22	2.860036	1.113208
2002-09-23	3.510163	1.132381
2002-09-24	6.524983	1.080963
2002-09-25	6.409626	1.082911
2002-09-26	5.093787	1.136199

```
[1000 rows x 2 columns]
```

The function names can also be strings. In order for a string to be valid it must be implemented on the windowed object

```
In [96]: r.agg({'A': 'sum', 'B': 'std'})
```

```
Out[96]:
```

	A	B
2000-01-01	-0.289838	NaN
2000-01-02	-0.216612	0.660747
2000-01-03	1.154661	0.689929
2000-01-04	2.969393	1.072199
2000-01-05	4.690630	0.939657
...	...	...
2002-09-22	2.860036	1.113208
2002-09-23	3.510163	1.132381
2002-09-24	6.524983	1.080963
2002-09-25	6.409626	1.082911
2002-09-26	5.093787	1.136199

```
[1000 rows x 2 columns]
```

Furthermore you can pass a nested dict to indicate different aggregations on different columns.

```
In [97]: r.agg({'A': ['sum', 'std'], 'B': ['mean', 'std']})
Out[97]:
```

	A		B	
	sum	std	mean	std
2000-01-01	-0.289838	NaN	-0.370545	NaN
2000-01-02	-0.216612	0.256725	-0.837764	0.660747
2000-01-03	1.154661	0.873311	-0.544672	0.689929
2000-01-04	2.969393	1.009734	-1.000819	1.072199
2000-01-05	4.690630	0.977914	-0.936403	0.939657
...	...	...	...	...
2002-09-22	2.860036	1.132051	-0.154506	1.113208
2002-09-23	3.510163	1.134296	-0.135857	1.132381
2002-09-24	6.524983	1.144204	-0.169468	1.080963
2002-09-25	6.409626	1.142913	-0.165937	1.082911
2002-09-26	5.093787	1.151416	-0.117909	1.136199

```
[1000 rows x 4 columns]
```

## Expanding windows

A common alternative to rolling statistics is to use an expanding window, which yields the value of the statistic with all the data available up to that point in time.

These follow a similar interface to `.rolling`, with the `.expanding` method returning an `Expanding` object.

As these calculations are a special case of rolling statistics, they are implemented in pandas such that the following two calls are equivalent:

```
In [98]: df.rolling(window=len(df), min_periods=1).mean()[:5]
Out[98]:
```

	A	B	C	D
2000-01-01	0.314226	-0.001675	0.071823	0.892566
2000-01-02	0.654522	-0.171495	0.179278	0.853361
2000-01-03	0.708733	-0.064489	-0.238271	1.371111
2000-01-04	0.987613	0.163472	-0.919693	1.566485
2000-01-05	1.426971	0.288267	-1.358877	1.808650

```
In [99]: df.expanding(min_periods=1).mean()[:5]
Out[99]:
```

	A	B	C	D
2000-01-01	0.314226	-0.001675	0.071823	0.892566
2000-01-02	0.654522	-0.171495	0.179278	0.853361
2000-01-03	0.708733	-0.064489	-0.238271	1.371111
2000-01-04	0.987613	0.163472	-0.919693	1.566485
2000-01-05	1.426971	0.288267	-1.358877	1.808650

These have a similar set of methods to `.rolling` methods.



## Method summary

Function	Description
<code>count()</code>	Number of non-null observations
<code>sum()</code>	Sum of values
<code>mean()</code>	Mean of values
<code>median()</code>	Arithmetic median of values
<code>min()</code>	Minimum
<code>max()</code>	Maximum
<code>std()</code>	Unbiased standard deviation
<code>var()</code>	Unbiased variance
<code>skew()</code>	Unbiased skewness (3rd moment)
<code>kurt()</code>	Unbiased kurtosis (4th moment)
<code>quantile()</code>	Sample quantile (value at %)
<code>apply()</code>	Generic apply
<code>cov()</code>	Unbiased covariance (binary)
<code>corr()</code>	Correlation (binary)

Aside from not having a `window` parameter, these functions have the same interfaces as their `.rolling` counterparts. Like above, the parameters they all accept are:

- `min_periods`: threshold of non-null data points to require. Defaults to minimum needed to compute statistic. No `NaNs` will be output once `min_periods` non-null data points have been seen.
- `center`: boolean, whether to set the labels at the center (default is `False`).

::: tip Note

The output of the `.rolling` and `.expanding` methods do not return a `NaN` if there are at least `min_periods` non-null values in the current window. For example:

```
In [100]: sn = pd.Series([1, 2, np.nan, 3, np.nan, 4])
```

```
In [101]: sn
```

```
Out[101]:
```

```
0 1.0
```

```
1 2.0
```

```
2 NaN
```

```
3 3.0
```

```
4 NaN
```

```
5 4.0
```

```
dtype: float64
```

```
In [102]: sn.rolling(2).max()
```

```
Out[102]:
```

```
0 NaN
```

```
1 2.0
```

```
2 NaN
```

```
3 NaN
```

```
4 NaN
```

```
5 NaN
```

```
dtype: float64
```

```
In [103]: sn.rolling(2, min_periods=1).max()
```

```
Out[103]:
```

```
0 1.0
```

```
1 2.0
```

```
2 2.0
```

```
3 3.0
```

```
4 3.0
```

```
5 4.0
```

```
dtype: float64
```

In case of expanding functions, this differs from `cumsum()`, `cumprod()`, `cummax()`, and `cummin()`, which return `NaN` in the output wherever a `NaN` is encountered in the input. In order to match the output of `cumsum` with `expanding`, use `fillna()`:

```

In [104]: sn.expanding().sum()
Out[104]:
0 1.0
1 3.0
2 3.0
3 6.0
4 6.0
5 10.0
dtype: float64

In [105]: sn.cumsum()
Out[105]:
0 1.0
1 3.0
2 NaN
3 6.0
4 NaN
5 10.0
dtype: float64

In [106]: sn.cumsum().fillna(method='ffill')
Out[106]:
0 1.0
1 3.0
2 3.0
3 6.0
4 6.0
5 10.0
dtype: float64

```

...

An expanding window statistic will be more stable (and less responsive) than its rolling window counterpart as the increasing window size decreases the relative impact of an individual data point. As an example, here is the `mean()` output for the previous time series dataset:

```

In [107]: s.plot(style='k--')
Out[107]: <matplotlib.axes._subplots.AxesSubplot at 0x7f6607b14400>

In [108]: s.expanding().mean().plot(style='k')
Out[108]: <matplotlib.axes._subplots.AxesSubplot at 0x7f6607b14400>

```

expanding mean frame

## Exponentially weighted windows

A related set of functions are exponentially weighted versions of several of the above statistics. A similar interface to `.rolling` and `.expanding` is accessed through the `.ewm` method to receive an `EWM` object. A number of expanding EW (exponentially weighted) methods are provided:

Function	Description
<code>mean()</code>	EW moving average
<code>var()</code>	EW moving variance
<code>std()</code>	EW moving standard deviation
<code>corr()</code>	EW moving correlation
<code>cov()</code>	EW moving covariance

In general, a weighted moving average is calculated as

$$y_t = \frac{\sum_{i=0}^t w_i x_{t-i}}{\sum_{i=0}^t w_i},$$

where  $x_t$  is the input,  $y_t$  is the result and the  $w_i$  are the weights.

The EW functions support two variants of exponential weights. The default, `adjust=True`, uses the weights  $w_i = (1 - \alpha)^i$  which gives

$$y_t = \frac{x_t + (1 - \alpha)x_{t-1} + (1 - \alpha)^2 x_{t-2} + \dots + (1 - \alpha)^t x_0}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots + (1 - \alpha)^t}$$

When `adjust=False` is specified, moving averages are calculated as

$$y_0 = x_0 \quad y_t = (1 - \alpha) y_{t-1} + \alpha x_t,$$

which is equivalent to using weights

$$w_i = \begin{cases} \alpha (1 - \alpha)^i & \text{if } i < t \\ (1 - \alpha)^i & \text{if } i = t. \end{cases}$$

::: tip Note

These equations are sometimes written in terms of  $\alpha' = 1 - \alpha$ , e.g.

$$y_t = \alpha' y_{t-1} + (1 - \alpha') x_t.$$

:::

The difference between the above two variants arises because we are dealing with series which have finite history. Consider a series of infinite history, with `adjust=True`:

$$y_t = \alpha' y_{t-1} + (1 - \alpha') x_t.$$

Noting that the denominator is a geometric series with initial term equal to 1 and a ratio of  $(1 - \alpha)$  we have

$$y_t = \frac{x_t + (1 - \alpha)x_{t-1} + (1 - \alpha)^2 x_{t-2} + \dots}{1 + (1 - \alpha) + (1 - \alpha)^2 + \dots} = \frac{x_t + (1 - \alpha)x_{t-1} + (1 - \alpha)^2 x_{t-2} + \dots}{1 - (1 - \alpha)}$$

$$-\alpha^2 x_{t-2} + \dots] \alpha \quad \&= \quad \alpha x_t + (1 - \alpha)[x_{t-1} + (1 - \alpha)x_{t-2} + \dots] \alpha \quad \&= \quad \alpha x_t + (1 - \alpha)y_{t-1} \end{split} \]$$

which is the same expression as `adjust=False` above and therefore shows the equivalence of the two variants for infinite series. When `adjust=False`, we have  $y_0 = x_0$  and  $y_t = \alpha x_t + (1 - \alpha)y_{t-1}$ . Therefore, there is an assumption that  $x_0$  is not an ordinary value but rather an exponentially weighted moment of the infinite series up to that point.

One must have  $0 < \alpha \leq 1$ , and while since version 0.18.0 it has been possible to pass  $\alpha$  directly, it's often easier to think about either the **span**, **center of mass (com)** or **half-life** of an EW moment:

$$\begin{split} \alpha = \begin{cases} \frac{2}{s+1}, & \text{for span } s \geq 1 \\ \frac{1}{1+c}, & \text{for center of mass } c \geq 0 \\ 1 - \exp^{-\frac{\log 0.5}{h}}, & \text{for half-life } h > 0 \end{cases} \end{split}$$

One must specify precisely one of **span**, **center of mass**, **half-life** and **alpha** to the EW functions:

- **Span** corresponds to what is commonly called an “N-day EW moving average”.
- **Center of mass** has a more physical interpretation and can be thought of in terms of span:  $c = (s - 1) / 2$ .
- **Half-life** is the period of time for the exponential weight to reduce to one half.
- **Alpha** specifies the smoothing factor directly.

Here is an example for a univariate time series:

```
In [109]: s.plot(style='k--')
Out[109]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f7537278>

In [110]: s.ewm(span=20).mean().plot(style='k')
Out[110]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f7537278>
```

ewma ex

EWM has a `min_periods` argument, which has the same meaning it does for all the `.expanding` and `.rolling` methods: no output values will be set until at least `min_periods` non-null values are encountered in the (expanding) window.

EWM also has an `ignore_na` argument, which determines how intermediate null values affect the calculation of the weights. When `ignore_na=False` (the default), weights are calculated based on absolute positions, so that intermediate null values affect the result. When `ignore_na=True`, weights are calculated by ignoring intermediate null values. For example, assuming `adjust=True`, if `ignore_na=False`, the weighted average of 3, NaN, 5 would be calculated as

$$\frac{(1-\alpha)^2 \cdot 3 + 1 \cdot 5}{(1-\alpha)^2 + 1}$$

Whereas if `ignore_na=True`, the weighted average would be calculated as

$$\left[\frac{(1-\alpha) \cdot 3 + 1 \cdot 5}{(1-\alpha) + 1}\right]$$

The `var()`, `std()`, and `cov()` functions have a `bias` argument, specifying whether the result should contain biased or unbiased statistics. For example, if `bias=True`, `ewmvar(x)` is calculated as `ewmvar(x) = ewma(x**2) - ewma(x)**2`; whereas if `bias=False` (the default), the biased variance statistics are scaled by debiasing factors

$$\left[\frac{\left(\sum_{i=0}^t w_i\right)^2}{\left(\sum_{i=0}^t w_i\right)^2 - \sum_{i=0}^t w_i^2}\right]$$

(For  $(w_i = 1)$ , this reduces to the usual  $(N / (N - 1))$  factor, with  $(N = t + 1)$ .) See [Weighted Sample Variance](#) on Wikipedia for further details.

## Group By: split-apply-combine

By “group by” we are referring to a process involving one or more of the following steps:

- **Splitting** the data into groups based on some criteria.
- **Applying** a function to each group independently.
- **Combining** the results into a data structure.

Out of these, the split step is the most straightforward. In fact, in many situations we may wish to split the data set into groups and do something with those groups. In the apply step, we might wish to do one of the following:

- **Aggregation:** compute a summary statistic (or statistics) for each group. Some examples:
  - Compute group sums or means.
  - Compute group sizes / counts.
- **Transformation:** perform some group-specific computations and return a like-indexed object. Some examples:
  - Standardize data (zscore) within a group.
  - Filling NAs within groups with a value derived from each group.
- **Filtration:** discard some groups, according to a group-wise computation that evaluates True or False. Some examples:
  - Discard data that belongs to groups with only a few members.
  - Filter out data based on the group sum or mean.
- Some combination of the above: `GroupBy` will examine the results of the apply step and try to return a sensibly combined result if it doesn't fit into either of the above two categories.

Since the set of object instance methods on pandas data structures are generally rich and expressive, we often simply want to invoke, say, a `DataFrame` function on each group. The name `GroupBy` should

be quite familiar to those who have used a SQL-based tool (or `itertools`), in which you can write code like:

```
SELECT Column1, Column2, mean(Column3), sum(Column4)
FROM SomeTable
GROUP BY Column1, Column2
```

We aim to make operations like this natural and easy to express using pandas. We'll address each area of GroupBy functionality then provide some non-trivial examples / use cases.

See the [cookbook](#) for some advanced strategies.

## Splitting an object into groups

pandas objects can be split on any of their axes. The abstract definition of grouping is to provide a mapping of labels to group names. To create a GroupBy object (more on what the GroupBy object is later), you may do the following:

```
In [1]: df = pd.DataFrame([('bird', 'Falconiformes', 389.0),
...: ('bird', 'Psittaciformes', 24.0),
...: ('mammal', 'Carnivora', 80.2),
...: ('mammal', 'Primates', np.nan),
...: ('mammal', 'Carnivora', 58)],
...: index=['falcon', 'parrot', 'lion', 'monkey', 'leopard'],
...: columns=('class', 'order', 'max_speed'))
...:
```

```
In [2]: df
Out[2]:
```

	class	order	max_speed
falcon	bird	Falconiformes	389.0
parrot	bird	Psittaciformes	24.0
lion	mammal	Carnivora	80.2
monkey	mammal	Primates	NaN
leopard	mammal	Carnivora	58.0

```
default is axis=0
In [3]: grouped = df.groupby('class')

In [4]: grouped = df.groupby('order', axis='columns')

In [5]: grouped = df.groupby(['class', 'order'])
```

The mapping can be specified many different ways:

- A Python function, to be called on each of the axis labels.
- A list or NumPy array of the same length as the selected axis.
- A dict or `Series`, providing a `label -> group name` mapping.

- For `DataFrame` objects, a string indicating a column to be used to group. Of course `df.groupby('A')` is just syntactic sugar for `df.groupby(df['A'])`, but it makes life simpler.
- For `DataFrame` objects, a string indicating an index level to be used to group.
- A list of any of the above things.

Collectively we refer to the grouping objects as the **keys**. For example, consider the following

`DataFrame` :

::: tip Note

A string passed to `groupby` may refer to either a column or an index level. If a string matches both a column name and an index level name, a `ValueError` will be raised.

:::

```
In [6]: df = pd.DataFrame({'A': ['foo', 'bar', 'foo', 'bar',
...: 'foo', 'bar', 'foo', 'foo'],
...: 'B': ['one', 'one', 'two', 'three',
...: 'two', 'two', 'one', 'three'],
...: 'C': np.random.randn(8),
...: 'D': np.random.randn(8)})
...:
```

```
In [7]: df
```

```
Out[7]:
```

	A	B	C	D
0	foo	one	0.469112	-0.861849
1	bar	one	-0.282863	-2.104569
2	foo	two	-1.509059	-0.494929
3	bar	three	-1.135632	1.071804
4	foo	two	1.212112	0.721555
5	bar	two	-0.173215	-0.706771
6	foo	one	0.119209	-1.039575
7	foo	three	-1.044236	0.271860

On a `DataFrame`, we obtain a `GroupBy` object by calling `groupby()`. We could naturally group by either the `A` or `B` columns, or both:

```
In [8]: grouped = df.groupby('A')
```

```
In [9]: grouped = df.groupby(['A', 'B'])
```

New in version 0.24.

If we also have a `MultiIndex` on columns `A` and `B`, we can group by all but the specified columns



```

In [10]: df2 = df.set_index(['A', 'B'])

In [11]: grouped = df2.groupby(level=df2.index.names.difference(['B']))

In [12]: grouped.sum()
Out[12]:
 C D
A
bar -1.591710 -1.739537
foo -0.752861 -1.402938

```

These will split the DataFrame on its index (rows). We could also split by the columns:

```

In [13]: def get_letter_type(letter):
....: if letter.lower() in 'aeiou':
....: return 'vowel'
....: else:
....: return 'consonant'
....:

In [14]: grouped = df.groupby(get_letter_type, axis=1)

```

pandas [Index](#) objects support duplicate values. If a non-unique index is used as the group key in a groupby operation, all values for the same index value will be considered to be in one group and thus the output of aggregation functions will only contain unique index values:

```

In [15]: lst = [1, 2, 3, 1, 2, 3]

In [16]: s = pd.Series([1, 2, 3, 10, 20, 30], lst)

In [17]: grouped = s.groupby(level=0)

In [18]: grouped.first()
Out[18]:
1 1
2 2
3 3
dtype: int64

In [19]: grouped.last()
Out[19]:
1 10
2 20
3 30
dtype: int64

In [20]: grouped.sum()
Out[20]:
1 11
2 22
3 33
dtype: int64

```

Note that **no splitting occurs** until it's needed. Creating the GroupBy object only verifies that you've passed a valid mapping.

::: tip Note

Many kinds of complicated data manipulations can be expressed in terms of GroupBy operations (though can't be guaranteed to be the most efficient). You can get quite creative with the label mapping functions.

:::

## GroupBy sorting

By default the group keys are sorted during the `groupby` operation. You may however pass `sort=False` for potential speedups:

```
In [21]: df2 = pd.DataFrame({'X': ['B', 'B', 'A', 'A'], 'Y': [1, 2, 3, 4]})

In [22]: df2.groupby(['X']).sum()
Out[22]:
 Y
X
A 7
B 3

In [23]: df2.groupby(['X'], sort=False).sum()
Out[23]:
 Y
X
B 3
A 7
```

Note that `groupby` will preserve the order in which observations are sorted within each group. For example, the groups created by `groupby()` below are in the order they appeared in the original

`DataFrame`:

```
In [24]: df3 = pd.DataFrame({'X': ['A', 'B', 'A', 'B'], 'Y': [1, 4, 3, 2]})

In [25]: df3.groupby(['X']).get_group('A')
Out[25]:
 X Y
0 A 1
2 A 3

In [26]: df3.groupby(['X']).get_group('B')
Out[26]:
 X Y
1 B 4
3 B 2
```

## GroupBy object attributes

The `groups` attribute is a dict whose keys are the computed unique groups and corresponding values being the axis labels belonging to each group. In the above example we have:

```
In [27]: df.groupby('A').groups
Out[27]:
{'bar': Int64Index([1, 3, 5], dtype='int64'),
 'foo': Int64Index([0, 2, 4, 6, 7], dtype='int64')}

In [28]: df.groupby(get_letter_type, axis=1).groups
Out[28]:
{'consonant': Index(['B', 'C', 'D'], dtype='object'),
 'vowel': Index(['A'], dtype='object')}
```

Calling the standard Python `len` function on the GroupBy object just returns the length of the `groups` dict, so it is largely just a convenience:

```
In [29]: grouped = df.groupby(['A', 'B'])

In [30]: grouped.groups
Out[30]:
{('bar', 'one'): Int64Index([1], dtype='int64'),
 ('bar', 'three'): Int64Index([3], dtype='int64'),
 ('bar', 'two'): Int64Index([5], dtype='int64'),
 ('foo', 'one'): Int64Index([0, 6], dtype='int64'),
 ('foo', 'three'): Int64Index([7], dtype='int64'),
 ('foo', 'two'): Int64Index([2, 4], dtype='int64')}

In [31]: len(grouped)
Out[31]: 6
```

`GroupBy` will tab complete column names (and other attributes):

```
In [32]: df
Out[32]:
```

	height	weight	gender
2000-01-01	42.849980	157.500553	male
2000-01-02	49.607315	177.340407	male
2000-01-03	56.293531	171.524640	male
2000-01-04	48.421077	144.251986	female
2000-01-05	46.556882	152.526206	male
2000-01-06	68.448851	168.272968	female
2000-01-07	70.757698	136.431469	male
2000-01-08	58.909500	176.499753	female
2000-01-09	76.435631	174.094104	female
2000-01-10	45.306120	177.540920	male

```
In [33]: gb = df.groupby('gender')
```

```
In [34]: gb.<TAB> # noqa: E225, E999
gb.agg gb.boxplot gb.cummin gb.describe gb.filter gb.get_group gb.height
 gb.last gb.median gb.ngroups gb.plot gb.rank gb.std
 gb.transform
gb.aggregate gb.count gb.cumprod gb.dtype gb.first gb.groups gb.hist
 gb.max gb.min gb.nth gb.prod gb.resample gb.sum gb.var
gb.apply gb.cummax gb.cumsum gb.fillna gb.gender gb.head gb.indices
 gb.mean gb.name gb.ohlc gb.quantile gb.size gb.tail gb.weight
```

## GroupBy with MultiIndex

With [hierarchically-indexed data](#), it's quite natural to group by one of the levels of the hierarchy.

Let's create a Series with a two-level `MultiIndex`.

```
In [35]: arrays = [['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux'],
....: ['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two']]
....:

In [36]: index = pd.MultiIndex.from_arrays(arrays, names=['first', 'second'])

In [37]: s = pd.Series(np.random.randn(8), index=index)

In [38]: s
Out[38]:
first second
bar one -0.919854
 two -0.042379
baz one 1.247642
 two -0.009920
foo one 0.290213
 two 0.495767
qux one 0.362949
 two 1.548106
dtype: float64
```

We can then group by one of the levels in `s`.

```
In [39]: grouped = s.groupby(level=0)

In [40]: grouped.sum()
Out[40]:
first
bar -0.962232
baz 1.237723
foo 0.785980
qux 1.911055
dtype: float64
```

If the `MultiIndex` has names specified, these can be passed instead of the level number:

```
In [41]: s.groupby(level='second').sum()
Out[41]:
second
one 0.980950
two 1.991575
dtype: float64
```

The aggregation functions such as `sum` will take the `level` parameter directly. Additionally, the resulting index will be named according to the chosen level:

```
In [42]: s.sum(level='second')
Out[42]:
second
one 0.980950
two 1.991575
dtype: float64
```

Grouping with multiple levels is supported.

```
In [43]: s
Out[43]:
first second third
bar doo one -1.131345
 doo two -0.089329
baz bee one 0.337863
 bee two -0.945867
foo bop one -0.932132
 bop two 1.956030
qux bop one 0.017587
 bop two -0.016692
dtype: float64

In [44]: s.groupby(level=['first', 'second']).sum()
Out[44]:
first second
bar doo -1.220674
baz bee -0.608004
foo bop 1.023898
qux bop 0.000895
dtype: float64
```

New in version 0.20.

Index level names may be supplied as keys.

```
In [45]: s.groupby(['first', 'second']).sum()
Out[45]:
first second
bar doo -1.220674
baz bee -0.608004
foo bop 1.023898
qux bop 0.000895
dtype: float64
```

More on the `sum` function and aggregation later.

## Grouping DataFrame with Index levels and columns

A DataFrame may be grouped by a combination of columns and index levels by specifying the column names as strings and the index levels as `pd.Grouper` objects.

```
In [46]: arrays = [['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux'],
....: ['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two']]
....:

In [47]: index = pd.MultiIndex.from_arrays(arrays, names=['first', 'second'])

In [48]: df = pd.DataFrame({'A': [1, 1, 1, 1, 2, 2, 3, 3],
....: 'B': np.arange(8)},
....: index=index)
....:

In [49]: df
Out[49]:
 first second A B
bar one 1 0
 two 1 1
baz one 1 2
 two 1 3
foo one 2 4
 two 2 5
qux one 3 6
 two 3 7
```

The following example groups `df` by the `second` index level and the `A` column.

```
In [50]: df.groupby([pd.Grouper(level=1), 'A']).sum()
Out[50]:
```

		B
second	A	
one	1	2
	2	4
	3	6
two	1	4
	2	5
	3	7

Index levels may also be specified by name.

```
In [51]: df.groupby([pd.Grouper(level='second'), 'A']).sum()
Out[51]:
```

		B
second	A	
one	1	2
	2	4
	3	6
two	1	4
	2	5
	3	7

New in version 0.20.

Index level names may be specified as keys directly to `groupby`.

```
In [52]: df.groupby(['second', 'A']).sum()
Out[52]:
```

		B
second	A	
one	1	2
	2	4
	3	6
two	1	4
	2	5
	3	7

## DataFrame column selection in GroupBy

Once you have created the GroupBy object from a DataFrame, you might want to do something different for each of the columns. Thus, using `[]` similar to getting a column from a DataFrame, you can do:

```
In [53]: grouped = df.groupby(['A'])

In [54]: grouped_C = grouped['C']

In [55]: grouped_D = grouped['D']
```

This is mainly syntactic sugar for the alternative and much more verbose:

```
In [56]: df['C'].groupby(df['A'])
Out[56]: <pandas.core.groupby.generic.SeriesGroupBy object at 0x7f65f21ac518>
```

Additionally this method avoids recomputing the internal grouping information derived from the passed key.

## Iterating through groups

With the GroupBy object in hand, iterating through the grouped data is very natural and functions similarly to `itertools.groupby()`:

```
In [57]: grouped = df.groupby('A')

In [58]: for name, group in grouped:
....: print(name)
....: print(group)
....:
bar
 A B C D
1 bar one 0.254161 1.511763
3 bar three 0.215897 -0.990582
5 bar two -0.077118 1.211526
foo
 A B C D
0 foo one -0.575247 1.346061
2 foo two -1.143704 1.627081
4 foo two 1.193555 -0.441652
6 foo one -0.408530 0.268520
7 foo three -0.862495 0.024580
```

In the case of grouping by multiple keys, the group name will be a tuple:



```
In [59]: for name, group in df.groupby(['A', 'B']):
....: print(name)
....: print(group)
....:
```

	A	B	C	D
1	bar	one	0.254161	1.511763

```
('bar', 'three')
```

	A	B	C	D
3	bar	three	0.215897	-0.990582

```
('bar', 'two')
```

	A	B	C	D
5	bar	two	-0.077118	1.211526

```
('foo', 'one')
```

	A	B	C	D
0	foo	one	-0.575247	1.346061
6	foo	one	-0.408530	0.268520

```
('foo', 'three')
```

	A	B	C	D
7	foo	three	-0.862495	0.02458

```
('foo', 'two')
```

	A	B	C	D
2	foo	two	-1.143704	1.627081
4	foo	two	1.193555	-0.441652

See [Iterating through groups](#).

## Selecting a group

A single group can be selected using `get_group()`:

```
In [60]: grouped.get_group('bar')
Out[60]:
```

	A	B	C	D
1	bar	one	0.254161	1.511763
3	bar	three	0.215897	-0.990582
5	bar	two	-0.077118	1.211526

Or for an object grouped on multiple columns:

```
In [61]: df.groupby(['A', 'B']).get_group(('bar', 'one'))
Out[61]:
```

	A	B	C	D
1	bar	one	0.254161	1.511763

## Aggregation

Once the GroupBy object has been created, several methods are available to perform a computation on the grouped data. These operations are similar to the [aggregating API](#), [window functions API](#), and [resample API](#).

An obvious one is aggregation via the `aggregate()` or equivalently `agg()` method:

```
In [62]: grouped = df.groupby('A')

In [63]: grouped.agg(np.sum)
Out[63]:
```

	C	D
A		
bar	0.392940	1.732707
foo	-1.796421	2.824590

```
In [64]: grouped = df.groupby(['A', 'B'])

In [65]: grouped.agg(np.sum)
Out[65]:
```

		C	D
A	B		
bar	one	0.254161	1.511763
	three	0.215897	-0.990582
	two	-0.077118	1.211526
foo	one	-0.983776	1.614581
	three	-0.862495	0.024580
	two	0.049851	1.185429

As you can see, the result of the aggregation will have the group names as the new index along the grouped axis. In the case of multiple keys, the result is a [MultiIndex](#) by default, though this can be changed by using the `as_index` option:

```
In [66]: grouped = df.groupby(['A', 'B'], as_index=False)

In [67]: grouped.agg(np.sum)
Out[67]:
```

	A	B	C	D
0	bar	one	0.254161	1.511763
1	bar	three	0.215897	-0.990582
2	bar	two	-0.077118	1.211526
3	foo	one	-0.983776	1.614581
4	foo	three	-0.862495	0.024580
5	foo	two	0.049851	1.185429

```
In [68]: df.groupby('A', as_index=False).sum()
Out[68]:
```

	A	C	D
0	bar	0.392940	1.732707
1	foo	-1.796421	2.824590

Note that you could use the `reset_index` DataFrame function to achieve the same result as the column names are stored in the resulting MultiIndex:

```
In [69]: df.groupby(['A', 'B']).sum().reset_index()
Out[69]:
```

	A	B	C	D
0	bar	one	0.254161	1.511763
1	bar	three	0.215897	-0.990582
2	bar	two	-0.077118	1.211526
3	foo	one	-0.983776	1.614581
4	foo	three	-0.862495	0.024580
5	foo	two	0.049851	1.185429

Another simple aggregation example is to compute the size of each group. This is included in GroupBy as the `size` method. It returns a Series whose index are the group names and whose values are the sizes of each group.

```
In [70]: grouped.size()
Out[70]:
```

A	B	
bar	one	1
	three	1
	two	1
foo	one	2
	three	1
	two	2

dtype: int64

```
In [71]: grouped.describe()
Out[71]:
```

		C								D		
	count	mean	std	min	25%	50%	75%	max	count	mean		
		std	min	25%	50%	75%	max					
0	1.0	0.254161	NaN	0.254161	0.254161	0.254161	0.254161	0.254161	1.0	1.511763	NaN	
		1.511763	1.511763	1.511763	1.511763	1.511763	1.511763					
1	1.0	0.215897	NaN	0.215897	0.215897	0.215897	0.215897	0.215897	1.0	-0.990582	NaN	
		-0.990582	-0.990582	-0.990582	-0.990582	-0.990582	-0.990582					
2	1.0	-0.077118	NaN	-0.077118	-0.077118	-0.077118	-0.077118	-0.077118	1.0	1.211526	NaN	
		1.211526	1.211526	1.211526	1.211526	1.211526	1.211526					
3	2.0	-0.491888	0.117887	-0.575247	-0.533567	-0.491888	-0.450209	-0.408530	2.0	0.807291	0.761937	
		0.268520	0.537905	0.807291	1.076676	1.346061						
4	1.0	-0.862495	NaN	-0.862495	-0.862495	-0.862495	-0.862495	-0.862495	1.0	0.024580	NaN	
		0.024580	0.024580	0.024580	0.024580	0.024580						
5	2.0	0.024925	1.652692	-1.143704	-0.559389	0.024925	0.609240	1.193555	2.0	0.592714	1.462816	
		-0.441652	0.075531	0.592714	1.109898	1.627081						

::: tip Note

Aggregation functions **will not** return the groups that you are aggregating over if they are named columns, when `as_index=True`, the default. The grouped columns will be the **indices** of the returned object.

Passing `as_index=False` **will** return the groups that you are aggregating over, if they are named columns.

...

Aggregating functions are the ones that reduce the dimension of the returned objects. Some common aggregating functions are tabulated below:

Function	Description
<code>mean()</code>	Compute mean of groups
<code>sum()</code>	Compute sum of group values
<code>size()</code>	Compute group sizes
<code>count()</code>	Compute count of group
<code>std()</code>	Standard deviation of groups
<code>var()</code>	Compute variance of groups
<code>sem()</code>	Standard error of the mean of groups
<code>describe()</code>	Generates descriptive statistics
<code>first()</code>	Compute first of group values
<code>last()</code>	Compute last of group values
<code>nth()</code>	Take nth value, or a subset if n is a list
<code>min()</code>	Compute min of group values
<code>max()</code>	Compute max of group values

The aggregating functions above will exclude NA values. Any function which reduces a [Series](#) to a scalar value is an aggregation function and will work, a trivial example is

`df.groupby('A').agg(lambda ser: 1)`. Note that `nth()` can act as a reducer or a filter, see [here](#).

## Applying multiple functions at once

With grouped [Series](#) you can also pass a list or dict of functions to do aggregation with, outputting a [DataFrame](#):

```
In [72]: grouped = df.groupby('A')

In [73]: grouped['C'].agg([np.sum, np.mean, np.std])
Out[73]:
```

	sum	mean	std
A			
bar	0.392940	0.130980	0.181231
foo	-1.796421	-0.359284	0.912265

On a grouped `DataFrame`, you can pass a list of functions to apply to each column, which produces an aggregated result with a hierarchical index:

```
In [74]: grouped.agg([np.sum, np.mean, np.std])
Out[74]:
```

	C			D		
	sum	mean	std	sum	mean	std
A						
bar	0.392940	0.130980	0.181231	1.732707	0.577569	1.366330
foo	-1.796421	-0.359284	0.912265	2.824590	0.564918	0.884785

The resulting aggregations are named for the functions themselves. If you need to rename, then you can add in a chained operation for a `Series` like this:

```
In [75]: (grouped['C'].agg([np.sum, np.mean, np.std])
....: .rename(columns={'sum': 'foo',
....: 'mean': 'bar',
....: 'std': 'baz'}))
Out[75]:
```

	foo	bar	baz
A			
bar	0.392940	0.130980	0.181231
foo	-1.796421	-0.359284	0.912265

For a grouped `DataFrame`, you can rename in a similar manner:

```
In [76]: (grouped.agg([np.sum, np.mean, np.std])
....: .rename(columns={'sum': 'foo',
....: 'mean': 'bar',
....: 'std': 'baz'}))
Out[76]:
```

	C			D		
	foo	bar	baz	foo	bar	baz
A						
bar	0.392940	0.130980	0.181231	1.732707	0.577569	1.366330
foo	-1.796421	-0.359284	0.912265	2.824590	0.564918	0.884785

::: tip Note

In general, the output column names should be unique. You can't apply the same function (or two functions with the same name) to the same column.

```

In [77]: grouped['C'].agg(['sum', 'sum'])

SpecificationError Traceback (most recent call last)
<ipython-input-77-7be02859f395> in <module>
----> 1 grouped['C'].agg(['sum', 'sum'])

/pandas/pandas/core/groupby/generic.py in aggregate(self, func_or_funcs, *args, **kwargs)
 849 # but not the class list / tuple itself.
 850 func_or_funcs = _maybe_mangle_lambdas(func_or_funcs)
--> 851 ret = self._aggregate_multiple_funcs(func_or_funcs, (_level or 0) + 1)
 852 if relabeling:
 853 ret.columns = columns

/pandas/pandas/core/groupby/generic.py in _aggregate_multiple_funcs(self, arg, _level)
 919 raise SpecificationError(
 920 "Function names must be unique, found multiple named "
--> 921 "{}".format(name)
 922)
 923

SpecificationError: Function names must be unique, found multiple named sum

```

Pandas does allow you to provide multiple lambdas. In this case, pandas will mangle the name of the (nameless) lambda functions, appending `_` to each subsequent lambda.

```

In [78]: grouped['C'].agg([lambda x: x.max() - x.min(),
....: lambda x: x.median() - x.mean()])
....:
Out[78]:
 <lambda_0> <lambda_1>
A
bar 0.331279 0.084917
foo 2.337259 -0.215962

```

...

## Named aggregation

New in version 0.25.0.

To support column-specific aggregation with control over the output column names, pandas accepts the special syntax in `GroupBy.agg()`, known as “named aggregation”, where

- The keywords are the output column names
- The values are tuples whose first element is the column to select and the second element is the aggregation to apply to that column. Pandas provides the `pandas.NamedAgg` namedtuple with the fields `['column', 'aggfunc']` to make it clearer what the arguments are. As usual, the aggregation can be a callable or a string alias.

```
In [79]: animals = pd.DataFrame({'kind': ['cat', 'dog', 'cat', 'dog'],
....: 'height': [9.1, 6.0, 9.5, 34.0],
....: 'weight': [7.9, 7.5, 9.9, 198.0]})
....:

In [80]: animals
Out[80]:
 kind height weight
0 cat 9.1 7.9
1 dog 6.0 7.5
2 cat 9.5 9.9
3 dog 34.0 198.0

In [81]: animals.groupby("kind").agg(
....: min_height=pd.NamedAgg(column='height', aggfunc='min'),
....: max_height=pd.NamedAgg(column='height', aggfunc='max'),
....: average_weight=pd.NamedAgg(column='weight', aggfunc=np.mean),
....:)
....:
Out[81]:
 min_height max_height average_weight
kind
cat 9.1 9.5 8.90
dog 6.0 34.0 102.75
```

`pandas.NamedAgg` is just a `namedtuple`. Plain tuples are allowed as well.

```
In [82]: animals.groupby("kind").agg(
....: min_height=('height', 'min'),
....: max_height=('height', 'max'),
....: average_weight=('weight', np.mean),
....:)
....:
Out[82]:
 min_height max_height average_weight
kind
cat 9.1 9.5 8.90
dog 6.0 34.0 102.75
```

If your desired output column names are not valid python keywords, construct a dictionary and unpack the keyword arguments

```
In [83]: animals.groupby("kind").agg(**{
....: 'total_weight': pd.NamedAgg(column='weight', aggfunc=sum),
....: })
....:
Out[83]:
 total_weight
kind
cat 17.8
dog 205.5
```

Additional keyword arguments are not passed through to the aggregation functions. Only pairs of (column, aggfunc) should be passed as `**kwargs`. If your aggregation functions requires additional arguments, partially apply them with `functools.partial()`.

::: tip Note

For Python 3.5 and earlier, the order of `**kwargs` in a functions was not preserved. This means that the output column ordering would not be consistent. To ensure consistent ordering, the keys (and so output columns) will always be sorted for Python 3.5.

:::

Named aggregation is also valid for Series groupby aggregations. In this case there's no column selection, so the values are just the functions.

```
In [84]: animals.groupby("kind").height.agg(
....: min_height='min',
....: max_height='max',
....:)
....:
Out[84]:
 min_height max_height
kind
cat 9.1 9.5
dog 6.0 34.0
```

## Applying different functions to DataFrame columns

By passing a dict to `aggregate` you can apply a different aggregation to the columns of a DataFrame:

```
In [85]: grouped.agg({'C': np.sum,
....: 'D': lambda x: np.std(x, ddof=1)})
....:
Out[85]:
 C D
A
bar 0.392940 1.366330
foo -1.796421 0.884785
```

The function names can also be strings. In order for a string to be valid it must be either implemented on GroupBy or available via [dispatching](#):

```
In [86]: grouped.agg({'C': 'sum', 'D': 'std'})
Out[86]:
 C D
A
bar 0.392940 1.366330
foo -1.796421 0.884785
```



## Cython-optimized aggregation functions

Some common aggregations, currently only `sum`, `mean`, `std`, and `sem`, have optimized Cython implementations:

```
In [87]: df.groupby('A').sum()
Out[87]:
```

	C	D
A		
bar	0.392940	1.732707
foo	-1.796421	2.824590

```
In [88]: df.groupby(['A', 'B']).mean()
Out[88]:
```

		C	D
A	B		
bar	one	0.254161	1.511763
	three	0.215897	-0.990582
	two	-0.077118	1.211526
foo	one	-0.491888	0.807291
	three	-0.862495	0.024580
	two	0.024925	0.592714

Of course `sum` and `mean` are implemented on pandas objects, so the above code would work even without the special versions via dispatching (see below).

## Transformation

The `transform` method returns an object that is indexed the same (same size) as the one being grouped. The transform function must:

- Return a result that is either the same size as the group chunk or broadcastable to the size of the group chunk (e.g., a scalar, `grouped.transform(lambda x: x.iloc[-1])`).
- Operate column-by-column on the group chunk. The transform is applied to the first group chunk using `chunk.apply`.
- Not perform in-place operations on the group chunk. Group chunks should be treated as immutable, and changes to a group chunk may produce unexpected results. For example, when using `fillna`, `inplace` must be `False` (`grouped.transform(lambda x: x.fillna(inplace=False))`).
- (Optionally) operates on the entire group chunk. If this is supported, a fast path is used starting from the second chunk.

For example, suppose we wished to standardize the data within each group:

```

In [89]: index = pd.date_range('10/1/1999', periods=1100)

In [90]: ts = pd.Series(np.random.normal(0.5, 2, 1100), index)

In [91]: ts = ts.rolling(window=100, min_periods=100).mean().dropna()

In [92]: ts.head()
Out[92]:
2000-01-08 0.779333
2000-01-09 0.778852
2000-01-10 0.786476
2000-01-11 0.782797
2000-01-12 0.798110
Freq: D, dtype: float64

In [93]: ts.tail()
Out[93]:
2002-09-30 0.660294
2002-10-01 0.631095
2002-10-02 0.673601
2002-10-03 0.709213
2002-10-04 0.719369
Freq: D, dtype: float64

In [94]: transformed = (ts.groupby(lambda x: x.year)
.....: .transform(lambda x: (x - x.mean()) / x.std()))
.....:

```

We would expect the result to now have mean 0 and standard deviation 1 within each group, which we can easily check:

```

Original Data
In [95]: grouped = ts.groupby(lambda x: x.year)

In [96]: grouped.mean()
Out[96]:
2000 0.442441
2001 0.526246
2002 0.459365
dtype: float64

In [97]: grouped.std()
Out[97]:
2000 0.131752
2001 0.210945
2002 0.128753
dtype: float64

Transformed Data
In [98]: grouped_trans = transformed.groupby(lambda x: x.year)

In [99]: grouped_trans.mean()
Out[99]:
2000 1.168208e-15
2001 1.454544e-15
2002 1.726657e-15
dtype: float64

In [100]: grouped_trans.std()
Out[100]:
2000 1.0
2001 1.0
2002 1.0
dtype: float64

```

We can also visually compare the original and transformed data sets.

```

In [101]: compare = pd.DataFrame({'Original': ts, 'Transformed': transformed})

In [102]: compare.plot()
Out[102]: <matplotlib.axes._subplots.AxesSubplot at 0x7f65f731cba8>

```

groupby transform plot

Transformation functions that have lower dimension outputs are broadcast to match the shape of the input array.

```

In [103]: ts.groupby(lambda x: x.year).transform(lambda x: x.max() - x.min())
Out[103]:
2000-01-08 0.623893
2000-01-09 0.623893
2000-01-10 0.623893
2000-01-11 0.623893
2000-01-12 0.623893
...
2002-09-30 0.558275
2002-10-01 0.558275
2002-10-02 0.558275
2002-10-03 0.558275
2002-10-04 0.558275
Freq: D, Length: 1001, dtype: float64

```

Alternatively, the built-in methods could be used to produce the same outputs.

```

In [104]: max = ts.groupby(lambda x: x.year).transform('max')

In [105]: min = ts.groupby(lambda x: x.year).transform('min')

In [106]: max - min
Out[106]:
2000-01-08 0.623893
2000-01-09 0.623893
2000-01-10 0.623893
2000-01-11 0.623893
2000-01-12 0.623893
...
2002-09-30 0.558275
2002-10-01 0.558275
2002-10-02 0.558275
2002-10-03 0.558275
2002-10-04 0.558275
Freq: D, Length: 1001, dtype: float64

```

Another common data transform is to replace missing data with the group mean.

```

In [107]: data_df
Out[107]:
 A B C
0 1.539708 -1.166480 0.533026
1 1.302092 -0.505754 NaN
2 -0.371983 1.104803 -0.651520
3 -1.309622 1.118697 -1.161657
4 -1.924296 0.396437 0.812436
..
995 -0.093110 0.683847 -0.774753
996 -0.185043 1.438572 NaN
997 -0.394469 -0.642343 0.011374
998 -1.174126 1.857148 NaN
999 0.234564 0.517098 0.393534

[1000 rows x 3 columns]

In [108]: countries = np.array(['US', 'UK', 'GR', 'JP'])

In [109]: key = countries[np.random.randint(0, 4, 1000)]

In [110]: grouped = data_df.groupby(key)

Non-NA count in each group
In [111]: grouped.count()
Out[111]:
 A B C
GR 209 217 189
JP 240 255 217
UK 216 231 193
US 239 250 217

In [112]: transformed = grouped.transform(lambda x: x.fillna(x.mean()))

```

We can verify that the group means have not changed in the transformed data and that the transformed data contains no NAs.

```

In [113]: grouped_trans = transformed.groupby(key)

In [114]: grouped.mean() # original group means
Out[114]:
 A B C
GR -0.098371 -0.015420 0.068053
JP 0.069025 0.023100 -0.077324
UK 0.034069 -0.052580 -0.116525
US 0.058664 -0.020399 0.028603

In [115]: grouped_trans.mean() # transformation did not change group means
Out[115]:
 A B C
GR -0.098371 -0.015420 0.068053
JP 0.069025 0.023100 -0.077324
UK 0.034069 -0.052580 -0.116525
US 0.058664 -0.020399 0.028603

In [116]: grouped.count() # original has some missing data points
Out[116]:
 A B C
GR 209 217 189
JP 240 255 217
UK 216 231 193
US 239 250 217

In [117]: grouped_trans.count() # counts after transformation
Out[117]:
 A B C
GR 228 228 228
JP 267 267 267
UK 247 247 247
US 258 258 258

In [118]: grouped_trans.size() # Verify non-NA count equals group size
Out[118]:
GR 228
JP 267
UK 247
US 258
dtype: int64

```

### ::: tip Note

Some functions will automatically transform the input when applied to a GroupBy object, but returning an object of the same shape as the original. Passing `as_index=False` will not affect these transformation methods.

For example: `fillna`, `ffill`, `bfill`, `shift`.

```
In [119]: grouped.ffmpeg()
Out[119]:
```

	A	B	C
0	1.539708	-1.166480	0.533026
1	1.302092	-0.505754	0.533026
2	-0.371983	1.104803	-0.651520
3	-1.309622	1.118697	-1.161657
4	-1.924296	0.396437	0.812436
..	...	...	...
995	-0.093110	0.683847	-0.774753
996	-0.185043	1.438572	-0.774753
997	-0.394469	-0.642343	0.011374
998	-1.174126	1.857148	-0.774753
999	0.234564	0.517098	0.393534

```
[1000 rows x 3 columns]
```

...

## New syntax to window and resample operations

New in version 0.18.1.

Working with the resample, expanding or rolling operations on the groupby level used to require the application of helper functions. However, now it is possible to use `resample()`, `expanding()` and `rolling()` as methods on groupbys.

The example below will apply the `rolling()` method on the samples of the column B based on the groups of column A.

```
In [120]: df_re = pd.DataFrame({'A': [1] * 10 + [5] * 10,
.....: 'B': np.arange(20)})
.....:
```

```
In [121]: df_re
```

```
Out[121]:
```

	A	B
0	1	0
1	1	1
2	1	2
3	1	3
4	1	4
..	..	..
15	5	15
16	5	16
17	5	17
18	5	18
19	5	19

```
[20 rows x 2 columns]
```

```
In [122]: df_re.groupby('A').rolling(4).B.mean()
```

```
Out[122]:
```

A		
1	0	NaN
	1	NaN
	2	NaN
	3	1.5
	4	2.5
		...
5	15	13.5
	16	14.5
	17	15.5
	18	16.5
	19	17.5

```
Name: B, Length: 20, dtype: float64
```

The `expanding()` method will accumulate a given operation (`sum()` in the example) for all the members of each particular group.



```
In [123]: df_re.groupby('A').expanding().sum()
```

```
Out[123]:
```

	A	B
A		
1 0	1.0	0.0
1	2.0	1.0
2	3.0	3.0
3	4.0	6.0
4	5.0	10.0
...	...	...
5 15	30.0	75.0
16	35.0	91.0
17	40.0	108.0
18	45.0	126.0
19	50.0	145.0

```
[20 rows x 2 columns]
```

Suppose you want to use the `resample()` method to get a daily frequency in each group of your dataframe and wish to complete the missing values with the `ffill()` method.

```
In [124]: df_re = pd.DataFrame({'date': pd.date_range(start='2016-01-01', periods=4,
.....: freq='W'),
.....: 'group': [1, 1, 2, 2],
.....: 'val': [5, 6, 7, 8]}).set_index('date')
.....:
```

```
In [125]: df_re
```

```
Out[125]:
```

	group	val
date		
2016-01-03	1	5
2016-01-10	1	6
2016-01-17	2	7
2016-01-24	2	8

```
In [126]: df_re.groupby('group').resample('1D').ffill()
```

```
Out[126]:
```

	group	date	val
1		2016-01-03	5
		2016-01-04	5
		2016-01-05	5
		2016-01-06	5
		2016-01-07	5
...		...	...
2		2016-01-20	7
		2016-01-21	7
		2016-01-22	7
		2016-01-23	7
		2016-01-24	8

```
[16 rows x 4 columns]
```

## Filtration

The `filter` method returns a subset of the original object. Suppose we want to take only elements that belong to groups with a group sum greater than 2.

```
In [127]: sf = pd.Series([1, 1, 2, 3, 3, 3])

In [128]: sf.groupby(sf).filter(lambda x: x.sum() > 2)
Out[128]:
3 3
4 3
5 3
dtype: int64
```

The argument of `filter` must be a function that, applied to the group as a whole, returns `True` or `False`.

Another useful operation is filtering out elements that belong to groups with only a couple members.

```
In [129]: dff = pd.DataFrame({'A': np.arange(8), 'B': list('aabbbbcc')})

In [130]: dff.groupby('B').filter(lambda x: len(x) > 2)
Out[130]:
 A B
2 2 b
3 3 b
4 4 b
5 5 b
```

Alternatively, instead of dropping the offending groups, we can return a like-indexed objects where the groups that do not pass the filter are filled with NaNs.

```
In [131]: dff.groupby('B').filter(lambda x: len(x) > 2, dropna=False)
Out[131]:
 A B
0 NaN NaN
1 NaN NaN
2 2.0 b
3 3.0 b
4 4.0 b
5 5.0 b
6 NaN NaN
7 NaN NaN
```

For DataFrames with multiple columns, filters should explicitly specify a column as the filter criterion.

```
In [132]: dff['C'] = np.arange(8)

In [133]: dff.groupby('B').filter(lambda x: len(x['C']) > 2)
Out[133]:
```

	A	B	C
2	2	b	2
3	3	b	3
4	4	b	4
5	5	b	5

⋮ tip Note

Some functions when applied to a groupby object will act as a **filter** on the input, returning a reduced shape of the original (and potentially eliminating groups), but with the index unchanged. Passing `as_index=False` will not affect these transformation methods.

For example: `head`, `tail`.

```
In [134]: dff.groupby('B').head(2)
Out[134]:
```

	A	B	C
0	0	a	0
1	1	a	1
2	2	b	2
3	3	b	3
6	6	c	6
7	7	c	7

⋮

## Dispatching to instance methods

When doing an aggregation or transformation, you might just want to call an instance method on each data group. This is pretty easy to do by passing lambda functions:

```
In [135]: grouped = df.groupby('A')

In [136]: grouped.agg(lambda x: x.std())
Out[136]:
```

	C	D
A		
bar	0.181231	1.366330
foo	0.912265	0.884785

But, it's rather verbose and can be untidy if you need to pass additional arguments. Using a bit of metaprogramming cleverness, GroupBy now has the ability to “dispatch” method calls to the groups:

```
In [137]: grouped.std()
Out[137]:
```

	C	D
A		
bar	0.181231	1.366330
foo	0.912265	0.884785

What is actually happening here is that a function wrapper is being generated. When invoked, it takes any passed arguments and invokes the function with any arguments on each group (in the above example, the `std` function). The results are then combined together much in the style of `agg` and `transform` (it actually uses `apply` to infer the gluing, documented next). This enables some operations to be carried out rather succinctly:

```
In [138]: tsdf = pd.DataFrame(np.random.randn(1000, 3),
.....: index=pd.date_range('1/1/2000', periods=1000),
.....: columns=['A', 'B', 'C'])
.....:

In [139]: tsdf.iloc[:,2] = np.nan

In [140]: grouped = tsdf.groupby(lambda x: x.year)

In [141]: grouped.fillna(method='pad')
Out[141]:
```

	A	B	C
2000-01-01	NaN	NaN	NaN
2000-01-02	-0.353501	-0.080957	-0.876864
2000-01-03	-0.353501	-0.080957	-0.876864
2000-01-04	0.050976	0.044273	-0.559849
2000-01-05	0.050976	0.044273	-0.559849
...	...	...	...
2002-09-22	0.005011	0.053897	-1.026922
2002-09-23	0.005011	0.053897	-1.026922
2002-09-24	-0.456542	-1.849051	1.559856
2002-09-25	-0.456542	-1.849051	1.559856
2002-09-26	1.123162	0.354660	1.128135

[1000 rows x 3 columns]

In this example, we chopped the collection of time series into yearly chunks then independently called `fillna` on the groups.

The `nlargest` and `nsmallest` methods work on `Series` style groupbys:

```
In [142]: s = pd.Series([9, 8, 7, 5, 19, 1, 4.2, 3.3])
```

```
In [143]: g = pd.Series(list('abababab'))
```

```
In [144]: gb = s.groupby(g)
```

```
In [145]: gb.nlargest(3)
```

```
Out[145]:
```

```
a 4 19.0
```

```
 0 9.0
```

```
 2 7.0
```

```
b 1 8.0
```

```
 3 5.0
```

```
 7 3.3
```

```
dtype: float64
```

```
In [146]: gb.nsmallest(3)
```

```
Out[146]:
```

```
a 6 4.2
```

```
 2 7.0
```

```
 0 9.0
```

```
b 5 1.0
```

```
 7 3.3
```

```
 3 5.0
```

```
dtype: float64
```

## Flexible `apply`

Some operations on the grouped data might not fit into either the aggregate or transform categories.

Or, you may simply want `GroupBy` to infer how to combine the results. For these, use the `apply` function, which can be substituted for both `aggregate` and `transform` in many standard use cases.

However, `apply` can handle some exceptional use cases, for example:

```

In [147]: df
Out[147]:
 A B C D
0 foo one -0.575247 1.346061
1 bar one 0.254161 1.511763
2 foo two -1.143704 1.627081
3 bar three 0.215897 -0.990582
4 foo two 1.193555 -0.441652
5 bar two -0.077118 1.211526
6 foo one -0.408530 0.268520
7 foo three -0.862495 0.024580

In [148]: grouped = df.groupby('A')

could also just call .describe()
In [149]: grouped['C'].apply(lambda x: x.describe())
Out[149]:
A
bar count 3.000000
 mean 0.130980
 std 0.181231
 min -0.077118
 25% 0.069390
 ...
foo min -1.143704
 25% -0.862495
 50% -0.575247
 75% -0.408530
 max 1.193555
Name: C, Length: 16, dtype: float64

```

The dimension of the returned result can also change:

```

In [150]: grouped = df.groupby('A')['C']

In [151]: def f(group):
.....: return pd.DataFrame({'original': group,
.....: 'demeaned': group - group.mean()})
.....:

In [152]: grouped.apply(f)
Out[152]:
 original demeaned
0 -0.575247 -0.215962
1 0.254161 0.123181
2 -1.143704 -0.784420
3 0.215897 0.084917
4 1.193555 1.552839
5 -0.077118 -0.208098
6 -0.408530 -0.049245
7 -0.862495 -0.503211

```

`apply` on a Series can operate on a returned value from the applied function, that is itself a series, and possibly upcast the result to a DataFrame:

```
In [153]: def f(x):
.....: return pd.Series([x, x ** 2], index=['x', 'x^2'])
.....:

In [154]: s = pd.Series(np.random.rand(5))

In [155]: s
Out[155]:
0 0.321438
1 0.493496
2 0.139505
3 0.910103
4 0.194158
dtype: float64

In [156]: s.apply(f)
Out[156]:
 x x^2
0 0.321438 0.103323
1 0.493496 0.243538
2 0.139505 0.019462
3 0.910103 0.828287
4 0.194158 0.037697
```

::: tip Note

`apply` can act as a reducer, transformer, or filter function, depending on exactly what is passed to it. So depending on the path taken, and exactly what you are grouping. Thus the grouped columns(s) may be included in the output as well as set the indices.

:::

## Other useful features

---

### Automatic exclusion of “nuisance” columns

Again consider the example DataFrame we’ve been looking at:

```
In [157]: df
Out[157]:
```

	A	B	C	D
0	foo	one	-0.575247	1.346061
1	bar	one	0.254161	1.511763
2	foo	two	-1.143704	1.627081
3	bar	three	0.215897	-0.990582
4	foo	two	1.193555	-0.441652
5	bar	two	-0.077118	1.211526
6	foo	one	-0.408530	0.268520
7	foo	three	-0.862495	0.024580

Suppose we wish to compute the standard deviation grouped by the `A` column. There is a slight problem, namely that we don't care about the data in column `B`. We refer to this as a “nuisance” column. If the passed aggregation function can't be applied to some columns, the troublesome columns will be (silently) dropped. Thus, this does not pose any problems:

```
In [158]: df.groupby('A').std()
Out[158]:
```

	C	D
A		
bar	0.181231	1.366330
foo	0.912265	0.884785

Note that `df.groupby('A').colname.std().` is more efficient than `df.groupby('A').std().colname`, so if the result of an aggregation function is only interesting over one column (here `colname`), it may be filtered before applying the aggregation function.

::: tip Note

Any object column, also if it contains numerical values such as `Decimal` objects, is considered as a “nuisance” columns. They are excluded from aggregate functions automatically in `groupby`.

If you do wish to include decimal or object columns in an aggregation with other non-nuisance data types, you must do so explicitly.

:::



```

In [159]: from decimal import Decimal

In [160]: df_dec = pd.DataFrame(
.....: {'id': [1, 2, 1, 2],
.....: 'int_column': [1, 2, 3, 4],
.....: 'dec_column': [Decimal('0.50'), Decimal('0.15'),
.....: Decimal('0.25'), Decimal('0.40')]}
.....:)
.....:

Decimal columns can be sum'd explicitly by themselves...
In [161]: df_dec.groupby(['id'])['dec_column'].sum()
Out[161]:
dec_column
id
1 0.75
2 0.55

...but cannot be combined with standard data types or they will be excluded
In [162]: df_dec.groupby(['id'])['int_column', 'dec_column'].sum()
Out[162]:
int_column
id
1 4
2 6

Use .agg function to aggregate over standard and "nuisance" data types
at the same time
In [163]: df_dec.groupby(['id']).agg({'int_column': 'sum', 'dec_column': 'sum'})
Out[163]:
int_column dec_column
id
1 4 0.75
2 6 0.55

```

## Handling of (un)observed Categorical values

When using a `Categorical` grouper (as a single grouper, or as part of multiple groupers), the `observed` keyword controls whether to return a cartesian product of all possible groupers values (`observed=False`) or only those that are observed groupers (`observed=True`).

Show all values:

```

In [164]: pd.Series([1, 1, 1]).groupby(pd.Categorical(['a', 'a', 'a'],
.....: categories=['a', 'b']),
.....: observed=False).count()
.....:
Out[164]:
a 3
b 0
dtype: int64

```

Show only the observed values:

```
In [165]: pd.Series([1, 1, 1]).groupby(pd.Categorical(['a', 'a', 'a'],
.....: categories=['a', 'b']),
.....: observed=True).count()
.....:
Out[165]:
a 3
dtype: int64
```

The returned dtype of the grouped will always include all of the categories that were grouped.

```
In [166]: s = pd.Series([1, 1, 1]).groupby(pd.Categorical(['a', 'a', 'a'],
.....: categories=['a', 'b']),
.....: observed=False).count()
.....:

In [167]: s.index.dtype
Out[167]: CategoricalDtype(categories=['a', 'b'], ordered=False)
```

## NA and NaT group handling

If there are any NaN or NaT values in the grouping key, these will be automatically excluded. In other words, there will never be an “NA group” or “NaT group”. This was not the case in older versions of pandas, but users were generally discarding the NA group anyway (and supporting it was an implementation headache).

## Grouping with ordered factors

Categorical variables represented as instance of pandas’ `Categorical` class can be used as group keys. If so, the order of the levels will be preserved:

```
In [168]: data = pd.Series(np.random.randn(100))

In [169]: factor = pd.qcut(data, [0, .25, .5, .75, 1.])

In [170]: data.groupby(factor).mean()
Out[170]:
(-2.645, -0.523] -1.362896
(-0.523, 0.0296] -0.260266
(0.0296, 0.654] 0.361802
(0.654, 2.21] 1.073801
dtype: float64
```

## Grouping with a grouper specification

You may need to specify a bit more data to properly group. You can use the `pd.Grouper` to provide this local control.

```
In [171]: import datetime

In [172]: df = pd.DataFrame({'Branch': 'A A A A A A B'.split(),
.....: 'Buyer': 'Carl Mark Carl Carl Joe Joe Joe Carl'.split(),
.....: 'Quantity': [1, 3, 5, 1, 8, 1, 9, 3],
.....: 'Date': [
.....: datetime.datetime(2013, 1, 1, 13, 0),
.....: datetime.datetime(2013, 1, 1, 13, 5),
.....: datetime.datetime(2013, 10, 1, 20, 0),
.....: datetime.datetime(2013, 10, 2, 10, 0),
.....: datetime.datetime(2013, 10, 1, 20, 0),
.....: datetime.datetime(2013, 10, 2, 10, 0),
.....: datetime.datetime(2013, 12, 2, 12, 0),
.....: datetime.datetime(2013, 12, 2, 14, 0)]
.....: })
.....:
```

```
In [173]: df
Out[173]:
```

	Branch	Buyer	Quantity	Date
0	A	Carl	1	2013-01-01 13:00:00
1	A	Mark	3	2013-01-01 13:05:00
2	A	Carl	5	2013-10-01 20:00:00
3	A	Carl	1	2013-10-02 10:00:00
4	A	Joe	8	2013-10-01 20:00:00
5	A	Joe	1	2013-10-02 10:00:00
6	A	Joe	9	2013-12-02 12:00:00
7	B	Carl	3	2013-12-02 14:00:00

Groupby a specific column with the desired frequency. This is like resampling.

```
In [174]: df.groupby([pd.Grouper(freq='1M', key='Date'), 'Buyer']).sum()
Out[174]:
```

Date	Buyer	Quantity
2013-01-31	Carl	1
	Mark	3
2013-10-31	Carl	6
	Joe	9
2013-12-31	Carl	3
	Joe	9

You have an ambiguous specification in that you have a named index and a column that could be potential groupers.

```
In [175]: df = df.set_index('Date')

In [176]: df['Date'] = df.index + pd.offsets.MonthEnd(2)

In [177]: df.groupby([pd.Grouper(freq='6M', key='Date'), 'Buyer']).sum()
Out[177]:
```

Date	Buyer	Quantity
2013-02-28	Carl	1
	Mark	3
2014-02-28	Carl	9
	Joe	18

```
In [178]: df.groupby([pd.Grouper(freq='6M', level='Date'), 'Buyer']).sum()
Out[178]:
```

Date	Buyer	Quantity
2013-01-31	Carl	1
	Mark	3
2014-01-31	Carl	9
	Joe	18

## Taking the first rows of each group

Just like for a DataFrame or Series you can call head and tail on a groupby:

```
In [179]: df = pd.DataFrame([[1, 2], [1, 4], [5, 6]], columns=['A', 'B'])

In [180]: df
Out[180]:
```

	A	B
0	1	2
1	1	4
2	5	6

```
In [181]: g = df.groupby('A')

In [182]: g.head(1)
Out[182]:
```

	A	B
0	1	2
2	5	6

```
In [183]: g.tail(1)
Out[183]:
```

	A	B
1	1	4
2	5	6

This shows the first or last n rows from each group.

## Taking the nth row of each group

To select from a DataFrame or Series the nth item, use `nth()`. This is a reduction method, and will return a single row (or no row) per group if you pass an int for n:

```
In [184]: df = pd.DataFrame([[1, np.nan], [1, 4], [5, 6]], columns=['A', 'B'])
```

```
In [185]: g = df.groupby('A')
```

```
In [186]: g.nth(0)
```

```
Out[186]:
```

```
 B
A
1 NaN
5 6.0
```

```
In [187]: g.nth(-1)
```

```
Out[187]:
```

```
 B
A
1 4.0
5 6.0
```

```
In [188]: g.nth(1)
```

```
Out[188]:
```

```
 B
A
1 4.0
```

If you want to select the nth not-null item, use the `dropna` kwarg. For a DataFrame this should be either `'any'` or `'all'` just like you would pass to `dropna`:

```

nth(0) is the same as g.first()
In [189]: g.nth(0, dropna='any')
Out[189]:
 B
A
1 4.0
5 6.0

In [190]: g.first()
Out[190]:
 B
A
1 4.0
5 6.0

nth(-1) is the same as g.last()
In [191]: g.nth(-1, dropna='any') # NaNs denote group exhausted when using dropna
Out[191]:
 B
A
1 4.0
5 6.0

In [192]: g.last()
Out[192]:
 B
A
1 4.0
5 6.0

In [193]: g.B.nth(0, dropna='all')
Out[193]:
A
1 4.0
5 6.0
Name: B, dtype: float64

```

As with other methods, passing `as_index=False`, will achieve a filtration, which returns the grouped row.

```
In [194]: df = pd.DataFrame([[1, np.nan], [1, 4], [5, 6]], columns=['A', 'B'])
```

```
In [195]: g = df.groupby('A', as_index=False)
```

```
In [196]: g.nth(0)
```

```
Out[196]:
```

	A	B
0	1	NaN
2	5	6.0

```
In [197]: g.nth(-1)
```

```
Out[197]:
```

	A	B
1	1	4.0
2	5	6.0

You can also select multiple rows from each group by specifying multiple `nth` values as a list of ints.

```
In [198]: business_dates = pd.date_range(start='4/1/2014', end='6/30/2014', freq='B')
```

```
In [199]: df = pd.DataFrame(1, index=business_dates, columns=['a', 'b'])
```

```
get the first, 4th, and last date index for each month
```

```
In [200]: df.groupby([df.index.year, df.index.month]).nth([0, 3, -1])
```

```
Out[200]:
```

		a	b
2014	4	1	1
	4	1	1
	4	1	1
	5	1	1
	5	1	1
	5	1	1
	6	1	1
	6	1	1
	6	1	1

## Enumerate group items

To see the order in which each row appears within its group, use the `cumcount` method:

```
In [201]: dfg = pd.DataFrame(list('aaabba'), columns=['A'])
```

```
In [202]: dfg
```

```
Out[202]:
```

```
 A
0 a
1 a
2 a
3 b
4 b
5 a
```

```
In [203]: dfg.groupby('A').cumcount()
```

```
Out[203]:
```

```
0 0
1 1
2 2
3 0
4 1
5 3
```

```
dtype: int64
```

```
In [204]: dfg.groupby('A').cumcount(ascending=False)
```

```
Out[204]:
```

```
0 3
1 2
2 1
3 1
4 0
5 0
```

```
dtype: int64
```

## Enumerate groups

New in version 0.20.2.

To see the ordering of the groups (as opposed to the order of rows within a group given by `cumcount`) you can use `ngroup()`.

Note that the numbers given to the groups match the order in which the groups would be seen when iterating over the groupby object, not the order they are first observed.



```
In [205]: dfg = pd.DataFrame(list('aaabba'), columns=['A'])
```

```
In [206]: dfg
```

```
Out[206]:
```

```
 A
0 a
1 a
2 a
3 b
4 b
5 a
```

```
In [207]: dfg.groupby('A').ngroup()
```

```
Out[207]:
```

```
0 0
1 0
2 0
3 1
4 1
5 0
```

```
dtype: int64
```

```
In [208]: dfg.groupby('A').ngroup(ascending=False)
```

```
Out[208]:
```

```
0 1
1 1
2 1
3 0
4 0
5 1
```

```
dtype: int64
```

## Plotting

Groupby also works with some plotting methods. For example, suppose we suspect that some features in a DataFrame may differ by group, in this case, the values in column 1 where the group is “B” are 3 higher on average.

```
In [209]: np.random.seed(1234)
```

```
In [210]: df = pd.DataFrame(np.random.randn(50, 2))
```

```
In [211]: df['g'] = np.random.choice(['A', 'B'], size=50)
```

```
In [212]: df.loc[df['g'] == 'B', 1] += 3
```

We can easily visualize this with a boxplot:

```
In [213]: df.groupby('g').boxplot()
Out[213]:
A AxesSubplot(0.1,0.15;0.363636x0.75)
B AxesSubplot(0.536364,0.15;0.363636x0.75)
dtype: object
```

## groupby boxplot

The result of calling `boxplot` is a dictionary whose keys are the values of our grouping column `g` ( “A” and “B” ). The values of the resulting dictionary can be controlled by the `return_type` keyword of `boxplot` . See the [visualization documentation](#) for more.

::: danger Warning

For historical reasons, `df.groupby("g").boxplot()` is not equivalent to `df.boxplot(by="g")` . See [here](#) for an explanation.

:::

## Piping function calls

New in version 0.21.0.

Similar to the functionality provided by `DataFrame` and `Series` , functions that take `GroupBy` objects can be chained together using a `pipe` method to allow for a cleaner, more readable syntax. To read about `.pipe` in general terms, see [here](#).

Combining `.groupby` and `.pipe` is often useful when you need to reuse `GroupBy` objects.

As an example, imagine having a `DataFrame` with columns for stores, products, revenue and quantity sold. We’ d like to do a groupwise calculation of prices (i.e. revenue/quantity) per store and per product. We could do this in a multi-step operation, but expressing it in terms of piping can make the code more readable. First we set the data:

```
In [214]: n = 1000

In [215]: df = pd.DataFrame({'Store': np.random.choice(['Store_1', 'Store_2'], n),
.....: 'Product': np.random.choice(['Product_1',
.....: 'Product_2'], n),
.....: 'Revenue': (np.random.random(n) * 50 + 10).round(2),
.....: 'Quantity': np.random.randint(1, 10, size=n)})
.....:

In [216]: df.head(2)
Out[216]:
 Store Product Revenue Quantity
0 Store_2 Product_1 26.12 1
1 Store_2 Product_1 28.86 1
```

Now, to find prices per store/product, we can simply do:

```
In [217]: (df.groupby(['Store', 'Product']))
.....: .pipe(lambda grp: grp.Revenue.sum() / grp.Quantity.sum())
.....: .unstack().round(2))
.....:
Out[217]:
Product Product_1 Product_2
Store
Store_1 6.82 7.05
Store_2 6.30 6.64
```

Piping can also be expressive when you want to deliver a grouped object to some arbitrary function, for example:

```
In [218]: def mean(groupby):
.....: return groupby.mean()
.....:

In [219]: df.groupby(['Store', 'Product']).pipe(mean)
Out[219]:
 Revenue Quantity
Store Product
Store_1 Product_1 34.622727 5.075758
 Product_2 35.482815 5.029630
Store_2 Product_1 32.972837 5.237589
 Product_2 34.684360 5.224000
```

where `mean` takes a `GroupBy` object and finds the mean of the `Revenue` and `Quantity` columns respectively for each `Store-Product` combination. The `mean` function can be any function that takes in a `GroupBy` object; the `.pipe` will pass the `GroupBy` object as a parameter into the function you specify.

## Examples

---

### Regrouping by factor

Regroup columns of a `DataFrame` according to their sum, and sum the aggregated ones.

```
In [220]: df = pd.DataFrame({'a': [1, 0, 0], 'b': [0, 1, 0],
.....: 'c': [1, 0, 0], 'd': [2, 3, 4]})
.....:
```

```
In [221]: df
Out[221]:
```

	a	b	c	d
0	1	0	1	2
1	0	1	0	3
2	0	0	0	4

```
In [222]: df.groupby(df.sum(), axis=1).sum()
```

```
Out[222]:
```

1	9
0	2
1	3
2	4

## Multi-column factorization

By using `ngroup()`, we can extract information about the groups in a way similar to `factorize()` (as described further in the [reshaping API](#)) but which applies naturally to multiple columns of mixed type and different sources. This can be useful as an intermediate categorical-like step in processing, when the relationships between the group rows are more important than their content, or as input to an algorithm which only accepts the integer encoding. (For more information about support in pandas for full categorical data, see the [Categorical introduction](#) and the [API documentation](#).)

```
In [223]: dfg = pd.DataFrame({"A": [1, 1, 2, 3, 2], "B": list("aaaba")})
```

```
In [224]: dfg
```

```
Out[224]:
```

	A	B
0	1	a
1	1	a
2	2	a
3	3	b
4	2	a

```
In [225]: dfg.groupby(["A", "B"]).ngroup()
```

```
Out[225]:
```

0	0
1	0
2	1
3	2
4	1

```
dtype: int64
```

```
In [226]: dfg.groupby(["A", [0, 0, 0, 1, 1]]).ngroup()
```

```
Out[226]:
```

0	0
1	0
2	1
3	3
4	2

```
dtype: int64
```

## Groupby by indexer to ‘resample’ data

Resampling produces new hypothetical samples (resamples) from already existing observed data or from a model that generates data. These new samples are similar to the pre-existing samples.

In order to resample to work on indices that are non-datetime-like, the following procedure can be utilized.

In the following examples, **df.index // 5** returns a binary array which is used to determine what gets selected for the groupby operation.

::: tip Note

The below example shows how we can downsample by consolidation of samples into fewer samples. Here by using **df.index // 5**, we are aggregating the samples in bins. By applying **std()** function, we aggregate the information contained in many samples into a small subset of values which is their standard deviation thereby reducing the number of samples.

:::

```

In [227]: df = pd.DataFrame(np.random.randn(10, 2))

In [228]: df
Out[228]:
 0 1
0 -0.793893 0.321153
1 0.342250 1.618906
2 -0.975807 1.918201
3 -0.810847 -1.405919
4 -1.977759 0.461659
5 0.730057 -1.316938
6 -0.751328 0.528290
7 -0.257759 -1.081009
8 0.505895 -1.701948
9 -1.006349 0.020208

In [229]: df.index // 5
Out[229]: Int64Index([0, 0, 0, 0, 0, 1, 1, 1, 1, 1], dtype='int64')

In [230]: df.groupby(df.index // 5).std()
Out[230]:
 0 1
0 0.823647 1.312912
1 0.760109 0.942941

```

## Returning a Series to propagate names

Group DataFrame columns, compute a set of metrics and return a named Series. The Series name is used as the name for the column index. This is especially useful in conjunction with reshaping operations such as stacking in which the column index name will be used as the name of the inserted column:

```
In [231]: df = pd.DataFrame({'a': [0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2],
.....: 'b': [0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1],
.....: 'c': [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
.....: 'd': [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1]})
.....:
```

```
In [232]: def compute_metrics(x):
.....: result = {'b_sum': x['b'].sum(), 'c_mean': x['c'].mean()}
.....: return pd.Series(result, name='metrics')
.....:
```

```
In [233]: result = df.groupby('a').apply(compute_metrics)
```

```
In [234]: result
Out[234]:
metrics b_sum c_mean
a
0 2.0 0.5
1 2.0 0.5
2 2.0 0.5
```

```
In [235]: result.stack()
Out[235]:
a metrics
0 b_sum 2.0
 c_mean 0.5
1 b_sum 2.0
 c_mean 0.5
2 b_sum 2.0
 c_mean 0.5
dtype: float64
```

## 时间序列与日期用法

依托 NumPy 的 `datetime64`、`timedelta64` 等数据类型，pandas 可以处理各种时间序列数据，还能调用 `scikits.timeseries` 等 Python 支持库的时间序列功能。

Pandas 支持以下操作：

解析 时间格式字符串、`np.datetime64`、`datetime.datetime` 等多种时间序列数据。

```
In [1]: import datetime

In [2]: dti = pd.to_datetime(['1/1/2018', np.datetime64('2018-01-01'),
...: datetime.datetime(2018, 1, 1)])
...:

In [3]: dti
Out[3]: DatetimeIndex(['2018-01-01', '2018-01-01', '2018-01-01'], dtype='datetime64[ns]', freq=None)
```

生成 `DatetimeIndex`、`TimedeltaIndex`、`PeriodIndex` 等定频日期与时间段序列。

```
In [4]: dti = pd.date_range('2018-01-01', periods=3, freq='H')

In [5]: dti
Out[5]:
DatetimeIndex(['2018-01-01 00:00:00', '2018-01-01 01:00:00',
 '2018-01-01 02:00:00'],
 dtype='datetime64[ns]', freq='H')
```

处理、转换带时区的日期时间数据。

```
In [6]: dti = dti.tz_localize('UTC')

In [7]: dti
Out[7]:
DatetimeIndex(['2018-01-01 00:00:00+00:00', '2018-01-01 01:00:00+00:00',
 '2018-01-01 02:00:00+00:00'],
 dtype='datetime64[ns, UTC]', freq='H')

In [8]: dti.tz_convert('US/Pacific')
Out[8]:
DatetimeIndex(['2017-12-31 16:00:00-08:00', '2017-12-31 17:00:00-08:00',
 '2017-12-31 18:00:00-08:00'],
 dtype='datetime64[ns, US/Pacific]', freq='H')
```

按指定频率重采样，并转换为时间序列。

```
In [9]: idx = pd.date_range('2018-01-01', periods=5, freq='H')

In [10]: ts = pd.Series(range(len(idx)), index=idx)

In [11]: ts
Out[11]:
2018-01-01 00:00:00 0
2018-01-01 01:00:00 1
2018-01-01 02:00:00 2
2018-01-01 03:00:00 3
2018-01-01 04:00:00 4
Freq: H, dtype: int64

In [12]: ts.resample('2H').mean()
Out[12]:
2018-01-01 00:00:00 0.5
2018-01-01 02:00:00 2.5
2018-01-01 04:00:00 4.0
Freq: 2H, dtype: float64
```

用绝对或相对时间差计算日期与时间。



```

In [13]: friday = pd.Timestamp('2018-01-05')

In [14]: friday.day_name()
Out[14]: 'Friday'

添加 1 个日历日
In [15]: saturday = friday + pd.Timedelta('1 day')

In [16]: saturday.day_name()
Out[16]: 'Saturday'

添加 1 个工作日，从星期五跳到星期一
In [17]: monday = friday + pd.offsets.BDay()

In [18]: monday.day_name()
Out[18]: 'Monday'

```

pandas 提供了一组精悍、实用的工具集以完成上述操作。

## 纵览

pandas 支持 4 种常见时间概念：

1. 日期时间（Datetime）：带时区的日期时间，类似于标准库的 `datetime.datetime`。
2. 时间差（Timedelta）：绝对时间周期，类似于标准库的 `datetime.timedelta`。
3. 时间段（Timespan）：在某一时刻以指定频率定义的时间跨度。
4. 日期偏移（Dateoffset）：与日历运算对应的时间段，类似于 `dateutil` 的 `dateutil.relativedelta.relativedelta`。

时间概念	标量类	数组类	Pandas 数据类型	主要构建方法
Date times	Timestamp	DatetimeIndex	datetime64[ns] 或 datetime64[ns,tz]	to_datetime 或 date_range
Time deltas	Timedelta	TimedeltaIndex	timedelta64[ns]	to_timedelta 或 timedelta_range
Time spans	Period	PeriodIndex	period[freq]	Period 或 period_range
Date offsets	DateOffset	None	None	DateOffset

一般情况下，时间序列主要是 `Series` 或 `DataFrame` 的时间型索引，可以用时间元素进行操控。

```
In [19]: pd.Series(range(3), index=pd.date_range('2000', freq='D', periods=3))
Out[19]:
2000-01-01 0
2000-01-02 1
2000-01-03 2
Freq: D, dtype: int64
```

当然，`Series` 与 `DataFrame` 也可以直接把时间序列当成数据。

```
In [20]: pd.Series(pd.date_range('2000', freq='D', periods=3))
Out[20]:
0 2000-01-01
1 2000-01-02
2 2000-01-03
dtype: datetime64[ns]
```

`Series` 与 `DataFrame` 提供了 `datetime`、`timedelta`、`Period` 扩展类型与专有用法，不过，`Dateoffset` 则保存为 `object`。

```
In [21]: pd.Series(pd.period_range('1/1/2011', freq='M', periods=3))
Out[21]:
0 2011-01
1 2011-02
2 2011-03
dtype: period[M]

In [22]: pd.Series([pd.DateOffset(1), pd.DateOffset(2)])
Out[22]:
0 <DateOffset>
1 <2 * DateOffsets>
dtype: object

In [23]: pd.Series(pd.date_range('1/1/2011', freq='M', periods=3))
Out[23]:
0 2011-01-31
1 2011-02-28
2 2011-03-31
dtype: datetime64[ns]
```

Pandas 用 `NaT` 表示日期时间、时间差及时间段的空值，代表了缺失日期或空日期的值，类似于浮点数的 `np.nan`。

```
In [24]: pd.Timestamp(pd.NaT)
Out[24]: NaT

In [25]: pd.Timedelta(pd.NaT)
Out[25]: NaT

In [26]: pd.Period(pd.NaT)
Out[26]: NaT

与 np.nan 一样, pd.NaT 不等于 pd.NaT
In [27]: pd.NaT == pd.NaT
Out[27]: False
```

## 时间戳 vs. 时间段

时间戳是最基本的时间序列数据，用于把数值与时点关联在一起。Pandas 对象通过时间戳调用时点数据。

```
In [28]: pd.Timestamp(datetime.datetime(2012, 5, 1))
Out[28]: Timestamp('2012-05-01 00:00:00')

In [29]: pd.Timestamp('2012-05-01')
Out[29]: Timestamp('2012-05-01 00:00:00')

In [30]: pd.Timestamp(2012, 5, 1)
Out[30]: Timestamp('2012-05-01 00:00:00')
```

不过，大多数情况下，用时间段改变变量更自然。`Period` 表示的时间段更直观，还可以用日期时间格式的字符串进行推断。

示例如下：

```
In [31]: pd.Period('2011-01')
Out[31]: Period('2011-01', 'M')

In [32]: pd.Period('2012-05', freq='D')
Out[32]: Period('2012-05-01', 'D')
```

`Timestamp` 与 `Period` 可以用作索引。作为索引的 `Timestamp` 与 `Period` 列表则被强制转换为对应的 `DatetimeIndex` 与 `PeriodIndex`。

```

In [33]: dates = [pd.Timestamp('2012-05-01'),
....: pd.Timestamp('2012-05-02'),
....: pd.Timestamp('2012-05-03')]
....:

In [34]: ts = pd.Series(np.random.randn(3), dates)

In [35]: type(ts.index)
Out[35]: pandas.core.indexes.datetimes.DatetimeIndex

In [36]: ts.index
Out[36]: DatetimeIndex(['2012-05-01', '2012-05-02', '2012-05-03'], dtype='datetime64[ns]', freq=None)

In [37]: ts
Out[37]:
2012-05-01 0.469112
2012-05-02 -0.282863
2012-05-03 -1.509059
dtype: float64

In [38]: periods = [pd.Period('2012-01'), pd.Period('2012-02'), pd.Period('2012-03')]

In [39]: ts = pd.Series(np.random.randn(3), periods)

In [40]: type(ts.index)
Out[40]: pandas.core.indexes.period.PeriodIndex

In [41]: ts.index
Out[41]: PeriodIndex(['2012-01', '2012-02', '2012-03'], dtype='period[M]', freq='M')

In [42]: ts
Out[42]:
2012-01 -1.135632
2012-02 1.212112
2012-03 -0.173215
Freq: M, dtype: float64

```

Pandas 可以识别这两种表现形式，并在两者之间进行转化。Pandas 后台用 `Timestamp` 实例代表时间戳，用 `DatetimeIndex` 实例代表时间戳序列。pandas 用 `Period` 对象表示符合规律的时间段标量值，用 `PeriodIndex` 表示时间段序列。未来版本将支持用任意起止时间实现不规律时间间隔。

## 转换时间戳

`to_datetime` 函数用于转换字符串、纪元式及混合的日期 `Series` 或日期列表。转换的是 `Series` 时，返回的是具有相同的索引的 `Series`，日期时间列表则会被转换为 `DatetimeIndex`：

```
In [43]: pd.to_datetime(pd.Series(['Jul 31, 2009', '2010-01-10', None]))
Out[43]:
0 2009-07-31
1 2010-01-10
2 NaT
dtype: datetime64[ns]

In [44]: pd.to_datetime(['2005/11/23', '2010.12.31'])
Out[44]: DatetimeIndex(['2005-11-23', '2010-12-31'], dtype='datetime64[ns]', freq=None)
```

解析欧式日期（日-月-年），要用 `dayfirst` 关键字参数：

```
In [45]: pd.to_datetime(['04-01-2012 10:00'], dayfirst=True)
Out[45]: DatetimeIndex(['2012-01-04 10:00:00'], dtype='datetime64[ns]', freq=None)

In [46]: pd.to_datetime(['14-01-2012', '01-14-2012'], dayfirst=True)
Out[46]: DatetimeIndex(['2012-01-14', '2012-01-14'], dtype='datetime64[ns]', freq=None)
```

⋮ danger 警告

从上例可以看出，`dayfirst` 并没有那么严苛，如果不能把第一个数解析为日，就会以 `dayfirst` 为 `False` 进行解析。

⋮

`to_datetime` 转换单个字符串时，返回的是单个 `Timestamp`。`Timestamp` 仅支持字符串输入，不支持 `dayfirst`、`format` 等字符串解析选项，如果要使用这些选项，就要用 `to_datetime`。

```
In [47]: pd.to_datetime('2010/11/12')
Out[47]: Timestamp('2010-11-12 00:00:00')

In [48]: pd.Timestamp('2010/11/12')
Out[48]: Timestamp('2010-11-12 00:00:00')
```

Pandas 还支持直接使用 `DatetimeIndex` 构建器：

```
In [49]: pd.DatetimeIndex(['2018-01-01', '2018-01-03', '2018-01-05'])
Out[49]: DatetimeIndex(['2018-01-01', '2018-01-03', '2018-01-05'], dtype='datetime64[ns]', freq=None)
```

创建 `DatetimeIndex` 时，传递字符串 `infer` 即可推断索引的频率。

```
In [50]: pd.DatetimeIndex(['2018-01-01', '2018-01-03', '2018-01-05'], freq='infer')
Out[50]: DatetimeIndex(['2018-01-01', '2018-01-03', '2018-01-05'], dtype='datetime64[ns]', freq='2D')
```

## 提供格式参数

要实现精准转换，除了传递 `datetime` 字符串，还要指定 `format` 参数，指定此参数还可以加速转换速度。

```
In [51]: pd.to_datetime('2010/11/12', format='%Y/%m/%d')
Out[51]: Timestamp('2010-11-12 00:00:00')

In [52]: pd.to_datetime('12-11-2010 00:00', format='%d-%m-%Y %H:%M')
Out[52]: Timestamp('2010-11-12 00:00:00')
```

要了解更多 `format` 选项，请参阅 Python [日期时间文档](#)。

## 用多列组合日期时间

0.18.1 版新增。

pandas 还可以把 `DataFrame` 里的整数或字符串列组合成 `Timestamp Series`。

```
In [53]: df = pd.DataFrame({'year': [2015, 2016],
....: 'month': [2, 3],
....: 'day': [4, 5],
....: 'hour': [2, 3]})
....:

In [54]: pd.to_datetime(df)
Out[54]:
0 2015-02-04 02:00:00
1 2016-03-05 03:00:00
dtype: datetime64[ns]
```

只传递组合所需的列也可以。

```
In [55]: pd.to_datetime(df[['year', 'month', 'day']])
Out[55]:
0 2015-02-04
1 2016-03-05
dtype: datetime64[ns]
```

`pd.to_datetime` 查找列名里日期时间组件的标准名称，包括：

- 必填： `year`、`month`、`day`
- 可选： `hour`、`minute`、`second`、`millisecond`、`microsecond`、`nanosecond`

## 无效数据

不可解析时，默认值 `errors='raise'` 会触发错误：

```
In [2]: pd.to_datetime(['2009/07/31', 'asd'], errors='raise')
ValueError: Unknown string format
```

`errors='ignore'` 返回原始输入：

```
In [56]: pd.to_datetime(['2009/07/31', 'asd'], errors='ignore')
Out[56]: Index(['2009/07/31', 'asd'], dtype='object')
```

`errors='coerce'` 把无法解析的数据转换为 `NaT`，即不是时间（Not a Time）：

```
In [57]: pd.to_datetime(['2009/07/31', 'asd'], errors='coerce')
Out[57]: DatetimeIndex(['2009-07-31', 'NaT'], dtype='datetime64[ns]', freq=None)
```

## 纪元时间戳

pandas 支持把整数或浮点数纪元时间转换为 `Timestamp` 与 `DatetimeIndex`。鉴于 `Timestamp` 对象内部存储方式，这种转换的默认单位是纳秒。不过，一般都会用指定其它时间单位 `unit` 来存储纪元数据，纪元时间从 `origin` 参数指定的时点开始计算。

```
In [58]: pd.to_datetime([1349720105, 1349806505, 1349892905,
.....: 1349979305, 1350065705], unit='s')
.....:
Out[58]:
DatetimeIndex(['2012-10-08 18:15:05', '2012-10-09 18:15:05',
 '2012-10-10 18:15:05', '2012-10-11 18:15:05',
 '2012-10-12 18:15:05'],
 dtype='datetime64[ns]', freq=None)

In [59]: pd.to_datetime([1349720105100, 1349720105200, 1349720105300,
.....: 1349720105400, 1349720105500], unit='ms')
.....:
Out[59]:
DatetimeIndex(['2012-10-08 18:15:05.100000', '2012-10-08 18:15:05.200000',
 '2012-10-08 18:15:05.300000', '2012-10-08 18:15:05.400000',
 '2012-10-08 18:15:05.500000'],
 dtype='datetime64[ns]', freq=None)
```

用带 `tz` 参数的纪元时间戳创建 `Timestamp` 或 `DatetimeIndex` 时，要先把纪元时间戳转化为 UTC，然后再把结果转换为指定时区。不过这种操作方式现在已经废弃了，对于其它时区 Wall Time 里的纪元时间戳，建议先把纪元时间戳转换为无时区时间戳，然后再把时区本地化。

```
In [60]: pd.Timestamp(1262347200000000000).tz_localize('US/Pacific')
Out[60]: Timestamp('2010-01-01 12:00:00-0800', tz='US/Pacific')

In [61]: pd.DatetimeIndex([1262347200000000000]).tz_localize('US/Pacific')
Out[61]: DatetimeIndex(['2010-01-01 12:00:00-08:00'], dtype='datetime64[ns, US/Pacific]', freq=None)
```

::: tip 注意

纪元时间取整到最近的纳秒。

:::

::: danger 警告

Python 浮点数只精确到 15 位小数，因此，转换浮点纪元时间可能会导致不精准或失控的结果。转换过程中，免不了会对高精度 Timestamp 取整，只有用 int64 等定宽类型才有可能实现极其精准的效果。

```
In [62]: pd.to_datetime([1490195805.433, 1490195805.433502912], unit='s')
Out[62]: DatetimeIndex(['2017-03-22 15:16:45.433000088', '2017-03-22 15:16:45.433502913'],
 dtype='datetime64[ns]', freq=None)

In [63]: pd.to_datetime(1490195805433502912, unit='ns')
Out[63]: Timestamp('2017-03-22 15:16:45.433502912')
```

...

... tip 注意

纪元时间取整到最近的纳秒。

...

... tip 参阅

应用 origin 参数

...

## 把时间戳转换为纪元

反转上述操作，把 Timestamp 转换为 unix 纪元：

```
In [64]: stamps = pd.date_range('2012-10-08 18:15:05', periods=4, freq='D')

In [65]: stamps
Out[65]:
DatetimeIndex(['2012-10-08 18:15:05', '2012-10-09 18:15:05',
 '2012-10-10 18:15:05', '2012-10-11 18:15:05'],
 dtype='datetime64[ns]', freq='D')
```

首先与纪元开始时点（1970 年 1 月 1 日午夜，UTC）相减，然后以 1 秒为时间单位（unit='1s'）取底整除。

```
In [66]: (stamps - pd.Timestamp("1970-01-01")) // pd.Timedelta('1s')
Out[66]: Int64Index([1349720105, 1349806505, 1349892905, 1349979305], dtype='int64')
```

## 应用 origin 参数

0.20.0 版新增。

origin 参数可以指定 DatetimeIndex 的备选开始时点。例如，把 1960-01-01 作为开始日期：



```
In [67]: pd.to_datetime([1, 2, 3], unit='D', origin=pd.Timestamp('1960-01-01'))
Out[67]: DatetimeIndex(['1960-01-02', '1960-01-03', '1960-01-04'], dtype='datetime64[ns]', freq=None)
```

默认值为 `origin='unix'`，即 `1970-01-01 00:00:00`，一般把这个时点称为 `unix 纪元` 或 `POSIX 时间`。

```
In [68]: pd.to_datetime([1, 2, 3], unit='D')
Out[68]: DatetimeIndex(['1970-01-02', '1970-01-03', '1970-01-04'], dtype='datetime64[ns]', freq=None)
```

## 生成时间戳范围

`DatetimeIndex`、`Index` 构建器可以生成时间戳索引，此处要提供 `datetime` 对象列表。

```
In [69]: dates = [datetime.datetime(2012, 5, 1),
....: datetime.datetime(2012, 5, 2),
....: datetime.datetime(2012, 5, 3)]
....:

注意频率信息
In [70]: index = pd.DatetimeIndex(dates)

In [71]: index
Out[71]: DatetimeIndex(['2012-05-01', '2012-05-02', '2012-05-03'], dtype='datetime64[ns]', freq=None)

自动转换为 DatetimeIndex
In [72]: index = pd.Index(dates)

In [73]: index
Out[73]: DatetimeIndex(['2012-05-01', '2012-05-02', '2012-05-03'], dtype='datetime64[ns]', freq=None)
```

实际工作中，经常要生成含大量时间戳的超长索引，一个个输入时间戳又枯燥，又低效。如果时间戳是定频的，用 `date_range()` 与 `bdate_range()` 函数即可创建 `DatetimeIndex`。`date_range` 默认的频率是**日历日**，`bdate_range` 的默认频率是**工作日**：

```

In [74]: start = datetime.datetime(2011, 1, 1)

In [75]: end = datetime.datetime(2012, 1, 1)

In [76]: index = pd.date_range(start, end)

In [77]: index
Out[77]:
DatetimeIndex(['2011-01-01', '2011-01-02', '2011-01-03', '2011-01-04',
 '2011-01-05', '2011-01-06', '2011-01-07', '2011-01-08',
 '2011-01-09', '2011-01-10',
 ...
 '2011-12-23', '2011-12-24', '2011-12-25', '2011-12-26',
 '2011-12-27', '2011-12-28', '2011-12-29', '2011-12-30',
 '2011-12-31', '2012-01-01'],
 dtype='datetime64[ns]', length=366, freq='D')

In [78]: index = pd.bdate_range(start, end)

In [79]: index
Out[79]:
DatetimeIndex(['2011-01-03', '2011-01-04', '2011-01-05', '2011-01-06',
 '2011-01-07', '2011-01-10', '2011-01-11', '2011-01-12',
 '2011-01-13', '2011-01-14',
 ...
 '2011-12-19', '2011-12-20', '2011-12-21', '2011-12-22',
 '2011-12-23', '2011-12-26', '2011-12-27', '2011-12-28',
 '2011-12-29', '2011-12-30'],
 dtype='datetime64[ns]', length=260, freq='B')

```

`date_range`、`bdate_range` 等便捷函数可以调用各种频率别名：

```

In [80]: pd.date_range(start, periods=1000, freq='M')
Out[80]:
DatetimeIndex(['2011-01-31', '2011-02-28', '2011-03-31', '2011-04-30',
 '2011-05-31', '2011-06-30', '2011-07-31', '2011-08-31',
 '2011-09-30', '2011-10-31',
 ...
 '2093-07-31', '2093-08-31', '2093-09-30', '2093-10-31',
 '2093-11-30', '2093-12-31', '2094-01-31', '2094-02-28',
 '2094-03-31', '2094-04-30'],
 dtype='datetime64[ns]', length=1000, freq='M')

In [81]: pd.bdate_range(start, periods=250, freq='BQS')
Out[81]:
DatetimeIndex(['2011-01-03', '2011-04-01', '2011-07-01', '2011-10-03',
 '2012-01-02', '2012-04-02', '2012-07-02', '2012-10-01',
 '2013-01-01', '2013-04-01',
 ...
 '2071-01-01', '2071-04-01', '2071-07-01', '2071-10-01',
 '2072-01-01', '2072-04-01', '2072-07-01', '2072-10-03',
 '2073-01-02', '2073-04-03'],
 dtype='datetime64[ns]', length=250, freq='BQS-JAN')

```

`date_range` 与 `bdate_range` 通过指定 `start`、`end`、`period` 与 `freq` 等参数，简化了生成日期范围这项工作。开始与结束日期是必填项，因此，不会生成指定范围之外的日期。

```
In [82]: pd.date_range(start, end, freq='BM')
Out[82]:
DatetimeIndex(['2011-01-31', '2011-02-28', '2011-03-31', '2011-04-29',
 '2011-05-31', '2011-06-30', '2011-07-29', '2011-08-31',
 '2011-09-30', '2011-10-31', '2011-11-30', '2011-12-30'],
 dtype='datetime64[ns]', freq='BM')

In [83]: pd.date_range(start, end, freq='W')
Out[83]:
DatetimeIndex(['2011-01-02', '2011-01-09', '2011-01-16', '2011-01-23',
 '2011-01-30', '2011-02-06', '2011-02-13', '2011-02-20',
 '2011-02-27', '2011-03-06', '2011-03-13', '2011-03-20',
 '2011-03-27', '2011-04-03', '2011-04-10', '2011-04-17',
 '2011-04-24', '2011-05-01', '2011-05-08', '2011-05-15',
 '2011-05-22', '2011-05-29', '2011-06-05', '2011-06-12',
 '2011-06-19', '2011-06-26', '2011-07-03', '2011-07-10',
 '2011-07-17', '2011-07-24', '2011-07-31', '2011-08-07',
 '2011-08-14', '2011-08-21', '2011-08-28', '2011-09-04',
 '2011-09-11', '2011-09-18', '2011-09-25', '2011-10-02',
 '2011-10-09', '2011-10-16', '2011-10-23', '2011-10-30',
 '2011-11-06', '2011-11-13', '2011-11-20', '2011-11-27',
 '2011-12-04', '2011-12-11', '2011-12-18', '2011-12-25',
 '2012-01-01'],
 dtype='datetime64[ns]', freq='W-SUN')

In [84]: pd.bdate_range(end=end, periods=20)
Out[84]:
DatetimeIndex(['2011-12-05', '2011-12-06', '2011-12-07', '2011-12-08',
 '2011-12-09', '2011-12-12', '2011-12-13', '2011-12-14',
 '2011-12-15', '2011-12-16', '2011-12-19', '2011-12-20',
 '2011-12-21', '2011-12-22', '2011-12-23', '2011-12-26',
 '2011-12-27', '2011-12-28', '2011-12-29', '2011-12-30'],
 dtype='datetime64[ns]', freq='B')

In [85]: pd.bdate_range(start=start, periods=20)
Out[85]:
DatetimeIndex(['2011-01-03', '2011-01-04', '2011-01-05', '2011-01-06',
 '2011-01-07', '2011-01-10', '2011-01-11', '2011-01-12',
 '2011-01-13', '2011-01-14', '2011-01-17', '2011-01-18',
 '2011-01-19', '2011-01-20', '2011-01-21', '2011-01-24',
 '2011-01-25', '2011-01-26', '2011-01-27', '2011-01-28'],
 dtype='datetime64[ns]', freq='B')
```

### 0.23.0 版新增。

指定 `start`、`end`、`periods` 即可生成从 `start` 开始至 `end` 结束的等距日期范围，这个日期范围包含了 `start` 与 `end`，生成的 `DatetimeIndex` 里的元素数量为 `periods` 的值。

```
In [86]: pd.date_range('2018-01-01', '2018-01-05', periods=5)
Out[86]:
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03', '2018-01-04',
 '2018-01-05'],
 dtype='datetime64[ns]', freq=None)

In [87]: pd.date_range('2018-01-01', '2018-01-05', periods=10)
Out[87]:
DatetimeIndex(['2018-01-01 00:00:00', '2018-01-01 10:40:00',
 '2018-01-01 21:20:00', '2018-01-02 08:00:00',
 '2018-01-02 18:40:00', '2018-01-03 05:20:00',
 '2018-01-03 16:00:00', '2018-01-04 02:40:00',
 '2018-01-04 13:20:00', '2018-01-05 00:00:00'],
 dtype='datetime64[ns]', freq=None)
```

## 自定义频率范围

设定 `weekmask` 与 `holidays` 参数，`bdate_range` 还可以生成自定义频率日期范围。这些参数只用于传递自定义字符串。

```
In [88]: weekmask = 'Mon Wed Fri'

In [89]: holidays = [datetime.datetime(2011, 1, 5), datetime.datetime(2011, 3, 14)]

In [90]: pd.bdate_range(start, end, freq='C', weekmask=weekmask, holidays=holidays)
Out[90]:
DatetimeIndex(['2011-01-03', '2011-01-07', '2011-01-10', '2011-01-12',
 '2011-01-14', '2011-01-17', '2011-01-19', '2011-01-21',
 '2011-01-24', '2011-01-26',
 ...,
 '2011-12-09', '2011-12-12', '2011-12-14', '2011-12-16',
 '2011-12-19', '2011-12-21', '2011-12-23', '2011-12-26',
 '2011-12-28', '2011-12-30'],
 dtype='datetime64[ns]', length=154, freq='C')

In [91]: pd.bdate_range(start, end, freq='CBMS', weekmask=weekmask)
Out[91]:
DatetimeIndex(['2011-01-03', '2011-02-02', '2011-03-02', '2011-04-01',
 '2011-05-02', '2011-06-01', '2011-07-01', '2011-08-01',
 '2011-09-02', '2011-10-03', '2011-11-02', '2011-12-02'],
 dtype='datetime64[ns]', freq='CBMS')
```

::: tip 参阅

[自定义工作日](#)

:::

## 时间戳的界限

Pandas 时间戳的最低单位为纳秒，64 位整数显示的时间跨度约为 584 年，这就是 `Timestamp` 的界限：

```
In [92]: pd.Timestamp.min
Out[92]: Timestamp('1677-09-21 00:12:43.145225')

In [93]: pd.Timestamp.max
Out[93]: Timestamp('2262-04-11 23:47:16.854775807')
```

::: tip 参阅

[时间段越界展示](#)

...

## 索引

---

`DatetimeIndex` 主要用作 pandas 对象的索引。`DatetimeIndex` 类为时间序列做了很多优化：

- 预计算了各种偏移量的日期范围，并在后台缓存，让后台生成后续日期范围的速度非常快（仅需抓取切片）。
- 在 pandas 对象上使用 `shift` 与 `tshift` 方法进行快速偏移。
- 合并具有相同频率的重叠 `DatetimeIndex` 对象的速度非常快（这点对快速数据对齐非常重要）。
- 通过 `year`、`month` 等属性快速访问日期字段。
- `snap` 等正则函数与超快的 `asof` 逻辑。

`DatetimeIndex` 对象支持全部常规 `Index` 对象的基本用法，及一些列简化频率处理的高级时间序列专有方法。

::: tip 参阅

[重置索引](#)

...

::: tip 注意

Pandas 不强制排序日期索引，但如果日期没有排序，可能会引发可控范围之外的或不正确的操作。

...

`DatetimeIndex` 可以当作常规索引，支持选择、切片等方法。

```

In [94]: rng = pd.date_range(start, end, freq='BM')

In [95]: ts = pd.Series(np.random.randn(len(rng)), index=rng)

In [96]: ts.index
Out[96]:
DatetimeIndex(['2011-01-31', '2011-02-28', '2011-03-31', '2011-04-29',
 '2011-05-31', '2011-06-30', '2011-07-29', '2011-08-31',
 '2011-09-30', '2011-10-31', '2011-11-30', '2011-12-30'],
 dtype='datetime64[ns]', freq='BM')

In [97]: ts[:5].index
Out[97]:
DatetimeIndex(['2011-01-31', '2011-02-28', '2011-03-31', '2011-04-29',
 '2011-05-31'],
 dtype='datetime64[ns]', freq='BM')

In [98]: ts[::2].index
Out[98]:
DatetimeIndex(['2011-01-31', '2011-03-31', '2011-05-31', '2011-07-29',
 '2011-09-30', '2011-11-30'],
 dtype='datetime64[ns]', freq='2BM')

```

## 局部字符串索引

能解析为时间戳的日期与字符串可以作为索引的参数：

```

In [99]: ts['1/31/2011']
Out[99]: 0.11920871129693428

In [100]: ts[datetime.datetime(2011, 12, 25):]
Out[100]:
2011-12-30 0.56702
Freq: BM, dtype: float64

In [101]: ts['10/31/2011':'12/31/2011']
Out[101]:
2011-10-31 0.271860
2011-11-30 -0.424972
2011-12-30 0.567020
Freq: BM, dtype: float64

```

pandas 为访问较长的时间序列提供了便捷方法，**年**、**年月**字符串均可：

```
In [102]: ts['2011']
Out[102]:
2011-01-31 0.119209
2011-02-28 -1.044236
2011-03-31 -0.861849
2011-04-29 -2.104569
2011-05-31 -0.494929
2011-06-30 1.071804
2011-07-29 0.721555
2011-08-31 -0.706771
2011-09-30 -1.039575
2011-10-31 0.271860
2011-11-30 -0.424972
2011-12-30 0.567020
Freq: BM, dtype: float64

In [103]: ts['2011-6']
Out[103]:
2011-06-30 1.071804
Freq: BM, dtype: float64
```

带 `DatetimeIndex` 的 `DataFrame` 也支持这种切片方式。局部字符串是标签切片的一种形式，这种切片也**包含**截止时点，即，与日期匹配的时间也会包含在内：

```

In [104]: dft = pd.DataFrame(np.random.randn(100000, 1), columns=['A'],
.....: index=pd.date_range('20130101', periods=100000, freq='T'))
.....:

In [105]: dft
Out[105]:
 A
2013-01-01 00:00:00 0.276232
2013-01-01 00:01:00 -1.087401
2013-01-01 00:02:00 -0.673690
2013-01-01 00:03:00 0.113648
2013-01-01 00:04:00 -1.478427
...
2013-03-11 10:35:00 -0.747967
2013-03-11 10:36:00 -0.034523
2013-03-11 10:37:00 -0.201754
2013-03-11 10:38:00 -1.509067
2013-03-11 10:39:00 -1.693043

[100000 rows x 1 columns]

In [106]: dft['2013']
Out[106]:
 A
2013-01-01 00:00:00 0.276232
2013-01-01 00:01:00 -1.087401
2013-01-01 00:02:00 -0.673690
2013-01-01 00:03:00 0.113648
2013-01-01 00:04:00 -1.478427
...
2013-03-11 10:35:00 -0.747967
2013-03-11 10:36:00 -0.034523
2013-03-11 10:37:00 -0.201754
2013-03-11 10:38:00 -1.509067
2013-03-11 10:39:00 -1.693043

[100000 rows x 1 columns]

```

下列代码截取了自 1 月 1 日凌晨起，至 2 月 28 日午夜的日期与时间。



```
In [107]: dft['2013-1':'2013-2']
Out[107]:
```

	A
2013-01-01 00:00:00	0.276232
2013-01-01 00:01:00	-1.087401
2013-01-01 00:02:00	-0.673690
2013-01-01 00:03:00	0.113648
2013-01-01 00:04:00	-1.478427
...	...
2013-02-28 23:55:00	0.850929
2013-02-28 23:56:00	0.976712
2013-02-28 23:57:00	-2.693884
2013-02-28 23:58:00	-1.575535
2013-02-28 23:59:00	-1.573517

```
[84960 rows x 1 columns]
```

下列代码截取了包含截止日期及其时间在内的日期与时间。

```
In [108]: dft['2013-1':'2013-2-28']
Out[108]:
```

	A
2013-01-01 00:00:00	0.276232
2013-01-01 00:01:00	-1.087401
2013-01-01 00:02:00	-0.673690
2013-01-01 00:03:00	0.113648
2013-01-01 00:04:00	-1.478427
...	...
2013-02-28 23:55:00	0.850929
2013-02-28 23:56:00	0.976712
2013-02-28 23:57:00	-2.693884
2013-02-28 23:58:00	-1.575535
2013-02-28 23:59:00	-1.573517

```
[84960 rows x 1 columns]
```

下列代码指定了精准的截止时间，注意此处的结果与上述截取结果的区别：

```
In [109]: dft['2013-1': '2013-2-28 00:00:00']
Out[109]:
```

	A
2013-01-01 00:00:00	0.276232
2013-01-01 00:01:00	-1.087401
2013-01-01 00:02:00	-0.673690
2013-01-01 00:03:00	0.113648
2013-01-01 00:04:00	-1.478427
...	...
2013-02-27 23:56:00	1.197749
2013-02-27 23:57:00	0.720521
2013-02-27 23:58:00	-0.072718
2013-02-27 23:59:00	-0.681192
2013-02-28 00:00:00	-0.557501

```
[83521 rows x 1 columns]
```

截止时间是索引的一部分，包含在截取的内容之内：

```
In [110]: dft['2013-1-15': '2013-1-15 12:30:00']
Out[110]:
```

	A
2013-01-15 00:00:00	-0.984810
2013-01-15 00:01:00	0.941451
2013-01-15 00:02:00	1.559365
2013-01-15 00:03:00	1.034374
2013-01-15 00:04:00	-1.480656
...	...
2013-01-15 12:26:00	0.371454
2013-01-15 12:27:00	-0.930806
2013-01-15 12:28:00	-0.069177
2013-01-15 12:29:00	0.066510
2013-01-15 12:30:00	-0.003945

```
[751 rows x 1 columns]
```

0.18.0 版新增。

`DatetimeIndex` 局部字符串索引还支持多重索引 `DataFrame`。

```

In [111]: dft2 = pd.DataFrame(np.random.randn(20, 1),
.....: columns=['A'],
.....: index=pd.MultiIndex.from_product(
.....: [pd.date_range('20130101', periods=10, freq='12H'),
.....: ['a', 'b']]))
.....:

In [112]: dft2
Out[112]:
```

		A
2013-01-01 00:00:00	a	-0.298694
	b	0.823553
2013-01-01 12:00:00	a	0.943285
	b	-1.479399
2013-01-02 00:00:00	a	-1.643342
...	...	...
2013-01-04 12:00:00	b	0.069036
2013-01-05 00:00:00	a	0.122297
	b	1.422060
2013-01-05 12:00:00	a	0.370079
	b	1.016331

```

[20 rows x 1 columns]

In [113]: dft2.loc['2013-01-05']
Out[113]:
```

	A
2013-01-05 00:00:00	a 0.122297
	b 1.422060
2013-01-05 12:00:00	a 0.370079
	b 1.016331

```

In [114]: idx = pd.IndexSlice

In [115]: dft2 = dft2.swaplevel(0, 1).sort_index()

In [116]: dft2.loc[idx[:, '2013-01-05'], :]
Out[116]:
```

	A
a 2013-01-05 00:00:00	0.122297
2013-01-05 12:00:00	0.370079
b 2013-01-05 00:00:00	1.422060
2013-01-05 12:00:00	1.016331

0.25.0 版新增。

字符串索引切片支持 UTC 偏移。

```
In [117]: df = pd.DataFrame([0], index=pd.DatetimeIndex(['2019-01-01'], tz='US/Pacific'))

In [118]: df
Out[118]:
 0
2019-01-01 00:00:00-08:00 0

In [119]: df['2019-01-01 12:00:00+04:00':'2019-01-01 13:00:00+04:00']
Out[119]:
 0
2019-01-01 00:00:00-08:00 0
```

## 切片 vs. 精准匹配

0.20.0 版新增。

基于索引的精度，字符串既可用于切片，也可用于精准匹配。字符串精度比索引精度低，就是切片，比索引精度高，则是精准匹配。

```
In [120]: series_minute = pd.Series([1, 2, 3],
.....: pd.DatetimeIndex(['2011-12-31 23:59:00',
.....: '2012-01-01 00:00:00',
.....: '2012-01-01 00:02:00']))

In [121]: series_minute.index.resolution
Out[121]: 'minute'
```

下例中的时间戳字符串没有 Series 对象的精度高。series\_minute 到秒，时间戳字符串只到分。

```
In [122]: series_minute['2011-12-31 23']
Out[122]:
2011-12-31 23:59:00 1
dtype: int64
```

精度为分钟（或更高精度）的时间戳字符串，给出的是标量，不会被当作切片。

```
In [123]: series_minute['2011-12-31 23:59']
Out[123]: 1

In [124]: series_minute['2011-12-31 23:59:00']
Out[124]: 1
```

索引的精度为秒时，精度为分钟的时间戳返回的是 Series。

```

In [125]: series_second = pd.Series([1, 2, 3],
.....: pd.DatetimeIndex(['2011-12-31 23:59:59',
.....: '2012-01-01 00:00:00',
.....: '2012-01-01 00:00:01']))
.....:

In [126]: series_second.index.resolution
Out[126]: 'second'

In [127]: series_second['2011-12-31 23:59']
Out[127]:
2011-12-31 23:59:59 1
dtype: int64

```

用时间戳字符串切片时，还可以用 `[]` 索引 `DataFrame`。

```

In [128]: dft_minute = pd.DataFrame({'a': [1, 2, 3], 'b': [4, 5, 6]},
.....: index=series_minute.index)
.....:

In [129]: dft_minute['2011-12-31 23']
Out[129]:
 a b
2011-12-31 23:59:00 1 4

```

### ::: danger 警告

字符串执行精确匹配时，用 `[]` 按列，而不是按行截取 `DataFrame`，参阅 [索引基础](#)。如，`dft_minute['2011-12-31 23:59']` 会触发 `KeyError`，这是因为 `2012-12-31 23:59` 与索引的精度一样，但没有叫这个名字的列。

为了实现精准切片，要用 `.loc` 对行进行切片或选择。

```

In [130]: dft_minute.loc['2011-12-31 23:59']
Out[130]:
a 1
b 4
Name: 2011-12-31 23:59:00, dtype: int64

```

:::

注意： `DatetimeIndex` 精度不能低于日。

```

In [131]: series_monthly = pd.Series([1, 2, 3],
.....: pd.DatetimeIndex(['2011-12', '2012-01', '2012-02']))
.....:

In [132]: series_monthly.index.resolution
Out[132]: 'day'

In [133]: series_monthly['2011-12'] # 返回的是 Series
Out[133]:
2011-12-01 1
dtype: int64

```

## 精确索引

正如上节所述，局部字符串依靠时间段的**精度**索引 `DatetimeIndex`，即时间间隔与索引精度相关。反之，用 `Timestamp` 或 `datetime` 索引更精准，这些对象指定的时间更精确。注意，精确索引包含了起始时点。

就算没有显式指定，`Timestamp` 与 `datetime` 也支持 `hours`、`minutes`、`seconds`，默认值为 0。

```

In [134]: dft[datetime.datetime(2013, 1, 1):datetime.datetime(2013, 2, 28)]
Out[134]:

```

	A
2013-01-01 00:00:00	0.276232
2013-01-01 00:01:00	-1.087401
2013-01-01 00:02:00	-0.673690
2013-01-01 00:03:00	0.113648
2013-01-01 00:04:00	-1.478427
...	...
2013-02-27 23:56:00	1.197749
2013-02-27 23:57:00	0.720521
2013-02-27 23:58:00	-0.072718
2013-02-27 23:59:00	-0.681192
2013-02-28 00:00:00	-0.557501

```

[83521 rows x 1 columns]

```

不用默认值。

```
In [135]: dft[datetime.datetime(2013, 1, 1, 10, 12, 0):
.....: datetime.datetime(2013, 2, 28, 10, 12, 0)]
.....:
Out[135]:
```

	A
2013-01-01 10:12:00	0.565375
2013-01-01 10:13:00	0.068184
2013-01-01 10:14:00	0.788871
2013-01-01 10:15:00	-0.280343
2013-01-01 10:16:00	0.931536
...	...
2013-02-28 10:08:00	0.148098
2013-02-28 10:09:00	-0.388138
2013-02-28 10:10:00	0.139348
2013-02-28 10:11:00	0.085288
2013-02-28 10:12:00	0.950146

```
[83521 rows x 1 columns]
```

## 截取与花式索引

`truncate()` 便捷函数与切片类似。注意，与切片返回的是部分匹配日期不同，`truncate` 假设 `DatetimeIndex` 里未标明时间组件的值为 0。

```
In [136]: rng2 = pd.date_range('2011-01-01', '2012-01-01', freq='W')

In [137]: ts2 = pd.Series(np.random.randn(len(rng2)), index=rng2)

In [138]: ts2.truncate(before='2011-11', after='2011-12')
Out[138]:
```

2011-11-06	0.437823
2011-11-13	-0.293083
2011-11-20	-0.059881
2011-11-27	1.252450

```
Freq: W-SUN, dtype: float64

In [139]: ts2['2011-11':'2011-12']
Out[139]:
```

2011-11-06	0.437823
2011-11-13	-0.293083
2011-11-20	-0.059881
2011-11-27	1.252450
2011-12-04	0.046611
2011-12-11	0.059478
2011-12-18	-0.286539
2011-12-25	0.841669

```
Freq: W-SUN, dtype: float64
```

花式索引返回的是 `DatetimeIndex`，但因为打乱了 `DatetimeIndex` 的频率，所以频率信息没有了，见 `freq=None`：

```
In [140]: ts2[[0, 2, 6]].index
Out[140]: DatetimeIndex(['2011-01-02', '2011-01-16', '2011-02-13'], dtype='datetime64[ns]', freq=None)
```

## 日期/时间组件

以下日期/时间属性可以访问 `Timestamp` 或 `DatetimeIndex`。

属性	说明
<code>year</code>	datetime 的年
<code>month</code>	datetime 的月
<code>day</code>	datetime 的日
<code>hour</code>	datetime 的小时
<code>minute</code>	datetime 的分钟
<code>second</code>	datetime 的秒
<code>microsecond</code>	datetime 的微妙
<code>nanosecond</code>	datetime 的纳秒
<code>date</code>	返回 <code>datetime.date</code> （不包含时区信息）
<code>time</code>	返回 <code>datetime.time</code> （不包含时区信息）
<code>timetz</code>	返回带本地时区信息的 <code>datetime.time</code>
<code>dayofyear</code>	一年里的第几天
<code>weekofyear</code>	一年里的第几周
<code>week</code>	一年里的第几周
<code>dayofweek</code>	一周里的第几天，Monday=0, Sunday=6
<code>weekday</code>	一周里的第几天，Monday=0, Sunday=6
<code>weekday_name</code>	这一天是星期几（如，Friday）
<code>quarter</code>	日期所处的季节：Jan-Mar = 1, Apr-Jun = 2 等
<code>days_in_month</code>	日期所在的月有多少天
<code>is_month_start</code>	逻辑判断是不是月初（由频率定义）
<code>is_month_end</code>	逻辑判断是不是月末（由频率定义）
<code>is_quarter_start</code>	逻辑判断是不是季初（由频率定义）
<code>is_quarter_end</code>	逻辑判断是不是季末（由频率定义）
<code>is_year_start</code>	逻辑判断是不是年初（由频率定义）
<code>is_year_end</code>	逻辑判断是不是年末（由频率定义）
<code>is_leap_year</code>	逻辑判断是不是日期所在年是不是闰年



参照 [.dt 访问器](#) 一节介绍的知识点，`Series` 的值为 `datetime` 时，还可以用 `.dt` 访问这些属性。

## DateOffset 对象

上例中，频率字符串（如，`D`）用于定义指定的频率：

- 用 `date_range()` 按指定频率分隔 `DatetimeIndex` 里的日期与时间
- `Period` 或 `PeriodIndex` 的频率

频率字符串表示的是 `DateOffset` 对象及其子类。`DateOffset` 类似于时间差 `Timedelta`，但遵循指定的日历日规则。例如，`Timedelta` 表示的每日时间差一直都是 24 小时，而 `DateOffset` 的每日偏移量则是与下一天相同的时间差，使用夏时制时，每日偏移时间有可能是 23 或 24 小时，甚至还有可能是 25 小时。不过，`DateOffset` 子类只能是等于或小于小时的时间单位（`Hour`、`Minute`、`Second`、`Milli`、`Micro`、`Nano`），操作类似于 `Timedelta` 及对应的绝对时间。

`DateOffset` 基础操作类似于 `dateutil.relativedelta`（[relativedelta 文档](#)），可按指定的日历日时间段偏移日期时间。可用算数运算符（`+`）或 `apply` 方法执行日期偏移操作。

```
指定包含夏时制变迁的某天
In [141]: ts = pd.Timestamp('2016-10-30 00:00:00', tz='Europe/Helsinki')

对应的绝对时间
In [142]: ts + pd.Timedelta(days=1)
Out[142]: Timestamp('2016-10-30 23:00:00+0200', tz='Europe/Helsinki')

对应的日历时间
In [143]: ts + pd.DateOffset(days=1)
Out[143]: Timestamp('2016-10-31 00:00:00+0200', tz='Europe/Helsinki')

In [144]: friday = pd.Timestamp('2018-01-05')

In [145]: friday.day_name()
Out[145]: 'Friday'

与两个工作日相加（星期五 --> 星期二）
In [146]: two_business_days = 2 * pd.offsets.BDay()

In [147]: two_business_days.apply(friday)
Out[147]: Timestamp('2018-01-09 00:00:00')

In [148]: friday + two_business_days
Out[148]: Timestamp('2018-01-09 00:00:00')

In [149]: (friday + two_business_days).day_name()
Out[149]: 'Tuesday'
```

大多数 `DateOffset` 都支持频率字符串或偏移别名，可用作 `freq` 关键字参数。有效的日期偏移及频率字符串如下：

日期偏移量	频率字符串	说明
DateOffset	无	通用偏移类，默认为一个日历日
BDay 或 BusinessDay	'B'	工作日
CDay 或 CustomBusinessDay	'C'	自定义工作日
Week	'W'	一周，可选周内固定某日
WeekOfMonth	'WOM'	每月第几周的第几天
LastWeekOfMonth	'LWOM'	每月最后一周的第几天
MonthEnd	'M'	日历日月末
MonthBegin	'MS'	日历日月初
BMonthEnd 或 BusinessMonthEnd	'BM'	工作日月末
BMonthBegin 或 BusinessMonthBegin	'BMS'	工作日月初
CBMonthEnd 或 CustomBusinessMonthEnd	'CBM'	自定义工作日月末
CBMonthBegin 或 CustomBusinessMonthBegin	'CBMS'	自定义工作日月初
SemiMonthEnd	'SM'	某月第 15 天（或其它半数日期）与日历日月末
SemiMonthBegin	'SMS'	日历日月初与第 15 天（或其它半数日期）
QuarterEnd	'Q'	日历日季末
QuarterBegin	'QS'	日历日季初
BQuarterEnd	'BQ'	工作日季末
BQuarterBegin	'BQS'	工作日季初
FY5253Quarter	'REQ'	零售季，又名 52-53 周
YearEnd	'A'	日历日年末
YearBegin	'AS' 或 'BYS'	日历日年初
BYearEnd	'BA'	工作日年末
BYearBegin	'BAS'	工作日年初
FY5253	'RE'	零售年（又名 52-53 周）
Easter	无	复活节假日
BusinessHour	'BH'	工作小时
CustomBusinessHour	'CBH'	自定义工作小时
Day	'D'	一天
Hour	'H'	一小时
Minute	'T' 或 'min'	一分钟
Second	'S'	一秒

日期偏移量	频率字符串	说明
Milli	'L' 或 'ms'	一毫秒
Micro	'U' 或 'us'	一微秒
Nano	'N'	一纳秒

`DateOffset` 还支持 `rollforward()` 与 `rollback()` 方法，按偏移量把某一日期**向前或向后**移动至有效偏移日期。例如，工作日偏移滚动日期时会跳过周末（即，星期六与星期日），直接到星期一，因为工作日偏移针对的是工作日。

```
In [150]: ts = pd.Timestamp('2018-01-06 00:00:00')

In [151]: ts.day_name()
Out[151]: 'Saturday'

工作时间的有效偏移日期为星期一至星期五
In [152]: offset = pd.offsets.BusinessHour(start='09:00')

向前偏移到最近的工作日，即星期一
In [153]: offset.rollforward(ts)
Out[153]: Timestamp('2018-01-08 09:00:00')

向前偏移至最近的工作日，同时，小时也相应增加了
In [154]: ts + offset
Out[154]: Timestamp('2018-01-08 10:00:00')
```

这些操作默认保存时间（小时、分钟等）信息。`normalize()` 可以把时间重置为午夜零点，是否应用此操作，取决于是否需要保留时间信息。

```
In [155]: ts = pd.Timestamp('2014-01-01 09:00')

In [156]: day = pd.offsets.Day()

In [157]: day.apply(ts)
Out[157]: Timestamp('2014-01-02 09:00:00')

In [158]: day.apply(ts).normalize()
Out[158]: Timestamp('2014-01-02 00:00:00')

In [159]: ts = pd.Timestamp('2014-01-01 22:00')

In [160]: hour = pd.offsets.Hour()

In [161]: hour.apply(ts)
Out[161]: Timestamp('2014-01-01 23:00:00')

In [162]: hour.apply(ts).normalize()
Out[162]: Timestamp('2014-01-01 00:00:00')

In [163]: hour.apply(pd.Timestamp("2014-01-01 23:30")).normalize()
Out[163]: Timestamp('2014-01-02 00:00:00')
```

## 参数偏移

偏移量支持参数，可以让不同操作生成不同结果。例如，`Week` 偏移生成每周数据时支持 `weekday` 参数，生成日期始终位于一周中的指定日期。

```
In [164]: d = datetime.datetime(2008, 8, 18, 9, 0)

In [165]: d
Out[165]: datetime.datetime(2008, 8, 18, 9, 0)

In [166]: d + pd.offsets.Week()
Out[166]: Timestamp('2008-08-25 09:00:00')

In [167]: d + pd.offsets.Week(weekday=4)
Out[167]: Timestamp('2008-08-22 09:00:00')

In [168]: (d + pd.offsets.Week(weekday=4)).weekday()
Out[168]: 4

In [169]: d - pd.offsets.Week()
Out[169]: Timestamp('2008-08-11 09:00:00')
```

加减法也支持 `normalize` 选项。

```
In [170]: d + pd.offsets.Week(normalize=True)
Out[170]: Timestamp('2008-08-25 00:00:00')

In [171]: d - pd.offsets.Week(normalize=True)
Out[171]: Timestamp('2008-08-11 00:00:00')
```

`YearEnd` 也支持参数，如 `month` 参数，用于指定月份。

```
In [172]: d + pd.offsets.YearEnd()
Out[172]: Timestamp('2008-12-31 09:00:00')

In [173]: d + pd.offsets.YearEnd(month=6)
Out[173]: Timestamp('2009-06-30 09:00:00')
```

## Series 与 DatetimeIndex 偏移

可以为 `Series` 或 `DatetimeIndex` 里的每个元素应用偏移。

```
In [174]: rng = pd.date_range('2012-01-01', '2012-01-03')

In [175]: s = pd.Series(rng)

In [176]: rng
Out[176]: DatetimeIndex(['2012-01-01', '2012-01-02', '2012-01-03'], dtype='datetime64[ns]', freq='D')

In [177]: rng + pd.DateOffset(months=2)
Out[177]: DatetimeIndex(['2012-03-01', '2012-03-02', '2012-03-03'], dtype='datetime64[ns]', freq='D')

In [178]: s + pd.DateOffset(months=2)
Out[178]:
0 2012-03-01
1 2012-03-02
2 2012-03-03
dtype: datetime64[ns]

In [179]: s - pd.DateOffset(months=2)
Out[179]:
0 2011-11-01
1 2011-11-02
2 2011-11-03
dtype: datetime64[ns]
```

如果偏移直接映射 `Timedelta`（`Day`、`Hour`、`Minute`、`Second`、`Micro`、`Milli`、`Nano`），则该偏移与 `Timedelta` 的使用方式完全一样。参阅[时间差 - Timedelta](#)，查看更多示例。

```

In [180]: s - pd.offsets.Day(2)
Out[180]:
0 2011-12-30
1 2011-12-31
2 2012-01-01
dtype: datetime64[ns]

In [181]: td = s - pd.Series(pd.date_range('2011-12-29', '2011-12-31'))

In [182]: td
Out[182]:
0 3 days
1 3 days
2 3 days
dtype: timedelta64[ns]

In [183]: td + pd.offsets.Minute(15)
Out[183]:
0 3 days 00:15:00
1 3 days 00:15:00
2 3 days 00:15:00
dtype: timedelta64[ns]

```

注意，某些偏移量（如 `BQuarterEnd`）不支持矢量操作，即使可以执行运算，速度也非常慢，并可能显示 `PerformanceWarning`（性能警告）。

```

In [184]: rng + pd.offsets.BQuarterEnd()
Out[184]: DatetimeIndex(['2012-03-30', '2012-03-30', '2012-03-30'], dtype='datetime64[ns]', freq='D')

```

## 自定义工作日

`Cday` 或 `CustomBusinessDay` 类可以参数化 `BusinessDay` 类，用于创建支持本地周末与传统节假日的自定义工作日历。

下面这个例子就很有意思，知道吗？埃及的周末是星期五与星期六。

```
In [185]: weekmask_egypt = 'Sun Mon Tue Wed Thu'

下面是 2012 - 2014 年的五一劳动节
In [186]: holidays = ['2012-05-01',
.....: datetime.datetime(2013, 5, 1),
.....: np.datetime64('2014-05-01')]
.....:

In [187]: bday_egypt = pd.offsets.CustomBusinessDay(holidays=holidays,
.....: weekmask=weekmask_egypt)
.....:

In [188]: dt = datetime.datetime(2013, 4, 30)

In [189]: dt + 2 * bday_egypt
Out[189]: Timestamp('2013-05-05 00:00:00')
```

下列代码实现了日期与工作日之间的映射关系。

```
In [190]: dts = pd.date_range(dt, periods=5, freq=bday_egypt)

In [191]: pd.Series(dts.weekday, dts).map(
.....: pd.Series('Mon Tue Wed Thu Fri Sat Sun'.split()))
.....:
Out[191]:
2013-04-30 Tue
2013-05-02 Thu
2013-05-05 Sun
2013-05-06 Mon
2013-05-07 Tue
Freq: C, dtype: object
```

节日日历支持节假日列表。更多信息，请参阅[节日日历文档](#)。

```
In [192]: from pandas.tseries.holiday import USFederalHolidayCalendar

In [193]: bday_us = pd.offsets.CustomBusinessDay(calendar=USFederalHolidayCalendar())

马丁路德金纪念日前的星期五
In [194]: dt = datetime.datetime(2014, 1, 17)

马丁路德金纪念日后的星期二，因为星期一放假，所以跳过了
In [195]: dt + bday_us
Out[195]: Timestamp('2014-01-21 00:00:00')
```

遵循节日日历规则的月偏移可以用正常方式定义。

```

In [196]: bmth_us = pd.offsets.CustomBusinessMonthBegin(
.....: calendar=USFederalHolidayCalendar())
.....:

跳过新年
In [197]: dt = datetime.datetime(2013, 12, 17)

In [198]: dt + bmth_us
Out[198]: Timestamp('2014-01-02 00:00:00')

定义带自定义偏移的日期索引
In [199]: pd.date_range(start='20100101', end='20120101', freq=bmth_us)
Out[199]:
DatetimeIndex(['2010-01-04', '2010-02-01', '2010-03-01', '2010-04-01',
 '2010-05-03', '2010-06-01', '2010-07-01', '2010-08-02',
 '2010-09-01', '2010-10-01', '2010-11-01', '2010-12-01',
 '2011-01-03', '2011-02-01', '2011-03-01', '2011-04-01',
 '2011-05-02', '2011-06-01', '2011-07-01', '2011-08-01',
 '2011-09-01', '2011-10-03', '2011-11-01', '2011-12-01'],
 dtype='datetime64[ns]', freq='CBMS')

```

::: tip 注意

频率字符串 'C' 验证 CustomBusinessDay 日期偏移调用，注意，CustomBusinessDay 可实现参数化，CustomBusinessDay 实例会各不相同，且频率字符串 'C' 无法识别这个问题。用户应确保应用里调用的频率字符串 'C' 的一致性。

## 工作时间

BusinessHour 表示 BusinessDay 基础上的工作时间，用于指定开始与结束工作时间。

BusinessHour 默认的工作时间是 9:00 - 17:00。BusinessHour 加法以小时频率增加 Timestamp。如果目标 Timestamp 超出了一小时，则要先移动到下一个工作小时，再行增加。如果超过了当日工作时间的范围，剩下的时间则添加到下一个工作日。



```

In [200]: bh = pd.offsets.BusinessHour()

In [201]: bh
Out[201]: <BusinessHour: BH=09:00-17:00>

2014 年 8 月 1 日是星期五
In [202]: pd.Timestamp('2014-08-01 10:00').weekday()
Out[202]: 4

In [203]: pd.Timestamp('2014-08-01 10:00') + bh
Out[203]: Timestamp('2014-08-01 11:00:00')

下例等同于: pd.Timestamp('2014-08-01 09:00') + bh
In [204]: pd.Timestamp('2014-08-01 08:00') + bh
Out[204]: Timestamp('2014-08-01 10:00:00')

如果计算结果为当日下午下班时间, 则转移到下一个工作日的上班时间
In [205]: pd.Timestamp('2014-08-01 16:00') + bh
Out[205]: Timestamp('2014-08-04 09:00:00')

剩下的时间也会添加到下一天
In [206]: pd.Timestamp('2014-08-01 16:30') + bh
Out[206]: Timestamp('2014-08-04 09:30:00')

添加 2 个工作小时
In [207]: pd.Timestamp('2014-08-01 10:00') + pd.offsets.BusinessHour(2)
Out[207]: Timestamp('2014-08-01 12:00:00')

减掉 3 个工作小时
In [208]: pd.Timestamp('2014-08-01 10:00') + pd.offsets.BusinessHour(-3)
Out[208]: Timestamp('2014-07-31 15:00:00')

```

还可以用关键字指定 `start` 与 `end` 时间。参数必须是 `hour:minute` 格式的字符串或 `datetime.time` 实例。把秒、微秒、纳秒设置为工作时间会导致 `ValueError`。

```

In [209]: bh = pd.offsets.BusinessHour(start='11:00', end=datetime.time(20, 0))

In [210]: bh
Out[210]: <BusinessHour: BH=11:00-20:00>

In [211]: pd.Timestamp('2014-08-01 13:00') + bh
Out[211]: Timestamp('2014-08-01 14:00:00')

In [212]: pd.Timestamp('2014-08-01 09:00') + bh
Out[212]: Timestamp('2014-08-01 12:00:00')

In [213]: pd.Timestamp('2014-08-01 18:00') + bh
Out[213]: Timestamp('2014-08-01 19:00:00')

```

`start` 时间晚于 `end` 时间表示夜班工作时间。此时, 工作时间将从午夜延至第二天。工作时间是否有效取决于该时间是否开始于有效的 `BusinessDay`。

```
In [214]: bh = pd.offsets.BusinessHour(start='17:00', end='09:00')
```

```
In [215]: bh
```

```
Out[215]: <BusinessHour: BH=17:00-09:00>
```

```
In [216]: pd.Timestamp('2014-08-01 17:00') + bh
```

```
Out[216]: Timestamp('2014-08-01 18:00:00')
```

```
In [217]: pd.Timestamp('2014-08-01 23:00') + bh
```

```
Out[217]: Timestamp('2014-08-02 00:00:00')
```

```
虽然 2014 年 8 月 2 日是星期六，
```

```
但因为工作时间开始于星期五，因此，也是有效的
```

```
In [218]: pd.Timestamp('2014-08-02 04:00') + bh
```

```
Out[218]: Timestamp('2014-08-02 05:00:00')
```

```
虽然 2014 年 8 月 4 日是星期一，
```

```
但开始时间是星期日，因此，超出了工作时间
```

```
In [219]: pd.Timestamp('2014-08-04 04:00') + bh
```

```
Out[219]: Timestamp('2014-08-04 18:00:00')
```

`BusinessHour.rollforward` 与 `rollback` 操作将前滚至下一天的上班时间，或回滚至前一天的下班时间。与其它偏移量不同，`BusinessHour.rollforward` 输出与 `apply` 定义不同的结果。

这是因为一天工作时间的结束等同于第二天工作时间的开始。默认情况下，工作时间为 9:00 - 17:00，pandas 认为 2014-08-01 17:00 与 2014-08-04 09:00 之间的时间间隔为 0 分钟。

```
把时间戳回滚到前一天的下班时间
```

```
In [220]: pd.offsets.BusinessHour().rollback(pd.Timestamp('2014-08-02 15:00'))
```

```
Out[220]: Timestamp('2014-08-01 17:00:00')
```

```
把时间戳前滚到下一个工作日的上班时间
```

```
In [221]: pd.offsets.BusinessHour().rollforward(pd.Timestamp('2014-08-02 15:00'))
```

```
Out[221]: Timestamp('2014-08-04 09:00:00')
```

```
等同于: BusinessHour().apply(pd.Timestamp('2014-08-01 17:00'))
```

```
与 BusinessHour().apply(pd.Timestamp('2014-08-04 09:00'))
```

```
In [222]: pd.offsets.BusinessHour().apply(pd.Timestamp('2014-08-02 15:00'))
```

```
Out[222]: Timestamp('2014-08-04 10:00:00')
```

```
工作日的结果（仅供参考）
```

```
In [223]: pd.offsets.BusinessHour().rollforward(pd.Timestamp('2014-08-02'))
```

```
Out[223]: Timestamp('2014-08-04 09:00:00')
```

```
等同于 BusinessDay().apply(pd.Timestamp('2014-08-01'))
```

```
等同于 rollforward 因为工作日不会重叠
```

```
In [224]: pd.offsets.BusinessHour().apply(pd.Timestamp('2014-08-02'))
```

```
Out[224]: Timestamp('2014-08-04 10:00:00')
```

`BusinessHour` 把星期六与星期日当成假日。`CustomBusinessHour` 可以把节假日设为工作时间，详见下文。

## 自定义工作时间

0.18.1 版新增。

`CustomBusinessHour` 是 `BusinessHour` 和 `CustomBusinessDay` 的混合体，可以指定任意节假日。除了跳过自定义节假日之外，`CustomBusinessHour` 的运作方式与 `BusinessHour` 一样。

```
In [225]: from pandas.tseries.holiday import USFederalHolidayCalendar

In [226]: bhour_us = pd.offsets.CustomBusinessHour(calendar=USFederalHolidayCalendar())

马丁路德金纪念日之前的星期五
In [227]: dt = datetime.datetime(2014, 1, 17, 15)

In [228]: dt + bhour_us
Out[228]: Timestamp('2014-01-17 16:00:00')

跳至马丁路德金纪念日之后的星期二，星期一过节，所以跳过了
In [229]: dt + bhour_us * 2
Out[229]: Timestamp('2014-01-21 09:00:00')
```

`BusinessHour` 支持与 `CustomBusinessDay` 一样的关键字参数。

```
In [230]: bhour_mon = pd.offsets.CustomBusinessHour(start='10:00',
.....: weekmask='Tue Wed Thu Fri')
.....:

跳过了星期一，因为星期一过节，工作时间从 10 点开始
In [231]: dt + bhour_mon * 2
Out[231]: Timestamp('2014-01-21 10:00:00')
```

## 偏移量别名

时间序列频率的字符串别名在这里叫**偏移量别名**。

别名	说明
B	工作日频率
C	自定义工作日频率
D	日历日频率
W	周频率
M	月末频率
SM	半月末频率（15 号与月末）
BM	工作日月末频率
CBM	自定义工作日月末频率
MS	月初频率
SMS	半月初频率（1 号与 15 号）
BMS	工作日月初频率
CBMS	自定义工作日月初频率
Q	季末频率
BQ	工作日季末频率
QS	季初频率
BQS	工作日季初频率
A, Y	年末频率
BA, BY	工作日年末频率
AS, YS	年初频率
BAS, BYS	工作日年初频率
BH	工作时间频率
H	小时频率
T, min	分钟频率
S	秒频率
L, ms	毫秒
U, us	微秒
N	纳秒

## 别名组合

如前所述，别名与偏移量实例在绝大多数函数里可以互换：

```

In [232]: pd.date_range(start, periods=5, freq='B')
Out[232]:
DatetimeIndex(['2011-01-03', '2011-01-04', '2011-01-05', '2011-01-06',
 '2011-01-07'],
 dtype='datetime64[ns]', freq='B')

In [233]: pd.date_range(start, periods=5, freq=pd.offsets.BDay())
Out[233]:
DatetimeIndex(['2011-01-03', '2011-01-04', '2011-01-05', '2011-01-06',
 '2011-01-07'],
 dtype='datetime64[ns]', freq='B')

```

可以组合日与当日偏移量。

```

In [234]: pd.date_range(start, periods=10, freq='2h20min')
Out[234]:
DatetimeIndex(['2011-01-01 00:00:00', '2011-01-01 02:20:00',
 '2011-01-01 04:40:00', '2011-01-01 07:00:00',
 '2011-01-01 09:20:00', '2011-01-01 11:40:00',
 '2011-01-01 14:00:00', '2011-01-01 16:20:00',
 '2011-01-01 18:40:00', '2011-01-01 21:00:00'],
 dtype='datetime64[ns]', freq='140T')

In [235]: pd.date_range(start, periods=10, freq='1D10U')
Out[235]:
DatetimeIndex(['2011-01-01 00:00:00', '2011-01-02 00:00:00.000010',
 '2011-01-03 00:00:00.000020', '2011-01-04 00:00:00.000030',
 '2011-01-05 00:00:00.000040', '2011-01-06 00:00:00.000050',
 '2011-01-07 00:00:00.000060', '2011-01-08 00:00:00.000070',
 '2011-01-09 00:00:00.000080', '2011-01-10 00:00:00.000090'],
 dtype='datetime64[ns]', freq='86400000010U')

```

## 锚定偏移量

可以指定某些频率的锚定后缀：

别名	说明
W-SUN	周频率（星期日），与“W”相同
W-MON	周频率（星期一）
W-TUE	周频率（星期二）
W-WED	周频率（星期三）
W-THU	周频率（星期四）
W-FRI	周频率（星期五）
W-SAT	周频率（星期六）
(B)Q(S)-DEC	季频率，该年结束于十二月，与“Q”相同
(B)Q(S)-JAN	季频率，该年结束于一月
(B)Q(S)-FEB	季频率，该年结束于二月
(B)Q(S)-MAR	季频率，该年结束于三月
(B)Q(S)-APR	季频率，该年结束于四月
(B)Q(S)-MAY	季频率，该年结束于五月
(B)Q(S)-JUN	季频率，该年结束于六月
(B)Q(S)-JUL	季频率，该年结束于七月
(B)Q(S)-AUG	季频率，该年结束于八月
(B)Q(S)-SEP	季频率，该年结束于九月
(B)Q(S)-OCT	季频率，该年结束于十月
(B)Q(S)-NOV	季频率，该年结束于十一月
(B)A(S)-DEC	年频率，锚定结束于十二月，与“A”相同
(B)A(S)-JAN	年频率，锚定结束于一月
(B)A(S)-FEB	年频率，锚定结束于二月
(B)A(S)-MAR	年频率，锚定结束于三月
(B)A(S)-APR	年频率，锚定结束于四月
(B)A(S)-MAY	年频率，锚定结束于五月
(B)A(S)-JUN	年频率，锚定结束于六月
(B)A(S)-JUL	年频率，锚定结束于七月
(B)A(S)-AUG	年频率，锚定结束于八月
(B)A(S)-SEP	年频率，锚定结束于九月
(B)A(S)-OCT	年频率，锚定结束于十月
(B)A(S)-NOV	年频率，锚定结束于十一月

这些别名可以用作 `date_range`、`bdate_range`、`DatetimeIndex` 及其它时间序列函数的参数。

## 锚定偏移量的含义

对于偏移量锚定于开始或结束指定频率（`MonthEnd`、`MonthBegin`、`WeekEnd` 等）下列规则应用于前滚与后滚。

`n` 不为 0 时，如果给定日期不是锚定日期，将寻找下一个或上一个锚点，并向前或向后移动  $|n|-1$  步。

```
In [236]: pd.Timestamp('2014-01-02') + pd.offsets.MonthBegin(n=1)
Out[236]: Timestamp('2014-02-01 00:00:00')

In [237]: pd.Timestamp('2014-01-02') + pd.offsets.MonthEnd(n=1)
Out[237]: Timestamp('2014-01-31 00:00:00')

In [238]: pd.Timestamp('2014-01-02') - pd.offsets.MonthBegin(n=1)
Out[238]: Timestamp('2014-01-01 00:00:00')

In [239]: pd.Timestamp('2014-01-02') - pd.offsets.MonthEnd(n=1)
Out[239]: Timestamp('2013-12-31 00:00:00')

In [240]: pd.Timestamp('2014-01-02') + pd.offsets.MonthBegin(n=4)
Out[240]: Timestamp('2014-05-01 00:00:00')

In [241]: pd.Timestamp('2014-01-02') - pd.offsets.MonthBegin(n=4)
Out[241]: Timestamp('2013-10-01 00:00:00')
```

如果给定日期是锚定日期，则向前（或向后）移动  $|n|$  个点。

```
In [242]: pd.Timestamp('2014-01-01') + pd.offsets.MonthBegin(n=1)
Out[242]: Timestamp('2014-02-01 00:00:00')

In [243]: pd.Timestamp('2014-01-31') + pd.offsets.MonthEnd(n=1)
Out[243]: Timestamp('2014-02-28 00:00:00')

In [244]: pd.Timestamp('2014-01-01') - pd.offsets.MonthBegin(n=1)
Out[244]: Timestamp('2013-12-01 00:00:00')

In [245]: pd.Timestamp('2014-01-31') - pd.offsets.MonthEnd(n=1)
Out[245]: Timestamp('2013-12-31 00:00:00')

In [246]: pd.Timestamp('2014-01-01') + pd.offsets.MonthBegin(n=4)
Out[246]: Timestamp('2014-05-01 00:00:00')

In [247]: pd.Timestamp('2014-01-31') - pd.offsets.MonthBegin(n=4)
Out[247]: Timestamp('2013-10-01 00:00:00')
```

`n=0` 时，如果日期在锚点，则不移动，否则将前滚至下一个锚点。

```
In [248]: pd.Timestamp('2014-01-02') + pd.offsets.MonthBegin(n=0)
Out[248]: Timestamp('2014-02-01 00:00:00')

In [249]: pd.Timestamp('2014-01-02') + pd.offsets.MonthEnd(n=0)
Out[249]: Timestamp('2014-01-31 00:00:00')

In [250]: pd.Timestamp('2014-01-01') + pd.offsets.MonthBegin(n=0)
Out[250]: Timestamp('2014-01-01 00:00:00')

In [251]: pd.Timestamp('2014-01-31') + pd.offsets.MonthEnd(n=0)
Out[251]: Timestamp('2014-01-31 00:00:00')
```

## 假日与节日日历

用假日与日历可以轻松定义 `CustomBusinessDay` 假日规则，或其它分析所需的预设假日。

`AbstractHolidayCalendar` 类支持所有返回假日列表的方法，并且仅需在指定假日日历类里定义 `rules`。

`start_date` 与 `end_date` 类属性决定了假日的范围。该操作会覆盖 `AbstractHolidayCalendar` 类，适用于所有日历子类。`USFederalHolidayCalendar` 是仅有的假日日历，主要用作开发其它日历的示例。

固定日期的假日，如美国阵亡将士纪念日或美国国庆日（7月4日），取决于该假日是否是在周末，可以使用以下规则：

规则	说明
<code>nearest_workday</code>	把星期六移至星期五，星期日移至星期一
<code>sunday_to_monday</code>	星期六紧接着星期一
<code>next_monday_or_tuesday</code>	把星期六移至星期一，并把星期日/星期一移至星期二
<code>previous_friday</code>	把星期六与星期日移至上一个星期五
<code>next_monday</code>	把星期六与星期日移至下一个星期一

下例展示如何定义假日与假日日历：

```
In [252]: from pandas.tseries.holiday import Holiday, USMemorialDay, \
.....: AbstractHolidayCalendar, nearest_workday, MO
.....:

In [253]: class ExampleCalendar(AbstractHolidayCalendar):
.....: rules = [
.....: USMemorialDay,
.....: Holiday('July 4th', month=7, day=4, observance=nearest_workday),
.....: Holiday('Columbus Day', month=10, day=1,
.....: offset=pd.DateOffset(weekday=MO(2)))]
.....:

In [254]: cal = ExampleCalendar()

In [255]: cal.holidays(datetime.datetime(2012, 1, 1), datetime.datetime(2012, 12, 31))
Out[255]: DatetimeIndex(['2012-05-28', '2012-07-04', '2012-10-08'], dtype='datetime64[ns]', freq=None)
```



⋮ tip 提示

`weekday=M0(2)` 与 `2 * Week(weekday=2)` 相同。

⋮

用这个日历创建索引，或计算偏移量，将跳过周末与假日（如，纪念日与国庆节）。下列代码用 `ExampleCalendar` 设定自定义工作日偏移量。至于其它偏移量，可以用于创建 `DatetimeIndex` 或添加到 `datetime` 与 `Timestamp` 对象。

```
In [256]: pd.date_range(start='7/1/2012', end='7/10/2012',
.....: freq=pd.offsets.CDay(calendar=cal)).to_pydatetime()
.....:
Out[256]:
array([datetime.datetime(2012, 7, 2, 0, 0),
 datetime.datetime(2012, 7, 3, 0, 0),
 datetime.datetime(2012, 7, 5, 0, 0),
 datetime.datetime(2012, 7, 6, 0, 0),
 datetime.datetime(2012, 7, 9, 0, 0),
 datetime.datetime(2012, 7, 10, 0, 0)], dtype=object)

In [257]: offset = pd.offsets.CustomBusinessDay(calendar=cal)

In [258]: datetime.datetime(2012, 5, 25) + offset
Out[258]: Timestamp('2012-05-29 00:00:00')

In [259]: datetime.datetime(2012, 7, 3) + offset
Out[259]: Timestamp('2012-07-05 00:00:00')

In [260]: datetime.datetime(2012, 7, 3) + 2 * offset
Out[260]: Timestamp('2012-07-06 00:00:00')

In [261]: datetime.datetime(2012, 7, 6) + offset
Out[261]: Timestamp('2012-07-09 00:00:00')
```

`AbstractHolidayCalendar` 的类属性 `start_date` 与 `end_date` 定义日期范围。默认值如下：

```
In [262]: AbstractHolidayCalendar.start_date
Out[262]: Timestamp('1970-01-01 00:00:00')

In [263]: AbstractHolidayCalendar.end_date
Out[263]: Timestamp('2030-12-31 00:00:00')
```

这两个日期可以用 `datetime`、`Timestamp`、字符串 修改。

```
In [264]: AbstractHolidayCalendar.start_date = datetime.datetime(2012, 1, 1)

In [265]: AbstractHolidayCalendar.end_date = datetime.datetime(2012, 12, 31)

In [266]: cal.holidays()
Out[266]: DatetimeIndex(['2012-05-28', '2012-07-04', '2012-10-08'], dtype='datetime64[ns]', freq=None)
```

`get_calender` 函数通过日历名称访问日历，返回的是日历实例。任意导入的日历都自动适用于此函数。同时，`HolidayCalendarFactory` 还提供了一个创建日历组合或含附加规则日历的简易接口。

```
In [267]: from pandas.tseries.holiday import get_calendar, HolidayCalendarFactory,\
.....: USLaborDay
.....:

In [268]: cal = get_calendar('ExampleCalendar')

In [269]: cal.rules
Out[269]:
[Holiday: Memorial Day (month=5, day=31, offset=<DateOffset: weekday=MO(-1)>),
 Holiday: July 4th (month=7, day=4, observance=<function nearest_workday at 0x7f2460862c20>),
 Holiday: Columbus Day (month=10, day=1, offset=<DateOffset: weekday=MO(+2)>)]

In [270]: new_cal = HolidayCalendarFactory('NewExampleCalendar', cal, USLaborDay)

In [271]: new_cal.rules
Out[271]:
[Holiday: Labor Day (month=9, day=1, offset=<DateOffset: weekday=MO(+1)>),
 Holiday: Memorial Day (month=5, day=31, offset=<DateOffset: weekday=MO(-1)>),
 Holiday: July 4th (month=7, day=4, observance=<function nearest_workday at 0x7f2460862c20>),
 Holiday: Columbus Day (month=10, day=1, offset=<DateOffset: weekday=MO(+2)>)]
```

## 时间序列实例方法

### 移位与延迟

有时，需要整体向前或向后移动时间序列里的值，这就是移位与延迟。实现这一操作的方法是 `shift()`，该方法适用于所有 pandas 对象。

```
In [272]: ts = pd.Series(range(len(rng)), index=rng)

In [273]: ts = ts[:5]

In [274]: ts.shift(1)
Out[274]:
2012-01-01 NaN
2012-01-02 0.0
2012-01-03 1.0
Freq: D, dtype: float64
```

`shift` 方法支持 `freq` 参数，可以把 `DateOffset`、`timedelta` 对象、[偏移量别名](#) 作为参数值：

```
In [275]: ts.shift(5, freq=pd.offsets.BDay())
Out[275]:
2012-01-06 0
2012-01-09 1
2012-01-10 2
Freq: B, dtype: int64
```

```
In [276]: ts.shift(5, freq='BM')
Out[276]:
2012-05-31 0
2012-05-31 1
2012-05-31 2
Freq: D, dtype: int64
```

除更改数据与索引的对齐方式外，`DataFrame` 与 `Series` 对象还提供了 `tshift()` 便捷方法，可以指定偏移量修改索引日期。

```
In [277]: ts.tshift(5, freq='D')
Out[277]:
2012-01-06 0
2012-01-07 1
2012-01-08 2
Freq: D, dtype: int64
```

注意，使用 `tshift()` 时，因为数据没有重对齐，`NaN` 不会排在前面。

## 频率转换

改变频率的函数主要是 `asfreq()`。对于 `DatetimeIndex`，这就是一个调用 `reindex()`，并生成 `date_range` 的便捷打包器。

```

In [278]: dr = pd.date_range('1/1/2010', periods=3, freq=3 * pd.offsets.BDay())

In [279]: ts = pd.Series(np.random.randn(3), index=dr)

In [280]: ts
Out[280]:
2010-01-01 1.494522
2010-01-06 -0.778425
2010-01-11 -0.253355
Freq: 3B, dtype: float64

In [281]: ts.ashfreq(pd.offsets.BDay())
Out[281]:
2010-01-01 1.494522
2010-01-04 NaN
2010-01-05 NaN
2010-01-06 -0.778425
2010-01-07 NaN
2010-01-08 NaN
2010-01-11 -0.253355
Freq: B, dtype: float64

```

`ashfreq` 用起来很方便，可以为频率转化后出现的任意间隔指定插值方法。

```

In [282]: ts.ashfreq(pd.offsets.BDay(), method='pad')
Out[282]:
2010-01-01 1.494522
2010-01-04 1.494522
2010-01-05 1.494522
2010-01-06 -0.778425
2010-01-07 -0.778425
2010-01-08 -0.778425
2010-01-11 -0.253355
Freq: B, dtype: float64

```

## 向前与向后填充

与 `ashfreq` 与 `reindex` 相关的是 `fillna()`，有关文档请参阅[缺失值](#)。

## 转换 Python 日期与时间

用 `to_datetime` 方法可以把 `DatetimeIndex` 转换为 Python 原生 `datetime.datetime` 对象数组。

## 重采样

::: danger 警告

0.18.0 版修改了 `.resample` 接口，现在的 `.resample` 更灵活，更像 `groupby`。参阅[更新文档](#)，对比新旧版本操作的区别。

...

Pandas 有一个虽然简单，但却强大、高效的功能，可在频率转换时执行重采样，如，将秒数据转换为 5 分钟数据，这种操作在金融等领域里的应用非常广泛。

`resample()` 是基于时间的分组操作，每个组都遵循归纳方法。参阅 [Cookbook 示例](#) 了解高级应用。

从 0.18.0 版开始，`resample()` 可以直接用于 `DataFrameGroupBy` 对象，参阅 [groupby 文档](#)。

::: tip 注意

`.resample()` 类似于基于时间偏移量的 `rolling()` 操作，请参阅[这里](#)的讨论。

...

## 基础知识

```
In [283]: rng = pd.date_range('1/1/2012', periods=100, freq='S')

In [284]: ts = pd.Series(np.random.randint(0, 500, len(rng)), index=rng)

In [285]: ts.resample('5Min').sum()
Out[285]:
2012-01-01 25103
Freq: 5T, dtype: int64
```

`resample` 函数非常灵活，可以指定多种频率转换与重采样参数。

任何支持派送 ([dispatch](#)) 的函数都可用于 `resample` 返回对象，包括 `sum`、`mean`、`std`、`sem`、`max`、`min`、`mid`、`median`、`first`、`last`、`ohlc`：

```
In [286]: ts.resample('5Min').mean()
Out[286]:
2012-01-01 251.03
Freq: 5T, dtype: float64

In [287]: ts.resample('5Min').ohlc()
Out[287]:
 open high low close
2012-01-01 308 460 9 205

In [288]: ts.resample('5Min').max()
Out[288]:
2012-01-01 460
Freq: 5T, dtype: int64
```

对于下采样，`closed` 可以设置为 `left` 或 `right`，用于指定关闭哪一端间隔：

```
In [289]: ts.resample('5Min', closed='right').mean()
Out[289]:
2011-12-31 23:55:00 308.000000
2012-01-01 00:00:00 250.454545
Freq: 5T, dtype: float64

In [290]: ts.resample('5Min', closed='left').mean()
Out[290]:
2012-01-01 251.03
Freq: 5T, dtype: float64
```

`label`、`loffset` 等参数用于生成标签。`label` 指定生成的结果是否要为间隔标注起始时间。`loffset` 调整输出标签的时间。

```
In [291]: ts.resample('5Min').mean() # 默认为 label='left'
Out[291]:
2012-01-01 251.03
Freq: 5T, dtype: float64

In [292]: ts.resample('5Min', label='left').mean()
Out[292]:
2012-01-01 251.03
Freq: 5T, dtype: float64

In [293]: ts.resample('5Min', label='left', loffset='1s').mean()
Out[293]:
2012-01-01 00:00:01 251.03
dtype: float64
```

### ::: danger 警告

除了 `M`、`A`、`Q`、`BM`、`BA`、`BQ`、`W` 的默认值是 `right` 外，其它频率偏移量的 `label` 与 `closed` 默认值都是 `left`。

这种操作可能会导致时间回溯，即后面的时间会被拉回到前面的时间，如下例的 `BusinessDay` 频率所示。

```
In [294]: s = pd.date_range('2000-01-01', '2000-01-05').to_series()
```

```
In [295]: s.iloc[2] = pd.NaT
```

```
In [296]: s.dt.weekday_name
```

```
Out[296]:
```

```
2000-01-01 Saturday
```

```
2000-01-02 Sunday
```

```
2000-01-03 NaN
```

```
2000-01-04 Tuesday
```

```
2000-01-05 Wednesday
```

```
Freq: D, dtype: object
```

```
默认为: label='left', closed='left'
```

```
In [297]: s.resample('B').last().dt.weekday_name
```

```
Out[297]:
```

```
1999-12-31 Sunday
```

```
2000-01-03 NaN
```

```
2000-01-04 Tuesday
```

```
2000-01-05 Wednesday
```

```
Freq: B, dtype: object
```

看到了吗？星期日被拉回到了上一个星期五。要想把星期日移至星期一，改用以下代码：

```
In [298]: s.resample('B', label='right', closed='right').last().dt.weekday_name
```

```
Out[298]:
```

```
2000-01-03 Sunday
```

```
2000-01-04 Tuesday
```

```
2000-01-05 Wednesday
```

```
Freq: B, dtype: object
```

...

`axis` 参数的值为 0 或 1，并可指定 `DataFrame` 重采样的轴。

`kind` 参数可以是 `timestamp` 或 `period`，转换为时间戳或时间段形式的索引。`resample` 默认保留输入的日期时间形式。

重采样 `period` 数据时（详情见下文），`convention` 可以设置为 `start` 或 `end`。指定低频时间段如何转换为高频时间段。

## 上采样

上采样可以指定上采样的方式及插入时间间隔的 `limit` 参数：

```
从秒到每 250 毫秒
In [299]: ts[:2].resample('250L').asfreq()
Out[299]:
2012-01-01 00:00:00.000 308.0
2012-01-01 00:00:00.250 NaN
2012-01-01 00:00:00.500 NaN
2012-01-01 00:00:00.750 NaN
2012-01-01 00:00:01.000 204.0
Freq: 250L, dtype: float64

In [300]: ts[:2].resample('250L').ffill()
Out[300]:
2012-01-01 00:00:00.000 308
2012-01-01 00:00:00.250 308
2012-01-01 00:00:00.500 308
2012-01-01 00:00:00.750 308
2012-01-01 00:00:01.000 204
Freq: 250L, dtype: int64

In [301]: ts[:2].resample('250L').ffill(limit=2)
Out[301]:
2012-01-01 00:00:00.000 308.0
2012-01-01 00:00:00.250 308.0
2012-01-01 00:00:00.500 308.0
2012-01-01 00:00:00.750 NaN
2012-01-01 00:00:01.000 204.0
Freq: 250L, dtype: float64
```

## 稀疏重采样

相对于时间点总量，稀疏时间序列重采样的点要少很多。单纯上采样稀疏系列可能会生成很多中间值。未指定填充值，即 `fill_method` 是 `None` 时，中间值将填充为 `NaN`。

鉴于 `resample` 是基于时间的分组，下列这种方法可以有效重采样，只是分组不是都为 `NaN`。

```
In [302]: rng = pd.date_range('2014-1-1', periods=100, freq='D') + pd.Timedelta('1s')

In [303]: ts = pd.Series(range(100), index=rng)
```

对 `Series` 全范围重采样。



```
In [304]: ts.resample('3T').sum()
Out[304]:
2014-01-01 00:00:00 0
2014-01-01 00:03:00 0
2014-01-01 00:06:00 0
2014-01-01 00:09:00 0
2014-01-01 00:12:00 0
..
2014-04-09 23:48:00 0
2014-04-09 23:51:00 0
2014-04-09 23:54:00 0
2014-04-09 23:57:00 0
2014-04-10 00:00:00 99
Freq: 3T, Length: 47521, dtype: int64
```

对以下包含点的分组重采样：

```
In [305]: from functools import partial

In [306]: from pandas.tseries.frequencies import to_offset

In [307]: def round(t, freq):
.....: freq = to_offset(freq)
.....: return pd.Timestamp((t.value // freq.delta.value) * freq.delta.value)
.....:

In [308]: ts.groupby(partial(round, freq='3T')).sum()
Out[308]:
2014-01-01 0
2014-01-02 1
2014-01-03 2
2014-01-04 3
2014-01-05 4
..
2014-04-06 95
2014-04-07 96
2014-04-08 97
2014-04-09 98
2014-04-10 99
Length: 100, dtype: int64
```

## 聚合

类似于聚合 API，Groupby API 及窗口函数 API，Resampler 可以有选择地重采样。

DataFrame 重采样，默认用相同函数操作所有列。

```
In [309]: df = pd.DataFrame(np.random.randn(1000, 3),
.....: index=pd.date_range('1/1/2012', freq='S', periods=1000),
.....: columns=['A', 'B', 'C'])
.....:
```

```
In [310]: r = df.resample('3T')
```

```
In [311]: r.mean()
```

```
Out[311]:
```

	A	B	C
2012-01-01 00:00:00	-0.033823	-0.121514	-0.081447
2012-01-01 00:03:00	0.056909	0.146731	-0.024320
2012-01-01 00:06:00	-0.058837	0.047046	-0.052021
2012-01-01 00:09:00	0.063123	-0.026158	-0.066533
2012-01-01 00:12:00	0.186340	-0.003144	0.074752
2012-01-01 00:15:00	-0.085954	-0.016287	-0.050046

标准 `getitem` 操作可以指定的一列或多列。

```
In [312]: r['A'].mean()
```

```
Out[312]:
```

2012-01-01 00:00:00	-0.033823
2012-01-01 00:03:00	0.056909
2012-01-01 00:06:00	-0.058837
2012-01-01 00:09:00	0.063123
2012-01-01 00:12:00	0.186340
2012-01-01 00:15:00	-0.085954

Freq: 3T, Name: A, dtype: float64

```
In [313]: r[['A', 'B']].mean()
```

```
Out[313]:
```

	A	B
2012-01-01 00:00:00	-0.033823	-0.121514
2012-01-01 00:03:00	0.056909	0.146731
2012-01-01 00:06:00	-0.058837	0.047046
2012-01-01 00:09:00	0.063123	-0.026158
2012-01-01 00:12:00	0.186340	-0.003144
2012-01-01 00:15:00	-0.085954	-0.016287

聚合还支持函数列表与字典，输出的是 `DataFrame`。

```
In [314]: r['A'].agg([np.sum, np.mean, np.std])
```

```
Out[314]:
```

	sum	mean	std
2012-01-01 00:00:00	-6.088060	-0.033823	1.043263
2012-01-01 00:03:00	10.243678	0.056909	1.058534
2012-01-01 00:06:00	-10.590584	-0.058837	0.949264
2012-01-01 00:09:00	11.362228	0.063123	1.028096
2012-01-01 00:12:00	33.541257	0.186340	0.884586
2012-01-01 00:15:00	-8.595393	-0.085954	1.035476

重采样后的 `DataFrame`，可以为每列指定函数列表，生成结构化索引的聚合结果：

```
In [315]: r.agg([np.sum, np.mean])
Out[315]:
```

	A		B		C	
	sum	mean	sum	mean	sum	mean
2012-01-01 00:00:00	-6.088060	-0.033823	-21.872530	-0.121514	-14.660515	-0.081447
2012-01-01 00:03:00	10.243678	0.056909	26.411633	0.146731	-4.377642	-0.024320
2012-01-01 00:06:00	-10.590584	-0.058837	8.468289	0.047046	-9.363825	-0.052021
2012-01-01 00:09:00	11.362228	0.063123	-4.708526	-0.026158	-11.975895	-0.066533
2012-01-01 00:12:00	33.541257	0.186340	-0.565895	-0.003144	13.455299	0.074752
2012-01-01 00:15:00	-8.595393	-0.085954	-1.628689	-0.016287	-5.004580	-0.050046

把字典传递给 `aggregate`，可以为 `DataFrame` 里不同的列应用不同聚合函数。

```
In [316]: r.agg({'A': np.sum,
.....: 'B': lambda x: np.std(x, ddof=1)})
.....:
Out[316]:
```

	A	B
2012-01-01 00:00:00	-6.088060	1.001294
2012-01-01 00:03:00	10.243678	1.074597
2012-01-01 00:06:00	-10.590584	0.987309
2012-01-01 00:09:00	11.362228	0.944953
2012-01-01 00:12:00	33.541257	1.095025
2012-01-01 00:15:00	-8.595393	1.035312

还可以用字符串代替函数名。为了让字符串有效，必须在重采样对象上操作：

```
In [317]: r.agg({'A': 'sum', 'B': 'std'})
Out[317]:
```

	A	B
2012-01-01 00:00:00	-6.088060	1.001294
2012-01-01 00:03:00	10.243678	1.074597
2012-01-01 00:06:00	-10.590584	0.987309
2012-01-01 00:09:00	11.362228	0.944953
2012-01-01 00:12:00	33.541257	1.095025
2012-01-01 00:15:00	-8.595393	1.035312

甚至还可以为每列单独多个聚合函数。

```
In [318]: r.agg({'A': ['sum', 'std'], 'B': ['mean', 'std']})
Out[318]:
```

	A		B	
	sum	std	mean	std
2012-01-01 00:00:00	-6.088060	1.043263	-0.121514	1.001294
2012-01-01 00:03:00	10.243678	1.058534	0.146731	1.074597
2012-01-01 00:06:00	-10.590584	0.949264	0.047046	0.987309
2012-01-01 00:09:00	11.362228	1.028096	-0.026158	0.944953
2012-01-01 00:12:00	33.541257	0.884586	-0.003144	1.095025
2012-01-01 00:15:00	-8.595393	1.035476	-0.016287	1.035312

如果 `DataFrame` 用的不是 `datetime` 型索引，则可以基于 `datetime` 数据列重采样，用关键字 `on` 控制。

```
In [319]: df = pd.DataFrame({'date': pd.date_range('2015-01-01', freq='W', periods=5),
.....: 'a': np.arange(5)},
.....: index=pd.MultiIndex.from_arrays([
.....: [1, 2, 3, 4, 5],
.....: pd.date_range('2015-01-01', freq='W', periods=5)],
.....: names=['v', 'd']))
```

```
In [320]: df
```

```
Out[320]:
```

		date	a
v	d		
1	2015-01-04	2015-01-04	0
2	2015-01-11	2015-01-11	1
3	2015-01-18	2015-01-18	2
4	2015-01-25	2015-01-25	3
5	2015-02-01	2015-02-01	4

```
In [321]: df.resample('M', on='date').sum()
```

```
Out[321]:
```

	a
date	
2015-01-31	6
2015-02-28	4

同样，还可以对 `datetime MultiIndex` 重采样，通过关键字 `level` 传递名字与位置。

```
In [322]: df.resample('M', level='d').sum()
```

```
Out[322]:
```

	a
d	
2015-01-31	6
2015-02-28	4

## 分组迭代

`Resampler` 对象迭代分组数据的操作非常自然，类似于 `itertools.groupby()`：

```
In [323]: small = pd.Series(
.....: range(6),
.....: index=pd.to_datetime(['2017-01-01T00:00:00',
.....: '2017-01-01T00:30:00',
.....: '2017-01-01T00:31:00',
.....: '2017-01-01T01:00:00',
.....: '2017-01-01T03:00:00',
.....: '2017-01-01T03:05:00']))
.....:)
.....:
```

```
In [324]: resampled = small.resample('H')
```

```
In [325]: for name, group in resampled:
.....: print("Group: ", name)
.....: print("-" * 27)
.....: print(group, end="\n\n")
.....:
```

```
Group: 2017-01-01 00:00:00
```

```

2017-01-01 00:00:00 0
2017-01-01 00:30:00 1
2017-01-01 00:31:00 2
```

```
dtype: int64
```

```
Group: 2017-01-01 01:00:00
```

```

2017-01-01 01:00:00 3
```

```
dtype: int64
```

```
Group: 2017-01-01 02:00:00
```

```

Series([], dtype: int64)
```

```
Group: 2017-01-01 03:00:00
```

```

2017-01-01 03:00:00 4
2017-01-01 03:05:00 5
```

```
dtype: int64
```

了解更多详情，请参阅[分组迭代](#)或 `itertools.groupby()`。

## 时间跨度表示

规律时间间隔可以用 pandas 的 `Period` 对象表示，`Period` 对象序列叫做 `PeriodIndex`，用便捷函数 `period_range` 创建。

### Period

`Period` 表示时间跨度，即时间段，如年、季、月、日等。关键字 `freq` 与频率别名可以指定时间段。`freq` 表示的是 `Period` 的时间跨度，不能为负，如，`-3D`。

```

In [326]: pd.Period('2012', freq='A-DEC')
Out[326]: Period('2012', 'A-DEC')

In [327]: pd.Period('2012-1-1', freq='D')
Out[327]: Period('2012-01-01', 'D')

In [328]: pd.Period('2012-1-1 19:00', freq='H')
Out[328]: Period('2012-01-01 19:00', 'H')

In [329]: pd.Period('2012-1-1 19:00', freq='5H')
Out[329]: Period('2012-01-01 19:00', '5H')

```

时间段加减法按自身频率位移。不同频率的时间段不可进行算术运算。

```

In [330]: p = pd.Period('2012', freq='A-DEC')

In [331]: p + 1
Out[331]: Period('2013', 'A-DEC')

In [332]: p - 3
Out[332]: Period('2009', 'A-DEC')

In [333]: p = pd.Period('2012-01', freq='2M')

In [334]: p + 2
Out[334]: Period('2012-05', '2M')

In [335]: p - 1
Out[335]: Period('2011-11', '2M')

In [336]: p == pd.Period('2012-01', freq='3M')

IncompatibleFrequency Traceback (most recent call last)
<ipython-input-336-4b67dc0b596c> in <module>
----> 1 p == pd.Period('2012-01', freq='3M')

/pandas/pandas/_libs/tslibs/period.pyx in pandas._libs.tslibs.period._Period.__richcmp__()

IncompatibleFrequency: Input has different freq=3M from Period(freq=2M)

```

freq 的频率为日或更高频率时，如 D、H、T、S、L、U、N，offsets 与 timedelta 可以用相同频率实现加法。否则，会触发 ValueError。

```

In [337]: p = pd.Period('2014-07-01 09:00', freq='H')

In [338]: p + pd.offsets.Hour(2)
Out[338]: Period('2014-07-01 11:00', 'H')

In [339]: p + datetime.timedelta(minutes=120)
Out[339]: Period('2014-07-01 11:00', 'H')

In [340]: p + np.timedelta64(7200, 's')
Out[340]: Period('2014-07-01 11:00', 'H')
In [1]: p + pd.offsets.Minute(5)
Traceback
...
ValueError: Input has different freq from Period(freq=H)

```

如果 `Period` 为其它频率，只有相同频率的 `offsets` 可以相加。否则，会触发 `ValueError`。

```

In [341]: p = pd.Period('2014-07', freq='M')

In [342]: p + pd.offsets.MonthEnd(3)
Out[342]: Period('2014-10', 'M')
In [1]: p + pd.offsets.MonthBegin(3)
Traceback
...
ValueError: Input has different freq from Period(freq=M)

```

用相同频率计算不同时间段实例之间的区别，将返回这些实例之间的频率单元数量。

```

In [343]: pd.Period('2012', freq='A-DEC') - pd.Period('2002', freq='A-DEC')
Out[343]: <10 * YearEnds: month=12>

```

## PeriodIndex 与 period\_range

`period_range` 便捷函数可以创建有规律的 `Period` 对象序列，即 `PeriodIndex`。

```

In [344]: prng = pd.period_range('1/1/2011', '1/1/2012', freq='M')

In [345]: prng
Out[345]:
PeriodIndex(['2011-01', '2011-02', '2011-03', '2011-04', '2011-05', '2011-06',
 '2011-07', '2011-08', '2011-09', '2011-10', '2011-11', '2011-12',
 '2012-01'],
 dtype='period[M]', freq='M')

```

也可以直接用 `PeriodIndex` 创建：

```

In [346]: pd.PeriodIndex(['2011-1', '2011-2', '2011-3'], freq='M')
Out[346]: PeriodIndex(['2011-01', '2011-02', '2011-03'], dtype='period[M]', freq='M')

```

频率为复数时，输出的 `Period` 序列为复数时间段。

```
In [347]: pd.period_range(start='2014-01', freq='3M', periods=4)
Out[347]: PeriodIndex(['2014-01', '2014-04', '2014-07', '2014-10'], dtype='period[3M]', freq='3M')
```

Period 对象的 start 或 end 会被当作 PeriodIndex 的锚定终点，其频率与 PeriodIndex 的频率一样。

```
In [348]: pd.period_range(start=pd.Period('2017Q1', freq='Q'),
.....: end=pd.Period('2017Q2', freq='Q'), freq='M')
.....:
Out[348]: PeriodIndex(['2017-03', '2017-04', '2017-05', '2017-06'], dtype='period[M]', freq='M')
```

和 DatetimeIndex 一样，PeriodIndex 也可以作为 pandas 对象的索引。

```
In [349]: ps = pd.Series(np.random.randn(len(prng)), prng)

In [350]: ps
Out[350]:
2011-01 -2.916901
2011-02 0.514474
2011-03 1.346470
2011-04 0.816397
2011-05 2.258648
2011-06 0.494789
2011-07 0.301239
2011-08 0.464776
2011-09 -1.393581
2011-10 0.056780
2011-11 0.197035
2011-12 2.261385
2012-01 -0.329583
Freq: M, dtype: float64
```

PeriodIndex 的加减法与 Period 一样。



```

In [351]: idx = pd.period_range('2014-07-01 09:00', periods=5, freq='H')

In [352]: idx
Out[352]:
PeriodIndex(['2014-07-01 09:00', '2014-07-01 10:00', '2014-07-01 11:00',
 '2014-07-01 12:00', '2014-07-01 13:00'],
 dtype='period[H]', freq='H')

In [353]: idx + pd.offsets.Hour(2)
Out[353]:
PeriodIndex(['2014-07-01 11:00', '2014-07-01 12:00', '2014-07-01 13:00',
 '2014-07-01 14:00', '2014-07-01 15:00'],
 dtype='period[H]', freq='H')

In [354]: idx = pd.period_range('2014-07', periods=5, freq='M')

In [355]: idx
Out[355]: PeriodIndex(['2014-07', '2014-08', '2014-09', '2014-10', '2014-11'], dtype='period[M]',
 freq='M')

In [356]: idx + pd.offsets.MonthEnd(3)
Out[356]: PeriodIndex(['2014-10', '2014-11', '2014-12', '2015-01', '2015-02'], dtype='period[M]',
 freq='M')

```

`PeriodIndex` 有自己的数据类型，即 `period`，请参阅 [Period 数据类型](#)。

## Period 数据类型

0.19.0 版新增。

`PeriodIndex` 的自定义数据类型是 `period`，是 pandas 扩展数据类型，类似于带时区信息的数据类型（`datetime64[ns, tz]`）。

`Period` 数据类型支持 `freq` 属性，还可以用 `period[freq]` 表示，如，`period[D]` 或 `period[M]`，这里用的是频率字符串。

```

In [357]: pi = pd.period_range('2016-01-01', periods=3, freq='M')

In [358]: pi
Out[358]: PeriodIndex(['2016-01', '2016-02', '2016-03'], dtype='period[M]', freq='M')

In [359]: pi.dtype
Out[359]: period[M]

```

`period` 数据类型在 `.astype(...)` 里使用。允许改变 `PeriodIndex` 的 `freq`，如 `.asfreq()`，并用 `to_period()` 把 `DatetimeIndex` 转化为 `PeriodIndex`：

```

把月频改为日频
In [360]: pi.astype('period[D]')
Out[360]: PeriodIndex(['2016-01-31', '2016-02-29', '2016-03-31'], dtype='period[D]', freq='D')

转换为 DatetimeIndex
In [361]: pi.astype('datetime64[ns]')
Out[361]: DatetimeIndex(['2016-01-01', '2016-02-01', '2016-03-01'], dtype='datetime64[ns]', freq='MS')

转换为 PeriodIndex
In [362]: dti = pd.date_range('2011-01-01', freq='M', periods=3)

In [363]: dti
Out[363]: DatetimeIndex(['2011-01-31', '2011-02-28', '2011-03-31'], dtype='datetime64[ns]', freq='M')

In [364]: dti.astype('period[M]')
Out[364]: PeriodIndex(['2011-01', '2011-02', '2011-03'], dtype='period[M]', freq='M')

```

## PeriodIndex 局部字符串索引

与 `DatetimeIndex` 一样，`PeriodIndex` 可以把日期与字符串传递给 `Series` 与 `DataFrame`。详情请参阅 [DatetimeIndex 局部字符串索引](#)。

```

In [365]: ps['2011-01']
Out[365]: -2.9169013294054507

In [366]: ps[datetime.datetime(2011, 12, 25):]
Out[366]:
2011-12 2.261385
2012-01 -0.329583
Freq: M, dtype: float64

In [367]: ps['10/31/2011':'12/31/2011']
Out[367]:
2011-10 0.056780
2011-11 0.197035
2011-12 2.261385
Freq: M, dtype: float64

```

传递比 `PeriodIndex` 更低频率的字符串会返回局部切片数据。

```

In [368]: ps['2011']
Out[368]:
2011-01 -2.916901
2011-02 0.514474
2011-03 1.346470
2011-04 0.816397
2011-05 2.258648
2011-06 0.494789
2011-07 0.301239
2011-08 0.464776
2011-09 -1.393581
2011-10 0.056780
2011-11 0.197035
2011-12 2.261385
Freq: M, dtype: float64

In [369]: dfp = pd.DataFrame(np.random.randn(600, 1),
.....: columns=['A'],
.....: index=pd.period_range('2013-01-01 9:00',
.....: periods=600,
.....: freq='T'))

In [370]: dfp
Out[370]:
 A
2013-01-01 09:00 -0.538468
2013-01-01 09:01 -1.365819
2013-01-01 09:02 -0.969051
2013-01-01 09:03 -0.331152
2013-01-01 09:04 -0.245334
...
2013-01-01 18:55 0.522460
2013-01-01 18:56 0.118710
2013-01-01 18:57 0.167517
2013-01-01 18:58 0.922883
2013-01-01 18:59 1.721104

[600 rows x 1 columns]

In [371]: dfp['2013-01-01 10H']
Out[371]:
 A
2013-01-01 10:00 -0.308975
2013-01-01 10:01 0.542520
2013-01-01 10:02 1.061068
2013-01-01 10:03 0.754005
2013-01-01 10:04 0.352933
...
2013-01-01 10:55 -0.865621
2013-01-01 10:56 -1.167818
2013-01-01 10:57 -2.081748
2013-01-01 10:58 -0.527146
2013-01-01 10:59 0.802298

```

```
[60 rows x 1 columns]
```

与 `DatetimeIndex` 一样，终点包含在结果范围之内。下例中的切片数据就是从 10:00 到 11:59。

```
In [372]: dfp['2013-01-01 10H':'2013-01-01 11H']
```

```
Out[372]:
```

```
 A
2013-01-01 10:00 -0.308975
2013-01-01 10:01 0.542520
2013-01-01 10:02 1.061068
2013-01-01 10:03 0.754005
2013-01-01 10:04 0.352933
...
2013-01-01 11:55 -0.590204
2013-01-01 11:56 1.539990
2013-01-01 11:57 -1.224826
2013-01-01 11:58 0.578798
2013-01-01 11:59 -0.685496
```

```
[120 rows x 1 columns]
```

## 频率转换与 `PeriodIndex` 重采样

`Period` 与 `PeriodIndex` 的频率可以用 `asfreq` 转换。下列代码开始于 2011 财年，结束时间为十二月：

```
In [373]: p = pd.Period('2011', freq='A-DEC')
```

```
In [374]: p
```

```
Out[374]: Period('2011', 'A-DEC')
```

可以把它转换为月频。使用 `how` 参数，指定是否返回开始或结束月份。

```
In [375]: p.asfreq('M', how='start')
```

```
Out[375]: Period('2011-01', 'M')
```

```
In [376]: p.asfreq('M', how='end')
```

```
Out[376]: Period('2011-12', 'M')
```

简称 `s` 与 `e` 用起来更方便：

```
In [377]: p.asfreq('M', 's')
```

```
Out[377]: Period('2011-01', 'M')
```

```
In [378]: p.asfreq('M', 'e')
```

```
Out[378]: Period('2011-12', 'M')
```

转换为“超级 period”，（如，年频就是季频的超级 period），自动返回包含输入时间段的超级 period：

```
In [379]: p = pd.Period('2011-12', freq='M')
```

```
In [380]: p.asfreq('A-NOV')
```

```
Out[380]: Period('2012', 'A-NOV')
```

注意，因为转换年频是在十一月结束的，2011 年 12 月的月时间段实际上是 2012 A-NOV period。

用锚定频率转换时间段，对经济学、商业等领域里的各种季度数据特别有用。很多公司都依据其财年开始月与结束月定义季度。因此，2011 年第一个季度有可能 2010 年就开始了，也有可能 2011 年过了几个月才开始。通过锚定频率，pandas 可以处理所有从 Q-JAN 至 Q-DEC 的季度频率。

Q-DEC 定义的是常规日历季度：

```
In [381]: p = pd.Period('2012Q1', freq='Q-DEC')
```

```
In [382]: p.asfreq('D', 's')
```

```
Out[382]: Period('2012-01-01', 'D')
```

```
In [383]: p.asfreq('D', 'e')
```

```
Out[383]: Period('2012-03-31', 'D')
```

Q-MAR 定义的是财年结束于三月：

```
In [384]: p = pd.Period('2011Q4', freq='Q-MAR')
```

```
In [385]: p.asfreq('D', 's')
```

```
Out[385]: Period('2011-01-01', 'D')
```

```
In [386]: p.asfreq('D', 'e')
```

```
Out[386]: Period('2011-03-31', 'D')
```

## 不同表现形式之间的转换

`to_period` 把时间戳转换为 `PeriodIndex`，`to_timestamp` 则执行反向操作。

```
In [387]: rng = pd.date_range('1/1/2012', periods=5, freq='M')
```

```
In [388]: ts = pd.Series(np.random.randn(len(rng)), index=rng)
```

```
In [389]: ts
```

```
Out[389]:
```

```
2012-01-31 1.931253
```

```
2012-02-29 -0.184594
```

```
2012-03-31 0.249656
```

```
2012-04-30 -0.978151
```

```
2012-05-31 -0.873389
```

```
Freq: M, dtype: float64
```

```
In [390]: ps = ts.to_period()
```

```
In [391]: ps
```

```
Out[391]:
```

```
2012-01 1.931253
```

```
2012-02 -0.184594
```

```
2012-03 0.249656
```

```
2012-04 -0.978151
```

```
2012-05 -0.873389
```

```
Freq: M, dtype: float64
```

```
In [392]: ps.to_timestamp()
```

```
Out[392]:
```

```
2012-01-01 1.931253
```

```
2012-02-01 -0.184594
```

```
2012-03-01 0.249656
```

```
2012-04-01 -0.978151
```

```
2012-05-01 -0.873389
```

```
Freq: MS, dtype: float64
```

记住 `s` 与 `e` 返回 `period` 开始或结束的时间戳：

```
In [393]: ps.to_timestamp('D', how='s')
```

```
Out[393]:
```

```
2012-01-01 1.931253
```

```
2012-02-01 -0.184594
```

```
2012-03-01 0.249656
```

```
2012-04-01 -0.978151
```

```
2012-05-01 -0.873389
```

```
Freq: MS, dtype: float64
```

用便捷算数函数可以转换时间段与时间戳。下例中，把以 11 月年度结束的季频转换为以下一个季度月末上午 9 点：

```

In [394]: prng = pd.period_range('1990Q1', '2000Q4', freq='Q-NOV')

In [395]: ts = pd.Series(np.random.randn(len(prng)), prng)

In [396]: ts.index = (prng.asfreq('M', 'e') + 1).asfreq('H', 's') + 9

In [397]: ts.head()
Out[397]:
1990-03-01 09:00 -0.109291
1990-06-01 09:00 -0.637235
1990-09-01 09:00 -1.735925
1990-12-01 09:00 2.096946
1991-03-01 09:00 -1.039926
Freq: H, dtype: float64

```

## 界外跨度表示

数据在 `Timestamp` 限定边界外时，参阅 [Timestamp 限制](#)，可以用 `PeriodIndex` 或 `Periods` 的 `Series` 执行计算。

```

In [398]: span = pd.period_range('1215-01-01', '1381-01-01', freq='D')

In [399]: span
Out[399]:
PeriodIndex(['1215-01-01', '1215-01-02', '1215-01-03', '1215-01-04',
 '1215-01-05', '1215-01-06', '1215-01-07', '1215-01-08',
 '1215-01-09', '1215-01-10',
 ...,
 '1380-12-23', '1380-12-24', '1380-12-25', '1380-12-26',
 '1380-12-27', '1380-12-28', '1380-12-29', '1380-12-30',
 '1380-12-31', '1381-01-01'],
 dtype='period[D]', length=60632, freq='D')

```

从基于 `int64` 的 `YYYYMMDD` 表示形式转换。

```

In [400]: s = pd.Series([20121231, 20141130, 99991231])

In [401]: s
Out[401]:
0 20121231
1 20141130
2 99991231
dtype: int64

In [402]: def conv(x):
.....: return pd.Period(year=x // 10000, month=x // 100 % 100,
.....: day=x % 100, freq='D')
.....:

In [403]: s.apply(conv)
Out[403]:
0 2012-12-31
1 2014-11-30
2 9999-12-31
dtype: period[D]

In [404]: s.apply(conv)[2]
Out[404]: Period('9999-12-31', 'D')

```

轻轻松松就可以将这些数据转换成 PeriodIndex：

```

In [405]: span = pd.PeriodIndex(s.apply(conv))

In [406]: span
Out[406]: PeriodIndex(['2012-12-31', '2014-11-30', '9999-12-31'], dtype='period[D]', freq='D')

```

## 时区控制

利用 `pytz` 与 `dateutil` 或标准库 `datetime.timezone` 对象，pandas 能以多种方式处理不同时区的时间戳。

### 处理时区

Pandas 对象默认不支持时区信息：

```

In [407]: rng = pd.date_range('3/6/2012 00:00', periods=15, freq='D')

In [408]: rng.tz is None
Out[408]: True

```

用 `date_range()`、`Timestamp`、`DatetimeIndex` 的 `tz_localize` 方法或 `tz` 关键字参数，可以为这些日期加上本地时区，即，把指定时区分配给不带时区的日期。还可以传递 `pytz`、`dateutil` 时区对象或奥尔森时区数据库



字符串。奥尔森时区字符串默认返回 `pytz` 时区对象。要返回 `dateutil` 时区对象，在字符串前加上 `dateutil/`。

- 用 `from pytz import common_timezones, all_timezones` 在 `pytz` 里查找通用时区。
- `dateutil` 使用操作系统时区，没有固定的列表，其通用时区名与 `pytz` 相同。

```
In [409]: import dateutil

pytz
In [410]: rng_pytz = pd.date_range('3/6/2012 00:00', periods=3, freq='D',
.....: tz='Europe/London')
.....:

In [411]: rng_pytz.tz
Out[411]: <DstTzInfo 'Europe/London' LMT-1 day, 23:59:00 STD>

dateutil
In [412]: rng_dateutil = pd.date_range('3/6/2012 00:00', periods=3, freq='D')

In [413]: rng_dateutil = rng_dateutil.tz_localize('dateutil/Europe/London')

In [414]: rng_dateutil.tz
Out[414]: tzfile('/usr/share/zoneinfo/Europe/London')

dateutil - utc special case
In [415]: rng_utc = pd.date_range('3/6/2012 00:00', periods=3, freq='D',
.....: tz=dateutil.tz.tzutc())
.....:

In [416]: rng_utc.tz
Out[416]: tzutc()
```

0.25.0 版新增。

```
datetime.timezone
In [417]: rng_utc = pd.date_range('3/6/2012 00:00', periods=3, freq='D',
.....: tz=datetime.timezone.utc)
.....:

In [418]: rng_utc.tz
Out[418]: datetime.timezone.utc
```

注意，`dateutil` 的 UTC 时区是个特例，要显式地创建 `dateutil.tz.tzutc` 实例。可以先创建其它时区对象。

```

In [419]: import pytz

pytz
In [420]: tz_pytz = pytz.timezone('Europe/London')

In [421]: rng_pytz = pd.date_range('3/6/2012 00:00', periods=3, freq='D')

In [422]: rng_pytz = rng_pytz.tz_localize(tz_pytz)

In [423]: rng_pytz.tz == tz_pytz
Out[423]: True

dateutil
In [424]: tz_dateutil = dateutil.tz.gettz('Europe/London')

In [425]: rng_dateutil = pd.date_range('3/6/2012 00:00', periods=3, freq='D',
.....: tz=tz_dateutil)
.....:

In [426]: rng_dateutil.tz == tz_dateutil
Out[426]: True

```

不同时区之间转换带时区的 pandas 对象时，用 `tz_convert` 方法。

```

In [427]: rng_pytz.tz_convert('US/Eastern')
Out[427]:
DatetimeIndex(['2012-03-05 19:00:00-05:00', '2012-03-06 19:00:00-05:00',
 '2012-03-07 19:00:00-05:00'],
 dtype='datetime64[ns, US/Eastern]', freq='D')

```

::: tip 注意

使用 `pytz` 时区时，对于相同的输入时区，`DatetimeIndex` 会构建一个与 `Timestamp` 不同的时区对象。`DatetimeIndex` 具有一组 `Timestamp` 对象，UTC 偏移量也不同，不能用一个 `pytz` 时区实例简洁地表示，`Timestamp` 则可以用来指定 UTC 偏移量表示一个时点。

```

In [428]: dti = pd.date_range('2019-01-01', periods=3, freq='D', tz='US/Pacific')

In [429]: dti.tz
Out[429]: <DstTzInfo 'US/Pacific' LMT-1 day, 16:07:00 STD>

In [430]: ts = pd.Timestamp('2019-01-01', tz='US/Pacific')

In [431]: ts.tz
Out[431]: <DstTzInfo 'US/Pacific' PST-1 day, 16:00:00 STD>

```

:::

::: danger 警告

注意不同支持库之间的转换。一些时区，`pytz` 与 `datetutil` 对时区的定义不一样。与 `US/Eastern` 等“标准”时区相比，那些更少见的时区的问题更严重。

...

### ::: danger 警告

注意不同版本时区支持库对时区的定义并不一致。在处理本地存储数据时使用一种版本的支持库，在运算时使用另一种版本的支持库，可能会引起问题。参阅[本文](#)了解如何处理这种问题。

...

### ::: danger 警告

对于 `pytz` 时区，直接把时区对象传递给 `datetime.datetime` 构建器是不对的，如，`datetime.datetime(2011, 1, 1, tz=pytz.timezone('US/Eastern'))`。反之，`datetime` 要在 `pytz` 时区对象上使用 `localize` 方法。

...

在后台，所有 `Timestamp` 都存储为 UTC。含时区信息的 `DatetimeIndex` 或 `Timestamp` 的值有其自己的本地化时区字段（日、小时、分钟等）。不过，对于不同时区时间戳，如果其 UTC 值相同，将被视作是相等的时间。

```
In [432]: rng_eastern = rng_utc.tz_convert('US/Eastern')

In [433]: rng_berlin = rng_utc.tz_convert('Europe/Berlin')

In [434]: rng_eastern[2]
Out[434]: Timestamp('2012-03-07 19:00:00-0500', tz='US/Eastern', freq='D')

In [435]: rng_berlin[2]
Out[435]: Timestamp('2012-03-08 01:00:00+0100', tz='Europe/Berlin', freq='D')

In [436]: rng_eastern[2] == rng_berlin[2]
Out[436]: True
```

不同时区 `Series` 之间的操作生成的是与 UTC 时间戳数据对齐的 UTC `Series`。

```

In [437]: ts_utc = pd.Series(range(3), pd.date_range('20130101', periods=3, tz='UTC'))

In [438]: eastern = ts_utc.tz_convert('US/Eastern')

In [439]: berlin = ts_utc.tz_convert('Europe/Berlin')

In [440]: result = eastern + berlin

In [441]: result
Out[441]:
2013-01-01 00:00:00+00:00 0
2013-01-02 00:00:00+00:00 2
2013-01-03 00:00:00+00:00 4
Freq: D, dtype: int64

In [442]: result.index
Out[442]:
DatetimeIndex(['2013-01-01 00:00:00+00:00', '2013-01-02 00:00:00+00:00',
 '2013-01-03 00:00:00+00:00'],
 dtype='datetime64[ns, UTC]', freq='D')

```

用 `tz_localize(None)` 或 `tz_convert(None)` 去掉时区信息。`tz_localize(None)` 去掉带本地时间表示的时区信息。`tz_convert(None)` 先把时间戳转为 UTC 时间，再去掉时区信息。

```

In [443]: didx = pd.date_range(start='2014-08-01 09:00', freq='H',
.....: periods=3, tz='US/Eastern')
.....:

In [444]: didx
Out[444]:
DatetimeIndex(['2014-08-01 09:00:00-04:00', '2014-08-01 10:00:00-04:00',
 '2014-08-01 11:00:00-04:00'],
 dtype='datetime64[ns, US/Eastern]', freq='H')

In [445]: didx.tz_localize(None)
Out[445]:
DatetimeIndex(['2014-08-01 09:00:00', '2014-08-01 10:00:00',
 '2014-08-01 11:00:00'],
 dtype='datetime64[ns]', freq='H')

In [446]: didx.tz_convert(None)
Out[446]:
DatetimeIndex(['2014-08-01 13:00:00', '2014-08-01 14:00:00',
 '2014-08-01 15:00:00'],
 dtype='datetime64[ns]', freq='H')

tz_convert(None) 等同于 tz_convert('UTC').tz_localize(None)
In [447]: didx.tz_convert('UTC').tz_localize(None)
Out[447]:
DatetimeIndex(['2014-08-01 13:00:00', '2014-08-01 14:00:00',
 '2014-08-01 15:00:00'],
 dtype='datetime64[ns]', freq='H')

```

## 本地化导致的混淆时间

`tz_localize` 不能决定时间戳的 UTC 偏移量，因为本地时区的夏时制（DST）会引起一些时间在一天内出现两次的问题（“时钟回调”）。下面的选项是有效的：

- `raise`：默认触发 `pytz.AmbiguousTimeError`
- `infer`：依据时间戳的单一性，尝试推断正确的偏移量
- `NaT`：用 `NaT` 替换混淆时间
- `bool`：`True` 代表夏时制（DST）时间，`False` 代表正常时间。数组型的 `bool` 值支持一组时间序列。

```
In [448]: rng_hourly = pd.DatetimeIndex(['11/06/2011 00:00', '11/06/2011 01:00',
.....: '11/06/2011 01:00', '11/06/2011 02:00'])
.....:
```

这种操作会引起混淆时间失败错误（'11/06/2011 01:00'）。

```
In [2]: rng_hourly.tz_localize('US/Eastern')
AmbiguousTimeError: Cannot infer dst time from Timestamp('2011-11-06 01:00:00'), try using the
 'ambiguous' argument
```

用下列指定的关键字控制混淆时间。

```
In [449]: rng_hourly.tz_localize('US/Eastern', ambiguous='infer')
Out[449]:
DatetimeIndex(['2011-11-06 00:00:00-04:00', '2011-11-06 01:00:00-04:00',
 '2011-11-06 01:00:00-05:00', '2011-11-06 02:00:00-05:00'],
 dtype='datetime64[ns, US/Eastern]', freq=None)

In [450]: rng_hourly.tz_localize('US/Eastern', ambiguous='NaT')
Out[450]:
DatetimeIndex(['2011-11-06 00:00:00-04:00', 'NaT', 'NaT',
 '2011-11-06 02:00:00-05:00'],
 dtype='datetime64[ns, US/Eastern]', freq=None)

In [451]: rng_hourly.tz_localize('US/Eastern', ambiguous=[True, True, False, False])
Out[451]:
DatetimeIndex(['2011-11-06 00:00:00-04:00', '2011-11-06 01:00:00-04:00',
 '2011-11-06 01:00:00-05:00', '2011-11-06 02:00:00-05:00'],
 dtype='datetime64[ns, US/Eastern]', freq=None)
```

## 本地化时不存在的时间

夏时制转换会移位本地时间一个小时，这样会创建一个不存在的本地时间（“时钟春季前滚”）。这种本地化操作会导致时间序列出现不存在的时间，此问题可以用 `nonexistent` 参数解决。下列都是有效的选项：

- `raise`：默认触发 `pytz.NonExistentTimeError`
- `NaT`：用 `NaT` 替换不存在的时间
- `shift_forward`：把不存在的时间前移至最近的真实时间
- `shift_backward`：把不存在的时间后滚至最近的真实时间

- `Timedelta` 对象：用 `timedelta` 移位不存在的时间

```
In [452]: dti = pd.date_range(start='2015-03-29 02:30:00', periods=3, freq='H')

2:30 是不存在的时间
```

对不存在的时间进行本地化操作默认会触发错误。

```
In [2]: dti.tz_localize('Europe/Warsaw')
NonExistentTimeError: 2015-03-29 02:30:00
```

把不存在的时间转换为 `NaT` 或移位时间

```
In [453]: dti
Out[453]:
DatetimeIndex(['2015-03-29 02:30:00', '2015-03-29 03:30:00',
 '2015-03-29 04:30:00'],
 dtype='datetime64[ns]', freq='H')

In [454]: dti.tz_localize('Europe/Warsaw', nonexistent='shift_forward')
Out[454]:
DatetimeIndex(['2015-03-29 03:00:00+02:00', '2015-03-29 03:30:00+02:00',
 '2015-03-29 04:30:00+02:00'],
 dtype='datetime64[ns, Europe/Warsaw]', freq='H')

In [455]: dti.tz_localize('Europe/Warsaw', nonexistent='shift_backward')
Out[455]:
DatetimeIndex(['2015-03-29 01:59:59.999999999+01:00',
 '2015-03-29 03:30:00+02:00',
 '2015-03-29 04:30:00+02:00'],
 dtype='datetime64[ns, Europe/Warsaw]', freq='H')

In [456]: dti.tz_localize('Europe/Warsaw', nonexistent=pd.Timedelta(1, unit='H'))
Out[456]:
DatetimeIndex(['2015-03-29 03:30:00+02:00', '2015-03-29 03:30:00+02:00',
 '2015-03-29 04:30:00+02:00'],
 dtype='datetime64[ns, Europe/Warsaw]', freq='H')

In [457]: dti.tz_localize('Europe/Warsaw', nonexistent='NaT')
Out[457]:
DatetimeIndex(['NaT', '2015-03-29 03:30:00+02:00',
 '2015-03-29 04:30:00+02:00'],
 dtype='datetime64[ns, Europe/Warsaw]', freq='H')
```

## 时区序列操作

无时区 `Series` 值的数据类型是 `datetime64[ns]`。

```
In [458]: s_naive = pd.Series(pd.date_range('20130101', periods=3))

In [459]: s_naive
Out[459]:
0 2013-01-01
1 2013-01-02
2 2013-01-03
dtype: datetime64[ns]
```

有时区 `Series` 值的数据类型是 `datetime64[ns, tz]`，`tz` 指的是时区。

```
In [460]: s_aware = pd.Series(pd.date_range('20130101', periods=3, tz='US/Eastern'))

In [461]: s_aware
Out[461]:
0 2013-01-01 00:00:00-05:00
1 2013-01-02 00:00:00-05:00
2 2013-01-03 00:00:00-05:00
dtype: datetime64[ns, US/Eastern]
```

这两种 `Series` 的时区信息都可以用 `.dt` 访问器操控，参阅 [dt 访问器](#)。

例如，本地化与把无时区时间戳转换为有时区时间戳。

```
In [462]: s_naive.dt.tz_localize('UTC').dt.tz_convert('US/Eastern')
Out[462]:
0 2012-12-31 19:00:00-05:00
1 2013-01-01 19:00:00-05:00
2 2013-01-02 19:00:00-05:00
dtype: datetime64[ns, US/Eastern]
```

时区信息还可以用 `astype` 操控。这种方法可以本地化并转换无时区时间戳或转换有时区时间戳。

```
本地化, 并把无时区转换为有时区
In [463]: s_naive.astype('datetime64[ns, US/Eastern]')
Out[463]:
0 2012-12-31 19:00:00-05:00
1 2013-01-01 19:00:00-05:00
2 2013-01-02 19:00:00-05:00
dtype: datetime64[ns, US/Eastern]

把有时区变为无时区
In [464]: s_aware.astype('datetime64[ns]')
Out[464]:
0 2013-01-01 05:00:00
1 2013-01-02 05:00:00
2 2013-01-03 05:00:00
dtype: datetime64[ns]

转换为新的时区
In [465]: s_aware.astype('datetime64[ns, CET]')
Out[465]:
0 2013-01-01 06:00:00+01:00
1 2013-01-02 06:00:00+01:00
2 2013-01-03 06:00:00+01:00
dtype: datetime64[ns, CET]
```

::: tip 注意

在 `Series` 上应用 `Series.to_numpy()`, 返回数据的 NumPy 数组。虽然 NumPy 可以输出本地时区! 但其实它当前并不支持时区, 因此, 有时区时间戳数据返回的是时间戳对象数组:

```
In [466]: s_naive.to_numpy()
Out[466]:
array(['2013-01-01T00:00:00.000000000', '2013-01-02T00:00:00.000000000',
 '2013-01-03T00:00:00.000000000'], dtype='datetime64[ns]')

In [467]: s_aware.to_numpy()
Out[467]:
array([Timestamp('2013-01-01 00:00:00-0500', tz='US/Eastern', freq='D'),
 Timestamp('2013-01-02 00:00:00-0500', tz='US/Eastern', freq='D'),
 Timestamp('2013-01-03 00:00:00-0500', tz='US/Eastern', freq='D')],
 dtype=object)
```

通过转换时间戳数组, 保留时区信息。例如, 转换回 `Series` 时:

```
In [468]: pd.Series(s_aware.to_numpy())
Out[468]:
0 2013-01-01 00:00:00-05:00
1 2013-01-02 00:00:00-05:00
2 2013-01-03 00:00:00-05:00
dtype: datetime64[ns, US/Eastern]
```

如果需要 NumPy `datetime64[ns]` 数组 (带已转为 UTC 的值) 而不是对象数组, 可以指定 `dtype` 参数:



```
In [469]: s_aware.to_numpy(dtype='datetime64[ns]')
Out[469]:
array(['2013-01-01T05:00:00.000000000', '2013-01-02T05:00:00.000000000',
 '2013-01-03T05:00:00.000000000'], dtype='datetime64[ns]')
```

...

---

## 时间差

`Timedelta`，时间差，即时间之间的差异，用 日、时、分、秒 等时间单位表示，时间单位可为正，也可为负。

`Timedelta` 是 `datetime.timedelta` 的子类，两者的操作方式相似，但 `Timedelta` 兼容 `np.timedelta64` 等数据类型，还支持自定义表示形式、能解析多种类型的数据，并支持自有属性。

### 解析数据，生成时间差

---

`Timedelta()` 支持用多种参数生成时间差：

```

In [1]: import datetime

字符串
In [2]: pd.Timedelta('1 days')
Out[2]: Timedelta('1 days 00:00:00')

In [3]: pd.Timedelta('1 days 00:00:00')
Out[3]: Timedelta('1 days 00:00:00')

In [4]: pd.Timedelta('1 days 2 hours')
Out[4]: Timedelta('1 days 02:00:00')

In [5]: pd.Timedelta('-1 days 2 min 3us')
Out[5]: Timedelta('-2 days +23:57:59.999997')

datetime.timedelta
注意：必须指定关键字参数
In [6]: pd.Timedelta(days=1, seconds=1)
Out[6]: Timedelta('1 days 00:00:01')

用整数与时间单位生成时间差
In [7]: pd.Timedelta(1, unit='d')
Out[7]: Timedelta('1 days 00:00:00')

datetime.timedelta 与 np.timedelta64
In [8]: pd.Timedelta(datetime.timedelta(days=1, seconds=1))
Out[8]: Timedelta('1 days 00:00:01')

In [9]: pd.Timedelta(np.timedelta64(1, 'ms'))
Out[9]: Timedelta('0 days 00:00:00.001000')

用字符串表示负数时间差
更接近 datetime.timedelta
In [10]: pd.Timedelta('-1us')
Out[10]: Timedelta('-1 days +23:59:59.999999')

时间差缺失值
In [11]: pd.Timedelta('nan')
Out[11]: NaT

In [12]: pd.Timedelta('nat')
Out[12]: NaT

ISO8601 时间格式字符串
In [13]: pd.Timedelta('P0DT0H1M0S')
Out[13]: Timedelta('0 days 00:01:00')

In [14]: pd.Timedelta('P0DT0H0M0.000000123S')
Out[14]: Timedelta('0 days 00:00:00.000000')

```

0.23.0 版新增：增加了用 **ISO8601 时间格式** 生成时间差。

**DateOffsets**（Day、Hour、Minute、Second、Milli、Micro、Nano）也可以用来生成时间差。

```
In [15]: pd.Timedelta(pd.offsets.Second(2))
Out[15]: Timedelta('0 days 00:00:02')
```

标量运算生成的也是 `Timedelta` 标量。

```
In [16]: pd.Timedelta(pd.offsets.Day(2)) + pd.Timedelta(pd.offsets.Second(2)) + \
....: pd.Timedelta('00:00:00.000123')
....:
Out[16]: Timedelta('2 days 00:00:02.000123')
```

## to\_timedelta

`pd.to_timedelta()` 可以把符合时间差格式的标量、数组、列表、序列等数据转换为 `Timedelta`。输入数据是序列，输出的就是序列，输入数据是标量，输出的就是标量，其它形式的输入数据则输出 `TimedeltaIndex`。

`to_timedelta()` 可以解析单个字符串：

```
In [17]: pd.to_timedelta('1 days 06:05:01.00003')
Out[17]: Timedelta('1 days 06:05:01.000030')

In [18]: pd.to_timedelta('15.5us')
Out[18]: Timedelta('0 days 00:00:00.000015')
```

还能解析字符串列表或数组：

```
In [19]: pd.to_timedelta(['1 days 06:05:01.00003', '15.5us', 'nan'])
Out[19]: TimedeltaIndex(['1 days 06:05:01.000030', '0 days 00:00:00.000015', NaT],
 dtype='timedelta64[ns]', freq=None)
```

`unit` 关键字参数指定时间差的单位：

```
In [20]: pd.to_timedelta(np.arange(5), unit='s')
Out[20]: TimedeltaIndex(['00:00:00', '00:00:01', '00:00:02', '00:00:03', '00:00:04'],
 dtype='timedelta64[ns]', freq=None)

In [21]: pd.to_timedelta(np.arange(5), unit='d')
Out[21]: TimedeltaIndex(['0 days', '1 days', '2 days', '3 days', '4 days'], dtype='timedelta64[ns]',
 freq=None)
```

## 时间差界限

Pandas 时间差的纳秒解析度是 64 位整数，这就决定了 `Timedelta` 的上下限。

```
In [22]: pd.Timedelta.min
Out[22]: Timedelta('-106752 days +00:12:43.145224')

In [23]: pd.Timedelta.max
Out[23]: Timedelta('106751 days 23:47:16.854775')
```

## 运算

---

以时间差为数据的 `Series` 与 `DataFrame` 支持各种运算，`datetime64 [ns]` 序列或 `Timestamps` 减法运算生成的是 `timedelta64 [ns]` 序列。

```
In [24]: s = pd.Series(pd.date_range('2012-1-1', periods=3, freq='D'))
```

```
In [25]: td = pd.Series([pd.Timedelta(days=i) for i in range(3)])
```

```
In [26]: df = pd.DataFrame({'A': s, 'B': td})
```

```
In [27]: df
```

```
Out[27]:
```

	A	B
0	2012-01-01	0 days
1	2012-01-02	1 days
2	2012-01-03	2 days

```
In [28]: df['C'] = df['A'] + df['B']
```

```
In [29]: df
```

```
Out[29]:
```

	A	B	C
0	2012-01-01	0 days	2012-01-01
1	2012-01-02	1 days	2012-01-03
2	2012-01-03	2 days	2012-01-05

```
In [30]: df.dtypes
```

```
Out[30]:
```

A	datetime64[ns]
B	timedelta64[ns]
C	datetime64[ns]

dtype: object

```
In [31]: s - s.max()
```

```
Out[31]:
```

0	-2 days
1	-1 days
2	0 days

dtype: timedelta64[ns]

```
In [32]: s - datetime.datetime(2011, 1, 1, 3, 5)
```

```
Out[32]:
```

0	364 days 20:55:00
1	365 days 20:55:00
2	366 days 20:55:00

dtype: timedelta64[ns]

```
In [33]: s + datetime.timedelta(minutes=5)
```

```
Out[33]:
```

0	2012-01-01 00:05:00
1	2012-01-02 00:05:00
2	2012-01-03 00:05:00

dtype: datetime64[ns]

```
In [34]: s + pd.offsets.Minute(5)
```

```
Out[34]:
```

0	2012-01-01 00:05:00
1	2012-01-02 00:05:00
2	2012-01-03 00:05:00

```
dtype: datetime64[ns]

In [35]: s + pd.offsets.Minute(5) + pd.offsets.Milli(5)
Out[35]:
0 2012-01-01 00:05:00.005
1 2012-01-02 00:05:00.005
2 2012-01-03 00:05:00.005
dtype: datetime64[ns]
```

`timedelta64 [ns]` 序列的标量运算：

```
In [36]: y = s - s[0]

In [37]: y
Out[37]:
0 0 days
1 1 days
2 2 days
dtype: timedelta64[ns]
```

时间差序列支持 `NaT` 值：

```
In [38]: y = s - s.shift()

In [39]: y
Out[39]:
0 NaT
1 1 days
2 1 days
dtype: timedelta64[ns]
```

与 `datetime` 类似，`np.nan` 把时间差设置为 `NaT`：

```
In [40]: y[1] = np.nan

In [41]: y
Out[41]:
0 NaT
1 NaT
2 1 days
dtype: timedelta64[ns]
```

运算符也可以显示为逆序（序列与单个对象的运算）：

```
In [42]: s.max() - s
Out[42]:
0 2 days
1 1 days
2 0 days
dtype: timedelta64[ns]

In [43]: datetime.datetime(2011, 1, 1, 3, 5) - s
Out[43]:
0 -365 days +03:05:00
1 -366 days +03:05:00
2 -367 days +03:05:00
dtype: timedelta64[ns]

In [44]: datetime.datetime.timedelta(minutes=5) + s
Out[44]:
0 2012-01-01 00:05:00
1 2012-01-02 00:05:00
2 2012-01-03 00:05:00
dtype: datetime64[ns]
```

DataFrame 支持 min、max 及 idxmin、idxmax 运算：

```

In [45]: A = s - pd.Timestamp('20120101') - pd.Timedelta('00:05:05')

In [46]: B = s - pd.Series(pd.date_range('2012-1-2', periods=3, freq='D'))

In [47]: df = pd.DataFrame({'A': A, 'B': B})

In [48]: df
Out[48]:
 A B
0 -1 days +23:54:55 -1 days
1 0 days 23:54:55 -1 days
2 1 days 23:54:55 -1 days

In [49]: df.min()
Out[49]:
A -1 days +23:54:55
B -1 days +00:00:00
dtype: timedelta64[ns]

In [50]: df.min(axis=1)
Out[50]:
0 -1 days
1 -1 days
2 -1 days
dtype: timedelta64[ns]

In [51]: df.idxmin()
Out[51]:
A 0
B 0
dtype: int64

In [52]: df.idxmax()
Out[52]:
A 2
B 0
dtype: int64

```

`Series` 也支持 `min`、`max` 及 `idxmin`、`idxmax` 运算。标量计算结果为 `Timedelta`。

```

In [53]: df.min().max()
Out[53]: Timedelta('-1 days +23:54:55')

In [54]: df.min(axis=1).min()
Out[54]: Timedelta('-1 days +00:00:00')

In [55]: df.min().idxmax()
Out[55]: 'A'

In [56]: df.min(axis=1).idxmin()
Out[56]: 0

```

时间差支持 `fillna` 函数，参数是 `Timedelta`，用于指定填充值。



```
In [57]: y.fillna(pd.Timedelta(0))
Out[57]:
0 0 days
1 0 days
2 1 days
dtype: timedelta64[ns]

In [58]: y.fillna(pd.Timedelta(10, unit='s'))
Out[58]:
0 0 days 00:00:10
1 0 days 00:00:10
2 1 days 00:00:00
dtype: timedelta64[ns]

In [59]: y.fillna(pd.Timedelta('-1 days, 00:00:05'))
Out[59]:
0 -1 days +00:00:05
1 -1 days +00:00:05
2 1 days 00:00:00
dtype: timedelta64[ns]
```

`Timedelta` 还支持取反、乘法及绝对值（`Abs`）运算：

```
In [60]: td1 = pd.Timedelta('-1 days 2 hours 3 seconds')

In [61]: td1
Out[61]: Timedelta('-2 days +21:59:57')

In [62]: -1 * td1
Out[62]: Timedelta('1 days 02:00:03')

In [63]: - td1
Out[63]: Timedelta('1 days 02:00:03')

In [64]: abs(td1)
Out[64]: Timedelta('1 days 02:00:03')
```

## 归约

`timedelta64 [ns]` 数值归约运算返回的是 `Timedelta` 对象。一般情况下，`NaT` 不计数。

```

In [65]: y2 = pd.Series(pd.to_timedelta(['-1 days +00:00:05', 'nat',
....: '-1 days +00:00:05', '1 days']))
....:

In [66]: y2
Out[66]:
0 -1 days +00:00:05
1 NaT
2 -1 days +00:00:05
3 1 days 00:00:00
dtype: timedelta64[ns]

In [67]: y2.mean()
Out[67]: Timedelta('-1 days +16:00:03.333333')

In [68]: y2.median()
Out[68]: Timedelta('-1 days +00:00:05')

In [69]: y2.quantile(.1)
Out[69]: Timedelta('-1 days +00:00:05')

In [70]: y2.sum()
Out[70]: Timedelta('-1 days +00:00:10')

```

## 频率转换

时间差除法把 `Timedelta` 序列、`TimedeltaIndex`、`Timedelta` 标量转换为其它“频率”，`astype` 也可以将之转换为指定的时间差。这些运算生成的是序列，并把 `NaT` 转换为 `nan`。注意，NumPy 标量除法是真除法，`astype` 则等同于取底整除（Floor Division）。

::: tip 说明

Floor Division，即两数的商为向下取整，如， $9 / 2 = 4$ 。又译作地板除或向下取整除，本文译作**取底整除**；

扩展知识：

Ceiling Division，即两数的商为向上取整，如， $9 / 2 = 5$ 。又译作屋顶除或向上取整除，本文译作**取顶整除**。

:::

```

In [71]: december = pd.Series(pd.date_range('20121201', periods=4))

In [72]: january = pd.Series(pd.date_range('20130101', periods=4))

In [73]: td = january - december

In [74]: td[2] += datetime.timedelta(minutes=5, seconds=3)

In [75]: td[3] = np.nan

In [76]: td
Out[76]:
0 31 days 00:00:00
1 31 days 00:00:00
2 31 days 00:05:03
3 NaT
dtype: timedelta64[ns]

转为日
In [77]: td / np.timedelta64(1, 'D')
Out[77]:
0 31.000000
1 31.000000
2 31.003507
3 NaN
dtype: float64

In [78]: td.astype('timedelta64[D]')
Out[78]:
0 31.0
1 31.0
2 31.0
3 NaN
dtype: float64

转为秒
In [79]: td / np.timedelta64(1, 's')
Out[79]:
0 2678400.0
1 2678400.0
2 2678703.0
3 NaN
dtype: float64

In [80]: td.astype('timedelta64[s]')
Out[80]:
0 2678400.0
1 2678400.0
2 2678703.0
3 NaN
dtype: float64

转为月 (此处用常量表示)
In [81]: td / np.timedelta64(1, 'M')
Out[81]:

```

```
0 1.018501
1 1.018501
2 1.018617
3 NaN
dtype: float64
```

`timedelta64 [ns]` 序列与整数或整数序列相乘或相除，生成的也是 `timedelta64 [ns]` 序列。

```
In [82]: td * -1
Out[82]:
0 -31 days +00:00:00
1 -31 days +00:00:00
2 -32 days +23:54:57
3 NaT
dtype: timedelta64[ns]

In [83]: td * pd.Series([1, 2, 3, 4])
Out[83]:
0 31 days 00:00:00
1 62 days 00:00:00
2 93 days 00:15:09
3 NaT
dtype: timedelta64[ns]
```

`timedelta64 [ns]` 序列与 `Timedelta` 标量相除的结果为取底整除的整数序列。

```
In [84]: td // pd.Timedelta(days=3, hours=4)
Out[84]:
0 9.0
1 9.0
2 9.0
3 NaN
dtype: float64

In [85]: pd.Timedelta(days=3, hours=4) // td
Out[85]:
0 0.0
1 0.0
2 0.0
3 NaN
dtype: float64
```

`Timedelta` 的求余（`mod(%)`）与除余（`divmod`）运算，支持时间差与数值参数。

```
In [86]: pd.Timedelta(hours=37) % datetime.timedelta(hours=2)
Out[86]: Timedelta('0 days 01:00:00')

除余运算的参数为时间差时，返回一对值 (int, Timedelta)
In [87]: divmod(datetime.timedelta(hours=2), pd.Timedelta(minutes=11))
Out[87]: (10, Timedelta('0 days 00:10:00'))

除余运算的参数为数值时，也返回一对值 (Timedelta, Timedelta)
In [88]: divmod(pd.Timedelta(hours=25), 864000000000000)
Out[88]: (Timedelta('0 days 00:00:00.000000'), Timedelta('0 days 01:00:00'))
```

## 属性

`Timedelta` 或 `TimedeltaIndex` 的组件可以直接访问 `days`、`seconds`、`microseconds`、`nanoseconds` 等属性。这些属性与 `datetime.timedelta` 的返回值相同，例如，`.seconds` 属性表示大于等于 0 天且小于 1 天的秒数。带符号的 `Timedelta` 返回的值也带符号。

`Series` 的 `.dt` 属性也可以直接访问这些数据。

⋮ tip 注意

这些属性**不是** `Timedelta` 显示的值。`.components` 可以提取显示的值。

⋮

对于 `Series`：

```
In [89]: td.dt.days
Out[89]:
0 31.0
1 31.0
2 31.0
3 NaN
dtype: float64

In [90]: td.dt.seconds
Out[90]:
0 0.0
1 0.0
2 303.0
3 NaN
dtype: float64
```

直接访问 `Timedelta` 标量字段值。

```
In [91]: tds = pd.Timedelta('31 days 5 min 3 sec')
```

```
In [92]: tds.days
```

```
Out[92]: 31
```

```
In [93]: tds.seconds
```

```
Out[93]: 303
```

```
In [94]: (-tds).seconds
```

```
Out[94]: 86097
```

`.components` 属性可以快速访问时间差的组件，返回结果是 `DataFrame`。下列代码输出 `Timedelta` 的显示值。

```
In [95]: td.dt.components
```

```
Out[95]:
```

	days	hours	minutes	seconds	milliseconds	microseconds	nanoseconds
0	31.0	0.0	0.0	0.0	0.0	0.0	0.0
1	31.0	0.0	0.0	0.0	0.0	0.0	0.0
2	31.0	0.0	5.0	3.0	0.0	0.0	0.0
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
In [96]: td.dt.components.seconds
```

```
Out[96]:
```

0	0.0
1	0.0
2	3.0
3	NaN

```
Name: seconds, dtype: float64
```

`.isoformat` 方法可以把 `Timedelta` 转换为 [ISO8601 时间格式](#) 字符串。

0.20.0 版新增。

```
In [97]: pd.Timedelta(days=6, minutes=50, seconds=3,
.....: milliseconds=10, microseconds=10,
.....: nanoseconds=12).isoformat()
.....:
```

```
Out[97]: 'P6DT0H50M3.010010012S'
```

## TimedeltaIndex

`TimedeltaIndex` 或 `timedelta_range()` 可以生成时间差索引。

`TimedeltaIndex` 支持字符串型的 `Timedelta`、`timedelta` 或 `np.timedelta64` 对象。

`np.nan`、`pd.NaT`、`nat` 代表缺失值。

```
In [98]: pd.TimedeltaIndex(['1 days', '1 days, 00:00:05', np.timedelta64(2, 'D'),
....: datetime.timedelta(days=2, seconds=2)])
....:
Out[98]:
TimedeltaIndex(['1 days 00:00:00', '1 days 00:00:05', '2 days 00:00:00',
 '2 days 00:00:02'],
 dtype='timedelta64[ns]', freq=None)
```

`freq` 关键字参数为 `infer` 时，`TimedeltaIndex` 可以自行推断时间频率：

```
In [99]: pd.TimedeltaIndex(['0 days', '10 days', '20 days'], freq='infer')
Out[99]: TimedeltaIndex(['0 days', '10 days', '20 days'], dtype='timedelta64[ns]', freq='10D')
```

## 生成时间差范围

与 `date_range()` 相似，`timedelta_range()` 可以生成定频 `TimedeltaIndex`，`timedelta_range` 的默认频率是日历日：

```
In [100]: pd.timedelta_range(start='1 days', periods=5)
Out[100]: TimedeltaIndex(['1 days', '2 days', '3 days', '4 days', '5 days'], dtype='timedelta64[ns]',
 freq='D')
```

`timedelta_range` 支持 `start`、`end`、`periods` 三个参数：

```
In [101]: pd.timedelta_range(start='1 days', end='5 days')
Out[101]: TimedeltaIndex(['1 days', '2 days', '3 days', '4 days', '5 days'], dtype='timedelta64[ns]',
 freq='D')

In [102]: pd.timedelta_range(end='10 days', periods=4)
Out[102]: TimedeltaIndex(['7 days', '8 days', '9 days', '10 days'], dtype='timedelta64[ns]', freq='D')
```

`freq` 参数支持各种频率别名：

```

In [103]: pd.timedelta_range(start='1 days', end='2 days', freq='30T')
Out[103]:
TimedeltaIndex(['1 days 00:00:00', '1 days 00:30:00', '1 days 01:00:00',
 '1 days 01:30:00', '1 days 02:00:00', '1 days 02:30:00',
 '1 days 03:00:00', '1 days 03:30:00', '1 days 04:00:00',
 '1 days 04:30:00', '1 days 05:00:00', '1 days 05:30:00',
 '1 days 06:00:00', '1 days 06:30:00', '1 days 07:00:00',
 '1 days 07:30:00', '1 days 08:00:00', '1 days 08:30:00',
 '1 days 09:00:00', '1 days 09:30:00', '1 days 10:00:00',
 '1 days 10:30:00', '1 days 11:00:00', '1 days 11:30:00',
 '1 days 12:00:00', '1 days 12:30:00', '1 days 13:00:00',
 '1 days 13:30:00', '1 days 14:00:00', '1 days 14:30:00',
 '1 days 15:00:00', '1 days 15:30:00', '1 days 16:00:00',
 '1 days 16:30:00', '1 days 17:00:00', '1 days 17:30:00',
 '1 days 18:00:00', '1 days 18:30:00', '1 days 19:00:00',
 '1 days 19:30:00', '1 days 20:00:00', '1 days 20:30:00',
 '1 days 21:00:00', '1 days 21:30:00', '1 days 22:00:00',
 '1 days 22:30:00', '1 days 23:00:00', '1 days 23:30:00',
 '2 days 00:00:00'],
 dtype='timedelta64[ns]', freq='30T')

In [104]: pd.timedelta_range(start='1 days', periods=5, freq='2D5H')
Out[104]:
TimedeltaIndex(['1 days 00:00:00', '3 days 05:00:00', '5 days 10:00:00',
 '7 days 15:00:00', '9 days 20:00:00'],
 dtype='timedelta64[ns]', freq='53H')

```

0.23.0 版新增。

用 `start`、`end`、`period` 可以生成等宽时间差范围，其中，`start` 与 `end`（含）是起止两端的时间，`periods` 为 `TimedeltaIndex` 里的元素数量：

```

In [105]: pd.timedelta_range('0 days', '4 days', periods=5)
Out[105]: TimedeltaIndex(['0 days', '1 days', '2 days', '3 days', '4 days'], dtype='timedelta64[ns]',
 freq=None)

In [106]: pd.timedelta_range('0 days', '4 days', periods=10)
Out[106]:
TimedeltaIndex(['0 days 00:00:00', '0 days 10:40:00', '0 days 21:20:00',
 '1 days 08:00:00', '1 days 18:40:00', '2 days 05:20:00',
 '2 days 16:00:00', '3 days 02:40:00', '3 days 13:20:00',
 '4 days 00:00:00'],
 dtype='timedelta64[ns]', freq=None)

```

## TimedeltaIndex 应用

与 `DatetimeIndex`、`PeriodIndex` 等 `datetime` 型索引类似，`TimedeltaIndex` 也可当作 pandas 对象的索引。



```

In [107]: s = pd.Series(np.arange(100),
.....: index=pd.timedelta_range('1 days', periods=100, freq='h'))
.....:

In [108]: s
Out[108]:
1 days 00:00:00 0
1 days 01:00:00 1
1 days 02:00:00 2
1 days 03:00:00 3
1 days 04:00:00 4
..
4 days 23:00:00 95
5 days 00:00:00 96
5 days 01:00:00 97
5 days 02:00:00 98
5 days 03:00:00 99
Freq: H, Length: 100, dtype: int64

```

选择操作也差不多，可以强制转换字符串与切片：

```

In [109]: s['1 day':'2 day']
Out[109]:
1 days 00:00:00 0
1 days 01:00:00 1
1 days 02:00:00 2
1 days 03:00:00 3
1 days 04:00:00 4
..
2 days 19:00:00 43
2 days 20:00:00 44
2 days 21:00:00 45
2 days 22:00:00 46
2 days 23:00:00 47
Freq: H, Length: 48, dtype: int64

In [110]: s['1 day 01:00:00']
Out[110]: 1

In [111]: s[pd.Timedelta('1 day 1h')]
Out[111]: 1

```

`TimedeltaIndex` 还支持局部字符串选择，并且可以推断选择范围：

```
In [112]: s['1 day':'1 day 5 hours']
Out[112]:
1 days 00:00:00 0
1 days 01:00:00 1
1 days 02:00:00 2
1 days 03:00:00 3
1 days 04:00:00 4
1 days 05:00:00 5
Freq: H, dtype: int64
```

## TimedeltaIndex 运算

TimedeltaIndex 与 DatetimeIndex 运算可以保留 NaT 值：

```
In [113]: tdi = pd.TimedeltaIndex(['1 days', pd.NaT, '2 days'])

In [114]: tdi.to_list()
Out[114]: [Timedelta('1 days 00:00:00'), NaT, Timedelta('2 days 00:00:00')]

In [115]: dti = pd.date_range('20130101', periods=3)

In [116]: dti.to_list()
Out[116]:
[Timestamp('2013-01-01 00:00:00', freq='D'),
 Timestamp('2013-01-02 00:00:00', freq='D'),
 Timestamp('2013-01-03 00:00:00', freq='D')]

In [117]: (dti + tdi).to_list()
Out[117]: [Timestamp('2013-01-02 00:00:00'), NaT, Timestamp('2013-01-05 00:00:00')]

In [118]: (dti - tdi).to_list()
Out[118]: [Timestamp('2012-12-31 00:00:00'), NaT, Timestamp('2013-01-01 00:00:00')]
```

## 转换

与 Series 频率转换类似，可以把 TimedeltaIndex 转换为其它索引。

```
In [119]: tdi / np.timedelta64(1, 's')
Out[119]: Float64Index([86400.0, nan, 172800.0], dtype='float64')

In [120]: tdi.astype('timedelta64[s]')
Out[120]: Float64Index([86400.0, nan, 172800.0], dtype='float64')
```

与标量操作类似，会返回**不同类型**的索引。

```

时间差与日期相加，结果为日期型索引 (DatetimeIndex)
In [121]: tdi + pd.Timestamp('20130101')
Out[121]: DatetimeIndex(['2013-01-02', 'NaT', '2013-01-03'], dtype='datetime64[ns]', freq=None)

日期与时间戳相减，结果为日期型数据 (Timestamp)
note that trying to subtract a date from a Timedelta will raise an exception
In [122]: (pd.Timestamp('20130101') - tdi).to_list()
Out[122]: [Timestamp('2012-12-31 00:00:00'), NaT, Timestamp('2012-12-30 00:00:00')]

时间差与时间差相加，结果还是时间差索引
In [123]: tdi + pd.Timedelta('10 days')
Out[123]: TimedeltaIndex(['11 days', NaT, '12 days'], dtype='timedelta64[ns]', freq=None)

除数是整数，则结果为时间差索引
In [124]: tdi / 2
Out[124]: TimedeltaIndex(['0 days 12:00:00', NaT, '1 days 00:00:00'], dtype='timedelta64[ns]', freq=None)

除数是时间差，则结果为 Float64Index
In [125]: tdi / tdi[0]
Out[125]: Float64Index([1.0, nan, 2.0], dtype='float64')

```

## 重采样

与[时间序列重采样](#)一样，`TimedeltaIndex` 也支持重采样。

```

In [126]: s.resample('D').mean()
Out[126]:
1 days 11.5
2 days 35.5
3 days 59.5
4 days 83.5
5 days 97.5
Freq: D, dtype: float64

```

## Styling

New in version 0.17.1

Provisional: This is a new feature and still under development. We’ ll be adding features and possibly making breaking changes in future releases. We’ d love to hear your feedback.

This document is written as a Jupyter Notebook, and can be viewed or downloaded [here](#).

You can apply **conditional formatting**, the visual styling of a DataFrame depending on the data within, by using the `DataFrame.style` property. This is a property that returns a `Styler` object, which has useful methods for formatting and displaying DataFrames.

The styling is accomplished using CSS. You write “style functions” that take scalars, `DataFrame`s or `Series`, and return like-indexed `DataFrame`s or `Series` with CSS “attribute: value” pairs for the values. These functions can be incrementally passed to the `Styler` which collects the styles before rendering.

## Building styles

---

Pass your style functions into one of the following methods:

- `Styler.applymap`: elementwise
- `Styler.apply`: column-/row-/table-wise

Both of those methods take a function (and some other keyword arguments) and applies your function to the `DataFrame` in a certain way. `Styler.applymap` works through the `DataFrame` elementwise. `Styler.apply` passes each column or row into your `DataFrame` one-at-a-time or the entire table at once, depending on the `axis` keyword argument. For columnwise use `axis=0`, rowwise use `axis=1`, and for the entire table at once use `axis=None`.

For `Styler.applymap` your function should take a scalar and return a single string with the CSS attribute-value pair.

For `Styler.apply` your function should take a `Series` or `DataFrame` (depending on the `axis` parameter), and return a `Series` or `DataFrame` with an identical shape where each value is a string with a CSS attribute-value pair.

Let’s see some examples.

style02

Here’s a boring example of rendering a `DataFrame`, without any (visible) styles:

style03

Note: The `DataFrame.style` attribute is a property that returns a `Styler` object. `Styler` has a `_repr_html_` method defined on it so they are rendered automatically. If you want the actual HTML back for further processing or for writing to file call the `.render()` method which returns a string.

The above output looks very similar to the standard `DataFrame` HTML representation. But we’ve done some work behind the scenes to attach CSS classes to each cell. We can view these by calling the `.render` method.

```
df.style.highlight_null().render().split('\n')[:10]
```

```
[
'<style type="text/css" >',
" #T_acfc12d6_a988_11e9_a75e_31802e421a9brow0_col2 {",
" background-color: red;",
' }</style><table id="T_acfc12d6_a988_11e9_a75e_31802e421a9b" ><thead> <tr> <th
class="blank level0" ></th> <th class="col_heading level0 col0" >A</th> <th
class="col_heading level0 col1" >B</th> <th class="col_heading level0 col2" >C</
th> <th class="col_heading level0 col3" >D</th> <th class="col_heading level0
col4" >E</th> </tr></thead><tbody>',
" <tr>",
' <th id="T_acfc12d6_a988_11e9_a75e_31802e421a9blevel0_row0" class="row_heading
level0 row0" >0</th>',
' <td id="T_acfc12d6_a988_11e9_a75e_31802e421a9brow0_col0" class="data row0
col0" >1</td>',
' <td id="T_acfc12d6_a988_11e9_a75e_31802e421a9brow0_col1" class="data row0
col1" >1.32921</td>',
' <td id="T_acfc12d6_a988_11e9_a75e_31802e421a9brow0_col2" class="data row0
col2" >nan</td>',
' <td id="T_acfc12d6_a988_11e9_a75e_31802e421a9brow0_col3" class="data row0
col3" >-0.31628</td>',
];
```

The `row0_col2` is the identifier for that particular cell. We've also prepended each row/column identifier with a UUID unique to each DataFrame so that the style from one doesn't collide with the styling from another within the same notebook or page (you can set the `uuid` if you'd like to tie together the styling of two DataFrames).

When writing style functions, you take care of producing the CSS attribute / value pairs you want. Pandas matches those up with the CSS classes that identify each cell.

Let's write a simple style function that will color negative numbers red and positive numbers black.

style04

In this case, the cell's style depends only on it's own value. That means we should use the `Styler.applymap` method which works elementwise.

style05

Notice the similarity with the standard `df.applymap`, which operates on DataFrames elementwise. We want you to be able to reuse your existing knowledge of how to interact with DataFrames.

Notice also that our function returned a string containing the CSS attribute and value, separated by a colon just like in a `````` tag. This will be a common theme.

Finally, the input shapes matched. `Styler.applymap` calls the function on each scalar input, and the function returns a scalar output.

Now suppose you wanted to highlight the maximum value in each column. We can't use `.applymap` anymore since that operated elementwise. Instead, we'll turn to `.apply` which operates columnwise

(or rowwise using the `axis` keyword). Later on we'll see that something like `highlight_max` is already defined on `Styler` so you wouldn't need to write this yourself.

style06

In this case the input is a `Series`, one column at a time. Notice that the output shape of `highlight_max` matches the input shape, an array with `len(s)` items.

We encourage you to use method chains to build up a style piecewise, before finally rendering at the end of the chain.

style07

Above we used `Styler.apply` to pass in each column one at a time.

Debugging Tip: If you're having trouble writing your style function, try just passing it into `DataFrame.apply`. Internally, `Styler.apply` uses `DataFrame.apply` so the result should be the same.

What if you wanted to highlight just the maximum value in the entire table? Use `.apply(function, axis=None)` to indicate that your function wants the entire table, not one column or row at a time. Let's try that next.

We'll rewrite our `highlight_max` to handle either `Series` (from `.apply(axis=0 or 1)`) or `DataFrames` (from `.apply(axis=None)`). We'll also allow the color to be adjustable, to demonstrate that `.apply`, and `.applymap` pass along keyword arguments.

style08

When using `Styler.apply(func, axis=None)`, the function must return a `DataFrame` with the same index and column labels.

style09

## Building Styles Summary

Style functions should return strings with one or more CSS `attribute: value` delimited by semicolons. Use

- `Styler.applymap(func)` for elementwise styles
- `Styler.apply(func, axis=0)` for columnwise styles
- `Styler.apply(func, axis=1)` for rowwise styles
- `Styler.apply(func, axis=None)` for tablewise styles

And crucially the input and output shapes of `func` must match. If `x` is the input then `func(x).shape == x.shape`.

## Finer control: slicing

---

Both `Styler.apply`, and `Styler.applymap` accept a `subset` keyword. This allows you to apply styles to specific rows or columns, without having to code that logic into your `style` function.

The value passed to `subset` behaves similar to slicing a `DataFrame`.

- A scalar is treated as a column label
- A list (or series or numpy array)
- A tuple is treated as `(row_indexer, column_indexer)`

Consider using `pd.IndexSlice` to construct the tuple for the last one.

style10

For row and column slicing, any valid indexer to `.loc` will work.

style11

Only label-based slicing is supported right now, not positional.

If your style function uses a `subset` or `axis` keyword argument, consider wrapping your function in a `functools.partial`, partialing out that keyword.

```
my_func2 = functools.partial(my_func, subset=42)
```

## Finer Control: Display Values

---

We distinguish the display value from the actual value in `Styler`. To control the display value, the text is printed in each cell, use `Styler.format`. Cells can be formatted according to a [format spec string](#) or a callable that takes a single value and returns a string.

style12

Use a dictionary to format specific columns.

style13

Or pass in a callable (or dictionary of callables) for more flexible handling.

style14

## Builtin styles

---

Finally, we expect certain styling functions to be common enough that we've included a few "built-in" to the `Styler`, so you don't have to write them yourself.

style15

You can create “heatmaps” with the `background_gradient` method. These require matplotlib, and we’ ll use [Seaborn](#) to get a nice colormap.

```
import seaborn as sns

cm = sns.light_palette("green", as_cmap=True)

s = df.style.background_gradient(cmap=cm)
s

/opt/conda/envs/pandas/lib/python3.7/site-packages/matplotlib/colors.py:479: RuntimeWarning: invalid
value encountered in less
 xa[xa < 0] = -1
```

style16

`Styler.background_gradient` takes the keyword arguments `low` and `high`. Roughly speaking these extend the range of your data by `low` and `high` percent so that when we convert the colors, the colormap’ s entire range isn’ t used. This is useful so that you can actually read the text still.

style17

There’ s also `.highlight_min` and `.highlight_max`.

style18

Use `Styler.set_properties` when the style doesn’ t actually depend on the values.

style19

## Bar charts

You can include “bar charts” in your DataFrame.

style20

New in version 0.20.0 is the ability to customize further the bar chart: You can now have the `df.style.bar` be centered on zero or midpoint value (in addition to the already existing way of having the min value at the left side of the cell), and you can pass a list of `[color_negative, color_positive]`.

Here’ s how you can change the above with the new `align='mid'` option:

style21

The following example aims to give a highlight of the behavior of the new align options:



```

import pandas as pd
from IPython.display import HTML

Test series
test1 = pd.Series([-100,-60,-30,-20], name='All Negative')
test2 = pd.Series([10,20,50,100], name='All Positive')
test3 = pd.Series([-10,-5,0,90], name='Both Pos and Neg')

head = """
<table>
 <thead>
 <th>Align</th>
 <th>All Negative</th>
 <th>All Positive</th>
 <th>Both Neg and Pos</th>
 </thead>
 </tbody>

"""

aligns = ['left','zero','mid']
for align in aligns:
 row = "<tr><th>{}</th>".format(align)
 for serie in [test1,test2,test3]:
 s = serie.copy()
 s.name=''
 row += "<td>{}</td>".format(s.to_frame().style.bar(align=align,
 color=['#d65f5f', '#5fba7d'],
 width=100).render()) #testn['width']

 row += '</tr>'
 head += row

head+= """
</tbody>
</table>"""

HTML(head)

```

style22

## Sharing styles

Say you have a lovely style built up for a DataFrame, and now you want to apply the same style to a second DataFrame. Export the style with `df1.style.export`, and import it on the second DataFrame with `df1.style.set`

style23

Notice that you're able share the styles even though they're data aware. The styles are re-evaluated on the new DataFrame they've been used upon.

## Other Options

---

You’ve seen a few methods for data-driven styling. `Styler` also provides a few other options for styles that don’t depend on the data.

- precision
- captions
- table-wide styles
- hiding the index or columns

Each of these can be specified in two ways:

- A keyword argument to `Styler.__init__`
- A call to one of the `.set_` or `.hide_` methods, e.g. `.set_caption` or `.hide_columns`

The best method to use depends on the context. Use the `Styler` constructor when building many styled DataFrames that should all share the same properties. For interactive use, the `.set_` and `.hide_` methods are more convenient.

### Precision

You can control the precision of floats using pandas’ regular `display.precision` option.

style24

Or through a `set_precision` method.

style25

Setting the precision only affects the printed number; the full-precision values are always passed to your style functions. You can always use `df.round(2).style` if you’d prefer to round from the start.

### Captions

Regular table captions can be added in a few ways.

style26

### Table styles

The next option you have are “table styles”. These are styles that apply to the table as a whole, but don’t look at the data. Certain sytlings, including pseudo-selectors like `:hover` can only be used this way.

style27

`table_styles` should be a list of dictionaries. Each dictionary should have the `selector` and `props` keys. The value for `selector` should be a valid CSS selector. Recall that all the styles are already attached to an `id`, unique to each `Styler`. This selector is in addition to that `id`. The value for `props` should be a list of tuples of `('attribute', 'value')`.

`table_styles` are extremely flexible, but not as fun to type out by hand. We hope to collect some useful ones either in pandas, or preferable in a new package that [builds on top](#) the tools here.

## Hiding the Index or Columns

The index can be hidden from rendering by calling `Styler.hide_index`. Columns can be hidden from rendering by calling `Styler.hide_columns` and passing in the name of a column, or a slice of columns.

style28

## CSS classes

Certain CSS classes are attached to cells.

- Index and Column names include `index_name` and `level` where `k` is its level in a MultiIndex
- Index label cells include `row_heading` `row` where `n` is the numeric position of the row `level` where `k` is the level in a MultiIndex
- `row_heading`
- `row` where `n` is the numeric position of the row
- `level` where `k` is the level in a MultiIndex
- Column label cells include `col_heading` `col` where `n` is the numeric position of the column `level` where `k` is the level in a MultiIndex
- `col_heading`
- `col` where `n` is the numeric position of the column
- `level` where `k` is the level in a MultiIndex
- Blank cells include `blank`
- Data cells include `data`

## Limitations

- DataFrame only (use `Series.to_frame().style`)
- The index and columns must be unique
- No large repr, and performance isn't great; this is intended for summary DataFrames
- You can only style the values, not the index or columns
- You can only apply styles, you can't insert new HTML entities

Some of these will be addressed in the future.

## Terms

- Style function: a function that's passed into `Styler.apply` or `Styler.applymap` and returns values like `'css attribute: value'`
- Builtin style functions: style functions that are methods on `Styler`
- table style: a dictionary with the two keys `selector` and `props`. `selector` is the CSS selector that `props` will apply to. `props` is a list of `(attribute, value)` tuples. A list of table styles passed into `Styler`.

## Fun stuff

---

Here are a few interesting examples.

`Styler` interacts pretty well with widgets. If you're viewing this online instead of running the notebook yourself, you're missing out on interactively adjusting the color palette.

style29

style30

## Export to Excel

---

New in version 0.20.0

Experimental: This is a new feature and still under development. We'll be adding features and possibly making breaking changes in future releases. We'd love to hear your feedback.

Some support is available for exporting styled `DataFrames` to Excel worksheets using the `OpenPyXL` or `XlsxWriter` engines. CSS2.2 properties handled include:

- `background-color`
- `border-style`, `border-width`, `border-color` and their `{ top, right, bottom, left }` variants
- `color`
- `font-family`
- `font-style`
- `font-weight`
- `text-align`
- `text-decoration`
- `vertical-align`
- `white-space: nowrap`
- Only CSS2 named colors and hex colors of the form `#rgb` or `#rrggbb` are currently supported.
- The following pseudo CSS properties are also available to set excel specific style properties:
  - `number-format`
- `number-format`

```
df.style.\
 applymap(color_negative_red).\
 apply(highlight_max).\
 to_excel('styled.xlsx', engine='openpyxl')
```

A screenshot of the output:

excel

## Extensibility

The core of pandas is, and will remain, its “high-performance, easy-to-use data structures” . With that in mind, we hope that `DataFrame.style` accomplishes two goals

- Provide an API that is pleasing to use interactively and is “good enough” for many tasks
- Provide the foundations for dedicated libraries to build on

If you build a great library on top of this, let us know and we’ ll [link](#) to it.

## Subclassing

If the default template doesn’ t quite suit your needs, you can subclass `Styler` and extend or override the template. We’ ll show an example of extending the default template to insert a custom header before each table.

```
from jinja2 import Environment, ChoiceLoader, FileSystemLoader
from IPython.display import HTML
from pandas.io.formats.style import Styler
```

We’ ll use the following template:

```
with open("templates/myhtml.tpl") as f:
 print(f.read())
```

Now that we’ ve created a template, we need to set up a subclass of `Styler` that knows about it.

```
class MyStyler(Styler):
 env = Environment(
 loader=ChoiceLoader([
 FileSystemLoader("templates"), # contains ours
 Styler.loader, # the default
])
)
 template = env.get_template("myhtml.tpl")
```

Notice that we include the original loader in our environment’ s loader. That’ s because we extend the original template, so the Jinja environment needs to be able to find it.

Now we can use that custom styler. It's `__init__` takes a `DataFrame`.

style31

Our custom template accepts a `table_title` keyword. We can provide the value in the `.render` method.

style32

For convenience, we provide the `Styler.from_custom_template` method that does the same as the custom subclass.

style33

Here's the template structure:

style34

See the template in the [GitHub repo](#) for more details.

---

## Options and settings

### Overview

---

pandas has an options system that lets you customize some aspects of its behaviour, display-related options being those the user is most likely to adjust.

Options have a full “dotted-style”, case-insensitive name (e.g. `display.max_rows`). You can get/set options directly as attributes of the top-level `options` attribute:

```
In [1]: import pandas as pd

In [2]: pd.options.display.max_rows
Out[2]: 15

In [3]: pd.options.display.max_rows = 999

In [4]: pd.options.display.max_rows
Out[4]: 999
```

The API is composed of 5 relevant functions, available directly from the `pandas` namespace:

- `get_option()` / `set_option()` - get/set the value of a single option.
- `reset_option()` - reset one or more options to their default value.
- `describe_option()` - print the descriptions of one or more options.
- `option_context()` - execute a codeblock with a set of options that revert to prior settings after execution.

**Note:** Developers can check out [pandas/core/config.py](#) for more information.

All of the functions above accept a regexp pattern ( `re.search` style) as an argument, and so passing in a substring will work - as long as it is unambiguous:

```
In [5]: pd.get_option("display.max_rows")
Out[5]: 999

In [6]: pd.set_option("display.max_rows", 101)

In [7]: pd.get_option("display.max_rows")
Out[7]: 101

In [8]: pd.set_option("max_r", 102)

In [9]: pd.get_option("display.max_rows")
Out[9]: 102
```

The following will **not work** because it matches multiple option names, e.g. `display.max_colwidth`, `display.max_rows`, `display.max_columns`:

```
In [10]: try:
....: pd.get_option("column")
....: except KeyError as e:
....: print(e)
....:
'Pattern matched multiple keys'
```

**Note:** Using this form of shorthand may cause your code to break if new options with similar names are added in future versions.

You can get a list of available options and their descriptions with `describe_option`. When called with no argument `describe_option` will print out the descriptions for all available options.

## Getting and setting options

As described above, `get_option()` and `set_option()` are available from the pandas namespace. To change an option, call `set_option('option regex', new_value)`.

```
In [11]: pd.get_option('mode.sim_interactive')
Out[11]: False

In [12]: pd.set_option('mode.sim_interactive', True)

In [13]: pd.get_option('mode.sim_interactive')
Out[13]: True
```

**Note:** The option `'mode.sim_interactive'` is mostly used for debugging purposes.

All options also have a default value, and you can use `reset_option` to do just that:

```
In [14]: pd.get_option("display.max_rows")
Out[14]: 60

In [15]: pd.set_option("display.max_rows", 999)

In [16]: pd.get_option("display.max_rows")
Out[16]: 999

In [17]: pd.reset_option("display.max_rows")

In [18]: pd.get_option("display.max_rows")
Out[18]: 60
```

It's also possible to reset multiple options at once (using a regex):

```
In [19]: pd.reset_option("^display")
```

`option_context` context manager has been exposed through the top-level API, allowing you to execute code with given option values. Option values are restored automatically when you exit the `with` block:

```
In [20]: with pd.option_context("display.max_rows", 10, "display.max_columns", 5):
....: print(pd.get_option("display.max_rows"))
....: print(pd.get_option("display.max_columns"))
....:
10
5

In [21]: print(pd.get_option("display.max_rows"))
60

In [22]: print(pd.get_option("display.max_columns"))
0
```

## Setting startup options in Python/IPython environment

Using startup scripts for the Python/IPython environment to import pandas and set options makes working with pandas more efficient. To do this, create a `.py` or `.ipy` script in the startup directory of the desired profile. An example where the startup folder is in a default ipython profile can be found at:

```
$IPYTHONDIR/profile_default/startup
```

More information can be found in the [ipython documentation](#). An example startup script for pandas is displayed below:



```
import pandas as pd
pd.set_option('display.max_rows', 999)
pd.set_option('precision', 5)
```

## Frequently Used Options

---

The following is a walk-through of the more frequently used display options.

`display.max_rows` and `display.max_columns` sets the maximum number of rows and columns displayed when a frame is pretty-printed. Truncated lines are replaced by an ellipsis.

```
In [23]: df = pd.DataFrame(np.random.randn(7, 2))
```

```
In [24]: pd.set_option('max_rows', 7)
```

```
In [25]: df
```

```
Out[25]:
```

```
 0 1
0 0.469112 -0.282863
1 -1.509059 -1.135632
2 1.212112 -0.173215
3 0.119209 -1.044236
4 -0.861849 -2.104569
5 -0.494929 1.071804
6 0.721555 -0.706771
```

```
In [26]: pd.set_option('max_rows', 5)
```

```
In [27]: df
```

```
Out[27]:
```

```
 0 1
0 0.469112 -0.282863
1 -1.509059 -1.135632
..
5 -0.494929 1.071804
6 0.721555 -0.706771
```

```
[7 rows x 2 columns]
```

```
In [28]: pd.reset_option('max_rows')
```

Once the `display.max_rows` is exceeded, the `display.min_rows` options determines how many rows are shown in the truncated repr.

```

In [29]: pd.set_option('max_rows', 8)

In [30]: pd.set_option('max_rows', 4)

below max_rows -> all rows shown
In [31]: df = pd.DataFrame(np.random.randn(7, 2))

In [32]: df
Out[32]:
 0 1
0 -1.039575 0.271860
1 -0.424972 0.567020
..
5 0.404705 0.577046
6 -1.715002 -1.039268

[7 rows x 2 columns]

above max_rows -> only min_rows (4) rows shown
In [33]: df = pd.DataFrame(np.random.randn(9, 2))

In [34]: df
Out[34]:
 0 1
0 -0.370647 -1.157892
1 -1.344312 0.844885
..
7 0.276662 -0.472035
8 -0.013960 -0.362543

[9 rows x 2 columns]

In [35]: pd.reset_option('max_rows')

In [36]: pd.reset_option('min_rows')

```

`display.expand_frame_repr` allows for the representation of dataframes to stretch across pages, wrapped over the full column vs row-wise.

```
In [37]: df = pd.DataFrame(np.random.randn(5, 10))
```

```
In [38]: pd.set_option('expand_frame_repr', True)
```

```
In [39]: df
```

```
Out[39]:
```

	0	1	2	3	4	5	6	7	8	9
0	-0.006154	-0.923061	0.895717	0.805244	-1.206412	2.565646	1.431256	1.340309	-1.170299	-0.226169
1	0.410835	0.813850	0.132003	-0.827317	-0.076467	-1.187678	1.130127	-1.436737	-1.413681	1.607920
2	1.024180	0.569605	0.875906	-2.211372	0.974466	-2.006747	-0.410001	-0.078638	0.545952	-1.219217
3	-1.226825	0.769804	-1.281247	-0.727707	-0.121306	-0.097883	0.695775	0.341734	0.959726	-1.110336
4	-0.619976	0.149748	-0.732339	0.687738	0.176444	0.403310	-0.154951	0.301624	-2.179861	-1.369849

```
In [40]: pd.set_option('expand_frame_repr', False)
```

```
In [41]: df
```

```
Out[41]:
```

	0	1	2	3	4	5	6	7	8	9
0	-0.006154	-0.923061	0.895717	0.805244	-1.206412	2.565646	1.431256	1.340309	-1.170299	-0.226169
1	0.410835	0.813850	0.132003	-0.827317	-0.076467	-1.187678	1.130127	-1.436737	-1.413681	1.607920
2	1.024180	0.569605	0.875906	-2.211372	0.974466	-2.006747	-0.410001	-0.078638	0.545952	-1.219217
3	-1.226825	0.769804	-1.281247	-0.727707	-0.121306	-0.097883	0.695775	0.341734	0.959726	-1.110336
4	-0.619976	0.149748	-0.732339	0.687738	0.176444	0.403310	-0.154951	0.301624	-2.179861	-1.369849

```
In [42]: pd.reset_option('expand_frame_repr')
```

`display.large_repr` lets you select whether to display dataframes that exceed `max_columns` or `max_rows` as a truncated frame, or as a summary.

```

In [43]: df = pd.DataFrame(np.random.randn(10, 10))

In [44]: pd.set_option('max_rows', 5)

In [45]: pd.set_option('large_repr', 'truncate')

In [46]: df
Out[46]:

```

	0	1	2	3	4	5	6	7	8	9
0	-0.954208	1.462696	-1.743161	-0.826591	-0.345352	1.314232	0.690579	0.995761	2.396780	0.014871
1	3.357427	-0.317441	-1.236269	0.896171	-0.487602	-0.082240	-2.182937	0.380396	0.084844	0.432390
..	...	...	...	...	...	...	...	...	...	...
8	-0.303421	-0.858447	0.306996	-0.028665	0.384316	1.574159	1.588931	0.476720	0.473424	-0.242861
9	-0.014805	-0.284319	0.650776	-1.461665	-1.137707	-0.891060	-0.693921	1.613616	0.464000	0.227371

```

[10 rows x 10 columns]

In [47]: pd.set_option('large_repr', 'info')

In [48]: df
Out[48]:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 10 columns):
0 10 non-null float64
1 10 non-null float64
2 10 non-null float64
3 10 non-null float64
4 10 non-null float64
5 10 non-null float64
6 10 non-null float64
7 10 non-null float64
8 10 non-null float64
9 10 non-null float64
dtypes: float64(10)
memory usage: 928.0 bytes

In [49]: pd.reset_option('large_repr')

In [50]: pd.reset_option('max_rows')

```

`display.max_colwidth` sets the maximum width of columns. Cells of this length or longer will be truncated with an ellipsis.

```
In [51]: df = pd.DataFrame(np.array([['foo', 'bar', 'bim', 'uncomfortably long string'],
....: ['horse', 'cow', 'banana', 'apple']]))
....:
```

```
In [52]: pd.set_option('max_colwidth', 40)
```

```
In [53]: df
```

```
Out[53]:
```

	0	1	2	3
0	foo	bar	bim	uncomfortably long string
1	horse	cow	banana	apple

```
In [54]: pd.set_option('max_colwidth', 6)
```

```
In [55]: df
```

```
Out[55]:
```

	0	1	2	3
0	foo	bar	bim	un...
1	horse	cow	ba...	apple

```
In [56]: pd.reset_option('max_colwidth')
```

`display.max_info_columns` sets a threshold for when by-column info will be given.

```
In [57]: df = pd.DataFrame(np.random.randn(10, 10))
```

```
In [58]: pd.set_option('max_info_columns', 11)
```

```
In [59]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10 entries, 0 to 9
```

```
Data columns (total 10 columns):
```

```
0 10 non-null float64
```

```
1 10 non-null float64
```

```
2 10 non-null float64
```

```
3 10 non-null float64
```

```
4 10 non-null float64
```

```
5 10 non-null float64
```

```
6 10 non-null float64
```

```
7 10 non-null float64
```

```
8 10 non-null float64
```

```
9 10 non-null float64
```

```
dtypes: float64(10)
```

```
memory usage: 928.0 bytes
```

```
In [60]: pd.set_option('max_info_columns', 5)
```

```
In [61]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10 entries, 0 to 9
```

```
Columns: 10 entries, 0 to 9
```

```
dtypes: float64(10)
```

```
memory usage: 928.0 bytes
```

```
In [62]: pd.reset_option('max_info_columns')
```

`display.max_info_rows: df.info()` will usually show null-counts for each column. For large frames this can be quite slow. `max_info_rows` and `max_info_cols` limit this null check only to frames with smaller dimensions than specified. Note that you can specify the option `df.info(null_counts=True)` to override on showing a particular frame.

```
In [63]: df = pd.DataFrame(np.random.choice([0, 1, np.nan], size=(10, 10)))
```

```
In [64]: df
```

```
Out[64]:
```

	0	1	2	3	4	5	6	7	8	9
0	0.0	NaN	1.0	NaN	NaN	0.0	NaN	0.0	NaN	1.0
1	1.0	NaN	1.0	1.0	1.0	1.0	NaN	0.0	0.0	NaN
2	0.0	NaN	1.0	0.0	0.0	NaN	NaN	NaN	NaN	0.0
3	NaN	NaN	NaN	0.0	1.0	1.0	NaN	1.0	NaN	1.0
4	0.0	NaN	NaN	NaN	0.0	NaN	NaN	NaN	1.0	0.0
5	0.0	1.0	1.0	1.0	1.0	0.0	NaN	NaN	1.0	0.0
6	1.0	1.0	1.0	NaN	1.0	NaN	1.0	0.0	NaN	NaN
7	0.0	0.0	1.0	0.0	1.0	0.0	1.0	1.0	0.0	NaN
8	NaN	NaN	NaN	0.0	NaN	NaN	NaN	NaN	1.0	NaN
9	0.0	NaN	0.0	NaN	NaN	0.0	NaN	1.0	1.0	0.0

```
In [65]: pd.set_option('max_info_rows', 11)
```

```
In [66]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10 entries, 0 to 9
```

```
Data columns (total 10 columns):
```

```
0 8 non-null float64
```

```
1 3 non-null float64
```

```
2 7 non-null float64
```

```
3 6 non-null float64
```

```
4 7 non-null float64
```

```
5 6 non-null float64
```

```
6 2 non-null float64
```

```
7 6 non-null float64
```

```
8 6 non-null float64
```

```
9 6 non-null float64
```

```
dtypes: float64(10)
```

```
memory usage: 928.0 bytes
```

```
In [67]: pd.set_option('max_info_rows', 5)
```

```
In [68]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10 entries, 0 to 9
```

```
Data columns (total 10 columns):
```

```
0 float64
```

```
1 float64
```

```
2 float64
```

```
3 float64
```

```
4 float64
```

```
5 float64
```

```
6 float64
```

```
7 float64
```

```
8 float64
```

```
9 float64
```

```
dtypes: float64(10)
```

```
memory usage: 928.0 bytes
```

```
In [69]: pd.reset_option('max_info_rows')
```

`display.precision` sets the output display precision in terms of decimal places. This is only a suggestion.

```
In [70]: df = pd.DataFrame(np.random.randn(5, 5))

In [71]: pd.set_option('precision', 7)

In [72]: df
Out[72]:
```

	0	1	2	3	4
0	-1.1506406	-0.7983341	-0.5576966	0.3813531	1.3371217
1	-1.5310949	1.3314582	-0.5713290	-0.0266708	-1.0856630
2	-1.1147378	-0.0582158	-0.4867681	1.6851483	0.1125723
3	-1.4953086	0.8984347	-0.1482168	-1.5960698	0.1596530
4	0.2621358	0.0362196	0.1847350	-0.2550694	-0.2710197

```
In [73]: pd.set_option('precision', 4)

In [74]: df
Out[74]:
```

	0	1	2	3	4
0	-1.1506	-0.7983	-0.5577	0.3814	1.3371
1	-1.5311	1.3315	-0.5713	-0.0267	-1.0857
2	-1.1147	-0.0582	-0.4868	1.6851	0.1126
3	-1.4953	0.8984	-0.1482	-1.5961	0.1597
4	0.2621	0.0362	0.1847	-0.2551	-0.2710

`display.chop_threshold` sets at what level pandas rounds to zero when it displays a Series of DataFrame. This setting does not change the precision at which the number is stored.



```
In [75]: df = pd.DataFrame(np.random.randn(6, 6))
```

```
In [76]: pd.set_option('chop_threshold', 0)
```

```
In [77]: df
```

```
Out[77]:
```

	0	1	2	3	4	5
0	1.2884	0.2946	-1.1658	0.8470	-0.6856	0.6091
1	-0.3040	0.6256	-0.0593	0.2497	1.1039	-1.0875
2	1.9980	-0.2445	0.1362	0.8863	-1.3507	-0.8863
3	-1.0133	1.9209	-0.3882	-2.3144	0.6655	0.4026
4	0.3996	-1.7660	0.8504	0.3881	0.9923	0.7441
5	-0.7398	-1.0549	-0.1796	0.6396	1.5850	1.9067

```
In [78]: pd.set_option('chop_threshold', .5)
```

```
In [79]: df
```

```
Out[79]:
```

	0	1	2	3	4	5
0	1.2884	0.0000	-1.1658	0.8470	-0.6856	0.6091
1	0.0000	0.6256	0.0000	0.0000	1.1039	-1.0875
2	1.9980	0.0000	0.0000	0.8863	-1.3507	-0.8863
3	-1.0133	1.9209	0.0000	-2.3144	0.6655	0.0000
4	0.0000	-1.7660	0.8504	0.0000	0.9923	0.7441
5	-0.7398	-1.0549	0.0000	0.6396	1.5850	1.9067

```
In [80]: pd.reset_option('chop_threshold')
```

`display.colheader_justify` controls the justification of the headers. The options are 'right', and 'left'.

```
In [81]: df = pd.DataFrame(np.array([np.random.randn(6),
.....: np.random.randint(1, 9, 6) * .1,
.....: np.zeros(6)]).T,
.....: columns=['A', 'B', 'C'], dtype='float')
```

```
In [82]: pd.set_option('colheader_justify', 'right')
```

```
In [83]: df
```

```
Out[83]:
```

	A	B	C
0	0.1040	0.1	0.0
1	0.1741	0.5	0.0
2	-0.4395	0.4	0.0
3	-0.7413	0.8	0.0
4	-0.0797	0.4	0.0
5	-0.9229	0.3	0.0

```
In [84]: pd.set_option('colheader_justify', 'left')
```

```
In [85]: df
```

```
Out[85]:
```

	A	B	C
0	0.1040	0.1	0.0
1	0.1741	0.5	0.0
2	-0.4395	0.4	0.0
3	-0.7413	0.8	0.0
4	-0.0797	0.4	0.0
5	-0.9229	0.3	0.0

```
In [86]: pd.reset_option('colheader_justify')
```

# Available options

---

Option	Default	Function
display.chop_threshold	None	If set to a float value, all float values smaller then the given threshold will be displayed as exactly 0 by repr and friends.
display.colheader_justify	right	Controls the justification of column headers. used by DataFrameFormatter.
display.column_space	12	No description available.
display.date_dayfirst	False	When True, prints and parses dates with the day first, eg 20/01/2005
display.date_yearfirst	False	When True, prints and parses dates with the year first, eg 2005/01/20
display.encoding	UTF-8	Defaults to the detected encoding of the console. Specifies the encoding to be used for strings returned by to_string, these are generally strings meant to be displayed on the console.
display.expand_frame_repr	True	Whether to print out the full DataFrame repr for wide DataFrames across multiple lines, max_columns is still respected, but the output will wrap-around across multiple “pages” if its width exceeds display.width.
display.float_format	None	The callable should accept a floating point number and return a string with the desired format of the number. This is used in some places like SeriesFormatter. See core.format.EngFormatter for an example.
display.large_repr	truncate	For DataFrames exceeding max_rows/max_cols, the repr (and HTML repr) can show a truncated table (the default), or switch to the view from df.info() (the behaviour in earlier versions of pandas). allowable settings, [ ‘truncate’ , ‘info’ ]
display.latex.repr	False	Whether to produce a latex DataFrame representation for jupyter frontends that support it.
display.latex.escape	True	Escapes special characters in DataFrames, when using the to_latex method.
display.latex.longtable	False	Specifies if the to_latex method of a DataFrame uses the longtable format.
display.latex.multicolumn	True	Combines columns when using a MultiIndex
display.latex.multicolumn_format	‘l’	Alignment of multicolumn labels
display.latex.multirow	False	Combines rows when using a MultiIndex. Centered instead of top-aligned, separated by clines.
display.max_columns	0 or 20	

Option	Default	Function
		max_rows and max_columns are used in <b>repr()</b> methods to decide if to_string() or info() is used to render an object to a string. In case Python/IPython is running in a terminal this is set to 0 by default and pandas will correctly auto-detect the width of the terminal and switch to a smaller format in case all columns would not fit vertically. The IPython notebook, IPython qtconsole, or IDLE do not run in a terminal and hence it is not possible to do correct auto-detection, in which case the default is set to 20. 'None' value means unlimited.
display.max_colwidth	50	The maximum width in characters of a column in the repr of a pandas data structure. When the column overflows, a “...” placeholder is embedded in the output.
display.max_info_columns	100	max_info_columns is used in DataFrame.info method to decide if per column information will be printed.
display.max_info_rows	1690785	df.info() will usually show null-counts for each column. For large frames this can be quite slow. max_info_rows and max_info_cols limit this null check only to frames with smaller dimensions then specified.
display.max_rows	60	This sets the maximum number of rows pandas should output when printing out various output. For example, this value determines whether the repr() for a dataframe prints out fully or just a truncated or summary repr. 'None' value means unlimited.
display.min_rows	10	The numbers of rows to show in a truncated repr (when max_rows is exceeded). Ignored when max_rows is set to None or 0. When set to None, follows the value of max_rows.
display.max_seq_items	100	when pretty-printing a long sequence, no more then max_seq_items will be printed. If items are omitted, they will be denoted by the addition of “...” to the resulting string. If set to None, the number of items to be printed is unlimited.
display.memory_usage	True	This specifies if the memory usage of a DataFrame should be displayed when the df.info() method is invoked.
display.multi_sparse	True	“Sparsify” MultiIndex display (don't display repeated elements in outer levels within groups)
display.notebook_repr_html	True	

Option	Default	Function
		When True, IPython notebook will use html representation for pandas objects (if it is available).
display.pprint_nest_depth	3	Controls the number of nested levels to process when pretty-printing
display.precision	6	Floating point output precision in terms of number of places after the decimal, for regular formatting as well as scientific notation. Similar to numpy's precision print option
display.show_dimensions	truncate	Whether to print out dimensions at the end of DataFrame repr. If 'truncate' is specified, only print out the dimensions if the frame is truncated (e.g. not display all rows and/or columns)
display.width	80	Width of the display in characters. In case python/ IPython is running in a terminal this can be set to None and pandas will correctly auto-detect the width. Note that the IPython notebook, IPython qtconsole, or IDLE do not run in a terminal and hence it is not possible to correctly detect the width.
display.html.table_schema	False	Whether to publish a Table Schema representation for frontends that support it.
display.html.border	1	A border=value attribute is inserted in the <code>&lt;table&gt;</code> tag for the DataFrame HTML repr.
display.html.use_mathjax	True	When True, Jupyter notebook will process table contents using MathJax, rendering mathematical expressions enclosed by the dollar symbol.
io.excel.xls.writer	xlwt	The default Excel writer engine for 'xls' files.
io.excel.xlsm.writer	openpyxl	The default Excel writer engine for 'xlsm' files. Available options: 'openpyxl' (the default).
io.excel.xlsx.writer	openpyxl	The default Excel writer engine for 'xlsx' files.
io.hdf.default_format	None	default format writing format, if None, then put will default to 'fixed' and append will default to 'table'
io.hdf.dropna_table	True	drop ALL nan rows when appending to a table
io.parquet.engine	None	The engine to use as a default for parquet reading and writing. If None then try 'pyarrow' and 'fastparquet'
mode.chained_assignment	warn	Controls SettingWithCopyWarning: 'raise' , 'warn' , or None. Raise an exception, warn, or no action if trying to use <a href="#">chained assignment</a> .

Option	Default	Function
mode.sim_interactive	False	Whether to simulate interactive mode for purposes of testing.
mode.use_inf_as_na	False	True means treat None, NaN, -INF, INF as NA (old way), False means None and NaN are null, but INF, -INF are not NA (new way).
compute.use_bottleneck	True	Use the bottleneck library to accelerate computation if it is installed.
compute.use_numexpr	True	Use the numexpr library to accelerate computation if it is installed.
plotting.backend	matplotlib	Change the plotting backend to a different backend than the current matplotlib one. Backends can be implemented as third-party libraries implementing the pandas plotting API. They can use other plotting libraries like Bokeh, Altair, etc.
plotting.matplotlib.register_converters	True	Register custom converters with matplotlib. Set to False to de-register.

## Number formatting

pandas also allows you to set how numbers are displayed in the console. This option is not set through the `set_options` API.

Use the `set_eng_float_format` function to alter the floating-point formatting of pandas objects to produce a particular format.

For instance:

```

In [87]: import numpy as np

In [88]: pd.set_eng_float_format(accuracy=3, use_eng_prefix=True)

In [89]: s = pd.Series(np.random.randn(5), index=['a', 'b', 'c', 'd', 'e'])

In [90]: s / 1.e3
Out[90]:
a 303.638u
b -721.084u
c -622.696u
d 648.250u
e -1.945m
dtype: float64

In [91]: s / 1.e6
Out[91]:
a 303.638n
b -721.084n
c -622.696n
d 648.250n
e -1.945u
dtype: float64

```

To round floats on a case-by-case basis, you can also use `round()` and `round()`.

## Unicode formatting

---

::: danger Warning

Enabling this option will affect the performance for printing of DataFrame and Series (about 2 times slower). Use only when it is actually required.

:::

Some East Asian countries use Unicode characters whose width corresponds to two Latin characters. If a DataFrame or Series contains these characters, the default output mode may not align them properly.

::: tip Note

Screen captures are attached for each output to show the actual results.

:::



```
In [92]: df = pd.DataFrame({'国籍': ['UK', '日本'], '名前': ['Alice', 'しのぶ']})
```

```
In [93]: df
```

```
Out[93]:
```

	国籍	名前
0	UK	Alice
1	日本	しのぶ

#### option unicode01

Enabling `display.unicode.east_asian_width` allows pandas to check each character's "East Asian Width" property. These characters can be aligned properly by setting this option to `True`. However, this will result in longer render times than the standard `len` function.

```
In [94]: pd.set_option('display.unicode.east_asian_width', True)
```

```
In [95]: df
```

```
Out[95]:
```

	国籍	名前
0	UK	Alice
1	日本	しのぶ

#### option unicode02

In addition, Unicode characters whose width is "Ambiguous" can either be 1 or 2 characters wide depending on the terminal setting or encoding. The option `display.unicode.ambiguous_as_wide` can be used to handle the ambiguity.

By default, an "Ambiguous" character's width, such as "¡" (inverted exclamation) in the example below, is taken to be 1.

```
In [96]: df = pd.DataFrame({'a': ['xxx', '¡¡'], 'b': ['yyy', '¡¡']})
```

```
In [97]: df
```

```
Out[97]:
```

	a	b
0	xxx	yyy
1	¡¡	¡¡

#### option unicode03

Enabling `display.unicode.ambiguous_as_wide` makes pandas interpret these characters' widths to be 2. (Note that this option will only be effective when `display.unicode.east_asian_width` is enabled.)

However, setting this option incorrectly for your terminal will cause these characters to be aligned incorrectly:

```
In [98]: pd.set_option('display.unicode.ambiguous_as_wide', True)
```

```
In [99]: df
```

```
Out[99]:
```

```
 a b
0 xxx yyy
1 ii ii
```

option unicode04

## Table schema display

---

New in version 0.20.0.

`DataFrame` and `Series` will publish a Table Schema representation by default. False by default, this can be enabled globally with the `display.html.table_schema` option:

```
In [100]: pd.set_option('display.html.table_schema', True)
```

Only `'display.max_rows'` are serialized and published.

---

## Enhancing performance

In this part of the tutorial, we will investigate how to speed up certain functions operating on pandas `DataFrames` using three different techniques: Cython, Numba and `pandas.eval()`. We will see a speed improvement of ~200 when we use Cython and Numba on a test function operating row-wise on the `DataFrame`. Using `pandas.eval()` we will speed up a sum by an order of ~2.

## Cython (writing C extensions for pandas)

---

For many use cases writing pandas in pure Python and NumPy is sufficient. In some computationally heavy applications however, it can be possible to achieve sizable speed-ups by offloading work to [cython](#).

This tutorial assumes you have refactored as much as possible in Python, for example by trying to remove for-loops and making use of NumPy vectorization. It's always worth optimising in Python first.

This tutorial walks through a “typical” process of cythonizing a slow computation. We use an [example from the Cython documentation](#) but in the context of pandas. Our final cythonized solution is around 100 times faster than the pure Python solution.

## Pure Python

We have a `DataFrame` to which we want to apply a function row-wise.

```
In [1]: df = pd.DataFrame({'a': np.random.randn(1000),
...: 'b': np.random.randn(1000),
...: 'N': np.random.randint(100, 1000, (1000)),
...: 'x': 'x'})
...:
```

```
In [2]: df
```

```
Out[2]:
```

	a	b	N	x
0	0.469112	-0.218470	585	x
1	-0.282863	-0.061645	841	x
2	-1.509059	-0.723780	251	x
3	-1.135632	0.551225	972	x
4	1.212112	-0.497767	181	x
..	...	...	...	..
995	-1.512743	0.874737	374	x
996	0.933753	1.120790	246	x
997	-0.308013	0.198768	157	x
998	-0.079915	1.757555	977	x
999	-1.010589	-1.115680	770	x

```
[1000 rows x 4 columns]
```

Here's the function in pure Python:

```
In [3]: def f(x):
...: return x * (x - 1)
...:

In [4]: def integrate_f(a, b, N):
...: s = 0
...: dx = (b - a) / N
...: for i in range(N):
...: s += f(a + i * dx)
...: return s * dx
...:
```

We achieve our result by using `apply` (row-wise):

```
In [7]: %timeit df.apply(lambda x: integrate_f(x['a'], x['b'], x['N']), axis=1)
10 loops, best of 3: 174 ms per loop
```

But clearly this isn't fast enough for us. Let's take a look and see where the time is spent during this operation (limited to the most time consuming four calls) using the [prun ipython magic function](#):

```
In [5]: %prun -l 4 df.apply(lambda x: integrate_f(x['a'], x['b'], x['N']), axis=1) # noqa E999
 672332 function calls (667306 primitive calls) in 0.285 seconds
```

Ordered by: internal time

List reduced **from** 221 to 4 due to restriction <4>

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1000	0.144	0.000	0.217	0.000	<ipython-input-4-c2a74e076cf0>:1(integrate_f)
552423	0.074	0.000	0.074	0.000	<ipython-input-3-c138bdd570e3>:1(f)
3000	0.008	0.000	0.045	0.000	base.py:4695(get_value)
6001	0.005	0.000	0.012	0.000	{pandas._libs.lib.values_from_object}

By far the majority of time is spend inside either `integrate_f` or `f`, hence we' ll concentrate our efforts cythonizing these two functions.

::: tip Note

In Python 2 replacing the `range` with its generator counterpart (`xrange`) would mean the `range` line would vanish. In Python 3 `range` is already a generator.

:::

## Plain Cython

First we' re going to need to import the Cython magic function to ipython:

```
In [6]: %load_ext Cython
```

Now, let' s simply copy our functions over to Cython as is (the suffix is here to distinguish between function versions):

```
In [7]: %%cython
...: def f_plain(x):
...: return x * (x - 1)
...: def integrate_f_plain(a, b, N):
...: s = 0
...: dx = (b - a) / N
...: for i in range(N):
...: s += f_plain(a + i * dx)
...: return s * dx
...:
```

::: tip Note

If you' re having trouble pasting the above into your ipython, you may need to be using bleeding edge ipython for paste to play well with cell magics.

:::

```
In [4]: %timeit df.apply(lambda x: integrate_f_plain(x['a'], x['b'], x['N']), axis=1)
10 loops, best of 3: 85.5 ms per loop
```

Already this has shaved a third off, not too bad for a simple copy and paste.

## Adding type

We get another huge improvement simply by providing type information:

```
In [8]: %%cython
...: cdef double f_typed(double x) except? -2:
...: return x * (x - 1)
...: cpdef double integrate_f_typed(double a, double b, int N):
...: cdef int i
...: cdef double s, dx
...: s = 0
...: dx = (b - a) / N
...: for i in range(N):
...: s += f_typed(a + i * dx)
...: return s * dx
...:
```

```
In [4]: %timeit df.apply(lambda x: integrate_f_typed(x['a'], x['b'], x['N']), axis=1)
10 loops, best of 3: 20.3 ms per loop
```

Now, we're talking! It's now over ten times faster than the original python implementation, and we haven't really modified the code. Let's have another look at what's eating up time:

```
In [9]: %prun -l 4 df.apply(lambda x: integrate_f_typed(x['a'], x['b'], x['N']), axis=1)
119905 function calls (114879 primitive calls) in 0.096 seconds

Ordered by: internal time
List reduced from 216 to 4 due to restriction <4>

ncalls tottime percall cumtime percall filename:lineno(function)
 3000 0.012 0.000 0.064 0.000 base.py:4695(get_value)
 6001 0.007 0.000 0.017 0.000 {pandas._libs.lib.values_from_object}
 3000 0.007 0.000 0.073 0.000 series.py:1061(__getitem__)
 3000 0.006 0.000 0.006 0.000 {method 'get_value' of 'pandas._libs.index.IndexEngine'
objects}
```

## Using ndarray

It's calling series... a lot! It's creating a Series from each row, and get-ting from both the index and the series (three times for each row). Function calls are expensive in Python, so maybe we could minimize these by cythonizing the apply part.

::: tip Note

We are now passing ndarrays into the Cython function, fortunately Cython plays very nicely with NumPy.

...

```
In [10]: %%cython
....: cimport numpy as np
....: import numpy as np
....: cdef double f_typed(double x) except? -2:
....: return x * (x - 1)
....: cpdef double integrate_f_typed(double a, double b, int N):
....: cdef int i
....: cdef double s, dx
....: s = 0
....: dx = (b - a) / N
....: for i in range(N):
....: s += f_typed(a + i * dx)
....: return s * dx
....: cpdef np.ndarray[double] apply_integrate_f(np.ndarray col_a, np.ndarray col_b,
....: np.ndarray col_N):
....: assert (col_a.dtype == np.float
....: and col_b.dtype == np.float and col_N.dtype == np.int)
....: cdef Py_ssize_t i, n = len(col_N)
....: assert (len(col_a) == len(col_b) == n)
....: cdef np.ndarray[double] res = np.empty(n)
....: for i in range(len(col_a)):
....: res[i] = integrate_f_typed(col_a[i], col_b[i], col_N[i])
....: return res
....:
```

The implementation is simple, it creates an array of zeros and loops over the rows, applying our `integrate_f_typed`, and putting this in the zeros array.

... danger Warning

You can **not pass** a `Series` directly as a `ndarray` typed parameter to a Cython function. Instead pass the actual `ndarray` using the `Series.to_numpy()`. The reason is that the Cython definition is specific to an `ndarray` and not the passed `Series`.

So, do not do this:

```
apply_integrate_f(df['a'], df['b'], df['N'])
```

But rather, use `Series.to_numpy()` to get the underlying `ndarray`:

```
apply_integrate_f(df['a'].to_numpy(),
 df['b'].to_numpy(),
 df['N'].to_numpy())
```

...

::: tip Note

Loops like this would be extremely slow in Python, but in Cython looping over NumPy arrays is fast.

:::

```
In [4]: %timeit apply_integrate_f(df['a'].to_numpy(),
 df['b'].to_numpy(),
 df['N'].to_numpy())
1000 loops, best of 3: 1.25 ms per loop
```

We've gotten another big improvement. Let's check again where the time is spent:

```
In [11]: %prun -l 4 apply_integrate_f(df['a'].to_numpy(),
....: df['b'].to_numpy(),
....: df['N'].to_numpy())
....:
File "<ipython-input-11-613f5c6ec02d>", line 2
 df['b'].to_numpy(),
 ^
IndentationError: unexpected indent
```

As one might expect, the majority of the time is now spent in `apply_integrate_f`, so if we wanted to make anymore efficiencies we must continue to concentrate our efforts here.

## More advanced techniques

There is still hope for improvement. Here's an example of using some more advanced Cython techniques:

```

In [12]: %%cython
....: cimport cython
....: cimport numpy as np
....: import numpy as np
....: cdef double f_typed(double x) except? -2:
....: return x * (x - 1)
....: cpdef double integrate_f_typed(double a, double b, int N):
....: cdef int i
....: cdef double s, dx
....: s = 0
....: dx = (b - a) / N
....: for i in range(N):
....: s += f_typed(a + i * dx)
....: return s * dx
....: @cython.boundscheck(False)
....: @cython.wraparound(False)
....: cpdef np.ndarray[double] apply_integrate_f_wrap(np.ndarray[double] col_a,
....: np.ndarray[double] col_b,
....: np.ndarray[int] col_N):
....: cdef int i, n = len(col_N)
....: assert len(col_a) == len(col_b) == n
....: cdef np.ndarray[double] res = np.empty(n)
....: for i in range(n):
....: res[i] = integrate_f_typed(col_a[i], col_b[i], col_N[i])
....: return res
....:

```

```

In [4]: %timeit apply_integrate_f_wrap(df['a'].to_numpy(),
 df['b'].to_numpy(),
 df['N'].to_numpy())

1000 loops, best of 3: 987 us per loop

```

Even faster, with the caveat that a bug in our Cython code (an off-by-one error, for example) might cause a segfault because memory access isn't checked. For more about `boundscheck` and `wraparound`, see the Cython docs on [compiler directives](#).

## Using Numba

A recent alternative to statically compiling Cython code, is to use a dynamic jit-compiler, Numba.

Numba gives you the power to speed up your applications with high performance functions written directly in Python. With a few annotations, array-oriented and math-heavy Python code can be just-in-time compiled to native machine instructions, similar in performance to C, C++ and Fortran, without having to switch languages or Python interpreters.

Numba works by generating optimized machine code using the LLVM compiler infrastructure at import time, runtime, or statically (using the included pycc tool). Numba supports compilation of Python to run on either CPU or GPU hardware, and is designed to integrate with the Python scientific software stack.



::: tip Note

You will need to install Numba. This is easy with `conda`, by using: `conda install numba`, see [installing using miniconda](#).

:::

::: tip Note

As of Numba version 0.20, pandas objects cannot be passed directly to Numba-compiled functions. Instead, one must pass the NumPy array underlying the pandas object to the Numba-compiled function as demonstrated below.

:::

## Jit

We demonstrate how to use Numba to just-in-time compile our code. We simply take the plain Python code from above and annotate with the `@jit` decorator.

```

import numba

@numba.jit
def f_plain(x):
 return x * (x - 1)

@numba.jit
def integrate_f_numba(a, b, N):
 s = 0
 dx = (b - a) / N
 for i in range(N):
 s += f_plain(a + i * dx)
 return s * dx

@numba.jit
def apply_integrate_f_numba(col_a, col_b, col_N):
 n = len(col_N)
 result = np.empty(n, dtype='float64')
 assert len(col_a) == len(col_b) == n
 for i in range(n):
 result[i] = integrate_f_numba(col_a[i], col_b[i], col_N[i])
 return result

def compute_numba(df):
 result = apply_integrate_f_numba(df['a'].to_numpy(),
 df['b'].to_numpy(),
 df['N'].to_numpy())
 return pd.Series(result, index=df.index, name='result')

```

Note that we directly pass NumPy arrays to the Numba function. `compute_numba` is just a wrapper that provides a nicer interface by passing/returning pandas objects.

```

In [4]: %timeit compute_numba(df)
1000 loops, best of 3: 798 us per loop

```

In this example, using Numba was faster than Cython.

## Vectorize

Numba can also be used to write vectorized functions that do not require the user to explicitly loop over the observations of a vector; a vectorized function will be applied to each row automatically. Consider the following toy example of doubling each observation:

```
import numba

def double_every_value_nonumba(x):
 return x * 2

@numba.vectorize
def double_every_value_withnumba(x): # noqa E501
 return x * 2
```

```
Custom function without numba
In [5]: %timeit df['col1_doubled'] = df.a.apply(double_every_value_nonumba) # noqa E501
1000 loops, best of 3: 797 us per loop

Standard implementation (faster than a custom function)
In [6]: %timeit df['col1_doubled'] = df.a * 2
1000 loops, best of 3: 233 us per loop

Custom function with numba
In [7]: %timeit (df['col1_doubled'] = double_every_value_withnumba(df.a.to_numpy()))
1000 loops, best of 3: 145 us per loop
```

## Caveats

::: tip Note

Numba will execute on any function, but can only accelerate certain classes of functions.

:::

Numba is best at accelerating functions that apply numerical functions to NumPy arrays. When passed a function that only uses operations it knows how to accelerate, it will execute in `nopython` mode.

If Numba is passed a function that includes something it doesn't know how to work with – a category that currently includes sets, lists, dictionaries, or string functions – it will revert to `object` mode. In `object` mode, Numba will execute but your code will not speed up significantly. If you would prefer that Numba throw an error if it cannot compile a function in a way that speeds up your code, pass Numba the argument `nopython=True` (e.g. `@numba.jit(nopython=True)`). For more on troubleshooting Numba modes, see the [Numba troubleshooting page](#).

Read more in the [Numba docs](#).

## Expression evaluation via `eval()`

The top-level function `pandas.eval()` implements expression evaluation of `Series` and `DataFrame` objects.

::: tip Note

To benefit from using `eval()` you need to install `numexpr`. See the [recommended dependencies section](#) for more details.

:::

The point of using `eval()` for expression evaluation rather than plain Python is two-fold: 1) large `DataFrame` objects are evaluated more efficiently and 2) large arithmetic and boolean expressions are evaluated all at once by the underlying engine (by default `numexpr` is used for evaluation).

::: tip Note

You should not use `eval()` for simple expressions or for expressions involving small `DataFrames`. In fact, `eval()` is many orders of magnitude slower for smaller expressions/objects than plain old Python. A good rule of thumb is to only use `eval()` when you have a `DataFrame` with more than 10,000 rows.

:::

`eval()` supports all arithmetic expressions supported by the engine in addition to some extensions available only in pandas.

::: tip Note

The larger the frame and the larger the expression the more speedup you will see from using `eval()`.

:::

## Supported syntax

These operations are supported by `pandas.eval()`:

- Arithmetic operations except for the left shift (`<<`) and right shift (`>>`) operators, e.g., `df + 2 * pi / s ** 4 % 42 - the_golden_ratio`
- Comparison operations, including chained comparisons, e.g., `2 < df < df2`
- Boolean operations, e.g., `df < df2 and df3 < df4 or not df_bool`
- List and tuple literals, e.g., `[1, 2]` or `(1, 2)`
- Attribute access, e.g., `df.a`
- Subscript expressions, e.g., `df[0]`
- Simple variable evaluation, e.g., `pd.eval('df')` (this is not very useful)
- Math functions: `sin`, `cos`, `exp`, `log`, `expm1`, `log1p`, `sqrt`, `sinh`, `cosh`, `tanh`, `arcsin`, `arccos`, `arctan`, `arccosh`, `arcsinh`, `arctanh`, `abs`, `arctan2` and `log10`.

This Python syntax is **not** allowed:

- Expressions
  - Function calls other than math functions.
  - `is / is not` operations
  - `if` expressions
  - `lambda` expressions
  - `list / set / dict` comprehensions
  - Literal `dict` and `set` expressions
  - `yield` expressions
  - Generator expressions
  - Boolean expressions consisting of only scalar values
- Statements
  - Neither `simple` nor `compound` statements are allowed. This includes things like `for`, `while`, and `if`.

## `eval()` examples

`pandas.eval()` works well with expressions containing large arrays.

First let's create a few decent-sized arrays to play with:

```
In [13]: nrows, ncols = 20000, 100
```

```
In [14]: df1, df2, df3, df4 = [pd.DataFrame(np.random.randn(nrows, ncols)) for _ in range(4)]
```

Now let's compare adding them together using plain ol' Python versus `eval()`:

```
In [15]: %timeit df1 + df2 + df3 + df4
21 ms +- 787 us per loop (mean +- std. dev. of 7 runs, 10 loops each)
```

```
In [16]: %timeit pd.eval('df1 + df2 + df3 + df4')
8.12 ms +- 249 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

Now let's do the same thing but with comparisons:

```
In [17]: %timeit (df1 > 0) & (df2 > 0) & (df3 > 0) & (df4 > 0)
272 ms +- 6.92 ms per loop (mean +- std. dev. of 7 runs, 1 loop each)
```

```
In [18]: %timeit pd.eval('(df1 > 0) & (df2 > 0) & (df3 > 0) & (df4 > 0)')
19.2 ms +- 1.87 ms per loop (mean +- std. dev. of 7 runs, 10 loops each)
```

`eval()` also works with unaligned pandas objects:

```
In [19]: s = pd.Series(np.random.randn(50))
```

```
In [20]: %timeit df1 + df2 + df3 + df4 + s
103 ms +- 12.7 ms per loop (mean +- std. dev. of 7 runs, 10 loops each)
```

```
In [21]: %timeit pd.eval('df1 + df2 + df3 + df4 + s')
10.2 ms +- 215 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

⋮ tip Note

Operations such as

```
1 and 2 # would parse to 1 & 2, but should evaluate to 2
3 or 4 # would parse to 3 | 4, but should evaluate to 3
~1 # this is okay, but slower when using eval
```

should be performed in Python. An exception will be raised if you try to perform any boolean/bitwise operations with scalar operands that are not of type `bool` or `np.bool_`. Again, you should perform these kinds of operations in plain Python.

⋮

## The `DataFrame.eval` method

In addition to the top level `pandas.eval()` function you can also evaluate an expression in the “context” of a `DataFrame`.

```
In [22]: df = pd.DataFrame(np.random.randn(5, 2), columns=['a', 'b'])
```

```
In [23]: df.eval('a + b')
```

```
Out[23]:
```

```
0 -0.246747
```

```
1 0.867786
```

```
2 -1.626063
```

```
3 -1.134978
```

```
4 -1.027798
```

```
dtype: float64
```

Any expression that is a valid `pandas.eval()` expression is also a valid `DataFrame.eval()` expression, with the added benefit that you don’t have to prefix the name of the `DataFrame` to the column(s) you’re interested in evaluating.

In addition, you can perform assignment of columns within an expression. This allows for formulaic evaluation. The assignment target can be a new column name or an existing column name, and it must be a valid Python identifier.

New in version 0.18.0.

The `inplace` keyword determines whether this assignment will be performed on the original `DataFrame` or return a copy with the new column.

::: danger Warning

For backwards compatibility, `inplace` defaults to `True` if not specified. This will change in a future version of pandas - if your code depends on an inplace assignment you should update to explicitly set `inplace=True`.

:::

```
In [24]: df = pd.DataFrame(dict(a=range(5), b=range(5, 10)))
```

```
In [25]: df.eval('c = a + b', inplace=True)
```

```
In [26]: df.eval('d = a + b + c', inplace=True)
```

```
In [27]: df.eval('a = 1', inplace=True)
```

```
In [28]: df
```

```
Out[28]:
```

	a	b	c	d
0	1	5	5	10
1	1	6	7	14
2	1	7	9	18
3	1	8	11	22
4	1	9	13	26

When `inplace` is set to `False`, a copy of the `DataFrame` with the new or modified columns is returned and the original frame is unchanged.

```

In [29]: df
Out[29]:
 a b c d
0 1 5 5 10
1 1 6 7 14
2 1 7 9 18
3 1 8 11 22
4 1 9 13 26

In [30]: df.eval('e = a - c', inplace=False)
Out[30]:
 a b c d e
0 1 5 5 10 -4
1 1 6 7 14 -6
2 1 7 9 18 -8
3 1 8 11 22 -10
4 1 9 13 26 -12

In [31]: df
Out[31]:
 a b c d
0 1 5 5 10
1 1 6 7 14
2 1 7 9 18
3 1 8 11 22
4 1 9 13 26

```

New in version 0.18.0.

As a convenience, multiple assignments can be performed by using a multi-line string.

```

In [32]: df.eval("""
....: c = a + b
....: d = a + b + c
....: a = 1""", inplace=False)
....:
Out[32]:
 a b c d
0 1 5 6 12
1 1 6 7 14
2 1 7 8 16
3 1 8 9 18
4 1 9 10 20

```

The equivalent in standard Python would be



```
In [33]: df = pd.DataFrame(dict(a=range(5), b=range(5, 10)))
```

```
In [34]: df['c'] = df.a + df.b
```

```
In [35]: df['d'] = df.a + df.b + df.c
```

```
In [36]: df['a'] = 1
```

```
In [37]: df
```

```
Out[37]:
```

	a	b	c	d
0	1	5	5	10
1	1	6	7	14
2	1	7	9	18
3	1	8	11	22
4	1	9	13	26

New in version 0.18.0.

The `query` method gained the `inplace` keyword which determines whether the query modifies the original frame.

```
In [38]: df = pd.DataFrame(dict(a=range(5), b=range(5, 10)))
```

```
In [39]: df.query('a > 2')
```

```
Out[39]:
```

	a	b
3	3	8
4	4	9

```
In [40]: df.query('a > 2', inplace=True)
```

```
In [41]: df
```

```
Out[41]:
```

	a	b
3	3	8
4	4	9

::: danger Warning

Unlike with `eval`, the default value for `inplace` for `query` is `False`. This is consistent with prior versions of pandas.

:::

## Local variables

You must explicitly reference any local variable that you want to use in an expression by placing the `@` character in front of the name. For example,

```
In [42]: df = pd.DataFrame(np.random.randn(5, 2), columns=list('ab'))
```

```
In [43]: newcol = np.random.randn(len(df))
```

```
In [44]: df.eval('b + @newcol')
```

```
Out[44]:
```

```
0 -0.173926
```

```
1 2.493083
```

```
2 -0.881831
```

```
3 -0.691045
```

```
4 1.334703
```

```
dtype: float64
```

```
In [45]: df.query('b < @newcol')
```

```
Out[45]:
```

	a	b
0	0.863987	-0.115998
2	-2.621419	-1.297879

If you don't prefix the local variable with `@`, pandas will raise an exception telling you the variable is undefined.

When using `DataFrame.eval()` and `DataFrame.query()`, this allows you to have a local variable and a `DataFrame` column with the same name in an expression.

```
In [46]: a = np.random.randn()
```

```
In [47]: df.query('@a < a')
```

```
Out[47]:
```

	a	b
0	0.863987	-0.115998

```
In [48]: df.loc[a < df.a] # same as the previous expression
```

```
Out[48]:
```

	a	b
0	0.863987	-0.115998

With `pandas.eval()` you cannot use the `@` prefix at all, because it isn't defined in that context. pandas will let you know this if you try to use `@` in a top-level call to `pandas.eval()`. For example,

```

In [49]: a, b = 1, 2

In [50]: pd.eval('@a + b')
Traceback (most recent call last):

File "/opt/conda/envs/pandas/lib/python3.7/site-packages/IPython/core/interactiveshell.py", line 3325,
 in run_code
 exec(code_obj, self.user_global_ns, self.user_ns)

File "<ipython-input-50-af17947a194f>", line 1, in <module>
 pd.eval('@a + b')

File "/pandas/pandas/core/computation/eval.py", line 311, in eval
 _check_for_locals(expr, level, parser)

File "/pandas/pandas/core/computation/eval.py", line 166, in _check_for_locals
 raise SyntaxError(msg)

File "<string>", line unknown
SyntaxError: The '@' prefix is not allowed in top-level eval calls,
please refer to your variables by name without the '@' prefix

```

In this case, you should simply refer to the variables like you would in standard Python.

```

In [51]: pd.eval('a + b')
Out[51]: 3

```

## **pandas.eval() parsers**

There are two different parsers and two different engines you can use as the backend.

The default 'pandas' parser allows a more intuitive syntax for expressing query-like operations (comparisons, conjunctions and disjunctions). In particular, the precedence of the `&` and `|` operators is made equal to the precedence of the corresponding boolean operations `and` and `or`.

For example, the above conjunction can be written without parentheses. Alternatively, you can use the 'python' parser to enforce strict Python semantics.

```

In [52]: expr = '(df1 > 0) & (df2 > 0) & (df3 > 0) & (df4 > 0)'

In [53]: x = pd.eval(expr, parser='python')

In [54]: expr_no_parens = 'df1 > 0 & df2 > 0 & df3 > 0 & df4 > 0'

In [55]: y = pd.eval(expr_no_parens, parser='pandas')

In [56]: np.all(x == y)
Out[56]: True

```

The same expression can be “anded” together with the word `and` as well:

```
In [57]: expr = '(df1 > 0) & (df2 > 0) & (df3 > 0) & (df4 > 0)'

In [58]: x = pd.eval(expr, parser='python')

In [59]: expr_with_and = 'df1 > 0 and df2 > 0 and df3 > 0 and df4 > 0'

In [60]: y = pd.eval(expr_with_and, parser='pandas')

In [61]: np.all(x == y)
Out[61]: True
```

The `and` and `or` operators here have the same precedence that they would in vanilla Python.

## `pandas.eval()` backends

There's also the option to make `eval()` operate identical to plain ol' Python.

::: tip Note

Using the `'python'` engine is generally not useful, except for testing other evaluation engines against it. You will achieve **no** performance benefits using `eval()` with `engine='python'` and in fact may incur a performance hit.

:::

You can see this by using `pandas.eval()` with the `'python'` engine. It is a bit slower (not by much) than evaluating the same expression in Python

```
In [62]: %timeit df1 + df2 + df3 + df4
9.5 ms +- 241 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

```
In [63]: %timeit pd.eval('df1 + df2 + df3 + df4', engine='python')
10.8 ms +- 898 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

## `pandas.eval()` performance

`eval()` is intended to speed up certain kinds of operations. In particular, those operations involving complex expressions with large `DataFrame`/`Series` objects should see a significant performance benefit. Here is a plot showing the running time of `pandas.eval()` as function of the size of the frame involved in the computation. The two lines are two different engines.

eval-perf

::: tip Note

Operations with smallish objects (around 15k-20k rows) are faster using plain Python:

eval-perf-small

...

This plot was created using a `DataFrame` with 3 columns each containing floating point values generated using `numpy.random.randn()`.

## Technical minutia regarding expression evaluation

Expressions that would result in an object dtype or involve datetime operations (because of `NaT`) must be evaluated in Python space. The main reason for this behavior is to maintain backwards compatibility with versions of NumPy < 1.7. In those versions of NumPy a call to `ndarray.astype(str)` will truncate any strings that are more than 60 characters in length. Second, we can't pass object arrays to `numexpr` thus string comparisons must be evaluated in Python space.

The upshot is that this only applies to object-dtype expressions. So, if you have an expression—for example

```
In [64]: df = pd.DataFrame({'strings': np.repeat(list('cba'), 3),
....: 'nums': np.repeat(range(3), 3)})
....:
```

```
In [65]: df
Out[65]:
```

	strings	nums
0	c	0
1	c	0
2	c	0
3	b	1
4	b	1
5	b	1
6	a	2
7	a	2
8	a	2

```
In [66]: df.query('strings == "a" and nums == 1')
```

```
Out[66]:
Empty DataFrame
Columns: [strings, nums]
Index: []
```

the numeric part of the comparison (`nums == 1`) will be evaluated by `numexpr`.

In general, `DataFrame.query()/pandas.eval()` will evaluate the subexpressions that can be evaluated by `numexpr` and those that must be evaluated in Python space transparently to the user. This is done by inferring the result type of an expression from its arguments and operators.

---

## Sparse data structures

... tip Note

`SparseSeries` and `SparseDataFrame` have been deprecated. Their purpose is served equally well by a `Series` or `DataFrame` with sparse values. See [Migrating](#) for tips on migrating.

...

Pandas provides data structures for efficiently storing sparse data. These are not necessarily sparse in the typical “mostly 0” . Rather, you can view these objects as being “compressed” where any data matching a specific value ( `NaN` / missing value, though any value can be chosen, including 0) is omitted. The compressed values are not actually stored in the array.

```
In [1]: arr = np.random.randn(10)

In [2]: arr[2:-2] = np.nan

In [3]: ts = pd.Series(pd.SparseArray(arr))

In [4]: ts
Out[4]:
0 0.469112
1 -0.282863
2 NaN
3 NaN
4 NaN
5 NaN
6 NaN
7 NaN
8 -0.861849
9 -2.104569
dtype: Sparse[float64, nan]
```

Notice the dtype, `Sparse[float64, nan]` . The `nan` means that elements in the array that are `nan` aren’ t actually stored, only the non-`nan` elements are. Those non-`nan` elements have a `float64` dtype.

The sparse objects exist for memory efficiency reasons. Suppose you had a large, mostly NA `DataFrame` :

```

In [5]: df = pd.DataFrame(np.random.randn(10000, 4))

In [6]: df.iloc[:9998] = np.nan

In [7]: sdf = df.astype(pd.SparseDtype("float", np.nan))

In [8]: sdf.head()
Out[8]:
 0 1 2 3
0 NaN NaN NaN NaN
1 NaN NaN NaN NaN
2 NaN NaN NaN NaN
3 NaN NaN NaN NaN
4 NaN NaN NaN NaN

In [9]: sdf.dtypes
Out[9]:
0 Sparse[float64, nan]
1 Sparse[float64, nan]
2 Sparse[float64, nan]
3 Sparse[float64, nan]
dtype: object

In [10]: sdf.sparse.density
Out[10]: 0.0002

```

As you can see, the density (% of values that have not been “compressed” ) is extremely low. This sparse object takes up much less memory on disk (pickled) and in the Python interpreter.

```

In [11]: 'dense : {:0.2f} bytes'.format(df.memory_usage().sum() / 1e3)
Out[11]: 'dense : 320.13 bytes'

In [12]: 'sparse: {:0.2f} bytes'.format(sdf.memory_usage().sum() / 1e3)
Out[12]: 'sparse: 0.22 bytes'

```

Functionally, their behavior should be nearly identical to their dense counterparts.

## SparseArray

`SparseArray` is a `ExtensionArray` for storing an array of sparse values (see [dtypes](#) for more on extension arrays). It is a 1-dimensional ndarray-like object storing only values distinct from the `fill_value`:

```

In [13]: arr = np.random.randn(10)

In [14]: arr[2:5] = np.nan

In [15]: arr[7:8] = np.nan

In [16]: sparr = pd.SparseArray(arr)

In [17]: sparr
Out[17]:
[-1.9556635297215477, -1.6588664275960427, nan, nan, nan, 1.1589328886422277, 0.14529711373305043, nan,
 0.6060271905134522, 1.3342113401317768]
Fill: nan
IntIndex
Indices: array([0, 1, 5, 6, 8, 9], dtype=int32)

```

A sparse array can be converted to a regular (dense) ndarray with `numpy.asarray()`

```

In [18]: np.asarray(sparr)
Out[18]:
array([-1.9557, -1.6589, nan, nan, nan, 1.1589, 0.1453,
 nan, 0.606 , 1.3342])

```

## SparseDtype

The `SparseArray.dtype` property stores two pieces of information

1. The dtype of the non-sparse values
2. The scalar fill value

```

In [19]: sparr.dtype
Out[19]: Sparse[float64, nan]

```

A `SparseDtype` may be constructed by passing each of these

```

In [20]: pd.SparseDtype(np.dtype('datetime64[ns]'))
Out[20]: Sparse[datetime64[ns], NaT]

```

The default fill value for a given NumPy dtype is the “missing” value for that dtype, though it may be overridden.

```

In [21]: pd.SparseDtype(np.dtype('datetime64[ns]'),
....: fill_value=pd.Timestamp('2017-01-01'))
....:
Out[21]: Sparse[datetime64[ns], 2017-01-01 00:00:00]

```

Finally, the string alias `'Sparse[dtype]'` may be used to specify a sparse dtype in many places



```
In [22]: pd.array([1, 0, 0, 2], dtype='Sparse[int]')
Out[22]:
[1, 0, 0, 2]
Fill: 0
IntIndex
Indices: array([0, 3], dtype=int32)
```

## Sparse accessor

---

New in version 0.24.0.

Pandas provides a `.sparse` accessor, similar to `.str` for string data, `.cat` for categorical data, and `.dt` for datetime-like data. This namespace provides attributes and methods that are specific to sparse data.

```
In [23]: s = pd.Series([0, 0, 1, 2], dtype="Sparse[int]")

In [24]: s.sparse.density
Out[24]: 0.5

In [25]: s.sparse.fill_value
Out[25]: 0
```

This accessor is available only on data with `SparseDtype`, and on the `Series` class itself for creating a Series with sparse data from a scipy COO matrix with.

New in version 0.25.0.

A `.sparse` accessor has been added for `DataFrame` as well. See [Sparse accessor](#) for more.

## Sparse calculation

---

You can apply NumPy `ufuncs` to `SparseArray` and get a `SparseArray` as a result.

```
In [26]: arr = pd.SparseArray([1., np.nan, np.nan, -2., np.nan])

In [27]: np.abs(arr)
Out[27]:
[1.0, nan, nan, 2.0, nan]
Fill: nan
IntIndex
Indices: array([0, 3], dtype=int32)
```

The ufunc is also applied to `fill_value`. This is needed to get the correct dense result.

```
In [28]: arr = pd.SparseArray([1., -1, -1, -2., -1], fill_value=-1)

In [29]: np.abs(arr)
Out[29]:
[1.0, 1, 1, 2.0, 1]
Fill: 1
IntIndex
Indices: array([0, 3], dtype=int32)

In [30]: np.abs(arr).to_dense()
Out[30]: array([1., 1., 1., 2., 1.])
```

## Migrating

In older versions of pandas, the `SparseSeries` and `SparseDataFrame` classes (documented below) were the preferred way to work with sparse data. With the advent of extension arrays, these subclasses are no longer needed. Their purpose is better served by using a regular `Series` or `DataFrame` with sparse values instead.

::: tip Note

There's no performance or memory penalty to using a `Series` or `DataFrame` with sparse values, rather than a `SparseSeries` or `SparseDataFrame`.

:::

This section provides some guidance on migrating your code to the new style. As a reminder, you can use the python warnings module to control warnings. But we recommend modifying your code, rather than ignoring the warning.

### Construction

From an array-like, use the regular `Series` or `DataFrame` constructors with `SparseArray` values.

```
Previous way
>>> pd.SparseDataFrame({"A": [0, 1]})
```

```
New way
In [31]: pd.DataFrame({"A": pd.SparseArray([0, 1])})
Out[31]:
 A
0 0
1 1
```

From a SciPy sparse matrix, use `DataFrame.sparse.from_spmatrix()`,

```
Previous way
>>> from scipy import sparse
>>> mat = sparse.eye(3)
>>> df = pd.SparseDataFrame(mat, columns=['A', 'B', 'C'])
```

```
New way
In [32]: from scipy import sparse

In [33]: mat = sparse.eye(3)

In [34]: df = pd.DataFrame.sparse.from_spmatrix(mat, columns=['A', 'B', 'C'])

In [35]: df.dtypes
Out[35]:
A Sparse[float64, 0.0]
B Sparse[float64, 0.0]
C Sparse[float64, 0.0]
dtype: object
```

## Conversion

From sparse to dense, use the `.sparse` accessors

```
In [36]: df.sparse.to_dense()
Out[36]:
 A B C
0 1 0 0
1 0 1 0
2 0 0 1

In [37]: df.sparse.to_coo()
Out[37]:
<3x3 sparse matrix of type '<class 'numpy.float64'>'
 with 3 stored elements in COOrdinate format>
```

From dense to sparse, use `DataFrame.astype()` with a `SparseDtype`.

```
In [38]: dense = pd.DataFrame({"A": [1, 0, 0, 1]})

In [39]: dtype = pd.SparseDtype(int, fill_value=0)

In [40]: dense.astype(dtype)
Out[40]:
 A
0 1
1 0
2 0
3 1
```

## Sparse Properties

Sparse-specific properties, like `density`, are available on the `.sparse` accessor.

```
In [41]: df.sparse.density
Out[41]: 0.3333333333333333
```

## General differences

In a `SparseDataFrame`, all columns were sparse. A `DataFrame` can have a mixture of sparse and dense columns. As a consequence, assigning new columns to a `DataFrame` with sparse values will not automatically convert the input to be sparse.

```
Previous Way
>>> df = pd.SparseDataFrame({"A": [0, 1]})
>>> df['B'] = [0, 0] # implicitly becomes Sparse
>>> df['B'].dtype
Sparse[int64, nan]
```

Instead, you'll need to ensure that the values being assigned are sparse

```
In [42]: df = pd.DataFrame({"A": pd.SparseArray([0, 1])})

In [43]: df['B'] = [0, 0] # remains dense

In [44]: df['B'].dtype
Out[44]: dtype('int64')

In [45]: df['B'] = pd.SparseArray([0, 0])

In [46]: df['B'].dtype
Out[46]: Sparse[int64, 0]
```

The `SparseDataFrame.default_kind` and `SparseDataFrame.default_fill_value` attributes have no replacement.

## Interaction with `scipy.sparse`

Use `DataFrame.sparse.from_spmatrix()` to create a `DataFrame` with sparse values from a sparse matrix.

New in version 0.25.0.

```

In [47]: from scipy.sparse import csr_matrix

In [48]: arr = np.random.random(size=(1000, 5))

In [49]: arr[arr < .9] = 0

In [50]: sp_arr = csr_matrix(arr)

In [51]: sp_arr
Out[51]:
<1000x5 sparse matrix of type '<class 'numpy.float64'>'
 with 517 stored elements in Compressed Sparse Row format>

In [52]: sdf = pd.DataFrame.sparse.from_spmatrix(sp_arr)

In [53]: sdf.head()
Out[53]:
 0 1 2 3 4
0 0.956380 0.0 0.0 0.000000 0.0
1 0.000000 0.0 0.0 0.000000 0.0
2 0.000000 0.0 0.0 0.000000 0.0
3 0.000000 0.0 0.0 0.000000 0.0
4 0.999552 0.0 0.0 0.956153 0.0

In [54]: sdf.dtypes
Out[54]:
0 Sparse[float64, 0.0]
1 Sparse[float64, 0.0]
2 Sparse[float64, 0.0]
3 Sparse[float64, 0.0]
4 Sparse[float64, 0.0]
dtype: object

```

All sparse formats are supported, but matrices that are not in `COOrdinate` format will be converted, copying data as needed. To convert back to sparse SciPy matrix in COO format, you can use the `DataFrame.sparse.to_coo()` method:

```

In [55]: sdf.sparse.to_coo()
Out[55]:
<1000x5 sparse matrix of type '<class 'numpy.float64'>'
 with 517 stored elements in COOrdinate format>

```

meth:Series.sparse.to\_coo is implemented for transforming a `Series` with sparse values indexed by a `MultiIndex` to a `scipy.sparse.coo_matrix`.

The method requires a `MultiIndex` with two or more levels.

```

In [56]: s = pd.Series([3.0, np.nan, 1.0, 3.0, np.nan, np.nan])

In [57]: s.index = pd.MultiIndex.from_tuples([(1, 2, 'a', 0),
.....: (1, 2, 'a', 1),
.....: (1, 1, 'b', 0),
.....: (1, 1, 'b', 1),
.....: (2, 1, 'b', 0),
.....: (2, 1, 'b', 1)],
.....: names=['A', 'B', 'C', 'D'])

In [58]: s
Out[58]:
A B C D
1 2 a 0 3.0
 1 NaN
 1 b 0 1.0
 1 3.0
2 1 b 0 NaN
 1 NaN
dtype: float64

In [59]: ss = s.astype('Sparse')

In [60]: ss
Out[60]:
A B C D
1 2 a 0 3.0
 1 NaN
 1 b 0 1.0
 1 3.0
2 1 b 0 NaN
 1 NaN
dtype: Sparse[float64, nan]

```

In the example below, we transform the `Series` to a sparse representation of a 2-d array by specifying that the first and second `MultiIndex` levels define labels for the rows and the third and fourth levels define labels for the columns. We also specify that the column and row labels should be sorted in the final sparse representation.

```

In [61]: A, rows, columns = ss.sparse.to_coo(row_levels=['A', 'B'],
.....: column_levels=['C', 'D'],
.....: sort_labels=True)
.....:

In [62]: A
Out[62]:
<3x4 sparse matrix of type '<class 'numpy.float64'>'
 with 3 stored elements in COOrdinate format>

In [63]: A.todense()
Out[63]:
matrix([[0., 0., 1., 3.],
 [3., 0., 0., 0.],
 [0., 0., 0., 0.]])

In [64]: rows
Out[64]: [(1, 1), (1, 2), (2, 1)]

In [65]: columns
Out[65]: [('a', 0), ('a', 1), ('b', 0), ('b', 1)]

```

Specifying different row and column labels (and not sorting them) yields a different sparse matrix:

```

In [66]: A, rows, columns = ss.sparse.to_coo(row_levels=['A', 'B', 'C'],
.....: column_levels=['D'],
.....: sort_labels=False)
.....:

In [67]: A
Out[67]:
<3x2 sparse matrix of type '<class 'numpy.float64'>'
 with 3 stored elements in COOrdinate format>

In [68]: A.todense()
Out[68]:
matrix([[3., 0.],
 [1., 3.],
 [0., 0.]])

In [69]: rows
Out[69]: [(1, 2, 'a'), (1, 1, 'b'), (2, 1, 'b')]

In [70]: columns
Out[70]: [0, 1]

```

A convenience method `Series.sparse.from_coo()` is implemented for creating a `Series` with sparse values from a `scipy.sparse.coo_matrix`.

```

In [71]: from scipy import sparse

In [72]: A = sparse.coo_matrix(([3.0, 1.0, 2.0], ([1, 0, 0], [0, 2, 3])),
.....: shape=(3, 4))
.....:

In [73]: A
Out[73]:
<3x4 sparse matrix of type '<class 'numpy.float64'>'
 with 3 stored elements in COOrdinate format>

In [74]: A.todense()
Out[74]:
matrix([[0., 0., 1., 2.],
 [3., 0., 0., 0.],
 [0., 0., 0., 0.]])

```

The default behaviour (with `dense_index=False`) simply returns a `Series` containing only the non-null entries.

```

In [75]: ss = pd.Series.sparse.from_coo(A)

In [76]: ss
Out[76]:
0 2 1.0
3 2.0
1 0 3.0
dtype: Sparse[float64, nan]

```

Specifying `dense_index=True` will result in an index that is the Cartesian product of the row and columns coordinates of the matrix. Note that this will consume a significant amount of memory (relative to `dense_index=False`) if the sparse matrix is large (and sparse) enough.

```

In [77]: ss_dense = pd.Series.sparse.from_coo(A, dense_index=True)

In [78]: ss_dense
Out[78]:
0 0 NaN
1 NaN
2 1.0
3 2.0
1 0 3.0
1 NaN
2 NaN
3 NaN
2 0 NaN
1 NaN
2 NaN
3 NaN
dtype: Sparse[float64, nan]

```



## Sparse subclasses

The `SparseSeries` and `SparseDataFrame` classes are deprecated. Visit their API pages for usage.

# Frequently Asked Questions (FAQ)

## DataFrame memory usage

The memory usage of a `DataFrame` (including the index) is shown when calling the `info()`. A configuration option, `display.memory_usage` (see [the list of options](#)), specifies if the `DataFrame`'s memory usage will be displayed when invoking the `df.info()` method.

For example, the memory usage of the `DataFrame` below is shown when calling `info()`:

```
In [1]: dtypes = ['int64', 'float64', 'datetime64[ns]', 'timedelta64[ns]',
...: 'complex128', 'object', 'bool']
...:

In [2]: n = 5000

In [3]: data = {t: np.random.randint(100, size=n).astype(t) for t in dtypes}

In [4]: df = pd.DataFrame(data)

In [5]: df['categorical'] = df['object'].astype('category')

In [6]: df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
int64 5000 non-null int64
float64 5000 non-null float64
datetime64[ns] 5000 non-null datetime64[ns]
timedelta64[ns] 5000 non-null timedelta64[ns]
complex128 5000 non-null complex128
object 5000 non-null object
bool 5000 non-null bool
categorical 5000 non-null category
dtypes: bool(1), category(1), complex128(1), datetime64[ns](1), float64(1), int64(1), object(1),
 timedelta64[ns](1)
memory usage: 289.1+ KB
```

The `+` symbol indicates that the true memory usage could be higher, because pandas does not count the memory used by values in columns with `dtype=object`.

Passing `memory_usage='deep'` will enable a more accurate memory usage report, accounting for the full usage of the contained objects. This is optional as it can be expensive to do this deeper introspection.

```

In [7]: df.info(memory_usage='deep')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 8 columns):
int64 5000 non-null int64
float64 5000 non-null float64
datetime64[ns] 5000 non-null datetime64[ns]
timedelta64[ns] 5000 non-null timedelta64[ns]
complex128 5000 non-null complex128
object 5000 non-null object
bool 5000 non-null bool
categorical 5000 non-null category
dtypes: bool(1), category(1), complex128(1), datetime64[ns](1), float64(1), int64(1), object(1),
 timedelta64[ns](1)
memory usage: 425.6 KB

```

By default the display option is set to `True` but can be explicitly overridden by passing the `memory_usage` argument when invoking `df.info()`.

The memory usage of each column can be found by calling the `memory_usage()` method. This returns a `Series` with an index represented by column names and memory usage of each column shown in bytes. For the `DataFrame` above, the memory usage of each column and the total memory usage can be found with the `memory_usage` method:

```

In [8]: df.memory_usage()
Out[8]:
Index 128
int64 40000
float64 40000
datetime64[ns] 40000
timedelta64[ns] 40000
complex128 80000
object 40000
bool 5000
categorical 10920
dtype: int64

total memory usage of dataframe
In [9]: df.memory_usage().sum()
Out[9]: 296048

```

By default the memory usage of the `DataFrame`'s index is shown in the returned `Series`, the memory usage of the index can be suppressed by passing the `index=False` argument:

```
In [10]: df.memory_usage(index=False)
Out[10]:
int64 40000
float64 40000
datetime64[ns] 40000
timedelta64[ns] 40000
complex128 80000
object 40000
bool 5000
categorical 10920
dtype: int64
```

The memory usage displayed by the `info()` method utilizes the `memory_usage()` method to determine the memory usage of a `DataFrame` while also formatting the output in human-readable units (base-2 representation; i.e. 1KB = 1024 bytes).

See also [Categorical Memory Usage](#).

## Using if/truth statements with pandas

pandas follows the NumPy convention of raising an error when you try to convert something to a `bool`. This happens in an `if`-statement or when using the boolean operations: `and`, `or`, and `not`. It is not clear what the result of the following code should be:

```
>>> if pd.Series([False, True, False]):
... pass
```

Should it be `True` because it's not zero-length, or `False` because there are `False` values? It is unclear, so instead, pandas raises a `ValueError`:

```
>>> if pd.Series([False, True, False]):
... print("I was true")
Traceback
...
ValueError: The truth value of an array is ambiguous. Use a.empty, a.any() or a.all().
```

You need to explicitly choose what you want to do with the `DataFrame`, e.g. use `any()`, `all()` or `empty()`. Alternatively, you might want to compare if the pandas object is `None`:

```
>>> if pd.Series([False, True, False]) is not None:
... print("I was not None")
I was not None
```

Below is how to check if any of the values are `True`:

```
>>> if pd.Series([False, True, False]).any():
... print("I am any")
I am any
```

To evaluate single-element pandas objects in a boolean context, use the method `bool()`:

```
In [11]: pd.Series([True]).bool()
Out[11]: True

In [12]: pd.Series([False]).bool()
Out[12]: False

In [13]: pd.DataFrame([[True]]).bool()
Out[13]: True

In [14]: pd.DataFrame([[False]]).bool()
Out[14]: False
```

## Bitwise boolean

Bitwise boolean operators like `==` and `!=` return a boolean `Series`, which is almost always what you want anyways.

```
>>> s = pd.Series(range(5))
>>> s == 4
0 False
1 False
2 False
3 False
4 True
dtype: bool
```

See [boolean comparisons](#) for more examples.

## Using the `in` operator

Using the Python `in` operator on a `Series` tests for membership in the index, not membership among the values.

```
In [15]: s = pd.Series(range(5), index=list('abcde'))

In [16]: 2 in s
Out[16]: False

In [17]: 'b' in s
Out[17]: True
```

If this behavior is surprising, keep in mind that using `in` on a Python dictionary tests keys, not values, and `Series` are dict-like. To test for membership in the values, use the method `isin()`:

```
In [18]: s.isin([2])
Out[18]:
a False
b False
c True
d False
e False
dtype: bool

In [19]: s.isin([2]).any()
Out[19]: True
```

For `DataFrames`, likewise, `in` applies to the column axis, testing for membership in the list of column names.

## NaN, Integer NA values and NA type promotions

---

### Choice of NA representation

For lack of `NA` (missing) support from the ground up in NumPy and Python in general, we were given the difficult choice between either:

- A masked array solution: an array of data and an array of boolean values indicating whether a value is there or is missing.
- Using a special sentinel value, bit pattern, or set of sentinel values to denote `NA` across the dtypes.

For many reasons we chose the latter. After years of production use it has proven, at least in my opinion, to be the best decision given the state of affairs in NumPy and Python in general. The special value `NaN` (Not-A-Number) is used everywhere as the `NA` value, and there are API functions `isna` and `notna` which can be used across the dtypes to detect NA values.

However, it comes with it a couple of trade-offs which I most certainly have not ignored.

### Support for integer NA

In the absence of high performance `NA` support being built into NumPy from the ground up, the primary casualty is the ability to represent NAs in integer arrays. For example:

```
In [20]: s = pd.Series([1, 2, 3, 4, 5], index=list('abcde'))
```

```
In [21]: s
```

```
Out[21]:
```

```
a 1
b 2
c 3
d 4
e 5
dtype: int64
```

```
In [22]: s.dtype
```

```
Out[22]: dtype('int64')
```

```
In [23]: s2 = s.reindex(['a', 'b', 'c', 'f', 'u'])
```

```
In [24]: s2
```

```
Out[24]:
```

```
a 1.0
b 2.0
c 3.0
f NaN
u NaN
dtype: float64
```

```
In [25]: s2.dtype
```

```
Out[25]: dtype('float64')
```

This trade-off is made largely for memory and performance reasons, and also so that the resulting `Series` continues to be “numeric” .

If you need to represent integers with possibly missing values, use one of the nullable-integer extension dtypes provided by pandas

- [Int8Dtype](#)
- [Int16Dtype](#)
- [Int32Dtype](#)
- [Int64Dtype](#)

```

In [26]: s_int = pd.Series([1, 2, 3, 4, 5], index=list('abcde'),
.....: dtype=pd.Int64Dtype())
.....:

In [27]: s_int
Out[27]:
a 1
b 2
c 3
d 4
e 5
dtype: Int64

In [28]: s_int.dtype
Out[28]: Int64Dtype()

In [29]: s2_int = s_int.reindex(['a', 'b', 'c', 'f', 'u'])

In [30]: s2_int
Out[30]:
a 1
b 2
c 3
f NaN
u NaN
dtype: Int64

In [31]: s2_int.dtype
Out[31]: Int64Dtype()

```

See [Nullable integer data type](#) for more.

## NA type promotions

When introducing NAs into an existing `Series` or `DataFrame` via `reindex()` or some other means, boolean and integer types will be promoted to a different dtype in order to store the NAs. The promotions are summarized in this table:

Typeclass	Promotion dtype for storing NAs
floating	no change
object	no change
integer	cast to float64
boolean	cast to object

While this may seem like a heavy trade-off, I have found very few cases where this is an issue in practice i.e. storing values greater than  $2^{53}$ . Some explanation for the motivation is in the next section.

## Why not make NumPy like R?

Many people have suggested that NumPy should simply emulate the `NA` support present in the more domain-specific statistical programming language [R](#). Part of the reason is the NumPy type hierarchy:

Typeclass	Dtypes
<code>numpy.floating</code>	<code>float16</code> , <code>float32</code> , <code>float64</code> , <code>float128</code>
<code>numpy.integer</code>	<code>int8</code> , <code>int16</code> , <code>int32</code> , <code>int64</code>
<code>numpy.unsignedinteger</code>	<code>uint8</code> , <code>uint16</code> , <code>uint32</code> , <code>uint64</code>
<code>numpy.object_</code>	<code>object_</code>
<code>numpy.bool_</code>	<code>bool_</code>
<code>numpy.character</code>	<code>string_</code> , <code>unicode_</code>

The R language, by contrast, only has a handful of built-in data types: `integer`, `numeric` (floating-point), `character`, and `boolean`. `NA` types are implemented by reserving special bit patterns for each type to be used as the missing value. While doing this with the full NumPy type hierarchy would be possible, it would be a more substantial trade-off (especially for the 8- and 16-bit data types) and implementation undertaking.

An alternate approach is that of using masked arrays. A masked array is an array of data with an associated boolean mask denoting whether each value should be considered `NA` or not. I am personally not in love with this approach as I feel that overall it places a fairly heavy burden on the user and the library implementer. Additionally, it exacts a fairly high performance cost when working with numerical data compared with the simple approach of using `NaN`. Thus, I have chosen the Pythonic “practicality beats purity” approach and traded integer `NA` capability for a much simpler approach of using a special value in float and object arrays to denote `NA`, and promoting integer arrays to floating when NAs must be introduced.

## Differences with NumPy

For `Series` and `DataFrame` objects, `var()` normalizes by  $N-1$  to produce unbiased estimates of the sample variance, while NumPy’s `var` normalizes by  $N$ , which measures the variance of the sample. Note that `cov()` normalizes by  $N-1$  in both pandas and NumPy.

## Thread-safety

As of pandas 0.11, pandas is not 100% thread safe. The known issues relate to the `copy()` method. If you are doing a lot of copying of `DataFrame` objects shared among threads, we recommend holding locks inside the threads where the data copying occurs.

See [this link](#) for more information.



## Byte-Ordering issues

---

Occasionally you may have to deal with data that were created on a machine with a different byte order than the one on which you are running Python. A common symptom of this issue is an error like::

```
Traceback
...
ValueError: Big-endian buffer not supported on little-endian compiler
```

To deal with this issue you should convert the underlying NumPy array to the native system byte order before passing it to `Series` or `DataFrame` constructors using something similar to the following:

```
In [32]: x = np.array(list(range(10)), '>i4') # big endian

In [33]: newx = x.byteswap().newbyteorder() # force native byteorder

In [34]: s = pd.Series(newx)
```

See [the NumPy documentation on byte order](#) for more details.

---

## 烹饪指南

本节列出了一些**短小精悍**的 Pandas 实例与链接。

我们希望 Pandas 用户能积极踊跃地为本文档添加更多内容。为本节添加实用示例的链接或代码，是 Pandas 用户提交第一个 **Pull Request** 最好的选择。

本节列出了简单、精练、易上手的实例代码，以及 Stack Overflow 或 GitHub 上的链接，这些链接包含实例代码的更多详情。

`pd` 与 `np` 是 Pandas 与 Numpy 的缩写。为了让新手易于理解，其它模块是显式导入的。

下列实例均为 Python 3 代码，简单修改即可用于 Python 早期版本。

## 惯用语

---

以下是 Pandas 的惯用语。

[对一系列数据执行 if-then / if-then-else 操作，把计算结果赋值给一系列或多列：](#)

```
In [1]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
...: 'BBB': [10, 20, 30, 40],
...: 'CCC': [100, 50, -30, -50]})
...:
```

```
In [2]: df
```

```
Out[2]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	20	50
2	6	30	-30
3	7	40	-50

## if-then...

在一列上执行 if-then 操作：

```
In [3]: df.loc[df.AAA >= 5, 'BBB'] = -1
```

```
In [4]: df
```

```
Out[4]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	-1	50
2	6	-1	-30
3	7	-1	-50

在两列上执行 if-then 操作：

```
In [5]: df.loc[df.AAA >= 5, ['BBB', 'CCC']] = 555
```

```
In [6]: df
```

```
Out[6]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	555	555
2	6	555	555
3	7	555	555

再添加一行代码，执行 -else 操作：

```
In [7]: df.loc[df.AAA < 5, ['BBB', 'CCC']] = 2000
```

```
In [8]: df
```

```
Out[8]:
```

	AAA	BBB	CCC
0	4	2000	2000
1	5	555	555
2	6	555	555
3	7	555	555

或用 Pandas 的 `where` 设置掩码 (mask) :

```
In [9]: df_mask = pd.DataFrame({'AAA': [True] * 4,
...: 'BBB': [False] * 4,
...: 'CCC': [True, False] * 2})
...:

In [10]: df.where(df_mask, -1000)
Out[10]:
```

	AAA	BBB	CCC
0	4	-1000	2000
1	5	-1000	-1000
2	6	-1000	555
3	7	-1000	-1000

用 NumPy `where()` 函数实现 if-then-else

```
In [11]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
...: 'BBB': [10, 20, 30, 40],
...: 'CCC': [100, 50, -30, -50]})
...:

In [12]: df
Out[12]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	20	50
2	6	30	-30
3	7	40	-50

```
In [13]: df['logic'] = np.where(df['AAA'] > 5, 'high', 'low')

In [14]: df
Out[14]:
```

	AAA	BBB	CCC	logic
0	4	10	100	low
1	5	20	50	low
2	6	30	-30	high
3	7	40	-50	high

## 切割

用布尔条件切割 DataFrame

```
In [15]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
.....: 'BBB': [10, 20, 30, 40],
.....: 'CCC': [100, 50, -30, -50]})
.....:
```

```
In [16]: df
Out[16]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	20	50
2	6	30	-30
3	7	40	-50

```
In [17]: df[df.AAA <= 5]
Out[17]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	20	50

```
In [18]: df[df.AAA > 5]
Out[18]:
```

	AAA	BBB	CCC
2	6	30	-30
3	7	40	-50

## 设置条件

### 多列条件选择

```
In [19]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
.....: 'BBB': [10, 20, 30, 40],
.....: 'CCC': [100, 50, -30, -50]})
.....:
```

```
In [20]: df
Out[20]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	20	50
2	6	30	-30
3	7	40	-50

和 (&) ，不赋值，直接返回 Series:

```
In [21]: df.loc[(df['BBB'] < 25) & (df['CCC'] >= -40), 'AAA']
Out[21]:
```

	AAA
0	4
1	5

Name: AAA, dtype: int64

或 (|) ，不赋值，直接返回 Series:

```
In [22]: df.loc[(df['BBB'] > 25) | (df['CCC'] >= -40), 'AAA']
Out[22]:
0 4
1 5
2 6
3 7
Name: AAA, dtype: int64
```

或 `()`，赋值，修改 DataFrame：

```
In [23]: df.loc[(df['BBB'] > 25) | (df['CCC'] >= 75), 'AAA'] = 0.1

In [24]: df
Out[24]:
 AAA BBB CCC
0 0.1 10 100
1 5.0 20 50
2 0.1 30 -30
3 0.1 40 -50
```

用 `argsort` 选择最接近指定值的行

```
In [25]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
.....: 'BBB': [10, 20, 30, 40],
.....: 'CCC': [100, 50, -30, -50]})
.....:

In [26]: df
Out[26]:
 AAA BBB CCC
0 4 10 100
1 5 20 50
2 6 30 -30
3 7 40 -50

In [27]: aValue = 43.0

In [28]: df.loc[(df.CCC - aValue).abs().argsort()]
Out[28]:
 AAA BBB CCC
1 5 20 50
0 4 10 100
2 6 30 -30
3 7 40 -50
```

用二进制运算符动态减少条件列表

```
In [29]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
.....: 'BBB': [10, 20, 30, 40],
.....: 'CCC': [100, 50, -30, -50]})
.....:
```

```
In [30]: df
Out[30]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	20	50
2	6	30	-30
3	7	40	-50

```
In [31]: Crit1 = df.AAA <= 5.5
```

```
In [32]: Crit2 = df.BBB == 10.0
```

```
In [33]: Crit3 = df.CCC > -40.0
```

硬编码方式为：

```
In [34]: AllCrit = Crit1 & Crit2 & Crit3
```

生成动态条件列表：

```
In [35]: import functools

In [36]: CritList = [Crit1, Crit2, Crit3]

In [37]: AllCrit = functools.reduce(lambda x, y: x & y, CritList)

In [38]: df[AllCrit]
Out[38]:
```

	AAA	BBB	CCC
0	4	10	100

## 选择

### DataFrames

更多信息，请参阅[索引](#)文档。

[行标签与值作为条件](#)

```
In [39]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
.....: 'BBB': [10, 20, 30, 40],
.....: 'CCC': [100, 50, -30, -50]})
.....:
```

```
In [40]: df
```

```
Out[40]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	20	50
2	6	30	-30
3	7	40	-50

```
In [41]: df[(df.AAA <= 6) & (df.index.isin([0, 2, 4]))]
```

```
Out[41]:
```

	AAA	BBB	CCC
0	4	10	100
2	6	30	-30

标签切片用 `loc`，位置切片用 `iloc`

```
In [42]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
.....: 'BBB': [10, 20, 30, 40],
.....: 'CCC': [100, 50, -30, -50]},
.....: index=['foo', 'bar', 'boo', 'kar'])
.....:
```

前 2 个是显式切片方法，第 3 个是通用方法：

1. 位置切片，Python 切片风格，不包括结尾数据；
2. 标签切片，非 Python 切片风格，包括结尾数据；
3. 通用切片，支持两种切片风格，取决于切片用的是标签还是位置。

```
In [43]: df.loc['bar':'kar'] # Label
Out[43]:
```

	AAA	BBB	CCC
bar	5	20	50
boo	6	30	-30
kar	7	40	-50

```
Generic
```

```
In [44]: df.iloc[0:3]
Out[44]:
```

	AAA	BBB	CCC
foo	4	10	100
bar	5	20	50
boo	6	30	-30

```
In [45]: df.loc['bar':'kar']
Out[45]:
```

	AAA	BBB	CCC
bar	5	20	50
boo	6	30	-30
kar	7	40	-50

包含整数，且不从 0 开始的索引，或不是逐步递增的索引会引发歧义。

```
In [46]: data = {'AAA': [4, 5, 6, 7],
....: 'BBB': [10, 20, 30, 40],
....: 'CCC': [100, 50, -30, -50]}
....:
```

```
In [47]: df2 = pd.DataFrame(data=data, index=[1, 2, 3, 4]) # Note index starts at 1.
```

```
In [48]: df2.iloc[1:3] # Position-oriented
Out[48]:
```

	AAA	BBB	CCC
2	5	20	50
3	6	30	-30

```
In [49]: df2.loc[1:3] # Label-oriented
Out[49]:
```

	AAA	BBB	CCC
1	4	10	100
2	5	20	50
3	6	30	-30

用逆运算符 (~) 提取掩码的反向内容



```
In [50]: df = pd.DataFrame({'AAA': [4, 5, 6, 7],
.....: 'BBB': [10, 20, 30, 40],
.....: 'CCC': [100, 50, -30, -50]})
.....:
```

```
In [51]: df
Out[51]:
```

	AAA	BBB	CCC
0	4	10	100
1	5	20	50
2	6	30	-30
3	7	40	-50

```
In [52]: df[~((df.AAA <= 6) & (df.index.isin([0, 2, 4])))]
Out[52]:
```

	AAA	BBB	CCC
1	5	20	50
3	7	40	-50

## 生成新列

### 用 applymap 高效动态生成新列

```
In [53]: df = pd.DataFrame({'AAA': [1, 2, 1, 3],
.....: 'BBB': [1, 1, 2, 2],
.....: 'CCC': [2, 1, 3, 1]})
.....:
```

```
In [54]: df
Out[54]:
```

	AAA	BBB	CCC
0	1	1	2
1	2	1	1
2	1	2	3
3	3	2	1

```
In [55]: source_cols = df.columns # Or some subset would work too
```

```
In [56]: new_cols = [str(x) + "_cat" for x in source_cols]
```

```
In [57]: categories = {1: 'Alpha', 2: 'Beta', 3: 'Charlie'}
```

```
In [58]: df[new_cols] = df[source_cols].applymap(categories.get)
```

```
In [59]: df
Out[59]:
```

	AAA	BBB	CCC	AAA_cat	BBB_cat	CCC_cat
0	1	1	2	Alpha	Alpha	Beta
1	2	1	1	Beta	Alpha	Alpha
2	1	2	3	Alpha	Beta	Charlie
3	3	2	1	Charlie	Beta	Alpha

### 分组时用 min()

```
In [60]: df = pd.DataFrame({'AAA': [1, 1, 1, 2, 2, 2, 3, 3],
....: 'BBB': [2, 1, 3, 4, 5, 1, 2, 3]})
....:
```

```
In [61]: df
```

```
Out[61]:
```

	AAA	BBB
0	1	2
1	1	1
2	1	3
3	2	4
4	2	5
5	2	1
6	3	2
7	3	3

方法1: 用 `idxmin()` 提取每组最小值的索引

```
In [62]: df.loc[df.groupby("AAA")["BBB"].idxmin()]
```

```
Out[62]:
```

	AAA	BBB
1	1	1
5	2	1
6	3	2

方法 2: 先排序, 再提取每组的第一个值

```
In [63]: df.sort_values(by="BBB").groupby("AAA", as_index=False).first()
```

```
Out[63]:
```

	AAA	BBB
0	1	1
1	2	1
2	3	2

注意, 提取的数据一样, 但索引不一样。

## 多层索引

更多信息, 请参阅[多层索引](#)文档。

[用带标签的字典创建多层索引](#)

```

In [64]: df = pd.DataFrame({'row': [0, 1, 2],
.....: 'One_X': [1.1, 1.1, 1.1],
.....: 'One_Y': [1.2, 1.2, 1.2],
.....: 'Two_X': [1.11, 1.11, 1.11],
.....: 'Two_Y': [1.22, 1.22, 1.22]})
.....:

In [65]: df
Out[65]:
 row One_X One_Y Two_X Two_Y
0 0 1.1 1.2 1.11 1.22
1 1 1.1 1.2 1.11 1.22
2 2 1.1 1.2 1.11 1.22

设置索引标签
In [66]: df = df.set_index('row')

In [67]: df
Out[67]:
 One_X One_Y Two_X Two_Y
row
0 1.1 1.2 1.11 1.22
1 1.1 1.2 1.11 1.22
2 1.1 1.2 1.11 1.22

多层索引的列
In [68]: df.columns = pd.MultiIndex.from_tuples([tuple(c.split('_'))
.....: for c in df.columns])
.....:

In [69]: df
Out[69]:
 One Two
 X Y X Y
row
0 1.1 1.2 1.11 1.22
1 1.1 1.2 1.11 1.22
2 1.1 1.2 1.11 1.22

先 stack, 然后 Reset 索引

In [70]: df = df.stack(0).reset_index(1)

In [71]: df
Out[71]:
 level_1 X Y
row
0 One 1.10 1.20
0 Two 1.11 1.22
1 One 1.10 1.20
1 Two 1.11 1.22
2 One 1.10 1.20
2 Two 1.11 1.22

修整标签, 注意自动添加了标签 `level_1`

```

```
In [72]: df.columns = ['Sample', 'All_X', 'All_Y']
```

```
In [73]: df
```

```
Out[73]:
```

	Sample	All_X	All_Y
row			
0	One	1.10	1.20
0	Two	1.11	1.22
1	One	1.10	1.20
1	Two	1.11	1.22
2	One	1.10	1.20
2	Two	1.11	1.22

## 运算

### 多层索引运算要用广播机制

```
In [74]: cols = pd.MultiIndex.from_tuples([(x, y) for x in ['A', 'B', 'C']
.....: for y in ['0', 'I']])
.....:
```

```
In [75]: df = pd.DataFrame(np.random.randn(2, 6), index=['n', 'm'], columns=cols)
```

```
In [76]: df
```

```
Out[76]:
```

	A		B		C	
	0	I	0	I	0	I
n	0.469112	-0.282863	-1.509059	-1.135632	1.212112	-0.173215
m	0.119209	-1.044236	-0.861849	-2.104569	-0.494929	1.071804

```
In [77]: df = df.div(df['C'], level=1)
```

```
In [78]: df
```

```
Out[78]:
```

	A		B		C	
	0	I	0	I	0	I
n	0.387021	1.633022	-1.244983	6.556214	1.0	1.0
m	-0.240860	-0.974279	1.741358	-1.963577	1.0	1.0

## 切片

### 用 xs 切片多层索引

```
In [79]: coords = [('AA', 'one'), ('AA', 'six'), ('BB', 'one'), ('BB', 'two'),
.....: ('BB', 'six')]
.....:
```

```
In [80]: index = pd.MultiIndex.from_tuples(coords)
```

```
In [81]: df = pd.DataFrame([11, 22, 33, 44, 55], index, ['MyData'])
```

```
In [82]: df
```

```
Out[82]:
```

	MyData
AA one	11
six	22
BB one	33
two	44
six	55

提取第一层与索引第一个轴的交叉数据：

```
注意：level 与 axis 是可选项，默认为 0
```

```
In [83]: df.xs('BB', level=0, axis=0)
```

```
Out[83]:
```

	MyData
one	33
two	44
six	55

……现在是第 1 个轴的第 2 层

```
In [84]: df.xs('six', level=1, axis=0)
```

```
Out[84]:
```

	MyData
AA	22
BB	55

用 `xs` 切片多层索引，方法 #2

```

In [85]: import itertools

In [86]: index = list(itertools.product(['Ada', 'Quinn', 'Violet'],
....: ['Comp', 'Math', 'Sci']))
....:

In [87]: headr = list(itertools.product(['Exams', 'Labs'], ['I', 'II']))

In [88]: indx = pd.MultiIndex.from_tuples(index, names=['Student', 'Course'])

In [89]: cols = pd.MultiIndex.from_tuples(headr) # Notice these are un-named

In [90]: data = [[70 + x + y + (x * y) % 3 for x in range(4)] for y in range(9)]

In [91]: df = pd.DataFrame(data, indx, cols)

In [92]: df
Out[92]:

```

		Exams		Labs	
		I	II	I	II
Student	Course				
Ada	Comp	70	71	72	73
	Math	71	73	75	74
	Sci	72	75	75	75
Quinn	Comp	73	74	75	76
	Math	74	76	78	77
	Sci	75	78	78	78
Violet	Comp	76	77	78	79
	Math	77	79	81	80
	Sci	78	81	81	81

```

In [93]: All = slice(None)

In [94]: df.loc['Violet']
Out[94]:

```

	Exams		Labs	
	I	II	I	II
Course				
Comp	76	77	78	79
Math	77	79	81	80
Sci	78	81	81	81

```

In [95]: df.loc[(All, 'Math'), All]
Out[95]:

```

		Exams		Labs	
		I	II	I	II
Student	Course				
Ada	Math	71	73	75	74
Quinn	Math	74	76	78	77
Violet	Math	77	79	81	80

```

In [96]: df.loc[(slice('Ada', 'Quinn'), 'Math'), All]
Out[96]:

```

		Exams		Labs	
		I	II	I	II
Student	Course				
Ada	Math	71	73	75	74
Quinn	Math	74	76	78	77

```

Student Course
Ada Math 71 73 75 74
Quinn Math 74 76 78 77

In [97]: df.loc[(All, 'Math'), ('Exams')]
Out[97]:
 I II
Student Course
Ada Math 71 73
Quinn Math 74 76
Violet Math 77 79

In [98]: df.loc[(All, 'Math'), (All, 'II')]
Out[98]:
 Exams Labs
 II II
Student Course
Ada Math 73 74
Quinn Math 76 77
Violet Math 79 80

```

用 xs 设置多层索引比例

## 排序

用多层索引按指定列或列序列列表排序x

```

In [99]: df.sort_values(by=('Labs', 'II'), ascending=False)
Out[99]:
 Exams Labs
 I II I II
Student Course
Violet Sci 78 81 81 81
 Math 77 79 81 80
 Comp 76 77 78 79
Quinn Sci 75 78 78 78
 Math 74 76 78 77
 Comp 73 74 75 76
Ada Sci 72 75 75 75
 Math 71 73 75 74
 Comp 70 71 72 73

```

部分选择，需要排序

## 层级

为多层索引添加一层

平铺结构化列

## 缺失数据

---

[缺失数据](#) 文档。

向前填充逆序时间序列。

```
In [100]: df = pd.DataFrame(np.random.randn(6, 1),
.....: index=pd.date_range('2013-08-01', periods=6, freq='B'),
.....: columns=list('A'))
.....:

In [101]: df.loc[df.index[3], 'A'] = np.nan

In [102]: df
Out[102]:
```

	A
2013-08-01	0.721555
2013-08-02	-0.706771
2013-08-05	-1.039575
2013-08-06	NaN
2013-08-07	-0.424972
2013-08-08	0.567020

```
In [103]: df.reindex(df.index[::-1]).ffill()
Out[103]:
```

	A
2013-08-08	0.567020
2013-08-07	-0.424972
2013-08-06	-0.424972
2013-08-05	-1.039575
2013-08-02	-0.706771
2013-08-01	0.721555

[空值时重置为 0，有值时累加](#)

## 替换

[用反引用替换](#)

## 分组

---

[分组](#) 文档。

[用 apply 执行分组基础操作](#)

与聚合不同，传递给 DataFrame 子集的 apply 可回调，可以访问所有列。



```

In [104]: df = pd.DataFrame({'animal': 'cat dog cat fish dog cat cat'.split(),
.....: 'size': list('SSMMMLL'),
.....: 'weight': [8, 10, 11, 1, 20, 12, 12],
.....: 'adult': [False] * 5 + [True] * 2})
.....:

In [105]: df
Out[105]:
 animal size weight adult
0 cat S 8 False
1 dog S 10 False
2 cat M 11 False
3 fish M 1 False
4 dog M 20 False
5 cat L 12 True
6 cat L 12 True

提取 size 列最重的动物列表
In [106]: df.groupby('animal').apply(lambda subf: subf['size'][subf['weight'].idxmax()])
Out[106]:
animal
cat L
dog M
fish M
dtype: object

```

## 使用 get\_group

```

In [107]: gb = df.groupby(['animal'])

In [108]: gb.get_group('cat')
Out[108]:
 animal size weight adult
0 cat S 8 False
2 cat M 11 False
5 cat L 12 True
6 cat L 12 True

```

## 为同一分组的不同内容使用 Apply 函数

```
In [109]: def GrowUp(x):
.....: avg_weight = sum(x[x['size'] == 'S'].weight * 1.5)
.....: avg_weight += sum(x[x['size'] == 'M'].weight * 1.25)
.....: avg_weight += sum(x[x['size'] == 'L'].weight)
.....: avg_weight /= len(x)
.....: return pd.Series(['L', avg_weight, True],
.....: index=['size', 'weight', 'adult'])
.....:
```

```
In [110]: expected_df = gb.apply(GrowUp)
```

```
In [111]: expected_df
```

```
Out[111]:
 size weight adult
animal
cat L 12.4375 True
dog L 20.0000 True
fish L 1.2500 True
```

## Apply 函数扩展

```
In [112]: S = pd.Series([i / 100.0 for i in range(1, 11)])
```

```
In [113]: def cum_ret(x, y):
.....: return x * (1 + y)
.....:
```

```
In [114]: def red(x):
.....: return functools.reduce(cum_ret, x, 1.0)
.....:
```

```
In [115]: S.expanding().apply(red, raw=True)
```

```
Out[115]:
0 1.010000
1 1.030200
2 1.061106
3 1.103550
4 1.158728
5 1.228251
6 1.314229
7 1.419367
8 1.547110
9 1.701821
dtype: float64
```

## 用分组里的剩余值的平均值进行替换

```
In [116]: df = pd.DataFrame({'A': [1, 1, 2, 2], 'B': [1, -1, 1, 2]})
```

```
In [117]: gb = df.groupby('A')
```

```
In [118]: def replace(g):
.....: mask = g < 0
.....: return g.where(mask, g[~mask].mean())
.....:
```

```
In [119]: gb.transform(replace)
```

```
Out[119]:
```

```
 B
0 1.0
1 -1.0
2 1.5
3 1.5
```

### 按聚合数据排序

```
In [120]: df = pd.DataFrame({'code': ['foo', 'bar', 'baz'] * 2,
.....: 'data': [0.16, -0.21, 0.33, 0.45, -0.59, 0.62],
.....: 'flag': [False, True] * 3})
.....:
```

```
In [121]: code_groups = df.groupby('code')
```

```
In [122]: agg_n_sort_order = code_groups[['data']].transform(sum).sort_values(by='data')
```

```
In [123]: sorted_df = df.loc[agg_n_sort_order.index]
```

```
In [124]: sorted_df
```

```
Out[124]:
 code data flag
1 bar -0.21 True
4 bar -0.59 False
0 foo 0.16 False
3 foo 0.45 True
2 baz 0.33 False
5 baz 0.62 True
```

### 创建多个聚合列

```
In [125]: rng = pd.date_range(start="2014-10-07", periods=10, freq='2min')
```

```
In [126]: ts = pd.Series(data=list(range(10)), index=rng)
```

```
In [127]: def MyCust(x):
.....: if len(x) > 2:
.....: return x[1] * 1.234
.....: return pd.NaT
.....:
```

```
In [128]: mhc = {'Mean': np.mean, 'Max': np.max, 'Custom': MyCust}
```

```
In [129]: ts.resample("5min").apply(mhc)
```

```
Out[129]:
```

Mean	2014-10-07 00:00:00	1
	2014-10-07 00:05:00	3.5
	2014-10-07 00:10:00	6
	2014-10-07 00:15:00	8.5
Max	2014-10-07 00:00:00	2
	2014-10-07 00:05:00	4
	2014-10-07 00:10:00	7
	2014-10-07 00:15:00	9
Custom	2014-10-07 00:00:00	1.234
	2014-10-07 00:05:00	NaT
	2014-10-07 00:10:00	7.404
	2014-10-07 00:15:00	NaT

```
dtype: object
```

```
In [130]: ts
```

```
Out[130]:
```

2014-10-07 00:00:00	0
2014-10-07 00:02:00	1
2014-10-07 00:04:00	2
2014-10-07 00:06:00	3
2014-10-07 00:08:00	4
2014-10-07 00:10:00	5
2014-10-07 00:12:00	6
2014-10-07 00:14:00	7
2014-10-07 00:16:00	8
2014-10-07 00:18:00	9

```
Freq: 2T, dtype: int64
```

## 为 DataFrame 创建值计数列

```
In [131]: df = pd.DataFrame({'Color': 'Red Red Red Blue'.split(),
.....: 'Value': [100, 150, 50, 50]})
.....:
```

```
In [132]: df
Out[132]:
```

	Color	Value
0	Red	100
1	Red	150
2	Red	50
3	Blue	50

```
In [133]: df['Counts'] = df.groupby(['Color']).transform(len)
```

```
In [134]: df
Out[134]:
```

	Color	Value	Counts
0	Red	100	3
1	Red	150	3
2	Red	50	3
3	Blue	50	1

### 基于索引唯一某列不同分组的值

```
In [135]: df = pd.DataFrame({'line_race': [10, 10, 8, 10, 10, 8],
.....: 'beyer': [99, 102, 103, 103, 88, 100]},
.....: index=['Last Gunfighter', 'Last Gunfighter',
.....: 'Last Gunfighter', 'Paynter', 'Paynter',
.....: 'Paynter'])
.....:
```

```
In [136]: df
Out[136]:
```

	line_race	beyer
Last Gunfighter	10	99
Last Gunfighter	10	102
Last Gunfighter	8	103
Paynter	10	103
Paynter	10	88
Paynter	8	100

```
In [137]: df['beyer_shifted'] = df.groupby(level=0)['beyer'].shift(1)
```

```
In [138]: df
Out[138]:
```

	line_race	beyer	beyer_shifted
Last Gunfighter	10	99	NaN
Last Gunfighter	10	102	99.0
Last Gunfighter	8	103	102.0
Paynter	10	103	NaN
Paynter	10	88	103.0
Paynter	8	100	88.0

### 选择每组最大值的行

```
In [139]: df = pd.DataFrame({'host': ['other', 'other', 'that', 'this', 'this'],
.....: 'service': ['mail', 'web', 'mail', 'mail', 'web'],
.....: 'no': [1, 2, 1, 2, 1]}).set_index(['host', 'service'])
.....:
```

```
In [140]: mask = df.groupby(level=0).agg('idxmax')
```

```
In [141]: df_count = df.loc[mask['no']].reset_index()
```

```
In [142]: df_count
```

```
Out[142]:
 host service no
0 other web 2
1 that mail 1
2 this mail 2
```

## Python itertools.groupby 式分组

```
In [143]: df = pd.DataFrame([0, 1, 0, 1, 1, 1, 0, 1, 1], columns=['A'])
```

```
In [144]: df.A.groupby((df.A != df.A.shift()).cumsum()).groups
```

```
Out[144]:
{1: Int64Index([0], dtype='int64'),
 2: Int64Index([1], dtype='int64'),
 3: Int64Index([2], dtype='int64'),
 4: Int64Index([3, 4, 5], dtype='int64'),
 5: Int64Index([6], dtype='int64'),
 6: Int64Index([7, 8], dtype='int64')}
```

```
In [145]: df.A.groupby((df.A != df.A.shift()).cumsum()).cumsum()
```

```
Out[145]:
0 0
1 1
2 0
3 1
4 2
5 3
6 0
7 1
8 2
Name: A, dtype: int64
```

## 扩展数据

### Alignment and to-date

### 基于计数值进行移动窗口计算

### 按时间间隔计算滚动平均

## 分割

### 分割 DataFrame

按指定逻辑，将不同的行，分割成 DataFrame 列表。

```
In [146]: df = pd.DataFrame(data={'Case': ['A', 'A', 'A', 'B', 'A', 'A', 'B', 'A',
.....: 'A'],
.....: 'Data': np.random.randn(9)})
.....:
```

```
In [147]: dfs = list(zip(*df.groupby((1 * (df['Case'] == 'B')).cumsum()
.....: .rolling(window=3, min_periods=1).median())))[-1]
.....:
```

```
In [148]: dfs[0]
Out[148]:
```

	Case	Data
0	A	0.276232
1	A	-1.087401
2	A	-0.673690
3	B	0.113648

```
In [149]: dfs[1]
Out[149]:
```

	Case	Data
4	A	-1.478427
5	A	0.524988
6	B	0.404705

```
In [150]: dfs[2]
Out[150]:
```

	Case	Data
7	A	0.577046
8	A	-1.715002

## 透视表

[透视表 文档](#)。

部分汇总与小计

```
In [151]: df = pd.DataFrame(data={'Province': ['ON', 'QC', 'BC', 'AL', 'AL', 'MN', 'ON'],
.....: 'City': ['Toronto', 'Montreal', 'Vancouver',
.....: 'Calgary', 'Edmonton', 'Winnipeg',
.....: 'Windsor'],
.....: 'Sales': [13, 6, 16, 8, 4, 3, 1]})
.....:
```

```
In [152]: table = pd.pivot_table(df, values=['Sales'], index=['Province'],
.....: columns=['City'], aggfunc=np.sum, margins=True)
.....:
```

```
In [153]: table.stack('City')
```

```
Out[153]:
```

		Sales
Province	City	
AL	All	12.0
	Calgary	8.0
	Edmonton	4.0
BC	All	16.0
	Vancouver	16.0
...		...
All	Montreal	6.0
	Toronto	13.0
	Vancouver	16.0
	Windsor	1.0
	Winnipeg	3.0

```
[20 rows x 1 columns]
```

类似 R 的 `plyr` 频率表



```

In [154]: grades = [48, 99, 75, 80, 42, 80, 72, 68, 36, 78]

In [155]: df = pd.DataFrame({'ID': ["%d" % r for r in range(10)],
.....: 'Gender': ['F', 'M', 'F', 'M', 'F',
.....: 'M', 'F', 'M', 'M', 'M'],
.....: 'ExamYear': ['2007', '2007', '2007', '2008', '2008',
.....: '2008', '2008', '2009', '2009', '2009'],
.....: 'Class': ['algebra', 'stats', 'bio', 'algebra',
.....: 'algebra', 'stats', 'stats', 'algebra',
.....: 'bio', 'bio'],
.....: 'Participated': ['yes', 'yes', 'yes', 'yes', 'no',
.....: 'yes', 'yes', 'yes', 'yes', 'yes'],
.....: 'Passed': ['yes' if x > 50 else 'no' for x in grades],
.....: 'Employed': [True, True, True, False,
.....: False, False, False, True, True, False],
.....: 'Grade': grades})

In [156]: df.groupby('ExamYear').agg({'Participated': lambda x: x.value_counts()['yes'],
.....: 'Passed': lambda x: sum(x == 'yes'),
.....: 'Employed': lambda x: sum(x),
.....: 'Grade': lambda x: sum(x) / len(x)})

Out[156]:

```

	Participated	Passed	Employed	Grade
ExamYear				
2007	3	2	3	74.000000
2008	3	3	0	68.500000
2009	3	2	2	60.666667

## 按年生成 DataFrame

跨列表创建年月：

```
In [157]: df = pd.DataFrame({'value': np.random.randn(36)},
.....: index=pd.date_range('2011-01-01', freq='M', periods=36))
.....:

In [158]: pd.pivot_table(df, index=df.index.month, columns=df.index.year,
.....: values='value', aggfunc='sum')
.....:
Out[158]:
```

	2011	2012	2013
1	-1.039268	-0.968914	2.565646
2	-0.370647	-1.294524	1.431256
3	-1.157892	0.413738	1.340309
4	-1.344312	0.276662	-1.170299
5	0.844885	-0.472035	-0.226169
6	1.075770	-0.013960	0.410835
7	-0.109050	-0.362543	0.813850
8	1.643563	-0.006154	0.132003
9	-1.469388	-0.923061	-0.827317
10	0.357021	0.895717	-0.076467
11	-0.674600	0.805244	-1.187678
12	-1.776904	-1.206412	1.130127

## Apply 函数

### 把嵌入列表转换为多层索引 DataFrame

```
In [159]: df = pd.DataFrame(data={'A': [[2, 4, 8, 16], [100, 200], [10, 20, 30]],
.....: 'B': [['a', 'b', 'c'], ['jj', 'kk'], ['ccc']]},
.....: index=['I', 'II', 'III'])
.....:

In [160]: def SeriesFromSubList(aList):
.....: return pd.Series(aList)
.....:

In [161]: df_orgz = pd.concat({ind: row.apply(SeriesFromSubList)
.....: for ind, row in df.iterrows()})
.....:

In [162]: df_orgz
Out[162]:
```

		0	1	2	3
I	A	2	4	8	16.0
	B	a	b	c	NaN
II	A	100	200	NaN	NaN
	B	jj	kk	NaN	NaN
III	A	10	20	30	NaN
	B	ccc	NaN	NaN	NaN

### 返回 Series

Rolling Apply to multiple columns where function calculates a Series before a Scalar from the Series is returned

```

In [163]: df = pd.DataFrame(data=np.random.randn(2000, 2) / 10000,
.....: index=pd.date_range('2001-01-01', periods=2000),
.....: columns=['A', 'B'])
.....:

In [164]: df
Out[164]:
 A B
2001-01-01 -0.000144 -0.000141
2001-01-02 0.000161 0.000102
2001-01-03 0.000057 0.000088
2001-01-04 -0.000221 0.000097
2001-01-05 -0.000201 -0.000041
...
2006-06-19 0.000040 -0.000235
2006-06-20 -0.000123 -0.000021
2006-06-21 -0.000113 0.000114
2006-06-22 0.000136 0.000109
2006-06-23 0.000027 0.000030

[2000 rows x 2 columns]

In [165]: def gm(df, const):
.....: v = (((df.A + df.B) + 1).cumprod()) - 1) * const
.....: return v.iloc[-1]
.....:

In [166]: s = pd.Series({df.index[i]: gm(df.iloc[i:min(i + 51, len(df) - 1)], 5)
.....: for i in range(len(df) - 50)})
.....:

In [167]: s
Out[167]:
2001-01-01 0.000930
2001-01-02 0.002615
2001-01-03 0.001281
2001-01-04 0.001117
2001-01-05 0.002772
...
2006-04-30 0.003296
2006-05-01 0.002629
2006-05-02 0.002081
2006-05-03 0.004247
2006-05-04 0.003928
Length: 1950, dtype: float64

```

## 返回标量值

Rolling Apply to multiple columns where function returns a Scalar (Volume Weighted Average Price)  
 对多列执行滚动 Apply，函数返回标量值（成交价加权平均价）

```

In [168]: rng = pd.date_range(start='2014-01-01', periods=100)

In [169]: df = pd.DataFrame({'Open': np.random.randn(len(rng)),
.....: 'Close': np.random.randn(len(rng)),
.....: 'Volume': np.random.randint(100, 2000, len(rng))},
.....: index=rng)
.....:

In [170]: df
Out[170]:
```

	Open	Close	Volume
2014-01-01	-1.611353	-0.492885	1219
2014-01-02	-3.000951	0.445794	1054
2014-01-03	-0.138359	-0.076081	1381
2014-01-04	0.301568	1.198259	1253
2014-01-05	0.276381	-0.669831	1728
...	...	...	...
2014-04-06	-0.040338	0.937843	1188
2014-04-07	0.359661	-0.285908	1864
2014-04-08	0.060978	1.714814	941
2014-04-09	1.759055	-0.455942	1065
2014-04-10	0.138185	-1.147008	1453

```

[100 rows x 3 columns]

In [171]: def vwap(bars):
.....: return ((bars.Close * bars.Volume).sum() / bars.Volume.sum())
.....:

In [172]: window = 5

In [173]: s = pd.concat([(pd.Series(vwap(df.iloc[i:i + window]),
.....: index=[df.index[i + window]]))
.....: for i in range(len(df) - window)])
.....:

In [174]: s.round(2)
Out[174]:
```

2014-01-06	0.02
2014-01-07	0.11
2014-01-08	0.10
2014-01-09	0.07
2014-01-10	-0.29
...	
2014-04-06	-0.63
2014-04-07	-0.02
2014-04-08	-0.03
2014-04-09	0.34
2014-04-10	0.29

```

Length: 95, dtype: float64

```

## 时间序列

---

删除指定时间之外的数据

用 `indexer` 提取在时间范围内的数据

创建不包括周末，且只包含指定时间的日期时间范围

矢量查询

聚合与绘制时间序列

把以小时为列，天为行的矩阵转换为连续的时间序列。 [如何重排 DataFrame?](#)

重建索引为指定频率时，如何处理重复值

为 `DatetimeIndex` 里每条记录计算当月第一天

```
In [175]: dates = pd.date_range('2000-01-01', periods=5)

In [176]: dates.to_period(freq='M').to_timestamp()
Out[176]:
DatetimeIndex(['2000-01-01', '2000-01-01', '2000-01-01', '2000-01-01',
 '2000-01-01'],
 dtype='datetime64[ns]', freq=None)
```

## 重采样

[重采样](#) 文档。

用 `Grouper` 代替 `TimeGrouper` 处理时间分组的值

含缺失值的时间分组

`Grouper` 的有效时间频率参数

用多层索引分组

用 `TimeGrouper` 与另一个分组创建子分组，再 `Apply` 自定义函数

按自定义时间段重采样

不添加新日期，重采样某日数据

按分钟重采样数据

分组重采样

## 合并

[连接 docs](#). The [Join](#)文档。

模拟 R 的 `rbind`：追加两个重叠索引的 DataFrame

```
In [177]: rng = pd.date_range('2000-01-01', periods=6)

In [178]: df1 = pd.DataFrame(np.random.randn(6, 3), index=rng, columns=['A', 'B', 'C'])

In [179]: df2 = df1.copy()
```

基于 df 构建器，需要 `ignore_index`。

```
In [180]: df = df1.append(df2, ignore_index=True)

In [181]: df
Out[181]:
```

	A	B	C
0	-0.870117	-0.479265	-0.790855
1	0.144817	1.726395	-0.464535
2	-0.821906	1.597605	0.187307
3	-0.128342	-1.511638	-0.289858
4	0.399194	-1.430030	-0.639760
5	1.115116	-2.012600	1.810662
6	-0.870117	-0.479265	-0.790855
7	0.144817	1.726395	-0.464535
8	-0.821906	1.597605	0.187307
9	-0.128342	-1.511638	-0.289858
10	0.399194	-1.430030	-0.639760
11	1.115116	-2.012600	1.810662

[自连接 DataFrame](#)

```
In [182]: df = pd.DataFrame(data={'Area': ['A'] * 5 + ['C'] * 2,
.....: 'Bins': [110] * 2 + [160] * 3 + [40] * 2,
.....: 'Test_0': [0, 1, 0, 1, 2, 0, 1],
.....: 'Data': np.random.randn(7)})
.....:
```

```
In [183]: df
```

```
Out[183]:
```

	Area	Bins	Test_0	Data
0	A	110	0	-0.433937
1	A	110	1	-0.160552
2	A	160	0	0.744434
3	A	160	1	1.754213
4	A	160	2	0.000850
5	C	40	0	0.342243
6	C	40	1	1.070599

```
In [184]: df['Test_1'] = df['Test_0'] - 1
```

```
In [185]: pd.merge(df, df, left_on=['Bins', 'Area', 'Test_0'],
.....: right_on=['Bins', 'Area', 'Test_1'],
.....: suffixes=('_L', '_R'))
.....:
```

```
Out[185]:
```

	Area	Bins	Test_0_L	Data_L	Test_1_L	Test_0_R	Data_R	Test_1_R
0	A	110	0	-0.433937	-1	1	-0.160552	0
1	A	160	0	0.744434	-1	1	1.754213	0
2	A	160	1	1.754213	0	2	0.000850	1
3	C	40	0	0.342243	-1	1	1.070599	0

[如何设置索引与连接](#)

[KDB 式的 asof 连接](#)

[基于符合条件的值进行连接](#)

[基于范围里的值，用 searchsorted 合并](#)

## 可视化

[可视化 文档。](#)

[让 Matplotlib 看上去像 R](#)

[设置 x 轴的主次标签](#)

[在 iPython Notebook 里创建多个可视图](#)

[创建多行可视图](#)

[绘制热力图](#)

标记时间序列图

标记时间序列图 #2

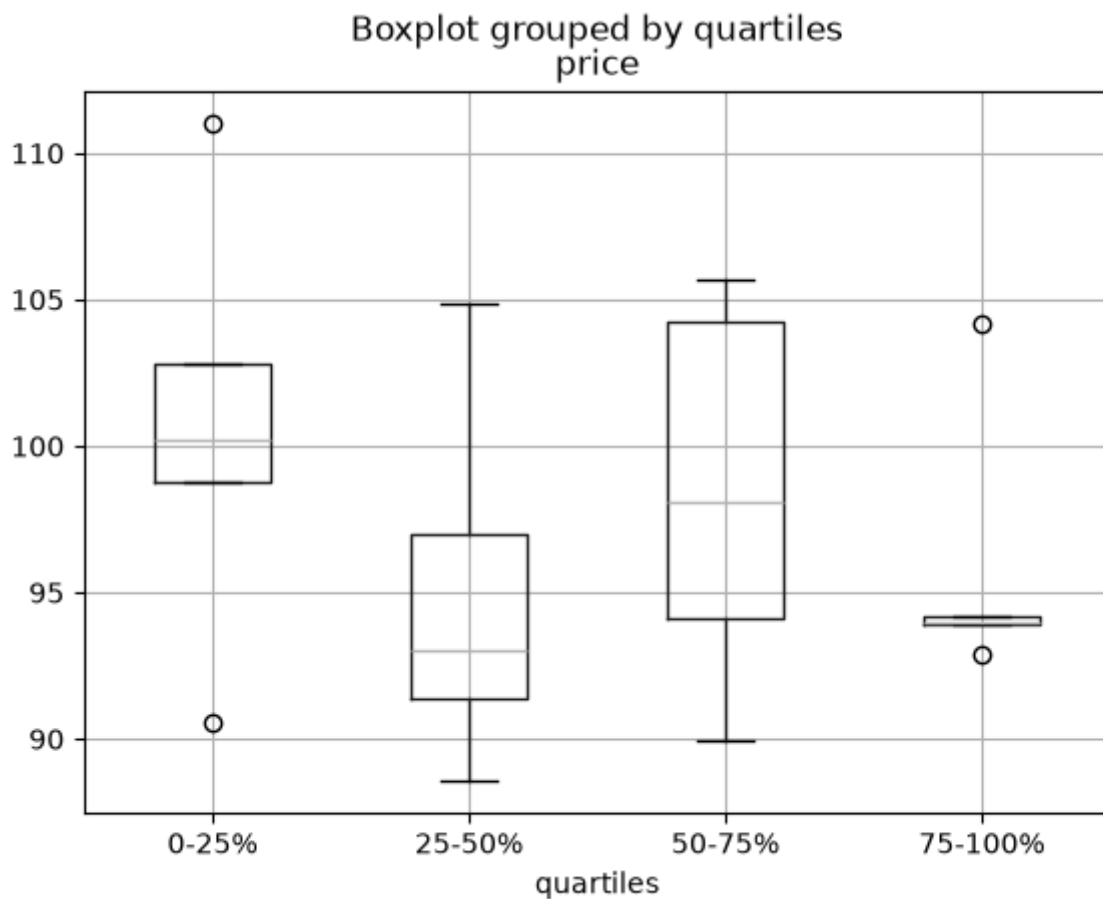
用 Pandas、Vincent、xlsxwriter 生成 Excel 文件里的嵌入可视图

为分层变量的每个四分位数绘制箱型图

```
In [186]: df = pd.DataFrame(
.....: {'stratifying_var': np.random.uniform(0, 100, 20),
.....: 'price': np.random.normal(100, 5, 20)})
.....:

In [187]: df['quartiles'] = pd.qcut(
.....: df['stratifying_var'],
.....: 4,
.....: labels=['0-25%', '25-50%', '50-75%', '75-100%'])
.....:

In [188]: df.boxplot(column='price', by='quartiles')
Out[188]: <matplotlib.axes._subplots.AxesSubplot at 0x7efff077f910>
```





## 数据输入输出

---

[SQL 与 HDF5 性能对比](#)

### CSV

[CSV文档](#)

[read\\_csv 函数实战](#)

[把 DataFrame 追加到 CSV 文件](#)

[分块读取 CSV](#)

[分块读取指定的行](#)

[只读取 DataFrame 的前几列](#)

读取不是 gzip 或 bz2 压缩（read\_csv 可识别的内置压缩格式）的文件。本例在介绍如何读取 WinZip 压缩文件的同时，还介绍了在环境管理器里打开文件，并读取内容的通用操作方式。[详见本链接](#)

[推断文件数据类型](#)

[处理出错数据](#)

[处理出错数据 II](#)

[用 Unix 时间戳读取 CSV，并转为本地时区](#)

[写入多行索引 CSV 时，不写入重复值](#)

### 从多个文件读取数据，创建单个 DataFrame

最好的方式是先一个个读取单个文件，然后再把每个文件的内容存成列表，再用 `pd.concat()` 组合成一个 DataFrame:

```
In [189]: for i in range(3):
.....: data = pd.DataFrame(np.random.randn(10, 4))
.....: data.to_csv('file_{}.csv'.format(i))
.....:

In [190]: files = ['file_0.csv', 'file_1.csv', 'file_2.csv']

In [191]: result = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)
```

还可以用同样的方法读取所有匹配同一模式的文件，下面这个例子使用的是 glob:

```
In [192]: import glob

In [193]: import os

In [194]: files = glob.glob('file_*.csv')

In [195]: result = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)
```

最后，这种方式也适用于 [io 文档](#) 介绍的其它 `pd.read_*` 函数。

## 解析多列里的日期组件

用一种格式解析多列的日期组件，速度更快。

```
In [196]: i = pd.date_range('20000101', periods=10000)

In [197]: df = pd.DataFrame({'year': i.year, 'month': i.month, 'day': i.day})

In [198]: df.head()
Out[198]:
 year month day
0 2000 1 1
1 2000 1 2
2 2000 1 3
3 2000 1 4
4 2000 1 5

In [199]: %timeit pd.to_datetime(df.year * 10000 + df.month * 100 + df.day, format='%Y%m%d')
.....: ds = df.apply(lambda x: "%04d%02d%02d" % (x['year'],
.....: x['month'], x['day']), axis=1)
.....: ds.head()
.....: %timeit pd.to_datetime(ds)
.....:
10.6 ms +- 698 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
3.21 ms +- 36.4 us per loop (mean +- std. dev. of 7 runs, 100 loops each)
```

## 跳过标题与数据之间的行

```
In [200]: data = """;;;;
.....: ;;;;
.....: ;;;;
.....: ;;;;
.....: ;;;;
.....: ;;;;
.....: ;;;;
.....: ;;;;
.....: ;;;;
.....: ;;;;
.....: ;;;;
.....: date;Param1;Param2;Param4;Param5
.....: ;m²;°C;m²;m
.....: ;;;;
.....: 01.01.1990 00:00;1;1;2;3
.....: 01.01.1990 01:00;5;3;4;5
.....: 01.01.1990 02:00;9;5;6;7
.....: 01.01.1990 03:00;13;7;8;9
.....: 01.01.1990 04:00;17;9;10;11
.....: 01.01.1990 05:00;21;11;12;13
.....: ""
.....:
```

### 选项 1: 显式跳过行

```
In [201]: from io import StringIO

In [202]: pd.read_csv(StringIO(data), sep=';', skiprows=[11, 12],
.....: index_col=0, parse_dates=True, header=10)
.....:
Out[202]:
```

	Param1	Param2	Param4	Param5
date				
1990-01-01 00:00:00	1	1	2	3
1990-01-01 01:00:00	5	3	4	5
1990-01-01 02:00:00	9	5	6	7
1990-01-01 03:00:00	13	7	8	9
1990-01-01 04:00:00	17	9	10	11
1990-01-01 05:00:00	21	11	12	13

## 选项 2: 读取列名, 然后再读取数据

```
In [203]: pd.read_csv(StringIO(data), sep=';', header=10, nrows=10).columns
Out[203]: Index(['date', 'Param1', 'Param2', 'Param4', 'Param5'], dtype='object')
```

```
In [204]: columns = pd.read_csv(StringIO(data), sep=';', header=10, nrows=10).columns
```

```
In [205]: pd.read_csv(StringIO(data), sep=';', index_col=0,
.....: header=12, parse_dates=True, names=columns)
.....:
Out[205]:
```

	Param1	Param2	Param4	Param5
date				
1990-01-01 00:00:00	1	1	2	3
1990-01-01 01:00:00	5	3	4	5
1990-01-01 02:00:00	9	5	6	7
1990-01-01 03:00:00	13	7	8	9
1990-01-01 04:00:00	17	9	10	11
1990-01-01 05:00:00	21	11	12	13

## SQL

[SQL 文档](#)

[用 SQL 读取数据库](#)

## Excel

[Excel 文档](#)

[读取文件式句柄](#)

[用 XlsxWriter 修改输出格式](#)

## HTML

[从不能处理默认请求 header 的服务器读取 HTML 表格](#)

## HDFStore

[HDFStores 文档](#)

[时间戳索引简单查询](#)

[用链式多表架构管理异构数据](#)

[在硬盘上合并数百万行的表格](#)

[避免多进程/线程存储数据出现不一致](#)

按块对大规模数据存储去重的本质是递归还原操作。[这里](#)介绍了一个函数，可以从 CSV 文件里按块提取数据，解析日期后，再按块存储。

[按块读取 CSV 文件，并保存](#)

[追加到已存储的文件，并确保索引唯一](#)

[大规模数据工作流](#)

[读取一系列文件，追加时采用全局唯一索引](#)

[用低分组密度分组 HDFStore 文件](#)

[用高分组密度分组 HDFStore 文件](#)

[HDFStore 文件结构化查询](#)

[HDFStore 计数](#)

[HDFStore 异常解答](#)

[用字符串设置 min\\_itemsize](#)

[用 ptrepack 创建完全排序索引](#)

[把属性存至分组节点](#)

```
In [206]: df = pd.DataFrame(np.random.randn(8, 3))

In [207]: store = pd.HDFStore('test.h5')

In [208]: store.put('df', df)

用 pickle 存储任意 Python 对象
In [209]: store.get_storer('df').attrs.my_attribute = {'A': 10}

In [210]: store.get_storer('df').attrs.my_attribute
Out[210]: {'A': 10}
```

## 二进制文件

读取 C 结构体数组组成的二进制文件，Pandas 支持 NumPy 记录数组。比如说，名为 `main.c` 的文件包含下列 C 代码，并在 64 位机器上用 `gcc main.c -std=gnu99` 进行编译。

```

#include <stdio.h>
#include <stdint.h>

typedef struct _Data
{
 int32_t count;
 double avg;
 float scale;
} Data;

int main(int argc, const char *argv[])
{
 size_t n = 10;
 Data d[n];

 for (int i = 0; i < n; ++i)
 {
 d[i].count = i;
 d[i].avg = i + 1.0;
 d[i].scale = (float) i + 2.0f;
 }

 FILE *file = fopen("binary.dat", "wb");
 fwrite(&d, sizeof(Data), n, file);
 fclose(file);

 return 0;
}

```

下列 Python 代码读取二进制文件 `binary.dat`，并将之存为 pandas `DataFrame`，每个结构体的元素对应 `DataFrame` 里的列：

```

names = 'count', 'avg', 'scale'

注意：因为结构体填充，位移量比类型尺寸大
offsets = 0, 8, 16
formats = 'i4', 'f8', 'f4'
dt = np.dtype({'names': names, 'offsets': offsets, 'formats': formats},
 align=True)
df = pd.DataFrame(np.fromfile('binary.dat', dt))

```

⋮ tip 注意

不同机器上创建的文件因其架构不同，结构化元素的位移量也不同，原生二进制格式文件不能跨平台使用，因此不建议作为通用数据存储格式。建议用 Pandas IO 功能支持的 HDF5 或 msgpack 文件。

⋮

## 计算

基于采样的时间序列数值整合

## 相关性

用 `DataFrame.corr()` 计算得出的相关矩阵的下（或上）三角形式一般都非常有用。下例通过把布尔掩码传递给 `where` 可以实现这一功能：

```
In [211]: df = pd.DataFrame(np.random.random(size=(100, 5)))

In [212]: corr_mat = df.corr()

In [213]: mask = np.tril(np.ones_like(corr_mat, dtype=np.bool), k=-1)

In [214]: corr_mat.where(mask)
Out[214]:
```

	0	1	2	3	4
0	NaN	NaN	NaN	NaN	NaN
1	-0.018923	NaN	NaN	NaN	NaN
2	-0.076296	-0.012464	NaN	NaN	NaN
3	-0.169941	-0.289416	0.076462	NaN	NaN
4	0.064326	0.018759	-0.084140	-0.079859	NaN

除了命名相关类型之外，`DataFrame.corr` 还接受回调，此处计算 `DataFrame` 对象的[距离相关矩阵](#)。

```
In [215]: def distcorr(x, y):
.....: n = len(x)
.....: a = np.zeros(shape=(n, n))
.....: b = np.zeros(shape=(n, n))
.....: for i in range(n):
.....: for j in range(i + 1, n):
.....: a[i, j] = abs(x[i] - x[j])
.....: b[i, j] = abs(y[i] - y[j])
.....: a += a.T
.....: b += b.T
.....: a_bar = np.vstack([np.nanmean(a, axis=0)] * n)
.....: b_bar = np.vstack([np.nanmean(b, axis=0)] * n)
.....: A = a - a_bar - a_bar.T + np.full(shape=(n, n), fill_value=a_bar.mean())
.....: B = b - b_bar - b_bar.T + np.full(shape=(n, n), fill_value=b_bar.mean())
.....: cov_ab = np.sqrt(np.nansum(A * B)) / n
.....: std_a = np.sqrt(np.sqrt(np.nansum(A**2)) / n)
.....: std_b = np.sqrt(np.sqrt(np.nansum(B**2)) / n)
.....: return cov_ab / std_a / std_b
.....:

In [216]: df = pd.DataFrame(np.random.normal(size=(100, 3)))

In [217]: df.corr(method=distcorr)
Out[217]:
```

	0	1	2
0	1.000000	0.199653	0.214871
1	0.199653	1.000000	0.195116
2	0.214871	0.195116	1.000000

## 时间差

[时间差文档](#)。

[使用时间差](#)

```
In [218]: import datetime

In [219]: s = pd.Series(pd.date_range('2012-1-1', periods=3, freq='D'))

In [220]: s - s.max()
Out[220]:
0 -2 days
1 -1 days
2 0 days
dtype: timedelta64[ns]

In [221]: s.max() - s
Out[221]:
0 2 days
1 1 days
2 0 days
dtype: timedelta64[ns]

In [222]: s - datetime.datetime(2011, 1, 1, 3, 5)
Out[222]:
0 364 days 20:55:00
1 365 days 20:55:00
2 366 days 20:55:00
dtype: timedelta64[ns]

In [223]: s + datetime.timedelta(minutes=5)
Out[223]:
0 2012-01-01 00:05:00
1 2012-01-02 00:05:00
2 2012-01-03 00:05:00
dtype: datetime64[ns]

In [224]: datetime.datetime(2011, 1, 1, 3, 5) - s
Out[224]:
0 -365 days +03:05:00
1 -366 days +03:05:00
2 -367 days +03:05:00
dtype: timedelta64[ns]

In [225]: datetime.timedelta(minutes=5) + s
Out[225]:
0 2012-01-01 00:05:00
1 2012-01-02 00:05:00
2 2012-01-03 00:05:00
dtype: datetime64[ns]
```

[日期加减](#)



```

In [226]: deltas = pd.Series([datetime.timedelta(days=i) for i in range(3)])

In [227]: df = pd.DataFrame({'A': s, 'B': deltas})

In [228]: df
Out[228]:
 A B
0 2012-01-01 0 days
1 2012-01-02 1 days
2 2012-01-03 2 days

In [229]: df['New Dates'] = df['A'] + df['B']

In [230]: df['Delta'] = df['A'] - df['New Dates']

In [231]: df
Out[231]:
 A B New Dates Delta
0 2012-01-01 0 days 2012-01-01 0 days
1 2012-01-02 1 days 2012-01-03 -1 days
2 2012-01-03 2 days 2012-01-05 -2 days

In [232]: df.dtypes
Out[232]:
A datetime64[ns]
B timedelta64[ns]
New Dates datetime64[ns]
Delta timedelta64[ns]
dtype: object

```

## 其它示例

与 `datetime` 类似，用 `np.nan` 可以把值设为 `NaT`。

```

In [233]: y = s - s.shift()

In [234]: y
Out[234]:
0 NaT
1 1 days
2 1 days
dtype: timedelta64[ns]

In [235]: y[1] = np.nan

In [236]: y
Out[236]:
0 NaT
1 NaT
2 1 days
dtype: timedelta64[ns]

```

## 轴别名

---

设置全局轴别名，可以定义以下两个函数：

```
In [237]: def set_axis_alias(cls, axis, alias):
.....: if axis not in cls._AXIS_NUMBERS:
.....: raise Exception("invalid axis [%s] for alias [%s]" % (axis, alias))
.....: cls._AXIS_ALIASES[alias] = axis
.....:
In [238]: def clear_axis_alias(cls, axis, alias):
.....: if axis not in cls._AXIS_NUMBERS:
.....: raise Exception("invalid axis [%s] for alias [%s]" % (axis, alias))
.....: cls._AXIS_ALIASES.pop(alias, None)
.....:
In [239]: set_axis_alias(pd.DataFrame, 'columns', 'myaxis2')

In [240]: df2 = pd.DataFrame(np.random.randn(3, 2), columns=['c1', 'c2'],
.....: index=['i1', 'i2', 'i3'])
.....:

In [241]: df2.sum(axis='myaxis2')
Out[241]:
i1 -0.461013
i2 2.040016
i3 0.904681
dtype: float64

In [242]: clear_axis_alias(pd.DataFrame, 'columns', 'myaxis2')
```

## 创建示例数据

---

类似 R 的 `expand.grid()` 函数，用不同类型的值组生成 DataFrame，需要创建键是列名，值是数据值列表的字典：

```
In [243]: def expand_grid(data_dict):
.....: rows = itertools.product(*data_dict.values())
.....: return pd.DataFrame.from_records(rows, columns=data_dict.keys())
.....:

In [244]: df = expand_grid({'height': [60, 70],
.....: 'weight': [100, 140, 180],
.....: 'sex': ['Male', 'Female']})
.....:

In [245]: df
Out[245]:
```

	height	weight	sex
0	60	100	Male
1	60	100	Female
2	60	140	Male
3	60	140	Female
4	60	180	Male
5	60	180	Female
6	70	100	Male
7	70	100	Female
8	70	140	Male
9	70	140	Female
10	70	180	Male
11	70	180	Female

---

## Pandas 生态

---

### Pandas 生态圈

越来越多的人在 Pandas 的基础上构建包，以满足数据准备、分析和可视化方面的特定需求。这是令人激动的，因为这意味着 Pandas 不仅帮助用户处理他们的数据任务，而且它为开发人员提供了一个更好的起点，来构建功能强大和更有针对性的数据工具。创建扩充 Pandas 功能的类库也能让 Pandas 的发展保持围绕其初心不改的原则。

这是一个不会断更的项目列表，它们给予 Pandas 构建，以便在 PyData 空间中能不断的提供新的工具。有关依赖 Pandas 的项目列表，请参阅 [Pandas 的 libraries.io 用法页面](#) 或 [在 pypi 中搜索 Pandas](#)。

我们希望让用户更容易找到这些项目，如果您还知道您认为的应该在此列表中展示的其他不错的项目，请将它告诉我们。

## 统计和机器学习

---

### Statsmodels

Statsmodels是著名的 Python “统计和计量经济学库”，它与Pandas长期有着的特殊关系。Statsmodels提供了超出Pandas能力范围的强大的统计数据功能，包括计量经济学，分析和建模等功能。Statsmodels利用Pandas对象作为计算的底层数据容器。

### sklearn-pandas

在 [scikit-learn](#) 机器学习中使用 pandas 的 DataFrames。

### Featuretools

Featuretools是一个基于 pandas 的自动功能工程的Python库。它擅长使用可重用的特征工程“基元”将时间和关系数据集转换为用于机器学习的特征矩阵。用户可以使用Python贡献自己的原语，并与社区的其他人共享。

## 可视化

---

### Altair

Altair是用于Python的声明性统计可视化库。使用 Altair，您可以花费更多时间来理解数据及其含义。Altair的API简单，友好且一致，并以强大的Vega-Lite JSON规范为基础。这种优雅的简单性用最少的代码即可产生美观，有效的可视化效果。Altair可与Pandas DataFrames配合使用。

### Bokeh

Bokeh是一个Python交互式可视化库，用于大型数据集，其本机使用最新的Web技术。它的目标是以 Protovis / D3 的样式提供优雅，简洁的新颖图形构造，同时为瘦客户端提供大数据上的高性能交互性。

### seaborn

Seaborn 是基于 [matplotlib](#) 的Python可视化库。它提供了一个高级的，面向数据集的界面，用于创建吸引人的统计图形。seaborn中的绘图功能可了解熊猫对象，并在内部利用熊猫分组操作来支持复杂可视化的简洁规范。Seaborn 深圳还超越了matplotlib 和 pandas，可以在绘制图表时进行统计估计，汇总观察结果并可视化统计模型的拟合以强调数据集中的模式。

### yhat/ggpy

Hadley Wickham的 [ggplot2](#) 是R语言的基础探索性可视化包。它以“[图形语法](#)”为基础，提供了一种强大的、说明性的、极其通用的方式来生成任何类型数据的定制绘图。这真是太不可思议了。其他语言的各种实现都是可用的，但是对于Python用户来说，一直缺少一个忠实的实现。尽管还很年轻(截至2014年1月)，[yhat/ggpy](#) 项目在这个方向上进展很快。

## IPython Vega

IPython Vega 利用 Vega 在 Jupyter Notebook 中创建图形。

## Plotly

Plotly's Python API 支持交互式图形和网络共享。地图、2D、3D 和实况流程图是使用 WebGL 和 D3.js 渲染的。该库支持直接从 pandas 的 DataFrame 打印和基于云的协作。matplotlib、Python 的 ggplot 和 Seborn 的用户可以将图形转换为基于 Web 的交互式绘图。绘图可以在 IPython Notebooks 本中绘制，可以使用 R 或 MATLAB 编辑，可以在 GUI 中修改，也可以嵌入到应用程序和仪表板中。Plotly 是免费的，可以无限地共享，并且有云、离线或内部帐户供私人使用。

## QtPandas

qtpandas 库是从主 Pandas 库派生出来的，它支持在 PyQt4 和 PySide 应用程序中实现 DataFrame 可视化和操作。

## IDE

---

## IPython

IPython is an interactive command shell and distributed computing environment. IPython tab completion works with Pandas methods and also attributes like DataFrame columns.

## Jupyter Notebook / Jupyter Lab

Jupyter Notebook is a web application for creating Jupyter notebooks. A Jupyter notebook is a JSON document containing an ordered list of input/output cells which can contain code, text, mathematics, plots and rich media. Jupyter notebooks can be converted to a number of open standard output formats (HTML, HTML presentation slides, LaTeX, PDF, ReStructuredText, Markdown, Python) through 'Download As' in the web interface and `jupyter convert` in a shell.

Pandas DataFrames implement `_repr_html_` and `_repr_latex` methods which are utilized by Jupyter Notebook for displaying (abbreviated) HTML or LaTeX tables. LaTeX output is properly escaped. (Note: HTML tables may or may not be compatible with non-HTML Jupyter output formats.)

See [Options and Settings](#) and [Available Options](#) for pandas `display` settings.

## quantopian/qgrid

qgrid is “an interactive grid for sorting and filtering DataFrames in IPython Notebook” built with SlickGrid.

## Spyder

Spyder is a cross-platform PyQt-based IDE combining the editing, analysis, debugging and profiling functionality of a software development tool with the data exploration, interactive execution, deep inspection and rich visualization capabilities of a scientific environment like MATLAB or Rstudio.

Its [Variable Explorer](#) allows users to view, manipulate and edit pandas `Index`, `Series`, and `DataFrame` objects like a “spreadsheet”, including copying and modifying values, sorting, displaying a “heatmap”, converting data types and more. Pandas objects can also be renamed, duplicated, new columns added, copied/pasted to/from the clipboard (as TSV), and saved/loaded to/from a file. Spyder can also import data from a variety of plain text and binary files or the clipboard into a new pandas `DataFrame` via a sophisticated import wizard.

Most pandas classes, methods and data attributes can be autocompleted in Spyder’s [Editor](#) and [IPython Console](#), and Spyder’s [Help pane](#) can retrieve and render Numpydoc documentation on pandas objects in rich text with Sphinx both automatically and on-demand.

## API

---

### pandas-datareader

`pandas-datareader` is a remote data access library for pandas (PyPI: `pandas-datareader`). It is based on functionality that was located in `pandas.io.data` and `pandas.io.web` but was split off in v0.19. See more in the [pandas-datareader docs](#):

The following data feeds are available:

- Google Finance
- Tiingo
- Morningstar
- IEX
- Robinhood
- Enigma
- Quandl
- FRED
- Fama/French
- World Bank
- OECD
- Eurostat
- TSP Fund Data
- Nasdaq Trader Symbol Definitions
- Stooq Index Data
- MOEX Data

## quandl/Python

Quandl API for Python wraps the Quandl REST API to return Pandas DataFrames with timeseries indexes.

## pydatastream

PyDatastream is a Python interface to the [Thomson Dataworks Enterprise \(DWE/Datastream\)](#) SOAP API to return indexed Pandas DataFrames or Panels with financial data. This package requires valid credentials for this API (non free).

## pandaSDMX

pandaSDMX is a library to retrieve and acquire statistical data and metadata disseminated in [SDMX 2.1](#), an ISO-standard widely used by institutions such as statistics offices, central banks, and international organisations. pandaSDMX can expose datasets and related structural metadata including data flows, code-lists, and data structure definitions as pandas Series or MultiIndexed DataFrames.

## fredapi

fredapi is a Python interface to the [Federal Reserve Economic Data \(FRED\)](#) provided by the Federal Reserve Bank of St. Louis. It works with both the FRED database and ALFRED database that contains point-in-time data (i.e. historic data revisions). fredapi provides a wrapper in Python to the FRED HTTP API, and also provides several convenient methods for parsing and analyzing point-in-time data from ALFRED. fredapi makes use of pandas and returns data in a Series or DataFrame. This module requires a FRED API key that you can obtain for free on the FRED website.

## Domain Specific

---

### Geopandas

Geopandas extends pandas data objects to include geographic information which support geometric operations. If your work entails maps and geographical coordinates, and you love pandas, you should take a close look at Geopandas.

### xarray

xarray brings the labeled data power of pandas to the physical sciences by providing N-dimensional variants of the core pandas data structures. It aims to provide a pandas-like and pandas-compatible toolkit for analytics on multi- dimensional arrays, rather than the tabular data for which pandas excels.

## Out-of-core

---

### Blaze

Blaze provides a standard API for doing computations with various in-memory and on-disk backends: NumPy, Pandas, SQLAlchemy, MongoDB, PyTables, PySpark.

### Dask

Dask is a flexible parallel computing library for analytics. Dask provides a familiar `DataFrame` interface for out-of-core, parallel and distributed computing.

### Dask-ML

Dask-ML enables parallel and distributed machine learning using Dask alongside existing machine learning libraries like Scikit-Learn, XGBoost, and TensorFlow.

### Odo

Odo provides a uniform API for moving data between different formats. It uses pandas own `read_csv` for CSV IO and leverages many existing packages such as PyTables, h5py, and pymongo to move data between non pandas formats. Its graph based approach is also extensible by end users for custom formats that may be too specific for the core of odo.

### Ray

Pandas on Ray is an early stage `DataFrame` library that wraps Pandas and transparently distributes the data and computation. The user does not need to know how many cores their system has, nor do they need to specify how to distribute the data. In fact, users can continue using their previous Pandas notebooks while experiencing a considerable speedup from Pandas on Ray, even on a single machine. Only a modification of the import statement is needed, as we demonstrate below. Once you've changed your import statement, you're ready to use Pandas on Ray just like you would Pandas.

```
import pandas as pd
import ray.dataframe as pd
```

### Vaex

Increasingly, packages are being built on top of pandas to address specific needs in data preparation, analysis and visualization. Vaex is a python library for Out-of-Core `DataFrames` (similar to Pandas), to visualize and explore big tabular datasets. It can calculate statistics such as mean, sum, count, standard deviation etc, on an N-dimensional grid up to a billion (10<sup>9</sup>) objects/rows per second. Visualization is done using histograms, density plots and 3d volume rendering, allowing interactive



exploration of big data. Vaex uses memory mapping, zero memory copy policy and lazy computations for best performance (no memory wasted).

- `vaex.from_pandas`
- `vaex.to_pandas_df`

## Data validation

---

### Engarde

Engarde is a lightweight library used to explicitly state your assumptions about your datasets and check that they're actually true.

## Extension Data Types

---

Pandas provides an interface for defining [extension types](#) to extend NumPy's type system. The following libraries implement that interface to provide types not found in NumPy or pandas, which work well with pandas' data containers.

### cyberpandas

Cyberpandas provides an extension type for storing arrays of IP Addresses. These arrays can be stored inside pandas' Series and DataFrame.

## Accessors

---

A directory of projects providing [extension accessors](#). This is for users to discover new accessors and for library authors to coordinate on the namespace.

Library	Accessor	Classes
<a href="#">cyberpandas</a>	ip	Series
<a href="#">pdvega</a>	vgplot	Series, DataFrame

## API 参考手册

---

## API 参考手册

本页面概述了所有公共的Pandas对象、功能和。方法。 `pandas.*` 命名空间中公开的所有类和函数都是公共的。

有些子模块是公开的，其中包括 `pandas.error`、`pandas.plotting` 和 `pandas.testing`。`pandas.io` 和 `pandas.tseries` 系列子模块中的公共函数在文档中有所提及。`pandas.api.types` 子模块含一些与pandas中的数据类型相关的公共函数。

⋮ danger 警告

`pandas.core`、`pandas.compat` 和 `pandas.util` 顶级模块是PRIVATE（私有的）。此类模块的功能稳定性无法保证。

⋮

- [Input/output](#)
  - [Pickling](#)
  - [Flat file](#)
  - [Clipboard](#)
  - [Excel](#)
  - [JSON](#)
  - [HTML](#)
  - [HDFStore: PyTables \(HDF5\)](#)
  - [Feather](#)
  - [Parquet](#)
  - [SAS](#)
  - [SQL](#)
  - [Google BigQuery](#)
  - [STATA](#)
- [General functions](#)
  - [Data manipulations](#)
  - [Top-level missing data](#)
  - [Top-level conversions](#)
  - [Top-level dealing with datetimelike](#)
  - [Top-level dealing with intervals](#)
  - [Top-level evaluation](#)
  - [Hashing](#)
  - [Testing](#)
- [Series](#)
  - [Constructor](#)
  - [Attributes](#)
  - [Conversion](#)
  - [Indexing, iteration](#)
  - [Binary operator functions](#)
  - [Function application, groupby & window](#)
  - [Computations / descriptive stats](#)
  - [Reindexing / selection / label manipulation](#)

- Missing data handling
- Reshaping, sorting
- Combining / joining / merging
- Time series-related
- Accessors
- Plotting
- Serialization / IO / conversion
- Sparse
- DataFrame
  - Constructor
  - Attributes and underlying data
  - Conversion
  - Indexing, iteration
  - Binary operator functions
  - Function application, GroupBy & window
  - Computations / descriptive stats
  - Reindexing / selection / label manipulation
  - Missing data handling
  - Reshaping, sorting, transposing
  - Combining / joining / merging
  - Time series-related
  - Plotting
  - Sparse accessor
  - Serialization / IO / conversion
  - Sparse
- Pandas arrays
  - pandas.array
  - Datetime data
  - Timedelta data
  - Timespan data
  - Period
  - Interval data
  - Nullable integer
  - Categorical data
  - Sparse data
- Panel
- Index objects
  - Index
  - Numeric Index
  - CategoricalIndex
  - IntervalIndex
  - MultiIndex

- DatetimeIndex
- TimedeltaIndex
- PeriodIndex
- Date offsets
  - DateOffset
  - BusinessDay
  - BusinessHour
  - CustomBusinessDay
  - CustomBusinessHour
  - MonthOffset
  - MonthEnd
  - MonthBegin
  - BusinessMonthEnd
  - BusinessMonthBegin
  - CustomBusinessMonthEnd
  - CustomBusinessMonthBegin
  - SemiMonthOffset
  - SemiMonthEnd
  - SemiMonthBegin
  - Week
  - WeekOfMonth
  - LastWeekOfMonth
  - QuarterOffset
  - BQuarterEnd
  - BQuarterBegin
  - QuarterEnd
  - QuarterBegin
  - YearOffset
  - BYearEnd
  - BYearBegin
  - YearEnd
  - YearBegin
  - FY5253
  - FY5253Quarter
  - Easter
  - Tick
  - Day
  - Hour
  - Minute
  - Second
  - Milli
  - Micro

- Nano
- BDay
- BMonthEnd
- BMonthBegin
- CBMonthEnd
- CBMonthBegin
- CDay
- Frequencies
  - `pandas.tseries.frequencies.to_offset`
- Window
  - Standard moving window functions
  - Standard expanding window functions
  - Exponentially-weighted moving window functions
- GroupBy
  - Indexing, iteration
  - Function application
  - Computations / descriptive stats
- Resampling
  - Indexing, iteration
  - Function application
  - Upsampling
  - Computations / descriptive stats
- Style
  - Styler constructor
  - Styler properties
  - Style application
  - Builtin styles
  - Style export and import
- Plotting
  - `pandas.plotting.andrews_curves`
  - `pandas.plotting.bootstrap_plot`
  - `pandas.plotting.deregister_matplotlib_converters`
  - `pandas.plotting.lag_plot`
  - `pandas.plotting.parallel_coordinates`
  - `pandas.plotting.radviz`
  - `pandas.plotting.register_matplotlib_converters`
  - `pandas.plotting.scatter_matrix`
- General utility functions
  - Working with options
  - Testing functions
  - Exceptions and warnings
  - Data types related functionality

- Extensions
    - `pandas.api.extensions.register_extension_dtype`
    - `pandas.api.extensions.register_dataframe_accessor`
    - `pandas.api.extensions.register_series_accessor`
    - `pandas.api.extensions.register_index_accessor`
    - `pandas.api.extensions.ExtensionDtype`
    - `pandas.api.extensions.ExtensionArray`
    - `pandas.arrays.PandasArray`
- 

## 开发者文档

---

## 开发者文档

- Contributing to pandas
    - Where to start?
    - Bug reports and enhancement requests
    - Working with the code
    - Contributing to the documentation
    - Contributing to the code base
    - Contributing your changes to pandas
  - Internals
    - Indexing
    - Subclassing pandas data structures
  - Extending pandas
    - Registering custom accessors
    - Extension types
    - Subclassing pandas data structures
    - Plotting backends
  - Developer
    - Storing pandas DataFrame objects in Apache Parquet format
- 

## 版本发布日志

---

## 发布日志

这是Pandas的每个发行版的更新列表。有关完整的详细信息，请参阅 <http://github.com/pandas-dev/pandas> 上的提交日志。有关安装和升级说明，请参阅 [Pandas 安装方法](#)。

## 版本 0.25

---

- [What's new in 0.25.0 \(July 18, 2019\)](#)
  - [Enhancements](#)
  - [Backwards incompatible API changes](#)
  - [Deprecations](#)
  - [Removal of prior version deprecations/changes](#)
  - [Performance improvements](#)
  - [Bug fixes](#)
  - [Contributors](#)

## 版本 0.24

---

- [Whats new in 0.24.2 \(March 12, 2019\)](#)
  - [Fixed regressions](#)
  - [Bug fixes](#)
  - [Contributors](#)
- [Whats new in 0.24.1 \(February 3, 2019\)](#)
  - [API changes](#)
  - [Fixed regressions](#)
  - [Bug fixes](#)
  - [Contributors](#)
- [What's new in 0.24.0 \(January 25, 2019\)](#)
  - [Enhancements](#)
  - [Backwards incompatible API changes](#)
  - [Extension type changes](#)
  - [Deprecations](#)
  - [Removal of prior version deprecations/changes](#)
  - [Performance improvements](#)
  - [Bug fixes](#)
  - [Contributors](#)

## 版本 0.23

---

- [What's new in 0.23.4 \(August 3, 2018\)](#)
  - [Fixed regressions](#)
  - [Bug fixes](#)
  - [Contributors](#)
- [What's new in 0.23.3 \(July 7, 2018\)](#)
  - [Contributors](#)

- What's new in 0.23.2 (July 5, 2018)
  - Logical reductions over entire DataFrame
  - Fixed regressions
  - Build changes
  - Bug fixes
  - Contributors
- What's new in 0.23.1 (June 12, 2018)
  - Fixed regressions
  - Performance improvements
  - Bug fixes
  - Contributors
- What's new in 0.23.0 (May 15, 2018)
  - New features
  - Backwards incompatible API changes
  - Deprecations
  - Removal of prior version deprecations/changes
  - Performance improvements
  - Documentation changes
  - Bug fixes
  - Contributors

## 版本 0.22

---

- v0.22.0 (December 29, 2017)
  - Backwards incompatible API changes
  - Compatibility
  - Contributors

## 版本 0.21

---

- v0.21.1 (December 12, 2017)
  - Restore Matplotlib datetime converter registration
  - New features
  - Deprecations
  - Performance improvements
  - Bug fixes
  - Contributors
- v0.21.0 (October 27, 2017)
  - New features
  - Backwards incompatible API changes
  - Deprecations
  - Removal of prior version deprecations/changes



- Performance improvements
- Documentation changes
- Bug fixes
- Contributors

## 版本 0.20

---

- v0.20.3 (July 7, 2017)
  - Bug fixes
  - Contributors
- v0.20.2 (June 4, 2017)
  - Enhancements
  - Performance improvements
  - Bug fixes
  - Contributors
- v0.20.1 (May 5, 2017)
  - New features
  - Backwards incompatible API changes
  - Reorganization of the library: privacy changes
  - Deprecations
  - Removal of prior version deprecations/changes
  - Performance improvements
  - Bug fixes
  - Contributors

## 版本 0.19

---

- v0.19.2 (December 24, 2016)
  - Enhancements
  - Performance improvements
  - Bug fixes
  - Contributors
- v0.19.1 (November 3, 2016)
  - Performance improvements
  - Bug fixes
  - Contributors
- v0.19.0 (October 2, 2016)
  - New features
  - API changes
  - Deprecations
  - Removal of prior version deprecations/changes
  - Performance improvements

- Bug fixes
- Contributors

## 版本 0.18

---

- v0.18.1 (May 3, 2016)
  - New features
  - Sparse changes
  - API changes
  - Performance improvements
  - Bug fixes
  - Contributors
- v0.18.0 (March 13, 2016)
  - New features
  - Backwards incompatible API changes
  - Performance improvements
  - Bug Fixes
  - Contributors

## 版本 0.17

---

- v0.17.1 (November 21, 2015)
  - New features
  - Enhancements
  - API changes
  - Performance improvements
  - Bug fixes
  - Contributors
- v0.17.0 (October 9, 2015)
  - New features
  - Backwards incompatible API changes
  - Performance improvements
  - Bug fixes
  - Contributors

## 版本 0.16

---

- v0.16.2 (June 12, 2015)
  - New features
  - API changes
  - Performance improvements

- Bug fixes
- Contributors
- v0.16.1 (May 11, 2015)
  - Enhancements
  - API changes
  - Index representation
  - Performance improvements
  - Bug fixes
  - Contributors
- v0.16.0 (March 22, 2015)
  - New features
  - Backwards incompatible API changes
  - Performance improvements
  - Bug fixes
  - Contributors

## 版本 0.15

---

- v0.15.2 (December 12, 2014)
  - API changes
  - Enhancements
  - Performance
  - Bug fixes
  - Contributors
- v0.15.1 (November 9, 2014)
  - API changes
  - Enhancements
  - Bug fixes
  - Contributors
- v0.15.0 (October 18, 2014)
  - New features
  - Backwards incompatible API changes
  - Enhancements
  - Performance
  - Bug fixes
  - Contributors

## 版本 0.14

---

- v0.14.1 (July 11, 2014)
  - API changes
  - Enhancements

- Performance
- Experimental
- Bug fixes
- Contributors
- v0.14.0 (May 31 , 2014)
  - API changes
  - Display changes
  - Text parsing API changes
  - Groupby API changes
  - SQL
  - MultiIndexing using slicers
  - Plotting
  - Prior version deprecations/changes
  - Deprecations
  - Known issues
  - Enhancements
  - Performance
  - Experimental
  - Bug Fixes
  - Contributors

## 版本 0.13

---

- v0.13.1 (February 3, 2014)
  - Output formatting enhancements
  - API changes
  - Prior version deprecations/changes
  - Deprecations
  - Enhancements
  - Performance
  - Experimental
  - Bug fixes
  - Contributors
- v0.13.0 (January 3, 2014)
  - API changes
  - Prior version deprecations/changes
  - Deprecations
  - Indexing API changes
  - Float64Index API change
  - HDFStore API changes
  - DataFrame repr changes
  - Enhancements

- Experimental
- Internal refactoring
- Bug fixes
- Contributors

## 版本 0.12

---

- v0.12.0 (July 24, 2013)
  - API changes
  - I/O enhancements
  - Other enhancements
  - Experimental features
  - Bug fixes
  - Contributors

## 版本 0.11

---

- v0.11.0 (April 22, 2013)
  - Selection choices
  - Selection deprecations
  - Dtypes
  - Dtype conversion
  - Dtype gotchas
  - Datetimes conversion
  - API changes
  - Enhancements
  - Contributors

## 版本 0.10

---

- v0.10.1 (January 22, 2013)
  - API changes
  - New features
  - HDFStore
  - Contributors
- v0.10.0 (December 17, 2012)
  - File parsing new features
  - API changes
  - New features
  - Wide DataFrame printing
  - Updated PyTables support

- N dimensional Panels (experimental)
- Contributors

## 版本 0.9

---

- v0.9.1 (November 14, 2012)
  - New features
  - API changes
  - Contributors
- v0.9.0 (October 7, 2012)
  - New features
  - API changes
  - Contributors

## 版本 0.8

---

- v0.8.1 (July 22, 2012)
  - New features
  - Performance improvements
  - Contributors
- v0.8.0 (June 29, 2012)
  - Support for non-unique indexes
  - NumPy datetime64 dtype and 1.6 dependency
  - Time series changes and improvements
  - Other new features
  - New plotting methods
  - Other API changes
  - Potential porting issues for pandas <= 0.7.3 users
  - Contributors

## 版本 0.7

---

- v.0.7.3 (April 12, 2012)
  - New features
  - NA Boolean comparison API change
  - Other API changes
  - Contributors
- v.0.7.2 (March 16, 2012)
  - New features
  - Performance improvements
  - Contributors

- v.0.7.1 (February 29, 2012)
  - New features
  - Performance improvements
  - Contributors
- v.0.7.0 (February 9, 2012)
  - New features
  - API changes to integer indexing
  - API tweaks regarding label-based slicing
  - Changes to Series [] operator
  - Other API changes
  - Performance improvements
  - Contributors

## 版本 0.6

---

- v.0.6.1 (December 13, 2011)
  - New features
  - Performance improvements
  - Contributors
- v.0.6.0 (November 25, 2011)
  - New features
  - Performance enhancements
  - Contributors

## 版本 0.5

---

- v.0.5.0 (October 24, 2011)
  - New features
  - Performance enhancements
  - Contributors

## 版本 0.4

---

- v.0.4.1 through v0.4.3 (September 25 - October 9, 2011)
  - New features
  - Performance enhancements
  - Contributors